Universidad Peruana de Ciencias Aplicadas

1ACC0238

Informe del Trabajo Final

Docente

Equipo

Integrantes

Código	Apellidos y nombres
u202318309	Aguilar Untiveros, Rodrigo Fabrizio
u202319950	Meza Solórzano, Didier Sebastian
u20241d934	Montoya Nina, Paula Fernanda
u20241e418	Ramos Mera, Neo Daniel
u202624323	Torres Juárez, Alisee Muriel

Período 202620

Registro de Versiones del Informe

El registro de versiones permite seguir las fechas, los responsables y las modificaciones realizadas durante la elaboración del informe, como se detalla en la Tabla 1.

Tabla 1

Historial de versiones del reporte

Versión	Fecha	Autor	Descripción de modificación
1.0.0	08-28-26	Montoya Nina, Paula Fernanda	Estructura base del informe: carátula, registro de versiones y formato del Student Outcome.
1.1.0	02-09-26	Meza Solórzano, Didier Sebastian	Cap. I — Startup Profile, Solution Profile y Segmentos objetivo; estructura de carpetas del informe.
1.2.0	13-09-26	Aguilar Untiveros, Rodrigo Fabrizio	Cap. II — Software Architecture (Context, Container y Deployment Diagrams, Context Mapping, Bounded Context Canvases, Domain Message Flows y Candidate Context Discovery).
1.2.1	16-09-26	Meza Solórzano, Didier Sebastian	Cap. II — Entrevistas y evidencias de los segmentos objetivo.
1.2.2	18-09-26	Torres Juárez, Alisee Muriel	Cap. II — Bounded Context Identity & Access Management (Domain, Interface, Application e Infrastructure).
1.2.3	18-09-26	Ramos Mera, Neo Daniel	Cap. II — Tactical-Level Domain-Driven Design.
1.2.4	18-09-26	Montoya Nina, Paula Fernanda	Cap. II — Tactical-Level DDD: diagramas de clases y de base de datos.
2.3.0	24-09-26	Aguilar Untiveros, Rodrigo Fabrizio	Cap. III — 3.1.1 Style Guidelines, 3.1.2 Information Architecture y 3.1.3 Landing Page UI Design.
2.3.1	06-10-26	Meza Solórzano, Didier Sebastian	Cap. III — 3.1.4.3 Mobile Applications Mock-ups.
2.3.2	07-10-26	Torres Juárez, Alisee Muriel	Cap. III — 3.1.4.4 User Flow Diagrams y 3.1.4.5 Prototyping.
2.3.3	07-10-26	Montoya Nina, Paula Fernanda / Ramos Mera, Neo Daniel	Cap. III — 3.1.4.1 Wireframes y 3.1.4.2 Wireflow Diagrams.
2.4.0	08-10-26	Torres Juárez, Alisee Muriel	Cap. IV — 4.1.1 Software Development Environment Configuration y 4.1.2 Source Code Management.

Versión	Fecha	Autor	Descripción de modificación
2.4.1	09-10-26	Aguilar Untiveros, Rodrigo Fabrizio	Cap. IV — 4.1.3 Source Code Style Guide y 4.1.4 Software Deployment Configuration.
2.4.2	09-10-26	Montoya Nina, Paula Fernanda	Cap. IV — 4.2.1.1 Sprint Planning 1 y 4.2.1.3 Sprint Backlog 1.
2.4.3	09-10-26	Meza Solórzano, Didier Sebastian	Cap. IV — 4.2.1.4 Development Evidence, 4.2.1.5 Testing Suite y 4.2.1.6 Execution Evidence.
2.4.4	10-10-26	Ramos Mera, Neo Daniel	Cap. IV — 4.2.1.7 Services Documentation, 4.2.1.8 Software Deployment Evidence y 4.2.1.9 Team Collaboration Insights.
2.4.5	10-10-26	Montoya Nina, Paula Fernanda	Cap. IV — 4.2.1.2 Aspect Leaders and Collaborators; Project Report Collaboration Insights (TB1) y correcciones de la entrega.
2.4.6	11-11-26	Montoya Nina, Paula Fernanda	Cap. IV — Sprint Planning 2 y Sprint Backlog 2; actualización del Product Backlog y diferenciación de TB1 y AV2 en Student Outcome.

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

Project Report Collaboration Insights

En esta sección se presenta la evidencia de la colaboración del equipo **RuwaLabs** en la elaboración del **Project Report** de SaludYa. Para cada entrega se describe cómo se desarrollaron las actividades del informe y se incluyen capturas de los analíticos de colaboración y de commits registrados en GitHub. Esta evidencia se amplía en cada entrega y mantiene coherencia con el Registro de Versiones del Informe.

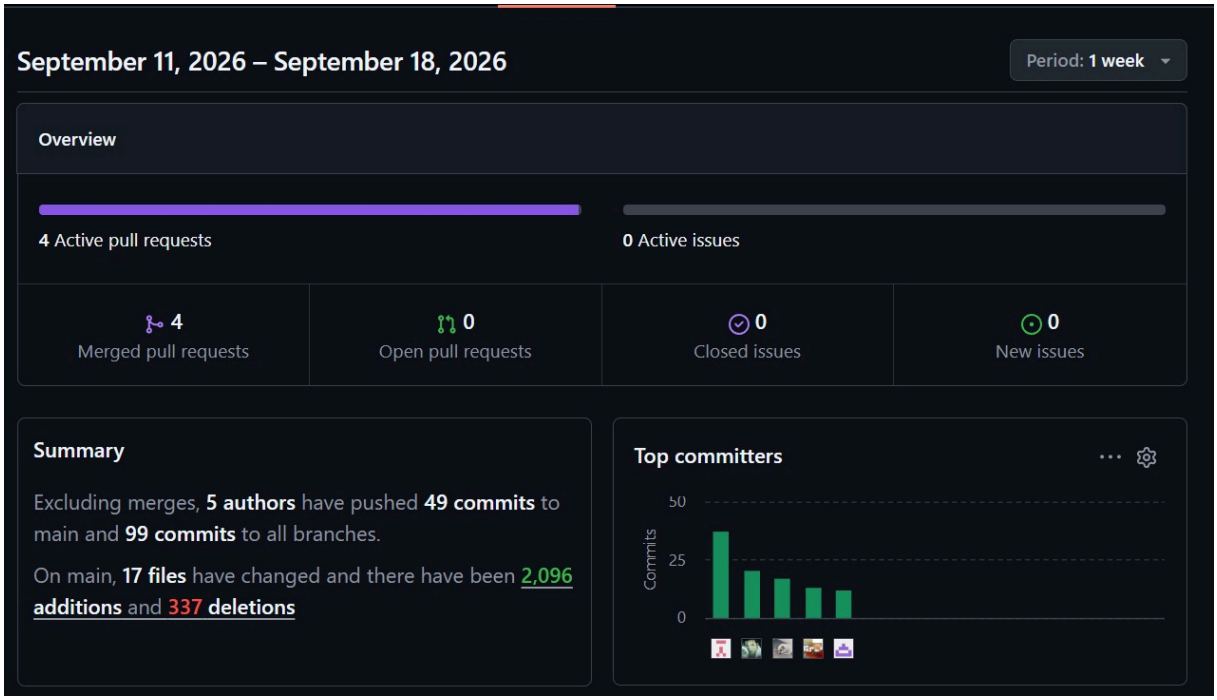
URL del Repositorio del Project Report: upc-pre-202620-1acc0238-4945-RuwaLabs-report

Primera entrega (AV1)

En la primera entrega, el equipo elaboró la estructura base del informe, la carátula, el registro de versiones y el formato del Student Outcome, así como los capítulos I y II (Presentación y Requirements Development and Software Solution Design). La redacción se distribuyó entre los cinco integrantes y se evidencia en el historial de colaboración y de commits del repositorio del informe, como se observa en la Figura 1, la Figura 2 y la Figura 3.

Figura 1

Colaboración en GitHub: evidencia 1 (AV1)



Nota. Captura del registro de colaboración del repositorio del informe en GitHub.

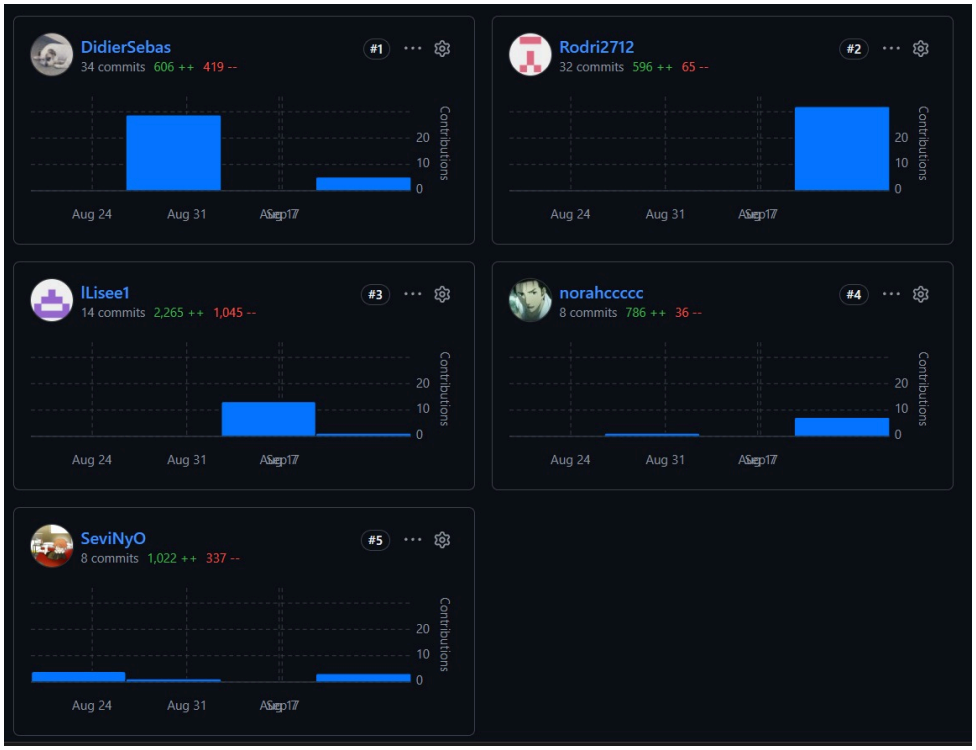
Figura 2

Colaboración en GitHub: evidencia 2 (AV1)

Nota. Captura del registro de colaboración del repositorio del informe en GitHub.

Figura 3

Colaboración en GitHub: evidencia 3 (AV1)



Nota. Captura del registro de colaboración del repositorio del informe en GitHub.

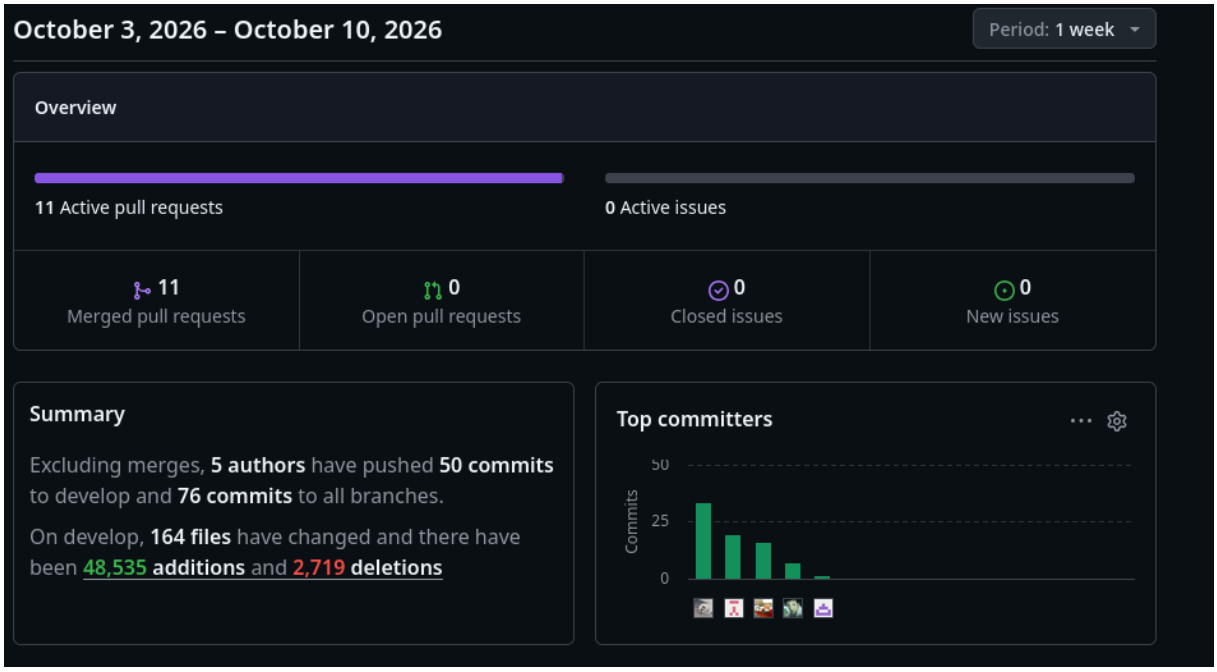
La actividad del equipo en GitHub queda registrada en la organización RuwaLabs.

Segunda entrega (TB1)

En la segunda entrega, el equipo actualizó el informe incorporando el **Capítulo III: Solution UI/UX Design** y el **Capítulo IV: Product Implementation & Validation**, junto con la documentación del **Sprint 1** (Sprint Planning, Aspect Leaders and Collaborators, Sprint Backlog y las evidencias de desarrollo, pruebas, ejecución y documentación de servicios). Asimismo, se corrigieron y mejoraron artefactos presentados en la entrega anterior. La elaboración se distribuyó entre los cinco integrantes, como se evidencia en los analíticos del repositorio del informe, que registran la actividad del periodo **Oct 3 – Oct 10, 2026** con **11 pull requests** integrados y **50 commits** en `develop` realizados por 5 autores, como se observa en la Figura 4 y la Figura 5.

Figura 4

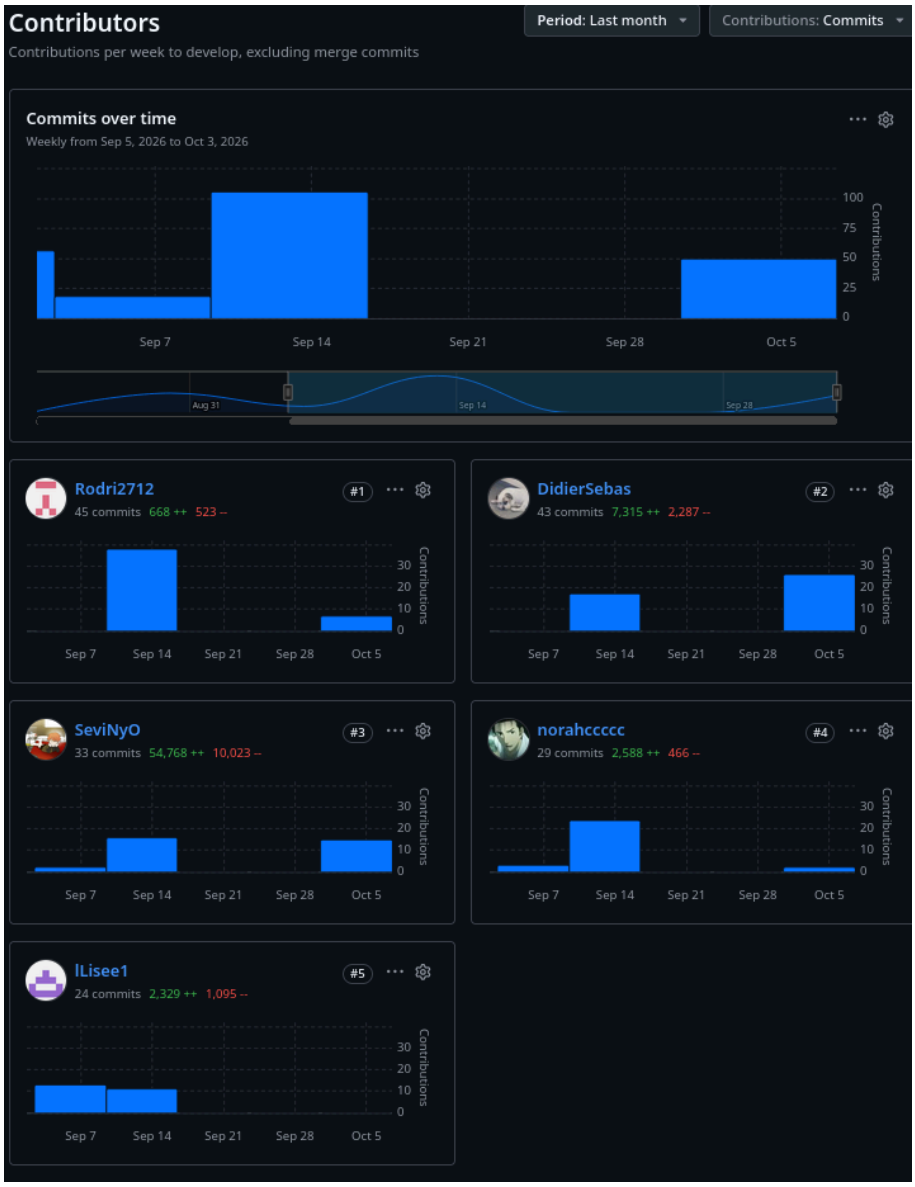
Resumen de actividad del repositorio del informe (TB1)



Nota. Captura del panel Overview del repositorio del informe en GitHub (periodo del 3 al 10 de octubre de 2026).

Figura 5

Contribuciones por integrante (TB1)



Nota. Captura del panel Contributors del repositorio del informe en GitHub.

Nota. Elaboración del equipo RuwaLabs a partir del panel Contributors del repositorio del informe en GitHub.

Contenido

- Registro de Versiones del Informe
- Project Report Collaboration Insights
- Contenido
- Student Outcome
- Objetivos SMART
- Capítulo I: Presentación
  - 1.1. Startup Profile
    - 1.1.1. Descripción de la Startup
    - 1.1.2. Perfiles de integrantes del equipo
  - 1.2. Solution Profile
    - 1.2.1. Antecedentes y problemática
    - 1.2.2. Lean UX Process
      - 1.2.2.1. Lean UX Problem Statements
      - 1.2.2.2. Lean UX Assumptions
      - 1.2.2.3. Lean UX Hypothesis Statements
      - 1.2.2.4. Lean UX Canvas
  - 1.3. Segmentos objetivo
- Capítulo II: Requirements Development and Software Solution Design
  - 2.1. Competidores
    - 2.1.1. Análisis competitivo
    - 2.1.2. Estrategias y tácticas frente a competidores
  - 2.2. Entrevistas
    - 2.2.1. Diseño de entrevistas
    - 2.2.2. Registro de entrevistas
    - 2.2.3. Análisis de entrevistas
  - 2.3. Needfinding
    - 2.3.1. User Personas
    - 2.3.2. User Task Matrix
    - 2.3.3. User Journey Mapping
    - 2.3.4. Empathy Mapping
    - 2.3.5. Big Picture EventStorming
    - 2.3.6. Ubiquitous Language
  - 2.4. Requirements specification
    - 2.4.1. User Stories
    - 2.4.2. Impact Mapping
    - 2.4.3. Product Backlog
  - 2.5. Strategic-Level Domain-Driven Design
    - 2.5.1. EventStorming
      - 2.5.1.1. Candidate Context Discovery
      - 2.5.1.2. Domain Message Flows Modeling
      - 2.5.1.3. Bounded Context Canvases
    - 2.5.2. Context Mapping
    - 2.5.3. Software Architecture
      - 2.5.3.1. Software Architecture Context Level Diagrams
      - 2.5.3.2. Software Architecture Container Level Diagrams
      - 2.5.3.3. Software Architecture Deployment Diagrams

- 2.6. Tactical-Level Domain-Driven Design
  - 2.6.1. Bounded Context: Identity & Access Management
    - 2.6.1.1. Domain Layer
    - 2.6.1.2. Interface Layer
    - 2.6.1.3. Application Layer
    - 2.6.1.4. Infrastructure Layer
    - 2.6.1.5. Bounded Context Software Architecture Component Level Diagrams
    - 2.6.1.6. Bounded Context Software Architecture Code Level Diagrams
      - 2.6.1.6.1. Bounded Context Domain Layer Class Diagrams
      - 2.6.1.6.2. Bounded Context Database Design Diagram
  - 2.6.2. Bounded Context: Appointments & Booking
    - 2.6.2.1. Domain Layer
    - 2.6.2.2. Interface Layer
    - 2.6.2.3. Application Layer
    - 2.6.2.4. Infrastructure Layer
    - 2.6.2.5. Bounded Context Software Architecture Component Level Diagrams
    - 2.6.2.6. Bounded Context Software Architecture Code Level Diagrams
      - 2.6.2.6.1. Bounded Context Domain Layer Class Diagrams
      - 2.6.2.6.2. Bounded Context Database Design Diagram
  - 2.6.3. Bounded Context: Reassignment
    - 2.6.3.1. Domain Layer
    - 2.6.3.2. Interface Layer
    - 2.6.3.3. Application Layer
    - 2.6.3.4. Infrastructure Layer
    - 2.6.3.5. Bounded Context Software Architecture Component Level Diagrams
    - 2.6.3.6. Bounded Context Software Architecture Code Level Diagrams
      - 2.6.3.6.1. Bounded Context Domain Layer Class Diagrams
      - 2.6.3.6.2. Bounded Context Database Design Diagram
  - 2.6.4. Bounded Context: Arrival & QR Check-in
    - 2.6.4.1. Domain Layer
    - 2.6.4.2. Interface Layer
    - 2.6.4.3. Application Layer
    - 2.6.4.4. Infrastructure Layer
    - 2.6.4.5. Bounded Context Software Architecture Component Level Diagrams
    - 2.6.4.6. Bounded Context Software Architecture Code Level Diagrams
      - 2.6.4.6.1. Bounded Context Domain Layer Class Diagrams
      - 2.6.4.6.2. Bounded Context Database Design Diagram
  - 2.6.5. Bounded Context: Hospital Operations & Configuration
    - 2.6.5.1. Domain Layer
    - 2.6.5.2. Interface Layer
    - 2.6.5.3. Application Layer
    - 2.6.5.4. Infrastructure Layer
    - 2.6.5.5. Bounded Context Software Architecture Component Level Diagrams
    - 2.6.5.6. Bounded Context Software Architecture Code Level Diagrams
      - 2.6.5.6.1. Bounded Context Domain Layer Class Diagrams
      - 2.6.5.6.2. Bounded Context Database Design Diagram
- Capítulo III: Solution UI/UX Design
  - 3.1. Product design
    - 3.1.1. Style Guidelines
      - 3.1.1.1. General Style Guidelines
    - 3.1.2. Information Architecture

        - 3.1.2.1. Organization Systems
        - 3.1.2.2. Labelling Systems
        - 3.1.2.3. SEO Tags and Meta Tags
        - 3.1.2.4. Searching Systems
        - 3.1.2.5. Navigation Systems
      - 3.1.3. Landing Page UI Design
        - 3.1.3.1. Landing Page Wireframe
        - 3.1.3.2. Landing Page Mock-up
      - 3.1.4. Mobile Applications UX/UI Design
        - 3.1.4.1. Mobile Applications Wireframes
        - 3.1.4.2. Mobile Applications Wireflow Diagrams
        - 3.1.4.3. Mobile Applications Mock-ups
        - 3.1.4.4. Mobile Applications User Flow Diagrams
        - 3.1.4.5. Mobile Applications Prototyping
- Capítulo IV: Product Implementation & Validation
  - 4. Product Implementation & Validation
  - 4.1. Software Configuration Management
    - 4.1.1. Software Development Environment Configuration
    - 4.1.2. Source Code Management
    - 4.1.3. Source Code Style Guide & Conventions
    - 4.1.4. Software Deployment Configuration
  - 4.2. Landing Page & Mobile Application Implementation
    - 4.2.1. Sprint 1
      - 4.2.1.1. Sprint Planning 1
      - 4.2.1.2. Aspect Leaders and Collaborators
      - 4.2.1.3. Sprint Backlog 1
      - 4.2.1.4. Development Evidence for Sprint Review
      - 4.2.1.5. Testing Suite Evidence for Sprint Review
      - 4.2.1.6. Execution Evidence for Sprint Review
      - 4.2.1.7. Services Documentation Evidence for Sprint Review
      - 4.2.1.8. Software Deployment Evidence for Sprint Review
      - 4.2.1.9. Team Collaboration Insights during Sprint
    - 4.2.2. Sprint 2
      - 4.2.2.1. Sprint Planning 2
      - 4.2.2.2. Aspect Leaders and Collaborators
      - 4.2.2.3. Sprint Backlog 2
      - 4.2.2.4. Development Evidence for Sprint Review
      - 4.2.2.5. Testing Suite Evidence for Sprint Review
      - 4.2.2.7. Services Documentation Evidence for Sprint Review
      - 4.2.2.8. Software Deployment Evidence for Sprint Review
  - 4.3. Validation Interviews
    - 4.3.1. Diseño de Entrevistas
    - 4.3.2. Registro de Entrevistas
    - 4.3.3. Evaluaciones según heurísticas
- Conclusiones
- Bibliografía
- Anexos

Student Outcome

El curso contribuye al cumplimiento del Student Outcome ABET: **ABET - EAC - Student Outcome 7 Criterio:** *La capacidad de adquirir y aplicar nuevos conocimientos según sea necesario, utilizando estrategias de aprendizaje apropiadas.* En el siguiente cuadro se describe las acciones realizadas y enunciados de conclusiones por parte del grupo, que permiten sustentar el haber alcanzado el logro del ABET – EAC - Student Outcome 7, como se detalla en la Tabla 3.

Tabla 3

Acciones y conclusiones para el Student Outcome 7

Criterio específico	Acciones realizadas	Conclusiones
Actualiza conceptos y conocimientos necesarios para su desarrollo profesional y en especial para su proyecto en soluciones de software.	**Aguilar Untiveros, Rodrigo Fabrizio** **AV1:** Apliqué nuevos conocimientos sobre EventStorming, Candidate Context Discovery, Domain Message Flows Modeling, Bounded Context Canvases, Context Mapping y el modelo C4 para documentar la arquitectura de SaludYa. Asimismo, incorporé los conceptos de las dos colas complementarias del dominio (Booking Order y Attendance Queue), refinando los bounded contexts existentes y actualizando los diagramas de EventStorming, Context Mapping y Container Level Diagram. **TB1:** Apliqué buenas prácticas de diseño de interfaces y de documentación para elaborar las guías de estilo, la arquitectura de información y la propuesta visual del Landing Page, así como las convenciones de código y la configuración de despliegue de la solución.	**AV1:** Como equipo, actualizamos de forma práctica nuestros conocimientos en el análisis y modelado de software, integrando metodologías como Lean UX, EventStorming y Domain-Driven Design (DDD). Esto nos permitió diseñar una arquitectura sólida, documentada con diagramas tácticos, estratégicos y de bases de datos, alineando la solución tecnológica a las necesidades reales del sector salud. **TB1:** Como equipo, aplicamos nuevos conocimientos de diseño UX/UI, documentación de servicios, configuración de software y análisis de nuestra colaboración en GitHub para construir los capítulos III y IV del informe y evidenciar el Sprint 1, demostrando que la actualización de conceptos nos permitió entregar artefactos de mayor calidad.

Criterio específico	Acciones realizadas	Conclusiones
	AV2: Apliqué conocimientos de configuración operativa al organizar el desarrollo en Flutter de las pantallas de intervalos y reglas del establecimiento, junto con la documentación de la ejecución del Sprint 2. **Meza Solórzano, Didier Sebastian** **AV1:** Apliqué nuevos conocimientos sobre la técnica 5W2H, el diagrama de Ishikawa, la metodología Lean UX, el análisis competitivo y el diseño de entrevistas para desarrollar de principio a fin la problemática, la solución y la validación con usuarios de SaludYa. **TB1:** Apliqué nuevos conocimientos de diseño de experiencia de usuario y de aseguramiento de calidad para elaborar los mock-ups y los user flows de las aplicaciones, y para documentar las evidencias de desarrollo, pruebas y ejecución del Sprint 1. **AV2:** Apliqué conocimientos de visualización de información al organizar el módulo Inicio en Flutter: pacientes con reserva por bloque, citas canceladas, indicadores del día, demanda por especialidad y exportación de reportes, además del diseño de las evidencias de desarrollo y pruebas. **Montoya Nina, Paula Fernanda**	**AV2:** Como equipo, aplicamos conocimientos de desarrollo móvil con Flutter/Dart y gestión ágil para distribuir las 24 historias y 84 puntos del Sprint 2 entre acceso, calendario, atención presencial, configuración e indicadores, con una visión compartida de la experiencia administrativa de SaludYa.

Criterio específico	Acciones realizadas	Conclusiones
	AV1: Investigué y apliqué conceptos avanzados de arquitectura de software para documentar el diseño a nivel de código de los Bounded Contexts. Esto incluyó la definición técnica de controladores, servicios y repositorios, plasmando esta lógica de negocio en diagramas relacionales de base de datos y diagramas de clases precisos para el proyecto. **TB1:** Apliqué conocimientos de diseño de wireframes y de gestión ágil de proyectos para elaborar los wireframes y wireflow diagrams de la aplicación, así como el Sprint Planning y el Sprint Backlog del primer sprint. **AV2:** Apliqué la planificación ágil al definir el objetivo y descomponer las 24 historias del Sprint 2; organicé el desarrollo del calendario operativo en Flutter, con selección de especialidad, consulta de bloques y edición de horarios. Actualicé las secciones de colaboración del informe y Student Outcome. **Ramos Mera, Neo Daniel** **AV1:** Apliqué nuevos conocimientos sobre el modelado táctico de Domain-Driven Design (DDD), estructurando correctamente las capas de Dominio, Aplicación, Interfaz e Infraestructura. Asimismo, aprendí y utilicé estándares de modelado C4 y UML para elaborar los diagramas de componentes, clases	

Criterio específico	Acciones realizadas	Conclusiones
	y base de datos de los Bounded Contexts asignados. **TB1:** Apliqué conocimientos de diseño de wireframes y de documentación de servicios para elaborar los wireframes y wireflow diagrams, y para evidenciar la documentación de los servicios web, el despliegue y la colaboración del equipo durante el Sprint 1. **AV2:** Apliqué conocimientos de autenticación e integración móvil al organizar el acceso del Super Admin y del personal de admisión, así como la unión de los módulos administrativos y la documentación de servicios y despliegue del Sprint 2. **Torres Juárez, Alisee Muriel** **AV1:** Apliqué nuevos conocimientos sobre EventStorming, diagramas de usuario y la redacción de User Stories para desarrollar de principio a fin la problemática, la solución y la validación de SaludYa **TB1:** Apliqué conocimientos de prototipado, de entornos de desarrollo y de control de versiones para elaborar los mock-ups, los user flows, el prototipo de la aplicación y las secciones de entorno de desarrollo y gestión del código. **AV2:** Apliqué conocimientos de interacción con la cámara y gestión de estados para organizar	

Criterio específico	Acciones realizadas	Conclusiones
	el registro de llegada por QR o código, la cola y la atención presencial, junto con la matriz de líderes y colaboradores y el análisis de colaboración del Sprint 2.	
Reconoce la necesidad del aprendizaje permanente para el desempeño profesional y el desarrollo de proyectos en soluciones de software.	**Aguilar Untiveros, Rodrigo Fabrizio** **AV1:** Reconocí la necesidad de investigar de forma autónoma técnicas de modelado estratégico y táctico que no había aplicado antes, como EventStorming y Domain-Driven Design, para sustentar con criterios arquitectónicos sólidos el diseño de SaludYa. La correcta delimitación de los bounded contexts y la identificación de las dos colas del dominio me exigieron un análisis profundo para alinear el lenguaje ubicuo con la arquitectura del sistema. **TB1:** Reconocí la importancia de investigar sobre diseño inclusivo, accesibilidad y arquitectura de la información para sustentar la propuesta visual del producto, así como de conocer las prácticas de despliegue y publicación de la solución. **AV2:** Reconocí que la implementación de reglas operativas requiere estudiar cómo representar parámetros y validaciones de manera comprensible para el Super Admin. La ejecución de estos flujos en Flutter me permitió relacionar el aprendizaje	**AV1:** El equipo reconoció que el aprendizaje autónomo es un pilar fundamental en el desarrollo del proyecto. La exigencia de investigar y dominar nuevas herramientas tecnológicas, técnicas de entrevistas y patrones arquitectónicos nos demostró que la actualización constante es la única vía para proponer soluciones de software de calidad. **TB1:** El equipo reconoció que el aprendizaje permanente también se reflejó en esta entrega, al investigar de forma autónoma sobre diseño de interfaces, documentación de API, entornos de despliegue y análisis de colaboración, competencias necesarias para documentar y sustentar el trabajo del sprint. **AV2:** El equipo reconoció que el desarrollo del módulo administrativo requiere aprender continuamente sobre Flutter/Dart, integración de servicios, estados de la atención y pruebas. La distribución de responsabilidades del Sprint 2 permitió aplicar estos

Criterio específico	Acciones realizadas	Conclusiones
	autónomo con las necesidades reales de configuración del establecimiento. **Meza Solórzano, Didier Sebastian** **AV1:** Reconocí la necesidad de seguir aprendiendo de forma autónoma metodologías que no había aplicado antes, como Lean UX y el diseño de entrevistas cualitativas, para poder sustentar con información real el desarrollo de SaludYa. **TB1:** Reconocí la necesidad de aprender sobre patrones de diseño de interfaces y de pruebas de software para sustentar los mock-ups y las evidencias de testing del sprint con criterios de calidad. **AV2:** Reconocí la necesidad de aprender a presentar indicadores y demanda por especialidad con claridad, además de verificar los filtros y formatos de los reportes operativos. El trabajo en Inicio y las pruebas del Sprint 2 reforzaron la importancia de contrastar las interfaces con sus historias de usuario. **Montoya Nina, Paula Fernanda** **AV1:** Comprendí que diseñar soluciones de software requiere una constante actualización en metodologías como DDD. Buscar y estudiar información sobre patrones de integración y persis-	conocimientos a los flujos del establecimiento y fortalecer las competencias de desarrollo móvil colaborativo.

Criterio específico	Acciones realizadas	Conclusiones
	tencia por mi cuenta me permitió resolver problemas técnicos complejos al definir las capas de infraestructura y dominio de SaludYa. **TB1:** Reconocí el valor de la planificación y priorización ágil para organizar el trabajo del equipo y de representar los flujos de usuario para sustentar el diseño del producto. **AV2:** Reconocí la necesidad de profundizar en la navegación entre especialidad, fecha y profesional, así como en las reglas de edición de bloques horarios. Al planificar y trabajar el calendario operativo del Sprint 2, relacioné el aprendizaje de Flutter con el flujo cotidiano del personal de admisión. **Ramos Mera, Neo Daniel** **AV1:** Reconocí la importancia de investigar de manera autónoma sobre arquitectura hexagonal y patrones de diseño. Entendí que este aprendizaje continuo es indispensable para definir correctamente entidades, agregados y repositorios, garantizando que el diseño del software sea escalable y mantenible en el entorno profesional. **TB1:** Reconocí la importancia de aprender sobre documentación de API y de analizar la colaboración del equipo en GitHub para evidenciar el avance del sprint.	

Criterio específico	Acciones realizadas	Conclusiones
	AV2: Reconocí que integrar los módulos administrativos exige profundizar en el control de acceso por rol y en la compatibilidad entre las pantallas Flutter y los servicios. La documentación de servicios y despliegue me permitió consolidar una estrategia de aprendizaje basada en la integración de funcionalidades. **Torres Juárez, Alisee Muriel** **AV1:** Reconocí la importancia de poner en práctica herramientas y metodologías que no había aplicado antes —como el EventStorming, los diagramas de usuario y la estructuración de User Stories para sustentar y modelar con precisión el desarrollo de SaludYa. La correcta definición de las User Stories técnicas, me exigió un análisis mucho más profundo para alinear los eventos del sistema con la arquitectura tecnológica **TB1:** Reconocí la importancia de documentar el entorno de desarrollo y aplicar buenas prácticas de control de versiones, así como del prototipado para validar la experiencia antes del desarrollo. **AV2:** Reconocí la necesidad de aprender sobre lectura de QR, registro manual y transiciones de estado de la cola para cubrir situaciones reales de atención. Este estudio guio mi trabajo en los flujos de llegada, llamado,	

Criterio específico	Acciones realizadas	Conclusiones
	atención y ausencia del Sprint 2.	

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

Objetivos SMART

En esta sección, cada integrante del equipo formula un plan de desarrollo profesional posterior a su graduación, compuesto por al menos dos objetivos **SMART**. Cada objetivo es **específico** (s), **medible** (m), **alcanzable** (a), **relevante** (r) y cuenta con un **plazo** definido (t), con el fin de evidenciar de manera clara y cuantificable cómo cada miembro planea mantener un crecimiento profesional continuo.

Objetivos SMART – Aguilar Untiveros, Rodrigo Fabrizio

Plan de desarrollo profesional: Consolidar su perfil como desarrollador de aplicaciones móviles e insertarse en el mercado laboral tecnológico, fortaleciendo sus competencias mediante certificaciones y experiencia práctica, como se detalla en la Tabla 4.

Tabla 4

Objetivos SMART – Aguilar Untiveros, Rodrigo Fabrizio

N.º	Objetivo SMART	Plazo
1	Obtener la certificación *Google Associate Android Developer* para especializarme en el desarrollo de aplicaciones móviles nativas, completando un plan de estudio de 8 horas semanales y aprobando el examen oficial con una puntuación superior a 80/100.	Diciembre 2027
2	Incorporarme como desarrollador móvil en una empresa de tecnología peruana, postulando a un mínimo de 10 ofertas por mes y participando en al menos 3 procesos de selección por trimestre hasta concretar mi primera contratación.	12 meses tras la graduación

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

Objetivos SMART – Meza Solórzano, Didier Sebastian

Plan de desarrollo profesional: Especializarse en el desarrollo multiplataforma y complementar su formación con estudios de posgrado orientados al desarrollo de software móvil, como se detalla en la Tabla 5.

Tabla 5

Objetivos SMART – Meza Solórzano, Didier Sebastian

N.º	Objetivo SMART	Plazo
1	Obtener la certificación oficial de *Flutter Developer* para dominar el desarrollo multiplataforma, desarrollando al menos 3 proyectos publicados en GitHub y aprobando la evaluación con un puntaje mínimo de 75/100.	Diciembre 2027
2	Iniciar un posgrado o especialización en desarrollo de software móvil, aprobando los primeros 4 ciclos con un promedio ponderado no menor a 16/20 y manteniendo una dedicación de 10 horas semanales de estudio.	2028

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

Objetivos SMART – Montoya Nina, Paula Fernanda

Plan de desarrollo profesional: Profundizar en la ingeniería y gestión de datos y en la arquitectura de software, combinando certificaciones técnicas con una maestría orientada a la especialización, como se detalla en la Tabla 6.

Tabla 6

Objetivos SMART – Montoya Nina, Paula Fernanda

N.º	Objetivo SMART	Plazo
1	Obtener la certificación *Google Cloud Professional Data Engineer* para especializarme en la gestión de datos, mediante un plan de estudio de 6 horas semanales y la aprobación del examen oficial con un puntaje mínimo de 70/100.	Diciembre 2027
2	Iniciar una maestría en Arquitectura de Software, culminando los dos primeros semestres con un promedio no menor a 16/20 y publicando un proyecto de arquitectura de datos al finalizar cada semestre.	2027

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

Objetivos SMART – Ramos Mera, Neo Daniel

Plan de desarrollo profesional: Consolidar su perfil en redes y desarrollo backend, integrando certificaciones técnicas con experiencia laboral especializada, como se detalla en la Tabla 7.

Tabla 7

Objetivos SMART – Ramos Mera, Neo Daniel

N.º	Objetivo SMART	Plazo
1	Obtener la certificación *Cisco Certified Network Associate (CCNA)* para fortalecer mis competencias en redes, completando 120 horas de laboratorio y aprobando el examen oficial 200-301 en el primer intento.	Junio 2027
2	Desempeñarme como desarrollador backend en una empresa tecnológica, consolidando al menos una certificación cloud (AWS o Azure) y participando en un mínimo de 8 procesos de selección anuales.	18 meses tras la graduación

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

Objetivos SMART – Torres Juárez, Alisee Muriel

Plan de desarrollo profesional: Orientar su crecimiento hacia la gestión de proyectos tecnológicos, complementando su perfil técnico con certificaciones y estudios de especialización, como se detalla en la Tabla 8.

Tabla 8

Objetivos SMART – Torres Juárez, Alisee Muriel

N.º	Objetivo SMART	Plazo
1	Obtener la certificación *Professional Scrum Master I (PSM I)* para fortalecer mis competencias en gestión de proyectos, aprobando el examen oficial con una puntuación mínima de 85% y gestionando al menos un proyecto académico bajo el marco Scrum.	Diciembre 2026
2	Ingresar a un programa de especialización en gestión de proyectos tecnológicos, aprobando los dos primeros ciclos con un promedio no menor a 16/20 y liderando al menos un proyecto real durante el programa.	2027

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

Capítulo 1: Presentación

1.1. Startup Profile

En esta sección, se presenta una descripción general de RuwaLabs, startup orientada al desarrollo de soluciones digitales que buscan atender problemáticas reales mediante el uso de la tecnología, considerando las necesidades de los usuarios y las características del contexto en el que se desenvuelven.

1.1.1. Descripción de la Startup

En el contexto actual, donde el acceso oportuno a los servicios de salud representa un desafío para diversos sectores de la población, especialmente en establecimientos públicos de salud con una alta demanda de pacientes, RuwaLabs propone desarrollar soluciones tecnológicas orientadas a mejorar la experiencia de atención y optimizar los procesos relacionados con la gestión de citas médicas.

RuwaLabs es una startup comprometida con el desarrollo de soluciones digitales innovadoras, enfocadas en facilitar el acceso a servicios y mejorar procesos mediante el uso de tecnologías móviles. Como parte de esta iniciativa, se desarrolla **SaludYa**, una solución compuesta por aplicaciones móviles dirigidas tanto a pacientes como al personal asistencial y administrativo de establecimientos públicos de salud.

SaludYa busca reducir las dificultades asociadas a la gestión tradicional de citas médicas, como las largas colas presenciales, la alta demanda de cupos y la falta de información sobre la disponibilidad de atención. Para ello, la solución contempla funcionalidades como la reserva digital de citas, lista de espera dinámica, recordatorios de atención, gestión de citas de familiares y check-in mediante código QR.

Asimismo, SaludYa busca facilitar la gestión interna de los establecimientos de salud mediante herramientas que permitan al personal administrar citas y pacientes, visualizar el estado de la atención y mejorar el aprovechamiento de los cupos disponibles, como se detalla en la Tabla 9.

Tabla 9

Misión, visión y valores de RuwaLabs

Misión	Visión	Valores
Desarrollar soluciones tecnológicas accesibles e innovadoras que permitan mejorar la experiencia de los usuarios y optimizar procesos relacionados con servicios de atención, utilizando la tecnología como herramienta para resolver necesidades reales.	Ser una startup reconocida por desarrollar soluciones digitales innovadoras que contribuyan a mejorar el acceso a servicios esenciales y la eficiencia de los procesos mediante el uso de tecnologías móviles.	Innovación, accesibilidad, compromiso, responsabilidad y orientación al usuario.

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

1.1.2. Perfiles de los integrantes del equipo

En esta sección, se presentan los perfiles de los integrantes del equipo, incluyendo sus habilidades y conocimientos técnicos relevantes para el desarrollo de **SaludYa**, como se detalla en la Tabla 10.

Tabla 10

Perfiles de los integrantes del equipo

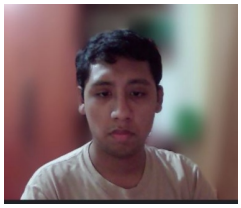
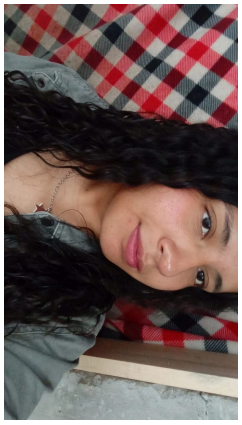
Foto	Integrante	Código	Carrera	Habilidades y conocimientos técnicos
	Aguilar Untiveros, Rodrigo Fabrizio	u202318309	Ingeniería de Software	Soy estudiante de Ingeniería de Software interesado en el desarrollo de aplicaciones móviles y en la construcción de soluciones tecnológicas que resuelvan necesidades reales de las personas. Me considero una persona responsable, organizada y con facilidad para trabajar en equipo, además de comprometida con la mejora continua y la aplicación de buenas prácticas de desarrollo. Durante el proyecto busco fortalecer mis conocimientos técnicos y aportar en la implementación de una solución funcional y de calidad.

Foto	Integrante	Código	Carrera	Habilidades y conocimientos técnicos
	Meza Solórzano, Didier Sebastian	u202319950	Ingeniería de Software	Soy estudiante de Ingeniería de Software interesado en el desarrollo de aplicaciones móviles y en soluciones tecnológicas orientadas a resolver problemas reales. Me considero una persona responsable, comprometida y con disposición para trabajar en equipo. Asimismo, busco aplicar buenas prácticas de desarrollo y mejorar continuamente mis conocimientos técnicos durante el desarrollo de proyectos.
	Montoya Nina, Paula Fernanda	u20241d934	Ingeniería de Software	Soy estudiante de Ingeniería de Software, interesada en la gestión de datos y en la arquitectura de Software. Tengo un enfoque de trabajo que prioriza la planificación y el orden estructural antes de iniciar cualquier implementación técnica. Mis fortalezas son la organización de flujos de trabajo eficiente y, además, puedo

Foto	Integrante	Código	Carrera	Habilidades y conocimientos técnicos
				desempeñar múltiples roles dentro de un proyecto, ya sea en frontend o backend, aunque prefiero dedicarme a la gestión de datos. Mi propósito es profundizar mis conocimientos en la arquitectura de Software, además de mejorar mi capacidad de colaboración en equipo para contribuir activamente en la creación de soluciones tecnológicas.
	Ramos Mera, Neo Daniel	u20241e418	Ingeniería de Software	Soy estudiante de Ingeniería de Software, interesado en la red y el backend. Me considero una persona responsable, atenta y también bastante cooperativa, ya que la comunicación y el trabajo en equipo son fundamentales para alcanzar el éxito en cualquier proyecto. Mi meta es especializarme en un campo en el que pueda trabajar con estos puntos, por lo que aspiro a participar en proyectos desafiantes que re-

Foto	Integrante	Código	Carrera	Habilidades y conocimientos técnicos
				ten mis habilidades y conocimientos.
	Torres Juárez, Alisee Muriel	U202624323	Ingeniería de Software	Soy estudiante de Ingeniería de Software interesada en la creación de soluciones tecnológicas que simplifiquen y agilicen procesos. Destaco por mi resiliencia, perseverancia y alta capacidad de adaptabilidad ante nuevos desafíos.

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

1.2. Solution Profile

En esta sección se describe el perfil de la solución propuesta por RuwaLabs, incluyendo los antecedentes y la problemática que aborda. Asimismo, se utiliza la técnica de las **5W's y 2H's** para comprender mejor el contexto, las necesidades de los usuarios y las condiciones en las que se presenta el problema.

La aplicación lleva por nombre **«SaludYa»**. El propósito de la aplicación se centra en la gestión eficiente de citas médicas en establecimientos públicos de salud, permitiendo a los pacientes acceder de forma oportuna a la reserva, seguimiento y *check-in* de sus atenciones. Al mismo tiempo, brinda al personal asistencial y administrativo herramientas para gestionar citas, pacientes y el flujo de atención.

Finalmente, **«SaludYa»** busca convertirse en una herramienta clave para conectar la gestión del paciente con la operación del establecimiento de salud, ayudando a reducir las colas presenciales, aprovechar mejor los cupos disponibles y facilitar el seguimiento de la atención. Además, permitirá contar con información oportuna y mejorar la experiencia general de acceso a los servicios de salud, como se observa en la Figura 6.

Figura 6

Identidad visual de SaludYa

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

1.2.1. Antecedentes y problemática

En esta sección se presentan los antecedentes y la problemática que la solución propuesta busca abordar. Para ello, se utiliza la técnica de las **5W's y 2H's** para realizar un análisis detallado del problema y el **diagrama de Ishikawa** para identificar sus principales causas raíz.

Antecedentes

En los establecimientos públicos de salud existe una alta demanda de atención médica, lo que puede generar dificultades para acceder oportunamente a una cita. Los pacientes pueden encontrarse con procesos presenciales, disponibilidad limitada de cupos, tiempos de espera elevados y poca información sobre las citas disponibles. Estas dificultades pueden afectar especialmente a las personas que deben trasladarse desde zonas urbanas periféricas hacia establecimientos públicos de salud.

Actualmente, el acceso a una cita médica en establecimientos del **Ministerio de Salud (MINSA)** puede realizarse mediante diferentes canales, como la atención presencial, vía telefónica y determinados servicios digitales. La existencia de estos canales representa un avance en la digitalización del proceso; sin embargo, todavía existen oportunidades para integrar la gestión de citas, la disponibilidad de cupos, las cancelaciones, las listas de espera y el seguimiento de la atención en una experiencia única para el paciente y el personal del establecimiento.

El MINSA ha implementado iniciativas orientadas a reducir los tiempos de espera y facilitar la gestión de citas. Por ejemplo, en 2017 se implementó el sistema **«Web Colas»** en el Hospital de Emergencias Villa El Salvador, permitiendo consultar información relacionada con los tiempos de atención y la cantidad de pacientes en espera (MINSA, 2017). Esta iniciativa evidencia la necesidad de utilizar herramientas digitales para mejorar la organización de la atención y disminuir las dificultades asociadas a las colas presenciales.

Asimismo, en 2023 el MINSA informó sobre diferentes acciones destinadas a reducir los tiempos de espera en establecimientos de salud de Lima Metropolitana, entre ellas sistemas de gestión de citas, listas de espera y atención mediante plataformas web (MINSA, 2023).

En 2024, el MINSA también informó sobre el fortalecimiento del sistema de **«Citas en Línea»**, mediante el cual se generaron citas digitales en establecimientos de primer nivel de atención (MINSA, 2024). Este tipo de iniciativas demuestra que existe una tendencia hacia la digitalización del acceso a los servicios de salud.

Por otro lado, durante el primer semestre de 2025, el Hospital San Juan de Lurigancho registró más de **93 000 pacientes** que accedieron a sus citas mediante canales telefónicos (MINSA, 2025), evidenciando la demanda por mecanismos alternativos que faciliten el acceso a la atención médica.

A nivel internacional, la **Organización Mundial de la Salud (OMS)** reconoce el potencial de la salud digital para fortalecer los sistemas sanitarios, mejorar su eficiencia y contribuir a un acceso más equitativo a servicios de salud de calidad.

Frente a esta problemática, surge **«SaludYa»**, una solución tecnológica compuesta por dos aplicaciones móviles: una orientada a pacientes y otra dirigida al personal asistencial y administrativo. La solución está diseñada para reducir las colas presenciales, aprovechar mejor los cupos disponibles y facilitar el seguimiento de la atención en establecimientos públicos de salud.

Técnica de las 5W's y 2H's

What?

¿Cuál es el problema?

Existe dificultad para acceder de manera oportuna y eficiente a citas médicas en establecimientos públicos de salud debido a la disponibilidad limitada de cupos, los procesos presenciales y la falta de información actualizada sobre las citas disponibles.

¿Cuál es la relación con la persona en cuestión?

Los pacientes, especialmente aquellos que residen en zonas urbanas periféricas, enfrentan directamente esta situación al buscar una atención oportuna. Por otro lado, el personal asistencial y administrativo es responsable de gestionar las citas y el flujo de atención del establecimiento.

When?

¿Cuándo sucede el problema?

La problemática se presenta principalmente durante la búsqueda y programación de una cita, especialmente en periodos de alta demanda o cuando existe una cantidad limitada de cupos. También puede presentarse el día de la atención debido a la congestión y los tiempos de espera.

¿Cuándo utiliza el cliente el producto?

Los usuarios utilizan **SaludYa** como parte de su proceso habitual de búsqueda de disponibilidad, reserva de citas, seguimiento de su atención y *check-in* al llegar al establecimiento de salud.

Where?
¿Dónde está el cliente cuando usa el producto?

Los pacientes pueden acceder a la aplicación desde sus hogares, centros de trabajo o estudio, e incluso durante su desplazamiento hacia el establecimiento de salud. Por su parte, el personal asistencial y administrativo utiliza la solución dentro de los establecimientos públicos de salud.

¿A dónde se dirige?

La problemática afecta principalmente a pacientes que utilizan establecimientos públicos de salud, en especial a quienes residen en zonas urbanas periféricas y buscan un acceso más oportuno a la atención médica.

¿Dónde surge el problema?

El problema surge en establecimientos públicos de salud que aún dependen parcialmente de procesos presenciales o telefónicos, especialmente en las áreas relacionadas con admisión, gestión de citas y atención de pacientes.

Who?
¿Quiénes están involucrados?

Los principales involucrados son:

- Los pacientes que utilizan los establecimientos públicos de salud.
- Los pacientes que residen en zonas urbanas periféricas.
- El personal administrativo encargado de gestionar las citas.
- El personal asistencial encargado de atender a los pacientes.
- Los establecimientos públicos de salud que administran los procesos de atención.

Why?
¿Cuál es la causa del problema?

Las principales causas están relacionadas con:

- La alta demanda de atención médica.
- La disponibilidad limitada de cupos.
- La dependencia de procesos presenciales o telefónicos.
- La falta de información en tiempo real sobre la disponibilidad de citas.
- La gestión limitada de cancelaciones y reprogramaciones.
- La ausencia de una gestión eficiente de listas de espera.
- La existencia de procesos administrativos fragmentados.

How?
¿En qué condiciones los clientes usan nuestros productos?

Los pacientes suelen consultar diferentes canales para encontrar una cita disponible, trasladarse al establecimiento de salud o comunicarse telefónicamente. Estas acciones se realizan muchas veces en contextos de alta demanda, tiempos de espera elevados e información limitada sobre la disponibilidad de citas.

¿Cómo prefieren los clientes acceder a nuestro producto?

Los usuarios prefieren una aplicación móvil intuitiva que les permita consultar la disponibilidad de citas, reservar una atención, recibir recordatorios y realizar el seguimiento de su atención sin necesidad de trasladarse previamente al establecimiento o realizar llamadas telefónicas.

¿Qué llevó al cliente a llegar a esta situación?

La dependencia de procesos presenciales y telefónicos, la falta de información en tiempo real sobre la disponibilidad de citas y la ausencia de una gestión integrada de cupos, cancelaciones y listas de espera han llevado a los pacientes y al personal de salud a requerir soluciones digitales que permitan optimizar el acceso y la gestión de la atención.

How much?

¿Cuánto impacto genera este problema en los establecimientos de salud?

El impacto de esta problemática es significativo, ya que genera costos de tiempo y desplazamiento para los pacientes y una mayor carga operativa para el personal de los establecimientos de salud.

Además, las iniciativas implementadas por el MINSA para digitalizar las citas y reducir los tiempos de espera evidencian la importancia de optimizar estos procesos mediante herramientas tecnológicas.

Diagrama de Ishikawa - Análisis de Causas

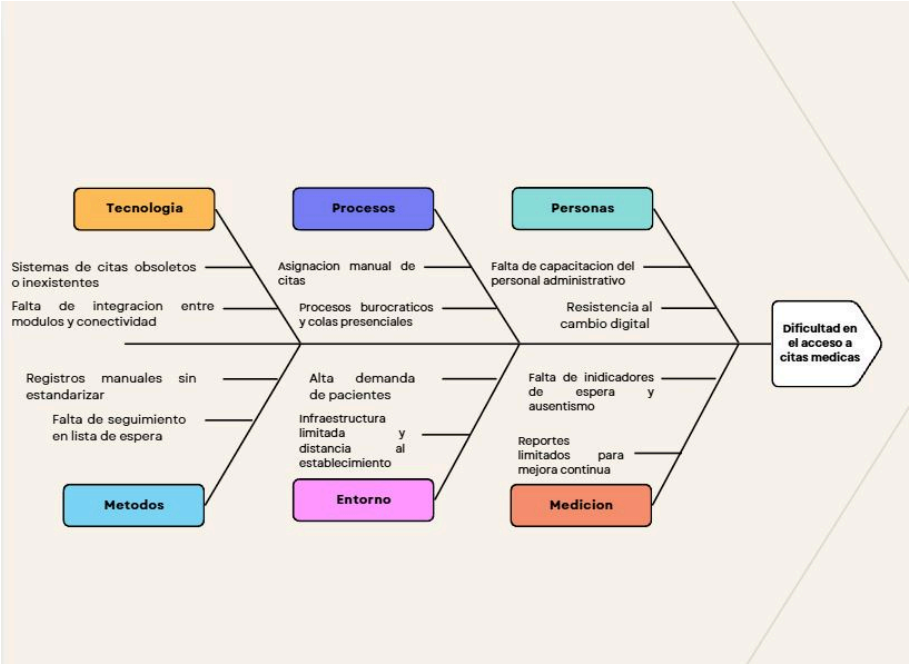
El **diagrama de Ishikawa**, también conocido como diagrama de espina de pescado o diagrama de causa--efecto, permite identificar y visualizar de manera sistemática las múltiples causas que contribuyen al problema central:

> **«Dificultad en el acceso oportuno y eficiente a citas médicas en establecimientos públicos de salud».**

Este análisis estructurado facilita la comprensión integral del problema y orienta el desarrollo de soluciones específicas para cada categoría de causas identificadas.

Figura 7

Diagrama de Ishikawa: causas de las dificultades para acceder a citas médicas



Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

El diagrama identifica **seis categorías principales de causas** que contribuyen al problema, como se observa en la Figura 7:

- **Tecnología:** Sistemas de citas obsoletos o inexistentes, plataformas que pueden presentar dificultades ante una alta demanda, falta de integración entre módulos y problemas de conectividad en determinados establecimientos.
- **Procesos:** Asignación manual de citas, falta de estandarización en la gestión de turnos, procesos burocráticos que pueden generar colas presenciales y dificultades para gestionar cancelaciones o reprogramaciones.
- **Personas:** Falta de capacitación del personal administrativo, resistencia al cambio hacia herramientas digitales, diferentes niveles de conocimiento digital de los usuarios y disponibilidad limitada de personal frente a una alta demanda.
- **Métodos:** Utilización de registros manuales, ausencia de mecanismos de priorización, falta de seguimiento de pacientes en listas de espera y posibles duplicidades o inconsistencias en los registros.
- **Entorno:** Infraestructura limitada, alta demanda de pacientes, condiciones socioeconómicas que pueden dificultar el acceso y distancia entre el domicilio del paciente y el establecimiento de salud.
- **Medición:** Falta de indicadores relacionados con el ausentismo, tiempos de espera y satisfacción de los pacientes, así como disponibilidad limitada de reportes para apoyar la mejora continua.

1.2.2. Lean UX Process

1.2.2.1. Lean UX Problem Statement

Actualmente, tanto los pacientes como el personal asistencial y administrativo de los establecimientos públicos de salud enfrentan dificultades por la dependencia de procesos presenciales y telefónicos que no permiten conocer en tiempo real la disponibilidad de citas, gestionar cancelaciones o listas de espera, ni dar seguimiento eficiente a la atención. Esto genera problemas como colas prolongadas, cupos desaprovechados, desinformación sobre la disponibilidad de citas y una carga operativa elevada para el personal. Además, muchos pacientes no cuentan con mecanismos accesibles que les permitan orientar el tipo de atención que podrían requerir antes de acudir al establecimiento.

¿Cómo podríamos diseñar una solución digital integral que permita a los pacientes reservar y gestionar sus citas médicas en tiempo real, acceder a listas de espera dinámicas y recibir orientación inicial sobre sus síntomas, mientras se brinda al personal asistencial y administrativo herramientas para gestionar el flujo de atención y aprovechar mejor los cupos disponibles?

1.2.2.2. Lean UX Assumptions

1.2.2.2.1 Business Assumptions

- Existe una demanda no atendida de pacientes en establecimientos públicos de salud que requieren mecanismos digitales para acceder a citas médicas de forma oportuna.
- Las dificultades de acceso a citas están vinculadas a la dependencia de procesos presenciales y telefónicos, generando una necesidad urgente de digitalización.
- El personal asistencial y administrativo de los establecimientos está interesado en herramientas que faciliten la gestión de citas y el flujo de pacientes.
- Los establecimientos públicos de salud buscan reducir tiempos de espera y mejorar el aprovechamiento de sus cupos disponibles.
- Existen iniciativas del MINSA orientadas a la digitalización de citas, lo que evidencia una tendencia favorable para la adopción de SaludYa.

1.2.2.2.2 Business Outcomes

- Queremos que los establecimientos de salud reduzcan en un 30% las colas presenciales durante los primeros 6 meses de implementación.
- Buscamos aumentar en un 25% el aprovechamiento de los cupos disponibles mediante la gestión de listas de espera y cancelaciones.
- Queremos reducir en un 35% el tiempo promedio que el personal dedica a la gestión manual de citas, en un plazo de 3 meses.
- Esperamos que el 50% de los establecimientos piloto recomienden la herramienta a otras sedes dentro de los primeros 3 meses de uso.

1.2.2.2.3 User Assumptions

- Los pacientes no cuentan actualmente con mecanismos digitales integrados, sino con canales presenciales o telefónicos fragmentados.
- Tienen interés en acceder a información oportuna sobre la disponibilidad de citas, pero no siempre cuentan con los medios para hacerlo.
- Necesitan una solución simple, rápida y accesible desde su dispositivo móvil que no requiera desplazamientos innecesarios.
- Les preocupa perder su turno o no ser informados sobre cambios en su cita.
- Valoran contar con recordatorios y una orientación inicial sobre el tipo de atención que podrían requerir.

1.2.2.2.4 User Outcomes

- El 80% de los pacientes reportan una reducción en el tiempo dedicado a gestionar sus citas después de 3 meses de uso.
- El 70% de los pacientes disminuyen sus desplazamientos innecesarios al establecimiento gracias a la reserva y el seguimiento digital.
- Reducir en un 30% el tiempo de espera percibido por los pacientes en el establecimiento gracias al check-in mediante código QR.
- El 65% del personal administrativo reporta una mejora en el control del flujo de pacientes y el aprovechamiento de cupos.
- El 70% de los usuarios completan el proceso de reserva de una cita sin asistencia técnica en menos de 5 minutos.

1.2.2.2.5 Features

- Si implementamos una funcionalidad que permita a los pacientes consultar disponibilidad y reservar citas médicas en tiempo real, entonces se reducirán los desplazamientos innecesarios y las colas presenciales.
- Si los pacientes cuentan con una lista de espera dinámica y reciben notificaciones ante cupos liberados por cancelaciones, entonces se aprovechará mejor la disponibilidad existente y se reducirán los tiempos de espera.
- Si el personal asistencial y administrativo dispone de una aplicación que les permita visualizar el flujo de pacientes y gestionar cambios o cancelaciones, entonces podrán optimizar la atención y reducir la carga operativa asociada a la gestión manual de citas.

1.2.2.3. Lean UX Hypothesis Statements

- **Hypothesis Statement 1**

Creemos que lograremos una reducción en las colas presenciales y un mejor aprovechamiento de los cupos disponibles. **Sabremos que** si los pacientes **Obtienen** visibilidad en tiempo real sobre la disponibilidad de citas y la posibilidad de unirse a una lista de espera dinámica **Cuando veamos** una solución digital que permita reservar, gestionar y recibir notificaciones sobre sus citas médicas.

- **Hypothesis Statement 2**

Creemos que lograremos un aumento en la satisfacción de los pacientes y una reducción en los tiempos de espera percibidos. **Sabremos que** los pacientes **Cuando veamos** mejor acceso a recordatorios, orientación inicial sobre síntomas y check-in mediante código QR **Con** una funcionalidad que agilice su llegada y atención en el establecimiento.

- **Hypothesis Statement 3**

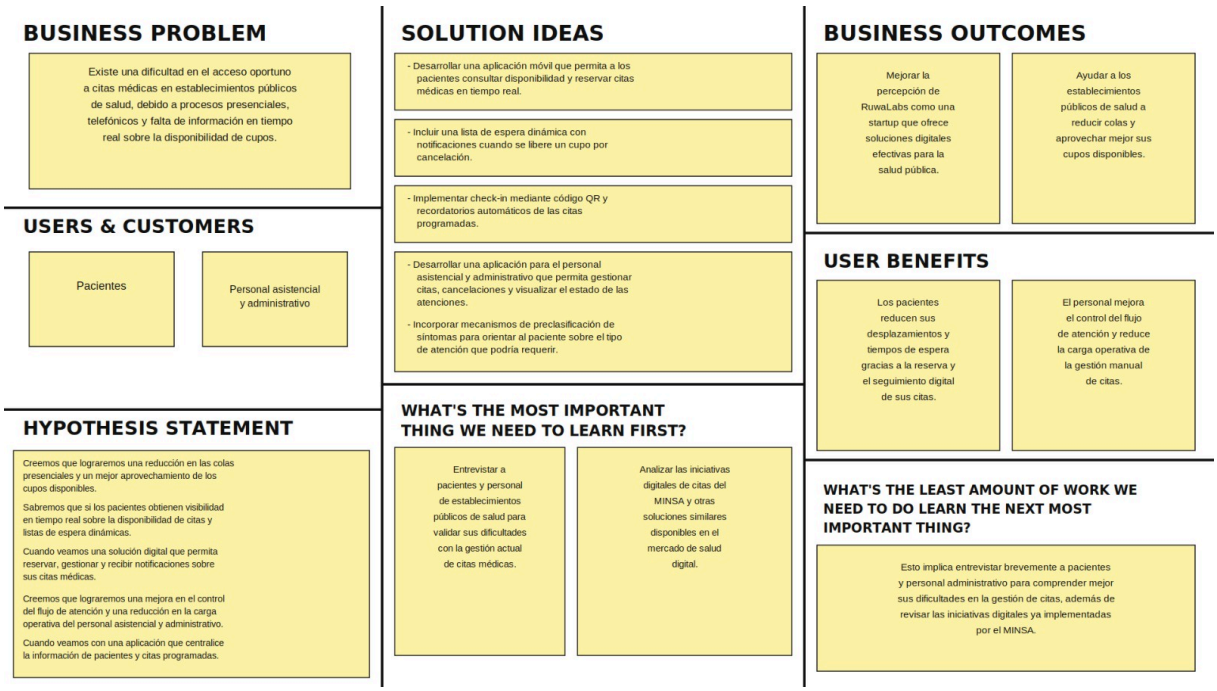
Creemos que lograremos una mejora en el control del flujo de atención y una reducción en la carga operativa del personal. **Sabremos que** si el personal asistencial y administrativo **Obtiene** la capacidad de gestionar cambios, cancelaciones y el estado de las atenciones en tiempo real **Cuando veamos** con una aplicación que centralice la información de pacientes y citas programadas.

1.2.2.4. Lean UX Canvas

El Lean UX Canvas es una herramienta metodológica que permite sintetizar y visualizar de manera estructurada los elementos clave del proyecto SaludYa. Este canvas facilita la comprensión integral del problema de negocio, las soluciones propuestas, los usuarios objetivo y los resultados esperados, proporcionando una base sólida para el desarrollo ágil del producto.

Figura 8

Lean UX Canvas de SaludYa



Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

Lean Ux Canvas

La imagen presenta una matriz dividida en ocho secciones que abordan desde la identificación del problema de acceso a citas médicas hasta los beneficios específicos para pacientes y personal de salud. Aquí se define el problema de negocio relacionado con la dificultad de acceso oportuno a citas médicas en establecimientos públicos de salud, seguido de las ideas de solución que incluyen la reserva digital de citas, la lista de espera dinámica y el check-in mediante código QR. Los resultados empresariales se enfocan en mejorar la percepción de RuwaLabs y ayudar a los establecimientos públicos de salud a reducir colas y optimizar el aprovechamiento de sus cupos disponibles, como se observa en la Figura 8.

1.3. Segmentos objetivos

A continuación, se determinan los segmentos objetivos a los que va dirigida la propuesta de solución **SaludYa** para la gestión de citas médicas en establecimientos públicos de salud.

Segmento objetivo 1: Pacientes de zonas urbanas periféricas que acuden a establecimientos públicos de salud

- **Datos demográficos:**
  - Edad: Entre 18 y 65 años, incluyendo adultos que gestionan sus propias citas y las de familiares o dependientes (hijos, adultos mayores).
  - Ocupación: Diversa, incluyendo personas que trabajan, estudian o se dedican a labores del hogar.
  - Lugar de residencia: Zonas urbanas periféricas de Lima Metropolitana, Perú.
- **Perfil:** Pacientes que acuden con cierta frecuencia a postas, centros de salud o establecimientos materno-infantiles públicos, y que muchas veces también gestionan la atención médica de familiares o personas a su cargo.
- **Problema:** La dependencia de procesos presenciales o telefónicos para conseguir una cita genera tiempos de espera elevados, cupos limitados, desinformación sobre la disponibilidad y el riesgo de perder citas ya conseguidas por falta de recordatorios.
- **Necesidad:** Una solución digital accesible desde el celular que permita reservar citas médicas, conocer la disponibilidad en tiempo real, recibir notificaciones cuando se libere un cupo y gestionar tanto sus propias citas como las de sus familiares o dependientes.

Segmento objetivo 2: Personal asistencial y administrativo de establecimientos públicos de salud

- **Datos demográficos:**
  - Edad: Mayores de 18 años.
  - Ocupación: Personal administrativo, de admisión, enfermeros y técnicos de establecimientos públicos de salud.
  - Lugar de residencia: Lima Metropolitana, Perú.
- **Perfil:** Personal responsable de la gestión de citas, la atención de pacientes y el control del flujo de atención dentro de los establecimientos públicos de salud.
- **Problema:** La asignación manual de cupos, la gestión de cancelaciones, el control de pacientes que no se presentan y el seguimiento de la sala de espera generan una carga operativa elevada y dificultan la visualización del estado de la atención en tiempo real.
- **Necesidad:** Una herramienta digital que centralice la gestión de citas y pacientes, permita visualizar en tiempo real el estado de la atención y la demanda, y facilite el control de cancelaciones, inasistencias y listas de espera.

Capítulo II: Requirements Development and Software Solution Design

2.1. Competidores

2.1.1. Análisis competitivo

> **¿Por qué llevar a cabo este análisis?**
> Mediante este análisis competitivo buscamos identificar las principales características, fortalezas, debilidades y propuestas de valor de las soluciones existentes en el mercado. Esto nos permitirá reconocer oportunidades de diferenciación y definir cómo **SaludYa** puede ofrecer un mayor valor a los pacientes y al personal de los establecimientos públicos de salud, como se detalla en la Tabla 11 y la Figura 9, la Figura 10, la Figura 11 y la Figura 12.

Tabla 11

Análisis competitivo de SaludYa, Doctoralia, Cita Médica y MINSA

Competitive Analysis Landscape	SaludYa	Doctoralia	Cita Médica	Citas en Línea (MINSA)
Logo	**Figura 9** *Logotipo de SaludYa*	**Figura 10** *Logotipo de Doctoralia*	**Figura 11** *Logotipo de Cita Médica*	**Figura 12** *Logotipo del MINSA*
Perfil / Overview	Es una solución compuesta por dos aplicaciones móviles que conectan a pacientes y personal de establecimientos públicos de salud, permitiendo la reserva de citas, la gestión de listas de espera y el seguimiento del flu-	Doctoralia es una plataforma internacional líder en reserva de citas médicas, que conecta a pacientes con más de 29 000 especialistas y clínicas privadas registrados, permitiendo agendar consultas presenciales o por videollamada.	Cita Médica es una aplicación peruana que permite reservar citas médicas en consultorios y clínicas privadas, de forma presencial o por videoconsulta.	Es una iniciativa del Ministerio de Salud (MINSA) que permite generar citas digitales en establecimientos de primer nivel de atención y hospitales seleccionados, como parte de la digitalización del Seguro Integral de Salud (SIS).

Competitive Analysis Landscape	SaludYa	Doctoralia	Cita Médica	Citas en Línea (MINSA)
	jo de atención en tiempo real.			
Ventaja competitiva *¿Qué valor ofrece a los clientes?*	Enfoque específico en establecimientos públicos de salud, conectando la gestión del paciente con la operación interna del establecimiento mediante lista de espera dinámica, check-in por QR y pre-filtro de síntomas.	Amplia red de especialistas verificados, videoconsultas, recordatorios automáticos y chat directo con el médico.	Consulta de precios de medicamentos y recetas en farmacias cercanas, además de recordatorios y chat privado con el médico.	Acceso gratuito y dirigido específicamente a la población que se atiende en establecimientos públicos de salud, con respaldo institucional del Estado.
Perfil de Marketing *Mercado objetivo*	Pacientes de zonas urbanas periféricas y personal asistencial y administrativo de establecimientos públicos de salud.	Pacientes que buscan atención médica privada, así como especialistas y clínicas privadas.	Pacientes que buscan atención médica privada en consultorios y clínicas de Perú.	Pacientes asegurados al SIS y usuarios de establecimientos públicos de salud en Lima y otras regiones.
Estrategias de marketing	Alianzas con establecimientos públicos de salud y difusión mediante campañas informativas y redes sociales.	Posicionamiento en redes sociales, SEO y alianzas con seguros médicos privados.	Posicionamiento en tiendas de aplicaciones (App Store, Google Play) y redes sociales.	Difusión institucional a través de comunicados del MINSA y de cada establecimiento de salud.
Productos & Servicios	App para pacientes (reserva de citas, recordatorios, lista de espera dinámica, pre-filtro de síntomas, gestión de familiares y check-in por QR) y app para personal (gestión de citas, visualiza-	Búsqueda de especialistas por ciudad o seguro médico, reserva de citas presenciales o virtuales, recordatorios, videoconsultas y mensajería con el especialista.	Búsqueda de médicos y consultorios cercanos, reserva de citas presenciales o por videoconsulta, recordatorios y consulta de precios de medicamentos.	Registro y reserva de citas médicas de forma virtual en hospitales y centros de salud seleccionados.

Competitive Analysis Landscape	SaludYa	Doctoralia	Cita Médica	Citas en Línea (MINSA)
	ción del flujo de atención y control de cancelaciones e inasistencias).			
Perfil de Producto *Precios & Costos*	Al estar dirigida a establecimientos públicos de salud, se plantea como un servicio sin costo directo para el paciente, con un modelo de implementación institucional.	Gratuito para pacientes; planes de suscripción mensual para profesionales y clínicas (Plus, VIP, entre otros).	Aplicación gratuita para el paciente.	Gratuito, al ser un servicio público.
Canales de distribución *Web y/o Móvil*	Web y Móvil	Web y Móvil	Móvil (iOS y Android)	Móvil (aplicaciones por hospital) y, en algunos casos, Web
Fortalezas	Conecta la experiencia del paciente con la operación interna del establecimiento, reduciendo colas y mejorando el aprovechamiento de los cupos disponibles.	Amplia base de especialistas y funcionalidades avanzadas como videoconsultas y lista de espera inteligente.	Integración de información sobre precios de medicamentos y recetas, además de comunicación directa con el médico.	Cobertura directa dentro del sistema público de salud y respaldo institucional del MINSA.
Análisis SWOT — Debilidades	Requiere una conexión estable a internet y la adopción digital del personal administrativo del establecimiento.	Enfocado en el sector privado, sin cobertura de establecimientos públicos de salud.	Enfocado únicamente en consultorios y clínicas privadas.	Cobertura limitada a determinados hospitales, sin funcionalidades como lista de espera dinámica, check-in por QR o gestión interna del flujo de atención.
Análisis SWOT — Oportunidades	Alineación con las iniciativas de digitalización del MIN-	Expansión hacia convenios con aseguradoras y	Expansión hacia convenios con es-	Expansión a más establecimientos de salud a nivel

Competitive Analysis Landscape	SaludYa	Doctoralia	Cita Médica	Citas en Línea (MINSA)
	SA y expansión a más establecimientos públicos de salud.	nuevas especialidades.	tablecimientos públicos de salud.	nacional e integración de nuevas funcionalidades.
Análisis SWOT — Amenazas	Competencia de plataformas privadas ya consolidadas y resistencia al cambio en instituciones públicas.	Aparición de nuevas plataformas especializadas por sector, como establecimientos públicos.	Competencia de plataformas más consolidadas como Doctoralia.	Falta de mantenimiento o actualización tecnológica constante al depender de presupuesto público.

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

2.1.2. Estrategias y tácticas frente a competidores

Una vez realizado la identificación de fortalezas, oportunidades, debilidades y amenazas con el análisis FODA de nuestros competidores en el sector del mercado, pasaremos a plantear las estrategias y tácticas para hacerle frente a estos mismos.

- Expansión de integraciones: Integrar SaludYa con los sistemas de información del MINSA y del Seguro Integral de Salud (SIS), así como con los sistemas hospitalarios (HIS) de los establecimientos, facilitando la interoperabilidad y el registro unificado de citas.
- Soporte Postventa: Brindar soporte técnico y asesoría continua al personal asistencial y administrativo, garantizando el máximo aprovechamiento de las funcionalidades y una adopción institucional sostenida en el tiempo.
- Visibilidad y seguimiento en tiempo real: Incorporar capacidades de seguimiento del flujo de pacientes, cupos y listas de espera mediante herramientas accesibles y de fácil comprensión tanto para el paciente como para el personal.
- Diferenciación por soporte y capacitación: Ofrecer capacitación digital al personal asistencial y administrativo de los establecimientos públicos de salud, muchos de los cuales presentan resistencia o poca familiaridad con herramientas tecnológicas.
- Alianzas estratégicas con el sector público: Establecer alianzas con el MINSA, redes de salud y DIRIS para promover una adopción institucional temprana y respaldada oficialmente.
- MVP enfocado en lo esencial: Iniciar con un MVP centrado en las funcionalidades esenciales —reserva de citas, lista de espera dinámica y notificaciones— para minimizar riesgos y validar la solución antes de escalar nuevas funciones.

2.2. Entrevistas

2.2.1. Diseño de entrevistas

Las entrevistas fueron diseñadas como guías semiestructuradas, considerando tres entrevistas por cada segmento objetivo: pacientes de zonas urbanas periféricas y personal asistencial y administrativo de establecimientos públicos de salud.

2.2.2. Registro de entrevistas

En esta sección, se registra cada entrevista realizada. En total, se realizaron tres entrevistas por cada segmento objetivo. Se detalla el nombre del miembro entrevistador y el del entrevistado. Además, se redacta un resumen general del contenido de la entrevista realizada.

Segmento Objetivo 1: Pacientes de zonas urbanas periféricas que acuden a establecimientos públicos de salud

Entrevista 1

La entrevista a Braulio Núñez recoge su experiencia con la gestión de citas médicas y las necesidades identificadas para SaludYa, como se detalla en la Tabla 12 y la Figura 13.

Tabla 12

Ficha de entrevista — Registro de la entrevista a Braulio Núñez

Entrevista	Registro
Figura 13 *Registro de la entrevista a Braulio Núñez* Nota. Evidencia de las entrevistas realizadas por el equipo RuwaLabs; vídeos en el Anexo A.	**Distrito:** Comas **Entrevistado:** Braulio Núñez
Link	**Entrevistador:** Didier Sebastián Meza Solórzano
Timing: Minuto 00:00-04:22	**Resumen:** Braulio Núñez, de 24 años, es técnico de mantenimiento y acude con frecuencia al Centro de Salud Comas para llevar a su sobrino a sus controles y vacunas. Relata que, para conseguir una cita, suele madrugar desde las cuatro y media de la mañana, ya que si llega más tarde ya no encuentra cupos disponibles en pediatría, y en alguna ocasión tuvo que regresar al día siguiente por no lograr atención. Menciona que casi nunca puede resolver esto por teléfono, pues las líneas del establecimiento no responden, por lo que siempre debe acudir de forma presencial. También comenta que una vez perdió una cita ya conseguida porque no contaba con ningún recordatorio. En cuanto a tecnología, utiliza un celular Android en el que emplea con frecuencia WhatsApp, Facebook y aplicaciones de delivery, y se muestra cómodo realizando trámites desde el celular. Considera que una aplicación que le muestre la disponibilidad de citas en tiempo real, le envíe notificaciones cuando se libere un cupo y respete el horario reservado, le ahorraría mucho tiempo y evitaría que tenga que madrugar sin certeza de conseguir atención.

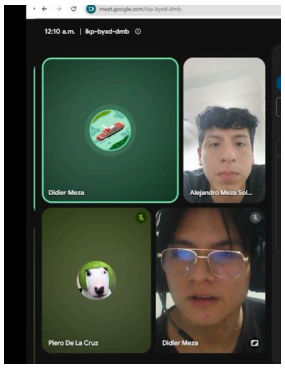
Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

Entrevista 2

La entrevista a Yordi Salazar recoge su experiencia con la gestión de citas médicas y las necesidades identificadas para SaludYa, como se detalla en la Tabla 13 y la Figura 14.

Tabla 13

Ficha de entrevista — Registro de la entrevista a Yordi Salazar

Entrevista	Registro
Figura 14 *Registro de la entrevista a Yordi Salazar*  Nota. Evidencia de las entrevistas realizadas por el equipo Ruwa-Labs; vídeos en el Anexo A.	**Distrito:** Villa María del Triunfo **Entrevistado:** Yordi Salazar
Link	**Entrevistador:** Didier Sebastián Meza Solórzano
Timing: Minuto 00:00-04:06	**Resumen:** Yordi Salazar, de 27 años, es dueño de una tienda de abarrotes y acude junto con su madre a la posta de salud de su zona de forma mensual. Cuenta que en su última experiencia tuvo que ir hasta tres veces distintas antes de conseguir cupo, ya que las dos primeras veces ya no había disponibilidad al momento de llegar. Señala que la parte más complicada es no saber cuánta gente hay antes en la fila, y que en una ocasión perdió una cita porque solo le avisaron la fecha de forma verbal, sin ningún respaldo escrito. Cuando no logra conseguir cupo, opta por acudir a una clínica particular, aunque esto le representa un gasto adicional. En cuanto a tecnología, utiliza un celular sencillo y no se siente del todo cómodo con trámites digitales, por lo que suele apoyarse en su hijo para este tipo de gestiones. Considera que una aplicación sencilla, con letras grandes y pocos pasos, que le permita conocer la disponibilidad de citas y avisarle mediante llamada o mensaje de texto cuando se libere un cupo, sería de gran ayuda para evitar las largas colas que actualmente enfrenta.

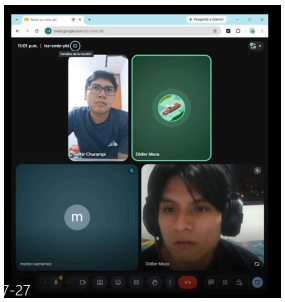
Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

Entrevista 3:

La entrevista a Kevin Huamán recoge su experiencia con la gestión de citas médicas y las necesidades identificadas para SaludYa, como se detalla en la Tabla 14 y la Figura 15.

Tabla 14

Ficha de entrevista — Registro de la entrevista a Kevin Huamán

Entrevista	Registro
Figura 15 *Registro de la entrevista a Kevin Huamán*  Nota. Evidencia de las entrevistas realizadas por el equipo RuwaLabs; vídeos en el Anexo A.	**Distrito:** San Juan de Lurigancho **Entrevistado:** Kevin Huamán
Link	**Entrevistador:** Didier Sebastián Meza Solórzano
Timing: Minuto 00:00-05:35	**Resumen:** Kevin Huamán, de 22 años, se dedica al reparto mediante aplicaciones de delivery y acude ocasionalmente al centro materno infantil de su zona. Relata que la última vez que necesitó una cita tuvo que pedir el día libre en su trabajo, ya que ni siquiera sabía que existía otra forma de agendar una atención que no fuera de manera presencial. Señala que en una oportunidad llegó al establecimiento y ya no había citas disponibles para medicina general, lo que le hizo perder tiempo de trabajo sin obtener ningún resultado. A diferencia de otros pacientes, se siente muy cómodo utilizando aplicaciones móviles, pues las emplea constantemente para su trabajo de reparto, mapas y redes sociales. Considera que una aplicación confiable, que no se cuelgue y tenga buenas reseñas, que le permita reservar su cita al toque y recibir notificaciones push cuando se libere un cupo, le permitiría organizar mejor su tiempo de trabajo y evitar viajes innecesarios solo para consultar disponibilidad.

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

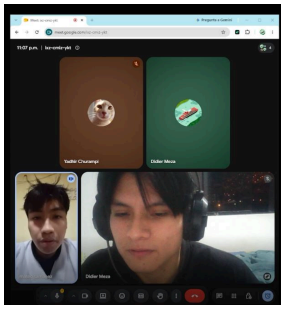
Segmento Objetivo 2: Personal asistencial y administrativo de establecimientos públicos de salud

Entrevista 4:

La entrevista a Franco Alanoca recoge su experiencia con la gestión de citas médicas y las necesidades identificadas para SaludYa, como se detalla en la Tabla 15 y la Figura 16.

Tabla 15

Ficha de entrevista — Registro de la entrevista a Franco Alanoca

Entrevista	Registro
Figura 16 *Registro de la entrevista a Franco Alanoca*  Nota. Evidencia de las entrevistas realizadas por el equipo Ruwa-Labs; vídeos en el Anexo A.	**Distrito:** San Juan de Lurigancho **Entrevistado:** Franco Alanoca
Link	**Entrevistador:** Didier Sebastián Meza Solórzano
Timing: Minuto 00:00-06:00	**Resumen:** Franco Alanoca, de 26 años, es técnico de admisión en un centro de salud de San Juan de Lurigancho, cargo que desempeña desde hace dos años. Su labor consiste en registrar a los pacientes conforme llegan y asignar los cupos del día según el orden de llegada, sin que exista un sistema de reserva previa. Explica que cuando los cupos se agotan simplemente se informa a los pacientes que regresen al día siguiente, y que no cuentan con un mecanismo formal para gestionar cancelaciones o inasistencias, por lo que esos cupos terminan perdiéndose. Señala que el principal problema es la desorganización que se genera cuando hay mucha demanda, especialmente en las mañanas, lo que ocasiona reclamos y un ambiente tenso. Actualmente, la gestión se apoya en cuadernos físicos y hojas de Excel, lo que en ocasiones provoca pérdida de información. Considera que contar con un sistema que muestre en tiempo real la disponibilidad de cupos y el estado de cada paciente eliminaría gran parte del registro manual repetitivo y ayudaría a evitar confusiones en la atención diaria.

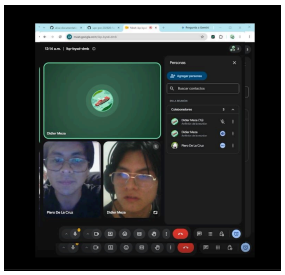
Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

Entrevista 5

La entrevista a Wilmer Contreras recoge su experiencia con la gestión de citas médicas y las necesidades identificadas para SaludYa, como se detalla en la Tabla 16 y la Figura 17.

Tabla 16

Ficha de entrevista — Registro de la entrevista a Wilmer Contreras

Entrevista	Registro
Figura 17 *Registro de la entrevista a Wilmer Contreras*  Nota. Evidencia de las entrevistas realizadas por el equipo Ruwa-Labs; vídeos en el Anexo A.	**Distrito:** Comas **Entrevistado:** Wilmer Contreras
Link	**Entrevistador:** Didier Sebastián Meza Solórzano
Timing: Minuto 0:00-04:25	**Resumen:** Wilmer Contreras, de 29 años, se desempeña como jefe de admisión en una posta de salud de Comas desde hace cinco años. Entre sus funciones se encuentra supervisar al personal de admisión, coordinar la distribución de cupos entre especialidades y atender los reclamos de los pacientes. Explica que el proceso actual depende de un cuaderno físico donde se revisa la disponibilidad y se asignan horarios aproximados que no siempre se cumplen, y que cuando los cupos se agotan se deriva a los pacientes a otros establecimientos o se les pide regresar otro día. Menciona que no existe un registro formal de cancelaciones ni de inasistencias, lo que dificulta reasignar los cupos liberados de manera oportuna. Identifica como principal problema la falta de un sistema centralizado que muestre la disponibilidad real de citas, lo que genera colas largas y personal saturado, especialmente los lunes y a inicios de mes. Considera que automatizar la asignación de cupos y las notificaciones a los pacientes sería clave para mejorar la atención y reducir la carga operativa del personal.

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

Entrevista 6

Tabla 17

Ficha de entrevista — Registro de la entrevista a Deyvis Ochante

Entrevista	Registro
Figura 18 *Registro de la entrevista a Deyvis Ochante* Nota. Evidencia de las entrevistas realizadas por el equipo Ruwa-Labs; vídeos en el Anexo A.	**Distrito:** Villa María del Triunfo **Entrevistado:** Deyvis Ochante **Edad:** 25 años
Ver entrevista	**Entrevistador:** Didier Sebastián Meza Solórzano
Timing: 00:00-04:10	**Resumen:** Deyvis Ochante, de 25 años, es técnico de enfermería encargado de la sala de espera en un centro materno infantil de Villa María del Triunfo, donde labora desde hace tres años. Su función principal es organizar el orden de atención de los pacientes y apoyar en el registro cuando es necesario. Relata que actualmente no existe un horario exacto asignado para cada paciente, y que el control de la sala de espera se realiza mediante una lista escrita a mano. Señala que el principal problema es no poder anticipar cuántos pacientes llegarán realmente cada día, lo que se agrava en las mañanas y durante campañas de vacunación. Además, indica que buscar las historias clínicas físicas de cada paciente le toma bastante tiempo, ya que en ocasiones se encuentran mal archivadas. Considera que contar con la información del paciente de forma digital, visible para todo el personal, ayudaría a reducir la dependencia del papel y a mejorar el seguimiento de la atención en los días de mayor demanda.

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

Los vídeos que respaldan las entrevistas se reúnen en el Anexo A, como se detalla en la Tabla 17 y la Figura 18.

2.2.3. Análisis de entrevistas

En primer lugar, con base en las tres entrevistas realizadas al primer segmento objetivo, conformado por los pacientes de zonas urbanas periféricas, se puede concluir lo siguiente:

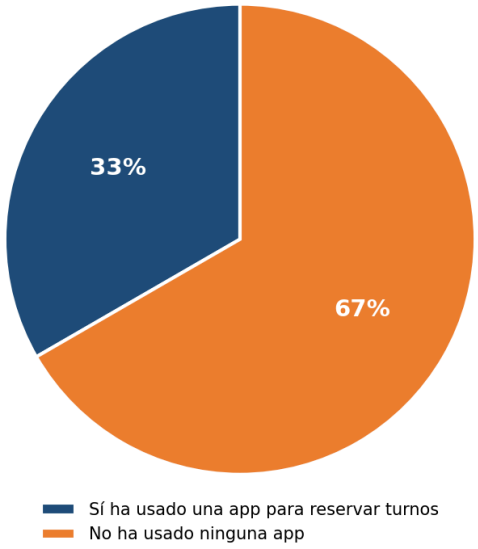
- Los usuarios perciben que el proceso actual para conseguir una cita médica presencial no satisface sus necesidades, ya que depende de madrugar y hacer largas colas sin certeza de conseguir cupo, lo cual afecta su tiempo laboral y personal.
- Aunque algunos usuarios no han utilizado antes una aplicación para reservar citas médicas, sí manifiestan interés en contar con una herramienta digital que les permita conocer la disponibilidad en tiempo real. Consideran que esto les brindaría mayor seguridad, ahorro de tiempo y una mejor planificación de sus actividades diarias.
- Los usuarios requieren una aplicación que les permita conocer la disponibilidad de citas, recibir notificaciones ante cupos liberados y gestionar sus citas sin necesidad de acudir físicamente al establecimiento. Esta solución debe estar orientada a reducir la incertidumbre del proceso actual y facilitar el acceso oportuno a la atención médica.

A continuación, se presentan los porcentajes destacados en las respuestas de los entrevistados a las preguntas planteadas:

- Uso previo de aplicaciones para reservar citas o turnos:

Figura 19

Uso de aplicaciones móviles para reservar citas entre los pacientes entrevistados



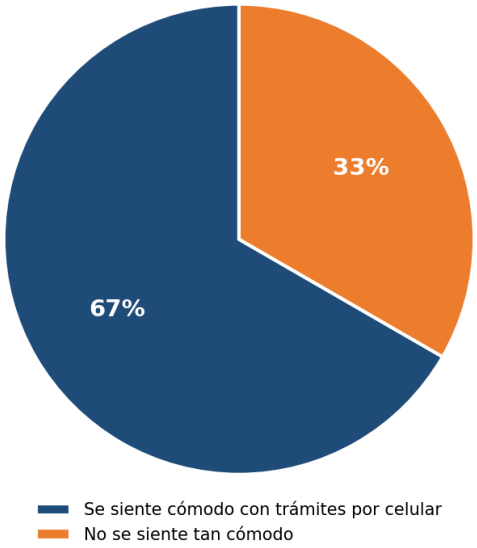
Nota. Elaboración del equipo RuwaLabs a partir de las entrevistas documentadas en el Anexo A.

En esta imagen, se visualiza una relación de respuestas sobre el tema planteado. Luego del análisis a este gráfico, se concluye que la mayoría de los entrevistados nunca ha utilizado una aplicación para reservar una cita médica, aunque sí han usado aplicaciones similares para otros rubros como restaurantes o bancos, como se observa en la Figura 19.

- Comodidad realizando trámites desde el celular:

Figura 20

Comodidad de los pacientes entrevistados al utilizar el celular



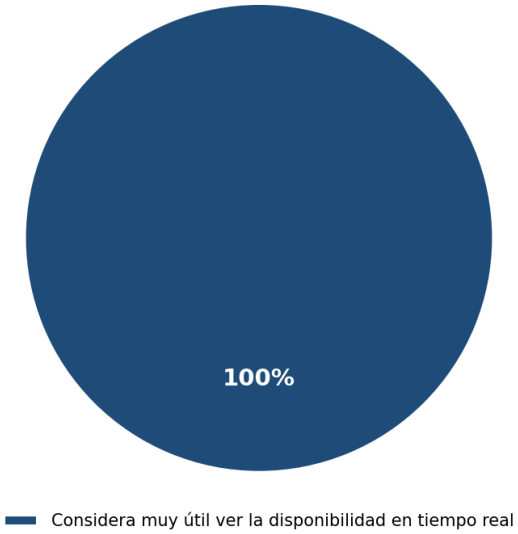
Nota. Elaboración del equipo RuwaLabs a partir de las entrevistas documentadas en el Anexo A.

En esta imagen, se visualiza una relación de respuestas sobre el tema planteado. Luego del análisis a este gráfico, se concluye que la mayoría de los entrevistados se siente cómodo realizando trámites desde su celular. Sin embargo, hay una pequeña parte que aún depende de un familiar para este tipo de gestiones, como se observa en la Figura 20.

- Utilidad de ver la disponibilidad de citas en tiempo real:

Figura 21

Utilidad percibida de consultar la disponibilidad de citas



Nota. Elaboración del equipo RuwaLabs a partir de las entrevistas documentadas en el Anexo A.

En esta imagen, se visualiza una relación de respuestas sobre el tema planteado. Luego del análisis a este gráfico, se concluye que todos los entrevistados consideran muy útil poder conocer la disponibilidad de citas desde su celular antes de acudir al establecimiento, como se observa en la Figura 21.

Segundo, con base en las tres entrevistas realizadas al segundo segmento objetivo, conformado por el personal asistencial y administrativo, se puede concluir lo siguiente:

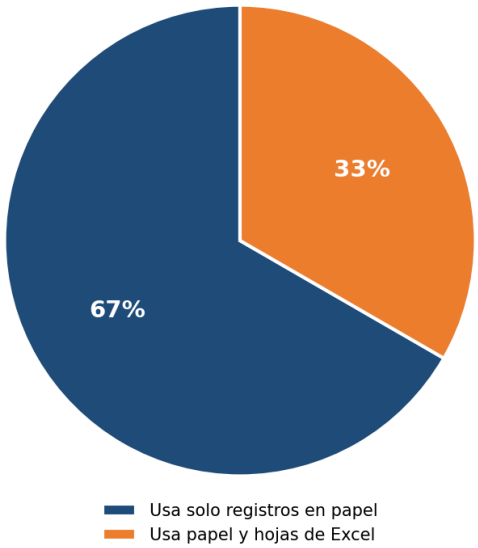
- El personal reconoce que el proceso actual de gestión de citas depende en gran medida de registros manuales, lo cual puede generar errores, pérdida de información y una atención desorganizada cuando hay mucha demanda.
- El personal enfrenta dificultades para controlar en tiempo real la disponibilidad de cupos, las cancelaciones y el flujo de pacientes en espera, lo que puede generar malentendidos, reclamos y sobrecarga operativa.
- El personal considera que contar con un sistema digital que centralice la información de los pacientes y muestre el estado de la atención en tiempo real facilitaría considerablemente su trabajo diario, permitiendo una mejor organización y un servicio más eficiente.

A continuación, se presentan los porcentajes destacados en las respuestas de los entrevistados a las preguntas planteadas:

- Sistema de gestión utilizado actualmente:

Figura 22

Sistema actual de gestión de citas del personal entrevistado



Nota. Elaboración del equipo RuwaLabs a partir de las entrevistas documentadas en el Anexo A.

En esta imagen, se visualiza una relación de respuestas sobre el tema planteado. Luego del análisis a este gráfico, se concluye que la mayoría del personal entrevistado gestiona las citas únicamente con registros en papel, mientras que una parte más pequeña combina el papel con hojas de Excel, como se observa en la Figura 22.

- Necesidad de un sistema con información en tiempo real:

Figura 23

Necesidad de un sistema digital según el personal entrevistado

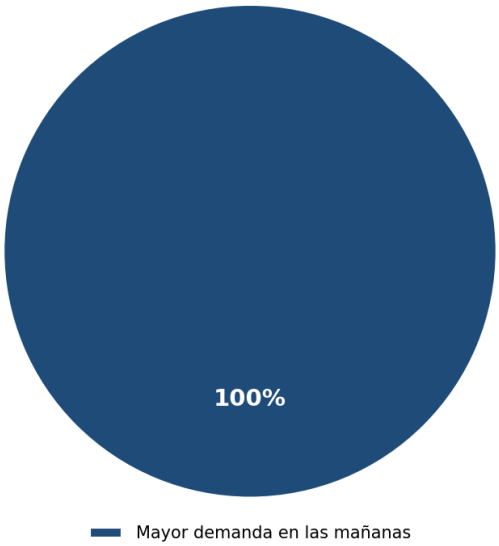
Nota. Elaboración del equipo RuwaLabs a partir de las entrevistas documentadas en el Anexo A.

En esta imagen, se visualiza una relación de respuestas sobre el tema planteado. Luego del análisis a este gráfico, se concluye que todos los entrevistados consideran necesario contar con un sistema que les muestre información en tiempo real sobre los cupos y el estado de los pacientes, como se observa en la Figura 23.

- Momento del día con mayor cantidad de pacientes:

Figura 24

Especialidades de mayor demanda según las entrevistas al personal



Nota. Elaboración del equipo RuwaLabs a partir de las entrevistas documentadas en el Anexo A.

En esta imagen, se visualiza una relación de respuestas sobre el tema planteado. Luego del análisis a este gráfico, se concluye que todos los entrevistados coinciden en que las mañanas son el momento de mayor afluencia de pacientes, como se observa en la Figura 24.

2.3. Needfinding

Con el análisis de datos completado, pasamos a la fase de Needfinding (Búsqueda de Necesidades). El objetivo es empatizar con los usuarios para diseñar una solución técnica y humana que elimine fricciones administrativas y brinde tranquilidad a las familias. Mediante herramientas como User Personas, User Task Matrix, User Journey Mapping y Empathy Mapping, definiremos quiénes son nuestros usuarios, sus acciones, emociones y necesidades reales.

2.3.1. User Personas

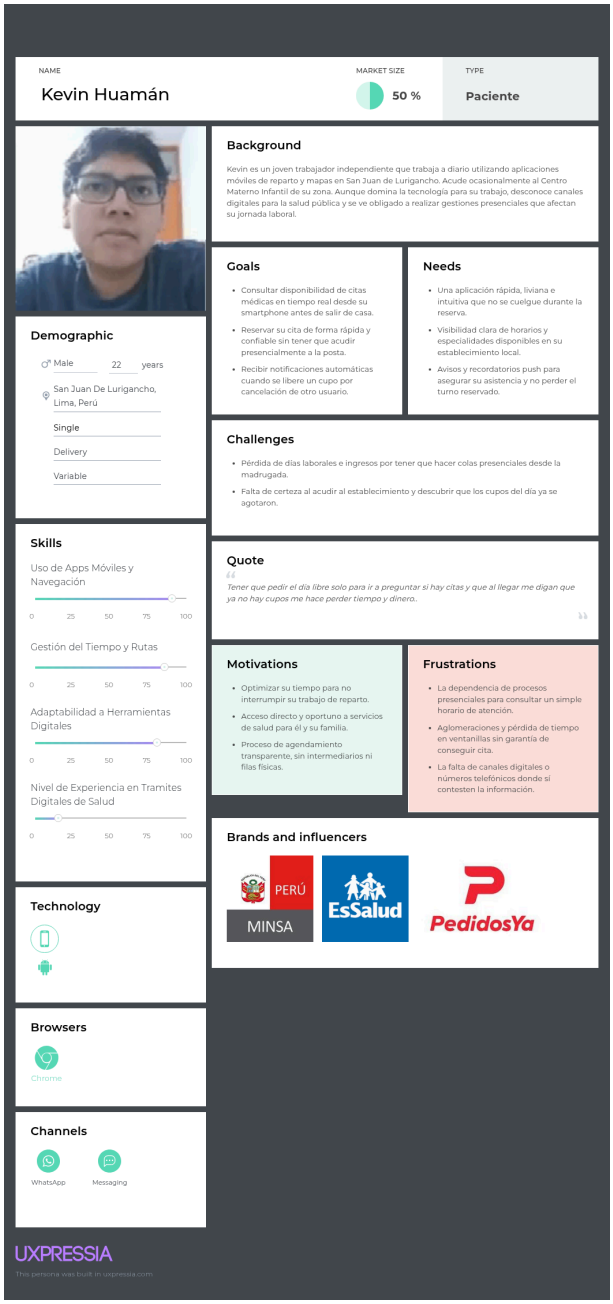
A partir de la investigación realizada con personas de zonas periféricas y del entorno de la salud pública, se definieron dos perfiles clave de usuario. Estos representan los segmentos principales que interactúan directa e indirectamente con la plataforma: los **pacientes de zonas urbanas periféricas** que buscan un acceso ágil y oportuno a sus citas médicas, y el **personal asistencial y administrativo**, encargado de la gestión operativa diaria.

Comprender sus dinámicas cotidianas, barreras y prioridades permite diseñar una experiencia accesible, eficiente y adaptada a la realidad de la atención pública.

Segmento Objetivo 1: Pacientes de zonas urbanas periféricas que acuden a establecimientos públicos de salud

Figura 25

User persona del paciente



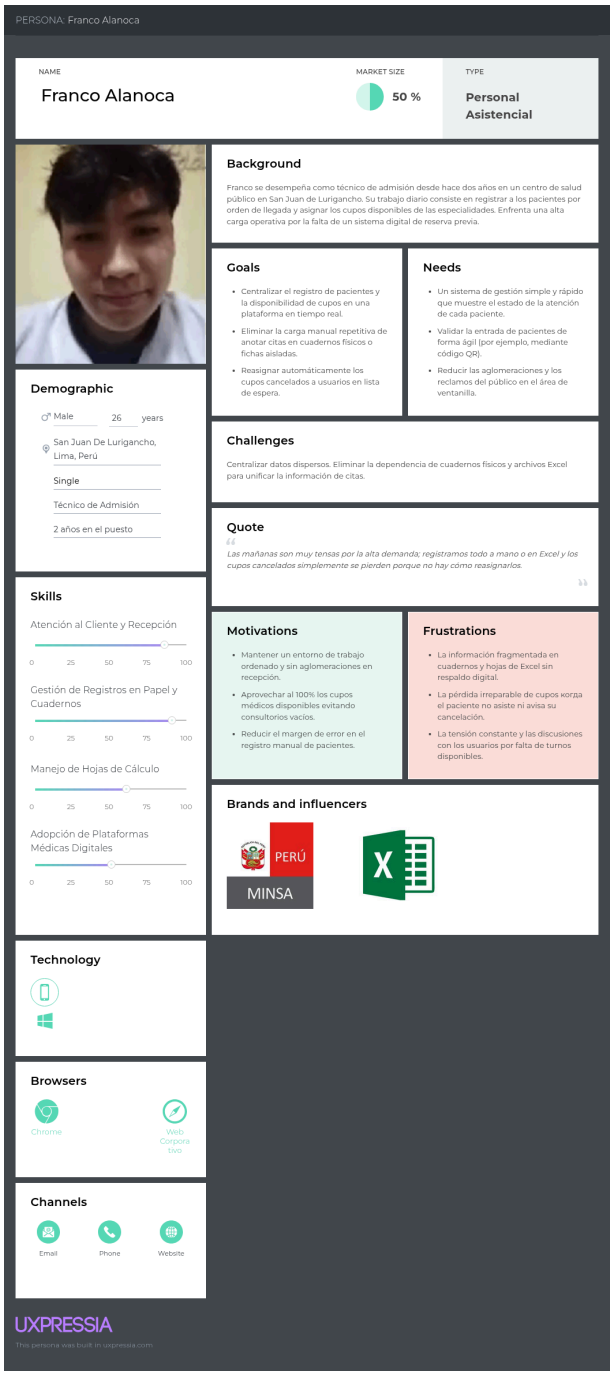
Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

Kevin Huamán, de 22 años, es repartidor en San Juan de Lurigancho y domina bien la tecnología para su trabajo, pero acude poco al centro materno infantil de su zona por depender de procesos presenciales. Busca poder consultar disponibilidad y reservar citas desde su celular, y recibir notificaciones cuando se libere un cupo, evitando perder tiempo y días de trabajo yendo sin certeza de conseguir atención, como se observa en la Figura 25.

Segmento Objetivo 2: Personal asistencial y administrativo de establecimientos públicos de salud

Figura 26

User persona del personal asistencial y administrativo



Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

Franco Alanoca, de 26 años, es técnico de admisión en un centro de salud de San Juan de Lurigancho, donde registra pacientes y asigna cupos de forma manual con cuadernos y Excel. Busca un sistema que centralice el registro y la disponibilidad de citas en tiempo real, reduzca la carga manual y permita reasignar automáticamente los cupos cancelados, como se observa en la Figura 26.

2.3.2. User Task Matrix

Para el siguiente análisis pensamos en dos segmentos principales los cuales podrán utilizar el software **SaludYa**: el **Paciente de zonas urbanas periféricas**, que requiere un acceso ágil para agendar citas propias o de sus dependientes, y el **Personal asistencial y administrativo**, encargado de operar la admisión, el flujo de atención y el control de cupos dentro de los establecimientos públicos de salud. Ambos interactúan con el dominio del problema desde perspectivas distintas pero complementarias, y las tareas identificadas son realizadas por cada segmento con independencia de la existencia de cualquier solución tecnológica, como se detalla en la Tabla 18.

Tabla 18

User Task Matrix

Tarea	Paciente (Zonas Periféricas)	Personal Asistencial / Administrativo	Frecuencia (Paciente)	Importancia (Paciente)	Frecuencia (Personal)	Importancia (Personal)
Consultar disponibilidad de cupos y horarios	X	X	High	High	High	High
Solicitar o asignar un cupo / cita médica	X	X	Medium	High	High	High
Registrar o gestionar citas de familiares y dependientes	X		Medium	High	Low	Medium
Confirmar la asistencia o recibir recordatorios de cita	X		Medium	High	Low	Medium
Realizar el check-in / registro de llegada al establecimiento	X	X	Medium	High	High	High
Gestionar la lista de espera y	X	X	Low	High	High	High

Tarea	Paciente (Zonas Periféricas)	Personal Asistencial / Administrativo	Frecuencia (Paciente)	Importancia (Paciente)	Frecuencia (Personal)	Importancia (Personal)
reasignación de cupos liberados						
Controlar el flujo y estado de la atención en sala de espera		X	Low	Low	High	High
Registrar admisión y actualizar datos en registros/sistemas		X	Low	Low	High	High

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

Análisis del Task Matrix

A partir del User Task Matrix elaborado, se identifican los siguientes hallazgos:

- **Tareas con mayor frecuencia e importancia para ambos segmentos:** Consultar la disponibilidad de cupos, solicitar/asignar citas y realizar el registro de llegada (check-in) representan el núcleo del proceso de atención. Ambos segmentos les asignan importancia **High**, reflejando que la desinformación en tiempo real y las largas colas presenciales son los cuellos de botella principales que SaludYa busca resolver tanto para el usuario como para el personal.
- **Principales diferencias entre segmentos:** El personal asistencial y administrativo concentra sus actividades en tareas de gestión operativa interna de frecuencia **High**, como controlar el flujo de la sala de espera y registrar admisiones para evitar la pérdida de información en cuadernos o archivos físicos. En cambio, el paciente enfoca su interacción en tareas puntuales de alta importancia (**High**), como la gestión de citas para familiares dependientes y el seguimiento de recordatorios para evitar la pérdida de turnos previamente conseguidos.
- **Coincidencias relevantes:** La gestión de la lista de espera ante cancelaciones e inasistencias es percibida con importancia **High** por ambas partes. Para el personal, representa una oportunidad crítica para reducir la sobrecarga operativa y optimizar el aprovechamiento de los cupos disponibles; para el paciente, constituye el mecanismo clave para acceder a un cupo liberado de forma oportuna sin necesidad de madrugar o realizar viajes innecesarios.

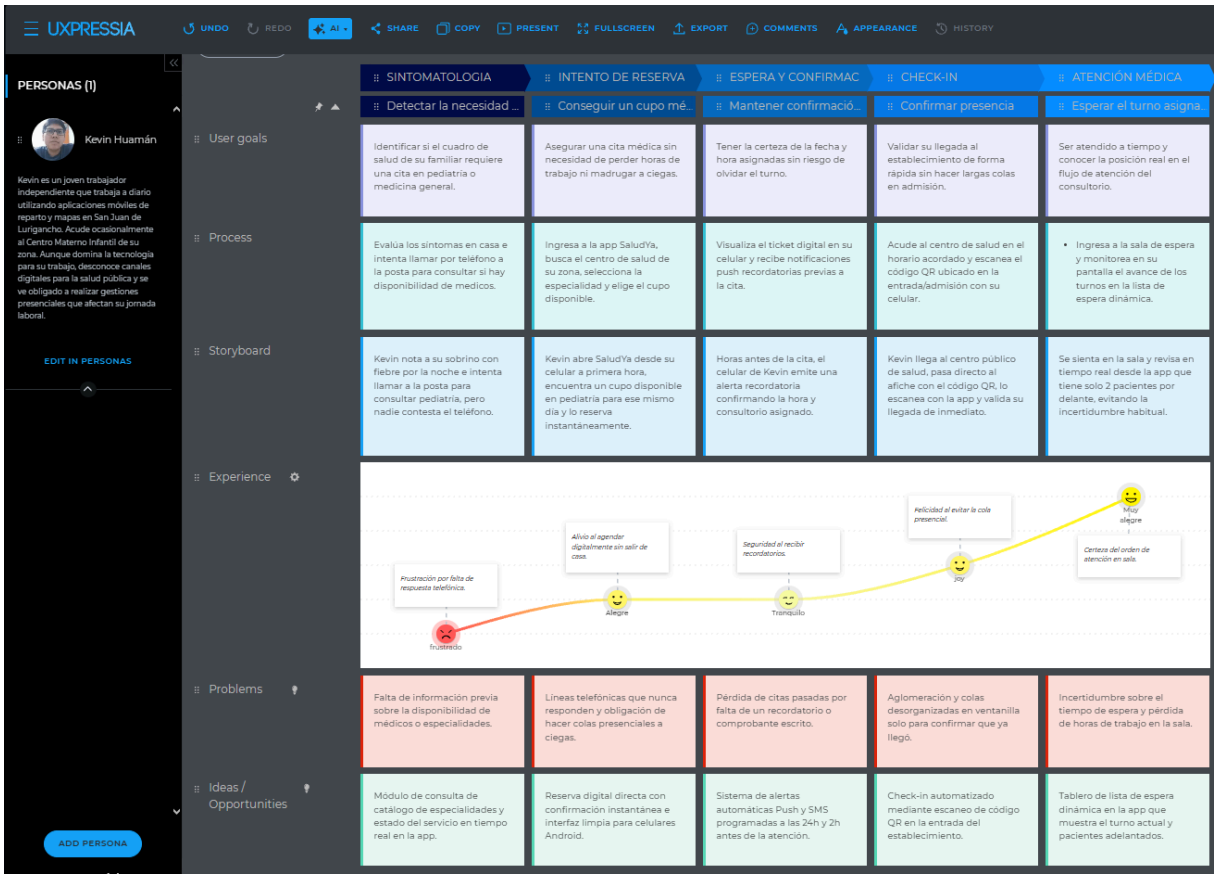
2.3.3. User Journey Mapping

A partir de los hallazgos obtenidos en las entrevistas con pacientes y personal de salud, se elaboraron los User Journey Maps utilizando la herramienta UXPressia. Estos esquemas analizan la experiencia de cada segmento a lo largo del proceso de atención, contrastando las fricciones de la gestión presencial/manual actual contra las oportunidades de optimización que introduce la plataforma **SaludYa**.

Segmento 1: Pacientes de zonas urbanas periféricas

Figura 27

User journey del paciente



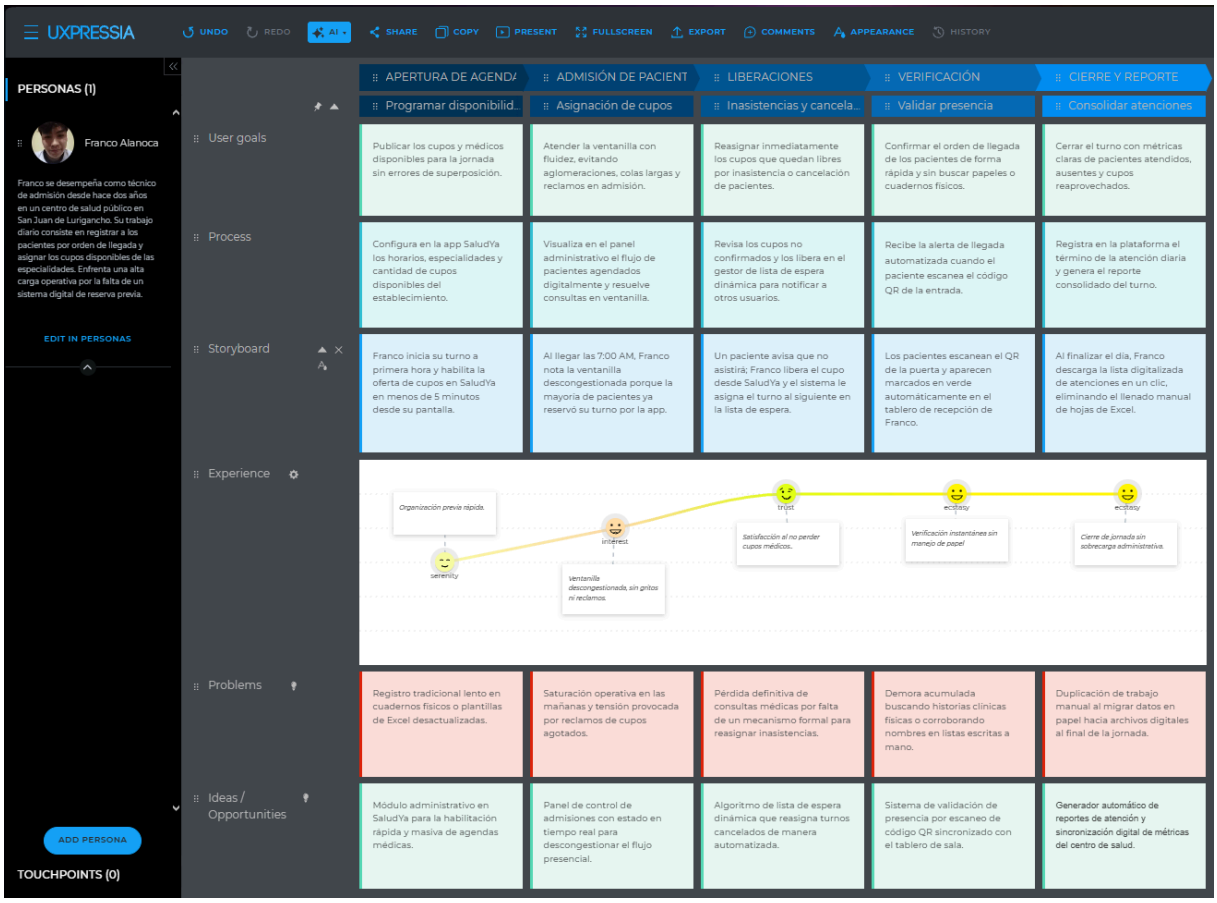
Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

El recorrido de Kevin abarca cinco etapas: sintomatología, intento de reserva, espera y confirmación, check-in y atención médica. Su experiencia inicia con frustración al no obtener respuesta telefónica de la posta, mejora al reservar digitalmente desde SaludYa y recibir confirmación con QR, y culmina en alegría al evitar la cola presencial y conocer su posición real en la sala de espera, como se observa en la Figura 27.

Segmento 2: Personal asistencial y administrativo

Figura 28

User journey del personal asistencial y administrativo



Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

El recorrido de Franco cubre cinco etapas: apertura de agenda, admisión de pacientes, liberaciones, verificación y cierre con reporte. Su experiencia pasa de la serenidad al organizar los cupos del día, a la satisfacción de una ventanilla descongestionada, hasta la total conformidad al cerrar el turno sin sobrecarga administrativa gracias a la digitalización del registro, como se observa en la Figura 28.

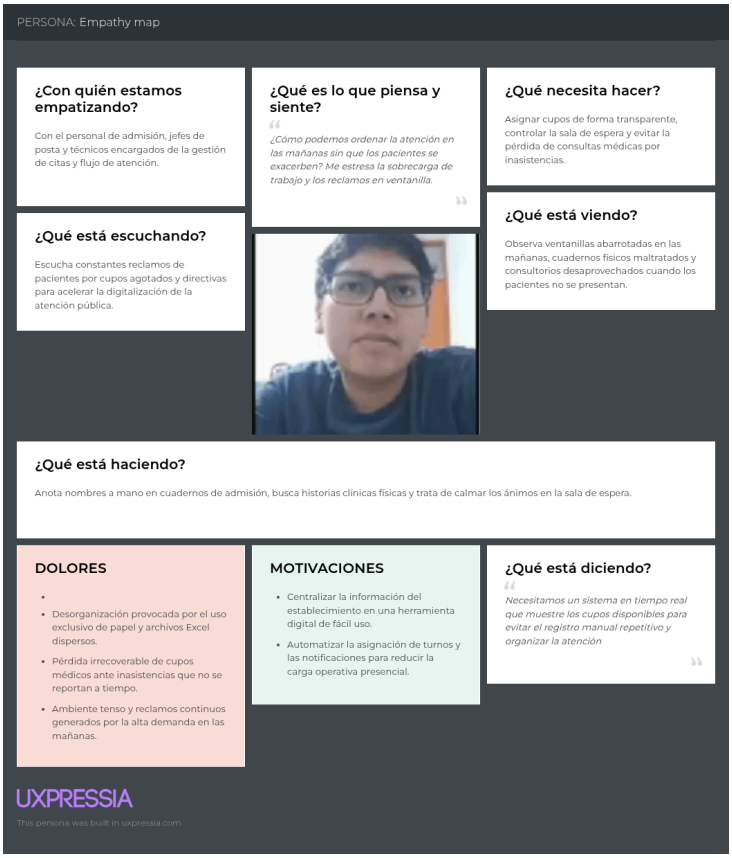
2.3.4. Empathy Mapping

El diseño de una solución de software orientada a la salud pública requiere comprender no solo las acciones operativas de los usuarios, sino también sus vivencias emocionales y percepciones del servicio. En este sentido, el Empathy Mapping nos permite trascender el perfil demográfico tradicional para examinar las dinámicas internas de nuestros segmentos objetivo. Al sistematizar lo que el paciente y el personal de salud oyen, ven, dicen, piensan y hacen, se identifican las barreras críticas y las expectativas del dominio. Este análisis asegura que SaludYa no solo sea una plataforma funcionalmente robusta, sino también una herramienta que genere confianza, previsibilidad y bienestar en la atención diaria.

Segmento 1: Pacientes de zonas urbanas periféricas

Figura 29

Empathy Map - Paciente de Zonas Periféricas



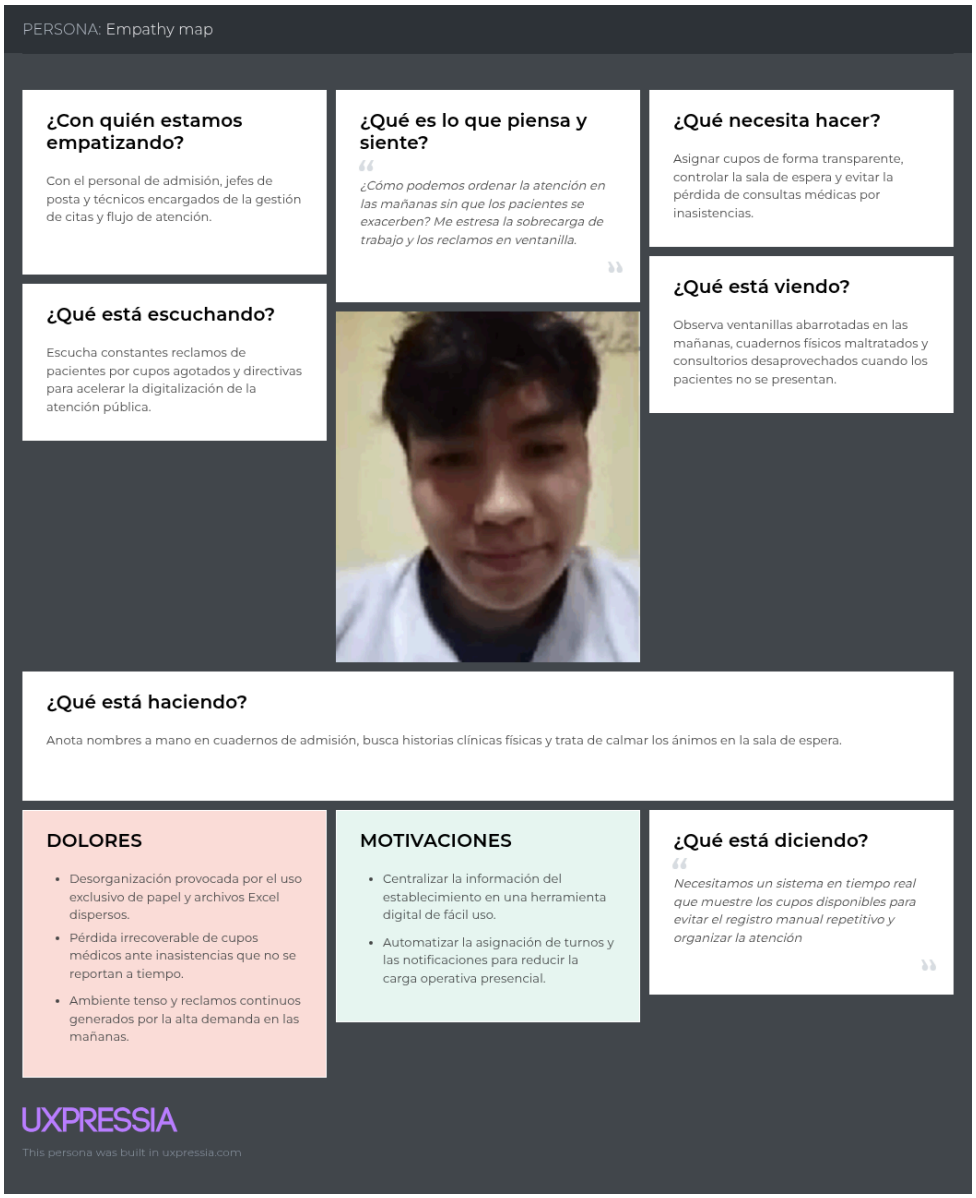
Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

Este mapa de empatía refleja la perspectiva de Kevin como paciente: escucha constantemente que «ya no hay citas para hoy» y comentarios de otros pacientes sobre lo difícil que es conseguir cupo, observa colas largas desde temprano y líneas telefónicas que nunca contestan, y piensa que debería existir una forma de saber la disponibilidad sin tener que ir físicamente y perder un día de trabajo. Su necesidad principal es reservar su cita desde el celular y recibir la confirmación sin depender de procesos presenciales, como se observa en la Figura 29.

Segmento 2: Personal asistencial y administrativo de establecimientos públicos de salud

Figura 30

Empathy Map - Personal Asistencial y Administrativo



Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

Este mapa de empatía muestra que el personal escucha reclamos por cupos agotados y presiona por acelerar la digitalización, mientras observa ventanillas saturadas y consultorios desaprovechados por inasistencias. Su dolor principal es la desorganización por el uso exclusivo de papel y Excel, y su motivación es centralizar la información en una herramienta digital que automatice la asignación de turnos, como se observa en la Figura 30.

2.3.5. Big Picture EventStorming

Para armar un sistema que funcione bien, primero hay que entender cómo trabaja el centro de salud en el día a día. El **Big Picture EventStorming** es una dinámica en equipo que nos sirve para ver todos los hechos importantes que pasan en la posta médica. Al organizar estos sucesos paso a paso, podemos descubrir en qué momentos el proceso se vuelve lento, dónde se pierden los datos o dónde se forman las largas filas de pacientes.

Step 1 – Free Exploration

En este primer paso, nos juntamos a hacer una lluvia de ideas para anotar absolutamente todo lo que sucede en el proceso de atención, sin importar el orden todavía. La idea fue soltar todos los eventos reales del negocio en tiempo pasado (por ejemplo: *Cita reservada* o *Check-in realizado*), sin preocuparnos por la parte técnica ni por el diseño del software, como se observa en la Figura 31.

Figura 31

Big Picture EventStorming - Step 1 Free Exploration

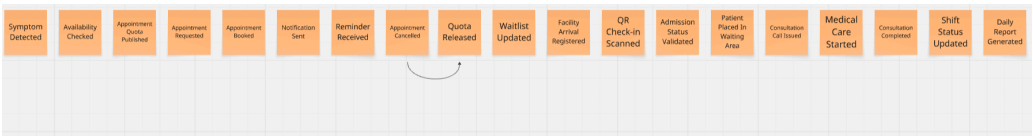
Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

Step 2 – Enforcing Timelines

En el segundo paso, organizamos todas las tarjetas naranjas de izquierda a derecha siguiendo la línea de tiempo real. Esto nos permitió establecer la secuencia cronológica del servicio, desde que el paciente detecta un síntoma en casa hasta que concluye la consulta y se cierra la jornada en el establecimiento de salud, como se observa en la Figura 32.

Figura 32

Big Picture EventStorming: organización de la línea temporal



Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

Step 3 – People and Systems (Actors & Read Models)

En este tercer paso, agrupamos el proceso por flujos de trabajo (*Workflows*) e identificamos a los **Actores** (tarjetas amarillas) que desencadenan cada acción (*Patient* y *Admission Staff*). También mapeamos la información visible o **Read Models** (tarjetas verdes) que necesitan consultar en pantalla para tomar decisiones, como la disponibilidad de cupos y el estado de la lista de espera, como se observa en la Figura 33.

Figura 33

Step 3 - People and Systems

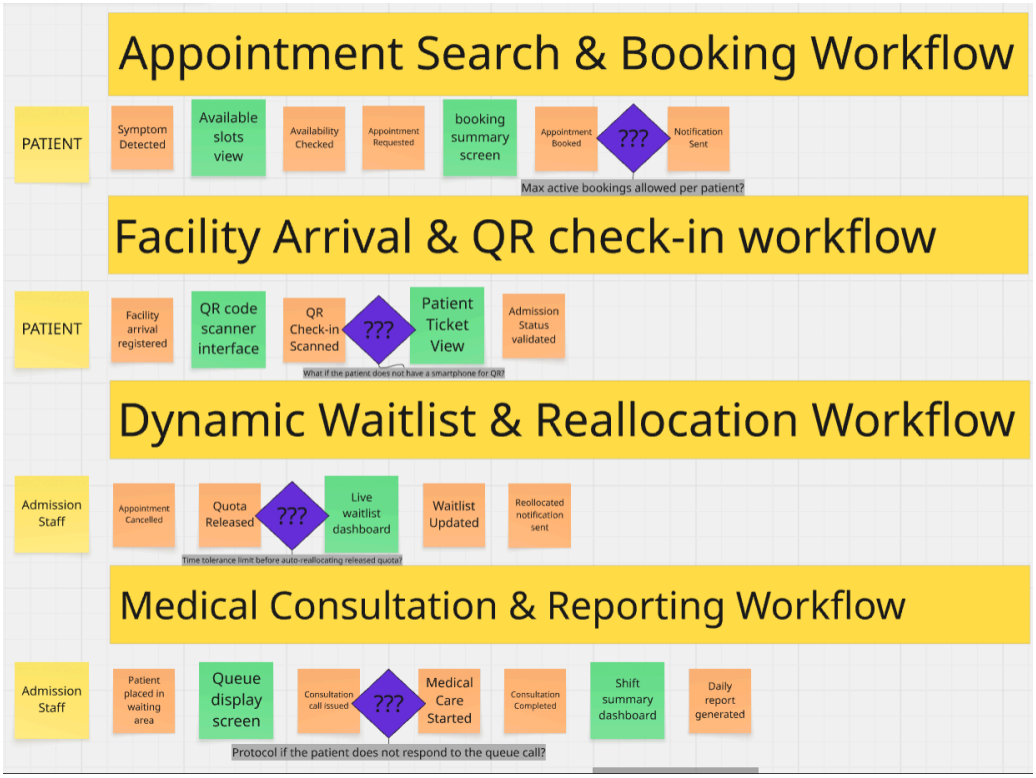
Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

Step 4 – Explicit Hotspots & Exceptions

En el paso final, identificamos los **Hotspots** (puntos críticos o dudas del negocio representados con tarjetas/rombos morados `????`). Esto nos ayudó a anticipar problemas y reglas no definidas, tales como el tiempo límite de tolerancia para el check-in QR, la gestión de pacientes sin teléfono inteligente o la confirmación de cupos liberados en la lista de espera dinámica, como se observa en la Figura 34.

Figura 34

Step 4 - Hotspots and Exceptions



Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

2.3.6. Ubiquitous Language

Para garantizar una comunicación fluida y sin ambigüedades entre el equipo de desarrollo, los diseñadores de UX y los actores del dominio (pacientes y personal de salud), se formalizó el **Lenguaje Ubicuo**. Este glosario unifica los términos clave del negocio que se reflejan tanto en las interfaces de usuario como en los modelos de código del sistema, como se detalla en la Tabla 19.

Tabla 19

Ubiquitous Language

Ubiquitous Language (Inglés / Código)	Definición en el Dominio de SaludYa
`Patient`	Usuario final que solicita, reserva o gestiona atenciones médicas para sí mismo o para sus familiares dependientes en un centro de salud público.
`Admission Staff`	Usuario operativo encargado de aperturar agendas, gestionar la atención en ventanilla y monitorear el flujo de la sala de espera.
`Time Slot`	Intervalo de tiempo asignado a una especialidad médica para la atención de un único paciente en una fecha y horario determinado.
`Booking` / `Appointment`	Proceso mediante el cual un paciente asegura un cupo médico a través de la aplicación móvil antes de acudir presencialmente.

Ubiquitous Language (Inglés / Código)	Definición en el Dominio de SaludYa
`Check-in`	Validación de asistencia presencial realizada por el paciente mediante el escaneo de un código QR al llegar al centro de salud.
`Reassignment`	Mecanismo automatizado que reasigna los cupos liberados por inasistencias a los pacientes de la cola de reserva, en orden de `bookingOrder`, formando una cadena de reasignación.
`Virtual Waiting Room` / `Queue Display`	Vista en tiempo real dentro de la app que informa al paciente su posición exacta en la cola y el tiempo aproximado para su llamado.
`Specialty Catalog` / `Quota Available`	Catálogo estructurado de servicios médicos y horarios configurados y publicados por el centro público de salud.

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

2.4. Requirements specification

En esta sección definimos la especificación formal de requisitos para la plataforma **SaludYa**, tomando como base los hallazgos del *Needfinding* y los flujos identificados en el *EventStorming*. A través de mapeos de escenarios futuros, Historias de Usuario (*User Stories*), *Impact Mapping* y un *Product Backlog* priorizado, transformamos las necesidades del dominio público de salud en entregables técnicos y funcionales ejecutables.

To-Be Scenario Mapping

El escenario propuesto relaciona las acciones del paciente con sus pensamientos y emociones durante el registro, la reserva, la reasignación y la atención médica, como se detalla en la Tabla 20.

Tabla 20

To-Be Scenario Mapping

Fase	Haciendo (Acción)	Pensando (Pensamiento)	Sintiendo (Emoción)
1. Registro y Autenticación	Inicia sesión o se registra ingresando su DNI para validación automática de identidad y vincula a los menores a su cargo.	«Qué rápido es validar mi identidad con el DNI sin hacer trámites presenciales ni llenar formularios largos.»	Tranquilidad y confianza
2. Búsqueda y Reserva de Citas	Selecciona la especialidad médica, explora el calendario de disponibilidad por bloques e ingresa la reserva propia o de su menor.	«Puedo ver todos los turnos disponibles en tiempo real y elegir la franja horaria que mejor me convenga.»	Comodidad y control

Fase	Haciendo (Acción)	Pensando (Pensamiento)	Sintiendo (Emoción)
3. Reasignación de Citas	Recibe una notificación de propuesta para adelantar su cita por un cupo liberado; acepta o rechaza la reasignación en su teléfono.	«Excelente que el sistema me avise para atenderme más temprano si alguien canceló su turno.»	Sorpresa y satisfacción
4. Check-in Presencial por QR	Llega al hospital dentro del margen de tolerancia, escanea el código QR y obtiene su ticket digital con el consultorio asignado.	«Evité la cola de admisión; solo escaneo el QR, confirmo mi presencia y voy directo a la sala.»	Agilidad y alivio
5. Atención y Control Operativo	Espera el llamado al consultorio según su ticket digital mientras el sistema audita tiempos de tolerancia y confirma la atención.	«El proceso es transparente, sé exactamente a dónde ir y se respetan los horarios de atención.»	Seguridad y complacencia

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

2.4.1. User Stories

Epics

Las épicas agrupan las historias de usuario según las capacidades de SaludYa y permiten delimitar los objetivos de cada área funcional, como se detalla en la Tabla 21.

Tabla 21

Epics

ID	Título de la Épica	Descripción Breve
EP0	Plataforma de Presentación y Captación	Sitio web estático (Landing Page) orientado a la captación de pacientes potenciales, la explicación de la propuesta de valor y la redirección de descarga de las aplicaciones móviles.
EP1	Authentication & Identity Management	Registro y autenticación de usuarios con verificación oficial por DNI, vinculación de menores de edad y recuperación de contraseñas.
EP2	Appointments & Booking Engine	Consulta de disponibilidad en calendario, reserva de citas médicas para titulares o menores, y gestión de cancelaciones.

ID	Título de la Épica	Descripción Breve
EP3	Reassignment Protocol	Gestión automatizada de la reasignación de cupos liberados por ausencias, ofreciendo propuestas de adelanto a los pacientes de la cola de reserva en orden de `bookingOrder`.
EP4	Arrival & QR Check-in System	Confirmación presencial de llegada mediante escaneo de código QR, gestión de la cola de asistencia y emisión del ticket digital de atención.
EP5	Hospital Operations & System Configuration	Control operativo de ausencias por vencimiento de tiempo, parametrización de reglas globales e intervalos del hospital, y visualización del dashboard operativo.

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

User stories

US-00: Visualización de Propuesta de Valor en Landing Page

Story ID	User	Priority	Epic
US-00	Paciente potencial	High	EP0: Plataforma de Presentación y Captación
Title	Visualización de Propuesta de Valor en Landing Page		
Description			
Como Paciente potencial, **Quiero** visualizar la propuesta de valor y los beneficios de SaludYa en el sitio web público, **Para** comprender cómo la plataforma optimiza la gestión de mis citas médicas y decidir descargar la aplicación móvil.			
Acceptance Criteria			
Scenario 1: Visualización de la propuesta de valor y enlaces de descarga • **Given** que un paciente potencial accede a la dirección web pública de la plataforma, **When** navega por la página principal, **Then** el sistema presenta de forma estructurada los beneficios de SaludYa y habilita los enlaces de descarga hacia las tiendas de aplicaciones móviles. **Scenario 2: Reproducción de material audiovisual institucional** • **Given** que el paciente potencial explora la página principal, **When** accede a las secciones de presentación audiovisual, **Then** el sistema reproduce los videos explicativos del producto (About-the-Product) y del equipo creador (About-the-Team).			

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

US-01: Verificación de Identidad por DNI

Story ID	User	Priority	Epic
US-01	Paciente	High	EP1: Authentication & Identity Management
Title	Verificación de Identidad por DNI		
Description			
Como Paciente, **Quiero** verificar mi identidad ingresando mi DNI, nombres, apellidos y fecha de nacimiento, **Para** que el sistema confirme que mis datos coinciden con mi identidad oficial antes de crear mi cuenta.			
Acceptance Criteria			
Scenario 1: Validación exitosa de identidad • **Given** que el paciente aún sin cuenta ingresa un DNI de 8 dígitos y nombres, apellidos y fecha de nacimiento, **When** el sistema consulta el registro oficial y los datos coinciden, **Then** el sistema marca la identidad como verificada y habilita la continuación del registro. **Scenario 2: Incoincidencia de datos de identidad** • **Given** que el paciente ingresa un DNI inexistente o datos que no coinciden con el registro oficial, **When** el sistema consulta el registro oficial, **Then** el sistema rechaza la verificación, indica que los datos no coinciden con el titular del DNI y permite corregirlos. **Scenario 3: Formato de DNI inválido** • **Given** que el paciente ingresa un número de documento que no tiene 8 dígitos numéricos, **When** solicita la verificación, **Then** el sistema la rechaza sin consultar el registro oficial e indica el formato esperado. **Scenario 4: Servicio de validación de identidad no disponible** • **Given** que el servicio externo de validación de identidad no responde, **When** el paciente solicita la verificación, **Then** el sistema informa que no pudo verificar la identidad, no habilita la continuación y permite reintentar más tarde.			

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

US-02: Registro de Credenciales de Acceso

Story ID	User	Priority	Epic
US-02	Paciente	High	EP1: Authentication & Identity Management
Title	Registro de Credenciales de Acceso		
Description			
Como Paciente, **Quiero** registrar mi correo, contraseña y número de celular una vez verificada mi identidad, **Para** contar con las credenciales con las que accederé a mi cuenta.			
Acceptance Criteria			
Scenario 1: Credenciales válidas • **Given** que el paciente con identidad verificada ingresa un correo disponible, una contraseña con su confirmación idéntica y un celular de 9 dígitos, **When** solicita continuar con el registro, **Then** el sistema envía un código de verificación de 6 dígitos por SMS al celular indicado. **Scenario 2: Correo ya registrado** • **Given** que el correo ingresado pertenece a otra cuenta, **When** el paciente solicita continuar, **Then** el sistema rechaza el registro e indica que el correo ya está en uso. **Scenario 3: Contraseñas que no coinciden** • **Given** que la contraseña y su confirmación son diferentes, **When** el paciente solicita continuar, **Then** el sistema rechaza el registro e indica que las contraseñas no coinciden. **Scenario 4: Formato de correo o celular inválido** • **Given** que el correo o el celular tiene un formato inválido, **When** el paciente solicita continuar, **Then** el sistema rechaza el registro e identifica el dato inválido.			

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

US-03: Verificación de Celular y Creación de Cuenta

Story ID	User	Priority	Epic
US-03	Paciente	High	EP1: Authentication & Identity Management
Title	Verificación de Celular y Creación de Cuenta		
Description			
Como Paciente, **Quiero** ingresar el código de 6 dígitos enviado a mi celular, **Para** confirmar mi número y obtener mi cuenta de paciente verificada.			
Acceptance Criteria			
Scenario 1: Código válido • **Given** que el paciente recibió un código de verificación vigente, **When** ingresa el código correcto, **Then** el sistema confirma el celular, crea la cuenta de paciente verificada y envía un correo de bienvenida. **Scenario 2: Código incorrecto** • **Given** que el paciente ingresa un código distinto al enviado, **When** solicita completar el registro, **Then** el sistema no crea la cuenta, indica que el código es incorrecto y permite reintentar. **Scenario 3: Reenvío de código** • **Given** que el paciente no recibió el código, **When** solicita un nuevo código, **Then** el sistema envía un código nuevo al mismo celular e invalida el anterior.			

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

US-04: Inicio de Sesión del Paciente

Story ID	User	Priority	Epic
US-04	Paciente	High	EP1: Authentication & Identity Management
Title	Inicio de Sesión del Paciente		
Description			
Como Paciente, **Quiero** iniciar sesión con mi correo y contraseña, **Para** acceder a mi cuenta de paciente.			
Acceptance Criteria			
Scenario 1: Inicio de sesión exitoso • **Given** que el paciente ingresa credenciales válidas de una cuenta activa, **When** envía sus credenciales, **Then** el sistema otorga el acceso y abre su sesión. **Scenario 2: Credenciales inválidas** • **Given** que el paciente ingresa un correo no registrado o una contraseña incorrecta, **When** envía sus credenciales, **Then** el sistema deniega el acceso e informa credenciales inválidas sin precisar cuál dato falló. **Scenario 3: Cuenta inactiva** • **Given** que el paciente ingresa credenciales válidas de una cuenta inactiva, **When** envía sus credenciales, **Then** el sistema deniega el acceso e indica consultar con el personal de admisión.			

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

US-05: Selección de Especialidad Médica

Story ID	User	Priority	Epic
US-05	Paciente	High	EP2: Appointments & Booking Engine
Title	Selección de Especialidad Médica		
Description			
Como Paciente, **Quiero** buscar y seleccionar una especialidad médica, **Para** consultar su disponibilidad de atención.			
Acceptance Criteria			
Scenario 1: Selección de especialidad • **Given** que el catálogo contiene especialidades publicadas, **When** el paciente selecciona una especialidad, **Then** el sistema habilita la consulta de disponibilidad de esa especialidad. **Scenario 2: Búsqueda sin coincidencias** • **Given** que el término buscado no coincide con ninguna especialidad publicada, **When** el paciente realiza la búsqueda, **Then** el sistema indica que no encontró la especialidad y permite volver al catálogo completo. **Scenario 3: Consulta sin especialidad seleccionada** • **Given** que el paciente no ha seleccionado una especialidad, **When** intenta consultar fechas, **Then** el sistema impide la consulta e indica que debe seleccionar una especialidad.			

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

US-06: Consulta de Horarios Disponibles

Story ID	User	Priority	Epic
US-06	Paciente	High	EP2: Appointments & Booking Engine
Title	Consulta de Horarios Disponibles		
Description			
Como Paciente, **Quiero** consultar los horarios libres de una fecha para la especialidad elegida, **Para** elegir el turno que mejor se ajuste a mi tiempo.			
Acceptance Criteria			
Scenario 1: Horarios libres de la fecha • **Given** que el paciente seleccionó una especialidad con agenda activa, **When** consulta una fecha, **Then** el sistema presenta los bloques horarios con cupos libres de esa fecha. **Scenario 2: Fecha sin turnos o con cupos agotados** • **Given** que la fecha no tiene bloques programados o sus cupos están agotados, **When** el paciente la consulta, **Then** el sistema indica que no hay turnos y resalta las fechas más próximas con horarios libres.			

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

US-07: Reserva de Cita para el Paciente

Story ID	User	Priority	Epic
US-07	Paciente	High	EP2: Appointments & Booking Engine
Title	Reserva de Cita para el Paciente		
Description			
Como Paciente, **Quiero** reservar un turno disponible para mí, **Para** asegurar mi atención médica en la fecha y hora elegidas.			
Acceptance Criteria			
Scenario 1: Reserva exitosa • **Given** que el paciente autenticado selecciona un horario con cupo libre, **When** confirma la reserva, **Then** el sistema registra la cita en estado reservado, descuenta el cupo y asigna un código de reserva único. **Scenario 2: Horario ocupado por otro paciente** • **Given** que el cupo seleccionado fue ocupado por otro paciente antes de confirmar, **When** el paciente confirma la reserva, **Then** el sistema rechaza la reserva, informa que el horario ya no está disponible y permite elegir otro. **Scenario 3: Solapamiento de horario** • **Given** que el paciente ya tiene una cita activa en el mismo bloque horario, **When** intenta confirmar una nueva reserva, **Then** el sistema rechaza la reserva e informa el cruce de horarios. **Scenario 4: Reserva fuera de la hora límite** • **Given** que se superó la hora límite para reservar configurada por el establecimiento, **When** el paciente intenta confirmar la reserva, **Then** el sistema rechaza la reserva e informa la hora límite vigente.			

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

US-08: Recepción de Confirmación de Cita por Correo

Story ID	User	Priority	Epic
US-08	Paciente	Medium	EP2: Appointments & Booking Engine
Title	Recepción de Confirmación de Cita por Correo		
Description			
Como Paciente, **Quiero** recibir por correo la confirmación de mi cita, **Para** contar con el comprobante y los detalles de mi atención.			
Acceptance Criteria			
Scenario 1: Envío de confirmación • **Given** que una cita fue reservada exitosamente, **When** concluye el proceso de reserva, **Then** el sistema envía al correo del paciente el código de reserva, el paciente beneficiario, la especialidad, el médico, la fecha y la hora. **Scenario 2: Servicio de correo no disponible** • **Given** que el servicio externo de correo no responde, **When** concluye el proceso de reserva, **Then** el sistema conserva la cita confirmada y reintenta el envío automáticamente.			

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

US-09: Consulta de Citas Agendadas

Story ID	User	Priority	Epic
US-09	Paciente	High	EP2: Appointments & Booking Engine
Title	Consulta de Citas Agendadas		
Description			
Como Paciente, **Quiero** consultar mis citas programadas, finalizadas y canceladas, **Para** conocer el estado y el detalle de mi atención.			
Acceptance Criteria			
Scenario 1: Listado de citas • **Given** que el paciente autenticado tiene citas registradas, **When** consulta sus citas, **Then** el sistema retorna las citas programadas, finalizadas y canceladas con su estado. **Scenario 2: Sin citas registradas** • **Given** que el paciente no tiene citas registradas, **When** consulta sus citas, **Then** el sistema indica que no hay citas registradas. **Scenario 3: Error en la consulta** • **Given** que la información no puede cargarse por una falla del servicio, **When** el paciente consulta sus citas, **Then** el sistema informa el error y permite reintentar la consulta.			

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

US-10: Cancelación de Cita

Story ID	User	Priority	Epic
US-10	Paciente	High	EP2: Appointments & Booking Engine
Title	Cancelación de Cita		
Description			
Como Paciente, **Quiero** cancelar una cita agendada, **Para** liberar el cupo cuando no pueda asistir.			
Acceptance Criteria			
Scenario 1: Cancelación dentro del plazo • **Given** que la cita está activa y cumple la anticipación mínima de cancelación configurada, **When** el paciente confirma la cancelación, **Then** el sistema cambia el estado de la cita a cancelada por el paciente, libera el cupo y envía el comprobante de cancelación al correo registrado. **Scenario 2: Cancelación fuera del plazo** • **Given** que la anticipación mínima de cancelación ya venció, **When** el paciente intenta cancelar, **Then** el sistema rechaza la cancelación e indica gestionarla de forma presencial con el personal de admisión. **Scenario 3: Cita no activa** • **Given** que la cita ya fue cancelada, atendida o declarada ausente, **When** el paciente intenta cancelarla, **Then** el sistema rechaza la solicitud e informa el estado actual de la cita.			

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

US-11: Consulta del Código QR de la Reserva

Story ID	User	Priority	Epic
US-11	Paciente	High	EP4: Arrival & QR Check-in System
Title	Consulta del Código QR de la Reserva		
Description			
Como Paciente, **Quiero** consultar el código QR de mi reserva, **Para** presentarlo al personal de admisión al llegar al establecimiento.			
Acceptance Criteria			
Scenario 1: Código QR disponible • **Given** que el paciente tiene una cita activa, **When** consulta su reserva, **Then** el sistema proporciona el código QR asociado a la reserva y su código de reserva. **Scenario 2: Cita no activa** • **Given** que la cita fue cancelada o ya finalizó, **When** el paciente consulta su reserva, **Then** el sistema no proporciona el código QR e informa el estado de la cita.			

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

US-12: Consulta del Ticket Digital de Atención

Story ID	User	Priority	Epic
US-12	Paciente	Medium	EP4: Arrival & QR Check-in System
Title	Consulta del Ticket Digital de Atención		
Description			
Como Paciente, **Quiero** consultar mi ticket digital de atención, **Para** conocer mi código de turno y el consultorio donde seré atendido.			
Acceptance Criteria			
Scenario 1: Ticket disponible • **Given** que la llegada del paciente fue registrada, **When** consulta su ticket, **Then** el sistema muestra el código de turno, la especialidad, el profesional, la sala y el consultorio. **Scenario 2: Llegada no registrada** • **Given** que el paciente no tiene llegada registrada, **When** consulta su ticket, **Then** el sistema indica que se requiere registrar la llegada presencial. **Scenario 3: Turno finalizado** • **Given** que el turno ya fue atendido o declarado ausente, **When** el paciente consulta su ticket, **Then** el sistema muestra el estado final del turno.			

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

US-13: Consulta de Posición en la Cola de Atención

Story ID	User	Priority	Epic
US-13	Paciente	High	EP4: Arrival & QR Check-in System
Title	Consulta de Posición en la Cola de Atención		
Description			
Como Paciente, **Quiero** consultar mi posición en la cola de atención, **Para** estimar el momento en que seré llamado.			
Acceptance Criteria			
Scenario 1: Posición en la cola • **Given** que el paciente tiene un turno activo en la cola y la posición de cola es visible según la regla del establecimiento, **When** consulta su posición, **Then** el sistema muestra su posición y el total de pacientes, ordenados por hora de llegada. **Scenario 2: Posición de cola no visible** • **Given** que el establecimiento configuró la posición de cola como no visible, **When** el paciente consulta su posición, **Then** el sistema no revela la posición e informa únicamente el estado del turno. **Scenario 3: Sin llegada registrada** • **Given** que el paciente no tiene llegada registrada, **When** consulta su posición, **Then** el sistema indica que no registra presencia en la cola. **Scenario 4: Turno finalizado** • **Given** que el turno ya fue atendido o declarado ausente, **When** el paciente consulta su posición, **Then** el sistema indica que ya no se encuentra en la cola.			

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

US-14: Recepción de Notificación de Llamado a Consultorio

Story ID	User	Priority	Epic
US-14	Paciente	High	EP4: Arrival & QR Check-in System
Title	Recepción de Notificación de Llamado a Consultorio		
Description			
Como Paciente, **Quiero** recibir una notificación cuando me llamen a consultorio, **Para** dirigirme a atención sin depender de un aviso manual.			
Acceptance Criteria			
Scenario 1: Notificación de llamado • **Given** que el personal de admisión registra el llamado del paciente, **When** se registra el llamado, **Then** el sistema envía al paciente una notificación con el consultorio asignado y el plazo para acudir. **Scenario 2: Canal de notificación no disponible** • **Given** que el canal de notificación no responde, **When** se registra el llamado, **Then** el sistema registra el intento y reintenta el envío sin detener la cola.			

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

US-15: Vinculación de un Menor de Edad

Story ID	User	Priority	Epic
US-15	Paciente	Medium	EP1: Authentication & Identity Management
Title	Vinculación de un Menor de Edad		
Description			
Como Paciente, **Quiero** vincular a un menor a mi cuenta ingresando su DNI, nombres, apellidos y fecha de nacimiento, **Para** gestionar sus citas médicas desde mi propia cuenta.			
Acceptance Criteria			
Scenario 1: Vinculación exitosa • **Given** que el paciente titular autenticado ingresa datos de un menor de 18 años que coinciden con el registro oficial, **When** confirma ser su adulto responsable, **Then** el sistema asocia al menor con la cuenta del titular. **Scenario 2: Menor ya vinculado a otra cuenta** • **Given** que el DNI del menor ya está asociado a otra cuenta, **When** el titular intenta vincularlo, **Then** el sistema rechaza la vinculación e indica validar la tutela legal con el personal de admisión. **Scenario 3: Edad no válida** • **Given** que el DNI corresponde a una persona de 18 años o más, **When** el titular intenta vincularla, **Then** el sistema rechaza la vinculación e indica que la edad no es válida. **Scenario 4: Incoincidencia de datos del menor** • **Given** que los datos ingresados no coinciden con el registro oficial, **When** el titular intenta vincular al menor, **Then** el sistema rechaza la vinculación e indica que la identidad no coincide.			

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

US-16: Reserva de Cita para un Menor Vinculado

Story ID	User	Priority	Epic
US-16	Paciente	Medium	EP2: Appointments & Booking Engine
Title	Reserva de Cita para un Menor Vinculado		
Description			
Como Paciente, **Quiero** reservar un turno disponible para un menor vinculado a mi cuenta, **Para** asegurar la atención médica del menor.			
Acceptance Criteria			
Scenario 1: Reserva exitosa para el menor • **Given** que el paciente titular selecciona un horario con cupo libre y un menor vinculado, **When** confirma la reserva, **Then** el sistema asigna la cita al menor, registra al titular como adulto responsable y asigna un código de reserva único. **Scenario 2: Solapamiento de horario del menor** • **Given** que el menor ya tiene una cita activa en el mismo bloque horario, **When** el titular intenta confirmar la reserva, **Then** el sistema rechaza la reserva e informa el cruce de horarios. **Scenario 3: Menor no vinculado** • **Given** que el menor seleccionado no está vinculado a la cuenta del titular, **When** el titular intenta reservar, **Then** el sistema rechaza la reserva e indica que el menor no está vinculado.			

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

US-17: Consulta de Menores Vinculados

Story ID	User	Priority	Epic
US-17	Paciente	Medium	EP1: Authentication & Identity Management
Title	Consulta de Menores Vinculados		
Description			
Como Paciente, **Quiero** consultar la lista de menores vinculados a mi cuenta, **Para** identificar a quiénes puedo gestionar citas.			
Acceptance Criteria			
Scenario 1: Lista de menores • **Given** que el paciente titular tiene menores vinculados, **When** consulta sus menores vinculados, **Then** el sistema retorna el nombre, la edad y el DNI enmascarado de cada menor. **Scenario 2: Sin menores vinculados** • **Given** que el paciente titular no tiene menores vinculados, **When** consulta sus menores vinculados, **Then** el sistema indica que no hay menores vinculados.			

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

US-18: Consulta del Perfil de un Menor

Story ID	User	Priority	Epic
US-18	Paciente	Medium	EP1: Authentication & Identity Management
Title	Consulta del Perfil de un Menor		
Description			
Como Paciente, **Quiero** consultar el perfil de un menor vinculado, **Para** revisar sus datos y mi condición de adulto responsable.			
Acceptance Criteria			
Scenario 1: Perfil del menor • **Given** que el menor está vinculado a la cuenta del titular, **When** el titular consulta su perfil, **Then** el sistema muestra el nombre, el DNI enmascarado, la fecha de nacimiento y el adulto responsable. **Scenario 2: Menor no vinculado a la cuenta** • **Given** que el menor no está vinculado a la cuenta del paciente, **When** el paciente intenta consultar su perfil, **Then** el sistema deniega el acceso a los datos del menor.			

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

US-19: Desvinculación de un Menor de Edad

Story ID	User	Priority	Epic
US-19	Paciente	Medium	EP1: Authentication & Identity Management
Title	Desvinculación de un Menor de Edad		
Description			
Como Paciente, **Quiero** desvincular a un menor de mi cuenta, **Para** que deje de estar disponible para agendar citas desde mi cuenta.			
Acceptance Criteria			
Scenario 1: Desvinculación confirmada • **Given** que el paciente titular tiene un menor vinculado y confirma la desvinculación, **When** el sistema procesa la solicitud, **Then** el sistema elimina el vínculo y el menor deja de estar disponible para agendar citas. **Scenario 2: Desvinculación no confirmada** • **Given** que el titular solicitó la desvinculación y esta requiere confirmación, **When** el titular no confirma, **Then** el sistema conserva el vínculo sin realizar cambios.			

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

US-20: Aceptación de Adelanto de Cita

Story ID	User	Priority	Epic
US-20	Paciente	High	EP3: Reassignment Protocol
Title	Aceptación de Adelanto de Cita		
Description			
Como Paciente, **Quiero** aceptar la propuesta de adelantar mi cita a un intervalo liberado, **Para** ser atendido más temprano.			
Acceptance Criteria			
Scenario 1: Adelanto aceptado • **Given** que el paciente recibió una propuesta de adelanto vigente, **When** acepta el cambio, **Then** el sistema le asigna el intervalo anterior, libera su horario original y confirma la reasignación. **Scenario 2: Intervalo ya asignado a otro paciente** • **Given** que otro paciente aceptó antes la misma propuesta, **When** el paciente intenta aceptar, **Then** el sistema informa que el horario ya no está disponible y mantiene su cita sin cambios.			

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

US-21: Rechazo de Adelanto de Cita

Story ID	User	Priority	Epic
US-21	Paciente	Medium	EP3: Reassignment Protocol
Title	Rechazo de Adelanto de Cita		
Description			
Como Paciente, **Quiero** rechazar la propuesta de adelantar mi cita, **Para** mantener mi turno programado originalmente.			
Acceptance Criteria			
Scenario 1: Adelanto rechazado • **Given** que el paciente recibió una propuesta de adelanto vigente, **When** rechaza el cambio, **Then** el sistema mantiene su cita en el intervalo original y ofrece el cupo al siguiente paciente según el orden de reserva. **Scenario 2: Propuesta sin respuesta** • **Given** que venció el plazo de respuesta a la reasignación configurado, **When** el paciente no responde, **Then** el sistema asume el rechazo, conserva su cita y ofrece el cupo al siguiente paciente según el orden de reserva.			

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

US-22: Salida Voluntaria de la Cola de Atención

Story ID	User	Priority	Epic
US-22	Paciente	Medium	EP4: Arrival & QR Check-in System
Title	Salida Voluntaria de la Cola de Atención		
Description			
Como Paciente, **Quiero** dejar la cola de atención confirmando mi decisión, **Para** no ocupar un lugar cuando ya no puedo esperar.			
Acceptance Criteria			
Scenario 1: Salida confirmada • **Given** que el paciente tiene un turno activo en la cola y confirma su salida, **When** el sistema procesa la solicitud, **Then** el sistema lo retira de la cola, recalcula la posición de los demás pacientes y conserva la reserva sin cancelarla. **Scenario 2: Salida no confirmada** • **Given** que el paciente solicitó salir y esto requiere confirmación, **When** no confirma, **Then** el sistema lo mantiene en la cola con su posición actual. **Scenario 3: Sin turno activo en la cola** • **Given** que el paciente no tiene un turno activo en la cola, **When** intenta salir de la cola, **Then** el sistema indica que no registra un turno activo.			

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

US-23: Solicitud de Enlace de Recuperación de Contraseña

Story ID	User	Priority	Epic
US-23	Paciente	High	EP1: Authentication & Identity Management
Title	Solicitud de Enlace de Recuperación de Contraseña		
Description			
Como Paciente, **Quiero** solicitar un enlace de recuperación a mi correo registrado, **Para** poder definir una nueva contraseña de forma segura.			
Acceptance Criteria			
Scenario 1: Envío del enlace • **Given** que el correo ingresado pertenece a una cuenta registrada, **When** el paciente solicita la recuperación, **Then** el sistema envía un enlace temporal con vigencia de 15 minutos. **Scenario 2: Correo no registrado** • **Given** que el correo ingresado no pertenece a ninguna cuenta, **When** el paciente solicita la recuperación, **Then** el sistema no genera enlace y responde con un mensaje genérico sin revelar la existencia de la cuenta. **Scenario 3: Solicitud de nuevo enlace** • **Given** que el paciente ya solicitó un enlace previamente, **When** solicita uno nuevo, **Then** el sistema genera un enlace nuevo e invalida el anterior.			

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

US-24: Restablecimiento de Contraseña

Story ID	User	Priority	Epic
US-24	Paciente	High	EP1: Authentication & Identity Management
Title	Restablecimiento de Contraseña		
Description			
Como Paciente, **Quiero** definir una nueva contraseña mediante el enlace recibido, **Para** recuperar el acceso a mi cuenta.			
Acceptance Criteria			
Scenario 1: Contraseña actualizada • **Given** que el enlace de recuperación está vigente y el paciente ingresa una contraseña con su confirmación idéntica, **When** solicita guardar la contraseña, **Then** el sistema actualiza la contraseña e indica que ya puede iniciar sesión. **Scenario 2: Contraseñas que no coinciden** • **Given** que la contraseña y su confirmación son diferentes, **When** el paciente solicita guardar, **Then** el sistema rechaza el cambio e indica que las contraseñas no coinciden. **Scenario 3: Enlace expirado** • **Given** que transcurrieron más de 15 minutos desde la emisión del enlace, **When** el paciente intenta definir la contraseña, **Then** el sistema rechaza la operación, indica que el enlace expiró y requiere una nueva solicitud.			

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

US-25: Solicitud de Ayuda por Pérdida de Acceso a la Cuenta

Story ID	User	Priority	Epic
US-25	Paciente	Medium	EP1: Authentication & Identity Management
Title	Solicitud de Ayuda por Pérdida de Acceso a la Cuenta		
Description			
Como Paciente, **Quiero** solicitar ayuda indicando mi DNI y un correo de contacto alternativo, **Para** recuperar mi cuenta cuando no tengo acceso a mi correo ni a mi celular.			
Acceptance Criteria			
Scenario 1: Solicitud registrada • **Given** que el paciente ingresa su DNI y un correo de contacto válido, **When** envía la solicitud, **Then** el sistema registra la solicitud para que un agente restablezca las credenciales previa verificación de identidad. **Scenario 2: Datos incompletos o inválidos** • **Given** que el DNI o el correo de contacto falta o tiene formato inválido, **When** el paciente envía la solicitud, **Then** el sistema rechaza la solicitud e identifica el dato a corregir.			

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

US-26: Consulta del Perfil del Paciente

Story ID	User	Priority	Epic
US-26	Paciente	Medium	EP1: Authentication & Identity Management
Title	Consulta del Perfil del Paciente		
Description			
Como Paciente, **Quiero** consultar mis datos de identidad y de contacto, **Para** verificar la información registrada en mi cuenta.			
Acceptance Criteria			
Scenario 1: Datos del perfil • **Given** que el paciente tiene sesión iniciada, **When** consulta su perfil, **Then** el sistema muestra su nombre, el DNI con solo los últimos 4 dígitos visibles, el estado de identidad verificada, su correo y su celular.			

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

US-27: Edición de Datos de Contacto

Story ID	User	Priority	Epic
US-27	Paciente	Medium	EP1: Authentication & Identity Management
Title	Edición de Datos de Contacto		
Description			
Como Paciente, **Quiero** actualizar mi correo y mi número de celular, **Para** mantener mis datos de contacto al día.			
Acceptance Criteria			
Scenario 1: Actualización exitosa • **Given** que el paciente ingresa un correo disponible y un celular válido, **When** guarda los cambios, **Then** el sistema actualiza sus datos de contacto y envía una notificación de seguridad. **Scenario 2: Correo ya registrado** • **Given** que el correo ingresado pertenece a otra cuenta, **When** el paciente guarda los cambios, **Then** el sistema rechaza la actualización e indica que el correo ya está en uso. **Scenario 3: Formato de contacto inválido** • **Given** que el correo o el celular tiene un formato inválido, **When** el paciente guarda los cambios, **Then** el sistema rechaza la actualización e identifica el dato inválido. **Scenario 4: Intento de modificar datos de identidad** • **Given** que el paciente intenta modificar su DNI, nombres, apellidos o fecha de nacimiento, **When** guarda los cambios, **Then** el sistema rechaza el cambio, pues los datos de identidad no se modifican.			

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

US-28: Cierre de Sesión del Paciente

Story ID	User	Priority	Epic
US-28	Paciente	Medium	EP1: Authentication & Identity Management
Title	Cierre de Sesión del Paciente		
Description			
Como Paciente, **Quiero** cerrar mi sesión, **Para** proteger el acceso a mi cuenta al terminar de usar la aplicación.			
Acceptance Criteria			
Scenario 1: Cierre de sesión confirmado • **Given** que el paciente tiene sesión iniciada y confirma el cierre, **When** el sistema procesa la solicitud, **Then** el sistema finaliza la sesión e invalida el token de acceso. **Scenario 2: Cierre de sesión no confirmado** • **Given** que el paciente solicitó cerrar sesión y esto requiere confirmación, **When** no confirma, **Then** el sistema conserva la sesión activa.			

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

US-29: Aviso de Sesión Expirada

Story ID	User	Priority	Epic
US-29	Paciente	Medium	EP1: Authentication & Identity Management
Title	Aviso de Sesión Expirada		
Description			
Como Paciente, **Quiero** ser informado cuando mi sesión ha expirado, **Para** volver a iniciar sesión de forma segura.			
Acceptance Criteria			
Scenario 1: Sesión expirada • **Given** que el token de acceso del paciente venció, **When** intenta realizar una acción, **Then** el sistema rechaza la acción, informa que la sesión expiró y requiere un nuevo inicio de sesión.			

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

US-30: Inicio de Sesión del Super Admin

Story ID	User	Priority	Epic
US-30	Super Admin	High	EP1: Authentication & Identity Management
Title	Inicio de Sesión del Super Admin		
Description			
Como Super Admin, **Quiero** iniciar sesión con mi correo y contraseña, **Para** acceder a la configuración del sistema.			
Acceptance Criteria			
Scenario 1: Inicio de sesión exitoso • **Given** que el Super Admin ingresa credenciales válidas de una cuenta activa, **When** envía sus credenciales, **Then** el sistema otorga el acceso con el rol de Super Admin. **Scenario 2: Credenciales inválidas** • **Given** que ingresa un correo no registrado, una contraseña incorrecta o un rol que no corresponde, **When** envía sus credenciales, **Then** el sistema deniega el acceso e informa credenciales inválidas. **Scenario 3: Cuenta inactiva** • **Given** que ingresa credenciales válidas de una cuenta inactiva, **When** envía sus credenciales, **Then** el sistema deniega el acceso e informa que la cuenta está inactiva.			

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

US-31: Creación de Cuenta de Personal de Admisión

Story ID	User	Priority	Epic
US-31	Super Admin	High	EP1: Authentication & Identity Management
Title	Creación de Cuenta de Personal de Admisión		
Description			
Como Super Admin, **Quiero** registrar al personal de admisión con su DNI, nombres, apellidos, fecha de nacimiento, correo corporativo y teléfono, **Para** otorgarle una cuenta verificada para operar el establecimiento.			
Acceptance Criteria			
Scenario 1: Alta exitosa • **Given** que el Super Admin autenticado ingresa datos que coinciden con el registro oficial y un correo corporativo disponible, **When** registra al personal, **Then** el sistema crea la cuenta de personal de admisión y envía las credenciales de acceso por correo. **Scenario 2: Incoincidencia de datos de identidad** • **Given** que algún dato ingresado no coincide con el registro oficial, **When** el Super Admin intenta registrar, **Then** el sistema rechaza el alta e indica que la identidad no coincide con el titular. **Scenario 3: Correo duplicado** • **Given** que el correo ingresado ya pertenece a una cuenta activa, **When** el Super Admin intenta registrar, **Then** el sistema rechaza el alta e indica que el correo ya está registrado.			

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

US-32: Inicio de Sesión del Personal de Admisión

Story ID	User	Priority	Epic
US-32	Personal de Admisión	High	EP1: Authentication & Identity Management
Title	Inicio de Sesión del Personal de Admisión		
Description			
Como Personal de Admisión, **Quiero** iniciar sesión con mi correo y contraseña, **Para** acceder a las herramientas de operación del establecimiento.			
Acceptance Criteria			
Scenario 1: Inicio de sesión exitoso • **Given** que el personal de admisión ingresa credenciales válidas de una cuenta activa, **When** envía sus credenciales, **Then** el sistema otorga el acceso con el rol de Personal de Admisión. **Scenario 2: Credenciales inválidas** • **Given** que ingresa un correo no registrado, una contraseña incorrecta o un rol que no corresponde, **When** envía sus credenciales, **Then** el sistema deniega el acceso e informa credenciales inválidas. **Scenario 3: Cuenta inactiva** • **Given** que ingresa credenciales válidas de una cuenta inactiva, **When** envía sus credenciales, **Then** el sistema deniega el acceso e informa que la cuenta está inactiva.			

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

US-33: Configuración de la Duración de los Intervalos de Atención

Story ID	User	Priority	Epic
US-33	Super Admin	High	EP5: Hospital Operations & System Configuration
Title	Configuración de la Duración de los Intervalos de Atención		
Description			
Como Super Admin, **Quiero** definir la duración de cada intervalo de atención, **Para** adaptar la agenda a la operación del establecimiento.			
Acceptance Criteria			
Scenario 1: Intervalo actualizado • **Given** que el Super Admin ingresa una duración válida coherente con la tolerancia de llegada, **When** guarda el intervalo, **Then** el sistema aplica la duración a los nuevos bloques y conserva las citas confirmadas. **Scenario 2: Duración incoherente** • **Given** que la tolerancia de llegada es mayor que la duración ingresada, **When** el Super Admin intenta guardar, **Then** el sistema rechaza el cambio e indica el conflicto.			

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

US-34: Edición de una Regla Operativa del Establecimiento

Story ID	User	Priority	Epic
US-34	Super Admin	High	EP5: Hospital Operations & System Configuration
Title	Edición de una Regla Operativa del Establecimiento		
Description			
Como Super Admin, **Quiero** modificar el valor de una regla operativa (capacidad por bloque, tolerancia de llegada, plazo después de llamada, respuesta a reasignación, hora límite para reservar o anticipación para cancelar), **Para** adaptar el sistema a la capacidad y a las políticas de la institución.			
Acceptance Criteria			
Scenario 1: Regla actualizada • **Given** que el Super Admin ingresa un valor válido para la regla, **When** guarda el cambio, **Then** el sistema persiste el valor, lo aplica a la operación y conserva las reservas confirmadas. **Scenario 2: Valor inválido** • **Given** que el valor ingresado es cero, no numérico o incoherente con la duración del intervalo, **When** el Super Admin intenta guardar, **Then** el sistema rechaza el cambio e indica el campo en conflicto.			

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

US-35: Consulta de Reglas Operativas Vigentes

Story ID	User	Priority	Epic
US-35	Super Admin	Medium	EP5: Hospital Operations & System Configuration
Title	Consulta de Reglas Operativas Vigentes		
Description			
Como Super Admin, **Quiero** consultar las reglas operativas vigentes del establecimiento, **Para** conocer cómo está configurado el sistema.			
Acceptance Criteria			
Scenario 1: Reglas vigentes • **Given** que el Super Admin está autenticado, **When** consulta las reglas del establecimiento, **Then** el sistema retorna el valor vigente de cada regla: capacidad por bloque, tolerancia de llegada, plazo después de llamada, respuesta a reasignación, hora límite para reservar, anticipación para cancelar, prioridad de reasignación y posición de cola visible.			

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

US-36: Selección de Especialidad en la Agenda Operativa

Story ID	User	Priority	Epic
US-36	Personal de Admisión	High	EP2: Appointments & Booking Engine
Title	Selección de Especialidad en la Agenda Operativa		
Description			
Como Personal de Admisión, **Quiero** buscar y seleccionar una especialidad, **Para** consultar su calendario operativo.			
Acceptance Criteria			
Scenario 1: Selección de especialidad • **Given** que el catálogo contiene especialidades publicadas, **When** el personal selecciona una especialidad, **Then** el sistema habilita el calendario operativo de esa especialidad. **Scenario 2: Búsqueda sin coincidencias** • **Given** que el término buscado no coincide con ninguna especialidad, **When** el personal realiza la búsqueda, **Then** el sistema indica que no encontró la especialidad y permite volver al catálogo completo.			

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

US-37: Consulta del Calendario Operativo

Story ID	User	Priority	Epic
US-37	Personal de Admisión	High	EP2: Appointments & Booking Engine
Title	Consulta del Calendario Operativo		
Description			
Como Personal de Admisión, **Quiero** consultar el calendario mensual de una especialidad, **Para** identificar las fechas con bloques programados.			
Acceptance Criteria			
Scenario 1: Fechas con bloques • **Given** que la especialidad seleccionada tiene agenda activa, **When** el personal consulta un mes, **Then** el sistema identifica las fechas que tienen bloques programados. **Scenario 2: Fecha sin bloques** • **Given** que la fecha consultada no tiene bloques programados, **When** el personal la consulta, **Then** el sistema indica que no hay bloques para esa fecha.			

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

US-38: Consulta de Bloques del Día

Story ID	User	Priority	Epic
US-38	Personal de Admisión	High	EP2: Appointments & Booking Engine
Title	Consulta de Bloques del Día		
Description			
Como Personal de Admisión, **Quiero** consultar los bloques horarios de un día, **Para** conocer la ocupación de la agenda.			
Acceptance Criteria			
Scenario 1: Bloques del día • **Given** que existen bloques en la fecha y especialidad seleccionadas, **When** el personal consulta el día, **Then** el sistema retorna cada bloque con su horario, profesional, consultorio, reservas, capacidad y estado.			

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

US-39: Consulta del Detalle de un Bloque

Story ID	User	Priority	Epic
US-39	Personal de Admisión	High	EP2: Appointments & Booking Engine
Title	Consulta del Detalle de un Bloque		
Description			
Como Personal de Admisión, **Quiero** consultar el detalle de un bloque horario, **Para** revisar su capacidad, reservas y estado.			
Acceptance Criteria			
Scenario 1: Detalle del bloque • **Given** que el bloque existe en la agenda, **When** el personal consulta su detalle, **Then** el sistema retorna el horario, la especialidad, el profesional, el consultorio, la capacidad, las reservas y el estado.			

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

US-40: Consulta de Horario por Profesional

Story ID	User	Priority	Epic
US-40	Personal de Admisión	Medium	EP2: Appointments & Booking Engine
Title	Consulta de Horario por Profesional		
Description			
Como Personal de Admisión, **Quiero** consultar los bloques de un profesional con sus cupos disponibles, **Para** seleccionar el bloque que corresponde.			
Acceptance Criteria			
Scenario 1: Bloques del profesional • **Given** que el profesional pertenece a la especialidad seleccionada, **When** el personal consulta su horario, **Then** el sistema retorna sus bloques con los cupos disponibles de cada uno. **Scenario 2: Bloque sin cupos** • **Given** que un bloque del profesional no tiene cupos disponibles, **When** el personal intenta seleccionarlo, **Then** el sistema lo identifica como sin cupos y no permite la selección.			

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

US-41: Consulta de Pacientes con Reserva en un Bloque

Story ID	User	Priority	Epic
US-41	Personal de Admisión	Medium	EP2: Appointments & Booking Engine
Title	Consulta de Pacientes con Reserva en un Bloque		
Description			
Como Personal de Admisión, **Quiero** consultar los pacientes que tienen reserva en un bloque, **Para** saber quiénes deben llegar y en qué estado se encuentran.			
Acceptance Criteria			
Scenario 1: Pacientes con reserva • **Given** que el bloque tiene reservas registradas, **When** el personal consulta la lista, **Then** el sistema retorna cada paciente con su código de reserva, si es titular o menor con su representante, y su estado de llegada o atención.			

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

US-42: Edición de un Bloque Horario

Story ID	User	Priority	Epic
US-42	Personal de Admisión	Medium	EP2: Appointments & Booking Engine
Title	Edición de un Bloque Horario		
Description			
Como Personal de Admisión, **Quiero** modificar la hora de inicio, la especialidad, el profesional o el estado de un bloque, **Para** mantener la agenda alineada con la operación real.			
Acceptance Criteria			
Scenario 1: Bloque actualizado • **Given** que el cambio no entra en conflicto con reservas confirmadas ni con otro bloque del profesional, **When** el personal confirma el cambio, **Then** el sistema guarda los nuevos datos del bloque. **Scenario 2: Cambio incompatible** • **Given** que el cambio entra en conflicto con reservas confirmadas u otro bloque del profesional, **When** el personal confirma el cambio, **Then** el sistema rechaza el cambio e indica que debe revisar el bloque.			

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

US-43: Registro de Llegada por Escaneo de QR

Story ID	User	Priority	Epic
US-43	Personal de Admisión	High	EP4: Arrival & QR Check-in System
Title	Registro de Llegada por Escaneo de QR		
Description			
Como Personal de Admisión, **Quiero** registrar la llegada de un paciente escaneando el código QR de su reserva, **Para** confirmar su presencia y generar su ticket de atención.			
Acceptance Criteria			
Scenario 1: Llegada confirmada • **Given** que el código QR corresponde a una reserva vigente y el paciente llega dentro de la tolerancia configurada, **When** el personal escanea el código, **Then** el sistema registra la llegada, genera el ticket del paciente y lo ingresa en la cola de atención. **Scenario 2: Reserva no vigente o inexistente** • **Given** que el código QR no corresponde a una reserva vigente, **When** el personal escanea el código, **Then** el sistema rechaza el registro y no ingresa al paciente en la cola. **Scenario 3: Llegada fuera de la tolerancia** • **Given** que el paciente llega fuera de la tolerancia configurada, **When** el personal escanea el código, **Then** el sistema rechaza el registro e informa que el horario de la reserva no permite la llegada.			

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

US-44: Registro de Llegada por Código Manual

Story ID	User	Priority	Epic
US-44	Personal de Admisión	Medium	EP4: Arrival & QR Check-in System
Title	Registro de Llegada por Código Manual		
Description			
Como Personal de Admisión, **Quiero** registrar la llegada de un paciente ingresando el código de su reserva, **Para** confirmar su presencia cuando no es posible escanear el QR.			
Acceptance Criteria			

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

US-45: Consulta de la Cola de Atención Presencial

Story ID	User	Priority	Epic
US-45	Personal de Admisión	High	EP4: Arrival & QR Check-in System
Title	Consulta de la Cola de Atención Presencial		
Description			
Como Personal de Admisión, **Quiero** consultar la cola de pacientes presentes de un consultorio, **Para** conocer el orden de atención.			
Acceptance Criteria			
Scenario 1: Cola con pacientes • **Given** que existen pacientes con llegada registrada, **When** el personal consulta la cola, **Then** el sistema retorna los pacientes ordenados por hora de llegada con su número de turno y estado. **Scenario 2: Cola vacía** • **Given** que no hay llegadas registradas, **When** el personal consulta la cola, **Then** el sistema indica que no hay pacientes en espera.			

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

US-46: Llamado del Siguiente Paciente

Story ID	User	Priority	Epic
US-46	Personal de Admisión	High	EP4: Arrival & QR Check-in System
Title	Llamado del Siguiente Paciente		
Description			
Como Personal de Admisión, **Quiero** llamar al siguiente paciente de la cola, **Para** que se dirija al consultorio dentro del plazo establecido.			
Acceptance Criteria			
Scenario 1: Llamado registrado • **Given** que existe un paciente en espera en primer lugar de la cola, **When** el personal lo llama, **Then** el sistema registra la hora del llamado, cambia el estado del paciente a llamado e inicia el plazo para acudir configurado. **Scenario 2: Sin pacientes en espera** • **Given** que no hay pacientes en espera en la cola, **When** el personal intenta llamar al siguiente, **Then** el sistema informa que no hay pacientes por llamar.			

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

US-47: Inicio de Atención de un Paciente Llamado

Story ID	User	Priority	Epic
US-47	Personal de Admisión	High	EP4: Arrival & QR Check-in System
Title	Inicio de Atención de un Paciente Llamado		
Description			
Como Personal de Admisión, **Quiero** iniciar la atención del paciente que acudió al consultorio, **Para** registrar que el paciente está siendo atendido.			
Acceptance Criteria			
Scenario 1: Atención iniciada • **Given** que el paciente llamado se encuentra en el consultorio dentro del plazo, **When** el personal inicia la atención, **Then** el sistema cambia el estado del turno a en atención y detiene el conteo del plazo. **Scenario 2: Atención iniciada con plazo vencido** • **Given** que el plazo venció pero el paciente aún no fue marcado ausente, **When** el personal inicia la atención, **Then** el sistema cambia el estado del turno a en atención. **Scenario 3: Turno ya declarado ausente** • **Given** que el turno fue declarado ausente, **When** el personal intenta iniciar la atención, **Then** el sistema rechaza la operación e informa que el turno se perdió.			

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

US-48: Finalización de la Atención de un Paciente

Story ID	User	Priority	Epic
US-48	Personal de Admisión	High	EP4: Arrival & QR Check-in System
Title	Finalización de la Atención de un Paciente		
Description			
Como Personal de Admisión, **Quiero** finalizar la atención de un paciente, **Para** retirarlo de la cola y continuar con el siguiente.			
Acceptance Criteria			
Scenario 1: Atención finalizada • **Given** que el turno del paciente está en atención, **When** el personal finaliza la atención, **Then** el sistema cambia el estado a atendido, retira al paciente de la cola y habilita el llamado del siguiente.			

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

US-49: Registro de Ausencia de un Paciente

Story ID	User	Priority	Epic
US-49	Personal de Admisión	High	EP5: Hospital Operations & System Configuration
Title	Registro de Ausencia de un Paciente		
Description			
Como Personal de Admisión, **Quiero** marcar como ausente a un paciente que no acudió tras ser llamado, **Para** liberar su cupo según las reglas del establecimiento.			
Acceptance Criteria			
Scenario 1: Ausencia confirmada • **Given** que el plazo posterior al llamado se cumplió sin que el paciente ingresara al consultorio, **When** el personal confirma la ausencia, **Then** el sistema marca el turno como ausente, retira al paciente de la cola y libera el cupo. **Scenario 2: Plazo aún vigente** • **Given** que el plazo posterior al llamado no se cumplió, **When** el personal intenta marcar la ausencia, **Then** el sistema rechaza la operación e informa el tiempo restante. **Scenario 3: Atención ya iniciada** • **Given** que la atención del paciente ya fue iniciada, **When** el personal intenta marcar la ausencia, **Then** el sistema rechaza la operación.			

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

US-50: Consulta de Citas Canceladas del Día

Story ID	User	Priority	Epic
US-50	Personal de Admisión	Medium	EP2: Appointments & Booking Engine
Title	Consulta de Citas Canceladas del Día		
Description			
Como Personal de Admisión, **Quiero** consultar las citas canceladas del día, **Para** conocer qué cupos fueron liberados.			
Acceptance Criteria			
Scenario 1: Cancelaciones del día • **Given** que existen cancelaciones registradas en el día, **When** el personal consulta las citas canceladas, **Then** el sistema retorna la especialidad, el paciente, el horario, el momento de la cancelación y el estado del cupo. **Scenario 2: Sin cancelaciones** • **Given** que no hay cancelaciones registradas en el día, **When** el personal consulta las citas canceladas, **Then** el sistema indica que no hay citas canceladas.			

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

US-51: Visualización de Indicadores del Día

Story ID	User	Priority	Epic
US-51	Personal de Admisión	High	EP5: Hospital Operations & System Configuration
Title	Visualización de Indicadores del Día		
Description			
Como Personal de Admisión, **Quiero** consultar las citas programadas, pendientes, canceladas e inasistencias del día junto con el resumen de atención, **Para** monitorear el flujo de citas e incidencias del establecimiento.			
Acceptance Criteria			
Scenario 1: Indicadores del día • **Given** que existen citas registradas en el día, **When** el personal consulta los indicadores, **Then** el sistema retorna las citas programadas, pendientes, canceladas, las inasistencias, las citas atendidas y los cupos recuperados, con su hora de actualización. **Scenario 2: Día sin actividad** • **Given** que la fecha consultada no tiene citas ni incidencias, **When** el personal consulta los indicadores, **Then** el sistema retorna todos los indicadores en cero.			

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

US-52: Visualización de Demanda por Especialidad

Story ID	User	Priority	Epic
US-52	Personal de Admisión	Medium	EP5: Hospital Operations & System Configuration
Title	Visualización de Demanda por Especialidad		
Description			
Como Personal de Admisión, **Quiero** consultar la cantidad de reservas del día por especialidad, **Para** identificar las especialidades con mayor demanda.			
Acceptance Criteria			
Scenario 1: Demanda por especialidad • **Given** que existen reservas registradas en el día, **When** el personal consulta la demanda, **Then** el sistema retorna el total de reservas y las reservas de cada especialidad.			

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

US-53: Exportación de Reporte Operativo

Story ID	User	Priority	Epic
US-53	Personal de Admisión	Medium	EP5: Hospital Operations & System Configuration
Title	Exportación de Reporte Operativo		
Description			
Como Personal de Admisión, **Quiero** exportar un reporte en PDF o CSV indicando el rango de fechas y la especialidad, **Para** contar con la información del periodo fuera de la aplicación.			
Acceptance Criteria			
Scenario 1: Reporte generado • **Given** que el personal indica un rango de fechas válido y una especialidad o todas, **When** solicita la exportación en PDF o CSV, **Then** el sistema genera el archivo con reservas, atenciones, ausencias, demanda y reasignaciones del periodo. **Scenario 2: Rango de fechas inválido** • **Given** que la fecha inicial es posterior a la fecha final, **When** el personal solicita la exportación, **Then** el sistema rechaza la solicitud e indica corregir las fechas.			

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

Technical stories

TECH-01: Configuración de Infraestructura y Clientes para Notificaciones (Email, SMS y FCM)

Story ID	User	Priority	Epic
TECH-01	Developer	High	EP1 / EP2 / EP3
Title	Configuración de Infraestructura y Clientes para Notificaciones (Email, SMS y FCM)		
Description			
Como Developer, **Quiero** integrar los clientes de servicios de mensajería (SMTP, Firebase Cloud Messaging y pasarela SMS), **Para** proveer componentes reutilizables de envío masivo y seguro de notificaciones transaccionales a través de la arquitectura del sistema.			
Acceptance Criteria			
Scenario 1: Envío de correo electrónico transaccional via SMTP • **Given** una solicitud de envío con la plantilla HTML y los datos dinámicos requeridos,			

Story ID	User	Priority	Epic
When el servicio invoca la interfaz de transporte SMTP, **Then** la pasarela entrega el correo al destinatario y retorna una respuesta con estado `200 OK` y el ID del mensaje enviado. **Scenario 2: Envío de alerta push a dispositivo móvil vía FCM API** • **Given** un payload de notificación Push conteniendo el token del dispositivo destino, **When** el servicio ejecuta la petición HTTP POST hacia la API de Firebase Cloud Messaging, **Then** FCM responde con código `200 OK` confirmando la entrega del mensaje al dispositivo.			

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

TECH-02: Desarrollo de Endpoints RESTful API con Especificación OpenAPI y Seguridad RBAC

Story ID	User	Priority	Epic
TECH-02	Developer	High	EP5: Hospital Operations & System Configuration
Title	Desarrollo de Endpoints RESTful API con Especificación OpenAPI y Seguridad RBAC		
Description			
Como Developer, **Quiero** construir los controladores API REST e integrar la especificación OpenAPI/Swagger con control de acceso por roles, **Para** exponer endpoints estandarizados, seguros y autodocumentados para los clientes del sistema.			
Acceptance Criteria			
Scenario 1: Petición HTTP autenticada con rol autorizado (Request/Response Exitoso) • **Given** una solicitud HTTP `GET /api/v1/appointments` con un encabezado `Authorization: Bearer ` que contiene el rol autorizado, **When** el controlador procesa la petición, **Then** retorna un código de estado `200 OK` junto con el payload JSON estandarizado y documentado en Swagger UI.			

Story ID	User	Priority	Epic
Scenario 2: Denegación de acceso por rol insuficiente o token inválido • **Given** una solicitud a un endpoint protegido enviada con un token caducado o sin el rol requerido, **When** el middleware RBAC valida el token JWT, **Then** el sistema interrumpe la petición y responde con código HTTP `401 Unauthorized` o `403 Forbidden` según la falla. **Scenario 3: Manejo estandarizado de errores de request (HTTP Status Codes)** • **Given** una petición HTTP POST con un cuerpo JSON malformado o faltante de campos obligatorios, **When** el middleware de validación procesa el request, **Then** la API responde con un código `400 Bad Request` indicando la lista detallada de errores de validación por campo.			

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

TECH-03: Desarrollo de Cron Jobs en Segundo Plano para Auditoría y Control de Ausencias

Story ID	User	Priority	Epic
TECH-03	Developer	Medium	EP3: Reassignment Protocol
Title	Desarrollo de Cron Jobs en Segundo Plano para Auditoría y Control de Ausencias		
Description			
Como Developer, **Quiero** implementar workers asíncronos programados (Cron Jobs), **Para** detectar turnos cuyo tiempo de tolerancia expiró, actualizar su estado a «Ausente» y desencadenar el protocolo de reasignación.			
Acceptance Criteria			
Scenario 1: Procesamiento y actualización masiva de citas expiradas • **Given** la ejecución periódica programada de la tarea en segundo plano,			

Story ID	User	Priority	Epic
When el worker audita las citas del intervalo actual cuyo margen de tolerancia expiró sin confirmación de check-in, **Then** el sistema actualiza el estado de la entidad a «Ausente» en la base de datos registrando el timestamp de la acción. **Scenario 2: Disparo de eventos de reasignación tras detección de ausencia** • **Given** una cita médica marcada como «Ausente» por el worker, **When** se confirma la persistencia del nuevo estado, **Then** el worker publica el evento `AppointmentExpiredEvent` en la cola de mensajes para iniciar el protocolo de reasignación de cupos. **Scenario 3: Bloqueo distribuido para evitar duplicidad de ejecución** • **Given** múltiples réplicas del servicio ejecutándose de manera concurrente, **When** la tarea Cron se dispara simultáneamente en varios nodos, **Then** el primer worker adquiere un bloqueo distribuido (Redis/Database Lock), evitando ejecuciones duplicadas sobre los mismos registros.			

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

TECH-04: Desarrollo de Consumidor Asíncrono de Eventos de Notificaciones de Citas

Story ID	User	Priority	Epic
TECH-04	Developer	Medium	EP2: Appointments & Booking Engine
Title	Desarrollo de Consumidor Asíncrono de Eventos de Notificaciones de Citas		
Description			
Como Developer, **Quiero** desarrollar un consumidor asíncrono de eventos de notificaciones,			

Story ID	User	Priority	Epic
Para procesar los eventos de reserva, cancelación o reasignación de citas enviando las alertas correspondientes sin penalizar el tiempo de respuesta del API REST.			
Acceptance Criteria			
Scenario 1: Consumo y despacho exitoso del evento de reserva (Request/Response Event) • **Given** el evento `AppointmentBookedEvent` publicado en la cola de notificaciones, **When** el consumidor recupera el mensaje y genera la plantilla correspondiente, **Then** el sistema efectúa el dispatch hacia el proveedor externo obteniendo un código de entrega exitoso y marcando el mensaje de la cola como procesado (`ACK`). **Scenario 2: Manejo de fallas temporales mediante Dead Letter Queue (DLQ)** • **Given** un fallo de conexión (5xx / Timeout) con la pasarela de notificaciones, **When** el consumidor detecta la excepción durante el procesamiento, **Then** el sistema aplica un reintento con *Exponential Backoff* y, si se supera el límite máximo de reintentos, transfiere el mensaje a la Dead Letter Queue (DLQ) registrando el log de auditoría.			

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

TECH-05: Desarrollo del Endpoint API REST para Consulta y Validación de DNI Externa

Story ID	User	Priority	Epic
TECH-05	Developer	High	EP1: Authentication & Identity Management
Title	Desarrollo del Endpoint API REST para Consulta y Validación de DNI Externa		
Description			
Como Developer, **Quiero** implementar el endpoint REST de consulta de DNI con manejo de cache y Circuit Breaker, **Para** exponer un servicio de validación de identidad rápido y tolerante a fallos para el registro de usuarios.			
Acceptance Criteria			
Scenario 1: Respuesta exitosa de validación de DNI (HTTP 200 OK)			

Story ID	User	Priority	Epic
• **Given** una solicitud `POST /api/v1/identity/verify-dni` conteniendo un DNI de 8 dígitos válido en el cuerpo JSON, **When** la API consulta el servicio (o la capa de cache en memoria), **Then** el sistema retorna un código de estado `200 OK` con los nombres y apellidos validados en el payload JSON. **Scenario 2: Validación de formato de entrada (HTTP 400 Bad Request)** • **Given** una petición HTTP enviada con un DNI con formato incorrecto (menos de 8 dígitos o caracteres alfanuméricos), **When** el middleware de validación procesa el request, **Then** el sistema rechaza la petición respondiendo con un código `400 Bad Request` y el detalle del error de formato. **Scenario 3: Respuesta ante caída o degradación del proveedor externo (HTTP 503 Service Unavailable)** • **Given** la API externa inactiva o con el patrón Circuit Breaker en estado abierto (`Open`), **When** el frontend realiza la consulta de un DNI, **Then** el endpoint responde con un código `503 Service Unavailable` adjuntando la bandera para permitir el registro condicional en el cliente.			

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

2.4.2. Impact Mapping

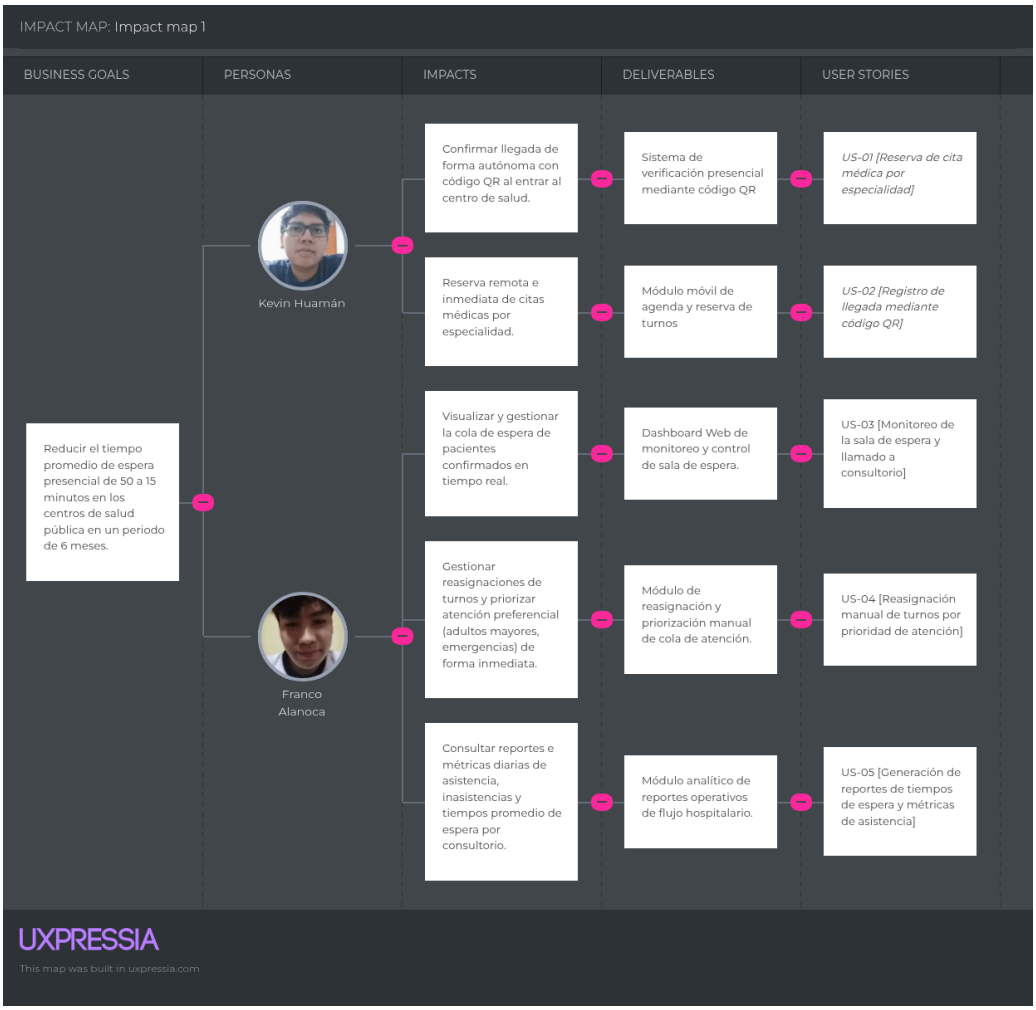
El mapa de impacto para nuestra plataforma de **gestión de citas médicas y control de sala de espera**, define una meta SMART: *«Optimizar la atención médica y reducir el tiempo de espera operativo; al primer año de despliegue contar con 15 clínicas afiliadas, 25 000 reservas procesadas y un 75% de confirmación de presencia vía QR»*. A partir de esta meta se explicitan los cambios de comportamiento esperados en los actores clave del sistema.

Como referencia cualitativa se emplean dos personas modelo: **Personal de Admisión** (gestión operativa y flujo en sala) y **Paciente** (reserva autónoma y confirmación de llegada). Para el primero, el mapa recoge necesidades relativas a visibilidad en tiempo real de pacientes confirmados, claridad en el llamado a consultorio, flexibilidad para aplicar prioridades o reasignaciones por imprevistos, y generación de métricas de ausentismo; para el segundo, autonomía para seleccionar especialidad y horario, confianza mediante el registro rápido por código QR sin filas presenciales, y claridad sobre su estado dentro de la cola de espera.

A partir de estos impactos se declaran entregables de producto susceptibles de materializar el cambio de conducta: catálogo de especialidades y cupos en tiempo real, protocolo de reasignación de cupos liberados por orden de `bookingOrder`, validador de presencia mediante lectura y verificación de hash QR con tolerancia de tiempo, monitor central para admisión con ordenamiento automático por llegada, dashboard operativo con indicadores de citas programadas, citas pendientes, citas canceladas e inasistencias del día, y motor analítico con exportación de reportes operativos (PDF/CSV). En el ámbito técnico y de arquitectura, se integran endpoints RESTful seguros (HTTP 200/401) para el registro de presencia y un motor de notificaciones en tiempo real para avisos instantáneos de llamados y liberación de cupos. La última dimensión del método vincula estos entregables con historias de usuario (US), historias técnicas (TS) y *spikes* (SP) en formato *Como… / quiero… / para…* (y su equivalente técnico *Objective / Given / When / Then*), asegurando la trazabilidad directa desde la meta estratégica hasta el desarrollo funcional, como se observa en la Figura 35.

Figura 35

Impact Mapping de SaludYa



Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

2.4.3. Product Backlog

El Product Backlog ha sido priorizado en función del **valor directo entregado al negocio y a los usuarios**, asegurando que los entregables visibles y de alto impacto (como la Landing Page y el flujo principal de reservas) se aborden desde los primeros Sprints. La priorización sigue el principio de **valor de negocio primero**: las historias relacionadas con la propuesta de valor visible para el paciente (Landing Page, reserva de citas, check-in QR) y la operación crítica del establecimiento (gestión de cupos, reasignación de turnos liberados y dashboard operativo) se ubican en los primeros lugares, mientras que las historias de soporte técnico y configuración avanzada se postergan a Sprints posteriores.

Tabla del Product Backlog

El backlog ordena las historias de usuario por prioridad e identifica su rol, alcance, estimación en story points y sprint previsto, como se detalla en la Tabla 22.

Tabla 22

Product Backlog de SaludYa

# Orden	User Story Id	Rol	Título	Story Points (1 / 2 / 3 / 5 / 8)	Sprint
1	US-00	Paciente potencial	Visualización de Propuesta de Valor en Landing Page	2	Sprint 1
2	US-01	Paciente	Verificación de Identidad por DNI	5	Sprint 1
3	US-02	Paciente	Registro de Credenciales de Acceso	3	Sprint 1
4	US-03	Paciente	Verificación de Celular y Creación de Cuenta	5	Sprint 1
5	US-04	Paciente	Inicio de Sesión del Paciente	3	Sprint 1
6	US-05	Paciente	Selección de Especialidad Médica	2	Sprint 1
7	US-06	Paciente	Consulta de Horarios Disponibles	5	Sprint 1
8	US-07	Paciente	Reserva de Cita para el Paciente	8	Sprint 1
9	US-08	Paciente	Recepción de Confirmación de Cita por Correo	3	Sprint 1
10	US-09	Paciente	Consulta de Citas Agendadas	3	Sprint 1
11	US-10	Paciente	Cancelación de Cita	5	Sprint 1
12	US-11	Paciente	Consulta del Código QR de la Reserva	2	Sprint 1
13	US-12	Paciente	Consulta del Ticket Digital de Atención	3	Sprint 1
14	US-13	Paciente	Consulta de Posición en la Cola de Atención	3	Sprint 1
15	US-14	Paciente	Recepción de Notificación de Llamado a Consultorio	3	Sprint 1
16	US-15	Paciente	Vinculación de un Menor de Edad	5	Sprint 1

# Orden	User Story Id	Rol	Título	Story Points (1 / 2 / 3 / 5 / 8)	Sprint
17	US-16	Paciente	Reserva de Cita para un Menor Vinculado	5	Sprint 1
18	US-17	Paciente	Consulta de Menores Vinculados	2	Sprint 1
19	US-18	Paciente	Consulta del Perfil de un Menor	2	Sprint 1
20	US-19	Paciente	Desvinculación de un Menor de Edad	2	Sprint 1
21	US-20	Paciente	Aceptación de Adelanto de Cita	5	Sprint 1
22	US-21	Paciente	Rechazo de Adelanto de Cita	3	Sprint 1
23	US-22	Paciente	Salida Voluntaria de la Cola de Atención	3	Sprint 1
24	US-23	Paciente	Solicitud de Enlace de Recuperación de Contraseña	3	Sprint 1
25	US-24	Paciente	Restablecimiento de Contraseña	3	Sprint 1
26	US-25	Paciente	Solicitud de Ayuda por Pérdida de Acceso a la Cuenta	2	Sprint 1
27	US-26	Paciente	Consulta del Perfil del Paciente	2	Sprint 1
28	US-27	Paciente	Edición de Datos de Contacto	3	Sprint 1
29	US-28	Paciente	Cierre de Sesión del Paciente	2	Sprint 1
30	US-29	Paciente	Aviso de Sesión Expirada	2	Sprint 1
31	US-30	Super Admin	Inicio de Sesión del Super Admin	3	Sprint 2
32	US-31	Super Admin	Creación de Cuenta de Personal de Admisión	3	Sprint 2
33	US-32	Personal de Admisión	Inicio de Sesión del Personal de Admisión	3	Sprint 2

# Orden	User Story Id	Rol	Título	Story Points (1 / 2 / 3 / 5 / 8)	Sprint
34	US-33	Super Admin	Configuración de la Duración de los Intervalos de Atención	3	Sprint 2
35	US-34	Super Admin	Edición de una Regla Operativa del Establecimiento	5	Sprint 2
36	US-35	Super Admin	Consulta de Reglas Operativas Vigentes	2	Sprint 2
37	US-36	Personal de Admisión	Selección de Especialidad en la Agenda Operativa	2	Sprint 2
38	US-37	Personal de Admisión	Consulta del Calendario Operativo	3	Sprint 2
39	US-38	Personal de Admisión	Consulta de Bloques del Día	3	Sprint 2
40	US-39	Personal de Admisión	Consulta del Detalle de un Bloque	2	Sprint 2
41	US-40	Personal de Admisión	Consulta de Horario por Profesional	3	Sprint 2
42	US-41	Personal de Admisión	Consulta de Pacientes con Reserva en un Bloque	2	Sprint 2
43	US-42	Personal de Admisión	Edición de un Bloque Horario	5	Sprint 2
44	US-43	Personal de Admisión	Registro de Llegada por Escaneo de QR	8	Sprint 2
45	US-44	Personal de Admisión	Registro de Llegada por Código Manual	3	Sprint 2
46	US-45	Personal de Admisión	Consulta de la Cola de Atención Presencial	3	Sprint 2
47	US-46	Personal de Admisión	Llamado del Siguiente Paciente	5	Sprint 2
48	US-47	Personal de Admisión	Inicio de Atención de un Paciente Llamado	3	Sprint 2
49	US-48	Personal de Admisión	Finalización de la Atención de un Paciente	3	Sprint 2
50	US-49	Personal de Admisión	Registro de Ausencia de un Paciente	5	Sprint 2

# Orden	User Story Id	Rol	Título	Story Points (1 / 2 / 3 / 5 / 8)	Sprint
51	US-50	Personal de Admisión	Consulta de Citas Canceladas del Día	2	Sprint 2
52	US-51	Personal de Admisión	Visualización de Indicadores del Día	5	Sprint 2
53	US-52	Personal de Admisión	Visualización de Demanda por Especialidad	3	Sprint 2
54	US-53	Personal de Admisión	Exportación de Reporte Operativo	5	Sprint 2
55	TECH-01	Developer	Configuración de Infraestructura y Clientes para Notificaciones (Email, SMS y FCM)	5	Sprint 4
56	TECH-02	Developer	Desarrollo de Endpoints RESTful API con Especificación OpenAPI y Seguridad RBAC	8	Sprint 4
57	TECH-03	Developer	Desarrollo de Cron Jobs en Segundo Plano para Auditoría y Control de Ausencias	5	Sprint 4
58	TECH-04	Developer	Desarrollo de Consumidor Asíncrono de Eventos de Notificaciones de Citas	5	Sprint 4
59	TECH-05	Developer	Desarrollo del Endpoint API REST para Consulta y Validación de DNI Externa	5	Sprint 4

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

/s

2.5. Strategic-Level Domain-Driven Design

2.5.1. EventStorming

Con el fin de plantear una aproximación del modelado de nivel general para el dominio del problema, se aplicó la técnica de EventStorming. Este proceso permitió al equipo comprender el flujo de eventos que

ocurren dentro del dominio de la gestión de citas médicas en establecimientos públicos de salud, y definir las interacciones principales entre los actores, comandos y políticas del sistema **SaludYa**.

La sesión se realizó con una duración aproximada de **2 horas**, con la participación de los cinco integrantes del equipo RuwaLabs, utilizando **Miro** como tablero colaborativo.

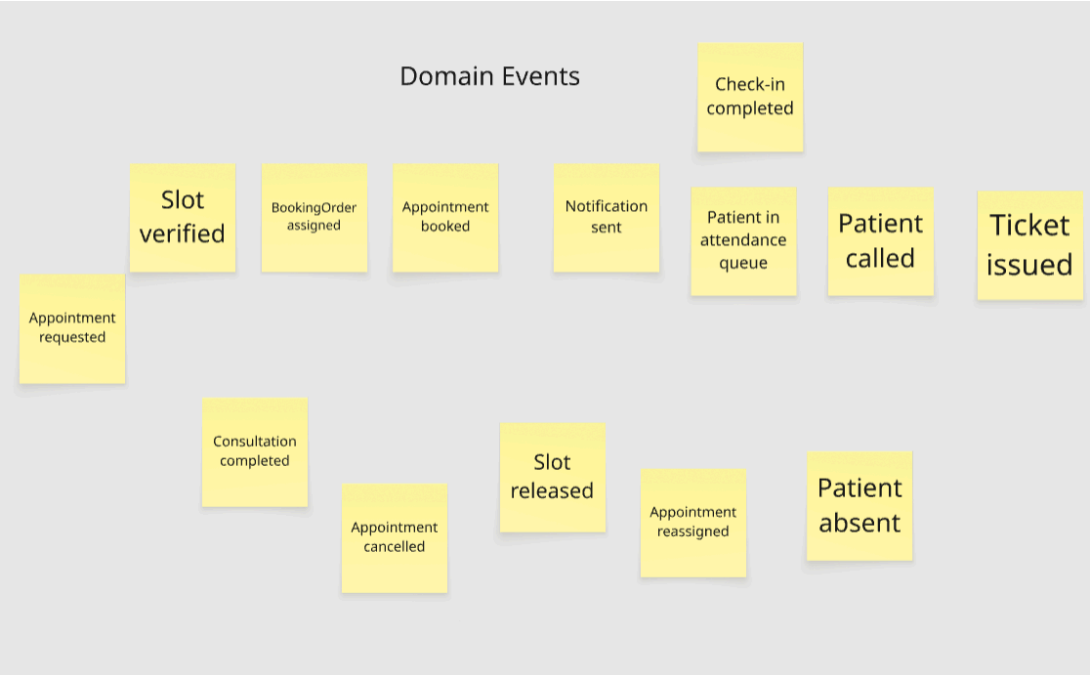
Pasos del proceso:

1. **Identificación de los Domain Events:**

En la primera fase, se identificaron los eventos clave que ocurren dentro del dominio (por ejemplo, «Appointment booked», «Slot released», «Check-in completed»). Estos se colocaron en secuencia sobre una línea de tiempo para visualizar el flujo del negocio. En esta fase se incorporaron los eventos `Booking Order assigned`, `Patient in attendance queue` y `Patient called`, que reflejan la existencia de dos colas complementarias dentro del dominio: la **cola por pedido de cita** (ordenada por `Booking Order`) y la **cola de asistencia** (ordenada por `checkInTimestamp`).

Figura 36

EventStorming - Domain Events



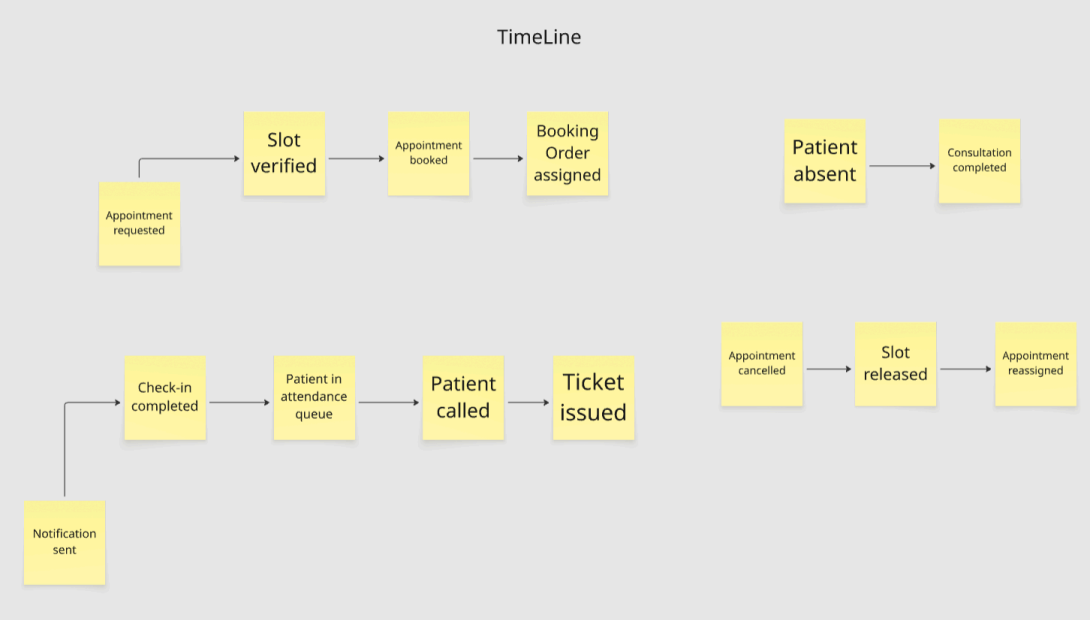
Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

2. **Organización cronológica de los eventos:**

Luego, los eventos fueron ordenados según el momento en que ocurren dentro del proceso real de atención, permitiendo entender la secuencia lógica de las operaciones desde la pre-atención hasta el cierre de la consulta. En esta línea de tiempo se distingue que el evento `Booking Order assigned` ocurre inmediatamente después de `Appointment booked`, mientras que `Patient in attendance queue` y `Patient called` ocurren después de `Check-in completed`, evidenciando que ambas colas operan en momentos distintos del flujo, como se observa en la Figura 36.

Figura 37

EventStorming - Timeline



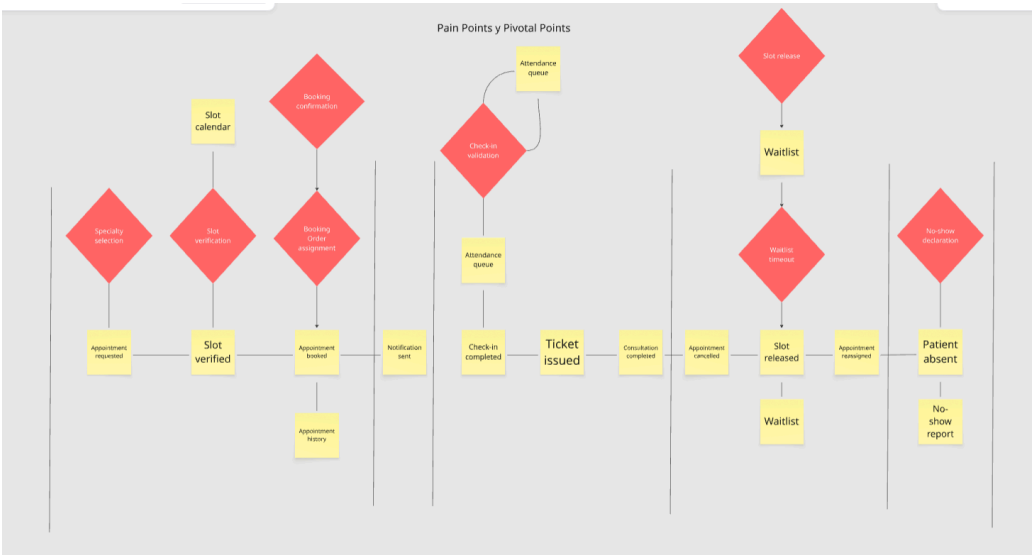
Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

3. Identificación de Pain Points y Pivotal Points:

En esta etapa se marcaron los **pain points**, es decir, las posibles dificultades o cuellos de botella del proceso actual, y los **pivotal points**, que representan los eventos más críticos o de cambio dentro del flujo. Se identificaron como puntos críticos la **asignación del** `Booking Order` (que determina la prioridad en la lista de espera) y el **waitlist timeout** (que define el paso al siguiente paciente cuando nadie responde una propuesta de cupo liberado). Asimismo, se incorporaron como read models la **attendance queue** y la **waitlist**, como se observa en la Figura 37.

Figura 38

EventStorming - Pain Points y Pivotal Points



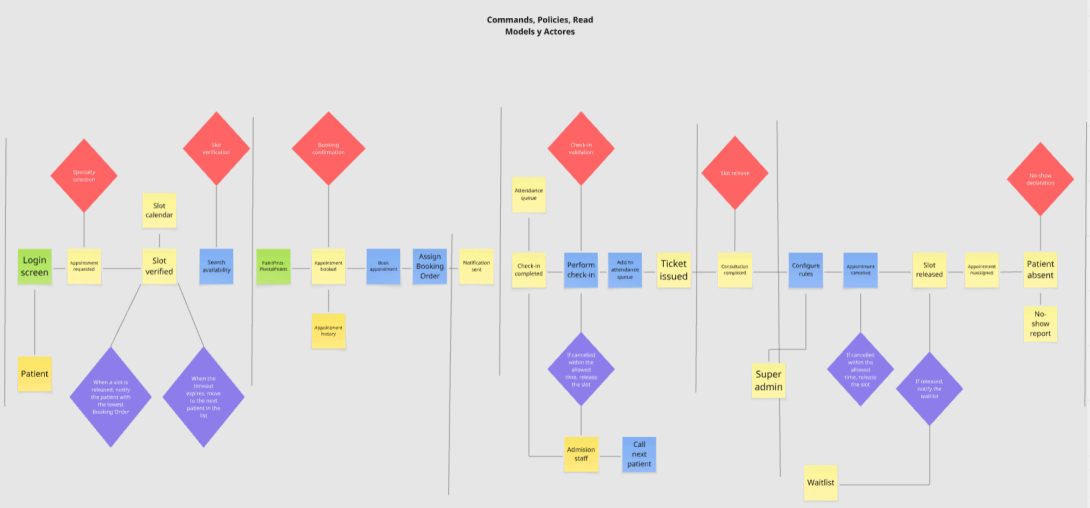
Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

4. Incorporación de Commands, Policies y Read Models:

Finalmente, se agregaron los **commands** (acciones que disparan eventos), las **policies** (reglas de negocio que responden a eventos) y los **read models** (consultas de información). Esto permitió obtener una visión más completa y técnica del dominio de SaludYa. En esta fase se incorporaron los commands `Assign Booking Order`, `Add to attendance queue` y `Call next patient` las policies `When a slot is released, notify the patient with the lowest Booking Order`, `When the timeout expires, move to the next patient in the list` y `If cancelled within the allowed time, release the slot` y los read models `Attendance Queue` y `Waitlist`, como se observa en la Figura 38 y la Figura 39.

Figura 39

EventStorming - Commands, Policies y Read Models



Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

2.5.1.1. Candidate Context Discovery

A partir del modelado realizado en el EventStorming, se llevó a cabo una sesión de Candidate Context Discovery con el objetivo de identificar los bounded contexts dentro del dominio de SaludYa. Para ello, se aplicó la técnica **«look-for-pivotal-events»**, que permitió detectar los eventos clave del negocio que marcan transiciones de estado y delimitan responsabilidades entre diferentes partes del proceso.

Durante la sesión, se reorganizó la línea de tiempo del EventStorming para agrupar los elementos relacionados —eventos, comandos, políticas y read models— en torno a sus respectivos aggregates. Esto facilitó distinguir los límites naturales entre los contextos y definir con mayor claridad las interacciones entre ellos.

Es importante precisar que **`Attendance Queue` no constituye un bounded context**, sino un **agregado dentro del bounded context `Arrival & QR Check-in`**. Su responsabilidad se limita a ordenar la atención presencial del día según el timestamp de check-in, y su ciclo de vida es efímero (por día y por franja horaria). De manera análoga, **`Booking Order` no constituye un bounded context**, sino un **atributo de `Appointment` dentro del bounded context `Appointments & Booking`**, cuyo propósito es determinar la prioridad de reasignación en la lista de espera dinámica. Un bounded context se justifica únicamente cuando existe lenguaje propio, reglas de negocio complejas y autonomía de modelo; ninguna de las dos colas cumple esas condiciones por separado.

Como resultado del proceso, se identificaron **cinco bounded contexts candidatos** para el dominio de SaludYa:

Tabla 23

Candidate Context Discovery

#	Bounded Context	Propósito	Eventos clave
1	**Identity & Access Management**	Gestionar el registro, autenticación y roles de pacientes y personal administrativo.	Account created, Session started, Minor linked
2	**Appointments & Booking**	Gestionar la búsqueda de disponibilidad, reserva y cancelación de citas médicas. Incluye el atributo `Booking Order`, que asigna un número secuencial a cada cita reservada.	Appointment requested, Slot verified, Appointment booked, Booking Order assigned, Appointment cancelled
3	**Dynamic Waitlist & Reassignment**	Gestionar la lista de espera dinámica y la reasignación de cupos liberados, ofreciendo las propuestas en orden de `Booking Order`.	Slot released, Appointment reassigned, Reassignment expired
4	**Arrival & QR Check-in**	Validar la presencia presencial del paciente, emitir el ticket digital de atención y gestionar la `Attendance Queue`.	Check-in completed, Patient in attendance queue, Patient called, Ticket issued, Patient absent
5	**Hospital Operations & Configuration**	Configurar parámetros operativos del establecimiento y monitorear la operación diaria.	Rules updated, Report generated

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

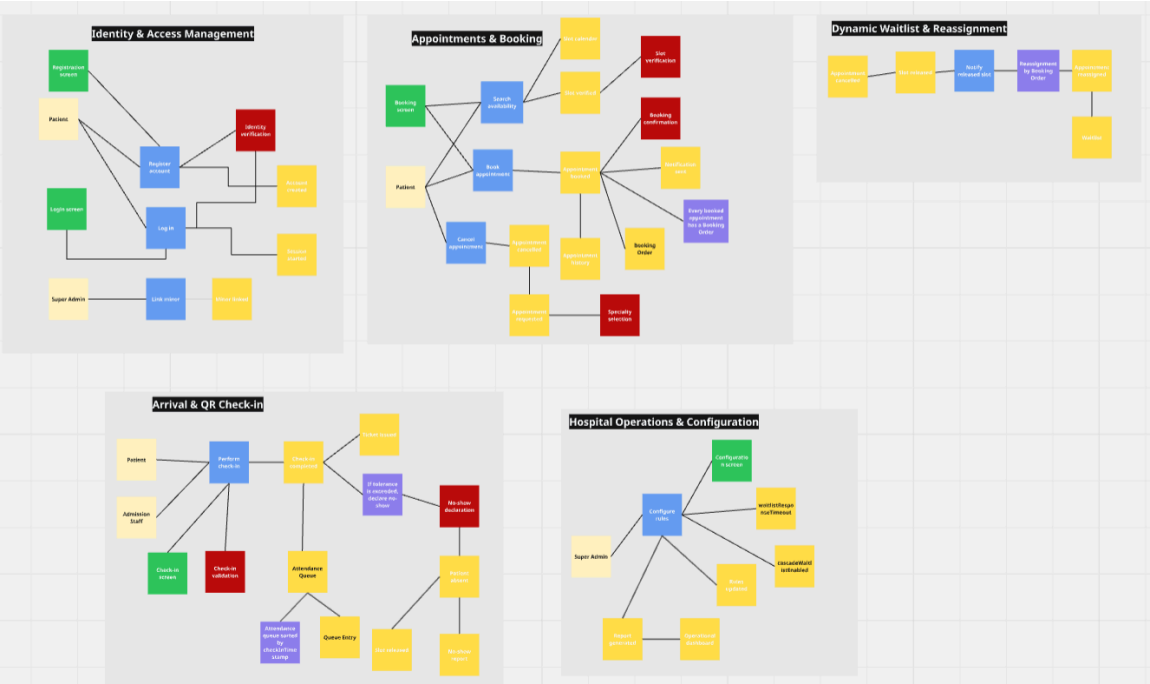
A continuación se detalla, para cada bounded context, los elementos incorporados en la sesión de Candidate Context Discovery, como se detalla en la Tabla 23:

- **Appointments & Booking :** se incorpora el atributo `Booking Order` y la regla de negocio *«Every booked appointment has a unique Booking Order per specialty, date and facility»*.
- **Dynamic Waitlist & Reassignment :** se incorporan los conceptos `Waitlist Entry` (entrada ordenada por `Booking Order`), `Cascade Reassignment` (reasignación en cascada si nadie acepta) y `Waitlist Response Timeout` (tiempo máximo para aceptar o rechazar). La policy de reasignación se define como *«Reassignment by Booking Order»*.
- **Arrival & QR Check-in :** se incorporan los conceptos `Attendance Queue` (cola virtual ordenada por `checkInTimestamp`) y `Queue Entry` (entrada individual en la cola de asistencia). La policy asociada se define como *«Attendance Queue sorted by checkInTimestamp»*.
- **Hospital Operations & Configuration :** se incorpora el parámetro `waitlistResponseTimeout` (y opcionalmente `cascadeWaitlistEnabled` y `maxCapacityPerSlot`) como parte de las reglas operativas configurables por el establecimiento.

La identificación de bounded contexts delimita las responsabilidades de identidad, reservas, reasignación, llegada y configuración operativa dentro del dominio de SaludYa, como se observa en la Figura 40.

Figura 40

Candidate Context Discovery - Bounded Contexts identificados



Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

2.5.1.2. Domain Message Flows Modeling

En esta sección se presentan los principales flujos de colaboración entre los bounded contexts, modelados con la técnica de **Domain Message Flow Modelling** propuesta por ddd-crew. Cada diagrama muestra los comandos, eventos y queries que fluyen entre actores, bounded contexts y sistemas para un escenario específico, incluyendo el nombre del mensaje, su orden y los datos que transporta.

Los flujos reflejan las dos colas complementarias del dominio: la **cola por pedido de cita** (ordenada por `bookingOrder`) y la **cola de asistencia** (ordenada por `checkInTimestamp`).

Flow 1: Registro y autenticación de paciente

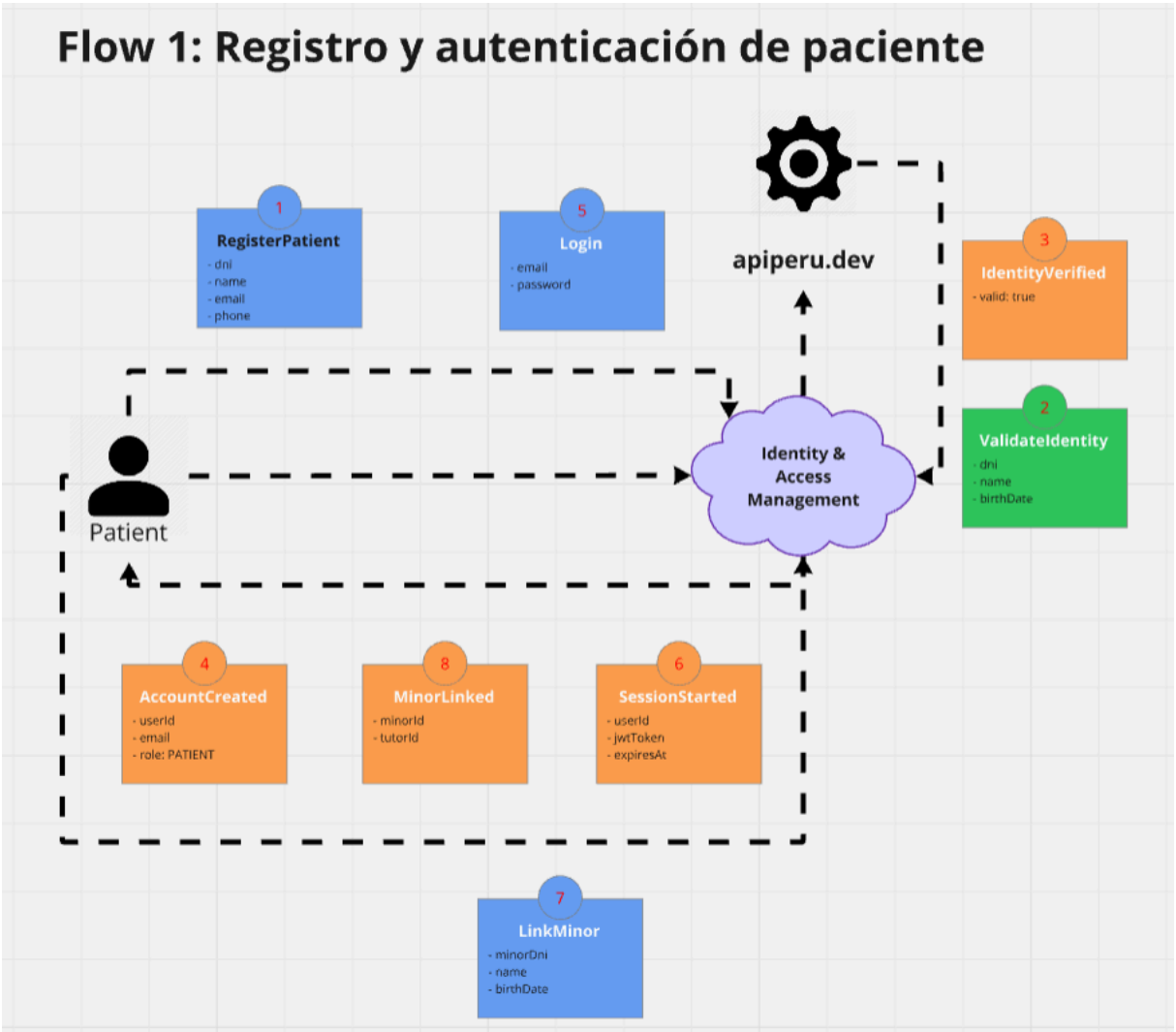


Figura 1: Flow 1

El paciente ejecuta `RegisterPatient` **Identity & Access Management** valida la identidad con `ValidateIdentity` contra **ApiPeru.dev** (ACL hacia RENIEC) y responde con `IdentityVerified` → `AccountCreated` . Luego `Login` → `SessionStarted` , y `LinkMinor` → `MinorLinked` .

Flow 2: Reserva de cita médica para el titular

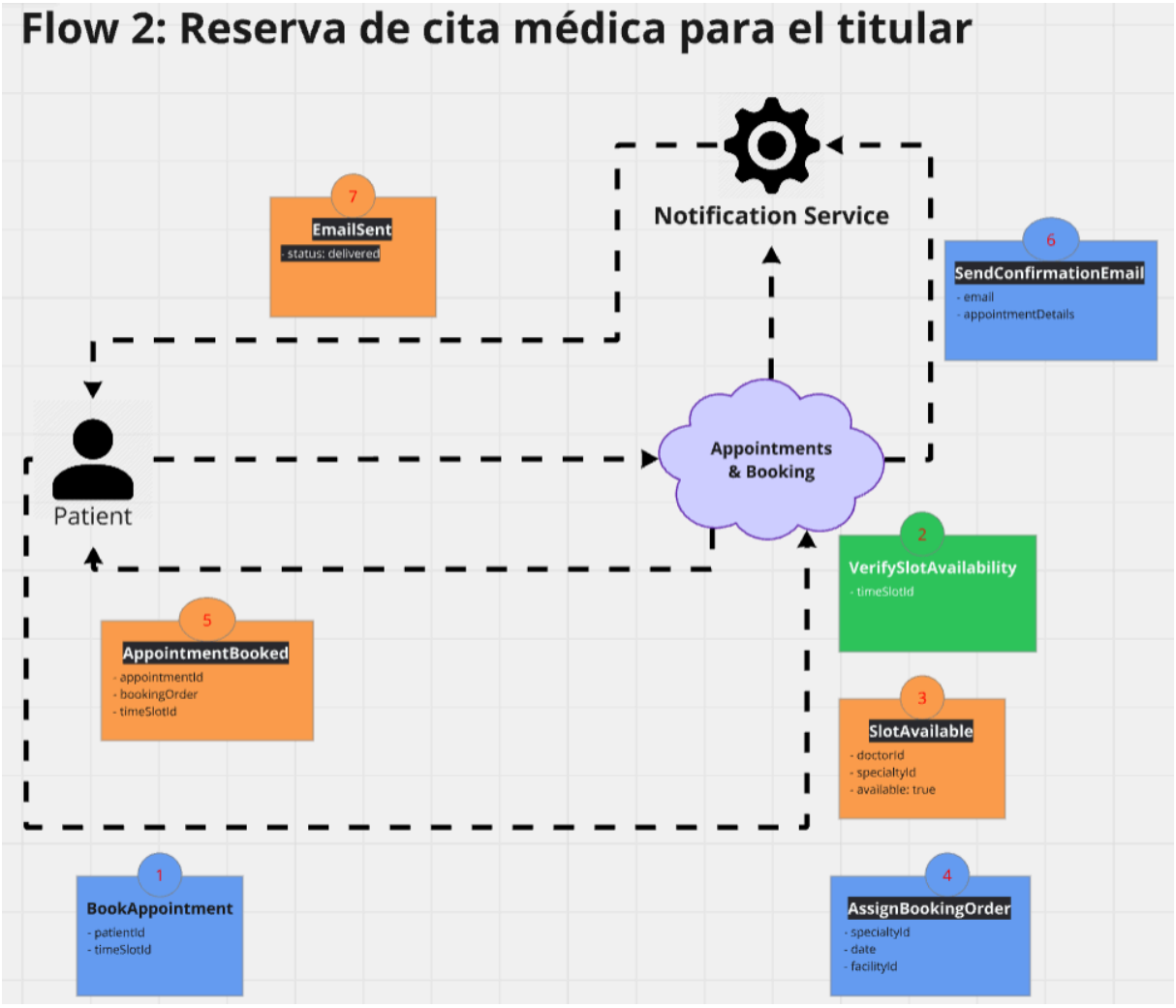


Figura 2: Flow 2

El paciente ejecuta `BookAppointment` **Appointments & Booking** valida con `VerifySlotAvailability` → `SlotAvailable`, asigna el `AssignBookingOrder` y emite `AppointmentBooked`. Finalmente notifica al paciente vía `SendConfirmationEmail` → `EmailSent`.

Flow 3: Reserva de cita médica para un menor a cargo

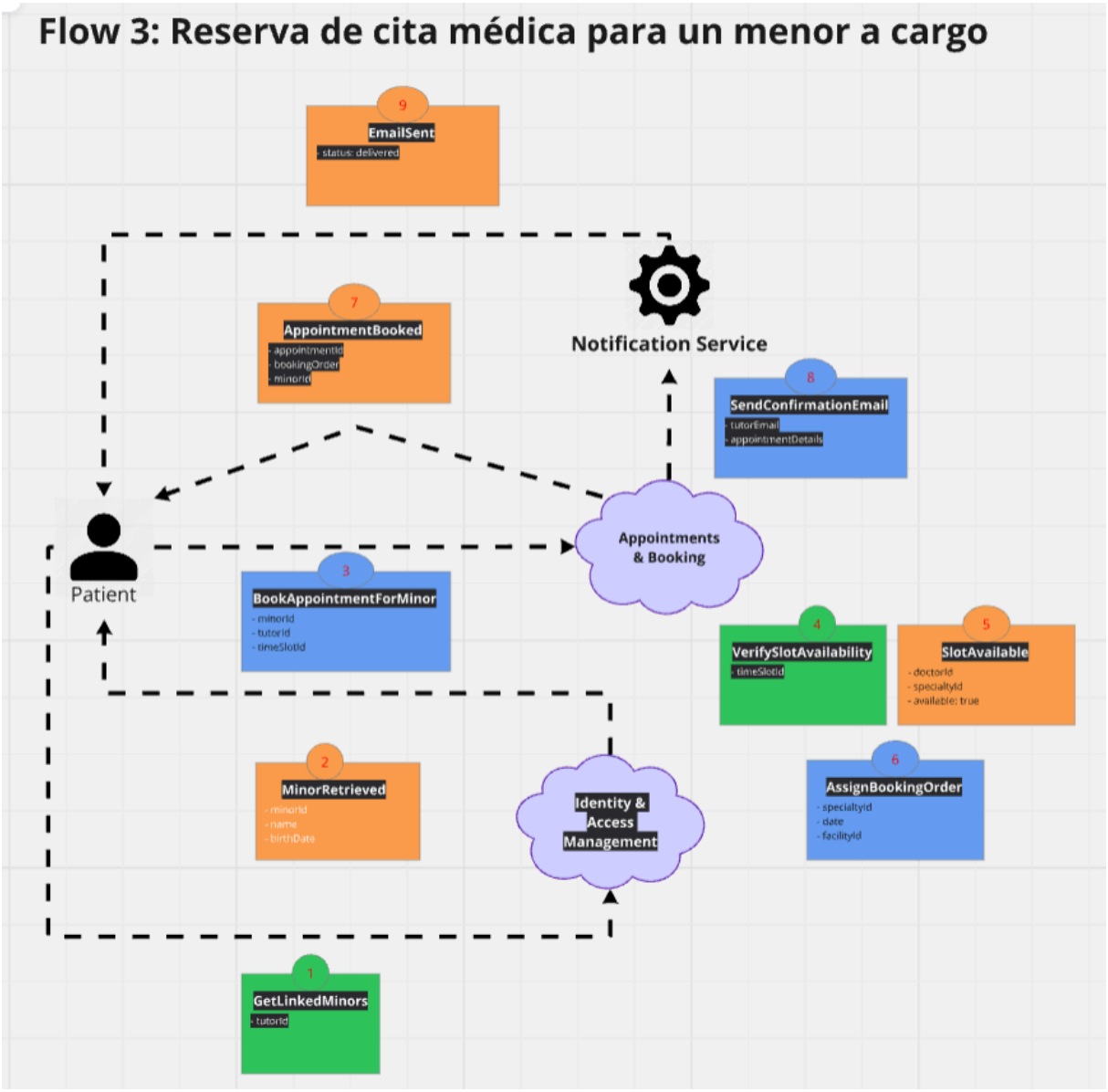


Figura 3: Flow 3

El titular ejecuta `GetLinkedMinors` (IAM) → `MinorRetrieved`, y luego `BookAppointmentForMinor` con el `minorId`. **Appointments & Booking** valida con `VerifySlotAvailability` → `SlotAvailable`, asigna el `bookingOrder` y emite `AppointmentBooked`, notificando al tutor.

Flow 4: Check-in presencial mediante código QR

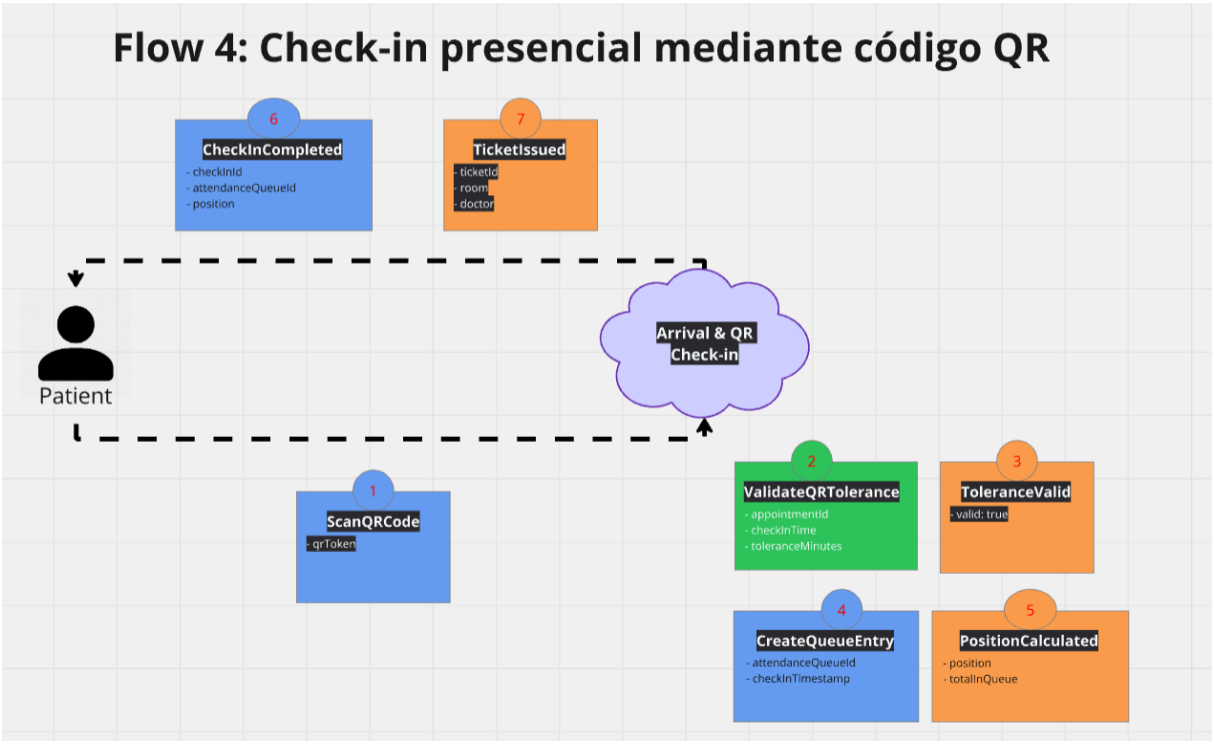


Figura 4: Flow 4

El paciente ejecuta `ScanQRCode` **Arrival & QR Check-in** valida con `ValidateQRTolerance` → `ToleranceValid`, crea la entrada con `CreateQueueEntry`, calcula la posición (`PositionCalculated`) y emite `CheckInCompleted` y `TicketIssued`.

Flow 5: Cancelación de cita y liberación de cupo

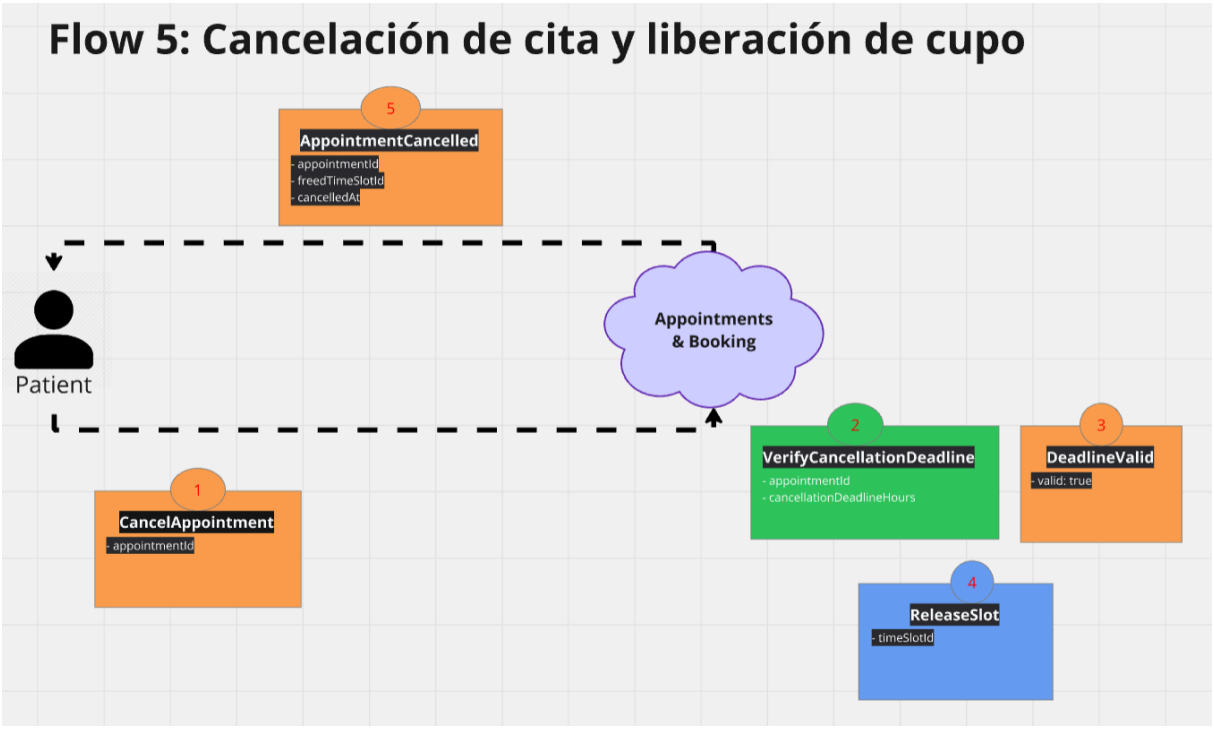


Figura 5: Flow 5

El paciente ejecuta `CancelAppointment` **Appointments & Booking** valida con `VerifyCancellationDeadline` → `DeadlineValid` , ejecuta `ReleaseSlot` y emite `AppointmentCancelled` .

> **Nota:** la cancelación **no dispara reasignación**. Solo la ausencia presencial lo hace (ver Flow 6).

Flow 6a: Declaración de ausencia

El personal de admisión ejecuta `CallPatient` **Arrival & QR Check-in** valida con `VerifyPostCallTolerance` → `ToleranceExpired` , y emite `PatientAbsentEvent` hacia **Reassignment**.

Flow 6b: Reasignación en cadena

Reassignment ejecuta `FindNextCandidate` en **Appointments & Booking** → `CandidateFound` , envía `ReassignmentOfferSent` al paciente, quien responde con `ReassignmentOfferAccepted` . **Reassignment** ejecuta `ReassignAppointment` y `ReleaseOriginalSlot` para liberar el slot del candidato y re-ofrecerlo al siguiente (cadena). Si el candidato acepta y no llega, se emite `ReassignmentOfferNoShow` .

Flow 7: Configuración operativa del establecimiento

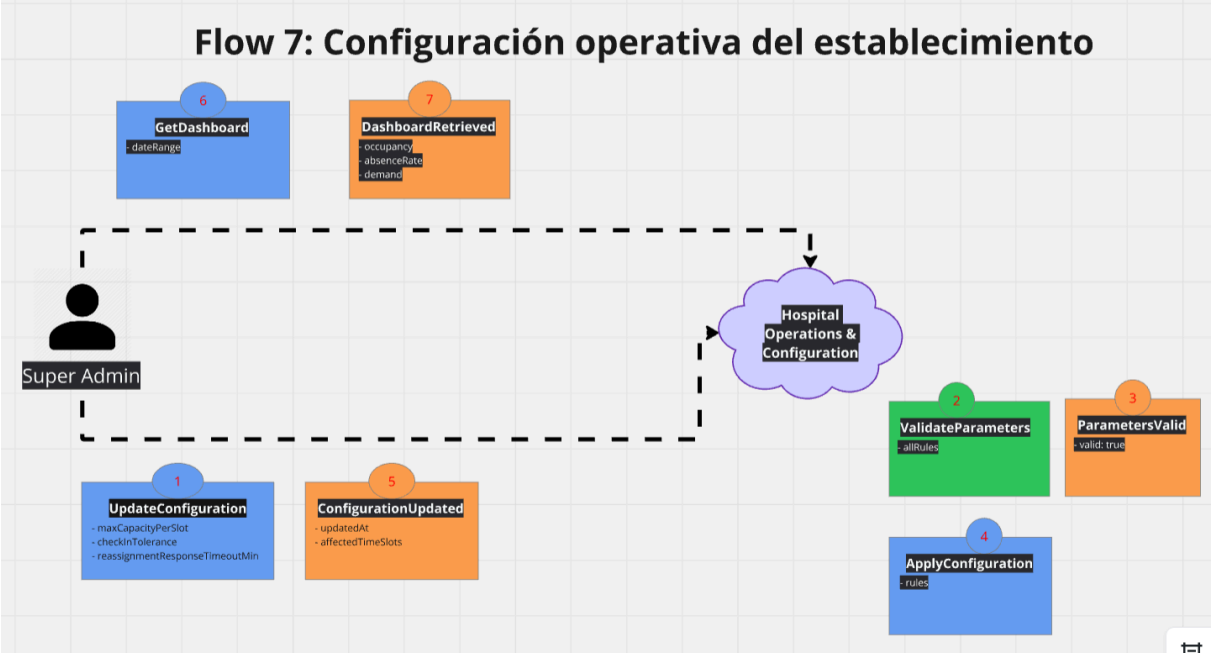


Figura 6: Flow 7

El Super Admin ejecuta `UpdateConfiguration` **Hospital Operations & Configuration** valida con `ValidateParameters` → `ParametersValid` , aplica con `ApplyConfiguration` y emite `ConfigurationUpdated` . Finalmente, `GetDashboard` → `DashboardRetrieved` .

En conjunto, los flujos evidencian que la **cola por pedido de cita** se gestiona como atributo dentro de `Appointments & Booking` y se consume desde `Reassignment` para priorizar la reasignación, mientras que la **cola de asistencia** se gestiona como agregado dentro de `Arrival & QR Check-in` para ordenar el llamado a consultorio.

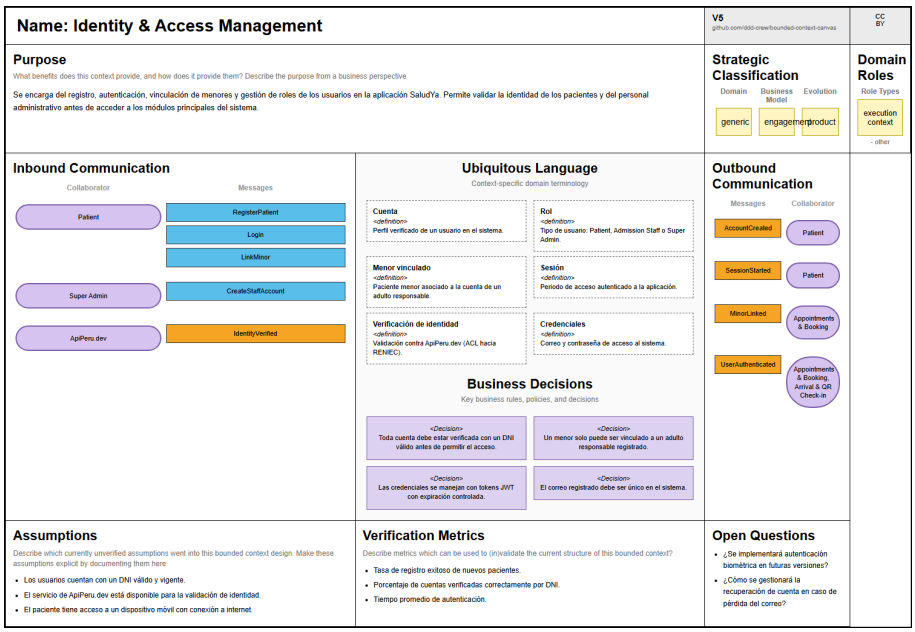
2.5.1.3. Bounded Context Canvases

Para la presente sección, elaboramos el Bounded Context Canvas de cada uno de los Bounded Context candidatos que identificamos. Aplicamos el modelo **versión 5** propuesto por el Domain Driven Design Group, que permite documentar cada contexto desde una perspectiva estratégica y táctica, cubriendo el propósito, la clasificación estratégica, las comunicaciones de entrada y salida, el lenguaje ubicuo, las decisiones de negocio, los supuestos, las métricas de verificación y las preguntas abiertas.

En cada uno de los canvases registramos las secciones específicas como el **Context Overview Definition**, **Business Rules Distillation** y el **Ubiquitous Language**, identificando claramente el tipo de Bounded Context y sus interacciones de entrada y salida con otros contextos. El resultado se presenta a continuación para cada uno de los cinco bounded contexts identificados en la sección 2.5.1.1.

Bounded Context Canvas – Identity & Access Management

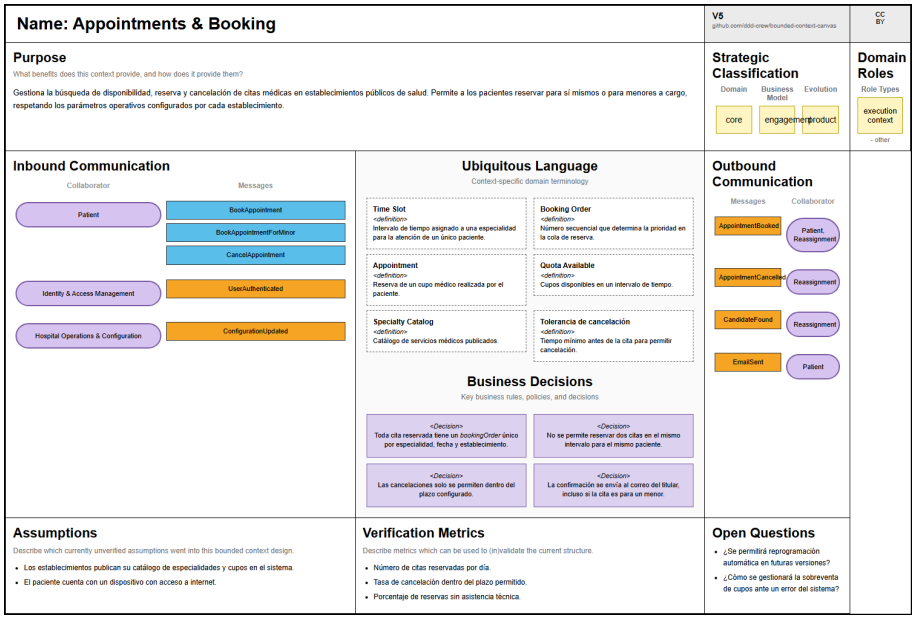
El bounded context **Identity & Access Management** gestiona el registro, autenticación, vinculación de menores y gestión de roles de los usuarios en SaludYa. Se clasifica como *Generic* con rol *Execution Context*, e interactúa principalmente con `Patient` , `Super Admin` y el servicio externo **ApiPeru.dev** (Anticorruption Layer hacia RENIEC) para la validación de identidad.



Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

Bounded Context Canvas – Appointments & Booking

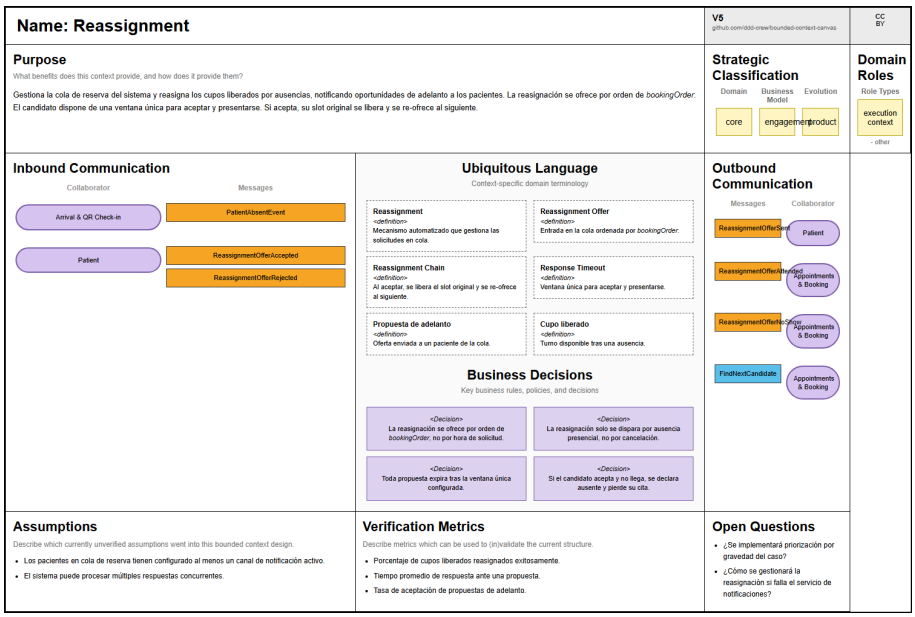
El bounded context **Appointments & Booking** gestiona la búsqueda de disponibilidad, reserva y cancelación de citas médicas. Se clasifica como *Core* con rol *Execution Context*, e incorpora el atributo `Booking Order`, que asigna un número secuencial a cada cita reservada y determina la prioridad en la cola de reserva consumida por `Reassignment`.



Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

Bounded Context Canvas – Reassignment

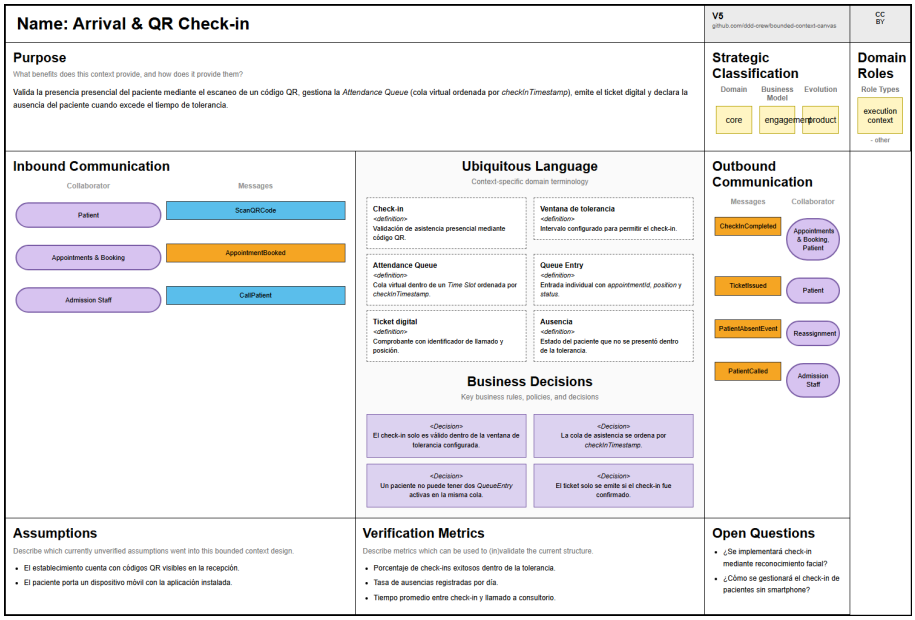
El bounded context **Reassignment** gestiona la cola de reserva del sistema y reasigna los cupos liberados por **ausencias**, notificando oportunidades de adelanto a los pacientes. Se clasifica como *Core* con rol *Execution Context*, y aplica una **cadena de reasignación** en la que el candidato que acepta libera su slot original, el cual se re-ofrece al siguiente paciente de la cola.



Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

Bounded Context Canvas – Arrival & QR Check-in

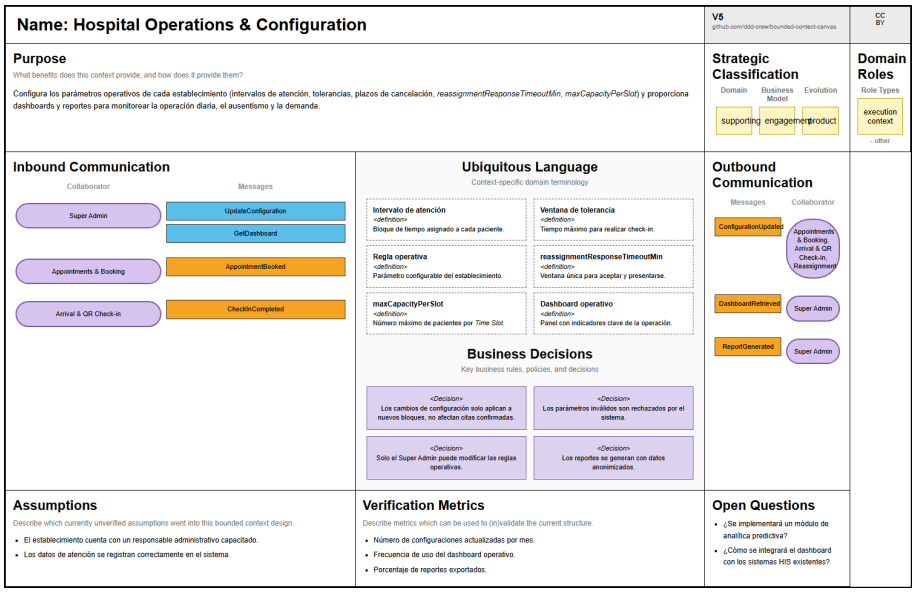
El bounded context **Arrival & QR Check-in** valida la presencia presencial del paciente mediante el escaneo de un código QR, gestiona la `Attendance Queue` (cola virtual ordenada por `checkInTimestamp`), emite el ticket digital y declara la ausencia del paciente cuando excede la tolerancia configurada. Se clasifica como *Core* con rol *Execution Context*.



Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

Bounded Context Canvas – Hospital Operations & Configuration

El bounded context **Hospital Operations & Configuration** configura los parámetros operativos de cada establecimiento de salud (intervalos de atención, tolerancias, plazos de cancelación, `reassignmentResponseTimeoutMin`, `maxCapacityPerSlot`) y proporciona dashboards y reportes para monitorear la operación diaria, el ausentismo y la demanda. Se clasifica como *Supporting* con rol *Execution Context*.



Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

Los canvases presentados permiten visualizar de forma estructurada las responsabilidades, reglas de negocio, lenguaje común y métricas de verificación de cada bounded context, asegurando un entendimiento compartido entre los miembros del equipo y facilitando la comunicación con los stakeholders del dominio de SaludYa. Asimismo, se evidencia que las dos colas complementarias del dominio quedan correctamente ubicadas: la **cola por pedido de cita** como atributo (`Booking Order`) dentro de `Appointments & Booking` , y la **cola de asistencia** como agregado (`Attendance Queue`) dentro de `Arrival & QR Check-in` . Esta separación permite que cada contexto mantenga su propio lenguaje ubicuo y sus reglas de negocio sin acoplarse innecesariamente al resto del sistema.

2.5.2. Context Mapping

El proceso de Context Mapping nos permitió analizar y definir las relaciones estructurales y los patrones de integración entre los bounded contexts del sistema SaludYa. Este análisis fue clave para garantizar una comunicación clara, minimizar dependencias innecesarias y mantener una alta cohesión interna dentro de cada contexto.

Análisis del proceso

Durante la elaboración de los context maps, el equipo se planteó las siguientes preguntas para refinar los límites y relaciones entre los bounded contexts:

- **¿Qué pasaría si movemos ciertas capacidades del Identity & Access Management al Appointments & Booking?** Se descartó esta opción, ya que se perdería la independencia del módulo de autenticación y se generaría una sobrecarga innecesaria en el proceso de reserva. La autenticación debe ser un servicio transversal consumido por el resto de contextos.

- **¿Qué pasaría si descomponemos el Appointments & Booking en subcontextos separados para reservas y cancelaciones?** Se concluyó que no era necesario, ya que ambos flujos comparten las mismas reglas de negocio, aggregates y políticas de agenda. Descomponerlo generaría duplicidad y aumentaría la complejidad de coordinación.
- **¿Qué pasaría si el Arrival & QR Check-in dependiera directamente del Appointments & Booking?** Se determinó mantener la dependencia mediante **eventos de dominio** (`Cita reservada` , `Cita cancelada`) para evitar acoplamientos fuertes y permitir que el check-in funcione de manera desacoplada.
- **¿Qué pasaría si creamos un servicio compartido para la gestión de notificaciones?** Se decidió implementar una comunicación **event-driven** con un **Shared Kernel** (plantillas, canales y prioridades compartidas), dado que varios contextos requieren enviar mensajes al usuario final: Identity & Access Management (confirmaciones de cuenta), Appointments & Booking (confirmación de reservas), Reassignment (propuestas de adelanto) y Arrival & QR Check-in (ticket emitido).
- **¿Qué pasaría si aislamos el core de reservas y movemos la configuración a un contexto aparte?** Se confirmó la separación actual: **Hospital Operations & Configuration** funciona como contexto de soporte que parametriza el comportamiento de los contextos core sin acoplarse a su lógica interna.

Patrones de integración aplicados

A partir del análisis, se definieron los siguientes patrones de relación entre los bounded contexts de SaludYa, como se detalla en la Tabla 24:

Tabla 24

Context Mapping: patrones y relaciones entre contextos

Bounded Context origen	Bounded Context destino	Patrón	Justificación
Identity & Access Management	RENIEC API (externo)	**ACL**	Se traduce el modelo externo de RENIEC al modelo interno de identidad.
Identity & Access Management	Appointments & Booking	**Shared Kernel**	Comparten el modelo de identidad y sesión activa del paciente.
Identity & Access Management	Arrival & QR Check-in	**Shared Kernel**	Comparten la validación de sesión para el check-in.
Identity & Access Management	Hospital Operations & Configuration	**Shared Kernel**	Comparten el modelo de roles para autorizar al Super Admin.
Arrival & QR Check-in	Reassignment	**Customer/Supplier**	Check-in publica el evento de ausencia (`PatientAbsentEvent`) que Reassignment consume.
Hospital Operations & Configuration	Appointments & Booking	**Conformist**	Booking adopta el modelo de parámetros operativos sin traducirlo.

Bounded Context origen	Bounded Context destino	Patrón	Justificación
Hospital Operations & Configuration	Arrival & QR Check-in	**Conformist**	Check-in adopta el modelo de tolerancias sin traducirlo.

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

Mensajes intercambiados entre bounded contexts

A continuación se detallan los mensajes que se intercambian entre los bounded contexts, reflejando la existencia de las dos colas complementarias del dominio, como se detalla en la Tabla 25:

Tabla 25

Context Mapping: mensajes entre contextos

Mensaje	Bounded Context origen	Bounded Context destino	Contenido
`AppointmentBooked`	Appointments & Booking	Arrival & QR Check-in	Incluye `bookingOrder` de la cita reservada.
`PatientAbsentEvent`	Arrival & QR Check-in	Reassignment	Evento de ausencia presencial que dispara la reasignación.
`CheckInCompleted`	Arrival & QR Check-in	Appointments & Booking, Patient	Incluye `attendanceQueueId` y `position` del paciente en la cola de asistencia.
`ReassignmentOfferSent`	Reassignment	Patient	Propuesta de cupo liberado enviada al paciente con menor `bookingOrder`.
`ReassignmentOfferAccepted`	Reassignment	Appointments & Booking, Patient	Confirmación de aceptación del cupo liberado (mueve la cita y libera el slot original).
`ReassignmentOfferRejected`	Reassignment	Patient	Rechazo del cupo liberado; se ofrece al siguiente candidato.
`ReassignmentOfferExpired`	Reassignment	Patient	Expiración de la ventana única sin respuesta del paciente.
`ReassignmentOfferAttended`	Reassignment	Appointments & Booking	El candidato aceptó y llegó; cierra la oferta.
`ReassignmentOfferNoShow`	Reassignment	Appointments & Booking	El candidato aceptó y no llegó; marca su cita como ausente.

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

Leyenda de patrones:

- **ACL (Anticorruption Layer):** Capa de traducción entre el modelo externo y el modelo interno.
- **SK (Shared Kernel):** Modelo compartido entre dos o más contextos.
- **CF (Conformist):** Un contexto adopta el modelo de otro sin traducirlo.
- **C/S (Customer/Supplier):** Relación donde el supplier publica y el customer consume.

El mapa de contextos relaciona las áreas del dominio y sus dependencias para explicar cómo colaboran los componentes de SaludYa, como se observa en la Figura 41.

Figura 41

Mapa de relaciones entre bounded contexts de SaludYa

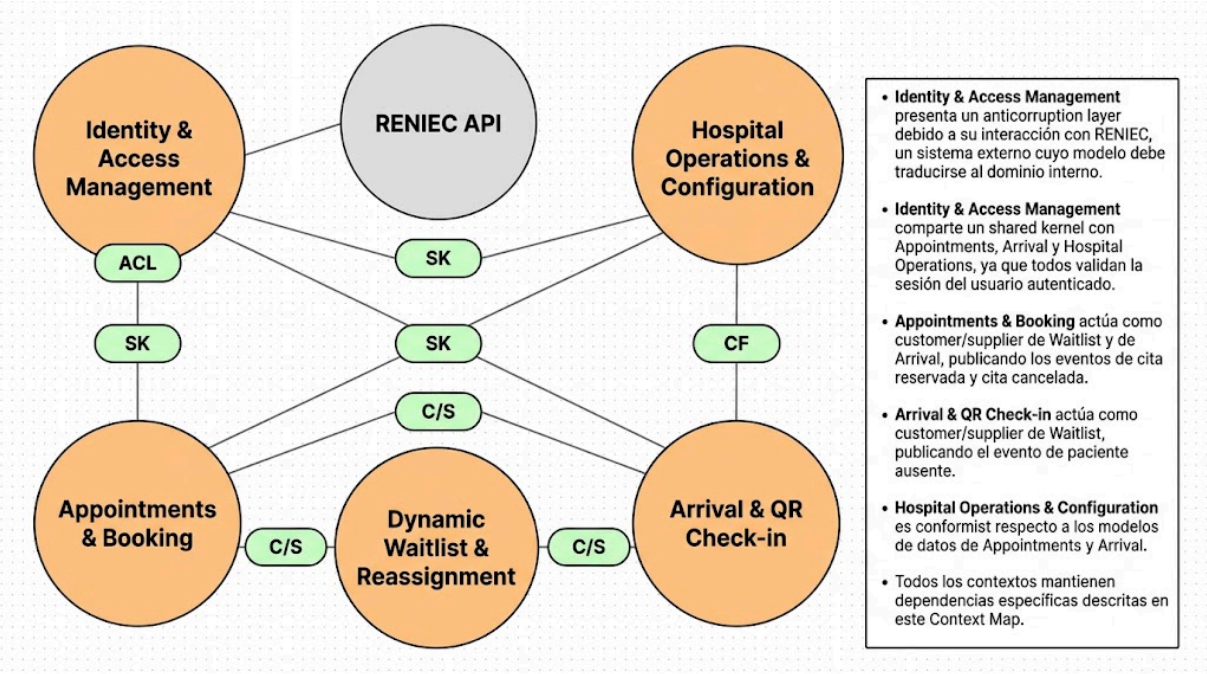


Figura 7: Context Map

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

2.5.3. Software Architecture

2.5.3.1. Software Architecture Context Level Diagrams

En el Software Architecture Context Diagram se pueden apreciar los componentes más importantes que interactúan con el sistema SaludYa, así como los usuarios principales y las funciones que desempeñan dentro del ecosistema de gestión de citas médicas en establecimientos públicos de salud.

El sistema SaludYa interactúa con tres tipos de usuarios principales: los **pacientes** que reservan y gestionan citas médicas, el **personal de admisión** que controla el flujo de atención en el establecimiento, y el **Super Admin** que configura los parámetros operativos del sistema. Asimismo, el sistema se integra con cuatro servicios externos: **RENIEC API** para la validación de identidad por DNI, **Firebase Cloud Messaging** para el envío de notificaciones push, un **Servicio de Correo** para notificaciones transaccionales, y una **Pasarela SMS** para el envío de mensajes de texto, como se observa en la Figura 42.

Figura 42

Diagrama C4 de contexto de SaludYa

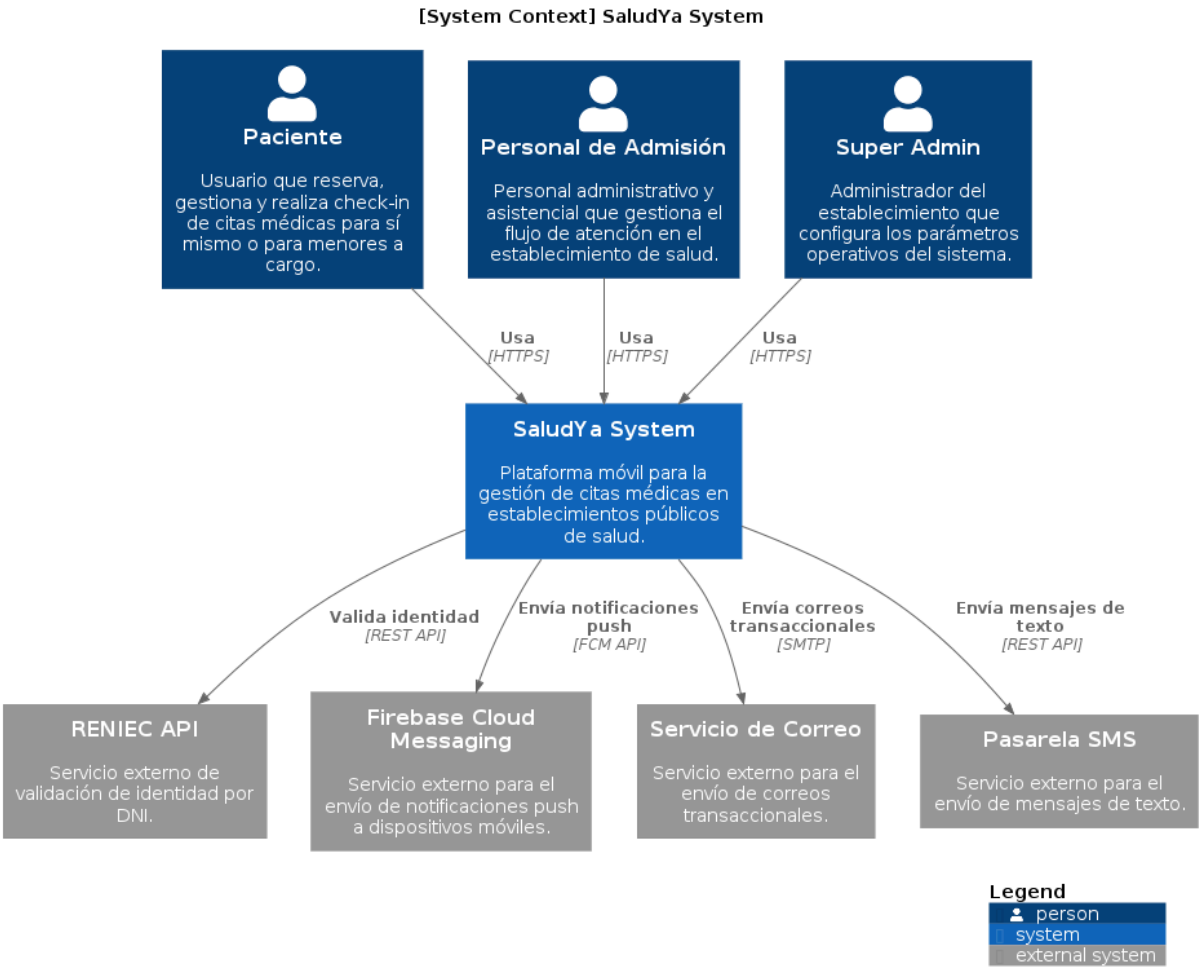


Figura 8: ContextSys

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

2.5.3.2. Software Architecture Container Level Diagrams

En el Software Architecture Container Diagram se detalla la estructura interna del sistema SaludYa, mostrando los contenedores principales que lo componen y cómo se comunican entre sí. Este nivel de abstracción permite visualizar las decisiones tecnológicas y la distribución de responsabilidades dentro del sistema.

El sistema SaludYa está compuesto por dos aplicaciones móviles (una para pacientes y otra para el personal de admisión), un API Gateway que centraliza las peticiones, un Backend API que orquesta la lógica de negocio de los bounded contexts, una base de datos PostgreSQL para la persistencia, un Message Broker RabbitMQ para la comunicación asíncrona entre contextos, y un worker de Cron Jobs que ejecuta tareas programadas como la expiración de tolerancias, la reasignación de cupos y la expiración de propuestas de la cola de reserva.

El Backend API orquesta internamente la lógica de los cinco bounded contexts del sistema, incluyendo la gestión de la cola de asistencia ordenada por `checkInTimestamp` y la cola de reserva ordenada por `bookingOrder`. La base de datos PostgreSQL incorpora las tablas `attendance_queue_entries` y

`reassignment_offers`, y la tabla `appointments` incluye el campo `booking_order` que determina la prioridad de reasignación.

El Backend API se integra con cuatro servicios externos: **RENIEC API** para la validación de identidad por DNI, **Firebase Cloud Messaging** para el envío de notificaciones push, un **Servicio de Correo** para notificaciones transaccionales, y una **Pasarela SMS** para el envío de mensajes de texto a los pacientes que no cuentan con smartphone, como se observa en la Figura 43.

Figura 43

Diagrama C4 de contenedores de SaludYa

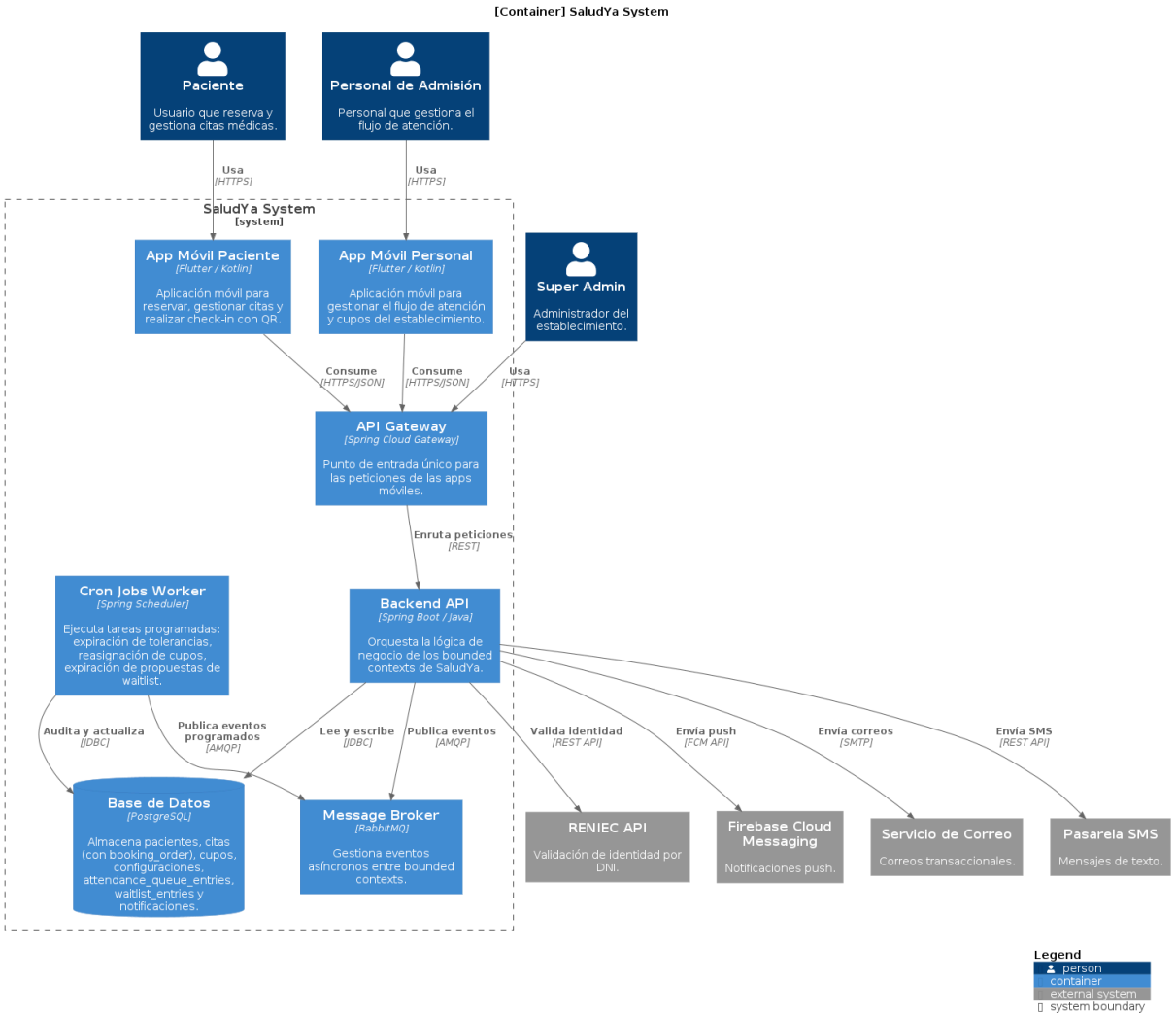


Figura 9: ContainerSys

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

2.5.3.3. Software Architecture Deployment Diagrams

En el Software Architecture Deployment Diagram se muestra la distribución física de los contenedores del sistema SaludYa sobre la infraestructura tecnológica que los aloja. Este diagrama permite visualizar cómo se despliegan las aplicaciones móviles, los servicios del backend y los componentes de infraestructura en los diferentes nodos del sistema.

El despliegue de SaludYa se distribuye en tres entornos principales. En primer lugar, las **aplicaciones móviles** se ejecutan directamente en los dispositivos de los usuarios: la app del paciente en smartphones

Android/iOS, y la app del personal de admisión en smartphones o tablets del establecimiento. En segundo lugar, el **backend del sistema** se despliega en una infraestructura cloud (AWS o GCP) compuesta por un servidor de aplicaciones con contenedores Docker que alojan el API Gateway, el Backend API y los Cron Jobs y un servidor de base de datos PostgreSQL. Finalmente, el sistema se integra con **servicios externos** como RENIEC API para la validación de identidad, Firebase Cloud Messaging para notificaciones push, el servicio de correo SMTP para notificaciones transaccionales, y una pasarela SMS.

Esta arquitectura de despliegue permite escalar horizontalmente los servicios del backend según la demanda, mantener la comunicación asíncrona entre bounded contexts mediante el message broker, y garantizar la disponibilidad de los servicios críticos mediante la infraestructura cloud, como se observa en la Figura 44.

Figura 44

Diagrama C4 de despliegue de SaludYa

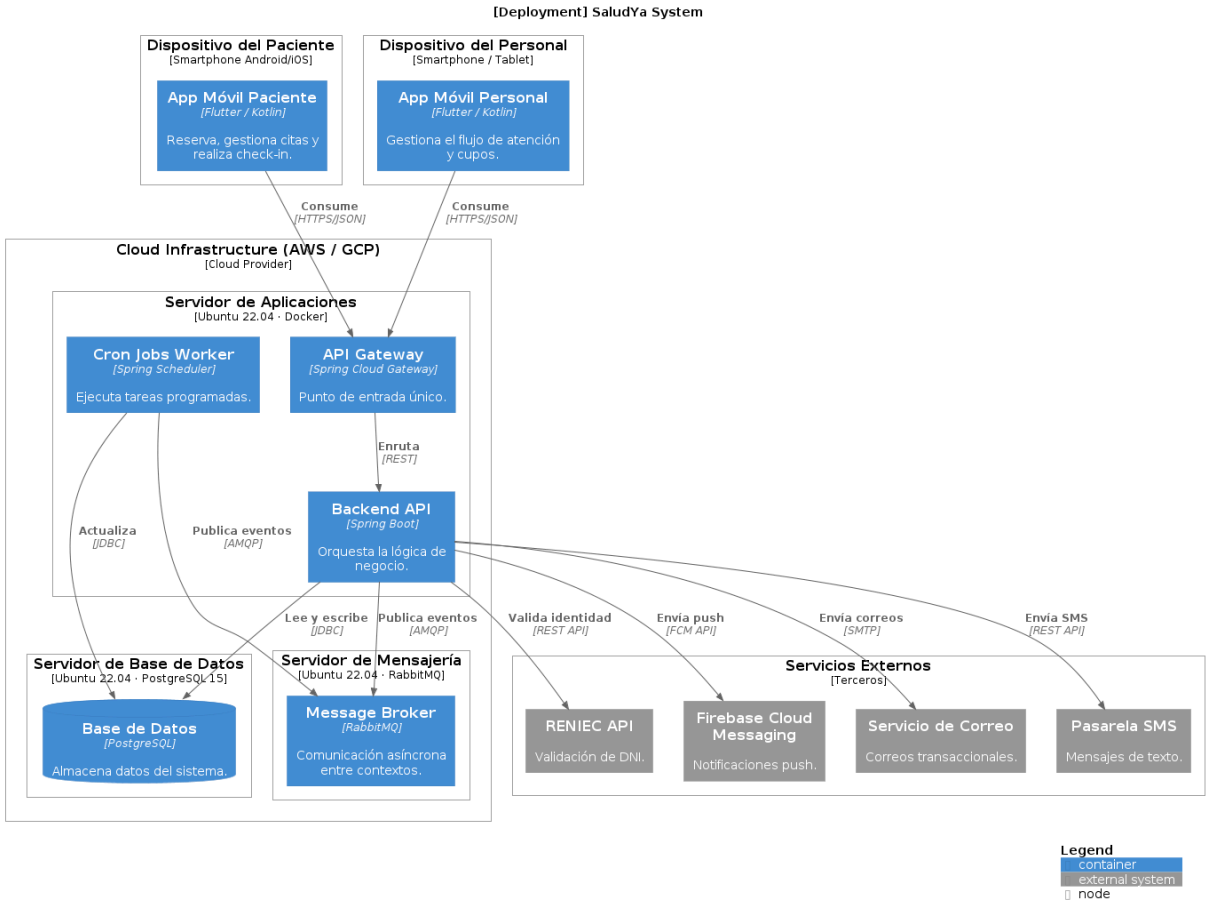


Figura 10: DeploymentSys

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

2.6. Tactical-Level Domain-Driven Design

2.6.1. Bounded Context: Identity & Access Management

El **bounded context de Identity & Access Management** gestiona el registro, autenticación, vinculación de menores y gestión de roles de los usuarios en SaludYa. Permite validar la identidad de los pacientes y del personal administrativo antes de acceder a los módulos principales del sistema.

2.6.1.1. Domain Layer

La capa de **Domain** representa el núcleo del negocio de identidad. Aquí se definen las entidades, value objects, enums, aggregates, factories e interfaces que encapsulan las reglas de negocio.

UserAccount (Aggregate Root)
Atributos: `id` , `email: Email` , `password: PasswordHash` , `role: Role` , `isActive` , `createdAt`

Métodos:

- `validatePassword(password)` → verifica si la contraseña ingresada coincide con la almacenada.
- `assignRole(role)` → asigna un rol al usuario.
- `activate()` / `deactivate()` → controlan si el usuario puede iniciar sesión.

Propósito: Representa la cuenta de acceso al sistema. Es aggregate root porque agrupa la lógica de autenticación y estado del usuario.

Patient (Aggregate Root)
Atributos: `id` , `idUser` , `dni: Dni` , `name` , `lastname` , `birthDate` , `phone`

Métodos:

- `updateContactInfo(phone, email)` → actualiza los datos de contacto.
- `isMinor()` → retorna `true` si el paciente es menor de edad.

Propósito: Representa el perfil del paciente. Es aggregate root porque agrupa la información personal del paciente.

PatientMinor (Entity)
Atributos: `id` , `idPatient` , `idTutor`

Métodos:

- `validateTutor(tutorId)` → valida que el tutor sea un adulto responsable registrado.

Propósito: Vincula a un menor de edad con un adulto responsable (tutor).

Email (Value Object)
Atributos: `value`

Métodos:

- `isValid()` → valida el formato del correo.

Propósito: Encapsula el correo electrónico como value object inmutable.

Dni (Value Object)
Atributos: `value`

Métodos:

- `isValid()` → valida que el DNI tenga 8 dígitos.

Propósito: Encapsula el DNI como value object inmutable.

PasswordHash (Value Object)
Atributos: `value`

Propósito: Encapsula el hash de la contraseña como value object inmutable.

Role (Enum)
Valores posibles: `PATIENT`, `ADMISSION_STAFF`, `SUPER_ADMIN`

Propósito: Define los tipos de usuario del sistema.

UserAccountFactory (Factory)
Métodos:

- `createPatientAccount(email, password, dni, name, lastname, birthDate, phone): UserAccount`
- `createStaffAccount(email, password, dni, name, lastname): UserAccount`

Propósito: Encapsula la creación de cuentas de usuario, validando los datos y asignando el rol correspondiente.

UserAccountRepository (Interface)
Métodos:

- `save(userAccount: UserAccount): UserAccount`
- `findById(id: Int): UserAccount?`
- `findByEmail(email: String): UserAccount?`
- `updateStatus(id: Int, isActive: Boolean)`

Propósito: Define las operaciones de persistencia para cuentas de usuario.

PatientRepository (Interface)
Métodos:

- `save(patient: Patient): Patient`
- `findById(id: Int): Patient?`
- `findByDni(dni: String): Patient?`
- `findByUserId(userId: Int): Patient?`
- `saveMinor(patientMinor: PatientMinor): PatientMinor`
- `findMinorsByTutor(tutorId: Int): List<PatientMinor>`

Propósito: Define las operaciones de persistencia para pacientes y menores vinculados.

EventPublisher (Interface)
Métodos:

- `publish(event: DomainEvent)`

Propósito: Define la interfaz para publicar eventos de dominio. La implementación concreta usa Spring Events.

2.6.1.2. Interface Layer
La **Interface Layer** expone las funcionalidades del bounded context mediante endpoints REST.

UserAccountsController (REST API Controller)
Endpoints:

- `POST /api/v1/user-accounts` → Registra un nuevo paciente con verificación de DNI.
- `POST /api/v1/user-accounts/login` → Inicia sesión y genera token JWT.
- `POST /api/v1/user-accounts/logout` → Cierra sesión.

- `POST /api/v1/user-accounts/recover-password` → Solicita recuperación de contraseña.
- `GET /api/v1/user-accounts/{id}` → Obtiene el perfil del usuario autenticado.
- `PUT /api/v1/user-accounts/{id}` → Actualiza datos de contacto.
- `POST /api/v1/user-accounts/staff` → Crea cuenta de personal administrativo.

Explicación: Este controlador gestiona las operaciones sobre el aggregate `UserAccount`.

PatientsController (REST API Controller)
Endpoints:

- `GET /api/v1/patients/{id}` → Obtiene el perfil de un paciente.
- `PUT /api/v1/patients/{id}` → Actualiza datos del paciente.
- `GET /api/v1/patients/{id}/minors` → Lista los menores vinculados al tutor.

Explicación: Este controlador gestiona las operaciones sobre el aggregate `Patient`.

PatientMinorsController (REST API Controller)
Endpoints:

- `POST /api/v1/patient-minors` → Vincula un menor a la cuenta del titular.
- `GET /api/v1/patient-minors/{id}` → Obtiene el detalle del vínculo.
- `DELETE /api/v1/patient-minors/{id}` → Desvincula un menor.

Explicación: Este controlador gestiona las operaciones sobre la entity `PatientMinor`.

IamContextFacade (Facade / ACL)
Métodos:

- `getUserById(id): UserAccount?`
- `getPatientById(id): Patient?`
- `getMinorsByTutor(tutorId): List<PatientMinor>`

Propósito: Punto de entrada interno para otros bounded contexts. Evita que otros contextos accedan directamente a los repositorios de IAM.

2.6.1.3. Application Layer
La **Application Layer** orquesta los casos de uso del dominio mediante **Command Services** y **Query Services**. Los **Command Handlers** viven dentro de los Command Services, y los **Event Handlers** en `application/internal/eventhandlers/`.

UserAccountCommandService (Interface)
Métodos (Command Handlers):

- `registerPatient(command: RegisterPatientCommand): Patient`
- `login(command: LoginCommand): AuthResult`
- `logout(command: LogoutCommand): void`
- `recoverPassword(command: RecoverPasswordCommand): void`
- `updateProfile(command: UpdateProfileCommand): Patient`
- `createStaffAccount(command: CreateStaffAccountCommand): UserAccount`

Propósito: Define los comandos relacionados con la cuenta de usuario.

PatientCommandService (Interface)
Métodos (Command Handlers):

- `linkMinor(command: LinkMinorCommand): PatientMinor`
- `unlinkMinor(command: UnlinkMinorCommand): void`

Propósito: Define los comandos relacionados con el paciente y sus menores vinculados.

UserAccountQueryService (Interface)
Métodos (Query Handlers):

- `getById(id: Int): UserAccount?`
- `getByEmail(email: String): UserAccount?`

Propósito: Define las consultas relacionadas con la cuenta de usuario.

PatientQueryService (Interface)
Métodos (Query Handlers):

- `getById(id: Int): Patient?`
- `getByDni(dni: String): Patient?`
- `getMinorsByTutor(tutorId: Int): List<PatientMinor>`

Propósito: Define las consultas relacionadas con el paciente.

UserAccountCommandServiceImpl (Implementation)
Responsabilidad: Implementar los comandos de cuenta de usuario.

Flujo de `registerPatient`:

1. Recibe los datos del paciente.
2. Valida el DNI contra el servicio externo RENIEC mediante `ReniecService` (ACL).
3. Valida que el correo no esté duplicado.
4. Crea la cuenta de usuario con rol `PATIENT` usando `UserAccountFactory`.
5. Crea el perfil de paciente.
6. Publica el evento `PatientRegisteredEvent`.
7. Notifica al paciente vía `NotificationAdapter`.

PatientCommandServiceImpl (Implementation)
Responsabilidad: Implementar los comandos de paciente.

Flujo de `linkMinor`:

1. Recibe el DNI del menor.
2. Valida el DNI contra RENIEC.
3. Valida que el menor no esté vinculado a otra cuenta.
4. Crea el `PatientMinor` asociando al tutor.
5. Publica el evento `MinorLinkedEvent`.
6. Notifica al tutor.

PatientRegisteredEventHandler (Event Handler)
Responsabilidad: Reaccionar al evento `PatientRegisteredEvent`. **Flujo:**

1. Escucha el evento.
2. Envía el correo de bienvenida.

---

3. Registra la acción en el log de auditoría.

MinorLinkedEventHandler (Event Handler)
Responsabilidad: Reaccionar al evento `MinorLinkedEvent` . **Flujo:**

1. Escucha el evento.
2. Notifica al tutor que la vinculación fue exitosa.
3. Registra la acción en el log de auditoría.

2.6.1.4. Infrastructure Layer
La capa de **Infrastructure** contiene las implementaciones concretas.

UserAccountRepositoryImpl
Implementa: `UserAccountRepository` **Tecnología:** Spring Data JPA + PostgreSQL **Explicación:** Ejecuta operaciones sobre la tabla `users` . Mapea entidades del dominio a entidades JPA.

PatientRepositoryImpl
Implementa: `PatientRepository` **Tecnología:** Spring Data JPA + PostgreSQL **Explicación:** Ejecuta operaciones sobre las tablas `patients` y `patient_minors` .

ReniecService (ACL)
Función: Valida la identidad por DNI contra el servicio externo de RENIEC. **Tecnología:** REST Client (RestTemplate / WebClient) **Explicación:** Implementa un Anticorruption Layer (ACL) que traduce el modelo externo de RENIEC al modelo interno de identidad.

BCryptHashingService
Función: Hashea y valida contraseñas con BCrypt. **Tecnología:** BCrypt

---

TokenServiceImpl
Función: Genera y valida tokens JWT. **Tecnología:** java-jwt / jjwt

---

WebSecurityConfiguration
Función: Configura Spring Security, filtros de autorización y pipeline de autenticación. **Tecnología:** Spring Security

---

BearerAuthorizationRequestFilter
Función: Filtra las peticiones HTTP y valida el token Bearer en cada request. **Tecnología:** Spring Security

---

NotificationAdapter
Función: Envía notificaciones al usuario (correo de bienvenida, recuperación de contraseña). **Tecnología:** SMTP + Firebase Cloud Messaging

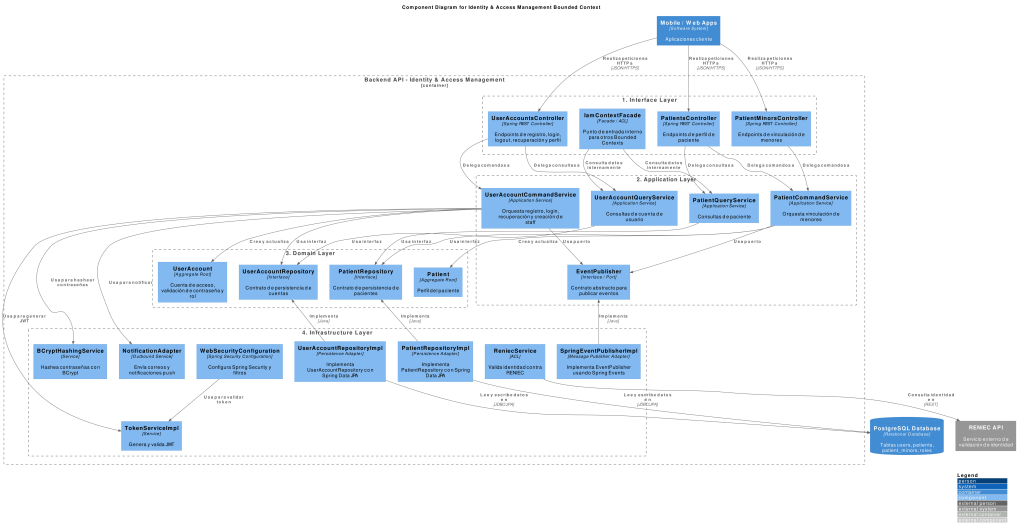
---

SpringEventPublisherImpl
Implementa: `EventPublisher` **Función:** Publica eventos de dominio usando Spring Events. **Tecnología:** `ApplicationEventPublisher` de Spring

---

2.6.1.5. Bounded Context Software Architecture Component Level Diagrams
Figura 45

Diagrama de componentes — IAM



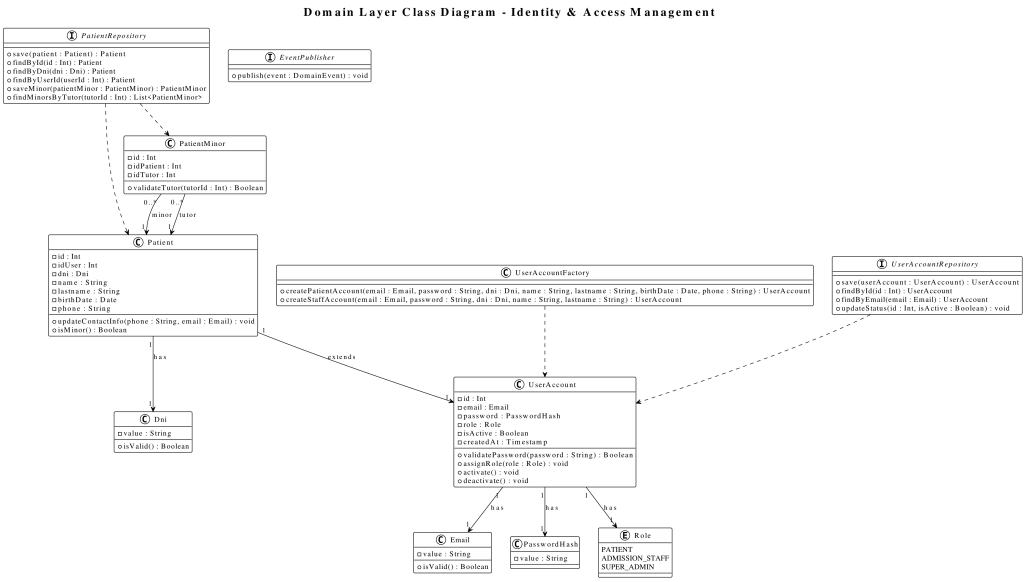
Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

El diagrama de componentes del bounded context Identity & Access Management muestra la organización interna del Backend API en sus cuatro capas: Interface, Application, Domain e Infrastructure. Se aprecian los controladores REST, los servicios de aplicación, los aggregates del dominio, las interfaces de repositorio y los adapters de infraestructura, junto con sus dependencias y la comunicación con la base de datos PostgreSQL y los servicios externos, como se observa en la Figura 45.

2.6.1.6. Bounded Context Software Architecture Code Level Diagrams

2.6.1.6.1. Bounded Context Domain Layer Class Diagrams
Figura 46

Diagrama de clases — IAM



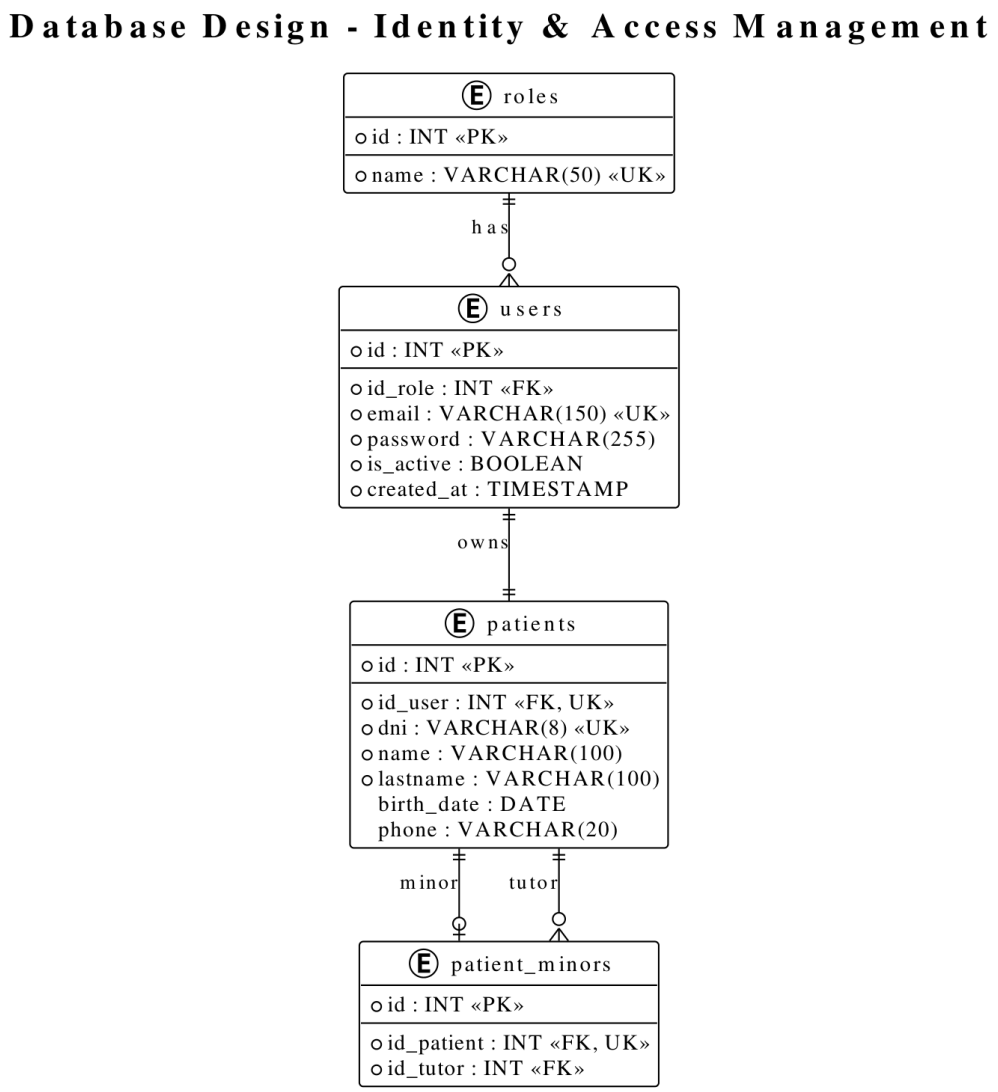
Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

El diagrama de clases del dominio del bounded context Identity & Access Management representa los aggregates, entities, value objects, enums, factory e interfaces de repositorio que encapsulan las reglas de negocio de identidad. Se muestran las relaciones entre UserAccount, Patient y PatientMinor, junto con los value objects Email, Dni y PasswordHash, el enum Role y la factory UserAccountFactory, como se observa en la Figura 46.

2.6.1.6.2. Bounded Context Database Design Diagram

Figura 47

Diagrama de base de datos — IAM



Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

El diagrama de base de datos del bounded context Identity & Access Management muestra las tablas roles, users, patients y patient_minors, junto con sus columnas, claves primarias, claves foráneas y restricciones de unicidad. Las relaciones reflejan la estructura de identidad: un rol tiene muchos usuarios, un usuario tiene un solo paciente, y un paciente puede ser tutor de muchos menores, como se observa en la Figura 47.

2.6.2. Bounded Context: Appointments & Booking

El **bounded context de Appointments & Booking** gestiona el ciclo de vida de una cita médica: consulta de disponibilidad, reserva, confirmación y cancelación. Es el responsable de la **cola de pedido de cita** (`booking_order`), que define la prioridad del paciente para asumir cupos liberados.

2.6.2.1. Domain Layer

La capa de **Domain** representa el núcleo del negocio de reserva de citas. Aquí se definen las entidades, value objects, enums, aggregates, domain events e interfaces que encapsulan las reglas de negocio.

Appointment (Aggregate Root)

Atributos: `id` , `timeSlotId` , `patientId` , `bookingOrder: BookingOrder` , `status: AppointmentStatus` , `createdAt` , `updatedAt`

Métodos:

- `create(timeSlotId, patientId, bookingOrder)` → crea una nueva cita en estado `RESERVED` (factory method estático).
- `rehydrate(id, timeSlotId, patientId, bookingOrder, status, createdAt, updatedAt)` → reconstruye la cita desde su estado persistido.
- `cancel()` → cambia el estado a `CANCELLED` y registra el evento `AppointmentCancelledEvent` .
- `markAsAbsent()` → cambia el estado a `ABSENT` y registra el evento `AppointmentAbsentEvent` .
- `markAsAttended()` → cambia el estado a `ATTENDED` .
- `confirm()` → cambia el estado de `RESERVED` a `CONFIRMED` .
- `expire()` → cambia el estado a `EXPIRED` .
- `isActive()` → retorna `true` si el estado es `RESERVED` o `CONFIRMED` .
- `canBeCancelled(scheduledAt, deadlineHours)` → valida si aún está dentro del plazo de cancelación.

Propósito: Representa una cita médica reservada. Es aggregate root porque agrupa el `bookingOrder` y controla el ciclo de vida de la cita.

TimeSlot (Aggregate Root)

Atributos: `id` , `doctorId` , `date` , `startHour` , `endHour` , `maxCapacity` , `currentBookings` , `status: TimeSlotStatus`

Métodos:

- `create(doctorId, date, startHour, endHour, maxCapacity)` → crea un bloque horario en estado `AVAILABLE` (factory method estático).
- `rehydrate(id, doctorId, date, startHour, endHour, maxCapacity, currentBookings, status)` → reconstruye el bloque desde su estado persistido.
- `hasAvailableCapacity()` → retorna `true` si `currentBookings < maxCapacity` .
- `incrementBookings()` → aumenta `currentBookings` en 1.
- `decrementBookings()` → disminuye `currentBookings` en 1.
- `updateMaxCapacity(newMaxCapacity)` → actualiza la capacidad máxima, validando que no sea menor que `currentBookings` .
- `isFull()` → retorna `true` si `currentBookings >= maxCapacity` .
- `isExpired()` → retorna `true` si la fecha y hora ya pasaron.
- `cancel()` → cambia el estado a `CANCELLED` .

Propósito: Representa un bloque horario concreto de un médico en una fecha. Es aggregate root porque controla la capacidad y el estado del bloque.

Doctor (Aggregate)
Atributos: `id` , `specialtyId` , `name` , `lastname`

Métodos:

- `create(specialtyId, name, lastname)` → crea un médico (factory method estático).
- `rehydrate(id, specialtyId, name, lastname)` → reconstruye el médico desde su estado persistido.

Propósito: Representa a un médico dentro del catálogo.

Specialty (Aggregate)
Atributos: `id` , `name` , `description`

Métodos:

- `create(name, description)` → crea una especialidad (factory method estático).
- `rehydrate(id, name, description)` → reconstruye la especialidad desde su estado persistido.

Propósito: Representa una especialidad médica dentro del catálogo.

BookingOrder (Value Object)
Atributos: `value`

Propósito: Encapsula el número secuencial de reserva. Es inmutable una vez asignado. El alcance «por especialidad» se resuelve al calcular el siguiente valor mediante `getNextBookingOrderBySpecialty` .

AppointmentStatus (Enum)
Valores posibles: `RESERVED` , `CONFIRMED` , `CANCELLED` , `ABSENT` , `ATTENDED` , `EXPIRED`

Propósito: Define los estados del ciclo de vida de una cita. Las transiciones hacia `ATTENDED` y `ABSENT` son disparadas por el bounded context `Arrival & QR Check-in` , y `EXPIRED` por un job programado.

TimeSlotStatus (Enum)
Valores posibles: `AVAILABLE` , `FULL` , `CANCELLED`

Propósito: Define el estado de disponibilidad de un bloque horario.

AppointmentRepository (Interface)
Métodos:

- `save(appointment: Appointment): Appointment`
- `findById(id: Long): Appointment?`
- `findByPatientId(patientId: Long): List<Appointment>`
- `findByTimeSlotId(timeSlotId: Long): List<Appointment>`
- `findActiveBySpecialtyId(specialtyId: Long): List<Appointment>`
- `getNextBookingOrderBySpecialty(specialtyId: Long): int`
- `existsActiveByPatientAndTimeSlot(patientId: Long, timeSlotId: Long): Boolean`
- `updateStatus(id: Long, status: AppointmentStatus)`
- `existsById(id: Long): Boolean`
- `deleteById(id: Long)`

Propósito: Define las operaciones de persistencia para citas.

TimeSlotRepository (Interface)
Métodos:

- `save(timeSlot: TimeSlot): TimeSlot`
- `findById(id: Long): TimeSlot?`
- `findAvailableBySpecialty(specialtyId: Long, date: LocalDate): List<TimeSlot>`
- `findByDoctorAndDate(doctorId: Long, date: LocalDate): List<TimeSlot>`
- `existsById(id: Long): Boolean`

Propósito: Define las operaciones de persistencia para bloques horarios.

SpecialtyRepository (Interface)
Métodos:

- `save(specialty: Specialty): Specialty`
- `findById(id: Long): Specialty?`
- `findAll(): List<Specialty>`
- `existsById(id: Long): Boolean`

Propósito: Define las operaciones de persistencia para el catálogo de especialidades.

DoctorRepository (Interface)
Métodos:

- `save(doctor: Doctor): Doctor`
- `findById(id: Long): Doctor?`
- `findBySpecialtyId(specialtyId: Long): List<Doctor>`
- `findAll(): List<Doctor>`
- `existsById(id: Long): Boolean`

Propósito: Define las operaciones de persistencia para el catálogo de médicos.

> **Nota sobre eventos de dominio:** `AppointmentBookedEvent`, `AppointmentCancelledEvent` y `AppointmentAbsentEvent` son registrados por el aggregate `Appointment` mediante `registerDomainEvent(...)` y publicados por el persistence adapter al guardar, usando `ApplicationEventPublisher` de Spring.

AppointmentBookedEvent (Domain Event)
Atributos: `appointmentId`, `timeSlotId`, `patientId`, `bookingOrder`, `bookedAt`

Propósito: Representa el hecho de negocio de que se reservó una nueva cita.

AppointmentCancelledEvent (Domain Event)
Atributos: `appointmentId`, `freedTimeSlotId`, `cancelledAt`

Propósito: Representa el hecho de negocio de que una cita fue cancelada y su cupo quedó libre.

AppointmentAbsentEvent (Domain Event)
Atributos: `appointmentId`, `freedTimeSlotId`, `absentAt`

Propósito: Representa el hecho de negocio de que un paciente no se presentó a su cita.

2.6.2.2. Interface Layer

La **Interface Layer** expone las funcionalidades del bounded context mediante endpoints REST.

AppointmentsController (REST API Controller)

Endpoints:

- `POST /api/v1/appointments` → Reserva una nueva cita.
- `GET /api/v1/appointments/{id}` → Obtiene el detalle de una cita.
- `GET /api/v1/appointments/patient/{patientId}` → Lista citas de un paciente.
- `DELETE /api/v1/appointments/{id}` → Cancela una cita.

Explicación: Este controlador gestiona las operaciones sobre el aggregate `Appointment`.

TimeSlotsController (REST API Controller)

Endpoints:

- `GET /api/v1/time-slots/available?specialtyId={id}&date={date}` → Lista bloques horarios disponibles de una especialidad.
- `GET /api/v1/time-slots?doctorId={id}&date={date}` → Lista bloques horarios de un médico en una fecha.
- `GET /api/v1/time-slots/{id}` → Obtiene el detalle de un bloque horario.
- `POST /api/v1/time-slots` → Crea un bloque horario.
- `PUT /api/v1/time-slots/{id}/capacity` → Actualiza la capacidad de un bloque horario.

Explicación: Este controlador gestiona las operaciones sobre el aggregate `TimeSlot`.

SpecialtiesController (REST API Controller)

Endpoints:

- `GET /api/v1/specialties` → Lista especialidades disponibles.
- `GET /api/v1/specialties/{id}` → Obtiene el detalle de una especialidad.

Explicación: Este controlador gestiona las operaciones sobre el aggregate `Specialty`.

DoctorsController (REST API Controller)

Endpoints:

- `GET /api/v1/doctors?specialtyId={id}` → Lista médicos de una especialidad.
- `GET /api/v1/doctors/{id}` → Obtiene el detalle de un médico.

Explicación: Este controlador gestiona las operaciones sobre el aggregate `Doctor`.

2.6.2.3. Application Layer

La **Application Layer** orquesta los casos de uso del dominio mediante **Command Services** y **Query Services**. Los command services retornan `Result<T, ApplicationError>` (patrón Result del módulo `shared`).

AppointmentCommandService (Interface)

Métodos (Command Handlers):

- `book(command: BookAppointmentCommand): Result<Appointment, ApplicationError>`
- `cancel(command: CancelAppointmentCommand): Result<Appointment, ApplicationError>`

Propósito: Define los comandos relacionados con la cita.

TimeSlotCommandService (Interface)
Métodos (Command Handlers):

- `create(command: CreateTimeSlotCommand): Result<TimeSlot, ApplicationError>`
- `updateCapacity(command: UpdateTimeSlotCapacityCommand): Result<TimeSlot, ApplicationError>`

Propósito: Define los comandos relacionados con el bloque horario.

AppointmentQueryService (Interface)
Métodos (Query Handlers):

- `getById(id: Long): Appointment?`
- `getByPatient(patientId: Long): List<Appointment>`
- `getByTimeSlot(timeSlotId: Long): List<Appointment>`

Propósito: Define las consultas relacionadas con la cita.

TimeSlotQueryService (Interface)
Métodos (Query Handlers):

- `getById(id: Long): TimeSlot?`
- `getAvailableBySpecialty(specialtyId: Long, date: LocalDate): List<TimeSlot>`
- `getByDoctorAndDate(doctorId: Long, date: LocalDate): List<TimeSlot>`

Propósito: Define las consultas relacionadas con el bloque horario.

SpecialtyQueryService (Interface)
Métodos (Query Handlers):

- `getAll(): List<Specialty>`
- `getById(id: Long): Specialty?`

Propósito: Define las consultas relacionadas con el catálogo de especialidades.

DoctorQueryService (Interface)
Métodos (Query Handlers):

- `getBySpecialty(specialtyId: Long): List<Doctor>`
- `getById(id: Long): Doctor?`

Propósito: Define las consultas relacionadas con el catálogo de médicos.

AppointmentCommandServiceImpl (Implementation)
Responsabilidad: Implementar los comandos de cita.

Flujo de `book`:

1. Recibe `patientId` y `timeSlotId`.
2. Carga el `TimeSlot` y valida que tenga capacidad disponible (`hasAvailableCapacity`).
3. Valida que no haya solapamiento (`existsActiveByPatientAndTimeSlot`).
4. Carga el `Doctor` del slot para obtener la `specialtyId`.
5. Calcula el siguiente `bookingOrder` con `getNextBookingOrderBySpecialty`.
6. Crea el `Appointment` en estado `RESERVED` con `Appointment.create(...)`.
7. Incrementa `current_bookings` y guarda el `TimeSlot`.
8. Guarda el `Appointment` el adapter publica el evento `AppointmentBookedEvent`.

Flujo de `cancel`:

1. Recibe `appointmentId`.
2. Busca la cita.
3. Valida que esté activa (`isActive`).
4. Cambia el estado a `CANCELLED` con `cancel()`, que registra el evento `AppointmentCancelledEvent`.
5. Decrementa `current_bookings`.
6. Guarda el `Appointment` el adapter publica el evento `AppointmentCancelledEvent`.

> **Nota:** la validación del plazo (`cancellationDeadlineHours`) proviene del bounded context `Hospital Operations & Configuration` y se integra mediante un ACL (pendiente).

> **Nota:** la reasignación de cupos liberados **por ausencia** es responsabilidad del bounded context `Reassignment`, que escucha `PatientAbsentEvent` publicado por `Arrival & QR Check-in`. La cancelación de una cita **no** dispara reasignación (solo libera el cupo para reserva externa). Este contexto no implementa handlers propios para reasignación.

TimeSlotCommandServiceImpl (Implementation)

Responsabilidad: Implementar los comandos de bloque horario.

Flujo de `create`:

1. Recibe `doctorId`, `date`, `startHour`, `endHour` y `maxCapacity`.
2. Crea el `TimeSlot` en estado `AVAILABLE` con `TimeSlot.create(...)`.
3. Guarda el `TimeSlot`.

Flujo de `updateCapacity`:

1. Recibe `timeSlotId` y `maxCapacity`.
2. Busca el `TimeSlot`.
3. Actualiza la capacidad con `updateMaxCapacity(...)`, validando que no sea menor que `currentBookings`.
4. Guarda el `TimeSlot`.

2.6.2.4. Infrastructure Layer

La capa de **Infrastructure** contiene las implementaciones concretas.

AppointmentPersistenceAdapter

Implementa: `AppointmentRepository` **Tecnología:** Spring Data JPA + PostgreSQL **Explicación:** Ejecuta operaciones sobre la tabla `appointments`, implementa el cálculo de `booking_order` secuencial por especialidad y publica los domain events del aggregate (`AppointmentBookedEvent`, `AppointmentCancelledEvent`, `AppointmentAbsentEvent`) vía `ApplicationEventPublisher` de Spring tras persistir.

TimeSlotPersistenceAdapter

Implementa: `TimeSlotRepository` **Tecnología:** Spring Data JPA + PostgreSQL **Explicación:** Ejecuta operaciones sobre la tabla `time_slots`.

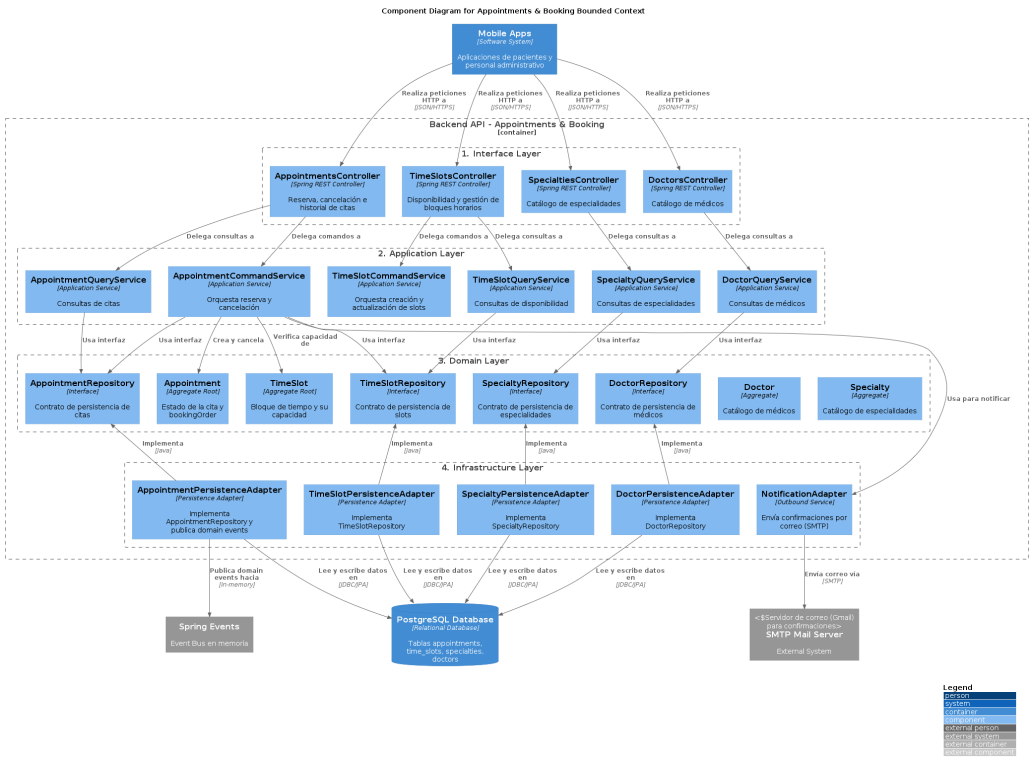
SpecialtyPersistenceAdapter
Implementa: `SpecialtyRepository` **Tecnología:** Spring Data JPA + PostgreSQL **Explicación:** Ejecuta operaciones sobre la tabla `specialties`.

DoctorPersistenceAdapter
Implementa: `DoctorRepository` **Tecnología:** Spring Data JPA + PostgreSQL **Explicación:** Ejecuta operaciones sobre la tabla `doctors`.

NotificationAdapter
Función: Envía notificaciones de confirmación y cancelación al paciente. **Tecnología:** SMTP **Explicación:** Requiere un ACL hacia el bounded context `Identity & Access Management` para obtener el correo del paciente. Su implementación está pendiente.

2.6.2.5. Bounded Context Software Architecture Component Level Diagrams
Figura 48

Diagrama de componentes — Appointment



Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

El diagrama de componentes del bounded context Appointments & Booking muestra la organización interna del Backend API en sus cuatro capas. En la Interface Layer, los controladores exponen los endpoints REST para reservar, cancelar, consultar disponibilidad, gestionar bloques horarios y explorar el catálogo médico. En la Application Layer, los Command Services y Query Services orquestan los casos de uso. En la Domain Layer, los aggregates Appointment y TimeSlot encapsulan las reglas de negocio, junto con los aggregates Doctor y Specialty y los domain events. En la Infrastructure Layer, los adapters implementan

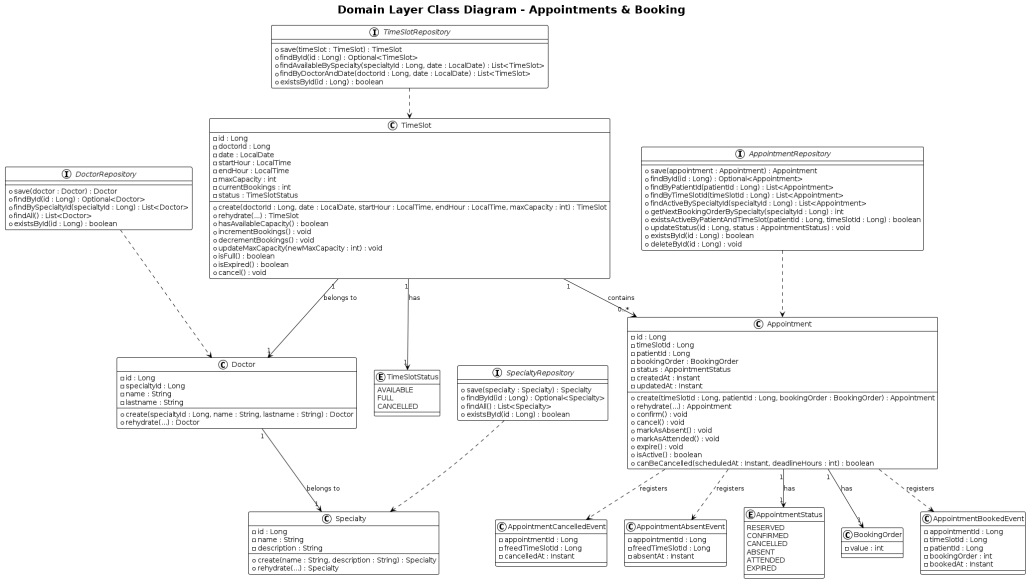
la persistencia con Spring Data JPA y la publicación de eventos de dominio con `ApplicationEventPublisher` de Spring, como se observa en la Figura 48.

2.6.2.6. Bounded Context Software Architecture Code Level Diagrams

2.6.2.6.1. Bounded Context Domain Layer Class Diagrams

Figura 49

Diagrama de clases — Appointment



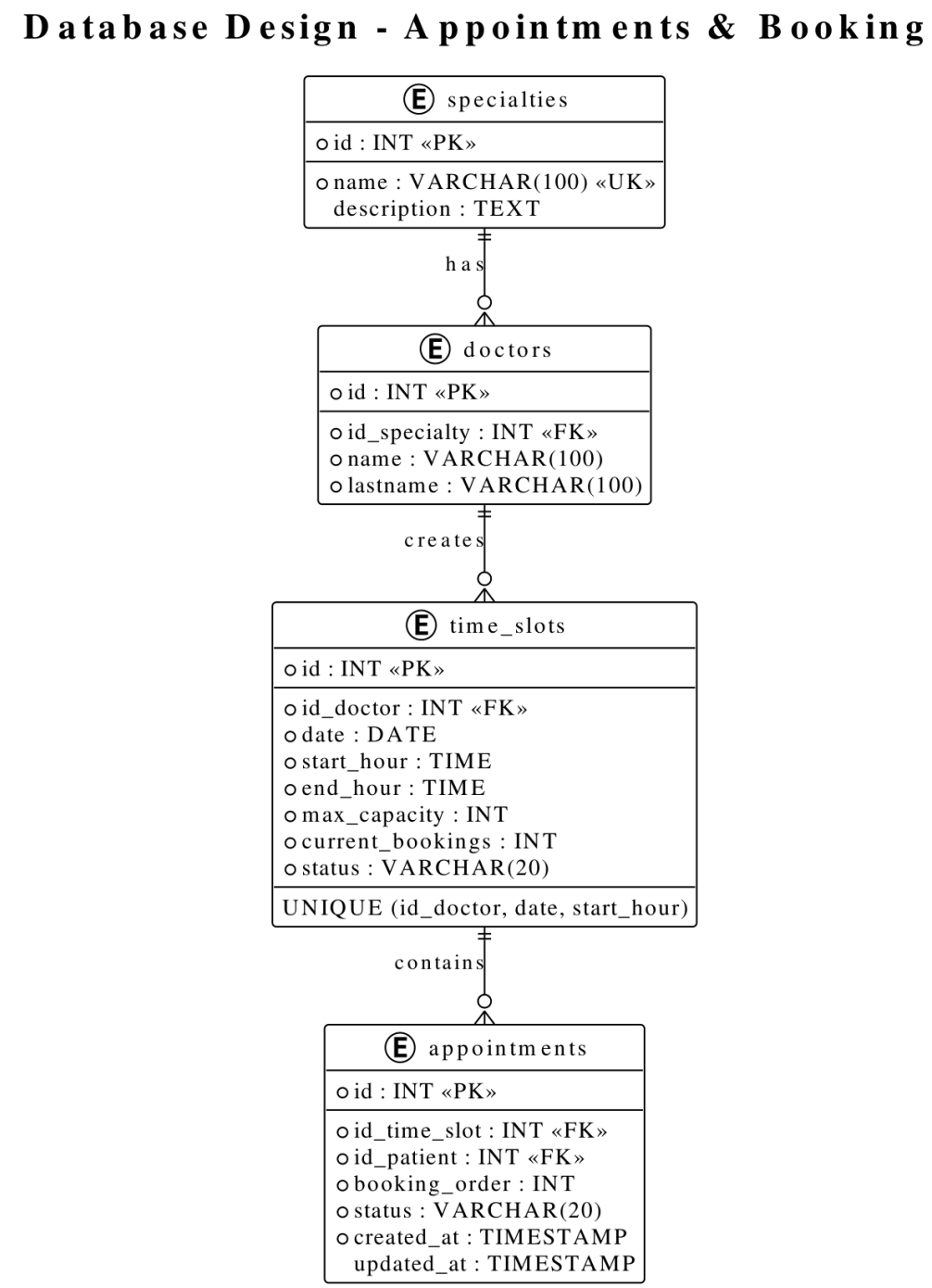
Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

El diagrama de clases del dominio del bounded context Appointments & Booking representa los aggregates, value objects, enums, domain events e interfaces de repositorio que encapsulan las reglas de negocio de reserva de citas. Se muestran las relaciones entre Appointment, TimeSlot, Doctor y Specialty, junto con el value object BookingOrder, los enums AppointmentStatus y TimeSlotStatus, los domain events y los repositorios, como se observa en la Figura 49.

2.6.2.6.2. Bounded Context Database Design Diagram

Figura 50

Diagrama de base de datos — Appointment



Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

El diagrama de base de datos del bounded context Appointments & Booking muestra las tablas specialties, doctors, time_slots y appointments, junto con sus columnas, claves primarias, claves foráneas y restricciones de unicidad. Las relaciones reflejan la estructura del catálogo médico y la reserva de citas:

una especialidad tiene muchos doctores, un doctor tiene muchos bloques horarios, y un bloque horario contiene muchas citas, como se observa en la Figura 50.

2.6.3. Bounded Context: Reassignment

El **bounded context de Reassignment** gestiona la reasignación de cupos liberados por **ausencias** presenciales. La reasignación opera sobre la **cola de reserva** (`appointments` ordenadas por `booking_order`); no existe lista de espera externa ni cascada hacia otros `Time Slot` . Cuando un paciente no se presenta y es declarado ausente, el sistema toma al siguiente paciente de la misma especialidad (por `booking_order`) y le ofrece el cupo liberado. El candidato dispone de una **ventana única** (`expiresAt`) para aceptar **y** presentarse. Si acepta y llega, se reasigna; al abandonar su slot original, este se libera y se re-ofrece al siguiente, formando una **cadena de reasignación**. Si rechaza o no responde, se pasa al siguiente candidato. Si acepta pero no llega, se declara ausente. Si nadie en la cola acepta, el cupo se cierra. La cancelación de citas **no** dispara este flujo.

2.6.3.1. Domain Layer

La capa de **Domain** representa el núcleo del negocio de reasignación. Aquí se definen el aggregate root, el enum y los domain events (records inmutables) que encapsulan las reglas de negocio.

ReassignmentOffer (Aggregate Root)
Atributos: `id` , `appointmentId` , `originalAppointmentId` , `freedTimeSlotId` , `candidateTimeSlotId` , `status: ReassignmentStatus` , `offeredAt` , `respondedAt` , `expiresAt`

Métodos:

- `offer(appointmentId, originalAppointmentId, freedTimeSlotId, candidateTimeSlotId, expiresAt)` → factory que crea una oferta `PENDING` .
- `onOffered()` → registra `ReassignmentOfferSentEvent` (tras persistir, cuando ya existe el `id`).
- `accept()` → valida `PENDING` y no expirada; cambia a `ACCEPTED` y registra `respondedAt` y `ReassignmentOfferAcceptedEvent` .
- `reject()` → valida `PENDING` cambia a `REJECTED` y registra `ReassignmentOfferRejectedEvent` .
- `expire()` → valida `PENDING` y `expiresAt < now` cambia a `EXPIRED` y registra `ReassignmentOfferExpiredEvent` .
- `markArrived()` → valida `ACCEPTED` cambia a `ATTENDED` y registra `ReassignmentOfferAttendedEvent` .
- `markNoShow()` → valida `ACCEPTED` y `expiresAt < now` cambia a `ABSENT` y registra `ReassignmentOfferNoShowEvent` .
- `isPending()` , `isAccepted()` , `isExpired()` → predicados de estado.

Propósito: Representa una oferta de reasignación enviada a un candidato de la cola de reserva. Es aggregate root porque controla el ciclo de vida completo de la oferta. Los identificadores de `Appointment` y `TimeSlot` se modelan como `Long` planos para no acoplar la persistencia entre bounded contexts.

ReassignmentStatus (Enum)
Valores posibles: `PENDING` , `ACCEPTED` , `REJECTED` , `EXPIRED` , `ATTENDED` , `ABSENT`

Propósito: Define los estados del ciclo de vida de una oferta de reasignación.

ReassignmentOfferRepository (Interface)
Métodos:

- `save(offer: ReassignmentOffer): ReassignmentOffer`
- `findById(id: Long): Optional<ReassignmentOffer>`
- `findPendingByAppointment(appointmentId: Long): List<ReassignmentOffer>`
- `findExpiredOffers(): List<ReassignmentOffer>`
- `findAcceptedOverdue(): List<ReassignmentOffer>`

Propósito: Define las operaciones de persistencia para ofertas de reasignación.

Domain Events (records)
Los domain events son **records inmutables** publicados por el persistence adapter vía `ApplicationEventPublisher` de Spring. El agregado los registra con `registerDomainEvent(...)` (heredado de `AbstractDomainAggregateRoot`).

- **ReassignmentOfferSentEvent:** `offerId`, `appointmentId`, `freedTimeSlotId`, `offeredAt` — se envió una oferta.
- **ReassignmentOfferAcceptedEvent:** `offerId`, `appointmentId`, `freedTimeSlotId`, `originalAppointmentId`, `candidateTimeSlotId`, `acceptedAt` — el candidato aceptó (lleva el snapshot para que Booking mueva la cita y libere el slot original).
- **ReassignmentOfferRejectedEvent:** `offerId`, `appointmentId`, `freedTimeSlotId`, `originalAppointmentId`, `rejectedAt` — el candidato rechazó.
- **ReassignmentOfferExpiredEvent:** `offerId`, `appointmentId`, `freedTimeSlotId`, `originalAppointmentId`, `expiredAt` — la oferta expiró sin respuesta.
- **ReassignmentOfferAttendedEvent:** `offerId`, `appointmentId`, `attendedAt` — el candidato aceptó y llegó.
- **ReassignmentOfferNoShowEvent:** `offerId`, `appointmentId`, `freedTimeSlotId`, `noShowAt` — el candidato aceptó y no llegó.

> **Nota:** la búsqueda del siguiente candidato (`findNextCandidate`) no es un domain service: se resuelve mediante un ACL hacia `Appointments & Booking`, porque consulta datos de otro bounded context.

2.6.3.2. Interface Layer
La **Interface Layer** expone las funcionalidades del bounded context mediante endpoints REST y un consumer de eventos.

ReassignmentOffersController (REST API Controller)
Endpoints:

- `GET /api/v1/reassignment-offers/pending?appointmentId={id}` → Lista ofertas pendientes de un candidato.
- `POST /api/v1/reassignment-offers/{id}/accept` → Acepta una oferta.
- `POST /api/v1/reassignment-offers/{id}/reject` → Rechaza una oferta.

Explicación: Este controlador gestiona las operaciones sobre el aggregate `ReassignmentOffer`.

PatientAbsentEventConsumer (Event Consumer)
Función: Escucha el evento `PatientAbsentEvent` publicado por `Arrival & QR Check-in` y dispara el protocolo de reasignación. **Tecnología:** `@EventListener` de Spring

---

2.6.3.3. Application Layer
La **Application Layer** orquesta los casos de uso mediante **Command Services** y **Query Services**. Los command services retornan `Result<T, ApplicationError>` (patrón Result del módulo `shared`), y los **Event Handlers** viven en `application/internal/eventhandlers/`.

ReassignmentCommandService (Interface)
Métodos (Command Handlers):

- `sendReassignmentOffer(command: SendReassignmentOfferCommand): Optional<Long>` — dispara la reasignación y devuelve el id de la oferta creada (vacío si no hay candidato).
- `acceptReassignment(command: AcceptReassignmentCommand): Result<ReassignmentOffer, ApplicationError>`
- `rejectReassignment(command: RejectReassignmentCommand): Result<ReassignmentOffer, ApplicationError>`

Propósito: Define los comandos relacionados con la reasignación.

ReassignmentQueryService (Interface)
Métodos (Query Handlers):

- `getPendingByAppointment(appointmentId: Long): List<ReassignmentOffer>`
- `getById(id: Long): Optional<ReassignmentOffer>`

Propósito: Define las consultas relacionadas con la reasignación.

AppointmentLookupService (ACL / Outbound)
Métodos:

- `findNextCandidateByBookingOrder(freedTimeSlotId): Optional<Long>` — devuelve el `appointmentId` del siguiente candidato por `bookingOrder`.
- `timeSlotOfAppointment(appointmentId): Long` — devuelve el slot actual del candidato.

Propósito: Puerto de acceso a `Appointments & Booking`. Implementación en memoria (pendiente de conectar a la facade real).

HospitalConfigurationService (ACL / Outbound)
Métodos:

- `reassignmentResponseTimeoutMinutes(): int`

Propósito: Puerto de acceso a `Hospital Operations & Configuration` para la ventana única. Implementación en memoria (pendiente).

ReassignmentCommandServiceImpl (Implementation)
Responsabilidad: Implementar los comandos de reasignación.

Flujo de `sendReassignmentOffer`:

1. Recibe `originalAppointmentId` y `freedTimeSlotId`.
2. Busca el siguiente candidato con `AppointmentLookupService.findNextCandidateByBookingOrder`.
3. Si no hay candidato, no hace nada (el cupo se cierra).

---

4. Lee la ventana única con `HospitalConfigurationService.reassignmentResponseTimeoutMinutes`.
5. Calcula `expiresAt = now + ventana` y obtiene `candidateTimeSlotId`.
6. Crea la oferta `PENDING` con `ReassignmentOffer.offer(...)` y la guarda (publica `ReassignmentOfferSentEvent`).

Flujo de `acceptReassignment`:

1. Busca la oferta; valida que esté `PENDING` y no expirada.
2. Llama `accept()` (cambia a `ACCEPTED`) y guarda (publica `ReassignmentOfferAcceptedEvent`).
3. El handler de `AcceptedEvent` libera el slot original del candidato y reinicia la cadena.

Flujo de `rejectReassignment`:

1. Busca la oferta; valida que esté `PENDING`.
2. Llama `reject()` y guarda (publica `ReassignmentOfferRejectedEvent`).

Event Handlers

- **PatientAbsentEventHandler** — escucha `PatientAbsentEvent` y dispara `sendReassignmentOffer`.
- **ReassignmentOfferAcceptedEventHandler** — escucha `AcceptedEvent` y re-ofrece el slot original del candidato (cadena).
- **ReassignmentOfferRejectedEventHandler** — escucha `RejectedEvent` y ofrece el mismo slot al siguiente candidato.
- **ReassignmentOfferExpiredEventHandler** — escucha `ExpiredEvent` y ofrece el mismo slot al siguiente candidato.
- **ReassignmentOfferNoShowEventHandler** — escucha `NoShowEvent` y cierra el cupo (Booking marcará la cita como ausente).

2.6.3.4. Infrastructure Layer

La capa de **Infrastructure** contiene las implementaciones concretas.

ReassignmentOfferRepositoryImpl

Implementa: `ReassignmentOfferRepository` **Tecnología:** Spring Data JPA + PostgreSQL **Explicación:** Ejecuta operaciones sobre la tabla `reassignment_offers` y publica los domain events del aggregate vía `ApplicationEventPublisher` de Spring tras persistir.

ReassignmentExpirationScheduler

Función: Ejecuta periódicamente dos tareas: (1) expira las ofertas `PENDING` cuya ventana venció, y (2) marca como `ABSENT` las ofertas `ACCEPTED` cuya ventana venció sin llegada. **Tecnología:** `@Scheduled` de Spring

---

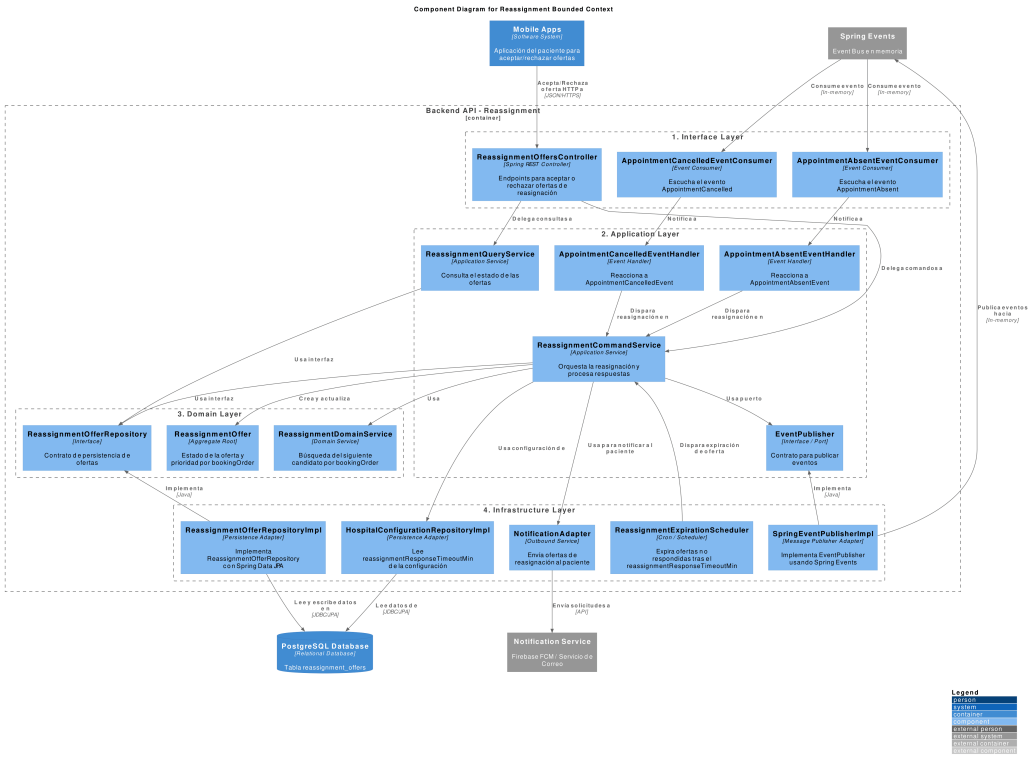
NotificationAdapter

Función: Envía notificaciones de ofertas de reasignación al paciente (pendiente de implementación; se integrará consumiendo los domain events). **Tecnología:** SMTP + Firebase Cloud Messaging

2.6.3.5. Bounded Context Software Architecture Component Level Diagrams

Figura 51

Diagrama de componentes — Reassignment



Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

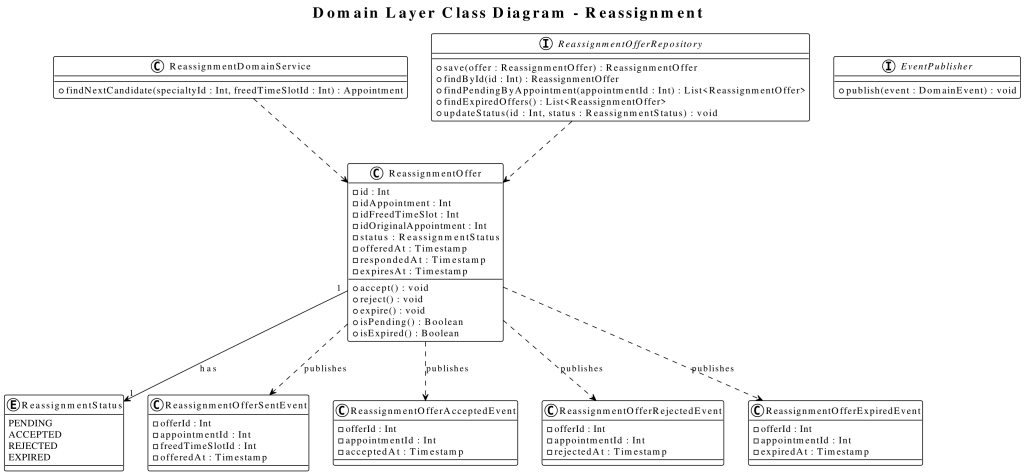
El diagrama de componentes del bounded context Reassignment muestra la organización interna del Backend API en sus cuatro capas. En la Interface Layer, el ReassignmentOffersController expone los endpoints REST para aceptar o rechazar ofertas, y el PatientAbsentEventConsumer escucha el evento de ausencia de Arrival. En la Application Layer, los Command Services y Query Services orquestan los casos de uso, junto con los Event Handlers que hacen avanzar la cadena de reasignación y los ACLs (AppointmentLookupService y HospitalConfigurationService). En la Domain Layer, el aggregate ReassignmentOffer encapsula las reglas de negocio, junto con el enum ReassignmentStatus, los domain events y la interfaz ReassignmentOfferRepository. En la Infrastructure Layer, los adapters implementan la persistencia con Spring Data JPA (ReassignmentOfferRepositoryImpl) y la expiración automática de ofertas (ReassignmentExpirationScheduler). La comunicación con la base de datos PostgreSQL se realiza mediante JDBC/JPA, como se observa en la Figura 51.

2.6.3.6. Bounded Context Software Architecture Code Level Diagrams

2.6.3.6.1. Bounded Context Domain Layer Class Diagrams

Figura 52

Diagrama de clases — Reassignment



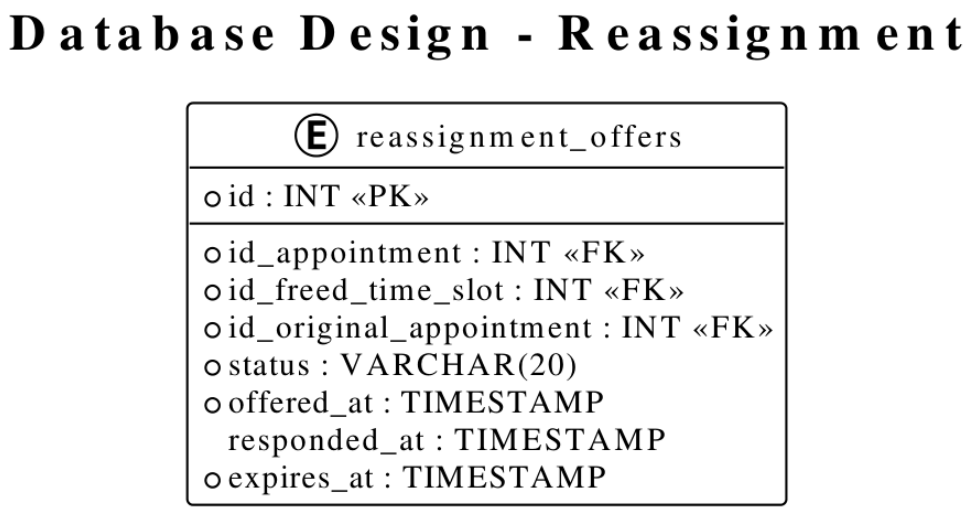
Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

El diagrama de clases del dominio del bounded context Reassignment representa el aggregate root ReassignmentOffer que encapsula el estado de la oferta, junto con el enum ReassignmentStatus que define los estados posibles (PENDING, ACCEPTED, REJECTED, EXPIRED, ATTENDED, ABSENT). Se muestran los seis Domain Events (records) que publica el aggregate (ReassignmentOfferSentEvent, ReassignmentOfferAcceptedEvent, ReassignmentOfferRejectedEvent, ReassignmentOfferExpiredEvent, ReassignmentOfferAttendedEvent, ReassignmentOfferNoShowEvent) y la interfaz ReassignmentOfferRepository, como se observa en la Figura 52.

2.6.3.6.2. Bounded Context Database Design Diagram

Figura 53

Diagrama de base de datos — Reassignment



Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

El diagrama de base de datos del bounded context Reassignment muestra la tabla reassignment_offers, que almacena las ofertas de reasignación enviadas a los pacientes de la cola de reserva. La tabla incluye dos foreign keys hacia appointments (el paciente candidato y la cita original liberada) y dos foreign keys hacia time_slots (el cupo liberado y el slot actual del candidato), junto con el estado de la oferta y los timestamps de envío, respuesta y expiración, como se observa en la Figura 53.

2.6.4. Bounded Context: Arrival & QR Check-in

El **bounded context de Arrival & QR Check-in** gestiona la llegada física del paciente al establecimiento de salud, la validación del código QR y la **cola de asistencia** (`attendance_queue`), que ordena a los pacientes por orden de llegada dentro de cada `time_slot` . Cuando el paciente escanea el código QR, el sistema valida el token firmado por el propio backend, registra su presencia, lo ingresa a la cola de asistencia y emite el ticket digital con su posición. Si el paciente no se presenta dentro de la ventana de tolerancia o no responde al llamado, el sistema declara su ausencia y libera el cupo.

2.6.4.1. Domain Layer

La capa de **Domain** representa el núcleo del negocio de llegada y check-in. Aquí se definen las entidades, value objects, enums, aggregates, factories, domain services e interfaces que encapsulan las reglas de la cola de asistencia y la validación de presencia.

CheckIn (Aggregate Root)

Atributos: `id` , `idAppointment` , `qrToken` , `status: CheckInStatus` , `checkedInAt`

Métodos:

- `isValid()` → retorna `true` si el estado es `VALID` .
- `expire()` → cambia el estado a `EXPIRED` .
- `invalidate()` → cambia el estado a `INVALID` .

Propósito: Representa la validación de presencia física del paciente. Es aggregate root porque controla el ciclo de vida del check-in.

AttendanceQueue (Aggregate Root)

Atributos: `id` , `idTimeSlot` , `date` , `status: AttendanceQueueStatus`

Métodos:

- `open()` → cambia el estado a `OPEN` .
- `close()` → cambia el estado a `CLOSED` .
- `pause()` → cambia el estado a `PAUSED` .
- `getEntriesOrdered()` → retorna las entradas ordenadas por `position` .

Propósito: Representa la fila de asistencia de un `time_slot` en una fecha. Es aggregate root porque agrupa las `QueueEntry` .

QueueEntry (Entity)

Atributos: `id` , `idAttendanceQueue` , `idCheckIn` , `position` , `status: QueueEntryStatus` , `calledAt` , `attendedAt`

Métodos:

- `call()` → cambia el estado a `CALLED` .
- `startAttention()` → cambia el estado a `IN_ATTENTION` .

- `markAsAttended()` → cambia el estado a `ATTENDED`.
- `markAsAbsent()` → cambia el estado a `ABSENT`.
- `isWaiting()` → retorna `true` si el estado es `WAITING`.

Propósito: Representa a cada persona en la fila de asistencia.

QueuePosition (Value Object)
Atributos: `value`, `totalInQueue`

Propósito: Encapsula la posición del paciente en la cola de asistencia. Es inmutable y se calcula por timestamp de check-in.

CheckInStatus (Enum)
Valores posibles: `VALID`, `EXPIRED`, `INVALID`

Propósito: Define el estado de la validación del check-in.

AttendanceQueueStatus (Enum)
Valores posibles: `OPEN`, `CLOSED`, `PAUSED`

Propósito: Define el estado de la fila de asistencia.

QueueEntryStatus (Enum)
Valores posibles: `WAITING`, `CALLED`, `IN_ATTENTION`, `ATTENDED`, `ABSENT`

Propósito: Define el estado de cada entrada en la cola de asistencia.

CheckInFactory (Factory)
Métodos:

- `createCheckIn(appointment, qrToken): CheckIn`

Propósito: Encapsula la creación de un check-in, validando que la cita exista.

QueueDomainService (Domain Service)
Métodos:

- `calculatePosition(attendanceQueueId): Int`
- `validateToleranceWindow(checkInTime, timeSlot): Boolean`

Propósito: Encapsula la lógica de cálculo de posición en la cola y la validación de la ventana de tolerancia.

CheckInRepository (Interface)
Métodos:

- `save(checkIn: CheckIn): CheckIn`
- `findById(id: Int): CheckIn?`
- `findByAppointment(appointmentId: Int): CheckIn?`
- `findByQRToken(qrToken: String): CheckIn?`
- `updateStatus(id: Int, status: CheckInStatus)`

Propósito: Define las operaciones de persistencia para check-ins.

AttendanceQueueRepository (Interface)
Métodos:

- `save(queue: AttendanceQueue): AttendanceQueue`
- `findById(id: Int): AttendanceQueue?`
- `findByTimeSlotAndDate(timeSlotId: Int, date: Date): AttendanceQueue?`
- `createIfNotExists(timeSlotId: Int, date: Date): AttendanceQueue`
- `updateStatus(id: Int, status: AttendanceQueueStatus)`

Propósito: Define las operaciones de persistencia para colas de asistencia.

QueueEntryRepository (Interface)
Métodos:

- `save(entry: QueueEntry): QueueEntry`
- `findById(id: Int): QueueEntry?`
- `findByAttendanceQueue(queueId: Int): List<QueueEntry>`
- `getNextPosition(queueId: Int): Int`
- `updateStatus(id: Int, status: QueueEntryStatus)`
- `findExpiredCalledEntries(toleranceMinutes: Int): List<QueueEntry>`

Propósito: Define las operaciones de persistencia para entradas de la cola.

EventPublisher (Interface)
Métodos:

- `publish(event: DomainEvent)`

Propósito: Define la interfaz para publicar eventos de dominio. La implementación concreta usa Spring Events.

CheckInCompletedEvent (Domain Event)
Atributos: `checkInId` , `appointmentId` , `attendanceQueueId` , `position` , `completedAt`

Propósito: Representa el hecho de negocio de que un paciente completó su check-in y fue ingresado a la cola de asistencia.

PatientCalledEvent (Domain Event)
Atributos: `queueEntryId` , `attendanceQueueId` , `position` , `calledAt`

Propósito: Representa el hecho de negocio de que un paciente fue llamado a consultorio.

PatientAbsentEvent (Domain Event)
Atributos: `queueEntryId` , `appointmentId` , `freedTimeSlotId` , `absentAt`

Propósito: Representa el hecho de negocio de que un paciente fue declarado ausente por no presentarse al llamado.

2.6.4.2. Interface Layer
La **Interface Layer** expone las funcionalidades del bounded context mediante endpoints REST.

CheckInsController (REST API Controller)
Endpoints:

- `POST /api/v1/check-ins/qr` → Valida el QR y registra el check-in.
- `GET /api/v1/check-ins/{id}` → Obtiene el detalle del check-in.
- `GET /api/v1/check-ins/appointment/{appointmentId}` → Obtiene el check-in de una cita.

Explicación: Este controlador gestiona las operaciones sobre el aggregate `CheckIn`.

AttendanceQueuesController (REST API Controller)
Endpoints:

- `GET /api/v1/attendance-queues/{id}` → Obtiene el estado de la cola.
- `GET /api/v1/attendance-queues/{id}/entries` → Lista las entradas de la cola.
- `POST /api/v1/attendance-queues/{id}/call-next` → Llama al siguiente paciente.

Explicación: Este controlador gestiona las operaciones sobre el aggregate `AttendanceQueue`.

QueueEntriesController (REST API Controller)
Endpoints:

- `GET /api/v1/queue-entries/{id}` → Obtiene el detalle de una entrada.
- `POST /api/v1/queue-entries/{id}/absent` → Marca una entrada como ausente.

Explicación: Este controlador gestiona las operaciones sobre la entity `QueueEntry`.

2.6.4.3. Application Layer
La **Application Layer** orquesta los casos de uso del dominio mediante **Command Services** y **Query Services**. Los **Command Handlers** viven dentro de los Command Services, y los **Event Handlers** en `application/internal/eventhandlers/`.

CheckInCommandService (Interface)
Métodos (Command Handlers):

- `validateQR(command: ValidateQRCommand): CheckInResult`
- `registerCheckIn(command: RegisterCheckInCommand): QueueEntry`
- `declareAbsence(command: DeclareAbsenceCommand): void`
- `detectAbsences(): void`

Propósito: Define los comandos relacionados con el check-in.

QueueCommandService (Interface)
Métodos (Command Handlers):

- `callNextPatient(command: CallNextPatientCommand): QueueEntry`

Propósito: Define los comandos relacionados con la cola de asistencia.

CheckInQueryService (Interface)
Métodos (Query Handlers):

- `getById(id: Int): CheckIn?`
- `getByAppointment(appointmentId: Int): CheckIn?`

Propósito: Define las consultas relacionadas con el check-in.

QueueQueryService (Interface)
Métodos (Query Handlers):

- `getQueuePosition(checkInId: Int): QueuePosition`
- `getQueueEntries(queueId: Int): List<QueueEntry>`

Propósito: Define las consultas relacionadas con la cola.

CheckInCommandServiceImpl (Implementation)
Responsabilidad: Implementar los comandos de check-in.

Flujo de `validateQR`:

1. Recibe `qrToken`.
2. Lee `checkInToleranceMinutes` de la configuración.
3. Busca la cita asociada al QR.
4. Valida el estado y la ventana de tolerancia con `QueueDomainService.validateToleranceWindow`.
5. Retorna el resultado.

Flujo de `registerCheckIn`:

1. Crea el `CheckIn` con estado `VALID` usando `CheckInFactory`.
2. Crea o recupera la `AttendanceQueue`.
3. Calcula la `position` con `QueueDomainService.calculatePosition`.
4. Crea el `QueueEntry` con estado `WAITING`.
5. Actualiza la cita a `CONFIRMED`.
6. Publica el evento `CheckInCompletedEvent`.
7. Notifica al paciente con su posición.

Flujo de `declareAbsence`:

1. Lee `postCallToleranceMinutes` de la configuración.
2. Valida que el `QueueEntry` esté `CALLED`.
3. Cambia el estado a `ABSENT`.
4. Publica el evento `PatientAbsentEvent`.
5. Dispara la reasignación.

QueueCommandServiceImpl (Implementation)
Responsabilidad: Implementar los comandos de cola.

Flujo de `callNextPatient`:

1. Busca el primer `QueueEntry` con estado `WAITING`.
2. Cambia su estado a `CALLED`.
3. Publica el evento `PatientCalledEvent`.
4. Notifica al paciente.

CheckInCompletedEventHandler (Event Handler)
Responsabilidad: Reaccionar al evento `CheckInCompletedEvent`. **Flujo:**

1. Escucha el evento.
2. Notifica al paciente que ha sido ingresado a la cola.
3. Registra la acción en el log de auditoría.

PatientCalledEventHandler (Event Handler)
Responsabilidad: Reaccionar al evento `PatientCalledEvent` . **Flujo:**

1. Escucha el evento.
2. Envía una notificación push al paciente.
3. Registra la acción en el log de auditoría.

2.6.4.4. Infrastructure Layer
La capa de **Infrastructure** contiene las implementaciones concretas.

CheckInRepositoryImpl
Implementa: `CheckInRepository` **Tecnología:** Spring Data JPA + PostgreSQL **Explicación:** Ejecuta operaciones sobre la tabla `check_ins` .

AttendanceQueueRepositoryImpl
Implementa: `AttendanceQueueRepository` **Tecnología:** Spring Data JPA + PostgreSQL **Explicación:** Ejecuta operaciones sobre la tabla `attendance_queues` .

QueueEntryRepositoryImpl
Implementa: `QueueEntryRepository` **Tecnología:** Spring Data JPA + PostgreSQL **Explicación:** Ejecuta operaciones sobre la tabla `queue_entries` .

HospitalConfigurationRepositoryImpl
Implementa: `HospitalConfigurationRepository` **Tecnología:** Spring Data JPA + PostgreSQL **Explicación:** Lee `checkInToleranceMinutes` , `postCallToleranceMinutes` y `attendanceQueueVisible` de la configuración del hospital.

JwtQRValidator (Utility)
Función: Valida el token QR firmado por el backend usando JWT. **Tecnología:** java-jwt / jjwt **Explicación:** Verifica la firma y la expiración del `qrToken` generado por el backend al confirmar la reserva. No se comunica con ningún sistema externo.

TicketGenerationAdapter
Función: Genera el ticket digital con identificador de llamado y posición en la cola. **Tecnología:** iText / Kotlin PDF

NotificationAdapter
Función: Envía notificaciones al paciente (check-in completado, llamado a consultorio). **Tecnología:** SMTP + Firebase Cloud Messaging

---

SpringEventPublisherImpl
Implementa: `EventPublisher` **Función:** Publica eventos de dominio usando Spring Events. **Tecnología:** `ApplicationEventPublisher` de Spring

---

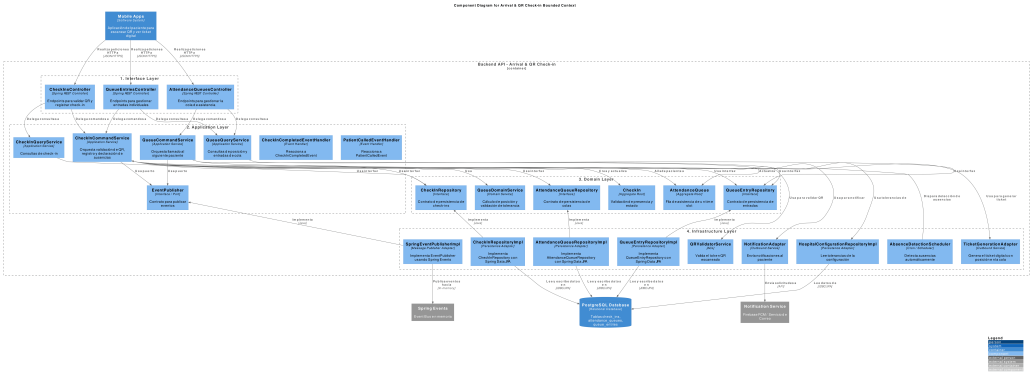
AbsenceDetectionScheduler
Función: Ejecuta periódicamente `detectAbsences` para detectar ausencias automáticamente. **Tecnología:** `@Scheduled` de Spring

---

2.6.4.5. Bounded Context Software Architecture Component Level Diagrams

Figura 54

Diagrama de componentes — Arrival



Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

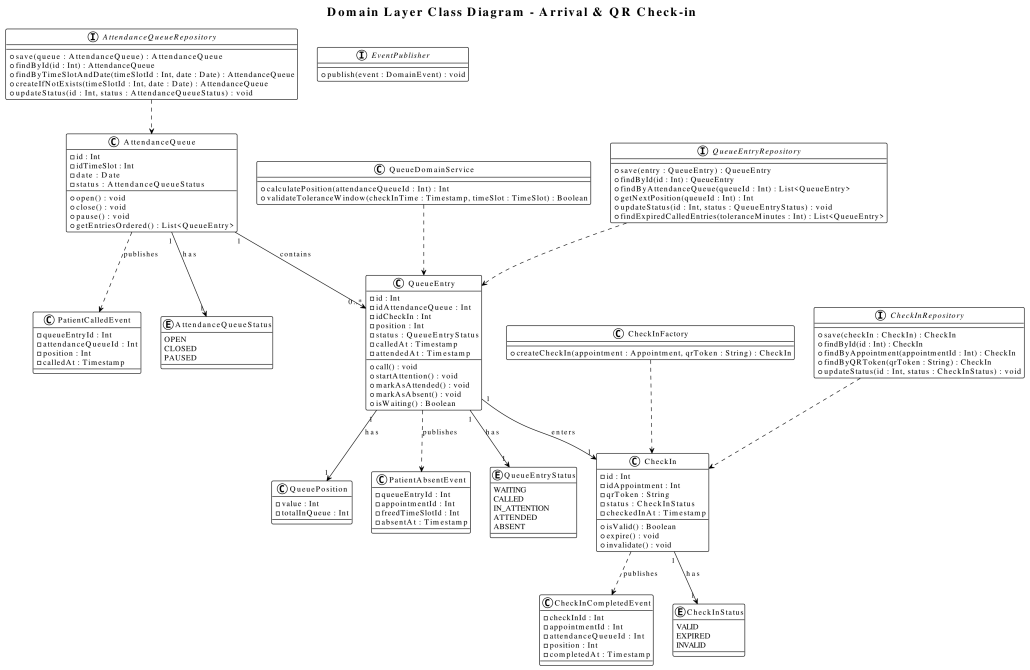
El diagrama de componentes del bounded context Arrival & QR Check-in muestra la organización interna del Backend API en sus cuatro capas. En la Interface Layer, los controladores CheckInsController, AttendanceQueuesController y QueueEntriesController exponen los endpoints REST para validar el QR, registrar el check-in y gestionar la cola de asistencia. En la Application Layer, los Command Services y Query Services orquestan los casos de uso, junto con los Event Handlers que reaccionan a los eventos de check-in completado y paciente llamado. En la Domain Layer, los aggregates CheckIn y AttendanceQueue encapsulan las reglas de negocio, junto con el QueueDomainService (que calcula la posición y valida la tolerancia) y las interfaces de repositorio. En la Infrastructure Layer, los adapters implementan la persistencia con Spring Data JPA (CheckInRepositoryImpl, AttendanceQueueRepositoryImpl, QueueEntryRepositoryImpl, HospitalConfigurationRepositoryImpl), la validación del QR firmado por el backend (JwtQRValidator), la generación del ticket digital (TicketGenerationAdapter), el envío de notificaciones (NotificationAdapter), la publicación de eventos con Spring Events (SpringEventPublisherImpl) y la detección automática de ausencias (AbsenceDetectionScheduler). La comunicación con la base de datos PostgreSQL se realiza mediante JDBC/JPA, como se observa en la Figura 54.

2.6.4.6. Bounded Context Software Architecture Code Level Diagrams

2.6.4.6.1. Bounded Context Domain Layer Class Diagrams

Figura 55

Diagrama de clases — Arrival



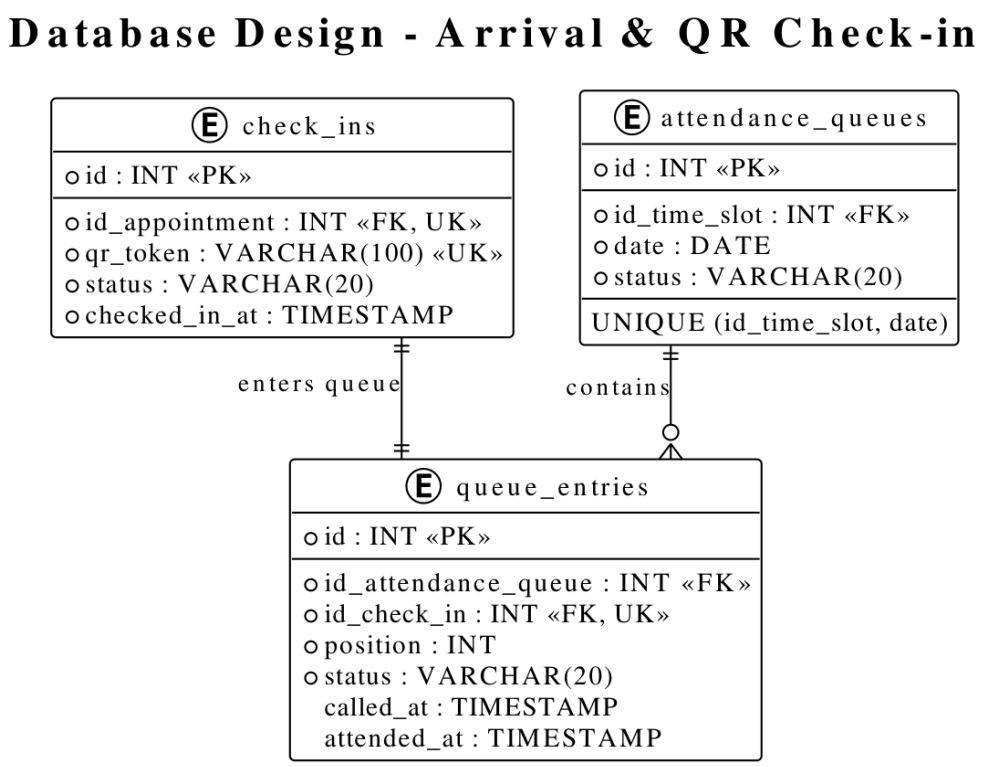
Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

El diagrama de clases del dominio del bounded context Arrival & QR Check-in representa los aggregates root CheckIn y AttendanceQueue, junto con la entity QueueEntry y el value object QueuePosition. Se muestran los enums CheckInStatus, AttendanceQueueStatus y QueueEntryStatus que definen los estados posibles de cada componente, la factory CheckInFactory que encapsula la creación de check-ins, el QueueDomainService que encapsula el cálculo de posición y la validación de la ventana de tolerancia, y las interfaces CheckInRepository, AttendanceQueueRepository, QueueEntryRepository y EventPublisher que definen los contratos de persistencia y publicación de eventos. Se muestran también los tres Domain Events que publica el aggregate: CheckInCompletedEvent, PatientCalledEvent y PatientAbsentEvent, como se observa en la Figura 55.

2.6.4.6.2. Bounded Context Database Design Diagram

Figura 56

Diagrama de base de datos — Arrival



Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

El diagrama de base de datos del bounded context Arrival & QR Check-in muestra las tablas check_ins, attendance_queues y queue_entries, junto con sus columnas, claves primarias, claves foráneas y restricciones de unicidad. La tabla check_ins almacena la validación de presencia del paciente con una foreign key hacia appointments. La tabla attendance_queues representa la fila de asistencia por time_slot y fecha, con una foreign key hacia time_slots. La tabla queue_entries almacena las entradas individuales de cada paciente en la cola, con foreign keys hacia attendance_queues y check_ins, y un campo position que determina el orden de atención por timestamp de check-in, como se observa en la Figura 56.

2.6.5. Bounded Context: Hospital Operations & Configuration

El **bounded context de Hospital Operations & Configuration** gestiona la configuración operativa del establecimiento de salud y provee dashboards y reportes para monitorear la operación diaria, el ausentismo y la demanda de servicios. Parametriza el comportamiento de los demás bounded contexts mediante reglas configurables como la capacidad máxima por bloque horario, las tolerancias de check-in y post-llamado, el tiempo de respuesta para la reasignación y los plazos de reserva y cancelación.

2.6.5.1. Domain Layer

La capa de **Domain** representa el núcleo del negocio de configuración y operación del hospital. Aquí se definen las entidades, value objects, enums e interfaces que encapsulan las reglas operativas.

HospitalConfiguration (Aggregate Root)
Atributos: `id` , `maxCapacityPerSlot` , `bookingOrderScope: BookingOrderScope` , `checkInToleranceMinutes` , `postCallToleranceMinutes` , `reassignmentResponseTimeoutMin` , `bookingCutoffTime` , `cancellationDeadlineHours` , `attendanceQueueVisible` , `updatedAt`

Métodos:

- `updateMaxCapacity(value)` → valida que sea mayor a 0 y actualiza la capacidad máxima por bloque.
- `updateTolerances(checkIn, postCall)` → valida que sean mayores o iguales a 0 y actualiza las tolerancias.
- `updateReassignmentTimeout(value)` → valida que sea mayor o igual a 0 y actualiza el timeout de reasignación.
- `updateBookingCutoffTime(value)` → actualiza la hora límite para reservar citas.
- `updateCancellationDeadline(value)` → actualiza el plazo mínimo para cancelar citas.
- `validate()` → valida que todos los parámetros sean coherentes entre sí.

Propósito: Representa la configuración operativa del establecimiento. Es aggregate root porque agrupa todos los parámetros operativos que parametrizan el comportamiento de los demás bounded contexts.

BookingOrderScope (Enum)
Valores posibles: `GLOBAL` , `PER_SPECIALTY`

Propósito: Define si el `bookingOrder` se calcula de forma global al establecimiento o por especialidad.

HospitalConfigurationRepository (Interface)
Métodos:

- `save(config: HospitalConfiguration): HospitalConfiguration`
- `findDefault(): HospitalConfiguration?`
- `update(config: HospitalConfiguration): HospitalConfiguration`

Propósito: Define las operaciones de persistencia para la configuración del hospital.

EventPublisher (Interface)
Métodos:

- `publish(event: DomainEvent)`

Propósito: Define la interfaz para publicar eventos de dominio. La implementación concreta usa Spring Events.

2.6.5.2. Interface Layer
La **Interface Layer** expone las funcionalidades del bounded context mediante endpoints REST.

ConfigurationController (REST API Controller)
Endpoints:

- `GET /api/v1/config` → Obtiene la configuración actual del hospital.
- `PUT /api/v1/config` → Actualiza la configuración operativa.
- `GET /api/v1/config/reports` → Genera un reporte operativo en PDF/CSV.
- `GET /api/v1/config/dashboard` → Obtiene métricas operativas del hospital.

Explicación: Este controlador gestiona las operaciones sobre el aggregate `HospitalConfiguration` , incluyendo la consulta y actualización de parámetros, la generación de reportes y la visualización de métricas en el dashboard.

2.6.5.3. Application Layer

La **Application Layer** orquesta los casos de uso del dominio mediante **Command Services** y **Query Services**. Los **Command Handlers** viven dentro de los Command Services, y los **Event Handlers** en `application/internal/eventhandlers/`.

ConfigurationCommandService (Interface)

Métodos (Command Handlers):

- `updateConfiguration(command: UpdateConfigurationCommand): HospitalConfiguration`

Propósito: Define los comandos relacionados con la configuración operativa.

ConfigurationQueryService (Interface)

Métodos (Query Handlers):

- `getConfiguration(): HospitalConfiguration?`
- `generateReport(query: GenerateReportQuery): Report`
- `getDashboard(query: GetDashboardQuery): Dashboard`

Propósito: Define las consultas relacionadas con la configuración, reportes y dashboard.

ConfigurationCommandServiceImpl (Implementation)

Responsabilidad: Implementar los comandos de configuración.

Flujo de `updateConfiguration`:

1. Recibe los nuevos parámetros de configuración.
2. Valida que los parámetros sean coherentes usando el método `validate()` del aggregate.
3. Actualiza la configuración en el repositorio.
4. Publica el evento `ConfigurationUpdatedEvent`.
5. Retorna la configuración actualizada.

ConfigurationUpdatedEventHandler (Event Handler)

Responsabilidad: Reaccionar al evento `ConfigurationUpdatedEvent`. **Flujo:**

1. Escucha el evento `ConfigurationUpdatedEvent`.
2. Actualiza la caché local de los bounded contexts que consumen la configuración.
3. Registra la acción en el log de auditoría.

2.6.5.4. Infrastructure Layer

La capa de **Infrastructure** contiene las implementaciones concretas.

HospitalConfigurationRepositoryImpl

Implementa: `HospitalConfigurationRepository` **Tecnología:** Spring Data JPA + PostgreSQL **Explicación:** Ejecuta operaciones sobre la tabla `hospital_configurations`. Como es un singleton, siempre retorna el único registro existente.

ReportGeneratorAdapter

Función: Genera reportes operativos en formato PDF y CSV a partir de los datos de atención, ausentismo y demanda. **Tecnología:** iText / Apache POI

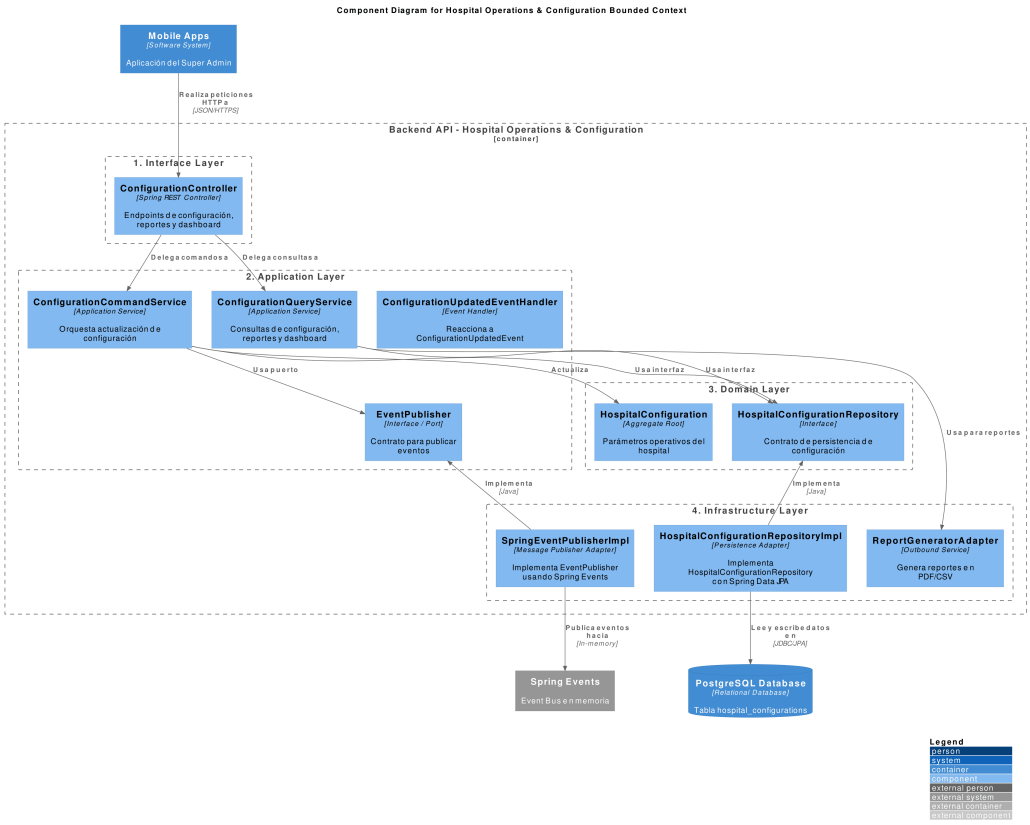
SpringEventPublisherImpl

Implementa: `EventPublisher` **Función:** Publica eventos de dominio usando Spring Events. **Tecnología:** `ApplicationEventPublisher` de Spring

2.6.5.5. Bounded Context Software Architecture Component Level Diagrams

Figura 57

Diagrama de componentes — Hospital Operations & Configuration



Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

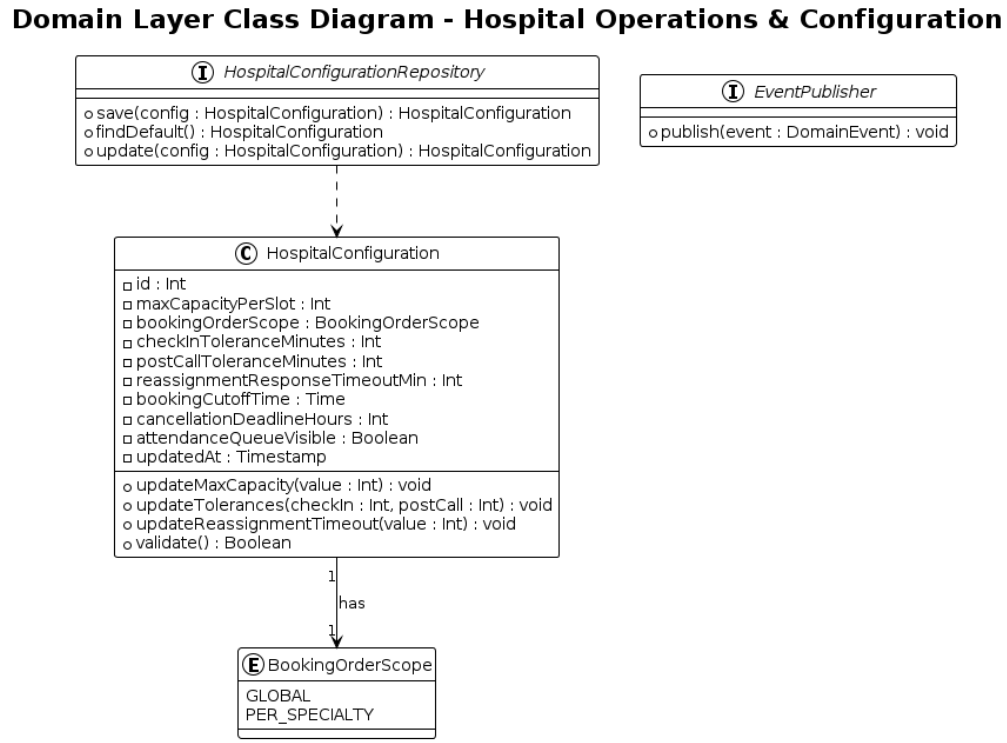
El diagrama de componentes del bounded context Hospital Operations & Configuration muestra la organización interna del Backend API en sus cuatro capas. En la Interface Layer, el ConfigurationController expone los endpoints REST para consultar y actualizar la configuración, generar reportes y visualizar el dashboard. En la Application Layer, los Command Services y Query Services orquestan los casos de uso, junto con el ConfigurationUpdatedEventHandler que reacciona a los cambios de configuración. En la Domain Layer, el aggregate HospitalConfiguration encapsula las reglas operativas del establecimiento, junto con la interfaz HospitalConfigurationRepository. En la Infrastructure Layer, los adapters implementan la persistencia con Spring Data JPA (HospitalConfigurationRepositoryImpl), la generación de reportes (ReportGeneratorAdapter) y la publicación de eventos con Spring Events (SpringEventPublisherImpl). La comunicación con la base de datos PostgreSQL se realiza mediante JDBC/JPA, como se observa en la Figura 57.

2.6.5.6. Bounded Context Software Architecture Code Level Diagrams

2.6.5.6.1. Bounded Context Domain Layer Class Diagrams

Figura 58

Diagrama de clases — Hospital Operations & Configuration



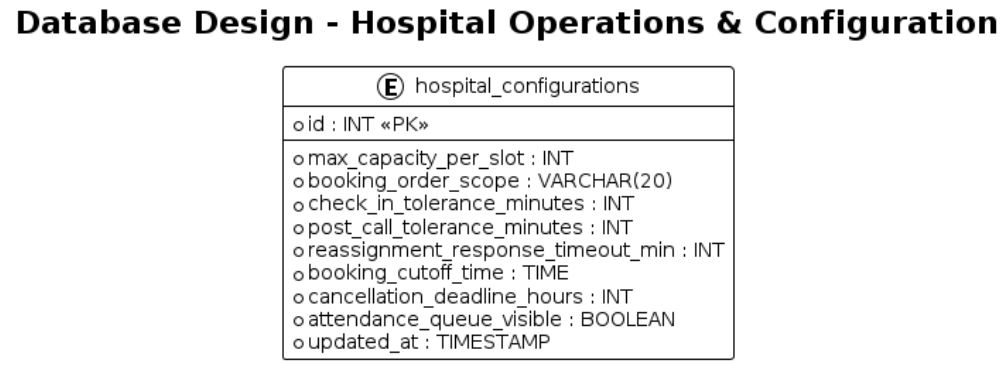
Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

El diagrama de clases del dominio del bounded context Hospital Operations & Configuration representa el aggregate root HospitalConfiguration que encapsula los parámetros operativos del establecimiento, junto con el enum BookingOrderScope que define el alcance del bookingOrder, la interfaz HospitalConfigurationRepository que define el contrato de persistencia y la interfaz EventPublisher que define el contrato para publicar eventos de dominio, como se observa en la Figura 58.

2.6.5.6.2. Bounded Context Database Design Diagram

Figura 59

Diagrama de base de datos — Hospital Operations & Configuration



Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

El diagrama de base de datos del bounded context Hospital Operations & Configuration muestra la tabla hospital_configurations, que almacena los parámetros operativos del establecimiento. La tabla es un singleton, es decir, contiene un único registro que define la configuración global del hospital. Los campos incluyen la capacidad máxima por bloque horario, el alcance del bookingOrder, las tolerancias de check--in y post-llamado, el timeout de reasignación, la hora de corte para reservas, el plazo de cancelación y la visibilidad de la cola de asistencia, como se observa en la Figura 59.

Capítulo III: Solution UI/UX Design

3.1. Product design

3.1.1. Style Guidelines

3.1.1.1. General Style Guidelines

Las presentes guías de estilo establecen los lineamientos visuales y de comunicación que rigen la identidad de **SaludYa**, producto digital desarrollado por la startup **RuwaLabs**. Su propósito es garantizar consistencia en todos los productos de la solución (Landing Page, aplicaciones móviles para pacientes y personal de salud, y servicios web), facilitando el trabajo colaborativo del equipo y asegurando una experiencia coherente para los usuarios.

Las decisiones aquí documentadas se sustentan en los principios de diseño inclusivo, accesibilidad (a11y) e internacionalización (i18n) establecidos en el proyecto, y toman como referencia buenas prácticas de Design Systems reconocidos, adaptadas al contexto de los establecimientos públicos de salud del Perú.

Branding

La identidad de marca de SaludYa se construye sobre los siguientes elementos:

Tabla 26

Branding

Elemento	Descripción
Nombre	SaludYa
Startup	RuwaLabs
Logotipo	Isotipo con icono de salud (cruz/maleta) en color primario, acompañado del nombre «SaludYa» en tipografía sans-serif
Isotipo	Símbolo gráfico que representa el acceso ágil a la salud, usado en favicon, app icon y elementos de navegación
Tagline	«Citas médicas sin colas para el Perú»
Tono de comunicación	Formal pero cercano, empático, claro y directo
Público objetivo	Pacientes de zonas urbanas periféricas y personal asistencial/administrativo de establecimientos públicos de salud
Valores de marca	Accesibilidad, transparencia, colaboración e impacto social

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

El logotipo se utiliza en el header y footer del Landing Page, así como en las pantallas de inicio de sesión de ambas aplicaciones móviles. Su versión reducida (`--logo-height-sm`) se emplea en contextos donde el espacio es limitado, como la versión móvil del Landing Page, como se detalla en la Tabla 26.

Typography

La tipografía seleccionada prioriza la legibilidad en pantallas de distintos tamaños y en contextos de baja iluminación, frecuentes en establecimientos de salud.

Landing Page (web)

Tabla 27

Typography — Landing Page (web)

Elemento	Fuente	Tamaño	Peso	Uso
Fuente base	Segoe UI / Helvetica Neue / Arial	1rem	400	Texto general, párrafos
Fuente de títulos	Segoe UI / Helvetica Neue / Arial	Variable	700	Encabezados h1–h4
h1	—	2.5rem	700	Título principal del Hero
h2	—	2rem	700	Títulos de sección
h3	—	1.35rem	700	Títulos de tarjetas y subsecciones
h4	—	1.1rem	600	Subtítulos internos
Interlineado	—	1.6	—	Cuerpo de texto
Interlineado títulos	—	1.25	—	Encabezados

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

La elección de fuentes del sistema (Segoe UI, Helvetica Neue, Arial) responde a criterios de rendimiento, disponibilidad multiplataforma y familiaridad para el usuario, evitando dependencias externas que afecten la carga del Landing Page, como se detalla en la Tabla 27.

Aplicaciones móviles (Android)

Tabla 28

Typography — Aplicaciones móviles (Android)

Elemento	Fuente	Tamaño	Peso	Uso
Fuente base	Inter	Variable	400	Texto general
Fuente de títulos	Inter	Variable	600	Encabezados y énfasis
Body Bold Large	Inter	18sp / 150% (27sp)	600	Cuerpo destacado, títulos de tarjeta
Body Extra Small	Inter	12sp / 150% (18sp)	400	Texto secundario, captions

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

En las aplicaciones móviles se utiliza la familia **Inter** con `letter-spacing` negativo en textos destacados (`-0.18px` en Body Bold Large) y `font-feature-settings: 'calt' off` para desactivar ligaduras contextuales. Los tamaños se expresan en **sp** (scale-independent pixels), conforme a las guías de Material Design para Android, como se detalla en la Tabla 28.

Colors

La paleta de colores de SaludYa se inspira en el sector salud, utilizando tonos verdes que transmiten confianza, bienestar y cercanía, complementados con un acento amarillo para elementos de foco y llamadas de atención.

Landing Page (web)

La paleta establece los colores de marca y los tonos utilizados en fondos, textos y estados de los componentes; estos criterios se aplican a landing page (web), como se detalla en la Tabla 29.

Tabla 29

Colors — Landing Page (web)

Color	Código HEX	Uso principal
Primario	#0b8f6b	Botones, enlaces, iconos, acentos
Primario oscuro	#076e52	Hover de botones, títulos de tarjetas
Primario claro	#e6f5f0	Fondos de sección, hover de navegación
Acento	#ffb703	Foco visible, detalles de marca
Texto	#1c2b2a	Texto principal
Texto atenuado	#56706d	Párrafos secundarios, roles
Fondo	#ffffff	Fondo base
Fondo alterno	#f4faf8	Secciones alternas
Borde	#d8e6e2	Bordes de tarjetas y separadores
Peligro	#c0392b	Mensajes de error (formularios)
Blanco	#ffffff	Texto sobre fondos oscuros
Footer	#0d2722	Fondo del pie de página

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

Figura 60

Paleta de colores del Landing Page de SaludYa

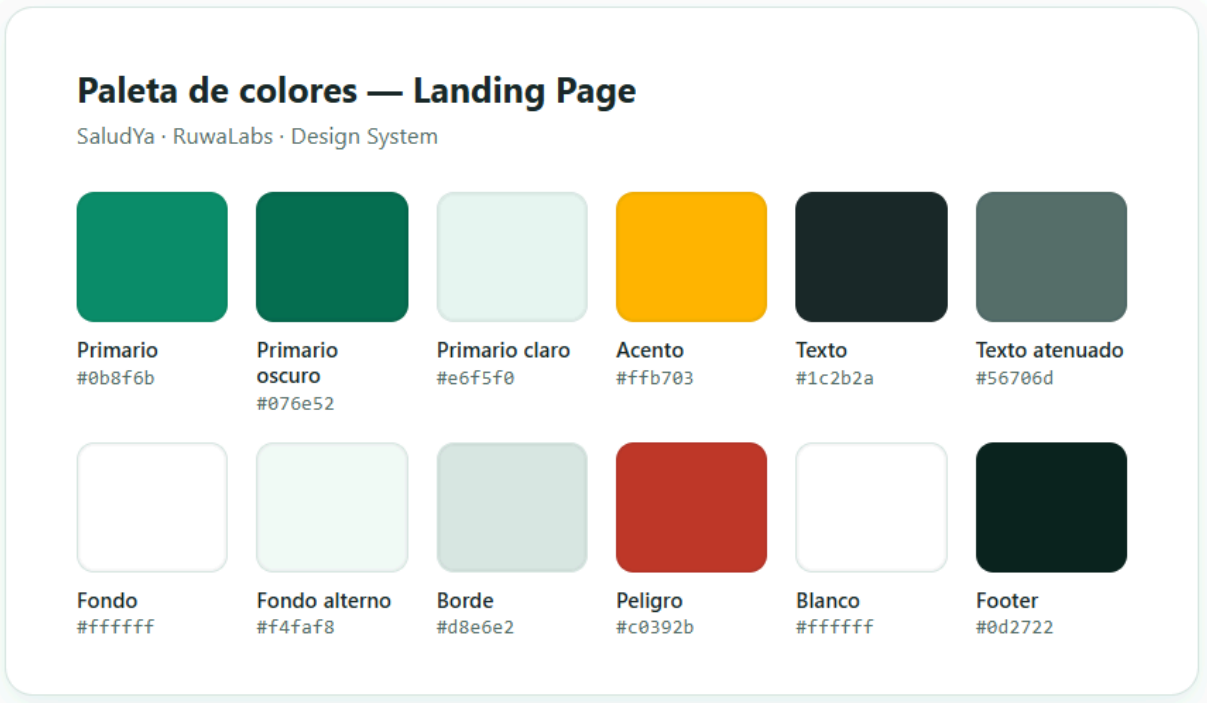


Figura 11: Paleta de colores del Landing Page de SaludYa

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

Aplicaciones móviles (Android)

Tabla 30

Colors — Aplicaciones móviles (Android)

Token	Código HEX	Uso principal
WF White	#FFFFFF	Fondo base, superficies

Token	Código HEX	Uso principal
WF 100	#F7F9FC	Fondo alterno muy claro
WF 200	#EDF0F7	Fondos de tarjetas y contenedores
WF 300	#E2E7F0	Separadores suaves
WF 400	#CBD2E0	Bordes suaves, elementos inactivos
WF 600	#717D96	Texto secundario
WF 700	#4A5468	Texto de énfasis medio
WF 800	#2D3648	Texto principal, bordes
WF 900	#1A202C	Texto de máximo contraste
Negro	#000000	Texto en superficies claras
Gris claro	#D9D9D9	Placeholders, elementos deshabilitados

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

Los colores fueron seleccionados para cumplir con el nivel de contraste **WCAG AA**, garantizando legibilidad para personas con baja visión o daltonismo, como se detalla en la Tabla 30.

Spacing

Landing Page (web)

Se define una escala de espaciado consistente basada en múltiplos de 0.25rem, aplicada a márgenes, padding y separación entre elementos en el Landing Page, como se detalla en la Tabla 31.

Tabla 31

Spacing — Landing Page (web)

Variable	Valor	Uso
--space-1	0.25rem	Separaciones mínimas
--space-2	0.5rem	Padding interno de botones pequeños
--space-3	0.75rem	Separación entre elementos inline
--space-4	1rem	Padding estándar
--space-5	1.5rem	Separación entre bloques
--space-6	2rem	Separación entre secciones

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

Aplicaciones móviles (Android)

Tabla 32

Spacing — Aplicaciones móviles (Android)

Token	Valor	Uso
--space-1	2dp	Separaciones mínimas, bordes internos
--space-2	8dp	Gap entre íconos y texto, separación inline
--space-3	11dp	Padding vertical de contenedores pequeños
--space-4	12dp	Padding interno de botones y cards
--space-5	28dp	Padding horizontal de secciones
--space-6	45dp	Separación entre bloques principales

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

Los valores en **dp** (density-independent pixels) provienen directamente de los tokens definidos en Figma y se aplican a padding, márgenes y gaps en las aplicaciones móviles, como se detalla en la Tabla 32.

Border Radius

Landing Page (web)

Los radios de esquina mantienen una forma consistente en botones, tarjetas y otros contenedores de la interfaz; estos criterios se aplican a landing page (web), como se detalla en la Tabla 33.

Tabla 33

Border Radius — Landing Page (web)

Variable	Valor	Uso
`--radius-sm`	6px	Botones pequeños, badges
`--radius-md`	12px	Botones, tarjetas
`--radius-lg`	20px	Contenedores destacados, hero

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

Aplicaciones móviles (Android)

Tabla 34

Border Radius — Aplicaciones móviles (Android)

Token	Valor	Uso
`--radius-sm`	12dp	Cards superiores (radio superior únicamente)
`--radius-md`	12dp	Botones y contenedores estándar

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

En las aplicaciones móviles los contenedores principales utilizan un radio superior de **12dp** (`border-radius: 12dp 12dp 0 0`), reservado para cards ancladas a la parte inferior de la pantalla, como se detalla en la Tabla 34.

Shadows

Landing Page (web)

Las sombras distinguen las superficies y refuerzan la jerarquía visual de tarjetas y elementos elevados; estos criterios se aplican a landing page (web), como se detalla en la Tabla 35.

Tabla 35

Shadows — Landing Page (web)

Variable	Valor	Uso
`--shadow-sm`	`0 1px 3px rgba(0, 0, 0, 0.08)`	Tarjetas en reposo
`--shadow-md`	`0 6px 18px rgba(11, 143, 107, 0.12)`	Tarjetas en hover, menú móvil
`--shadow-lg`	`0 12px 32px rgba(11, 143, 107, 0.18)`	Hero, elementos destacados

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

Aplicaciones móviles (Android)

Las aplicaciones móviles **no emplean sombras** en su diseño actual. La jerarquía visual se resuelve mediante contraste de color, bordes y espaciado, siguiendo un enfoque flat consistente con los wireframes de alta fidelidad definidos en Figma.

Tone of Voice

El tono de comunicación de SaludYa se define a partir de cuatro dimensiones:

Tabla 36

Tone of Voice

Dimensión	Elección	Justificación
Divertido / Serio	Serio con toques cercanos	El contexto de salud requiere credibilidad, pero se busca cercanía con el paciente
Formal / Casual	Formal	Se comunica con usuarios de diversos niveles educativos y con personal institucional
Respetuoso / Irreverente	Respetuoso	Se aborda un tema sensible como la salud pública
Entusiasta / Sereno	Sereno	Se transmite confianza y estabilidad, evitando promesas exageradas

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

El lenguaje empleado en el Landing Page y las aplicaciones evita tecnicismos innecesarios, prioriza frases cortas y utiliza un vocabulario accesible para ambos segmentos objetivo, como se detalla en la Tabla 36.

Iconography

Se utiliza la librería **Font Awesome 6.5.2** para la iconografía del Landing Page, seleccionando iconos universales y reconocibles, como se detalla en la Tabla 37:

Tabla 37

Iconography

Icono	Uso
`fa-circle-check`	Bullets de beneficios en el Hero
`fa-bullseye`	Misión en la sección Sobre nosotros
`fa-eye`	Visión en la sección Sobre nosotros
`fa-universal-access`	Valor de accesibilidad
`fa-shield-halved`	Valor de transparencia
`fa-handshake`	Valor de colaboración
`fa-heart-pulse`	Valor de impacto social

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

Design System de referencia

Las decisiones visuales de SaludYa toman como referencia principios de **Material Design** (jerarquía, elevación, uso del color) y buenas prácticas de Design Systems accesibles, adaptadas al contexto peruano y a las restricciones técnicas del proyecto (rendimiento en redes móviles, compatibilidad con dispositivos de gama media y baja).

3.1.2. Information Architecture

La arquitectura de información de **SaludYa** define la manera en que se organiza, etiqueta, navega y busca el contenido en los productos digitales que forman parte de la solución: el Landing Page, la aplicación móvil para pacientes y la aplicación móvil para el personal de salud. Su objetivo es que los visitantes

y usuarios encuentren sin esfuerzo la información o funcionalidad que necesitan, reduciendo la carga cognitiva y facilitando la adopción del producto.

Las decisiones aquí documentadas se sustentan en los principios de diseño inclusivo, accesibilidad (a11y) e internacionalización (i18n), y consideran las características de ambos segmentos objetivo: pacientes de zonas urbanas periféricas y personal asistencial y administrativo de establecimientos públicos de salud.

3.1.2.1. Organization Systems

La organización del contenido en SaludYa combina distintos sistemas según el tipo de información y el objetivo del usuario en cada producto. Se aplican principios de organización visual (jerárquica, secuencial y matricial) y esquemas de categorización (alfabético, cronológico, por tópicos y según audiencia), buscando siempre reducir la carga cognitiva y facilitar el acceso a la información.

Landing Page

La organización de las secciones agrupa la información según las tareas del usuario y define la jerarquía de acceso a cada función; estos criterios se aplican a landing page, como se detalla en la Tabla 38.

Tabla 38

Organization Systems — Landing Page

Sección	Sistema de organización visual	Esquema de categorización	Justificación
Header	Jerárquico + matricial	Por tópicos	Menú horizontal con enlaces principales y selector de idioma; organización matricial por categorías de contenido
Hero	Jerárquico	Por tópicos	Prioriza mensaje principal, subtítulo, CTAs y beneficios en orden de importancia
Problema	Jerárquico	Por tópicos	Tres tarjetas con igual jerarquía, organizadas por tema
Solución	Jerárquico	Según audiencia	Dos bloques comparativos (paciente vs. personal) con listas de funcionalidades
Videos	Secuencial	Cronológico	Dos bloques secuenciales: About the Product y About the Team
Modelo de negocio	Jerárquico	Por tópicos	Tres tarjetas con igual jerarquía
Testimonios	Matricial	Alfabético	Seis tarjetas organizadas por nombre del entrevistado

Sección	Sistema de organización visual	Esquema de categorización	Justificación
Sobre nosotros	Jerárquico	Por tópicos	Misión, visión, valores y equipo en orden de relevancia
Descarga	Jerárquico	Por tópicos	Botones de tiendas como acción principal
Footer	Jerárquico	Por tópicos	Cuatro columnas organizadas por categoría (marca, enlaces, proyecto, contacto)

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

Aplicación móvil para pacientes

La organización de las secciones agrupa la información según las tareas del usuario y define la jerarquía de acceso a cada función; estos criterios se aplican a aplicación móvil para pacientes, como se detalla en la Tabla 39.

Tabla 39

Organization Systems — Aplicación móvil para pacientes

Sección	Sistema de organización visual	Esquema de categorización	Justificación
Inicio	Jerárquico	Por tópicos	Accesos rápidos a reserva, lista de espera y citas próximas
Reserva de citas	Secuencial (step-by-step)	Cronológico	Flujo paso a paso: especialidad → establecimiento → fecha → confirmación
Lista de espera	Secuencial	Cronológico	Orden por fecha de inscripción y disponibilidad
Mis citas	Jerárquico	Cronológico	Orden por fecha de atención
Familiares a cargo	Jerárquico	Alfabético	Orden por nombre del familiar
Perfil	Jerárquico	Por tópicos	Datos personales, notificaciones y configuración

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

Aplicación móvil para personal de salud

La organización de las secciones agrupa la información según las tareas del usuario y define la jerarquía de acceso a cada función; estos criterios se aplican a aplicación móvil para personal de salud, como se detalla en la Tabla 40.

Tabla 40

Organization Systems — Aplicación móvil para personal de salud

Sección	Sistema de organización visual	Esquema de categorización	Justificación
Panel principal	Jerárquico	Por tópicos	Resumen del flujo de atención del día
Gestión de citas	Jerárquico	Cronológico	Orden por hora de atención
Lista de espera	Secuencial	Cronológico	Orden por fecha de inscripción
Cancelaciones e inasistencias	Jerárquico	Cronológico	Orden por fecha del evento
Pacientes	Jerárquico	Alfabético	Orden por apellido
Reportes	Jerárquico	Cronológico	Acceso a métricas y exportación por fecha

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

Esquemas de categorización aplicados

- **Alfabético:** testimonios en el Landing Page, pacientes y familiares a cargo en las aplicaciones.
- **Cronológico:** citas, lista de espera, cancelaciones, reportes y videos.
- **Por tópicos:** secciones del Landing Page (problema, solución, modelo de negocio, equipo) y pantallas principales de las aplicaciones.
- **Según audiencia:** separación entre la app para pacientes y la app para personal de salud, y bloques diferenciados en la sección Solución del Landing Page.
- **Jerárquico:** estructura general de navegación en todos los productos, priorizando la información más relevante para el usuario.
- **Secuencial:** flujos paso a paso en la reserva de citas y en la inscripción a la lista de espera.
- **Matricial:** grid de testimonios en el Landing Page, donde el usuario puede explorar varias tarjetas sin un orden estricto.

3.1.2.2. Labelling Systems

Las etiquetas de SaludYa buscan ser simples, claras y libres de ambigüedad, empleando el mínimo número de palabras posible y un vocabulario accesible para ambos segmentos objetivo. Se prioriza el uso de términos del dominio de la salud y de la gestión de citas, evitando tecnicismos innecesarios y anglicismos.

Landing Page

Las etiquetas permiten reconocer las secciones y acciones mediante nombres que expresan su función dentro de la aplicación; estos criterios se aplican a landing page, como se detalla en la Tabla 41.

Tabla 41

Labelling Systems — Landing Page

Etiqueta	Representa
Producto	Sección con el problema y la solución
Videos	Bloque con los videos About the Product y About the Team
Testimonios	Citas de entrevistados

Etiqueta	Representa
Sobre nosotros	Información de la startup y el equipo
Descargar app	CTA principal hacia las tiendas
ES / EN	Selector de idioma
Google Play	Botón de descarga para Android
App Store	Botón de descarga para iOS

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

Aplicación móvil para pacientes

Las etiquetas permiten reconocer las secciones y acciones mediante nombres que expresan su función dentro de la aplicación; estos criterios se aplican a aplicación móvil para pacientes, como se detalla en la Tabla 42.

Tabla 42

Labelling Systems — Aplicación móvil para pacientes

Etiqueta	Representa
Inicio	Pantalla principal con accesos rápidos
Reservar cita	Inicio del flujo de reserva
Lista de espera	Inscripción y seguimiento de cupos
Mis citas	Citas reservadas por el paciente
Familiares	Gestión de dependientes
Perfil	Datos personales y configuración
Check-in QR	Acceso rápido el día de la cita

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

Aplicación móvil para personal de salud

Las etiquetas permiten reconocer las secciones y acciones mediante nombres que expresan su función dentro de la aplicación; estos criterios se aplican a aplicación móvil para personal de salud, como se detalla en la Tabla 43.

Tabla 43

Labelling Systems — Aplicación móvil para personal de salud

Etiqueta	Representa
Panel	Resumen del día
Citas	Gestión de citas del establecimiento
Lista de espera	Pacientes en espera de cupo
Cancelaciones	Registro de cancelaciones e inasistencias
Pacientes	Búsqueda y consulta de pacientes
Reportes	Métricas y exportación
Reasignar cupo	Acción sobre cupos liberados

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

Asociaciones entre etiquetas

- **Reservar cita** se asocia con **Lista de espera** cuando no hay cupos disponibles.
- **Mis citas** se asocia con **Cancelaciones** en la app del personal.

- **Familiares** se asocia con **Reservar cita** para agendar a nombre de un dependiente.
- **Perfil** se asocia con **Notificaciones** y **Configuración**.
- **Check-in QR** se asocia con **Mis citas** el día de la atención.

3.1.2.3. SEO Tags and Meta Tags

Los SEO Tags y Meta Tags del Landing Page se definen en el `<head>` del documento y buscan posicionar el sitio en buscadores para consultas relacionadas con citas médicas en establecimientos públicos de salud del Perú. Asimismo, se definen los elementos de ASO (App Store Optimization) para las aplicaciones móviles publicadas en Google Play y App Store.

Landing Page

Los títulos, las descripciones y los metadatos identifican SaludYa y describen su propuesta de valor en buscadores y tiendas de aplicaciones; estos criterios se aplican a landing page, como se detalla en la Tabla 44.

Tabla 44

SEO Tags and Meta Tags — Landing Page

Tag	Valor
Title	SaludYa — Citas médicas sin colas
Meta Description	SaludYa — Plataforma digital que conecta pacientes y personal de establecimientos públicos de salud en Perú. Reserva de citas, lista de espera dinámica y check-in por QR.
Meta Keywords	SaludYa, citas médicas, MINSA, SIS, salud pública Perú, reserva de citas, lista de espera, RuwaLabs
Meta Author	RuwaLabs
Meta Robots	index, follow
Open Graph Title	SaludYa — Citas médicas sin colas
Open Graph Description	Reserva tu cita, recibe avisos de cupos liberados y llega justo a tu atención. Para pacientes y personal de establecimientos públicos de salud.
Open Graph Image	`assets/img/icon-saludya.png`
Open Graph Type	website
Open Graph URL	https://ruwalabs.github.io/saludya-landing/
Twitter Card	summary_large_image
Twitter Title	SaludYa — Citas médicas sin colas
Twitter Description	Reserva de citas, lista de espera dinámica y check-in por QR para establecimientos públicos de salud.
Twitter Image	`assets/img/icon-saludya.png`

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

ASO (App Store Optimization)

Los títulos, las descripciones y los metadatos identifican SaludYa y describen su propuesta de valor en buscadores y tiendas de aplicaciones; estos criterios se aplican a aso (app store optimization), como se detalla en la Tabla 45.

Tabla 45

SEO Tags and Meta Tags — ASO (App Store Optimization)

Elemento	App pacientes	App personal de salud
App Title	SaludYa — Citas médicas	SaludYa Staff — Gestión de citas
App Subtitle	Reserva sin colas	Gestión del flujo de atención
App Keywords	citas médicas, MINSA, SIS, salud pública, reserva, lista de espera	gestión de citas, personal de salud, MINSA, SIS, flujo de atención
App Description	Reserva tu cita en establecimientos públicos de salud, recibe avisos de cupos liberados y gestiona a tus familiares a cargo.	Administra las citas, la lista de espera y el flujo de atención de tu establecimiento de salud en tiempo real.

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

3.1.2.4. Searching Systems

Los sistemas de búsqueda de SaludYa están diseñados para evitar que el usuario se pierda entre el volumen de información, ofreciendo filtros claros, resultados consistentes y opciones de acción directa sobre los elementos encontrados.

Landing Page

Las opciones de búsqueda relacionan las consultas del usuario con los filtros disponibles y los resultados que ofrece cada sección; estos criterios se aplican a landing page, como se detalla en la Tabla 46.

Tabla 46

Searching Systems — Landing Page

Acción	Descripción
Navegación por anclas	Enlaces del header que llevan a secciones específicas (Producto, Videos, Testimonios, Sobre nosotros)
Selector de idioma	Búsqueda de contenido en ES o EN
Scroll suave	Desplazamiento con compensación de altura del header

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

Aplicación móvil para pacientes

Las opciones de búsqueda relacionan las consultas del usuario con los filtros disponibles y los resultados que ofrece cada sección; estos criterios se aplican a aplicación móvil para pacientes, como se detalla en la Tabla 47.

Tabla 47

Searching Systems — Aplicación móvil para pacientes

Búsqueda	Filtros disponibles	Resultado
Buscar establecimiento	Distrito, especialidad	Lista de establecimientos con disponibilidad
Buscar especialidad	Establecimiento, disponibilidad	Lista de especialidades y cupos
Buscar cita	Fecha, especialidad, establecimiento	Lista de citas reservadas
Buscar familiar	Nombre	Ficha del familiar a cargo

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

Aplicación móvil para personal de salud

Las opciones de búsqueda relacionan las consultas del usuario con los filtros disponibles y los resultados que ofrece cada sección; estos criterios se aplican a aplicación móvil para personal de salud, como se detalla en la Tabla 48.

Tabla 48

Searching Systems — Aplicación móvil para personal de salud

Búsqueda	Filtros disponibles	Resultado
Buscar paciente	Apellido, DNI, historia clínica	Ficha del paciente
Buscar cita	Fecha, especialidad, estado	Lista de citas
Buscar cancelación	Fecha, especialidad	Registro de cancelaciones
Buscar cupo liberado	Fecha, especialidad	Lista de cupos disponibles

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

Visualización de resultados

Los resultados se muestran en listas ordenadas cronológica o alfabéticamente, con indicadores visuales de estado (disponible, reservado, cancelado, atendido) y opciones de acción directa (reservar, cancelar, confirmar).

3.1.2.5. Navigation Systems

Los sistemas de navegación de SaludYa guían al usuario a través del Landing Page y las aplicaciones móviles, permitiéndole cumplir sus metas e interactuar de forma satisfactoria con el producto. Las decisiones de navegación se alinean con la arquitectura de información y los sistemas de búsqueda previamente definidos.

Landing Page

La navegación conecta las secciones mediante accesos, menús y recorridos que permiten completar las tareas del usuario; estos criterios se aplican a landing page, como se detalla en la Tabla 49.

Tabla 49

Navigation Systems — Landing Page

Acción	Descripción
Navegación sticky	El header permanece visible al hacer scroll
Menú de anclas	Enlaces directos a secciones
Botón hamburguesa	En vista móvil, despliega el menú verticalmente
Scroll suave	Desplazamiento con compensación del header
Enlace activo	Resaltado del enlace correspondiente a la sección visible
Selector de idioma	Cambio dinámico ES/EN sin recargar la página

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

Aplicación móvil para pacientes

La navegación conecta las secciones mediante accesos, menús y recorridos que permiten completar las tareas del usuario; estos criterios se aplican a aplicación móvil para pacientes, como se detalla en la Tabla 50.

Tabla 50

Navigation Systems — Aplicación móvil para pacientes

Acción	Descripción
Barra de navegación inferior	Accesos a Inicio, Reservar, Mis citas, Familiares y Perfil
Flujo secuencial	Reserva paso a paso con retroceso y confirmación
Notificaciones push	Avisos de cupos liberados y recordatorios
Check-in por QR	Acceso rápido a la atención el día de la cita

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

Aplicación móvil para personal de salud

La navegación conecta las secciones mediante accesos, menús y recorridos que permiten completar las tareas del usuario; estos criterios se aplican a aplicación móvil para personal de salud, como se detalla en la Tabla 51.

Tabla 51

Navigation Systems — Aplicación móvil para personal de salud

Acción	Descripción
Barra de navegación inferior	Accesos a Panel, Citas, Lista de espera, Pacientes y Reportes
Filtros por fecha y especialidad	Segmentación del flujo de atención
Actualización en tiempo real	Visualización del estado de cada paciente
Reasignación de cupos	Acción directa sobre cupos liberados

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

Recorrido del usuario

1. El visitante llega al Landing Page y comprende el problema y la solución.
2. Revisa los videos y testimonios de validación.
3. Conoce el equipo y el modelo de negocio.
4. Descarga la aplicación correspondiente a su perfil.
5. En la app, navega por las secciones principales mediante la barra inferior.
6. Completa sus tareas (reservar, gestionar, consultar) con flujos claros y retroalimentación visual.

3.1.3. Landing Page UI Design

3.1.3.1. Landing Page Wireframe

Los wireframes del Landing Page fueron elaborados en **Figma** para las versiones **Desktop Web Browser** y **Mobile Web Browser**, con el objetivo de definir previamente la estructura, distribución de contenidos, jerarquía visual y navegación de la interfaz antes de desarrollar los mock-ups de alta fidelidad.

Estructura general del wireframe

La estructura del Landing Page se organizó en diez secciones principales, siguiendo un flujo orientado a presentar el problema, introducir la solución y facilitar el acceso a la aplicación:

1. **Header / Navbar:** contiene el logotipo, menú de navegación, selector de idioma ES/EN y botón principal de descarga. En dispositivos móviles, el menú se adapta mediante un botón desplegable.
2. **Hero:** presenta el mensaje principal de la solución, una descripción breve, botones de acción, beneficios principales e imagen representativa.
3. **Problema:** presenta tres situaciones identificadas durante la investigación: incertidumbre al solicitar una cita, pérdida de cupos disponibles y procesos administrativos en papel.
4. **Solución:** presenta las funcionalidades principales de las aplicaciones destinadas a pacientes y personal de salud.
5. **Videos:** incorpora contenido audiovisual relacionado con el producto y el equipo de desarrollo.
6. **Modelo de negocio:** presenta las principales modalidades de implementación y servicios asociados a la solución.
7. **Testimonios:** presenta opiniones y experiencias obtenidas de los entrevistados.
8. **Sobre nosotros:** presenta la misión, visión, valores y miembros del equipo de RuwaLabs.
9. **Descarga:** incluye los accesos correspondientes a Google Play y App Store.
10. **Footer:** contiene el logotipo, enlaces de navegación, información del proyecto, medios de contacto y derechos de autor.

Wireframe Desktop Web Browser

Las vistas del wireframe distribuyen las secciones de la landing page para su consulta desde un navegador de escritorio, como se detalla en la Tabla 52.

Tabla 52

Wireframe Desktop Web Browser

Vista	Secciones destacadas
Vista superior	Header, Hero, Problema y Solución
Vista inferior	Videos, Modelo de negocio y Testimonios

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

Figura 61

Wireframe Desktop - Vista superior

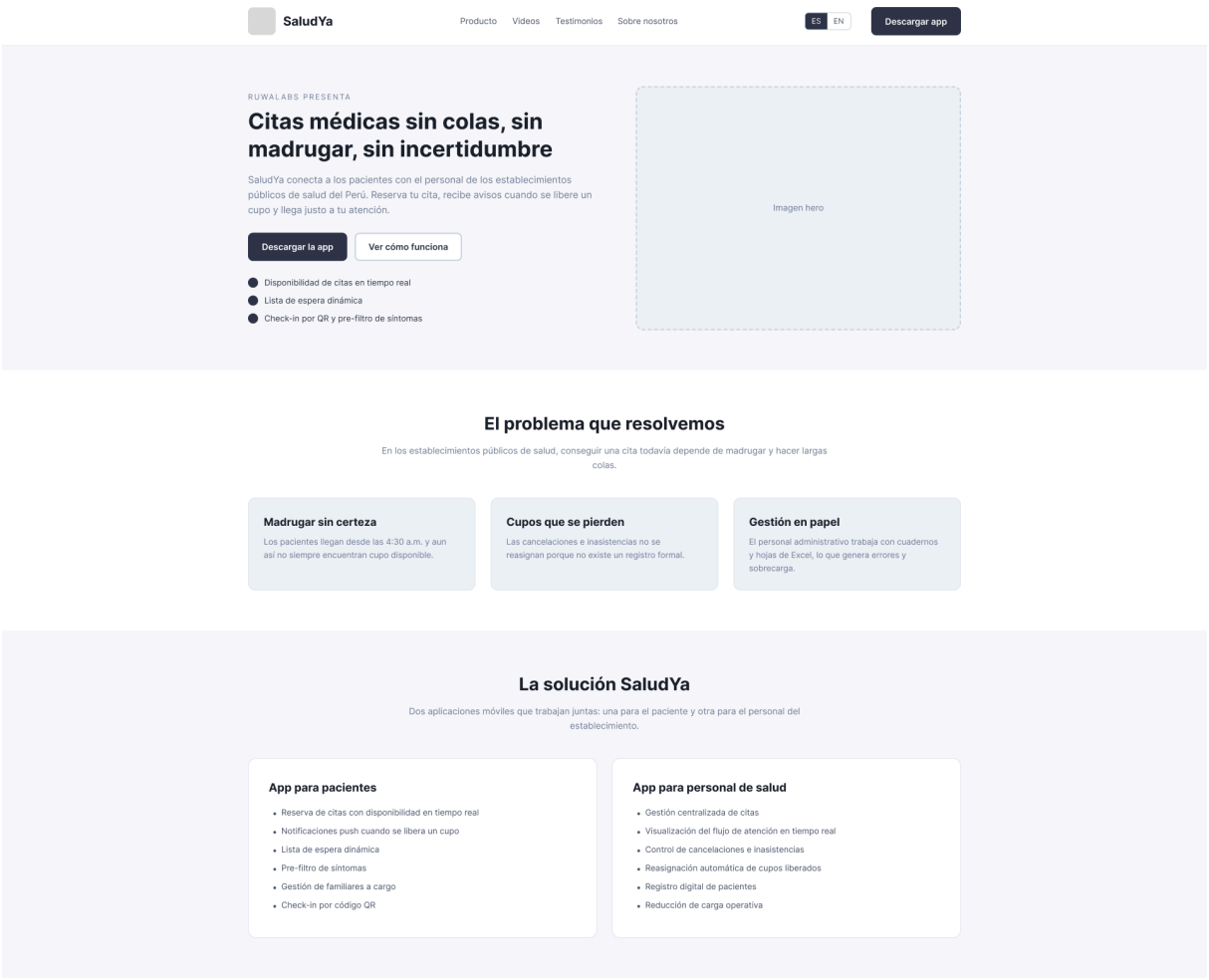


Figura 12: Wireframe Desktop - Vista superior

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

La vista inferior permite revisar la distribución de las secciones finales de la landing page en escritorio, como se observa en la Figura 61 y la Figura 62.

Figura 62

Wireframe Desktop - Vista inferior

Figura 13: Wireframe Desktop - Vista inferior

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

Wireframe Mobile Web Browser

Las vistas del wireframe distribuyen las secciones de la landing page para su consulta desde un navegador móvil, como se detalla en la Tabla 53.

Tabla 53

Wireframe Mobile Web Browser

Vista	Secciones destacadas
Vista superior	Header, Hero, Problema y Solución
Vista inferior	Videos, Modelo de negocio y Testimonios

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

Figura 63

Wireframe Mobile - Vista superior

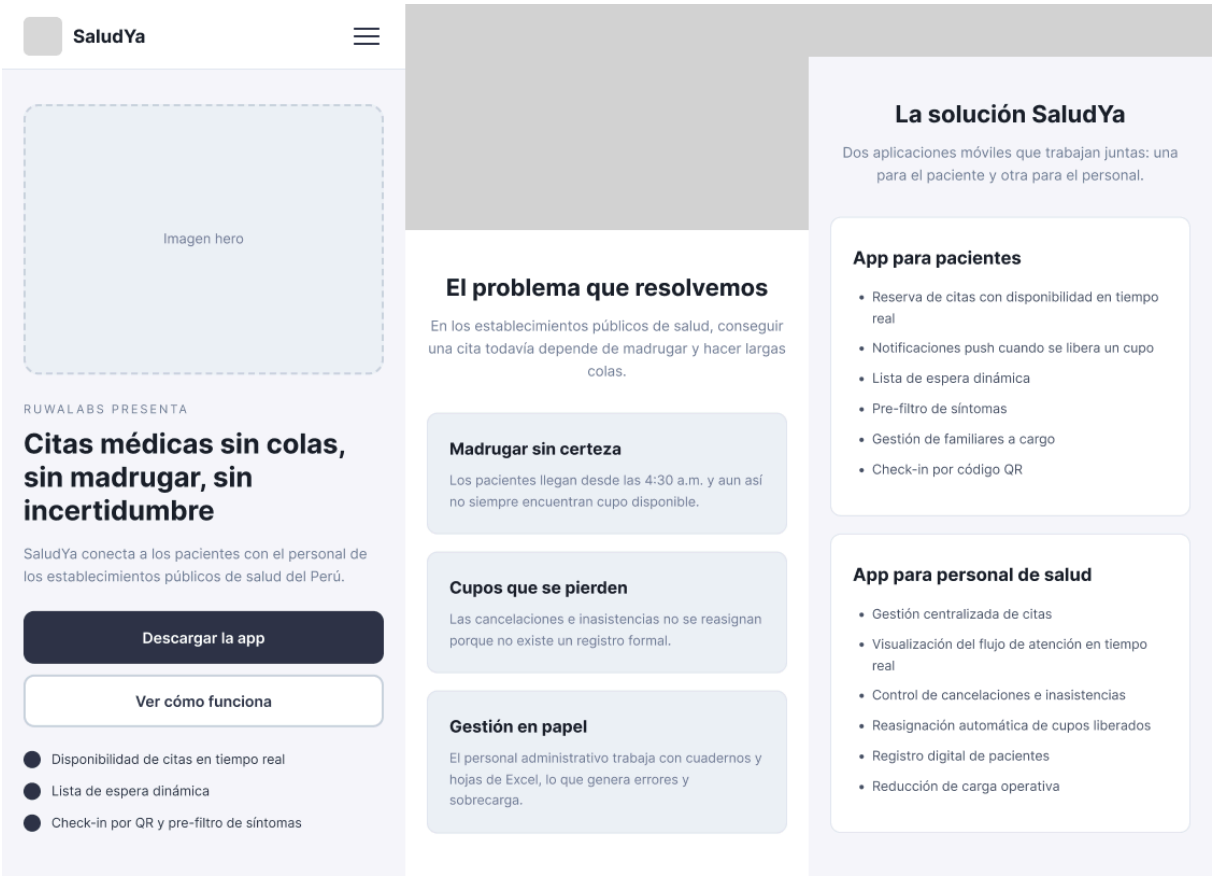


Figura 14: Wireframe Mobile - Vista superior

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

La vista inferior permite revisar la distribución de las secciones finales de la landing page en móvil, como se observa en la Figura 63 y la Figura 64.

Figura 64

Wireframe Mobile - Vista inferior

Figura 15: Wireframe Mobile - Vista inferior

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

Principios de diseño aplicados

- **Jerarquía visual:** se establecieron diferentes niveles tipográficos y variaciones de fondo para diferenciar títulos, contenidos y secciones.
- **Diseño inclusivo:** se consideraron contraste adecuado, tipografía legible, áreas de interacción apropiadas y estados de foco visibles.
- **Arquitectura de información:** la organización sigue una secuencia de problema → solución → evidencia → equipo → descarga, facilitando la comprensión progresiva del producto.
- **Consistencia visual:** se mantuvieron criterios uniformes de espaciado, tamaños, radios y componentes a lo largo de la interfaz.
- **Diseño responsive:** la estructura fue adaptada para mantener la legibilidad y funcionalidad tanto en pantallas de escritorio como en dispositivos móviles.

3.1.3.2. Landing Page Mock-up

Los mock-ups fueron desarrollados en **Figma** a partir de la estructura definida en los wireframes y aplicando el **Design System de SaludYa**. Este sistema establece los criterios visuales utilizados para mantener consistencia en colores, tipografía, espaciado, componentes e interacción.

Design System aplicado

El sistema de diseño reúne los colores, la tipografía y los criterios visuales utilizados para mantener una identidad consistente en la landing page, como se detalla en la Tabla 54.

Tabla 54

Design System aplicado

Elemento	Valor	Aplicación
Color primario	`#0b8f6b`	Botones, enlaces e iconos
Color primario oscuro	`#076e52`	Estados hover y elementos destacados
Color primario claro	`#e6f5f0`	Fondos de sección y estados hover
Color de acento	`#ffb703`	Indicadores de foco y elementos de énfasis
Texto principal	`#1c2b2a`	Títulos y contenido principal
Texto secundario	`#56706d`	Descripciones y contenido complementario
Fondos	`#ffffff` / `#f4faf8`	Fondo principal y secciones alternas
Borde	`#d8e6e2`	Tarjetas, controles y separadores
Tipografía	Segoe UI / Helvetica Neue / Arial	Títulos y contenido
Radios	6px / 12px / 20px	Tarjetas, botones y contenedores
Sombras	sm / md / lg	Jerarquía y profundidad visual

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

Mock-up Desktop Web Browser

Las vistas del mockup distribuyen las secciones de la landing page para su consulta desde un navegador de escritorio, como se detalla en la Tabla 55.

Tabla 55

Mock-up Desktop Web Browser

Vista	Secciones destacadas
Vista superior	Header, Hero, Problema y Solución
Vista inferior	Videos, Modelo de negocio y Testimonios

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

Figura 65

Mock-up Desktop - Vista superior

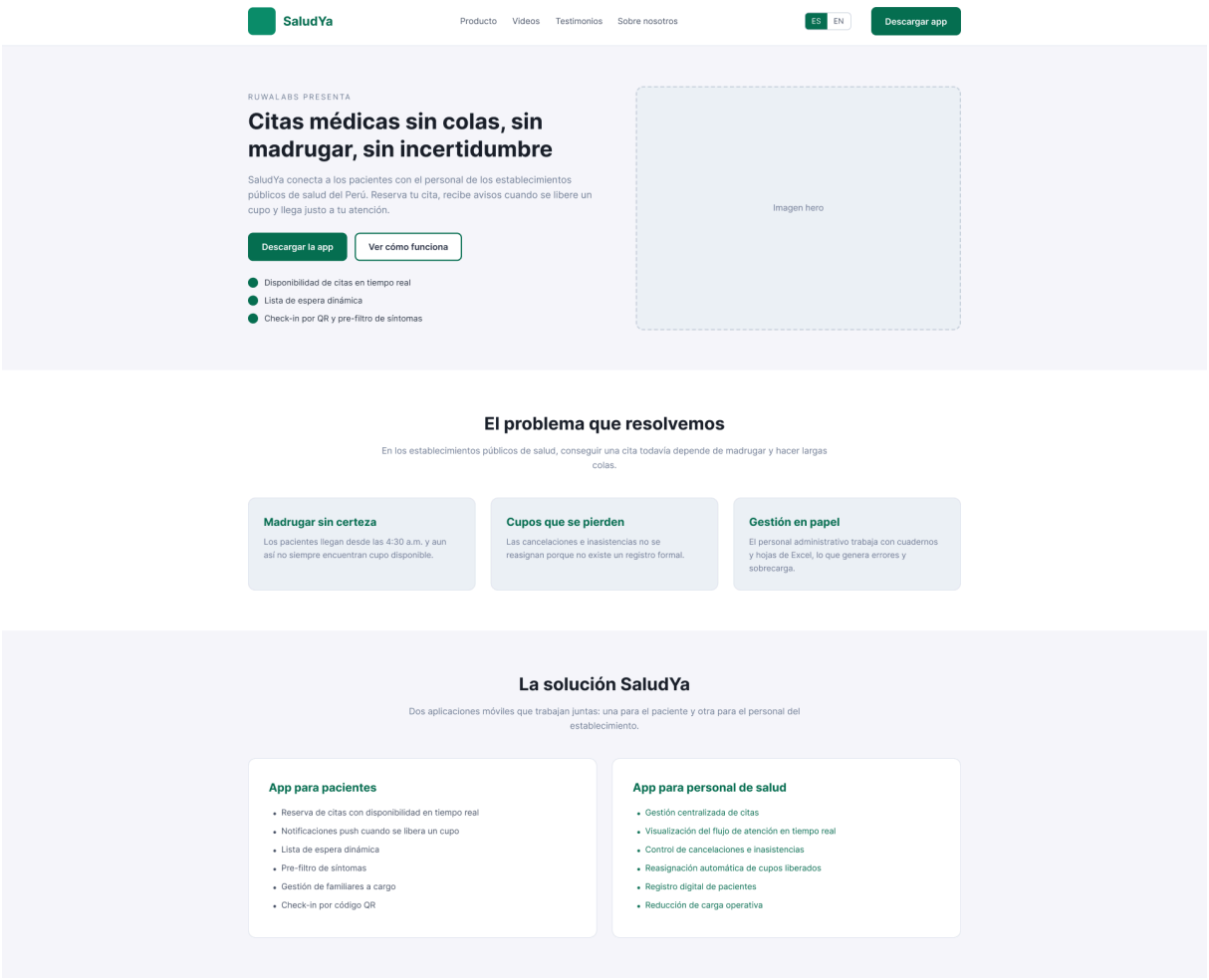


Figura 16: Mock-up Desktop - Vista superior

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

La vista inferior permite revisar la distribución de las secciones finales de la landing page en escritorio, como se observa en la Figura 65 y la Figura 66.

Figura 66

Mock-up Desktop - Vista inferior

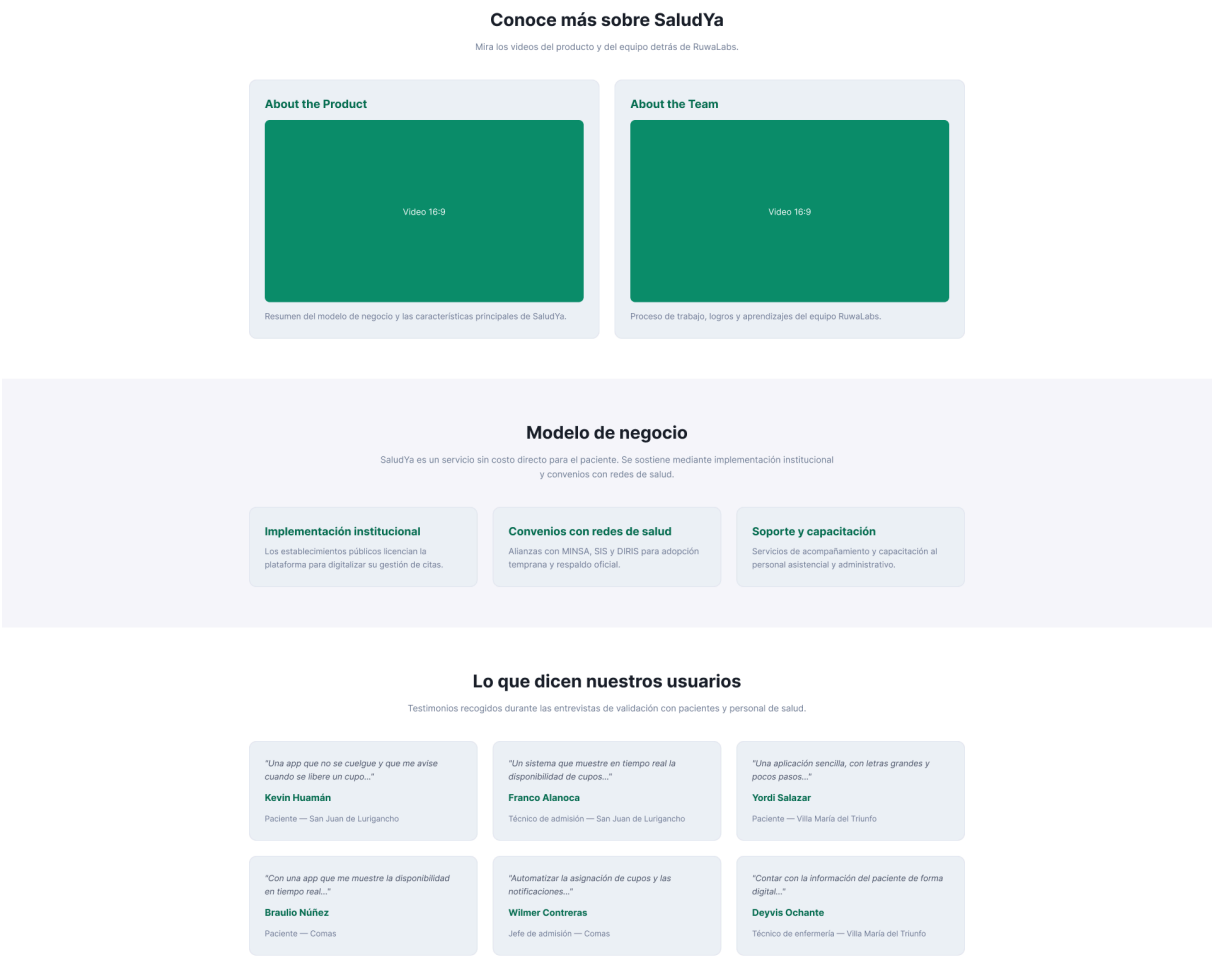


Figura 17: Mock-up Desktop - Vista inferior

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

Mock-up Mobile Web Browser

Las vistas del mockup distribuyen las secciones de la landing page para su consulta desde un navegador móvil, como se detalla en la Tabla 56.

Tabla 56

Mock-up Mobile Web Browser

Vista	Secciones destacadas
Vista superior	Header, Hero, Problema y Solución
Vista inferior	Videos, Modelo de negocio y Testimonios

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

Figura 67

Mock-up Mobile - Vista superior

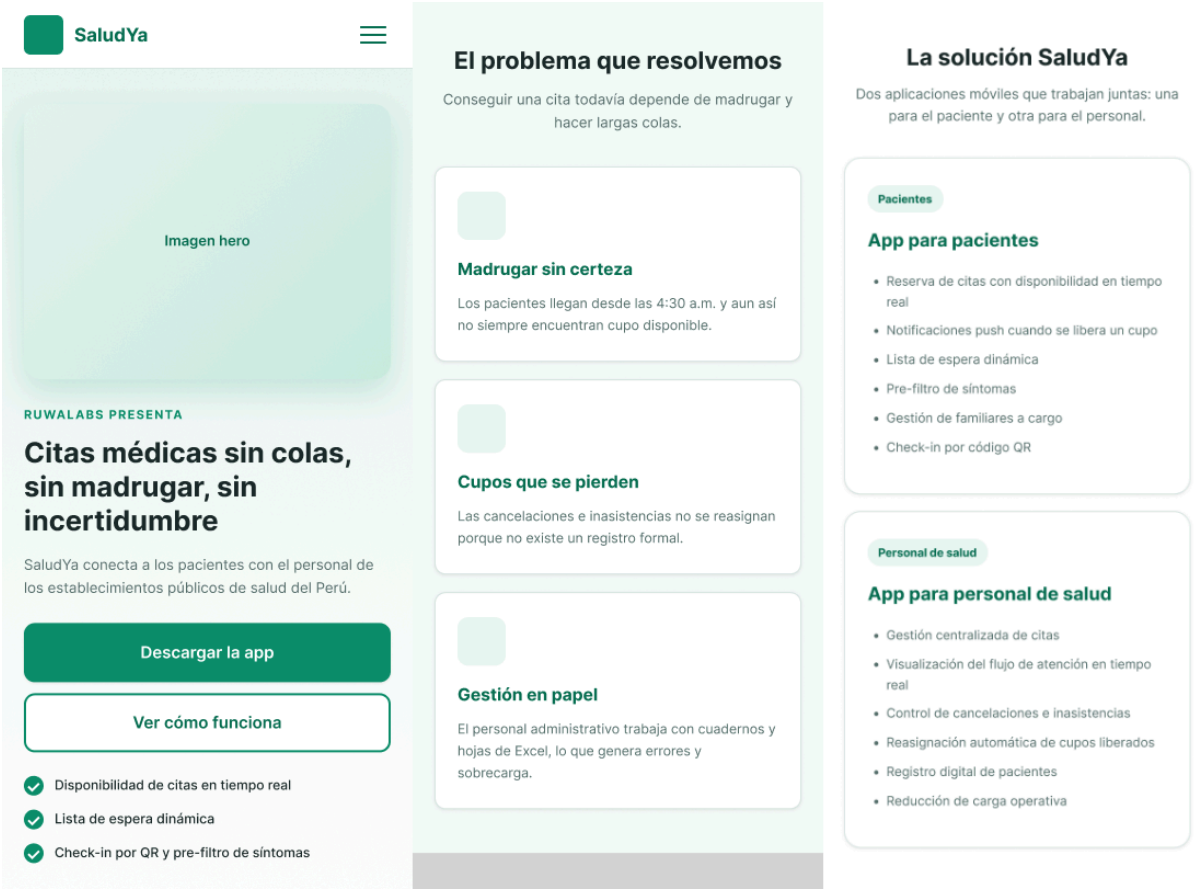


Figura 18: Mock-up Mobile - Vista superior

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

La vista inferior permite revisar la distribución de las secciones finales de la landing page en móvil, como se observa en la Figura 67 y la Figura 68.

Figura 68

Mock-up Mobile - Vista inferior

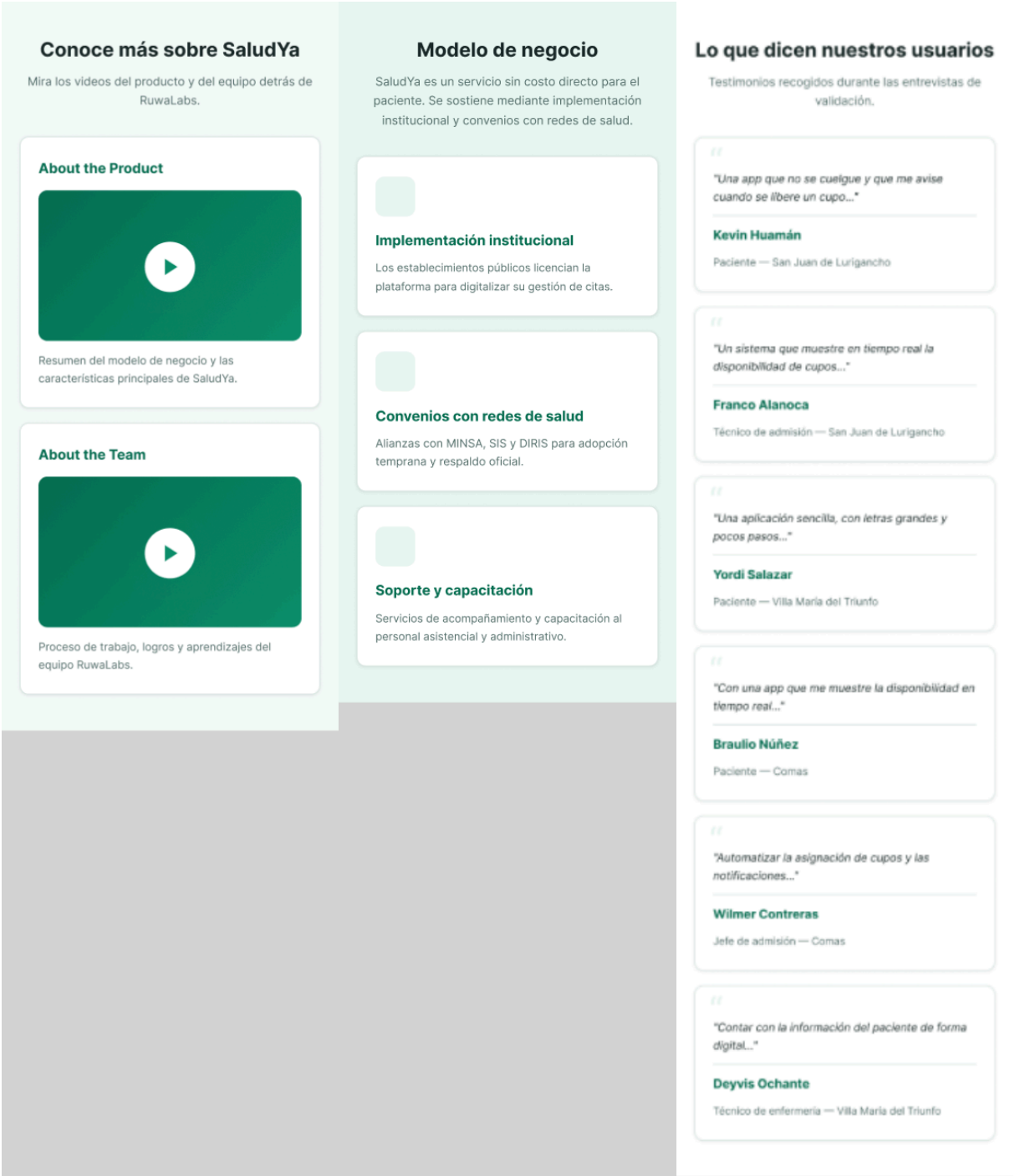


Figura 19: Mock-up Mobile - Vista inferior

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

Aplicación del Design System y diseño inclusivo

Los criterios de diseño combinan la identidad visual de SaludYa con decisiones de legibilidad y accesibilidad para facilitar el uso de la interfaz, como se detalla en la Tabla 57.

Tabla 57

Aplicación del Design System y diseño inclusivo

Criterio	Aplicación
Branding	Uso consistente del logotipo y de la paleta cromática de SaludYa
Tipografía	Jerarquía visual clara, tamaño legible e interlineado adecuado
Colores	Contraste adecuado para facilitar la lectura y diferenciación de acciones
Espaciado	Escala uniforme de márgenes, rellenos y separación entre componentes
Diseño inclusivo	Etiquetas accesibles, textos alternativos, áreas de interacción adecuadas y soporte para reducción de movimiento
Internacionalización	Selector ES/EN para adaptar dinámicamente los contenidos
Navegación	Menú sticky y desplazamiento suave entre las diferentes secciones
Responsive	Adaptación de estructura, componentes y contenidos a diferentes tamaños de pantalla

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

Componentes reutilizables

Los componentes reutilizables definen la apariencia y los estados de botones, tarjetas y controles para mantener su comportamiento consistente entre pantallas, como se detalla en la Tabla 58.

Tabla 58

Componentes reutilizables

Componente	Descripción	Estados
Botón primario	Botón de acción principal con fondo verde y texto blanco	Default, hover, focus
Botón secundario	Botón con fondo transparente y borde del color primario	Default, hover, focus
Tarjeta	Contenedor para información con borde y sombra	Default, hover
Testimonio	Tarjeta destinada a mostrar opiniones y citas de usuarios	Default
Miembro del equipo	Componente con fotografía, nombre y rol	Default, hover
Selector de idioma	Control para alternar entre español e inglés	Activo, inactivo
Menú de navegación	Control responsive para mostrar u ocultar las opciones de navegación	Cerrado, abierto

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

Landing Page implementado

El diseño definido en los wireframes y mock-ups fue posteriormente trasladado a una implementación funcional. Esta versión permite visualizar la aplicación de los lineamientos establecidos en el Design System y comprobar la adaptación de la interfaz a diferentes tamaños de pantalla, como se observa en la Figura 69.

Figura 69

Landing Page de SaludYa - Implementación

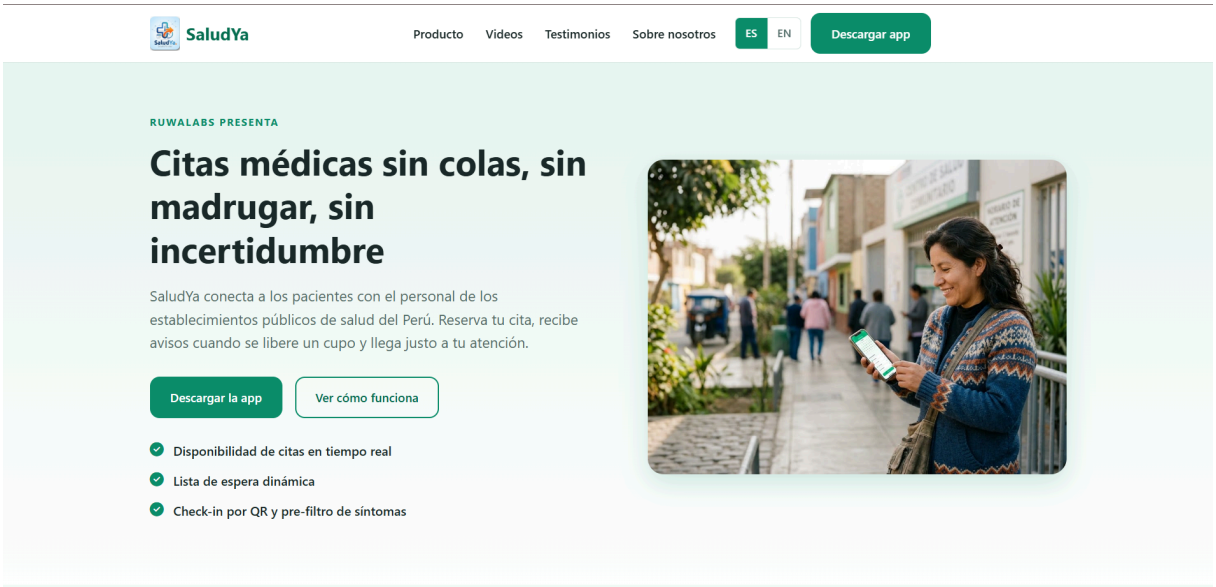


Figura 20: Landing Page de SaludYa - Implementación

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

Landing Page de SaludYa:

https://ruwalabs.github.io/saludya-landing/

Conclusión de la sección

Los wireframes y mock-ups del Landing Page evidencian la aplicación coherente del Design System, los principios de diseño inclusivo y la arquitectura de información. La propuesta comunica el modelo de negocio de RuwaLabs, el problema que resuelve SaludYa y los beneficios para ambos segmentos objetivo, facilitando la conversión del visitante hacia la descarga de las aplicaciones móviles. La inclusión de internacionalización y accesibilidad garantiza una experiencia inclusiva y consistente.

3.1.4. Mobile Applications UX/UI Design

3.1.4.1. Mobile Applications Wireframes

En esta sección se presenta la propuesta de wireframes de las aplicaciones móviles de **SaludYa**, elaborada en **Figma**. Los wireframes son de **baja fidelidad** y se construyeron con la escala de grises definida para esta etapa (de **WF 900** a **WF White**), de modo que la atención se concentre en la estructura, la jerarquía visual y la disposición de los elementos de cada pantalla, y no en el color ni en el acabado final. Esta decisión permite validar la organización del contenido antes de elaborar los mock-ups de alta fidelidad del apartado 3.1.4.3.

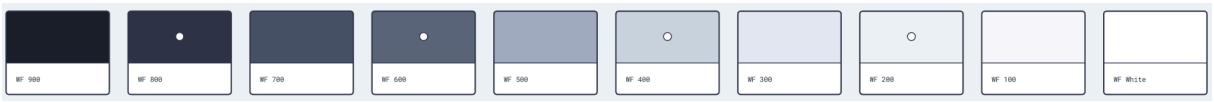
La propuesta traduce las decisiones de arquitectura de información definidas en el apartado 3.1.2 (sistemas de organización, etiquetado, búsqueda y navegación) y evidencia la aplicación de principios y elementos de diseño, así como de criterios de **diseño inclusivo**: contraste suficiente entre el contenido y las superficies, jerarquía tipográfica clara, áreas táctiles amplias y etiquetas simples y libres de ambigüedad. Los wireframes cubren las **dos aplicaciones del alcance** (pacientes y personal de salud) y los diálogos transversales. El archivo de trabajo se encuentra disponible en Mobile Application Wireframes.

Estilo de los wireframes

Los wireframes emplean una escala de grises de baja fidelidad que permite distinguir jerarquías, superficies y estados sin depender del color, manteniendo el foco en la estructura y en la organización del contenido. La escala abarca desde el tono más oscuro (WF 900) hasta el blanco (WF White), como se observa en la Figura 70.

Figura 70

Escala de grises para wireframes



Nota. Elaboración propia.

Aplicación móvil para pacientes

La aplicación para pacientes se organiza en torno a la barra de navegación inferior (**Inicio, Reservar cita, Asistencia y Configuración**), en línea con el sistema de navegación descrito en el apartado 3.1.2.5. Las vistas siguientes cubren el acceso, el inicio, la gestión de citas, la reserva, la asistencia, las notificaciones y la configuración.

El acceso a la aplicación reúne la pantalla de bienvenida, con las acciones **Iniciar Sesión** y **Registrarse**, y el formulario de **Inicio de Sesión** con tipo y número de documento y contraseña, como se observa en la Figura 71.

Figura 71

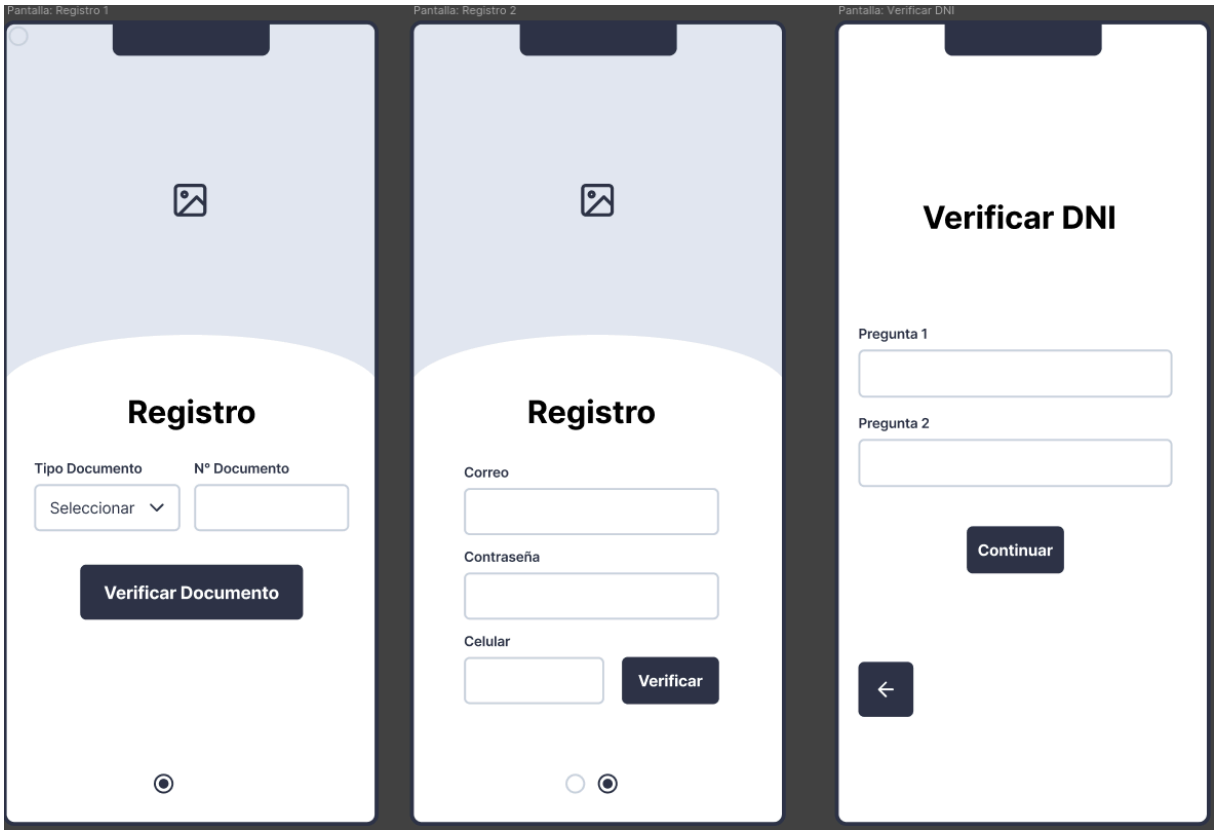
Bienvenida e inicio de sesión

Nota. Elaboración propia.

El registro se compone de la verificación de identidad por documento, el alta de credenciales (correo, contraseña y celular) y la verificación del DNI mediante preguntas de seguridad, como se observa en la Figura 72.

Figura 72

Registro y verificación de identidad

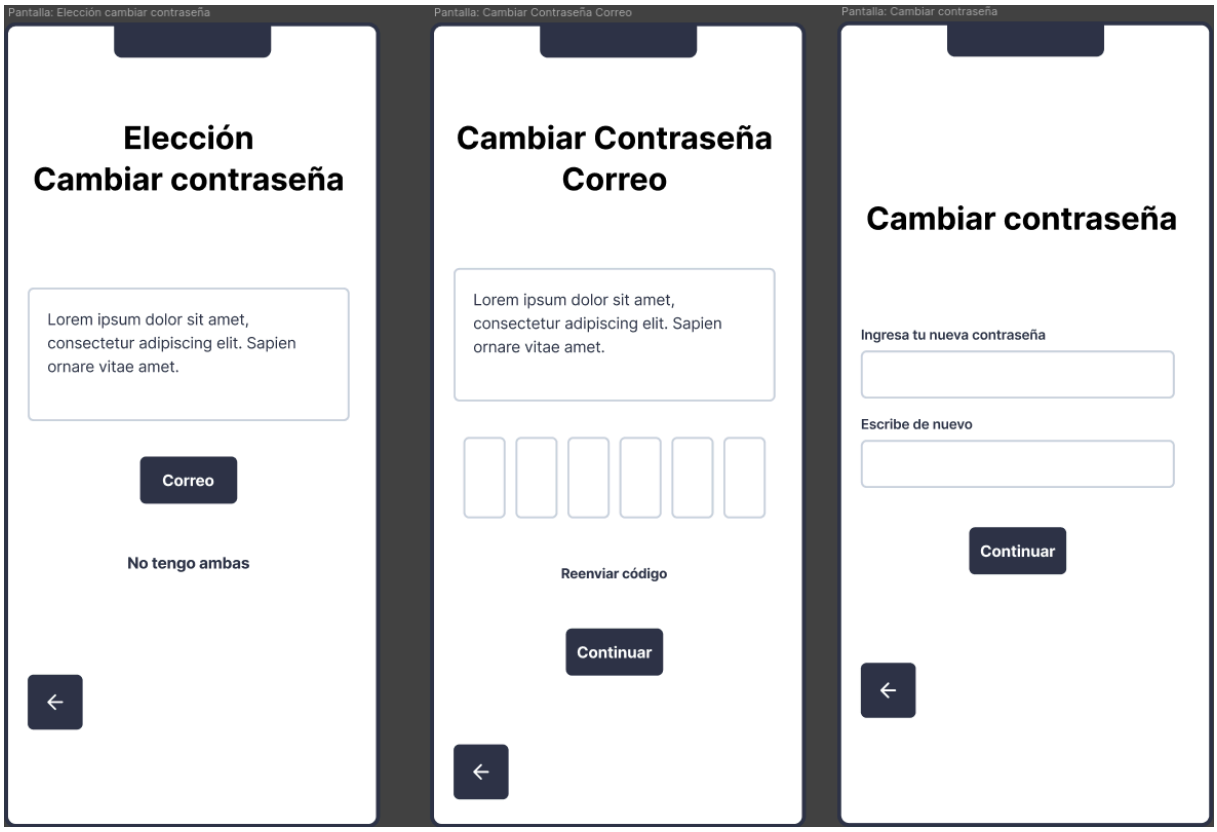


Nota. Elaboración propia.

La recuperación de acceso permite elegir el restablecimiento por correo, ingresar el código enviado y definir una nueva contraseña con su confirmación, como se observa en la Figura 73.

Figura 73

Recuperación y cambio de contraseña



Nota. Elaboración propia.

El inicio del paciente saluda al usuario, presenta las citas pendientes y el historial, y ofrece el acceso a las notificaciones y a las secciones principales mediante la barra inferior, como se observa en la Figura 74.

Figura 74

Inicio del paciente

Nota. Elaboración propia.

La vista de citas pendientes lista las reservas activas del paciente en tarjetas, con la cabecera de retroceso y la barra de navegación inferior, como se observa en la Figura 75.

Figura 75

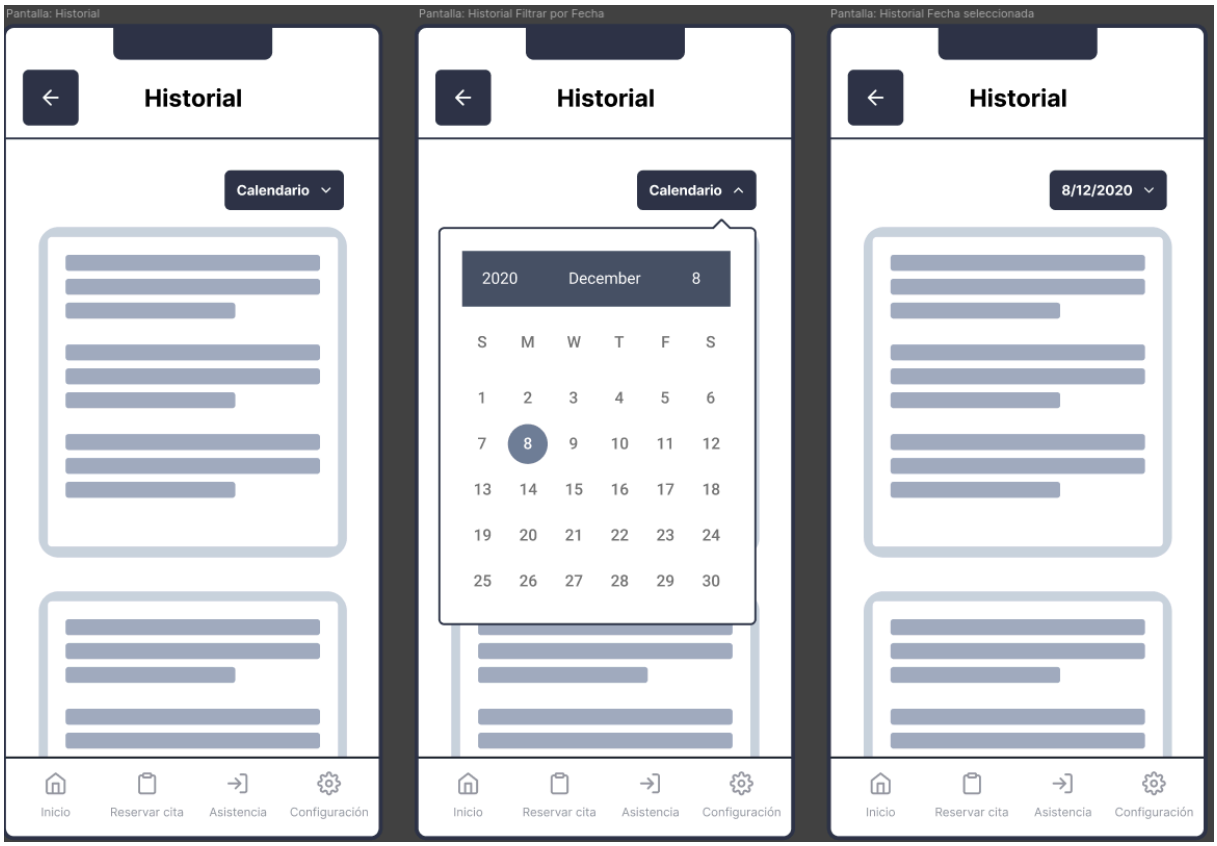
Citas pendientes

Nota. Elaboración propia.

El historial presenta las atenciones pasadas y permite filtrarlas por fecha mediante un selector de calendario, como se observa en la Figura 76.

Figura 76

Historial y filtro por fecha



Nota. Elaboración propia.

La reserva de cita permite buscar y elegir la especialidad, seleccionar la fecha en un calendario y escoger la hora entre los bloques disponibles, como se observa en la Figura 77.

Figura 77

Reserva de cita

Nota. Elaboración propia.

Cuando el paciente reserva por profesional, la interfaz permite buscar y elegir al doctor y luego seleccionar la hora, como se observa en la Figura 78.

Figura 78

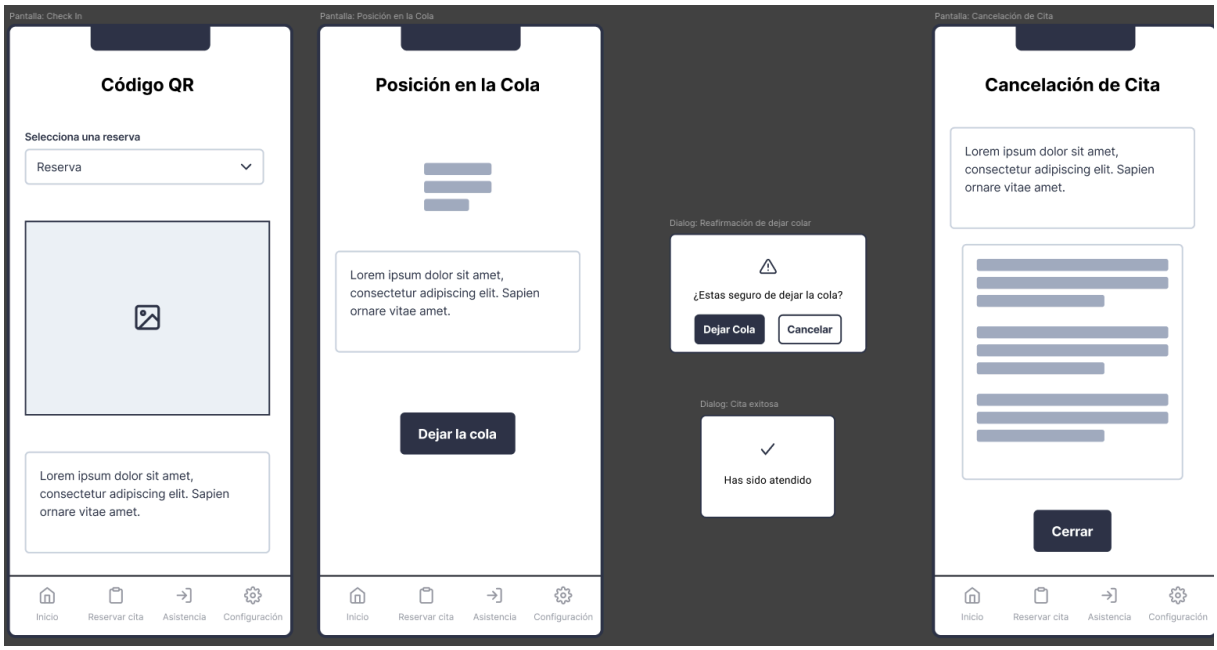
Reserva por doctor

Nota. Elaboración propia.

La asistencia reúne el registro de llegada mediante **código QR** y la consulta de la posición en la cola, junto con los diálogos de confirmación para dejar la cola y de atención finalizada, así como la cancelación de la cita, como se observa en la Figura 79.

Figura 79

Asistencia: check-in y cola

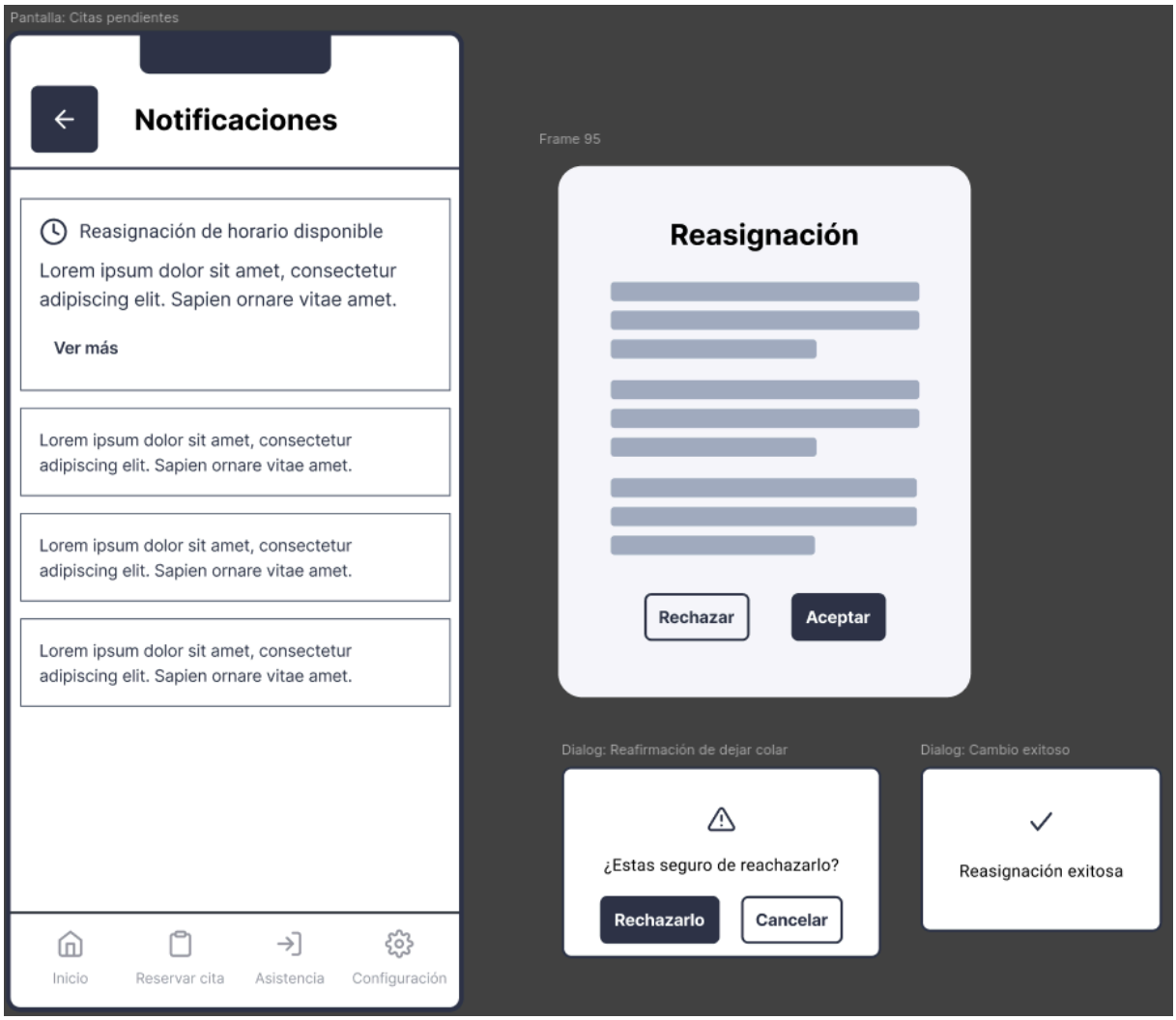


Nota. Elaboración propia.

Las notificaciones listan los avisos del paciente, entre ellos la disponibilidad de una reasignación de horario, que se resuelve en un modal con las acciones **Rechazar** y **Aceptar**, como se observa en la Figura 80.

Figura 80

Notificaciones y reasignación

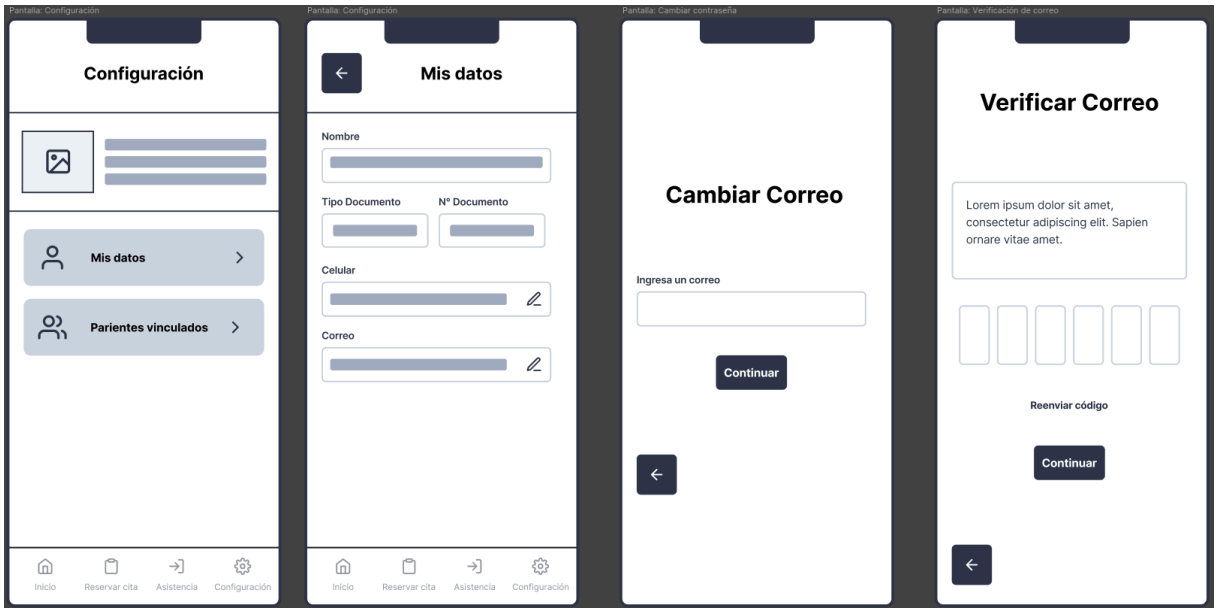


Nota. Elaboración propia.

La configuración permite acceder a los datos personales y a los pacientes vinculados, y el cambio de correo se completa verificando el nuevo correo mediante un código, como se observa en la Figura 81.

Figura 81

Configuración y cambio de correo



Nota. Elaboración propia.

La gestión de pacientes vinculados permite agregar y sincronizar menores, registrar un menor verificando su documento y administrar el vínculo desde el detalle, como se observa en la Figura 82.

Figura 82

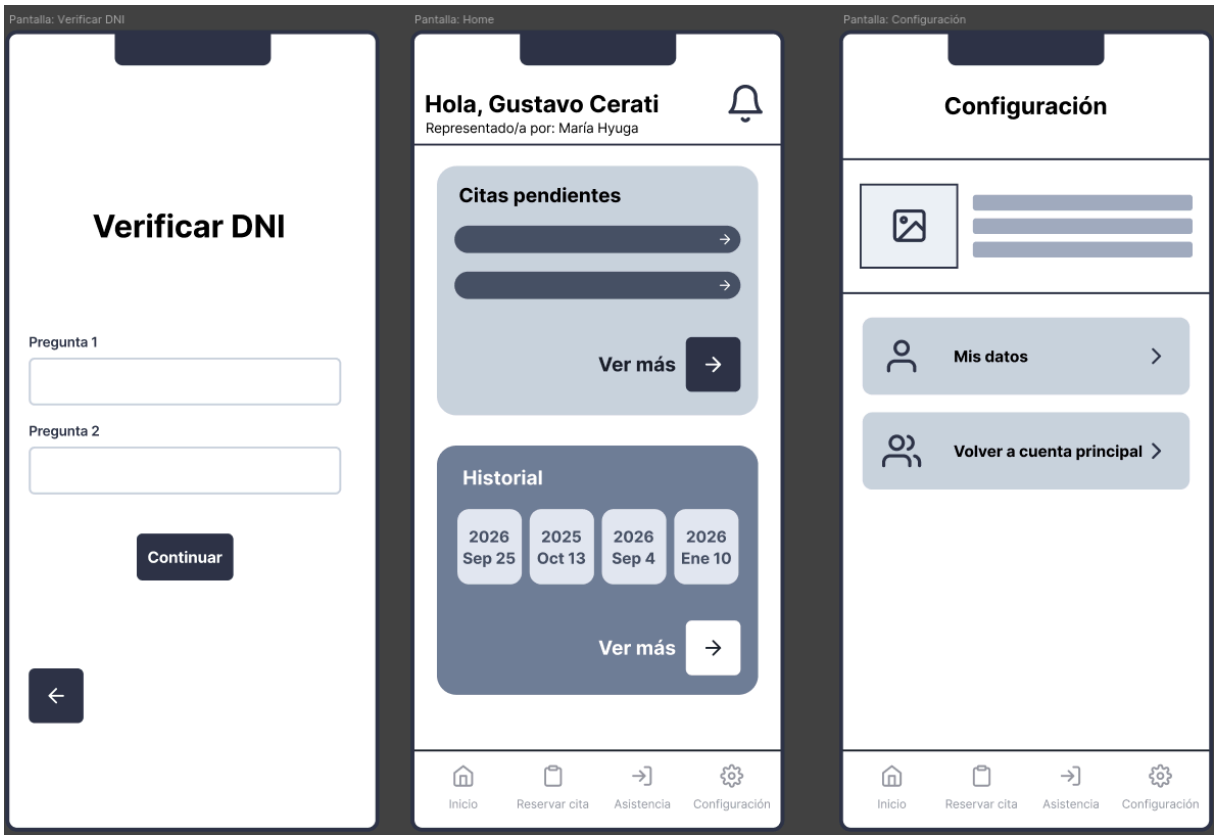
Pacientes vinculados y registro de menor

Nota. Elaboración propia.

El acceso al perfil de un menor incluye la verificación del vínculo, su propia vista de inicio y la opción de volver a la cuenta principal, como se observa en la Figura 83.

Figura 83

Perfil del menor vinculado



Nota. Elaboración propia.

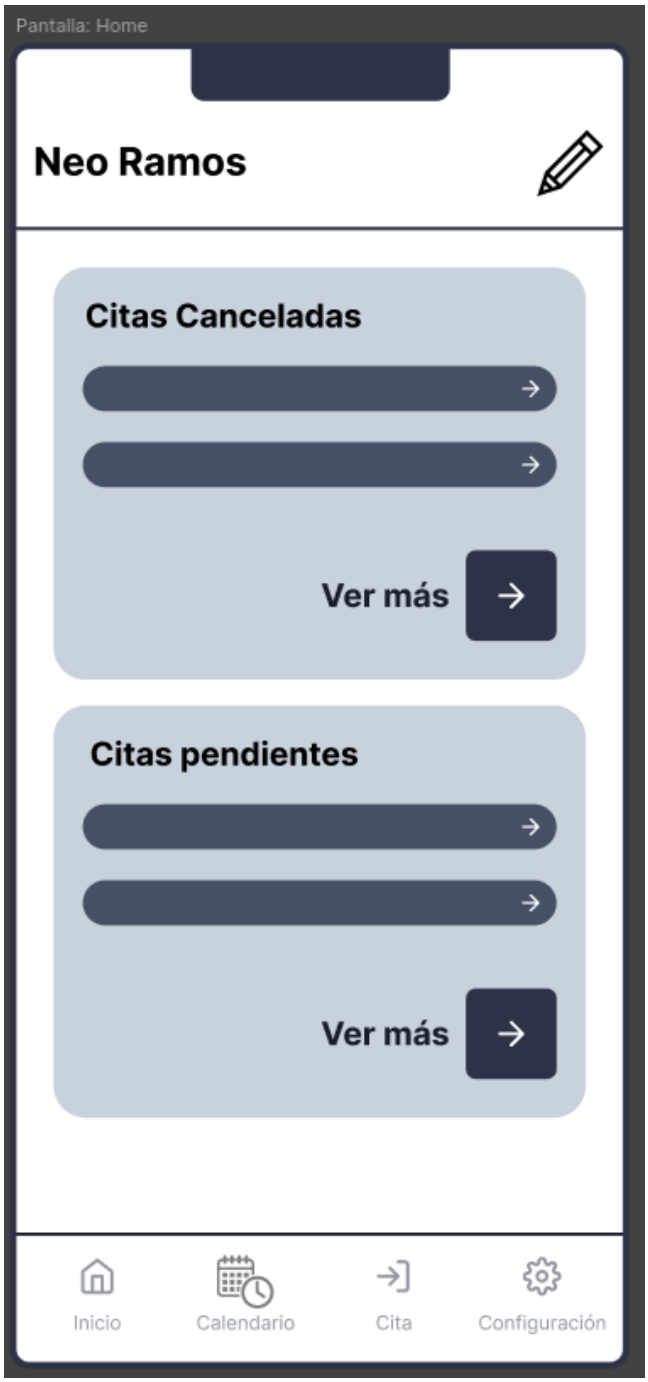
Aplicación móvil del personal de salud

La aplicación para el personal de salud se organiza en la barra inferior **Inicio, Calendario, Cita y Configuración**, y prioriza la gestión operativa del establecimiento.

El inicio del personal presenta el resumen del día con las citas canceladas y pendientes, junto con el acceso a la edición del perfil, como se observa en la Figura 84.

Figura 84

Inicio del personal de salud



Nota. Elaboración propia.

El calendario permite elegir la especialidad, la fecha y la hora para gestionar la agenda del establecimiento, como se observa en la Figura 85.

Figura 85

Calendario: especialidad, fecha y hora

Nota. Elaboración propia.

La vista por profesional permite buscar y seleccionar al doctor y luego elegir la hora de atención, como se observa en la Figura 86.

Figura 86

Calendario por doctor

Nota. Elaboración propia.

La edición de un bloque horario reúne el estado, la hora de inicio, la especialidad, el doctor y los pacientes con reserva, con la acción de editar y la confirmación de cambios, como se observa en la Figura 87.

Figura 87

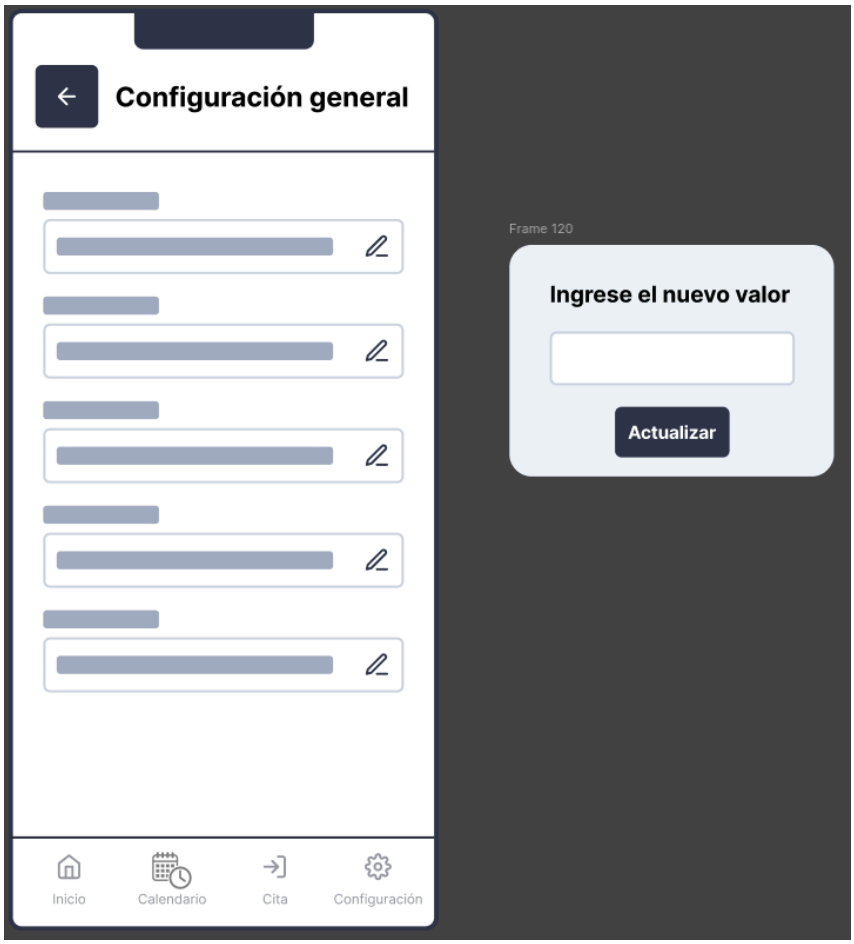
Edición de bloque horario

Nota. Elaboración propia.

La configuración general permite revisar y actualizar los parámetros operativos mediante campos editables y un diálogo para ingresar el nuevo valor, como se observa en la Figura 88.

Figura 88

Configuración general



Nota. Elaboración propia.

Diálogos y componentes

Los componentes transversales normalizan la confirmación de acciones y la comunicación de estados del sistema en ambas aplicaciones.

El diálogo de confirmación de cita solicita validar la reserva antes de registrarla y comunica el resultado exitoso, como se observa en la Figura 89.

Figura 89

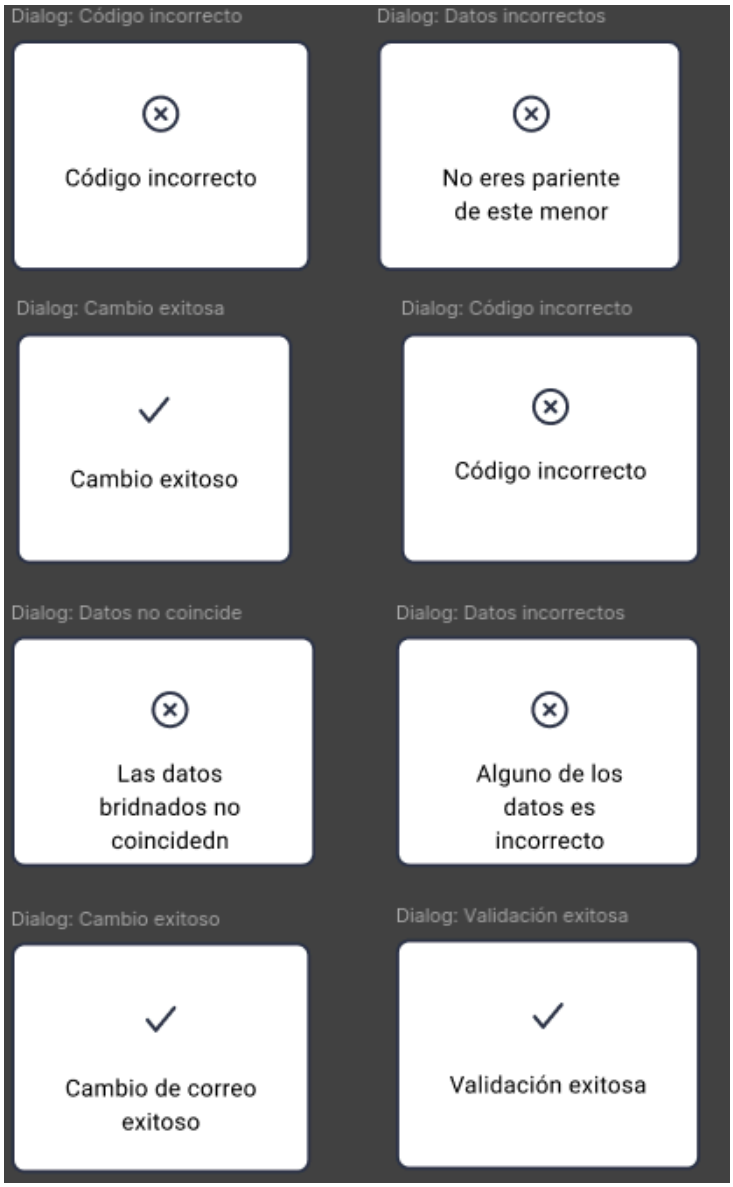
Diálogos de confirmación de cita

Nota. Elaboración propia.

Los diálogos de configuración comunican errores de validación (código incorrecto, datos incorrectos o inconsistencia de datos) y confirmaciones de éxito, como se observa en la Figura 90.

Figura 90

Diálogos de estado y validación



Nota. Elaboración propia.

3.1.4.2. Mobile Applications Wireflow Diagrams

En esta sección se presenta la propuesta de Wireflow Diagrams de las aplicaciones móviles de SaludYa, elaborada a partir de los Task Flows consensuados por el equipo. Cada Wireflow combina la secuencia de pasos de un User Goal con los wireframes de las pantallas que intervienen en el flujo; toda interacción que modifica el estado de una pantalla se representa agregando un paso con un wireframe del nuevo estado (validaciones, errores, estados vacíos y diálogos de confirmación). Se consideran las dos aplicaciones del alcance: la aplicación para pacientes (*User Persona*: Kevin Huamán) y la aplicación para el personal de admisión (*User Persona*: Franco Alanoca). Los wireflows se elaboraron en Figma y su orden responde a los User Goals priorizados para cada aplicación.

Wireflows de Pacientes

WF-01 · Registrarme en la aplicación

User Goal: Como paciente, quiero crear mi cuenta para reservar y gestionar mis citas desde el celular.

Figura 91

Wireflow de registro en la aplicación (paciente)

Nota. Elaboración propia.

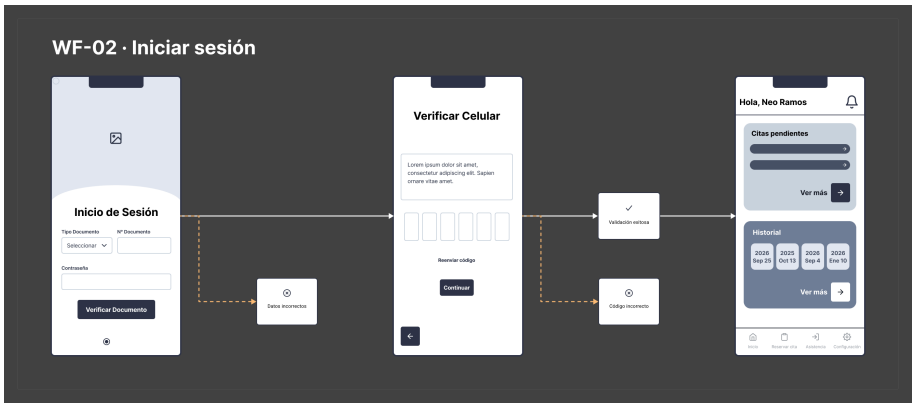
Explicación del flujo: En la Figura 91 se observa que el recorrido va de Bienvenida a Registro (tipo y número de documento) y luego a Verificar DNI, con diálogos de datos incorrectos o validación exitosa. Después sigue Registro de credenciales, Verificación de celular con código (incorrecto o exitoso) y, al final, Inicio de sesión.

WF-02 · Iniciar sesión

User Goal: Como paciente, quiero iniciar sesión para acceder a mi información y a mis citas.

Figura 92

Wireflow de inicio de sesión (paciente)



Nota. Elaboración propia.

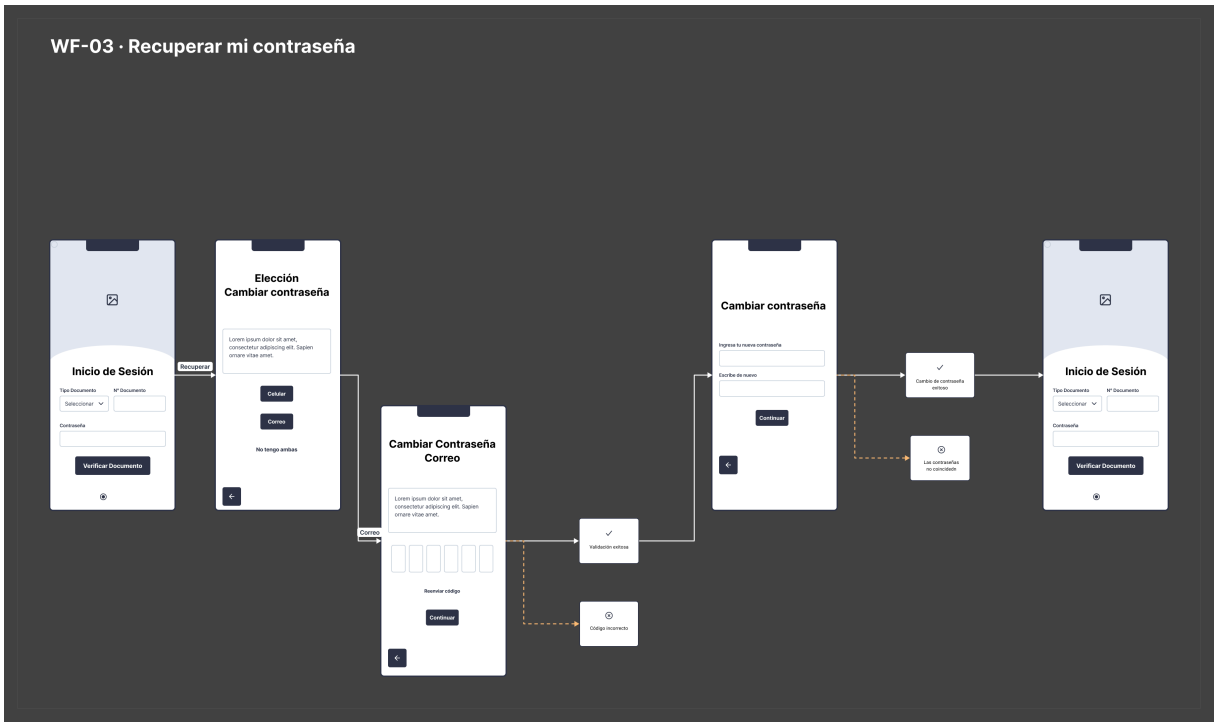
Explicación del flujo: En la Figura 92 se aprecia que, desde Inicio de sesión, si las credenciales son inválidas aparece un diálogo de error. Si son correctas, pasa a la verificación con código (segundo factor), con diálogo de código incorrecto o exitoso, y entra al Home.

WF-03 · Recuperar mi contraseña

User Goal: Como paciente, quiero recuperar el acceso cuando olvido mi contraseña.

Figura 93

Wireflow de recuperación de contraseña (paciente)



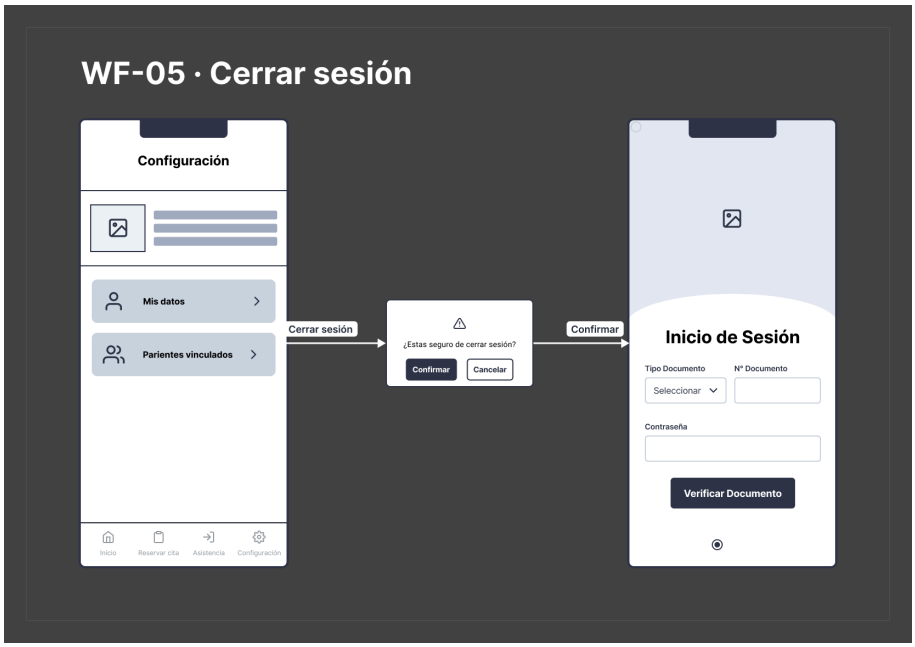
Nota. Elaboración propia.

Explicación del flujo: En la Figura 93 se muestra que, desde Inicio de sesión, se abre la elección de método (celular o correo) y se verifica el código, con camino alterno de código incorrecto. Luego se define la nueva contraseña (diálogo si no coincide o si el cambio fue exitoso) y se vuelve al Inicio de sesión.

WF-05 · Cerrar sesión
User Goal: Como paciente, quiero cerrar sesión de forma segura.

Figura 94

Wireflow de cierre de sesión (paciente)



Nota. Elaboración propia.

Explicación del flujo: En la Figura 94 se aprecia que, desde Configuración, se confirma el cierre y se vuelve al Inicio de sesión. El diálogo de confirmación aún no tiene wireframe, por eso aparece como recuadro pendiente.

WF-07 · Reservar una cita médica
User Goal: Como paciente, quiero reservar una cita eligiendo especialidad, fecha y horario.

Figura 95

Wireflow de reserva de una cita médica (paciente)

Nota. Elaboración propia.

Explicación del flujo: En la Figura 95 se observa que el flujo va de Home a Reservar cita (búsqueda de especialidad), luego Elige tu fecha, Elige la hora, Elige al doctor y la hora con ese doctor. Termina en Confirmar cita y el diálogo «Cita confirmada».

WF-08 · Consultar mis citas y su detalle

User Goal: Como paciente, quiero ver mis próximas citas y su detalle.

Figura 96

Wireflow de consulta de citas y detalle (paciente)

Nota. Elaboración propia.

Explicación del flujo: En la Figura 96 se observa que, desde Home, «Ver más» abre Citas pendientes, y al seleccionar una cita se abre su detalle con las acciones disponibles.

WF-09 · Revisar el historial de citas

User Goal: Como paciente, quiero revisar mis atenciones anteriores.

Figura 97

Wireflow del historial de citas (paciente)

Nota. Elaboración propia.

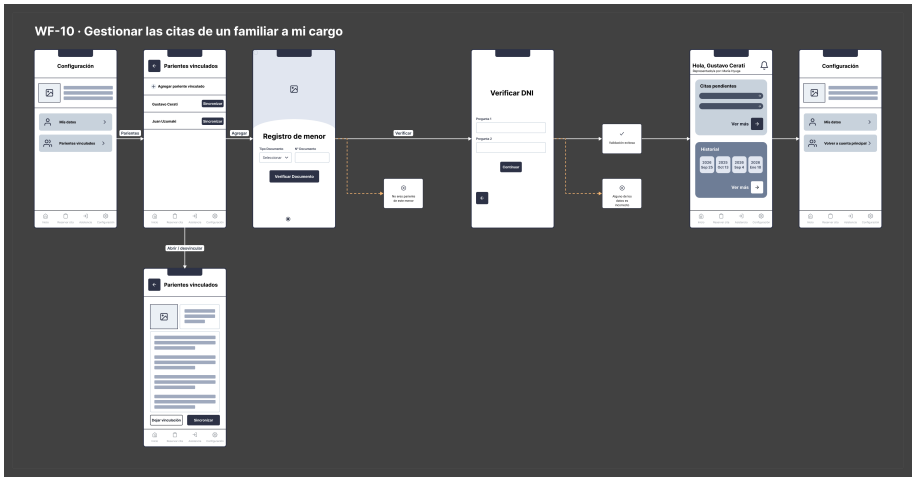
Explicación del flujo: En la Figura 97 se observa que, desde Home, se abre Historial, se despliega el calendario de filtro por fecha y se muestra el historial acotado a la fecha elegida.

WF-10 · Gestionar las citas de un familiar a mi cargo

User Goal: Como paciente, quiero vincular a un familiar dependiente y gestionar sus citas desde mi cuenta.

Figura 98

Wireflow de gestión de citas de un familiar a cargo (paciente)



Nota. Elaboración propia.

Explicación del flujo: En la Figura 98 se observa que, desde Configuración, se abre Parientes vinculados y se agrega un menor (Registro de menor). Luego viene Verificar DNI, con diálogos de «No eres pariente», datos incorrectos o validación exitosa, y se abre el Home del menor, desde donde se puede volver a la cuenta principal. Como camino alterno, se abre un pariente ya vinculado para desvincularlo.

WF-11 · Revisar mis notificaciones

User Goal: Como paciente, quiero revisar mis avisos para no perder citas ni cupos liberados.

Figura 99

Wireflow de revisión de notificaciones (paciente)

Nota. Elaboración propia.

Explicación del flujo: En la Figura 99 se observa que, desde el ícono de campana del Home, se abre el buzón de Notificaciones y, al seleccionar un aviso, se abre su detalle (por ejemplo, la oferta de reasignación).

WF-12 · Registrar mi llegada y seguir mi turno

User Goal: Como paciente, quiero registrar mi llegada y saber cuándo me atenderán sin hacer cola.

Figura 100

Wireflow de registro de llegada y seguimiento del turno (paciente)

Nota. Elaboración propia.

Explicación del flujo: En la Figura 100 se observa que, desde Asistencia, se elige la reserva y se registra con el código QR. Luego se muestra la Posición en la cola y se llega al aviso «Has sido atendido». Como camino alterno, se puede dejar la cola con un diálogo de confirmación.

WF-13 · Aceptar o rechazar un cupo liberado

User Goal: Como paciente, quiero aprovechar un cupo liberado por una cancelación.

Figura 101

Wireflow de aceptación o rechazo de un cupo liberado (paciente)

Nota. Elaboración propia.

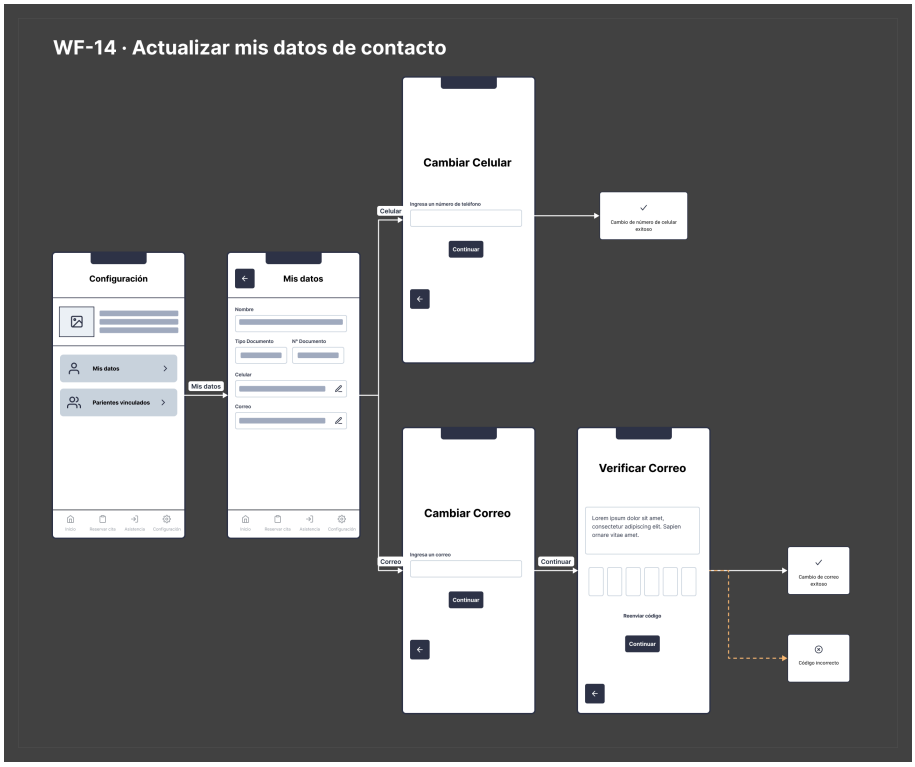
Explicación del flujo: En la Figura 101 se observa que, desde la notificación, se abre el detalle de la oferta. Si acepta, aparece «Reasignación exitosa». Si rechaza, aparece un diálogo de confirmación del rechazo.

WF-14 · Actualizar mis datos de contacto

User Goal: Como paciente, quiero mantener actualizados mi celular y mi correo.

Figura 102

Wireflow de actualización de datos de contacto (paciente)



Nota. Elaboración propia.

Explicación del flujo: En la Figura 102 se observa que, desde Configuración, se abre Mis datos y se elige cambiar celular o correo. Se edita el dato y se verifica con código en el nuevo contacto (con camino alterno de código incorrecto), y el cambio queda confirmado con un diálogo de éxito.

Wireflows de Healthcare Administrators

S01 · Registrarme como personal de admisión

User Goal: Como personal de admisión, quiero que se habilite mi cuenta con el correo corporativo y definir mi contraseña.

Figura 103

Wireflow de registro del personal de admisión

Nota. Elaboración propia.

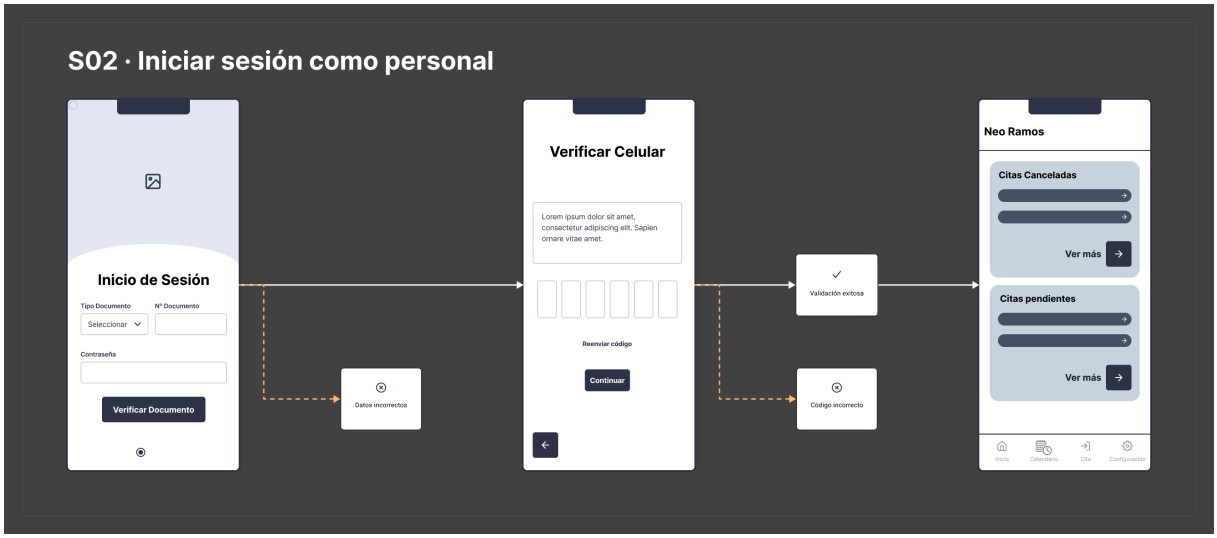
Explicación del flujo: En la Figura 103 se observa que el administrador crea la cuenta y envía una invitación (paso pendiente de wireframe). El personal abre el enlace, define su contraseña (diálogo si no coincide o si fue exitoso) y pasa al Inicio de sesión.

S02 · Iniciar sesión como personal

User Goal: Como personal de admisión, quiero iniciar sesión para acceder a la gestión de citas y a la sala de espera.

Figura 104

Wireflow de inicio de sesión del personal de admisión



Nota. Elaboración propia.

Explicación del flujo: En la Figura 104 se observa que va de Inicio de sesión (diálogo único si las credenciales son inválidas) a la verificación con código (diálogo de código incorrecto o exitoso) y entra al Home del administrador.

S03 · Registrar un bloque de cita

User Goal: Como personal de admisión, quiero registrar bloques de cita con doctor, fecha, horario y capacidad.

Figura 105

Wireflow de registro de un bloque de cita

Nota. Elaboración propia.

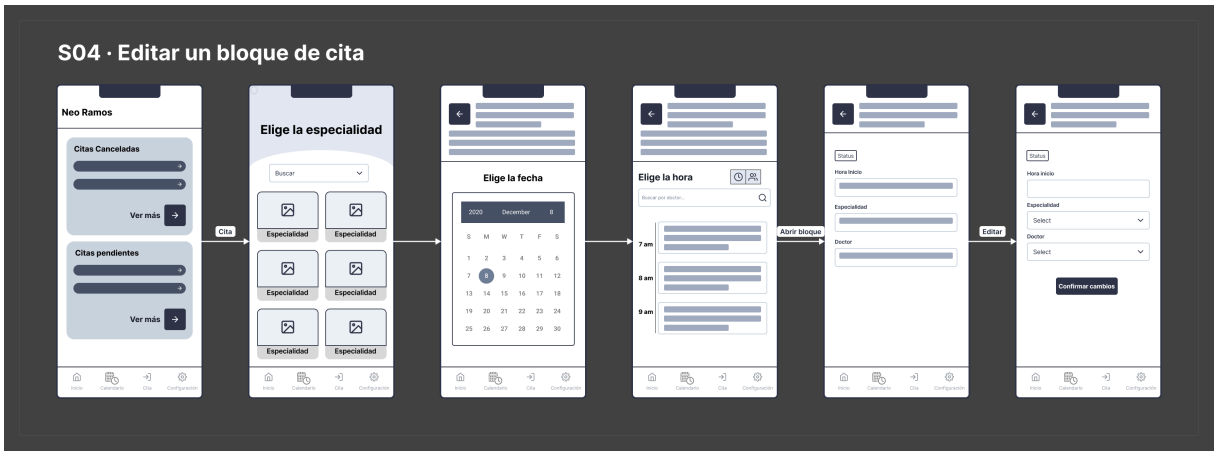
Explicación del flujo: En la Figura 105 se observa que, desde el Home, se elige la especialidad, la fecha, la hora y el doctor, y se termina en el diálogo «Confirmar bloque de cita».

S04 · Editar un bloque de cita

User Goal: Como personal de admisión, quiero ajustar bloques existentes para corregir la programación.

Figura 106

Wireflow de edición de un bloque de cita



Nota. Elaboración propia.

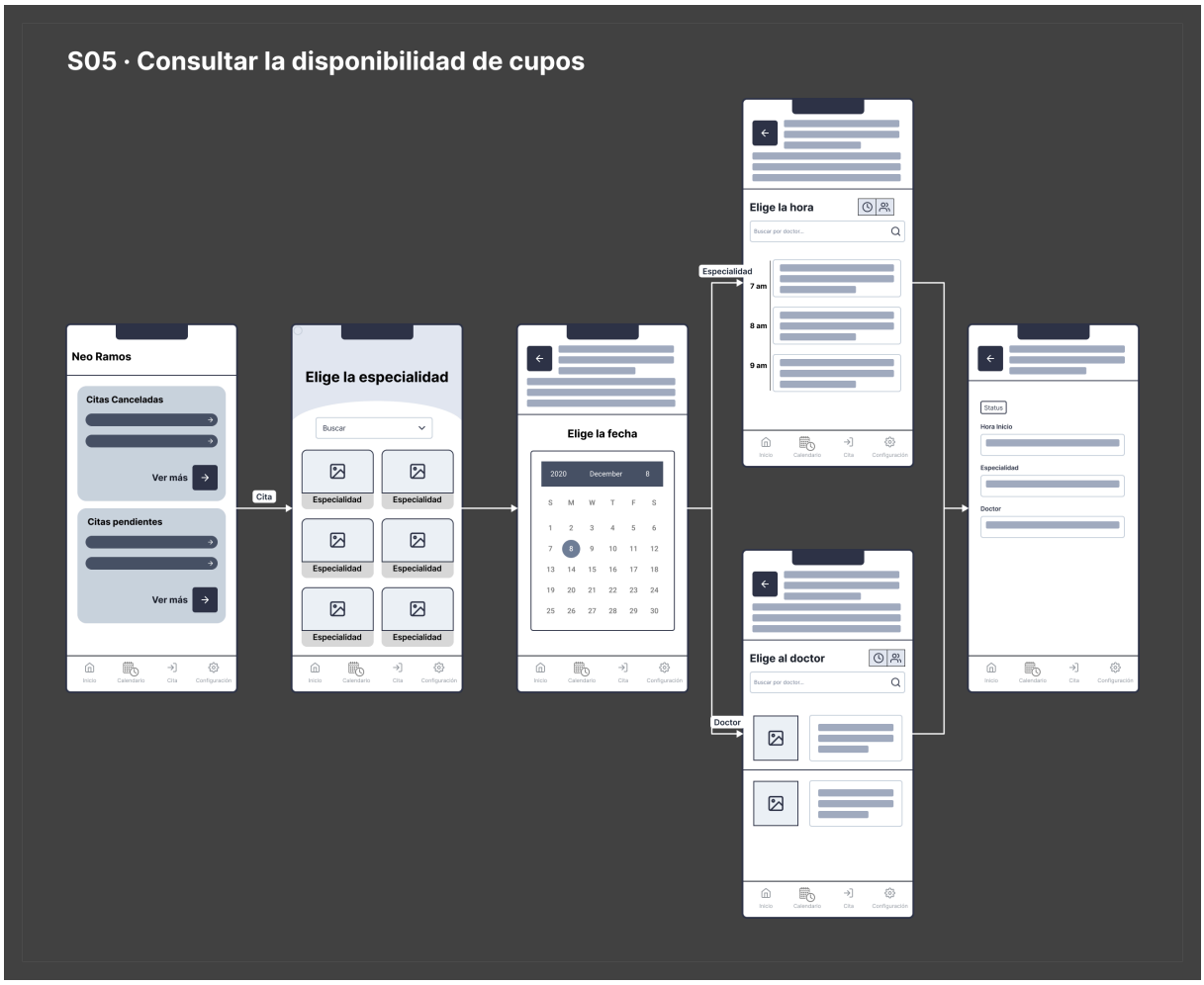
Explicación del flujo: En la Figura 106 se observa que se llega al bloque por especialidad, fecha y hora, se abre su detalle y se pasa a la pantalla de edición con «Confirmar cambios».

S05 · Consultar la disponibilidad de cupos

User Goal: Como personal de admisión, quiero consultar los cupos por especialidad, doctor y fecha.

Figura 107

Wireflow de consulta de disponibilidad de cupos



Nota. Elaboración propia.

Explicación del flujo: En la Figura 107 se observa que, tras elegir especialidad y fecha, el flujo se bifurca en una vista por especialidad y otra por doctor, y ambas llegan al detalle del bloque con su capacidad.

S06 · Consultar las citas pendientes

User Goal: Como personal de admisión, quiero ver las citas pendientes para organizar la agenda del día.

Figura 108

Wireflow de consulta de citas pendientes

Nota. Elaboración propia.

Explicación del flujo: En la Figura 108 se observa que, desde la tarjeta «Citas pendientes» del Home, se abre el detalle con los pacientes con reserva.

S07 · Consultar las citas atendidas

User Goal: Como personal de admisión, quiero consultar las citas ya atendidas y generar reportes por rango de fechas.

Figura 109

Wireflow de consulta de citas atendidas

Nota. Elaboración propia.

Explicación del flujo: En la Figura 109 se observa que el flujo parte del Home; las pantallas de listado y reporte están marcadas como pendientes de wireframe.

S08 · Registrar la llegada del paciente (check-in)

User Goal: Como personal de admisión, quiero registrar la llegada escaneando el QR o ingresando el código de reserva.

Figura 110

Wireflow de registro de llegada del paciente (check-in)

Nota. Elaboración propia.

Explicación del flujo: En la Figura 110 se observa que el flujo parte del Home; las pantallas de escaneo y alta en la cola están marcadas como pendientes de wireframe.

S09 · Gestionar la cola de asistencia

User Goal: Como personal de admisión, quiero manejar la cola de la sala de espera.

Figura 111

Wireflow de gestión de la cola de asistencia

Nota. Elaboración propia.

Explicación del flujo: En la Figura 111 se observa que, desde el Home, se elige la especialidad y se abre la lista «Hoy», donde se atiende y se finaliza cada cita en orden.

S10 · Configurar los parámetros del establecimiento

User Goal: Como personal de admisión, quiero configurar los parámetros operativos del centro.

Figura 112

Wireflow de configuración de los parámetros del establecimiento

Nota. Elaboración propia.

Explicación del flujo: En la Figura 112 se observa que, desde Configuración general, se edita un parámetro, que se actualiza con el diálogo «Ingrese el nuevo valor».

3.1.4.3. Mobile Applications Mock-ups

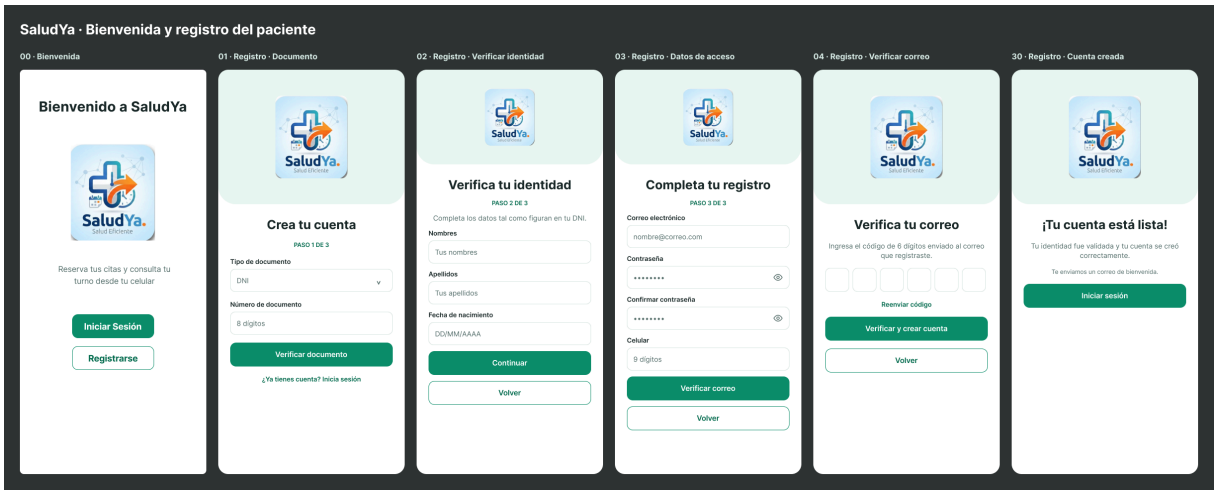
En este entregable se presentan los mock-ups de alta fidelidad de la aplicación móvil de SaludYa para el paciente, incluyendo la gestión de citas de menores a su cargo. El diseño parte de los wireframes del equipo y las historias de usuario del reporte. Los recorridos del personal de admisión, Super Admin, configuración operativa y exportación de reportes quedan pendientes para una entrega posterior.

La identidad visual utiliza el verde primario `#0B8F6B`, fondos claros y tipografía Inter. Las pantallas tienen un tamaño de referencia de 375 × 812 píxeles; los botones principales, una altura de 48 píxeles y esquinas de 12 píxeles. Se incluyen estados de confirmación, error y ausencia de información. Los datos son ficticios y las imágenes representan el diseño de la experiencia.

Sección Autenticación y Registro — Identity & Access Management

Figura 113

Bienvenida y registro del paciente

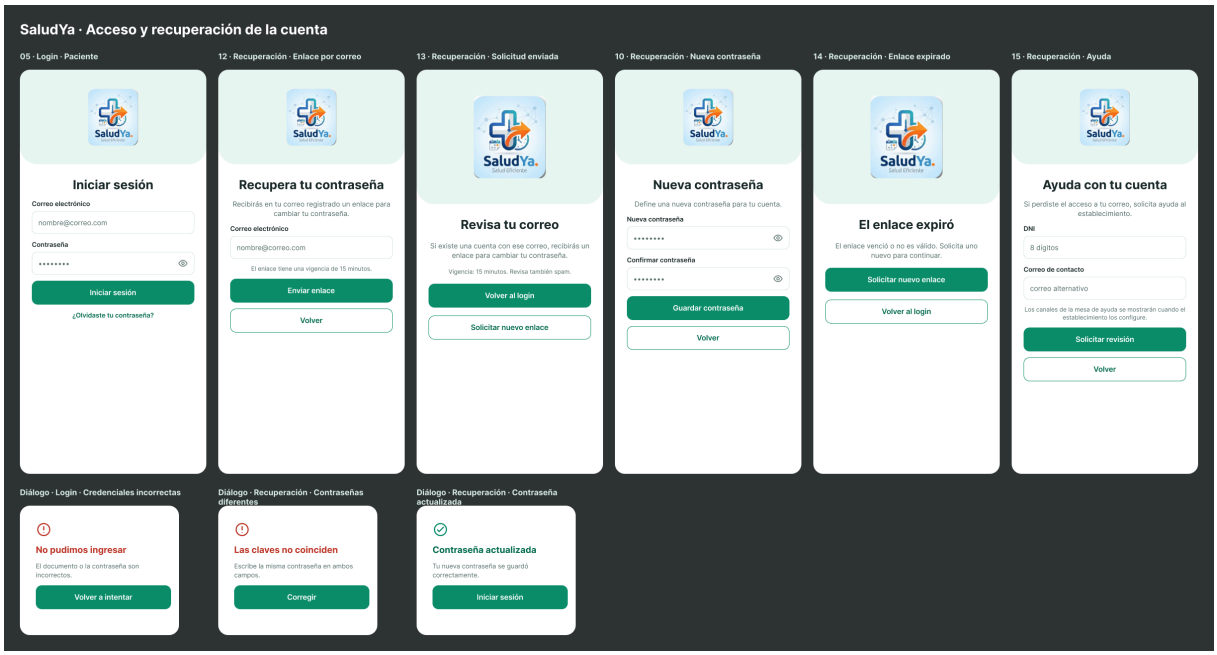


Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

Presenta la bienvenida, el ingreso del DNI, la verificación de datos personales y el registro del correo, contraseña y celular. Los botones de acceso y registro se agrupan en la bienvenida. La cuenta se verifica mediante un código enviado al correo electrónico registrado. El celular se conserva como dato de contacto; no se utiliza verificación por SMS, como se observa en la Figura 113.

Figura 114

Acceso y recuperación de la cuenta del paciente



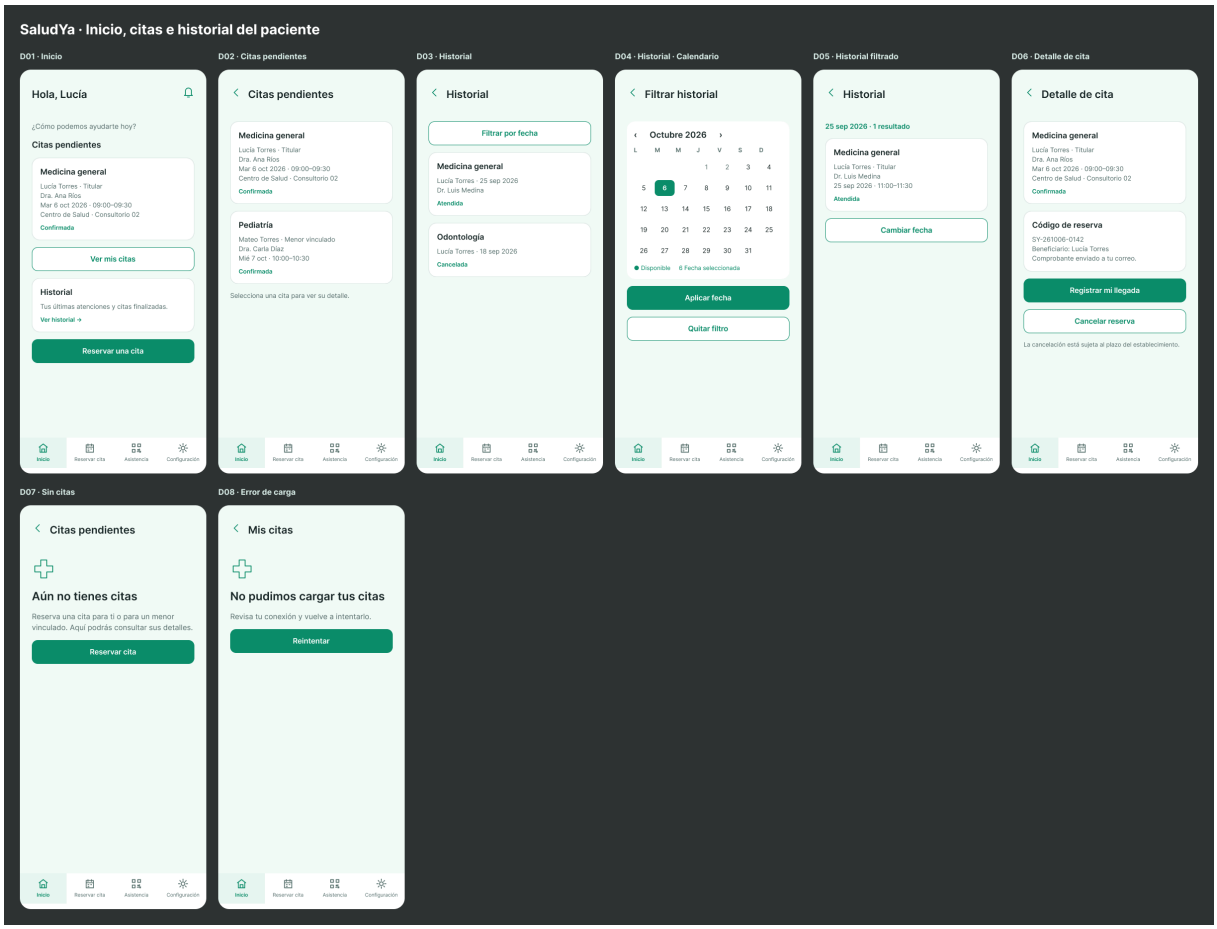
Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

El paciente inicia sesión con su correo y contraseña. El rol de este recorrido es Paciente y no se ofrece un selector de perfiles administrativos. La recuperación envía un enlace al correo registrado, con vigencia de 15 minutos; se muestran la solicitud enviada, el enlace vencido, la nueva contraseña y los errores de acceso, como se observa en la Figura 114.

Sección Dashboard del Paciente

Figura 115

Inicio, citas pendientes e historial del paciente



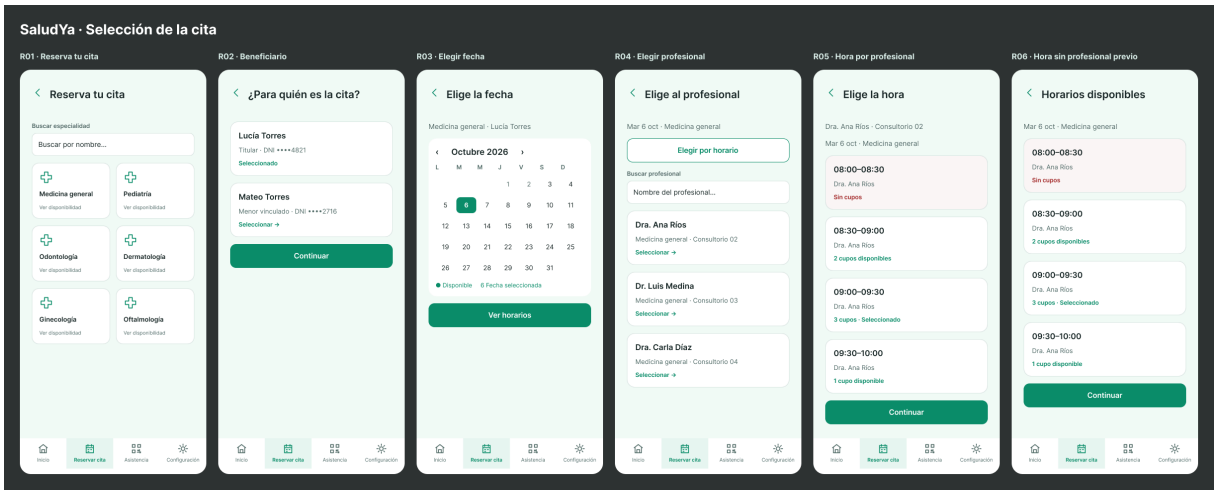
Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

El inicio reúne las citas pendientes, el acceso al historial y la reserva de una nueva cita. La campana de notificaciones se ubica en el extremo derecho de la cabecera. Las tarjetas identifican al beneficiario, la especialidad, el profesional, la fecha y el estado de la cita. Se incluyen el filtro por fecha, el detalle de la reserva y los estados sin citas o con error de carga, como se observa en la Figura 115.

Sección Reserva de Citas

Figura 116

Selección de especialidad, beneficiario, fecha, profesional y horario

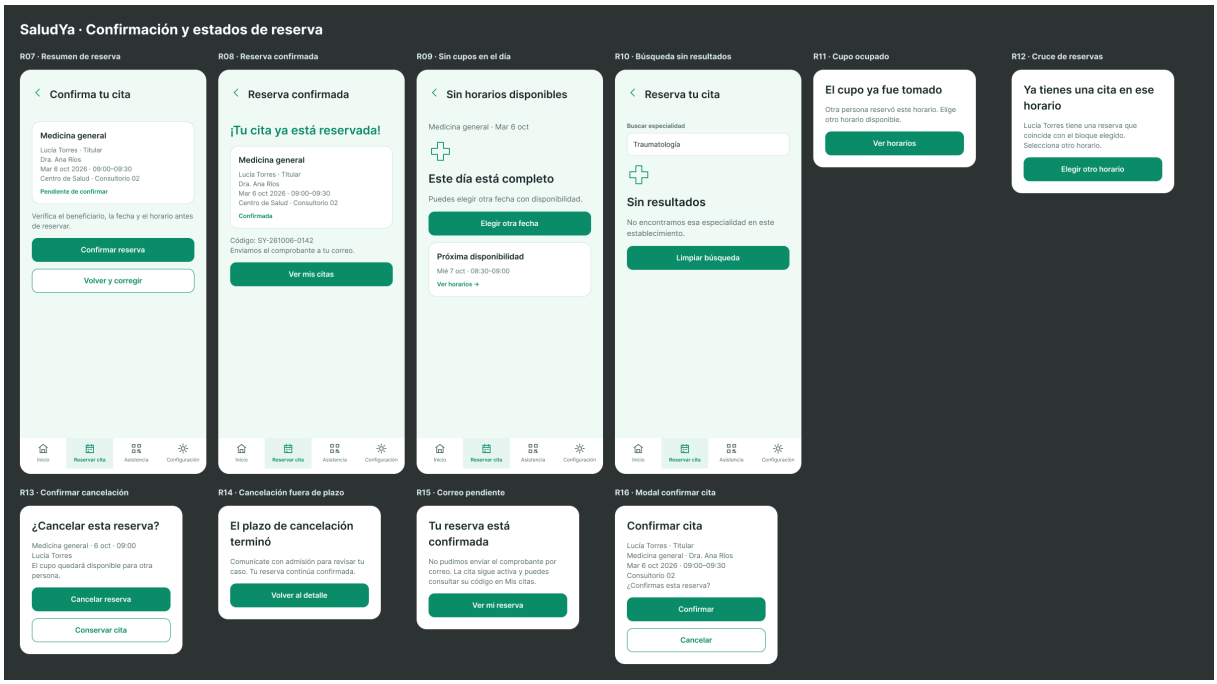


Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

El paciente selecciona la especialidad, al titular o menor vinculado y una fecha disponible. Puede elegir primero al profesional o consultar directamente los horarios mediante la opción ubicada antes de la lista. Los horarios sin cupos se distinguen con texto y color de estado y no permiten selección, como se observa en la Figura 116.

Figura 117

Resumen, confirmación y estados de la reserva



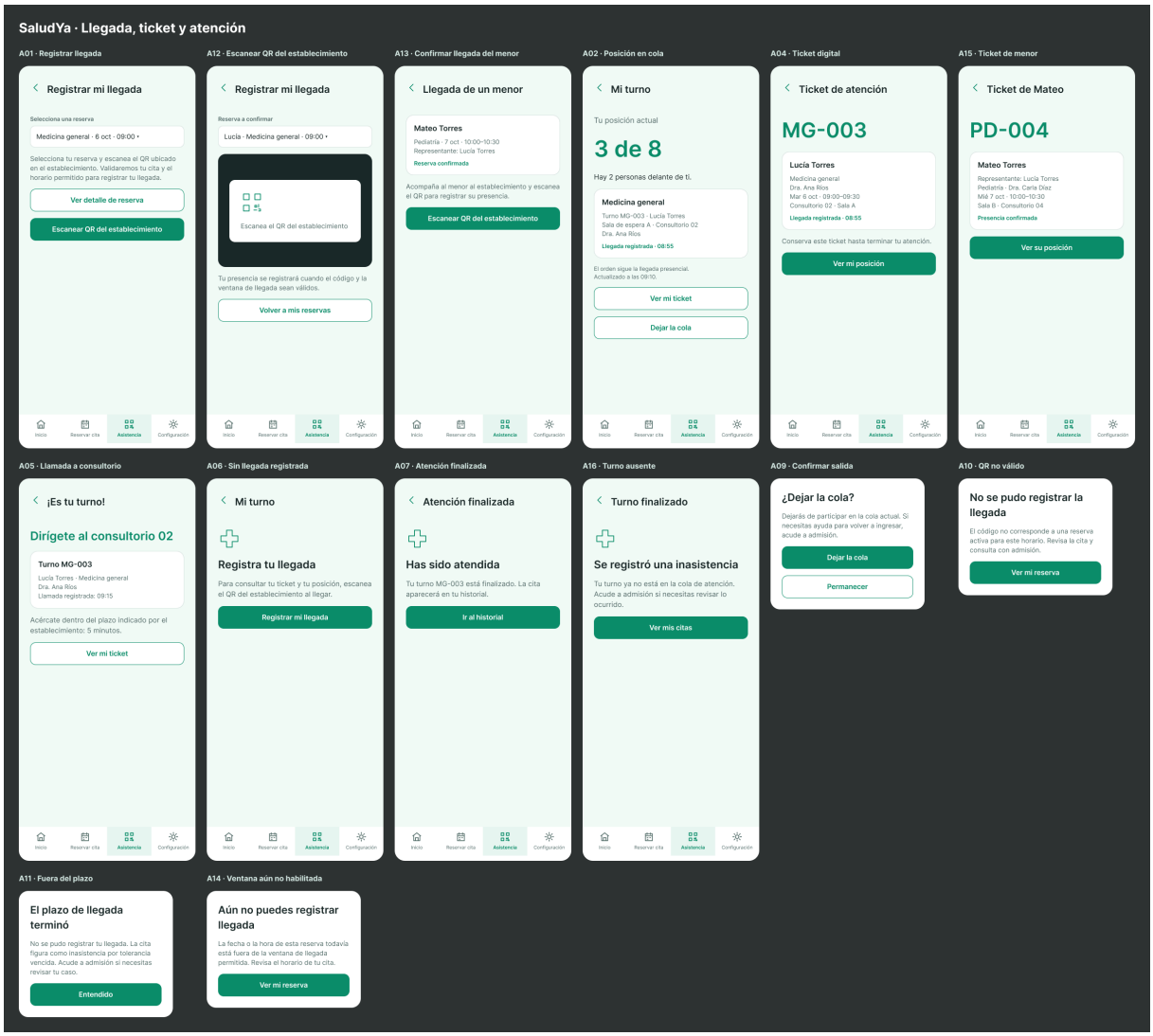
Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

El resumen permite revisar los datos antes de confirmar la cita. La reserva confirmada muestra su código identificador y los detalles de atención. Los estados alternativos contemplan cupos ocupados, cruces de horarios, falta de disponibilidad y restricciones de cancelación. Un fallo en el envío del comprobante no anula la reserva, como se observa en la Figura 117.

Sección Check-in y Atención del Paciente

Figura 118

Registro de llegada, escaneo del QR del establecimiento, ticket y cola



Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

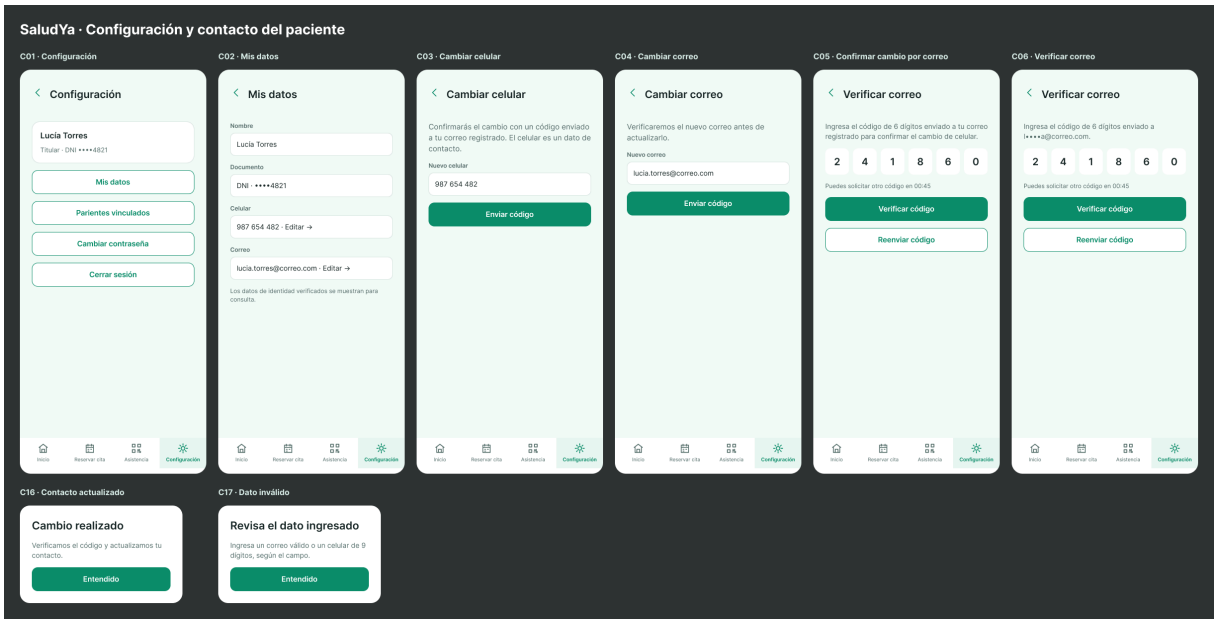
Para registrar su llegada, el paciente selecciona una reserva y escanea el QR ubicado en el establecimiento, conforme a US-12. La aplicación valida la cita y la ventana de tolerancia antes de confirmar la presencia. Este recorrido no solicita presentar un QR personal generado al reservar, como se observa en la Figura 118.

Después del check-in se habilitan el ticket digital y la posición en la cola, ordenada por llegada presencial. Para los menores se identifica al beneficiario y a su representante. Se muestran el llamado a consultorio, la atención finalizada, la ausencia y los errores de QR o de horario. La variante que ocultaba la posición de la cola queda fuera de este entregable.

Sección Configuración y Perfil del Paciente

Figura 119

Configuración, datos personales y actualización del contacto



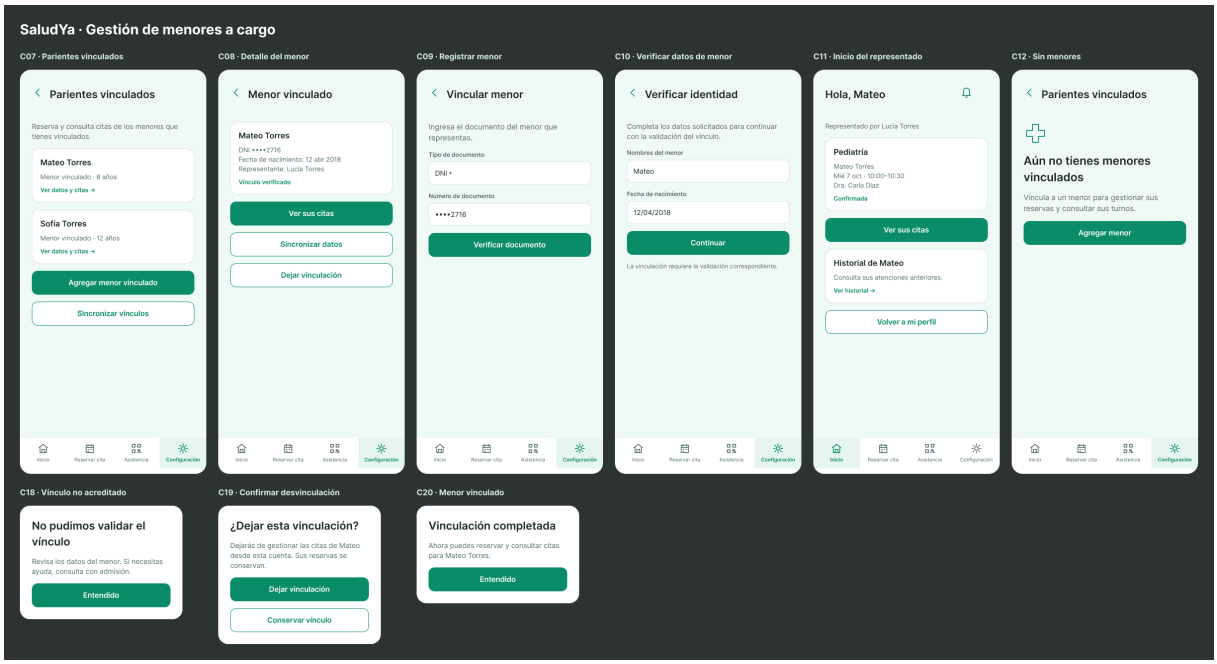
Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

El paciente consulta sus datos y actualiza su celular o correo. El nuevo correo se verifica con un código enviado a esa dirección. Para cambiar el celular, confirma la operación mediante un código enviado al correo registrado; el número se mantiene como dato de contacto. Se presentan los estados de actualización, código incorrecto o vencido y datos inválidos. La identidad verificada permanece como información de consulta, como se observa en la Figura 119.

Sección Gestión de Menores Vinculados

Figura 120

Vinculación, verificación y gestión de menores a cargo



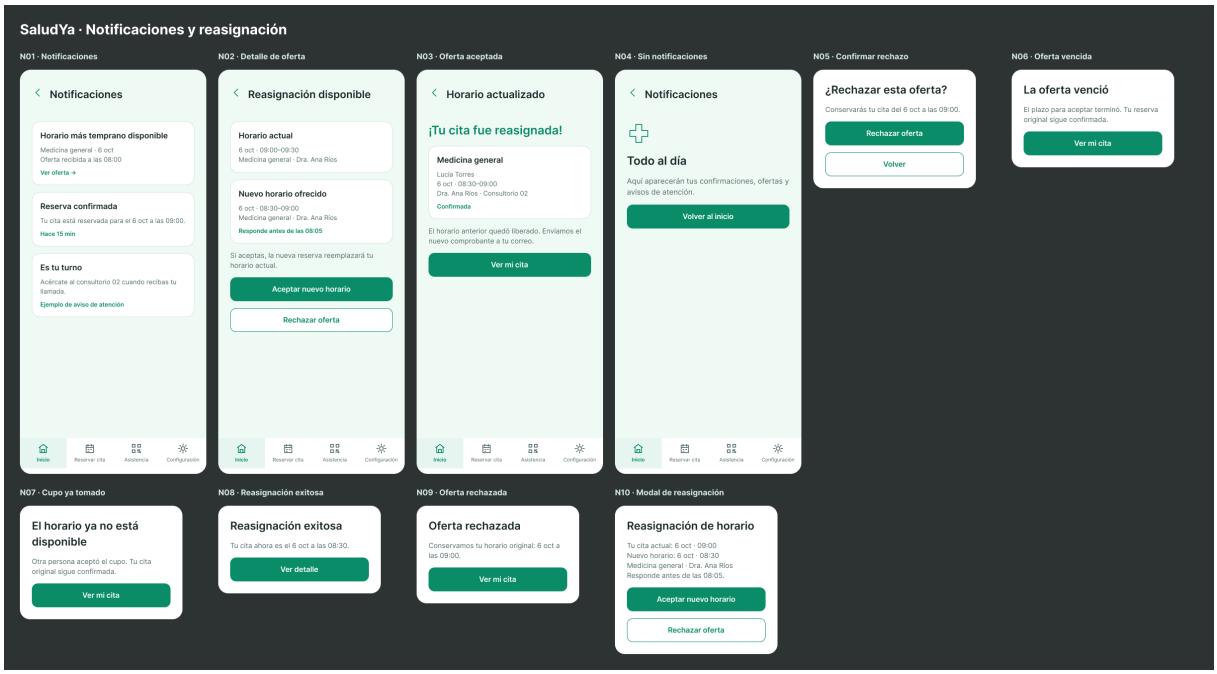
Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

El titular consulta sus menores vinculados, registra un menor y verifica sus datos para gestionar sus citas. Se incluyen el detalle del menor, el inicio del representado, la lista vacía, las restricciones de vinculación y la confirmación de desvinculación, como se observa en la Figura 120.

Sección Notificaciones y Reasignación de Citas

Figura 121

Notificaciones y ofertas de reasignación de citas



Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

Las notificaciones informan sobre reservas, llamados y propuestas de adelanto. El paciente compara el horario actual con el ofrecido y acepta o rechaza la propuesta dentro del plazo. El rechazo, el vencimiento de la oferta o la ocupación del cupo conservan la reserva original, como se observa en la Figura 121.

Archivo de diseño

Mobile Application Mockups — SaludYa

3.1.4.4. Mobile Applications User Flow Diagrams

Los user flow diagrams describen los recorridos del paciente y del titular que gestiona citas de menores a su cargo. Se elaboraron a partir de los mock-ups corregidos y las historias de usuario del reporte, manteniendo el alcance de este entregable.

Se presentan 15 objetivos mediante 30 diagramas. Cada objetivo incluye un **Happy Path**, correspondiente al recorrido esperado, y sus **Unhappy Paths**, con errores, restricciones y decisiones alternativas. La lectura es de izquierda a derecha: los extremos redondeados representan inicio o resultado, los rectángulos indican pantallas o acciones, los rombos muestran decisiones y las flechas enlazan los pasos.

Los recorridos comprenden registro, acceso, recuperación, perfil, menores, citas, reasignación, llegada, ticket, cola y sesión. No se incluyen los flujos del personal de admisión ni de administración.

- **User Goal 1:** Registrarse como paciente.

Happy Path

El paciente accede a Bienvenida, selecciona Registrarse e ingresa su DNI y datos personales. Tras validar su identidad, completa los datos de acceso y verifica su correo mediante el código recibido por email. El recorrido finaliza con la cuenta creada, como se observa en la Figura 122.

Figura 122

Registrarse como paciente — recorrido esperado

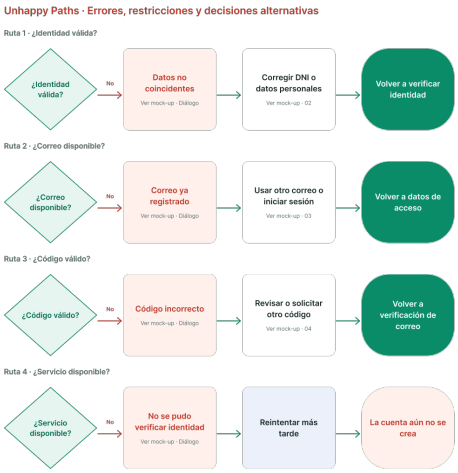
Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

Unhappy Paths

Se consideran datos de identidad no coincidentes, correo registrado, código incorrecto e indisponibilidad del servicio de identidad. El paciente corrige sus datos o reintenta la validación antes de crear la cuenta, como se observa en la Figura 123.

Figura 123

Registrarse como paciente — errores y alternativas



Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

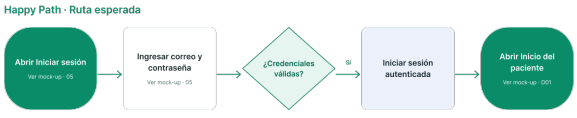
- **User Goal 2:** Iniciar sesión como paciente.

Happy Path

El paciente ingresa su correo y contraseña. Si la cuenta está activa y las credenciales son válidas, accede a Inicio. El recorrido corresponde exclusivamente al paciente, sin selección de perfiles administrativos, como se observa en la Figura 124.

Figura 124

Iniciar sesión como paciente — recorrido esperado



Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

Unhappy Paths

Las credenciales incorrectas mantienen al paciente en el acceso. Para una cuenta inactiva se indica la consulta con admisión. El acceso se realiza por correo y contraseña, sin verificación por SMS, como se observa en la Figura 125.

Figura 125

Iniciar sesión como paciente — errores y alternativas

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

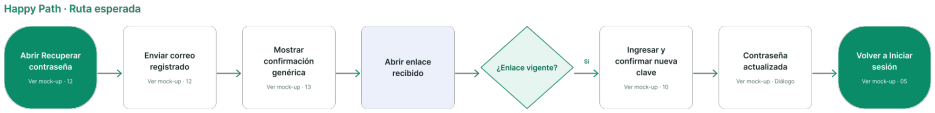
- **User Goal 3:** Recuperar la contraseña.

Happy Path

El paciente solicita la recuperación con su correo registrado. La aplicación muestra una confirmación genérica; el enlace recibido permite definir una nueva contraseña durante sus 15 minutos de vigencia, como se observa en la Figura 126.

Figura 126

Recuperar la contraseña — recorrido esperado



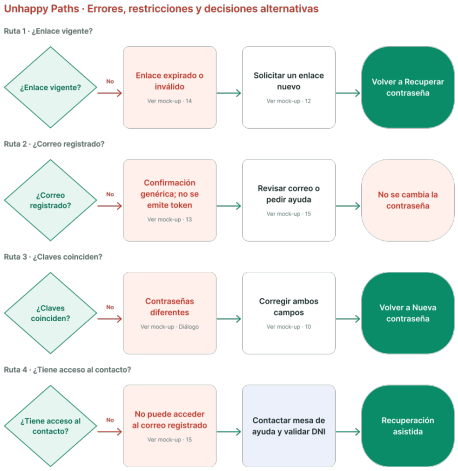
Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

Unhappy Paths

Se presentan enlaces vencidos o inválidos, contraseñas diferentes y pérdida de acceso al correo. Un correo no registrado recibe una respuesta genérica y no genera token. La recuperación asistida requiere verificar la identidad del paciente, como se observa en la Figura 127.

Figura 127

Recuperar la contraseña — errores y alternativas



Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

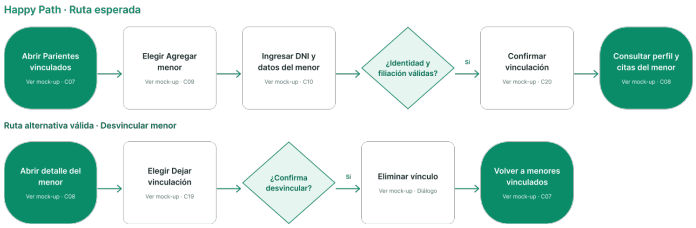
- **User Goal 4:** Vincular o desvincular a un menor.

Happy Path

El titular ingresa los datos del menor y confirma su vinculación después de verificar identidad y filiación. Desde el detalle puede consultar sus citas o confirmar la desvinculación, como se observa en la Figura 128.

Figura 128

Vincular o desvincular a un menor — recorrido esperado



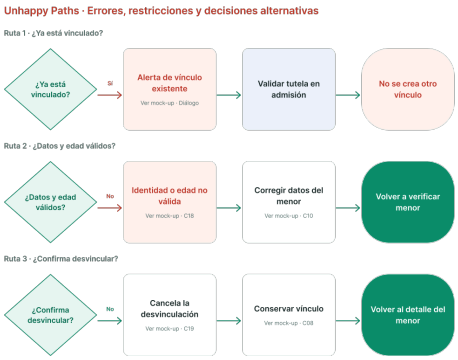
Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

Unhappy Paths

Se representan vínculos existentes, datos o edad inválidos y cancelación de la desvinculación. Cuando se requiere revisión de tutela, el titular consulta con admisión, como se observa en la Figura 129.

Figura 129

Vincular o desvincular a un menor — errores y alternativas



Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

- **User Goal 5:** Actualizar el correo o celular del perfil.

Happy Path

El paciente consulta sus datos y modifica su correo o celular. Si cambia el correo, verifica la nueva dirección; si cambia el celular, confirma la operación con un código enviado al correo registrado. El recorrido finaliza con la actualización y su confirmación, como se observa en la Figura 130.

Figura 130

Actualizar el correo o celular del perfil — recorrido esperado

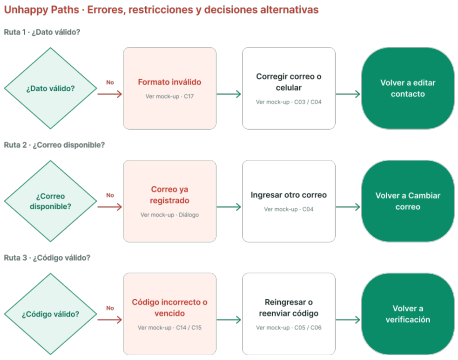
Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

Unhappy Paths

Se consideran formatos inválidos, correo duplicado y códigos incorrectos o vencidos. El paciente corrige el dato o solicita un nuevo código por correo. Los datos de identidad permanecen de consulta, como se observa en la Figura 131.

Figura 131

Actualizar el correo o celular del perfil — errores y alternativas



Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

- **User Goal 6:** Consultar disponibilidad de citas.

Happy Path

El paciente selecciona una especialidad y una fecha. Consulta profesionales y horarios disponibles; la alternativa de elegir por horario se encuentra antes de la lista de profesionales, como se observa en la Figura 132.

Figura 132

Consultar disponibilidad de citas — recorrido esperado

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

Unhappy Paths

Una búsqueda sin resultados permite cambiar el texto. Si no hay cupos para el día, el paciente vuelve al calendario y elige otra fecha. Los horarios sin cupos no se seleccionan, como se observa en la Figura 133.

Figura 133

Consultar disponibilidad de citas — errores y alternativas

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

- **User Goal 7:** Reservar una cita y recibir confirmación.

Happy Path

El titular indica el beneficiario, elige fecha, profesional y horario y revisa el resumen. Al confirmar, recibe el código de reserva y el comprobante de la cita, como se observa en la Figura 134.

Figura 134

Reservar una cita y recibir confirmación — recorrido esperado

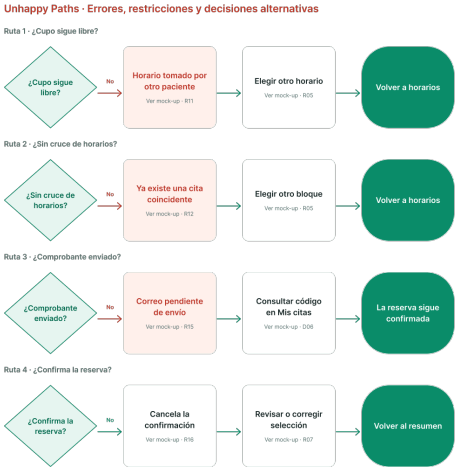
Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

Unhappy Paths

Se contemplan un cupo tomado, una cita coincidente, cancelación de la confirmación y fallo en el envío del comprobante. Si el correo queda pendiente, la reserva continúa confirmada, como se observa en la Figura 135.

Figura 135

Reservar una cita y recibir confirmación — errores y alternativas



Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

- **User Goal 8:** Consultar citas, detalles e historial.

Happy Path

Desde Inicio, el paciente consulta citas pendientes o historial, aplica un filtro por fecha y abre el detalle de una cita propia o de un menor, como se observa en la Figura 136.

Figura 136

Consultar citas, detalles e historial — recorrido esperado

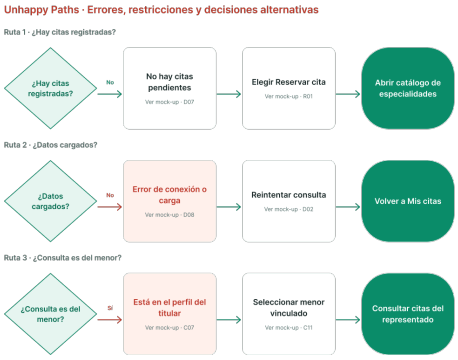
Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

Unhappy Paths

Se presentan ausencia de citas, error de carga y necesidad de seleccionar al menor representado. El paciente puede reservar, reintentar la consulta o cambiar de beneficiario, como se observa en la Figura 137.

Figura 137

Consultar citas, detalles e historial — errores y alternativas



Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

- **User Goal 9:** Cancelar una reserva dentro del plazo.

Happy Path

El paciente abre el detalle de una reserva y solicita cancelarla. Dentro del plazo permitido, confirma la operación y consulta el estado Cancelada, como se observa en la Figura 138.

Figura 138

Cancelar una reserva dentro del plazo — recorrido esperado

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

Unhappy Paths

Fuera del plazo, la reserva sigue activa y se indica la consulta con admisión. Si cancela la confirmación, conserva la cita. Cancelar una reserva se distingue de dejar la cola presencial, como se observa en la Figura 139.

Figura 139

Cancelar una reserva dentro del plazo — errores y alternativas

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

- **User Goal 10:** Responder a una oferta de adelanto.

Happy Path

El paciente recibe una oferta de adelanto, compara ambos horarios y acepta mientras la oferta y el cupo siguen vigentes. El nuevo horario reemplaza al anterior, como se observa en la Figura 140.

Figura 140

Responder a una oferta de adelanto — recorrido esperado

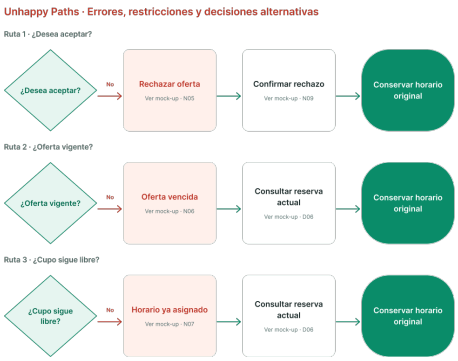
Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

Unhappy Paths

El rechazo, el vencimiento o un cupo ya tomado conservan la cita original. Cada estado permite volver a consultar la reserva actual, como se observa en la Figura 141.

Figura 141

Responder a una oferta de adelanto — errores y alternativas



Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

- **User Goal 11:** Registrar llegada presencial mediante QR.

Happy Path

Al llegar al establecimiento, el titular selecciona su reserva o la del menor y escanea el QR del establecimiento. Si la cita y la ventana horaria son válidas, se confirma la presencia, se ingresa a la cola y se habilita el ticket, como se observa en la Figura 142.

Figura 142

Registrar llegada presencial mediante QR — recorrido esperado

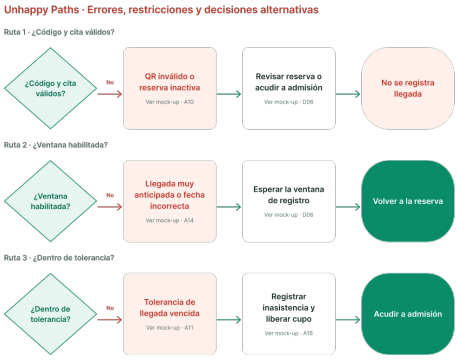
Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

Unhappy Paths

Un QR inválido o una reserva inactiva impiden registrar la llegada. Una llegada anticipada requiere esperar la ventana; una llegada fuera de tolerancia registra la inasistencia y activa la liberación del cupo, como se observa en la Figura 143.

Figura 143

Registrar llegada presencial mediante QR — errores y alternativas



Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

- **User Goal 12:** Obtener el ticket digital de atención.

Happy Path

Después del check-in, el paciente obtiene su código de turno y consulta el ticket con los datos del beneficiario, profesional, sala y consultorio, como se observa en la Figura 144.

Figura 144

Obtener el ticket digital de atención — recorrido esperado

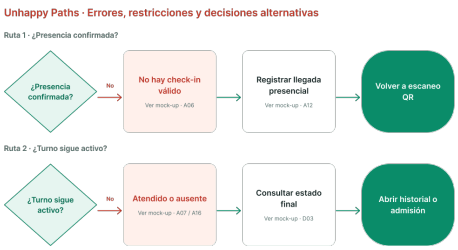
Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

Unhappy Paths

Sin presencia confirmada debe registrar primero la llegada. Si el turno ya finalizó o fue declarado ausente, consulta el estado correspondiente, como se observa en la Figura 145.

Figura 145

Obtener el ticket digital de atención — errores y alternativas



Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

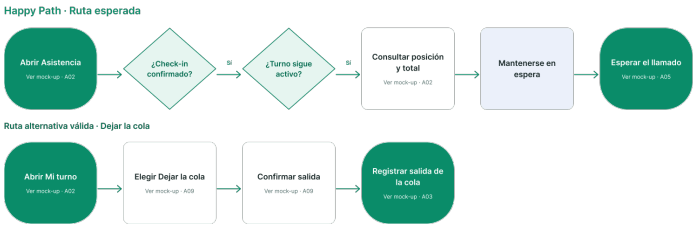
- **User Goal 13:** Consultar posición o dejar la cola.

Happy Path

El paciente con check-in confirmado y turno activo consulta su posición y el total de pacientes, ordenados por llegada presencial. También puede confirmar que deja la cola, como se observa en la Figura 146.

Figura 146

Consultar posición o dejar la cola — recorrido esperado



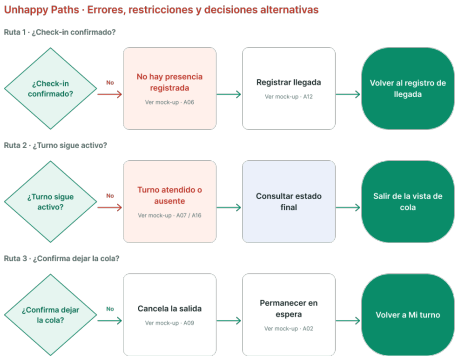
Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

Unhappy Paths

Sin check-in se solicita registrar la llegada. Un turno atendido o ausente muestra su estado final; cancelar la salida conserva al paciente en espera. No se incluye la variante de cola oculta, como se observa en la Figura 147.

Figura 147

Consultar posición o dejar la cola — errores y alternativas



Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

- **User Goal 14:** Recibir el llamado y acudir al consultorio.

Happy Path

El paciente recibe el llamado, consulta el ticket y se dirige al consultorio dentro del margen establecido para su atención, como se observa en la Figura 148.

Figura 148

Recibir el llamado y acudir al consultorio — recorrido esperado

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

Unhappy Paths

Se contemplan turno aún no llamado, vencimiento del plazo posterior al llamado y falta de respuesta. La aplicación muestra el estado y orienta al paciente sobre el siguiente paso, como se observa en la Figura 149.

Figura 149

Recibir el llamado y acudir al consultorio — errores y alternativas

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

- **User Goal 15:** Cerrar sesión o recuperar acceso a una sesión.

Happy Path

El paciente abre su perfil, solicita cerrar sesión y confirma. La aplicación finaliza la sesión y vuelve a Bienvenida, como se observa en la Figura 150.

Figura 150

Cerrar sesión o recuperar acceso a una sesión — recorrido esperado

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

Unhappy Paths

Cancelar el cierre conserva la sesión activa. Una sesión expirada requiere volver a ingresar con las credenciales del paciente, como se observa en la Figura 151.

Figura 151

Cerrar sesión o recuperar acceso a una sesión — errores y alternativas

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

Archivo de diagramas

Mobile Applications User Flow Diagrams — SaludYa

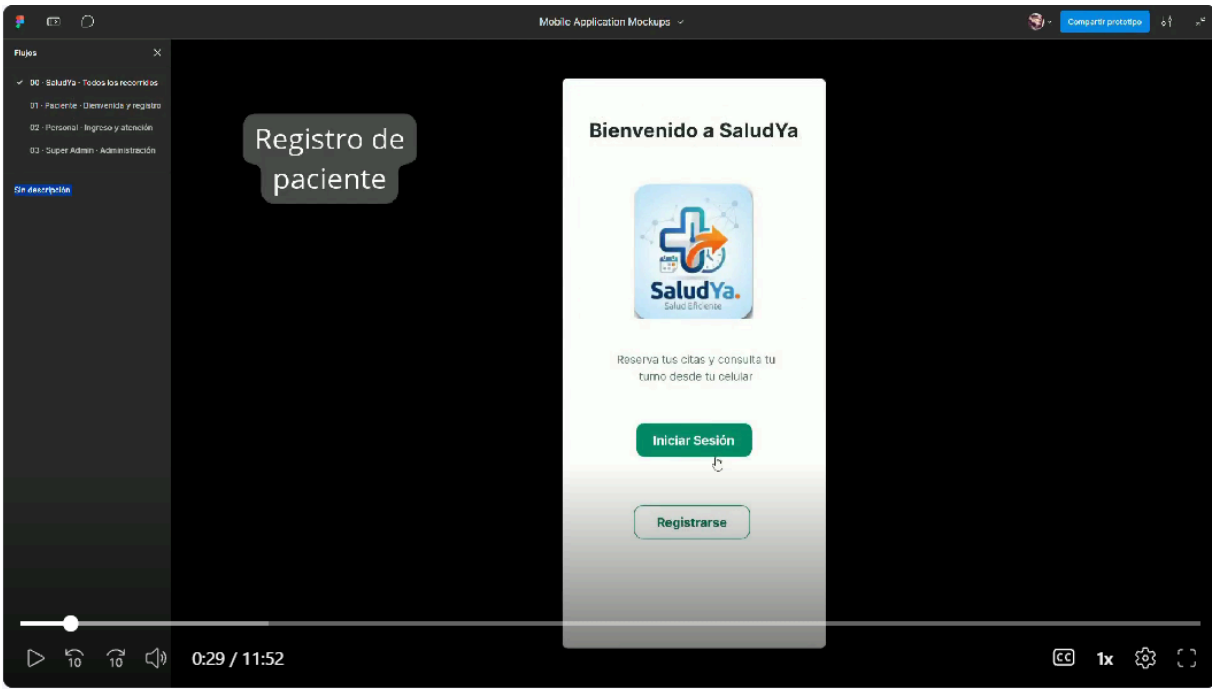
3.1.4.5. Mobile Applications Prototyping

En esta sección se presenta el prótotipo interactivo desarrollado en Figma para la aplicación móvil. El diseño y los flujos de navegación están alineados con la arquitectura de información y los user flow diagrams definidos.

A continuación, se adjunta el enlace al video de demostración, como se observa en la Figura 152.

Figura 152

Mobile applications prototyping



Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

Video Mobile Applications Prototyping

Capítulo IV: Product Implementation & Validation

4. Product Implementation & Validation

4.1. Software Configuration Management

4.1.1. Software Development Environment Configuration

En esta sección se especifican los productos de software que el equipo **RuwaLabs** utilizó para colaborar en el ciclo de vida de **SaludYa**, abarcando las actividades de **gestión del proyecto**, **gestión de requisitos**, **diseño UX/UI**, **desarrollo**, **despliegue** y **documentación**. Para cada producto se indica su **propósito de uso en el proyecto** y su **ruta de referencia** (cuando es software basado en modelos SaaS) o **ruta de descarga** (cuando es software que se ejecuta en el equipo de los integrantes), como se detalla en la Tabla 59.

Tabla 59

Software Development Environment Configuration

Actividad	Producto	Propósito de uso en el proyecto	Ruta de referencia / descarga
Project Management	Jira	Planificación y seguimiento del proyecto, organización del backlog y de los sprints	https://ruwalabs--salud-ya.atlassian.net/
Project Management	Jitsi Meet	Reuniones de coordinación del equipo (sprint planning, dailies y revisiones)	https://meet.jit.si/
Project Management	Discord	Comunicación diaria y coordinación del equipo	https://discord.com/download
Requirements Management	Jira	Registro, priorización y seguimiento de épicas, user stories y product backlog	https://ruwalabs--salud-ya.atlassian.net/
Requirements Management	Miro	EventStorming y modelado colaborativo del dominio	https://miro.com/
Product UX/UI Design	Figma	Diseño de wireframes, mock-ups, user flows y prototipado	https://www.figma.com/
Software Development	Android Studio	IDE para el desarrollo de la aplicación Android	https://developer.android.com/studio

Actividad	Producto	Propósito de uso en el proyecto	Ruta de referencia / descarga
		(Kotlin y Jetpack Compose)	
Software Development	IntelliJ IDEA	IDE para el desarrollo del backend (Spring Boot)	https://www.jetbrains.com/idea/download/
Software Development	Kotlin	Lenguaje de programación de las aplicaciones móviles	https://kotlinlang.org/
Software Development	Java (JDK)	Lenguaje de programación del backend	https://adoptium.net/
Software Development	Spring Boot	Framework para la construcción de los servicios web REST	https://spring.io/projects/spring-boot
Software Development	Maven	Gestión de dependencias y construcción del backend	https://maven.apache.org/download.cgi
Software Development	Gradle	Construcción de la aplicación Android	https://gradle.org/
Software Development	PostgreSQL	Base de datos relacional del sistema	https://www.postgresql.org/download/
Software Development	Git / GitHub	Control de versiones y hospedaje de los repositorios del proyecto	https://git-scm.com/downloads · https://github.com/RuwaLabs
Software Development	Postman	Pruebas manuales de los endpoints de los servicios web	https://www.postman.com/downloads/
Software Development	springdoc-openapi (Swagger UI)	Generación de la documentación de la API (OpenAPI)	https://springdoc.org/
Software Deployment	GitHub Pages	Publicación del Landing Page	https://pages.github.com/
Software Deployment	AWS (EC2)	Despliegue de los servicios web (Spring Boot) y de la base de datos PostgreSQL	https://aws.amazon.com/ec2/
Software Documentation	Obsidian	Redacción y edición del informe en formato Markdown	https://obsidian.md/download

Actividad	Producto	Propósito de uso en el proyecto	Ruta de referencia / descarga
Software Documentation	GitHub (Markdown)	Versionado y publicación de la documentación del proyecto	https://github.com/RuwaLabs
Software Documentation	PlantUML / diagrams.net	Elaboración de diagramas UML y del modelo C4	https://plantuml.com/ · https://app.diagrams.net/

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

4.1.2. Source Code Management

En esta sección se establecen los medios y el esquema de organización que el equipo **RuwaLabs** aplica para el seguimiento de modificaciones a lo largo del ciclo de vida de **SaludYa**. Como plataforma y sistema de control de versiones se utiliza **GitHub**, sobre el cual se aplica **GitFlow** como modelo de ramificación, **Conventional Commits** para los mensajes de commit y **Semantic Versioning** para nombrar las releases.

Cada producto de la solución cuenta con su propio repositorio, como se detalla en la Tabla 60.

Tabla 60

Repositorios de GitHub por producto

Producto	Repositorio de GitHub
Landing Page	https://github.com/RuwaLabs/saludya-landing
Web Services	https://github.com/RuwaLabs/backend-saludya
Mobile Applications	https://github.com/RuwaLabs/frontend-kotlin-saludya
Project Report	https://github.com/RuwaLabs/upc-pre-202620-1acc0238-4945-RuwaLabs--report

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

Para el control de versiones se adopta **GitFlow**. Sobre la rama principal `main` se crea la rama de integración `develop` cada funcionalidad o bounded context se desarrolla en su propia rama (*feature branch*), que se integra a `develop` mediante *pull requests*. Al cierre de un sprint, `develop` se integra a `main` mediante una rama de release, y las correcciones urgentes sobre producción se realizan mediante ramas de hotfix. Las convenciones de ramas se detallan en la Tabla 61.

Tabla 61

Convenciones de ramas (GitFlow)

Rama	Propósito	Convención / ramas utilizadas en el proyecto
`main`	Versión estable en producción	`main`
`develop`	Rama de integración del equipo	`develop`
Feature	Desarrollo de una funcionalidad o bounded context	Nombre del aspecto en kebab--case: `iam` , `booking` , `reassignment` , `arrival` , `hospitalconfig` (Web Services); `iam` , `patient-reservations` , `patient-check-in` ,

Rama	Propósito	Convención / ramas utilizadas en el proyecto
		`patient-configuration`, `patient-dashboard`, `notifications-reassignment` (Mobile Applications)
Release	Preparación de una versión estable	`release/<MAJOR.MINOR.PATCH>`
Hotfix	Corrección urgente sobre producción	`hotfix/<aspecto>`

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

Las releases se nombran aplicando **Semantic Versioning** (`MAJOR.MINOR.PATCH`), por ejemplo `v1.0.0` .

Finalmente, los mensajes de commit siguen **Conventional Commits**, como se detalla en la Tabla 62.

Tabla 62

Convenciones de commits (Conventional Commits)

Tipo	Uso
`feat`	Nueva funcionalidad
`fix`	Corrección de error
`docs`	Cambios en documentación
`style`	Formato sin cambios de lógica
`refactor`	Reestructuración sin cambiar comportamiento
`test`	Añadir o modificar pruebas
`chore`	Tareas de mantenimiento

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

4.1.3. Source Code Style Guide & Conventions

En esta sección se establecen las convenciones de estilo y nomenclatura que el equipo **RuwaLabs** adopta para el desarrollo de la solución **SaludYa**, compuesta por el Landing Page, las aplicaciones móviles (pacientes y personal de salud) y los servicios web. El objetivo es garantizar la legibilidad, mantenibilidad y consistencia del código a lo largo del ciclo de vida del proyecto, así como facilitar la colaboración entre los miembros del equipo.

Todas las convenciones se aplican en **inglés** para nombres de archivos, variables, funciones, clases y comentarios, siguiendo las buenas prácticas de la industria y las guías oficiales de cada tecnología.

Landing Page

El Landing Page se desarrolla con **HTML5, CSS3 y JavaScript (ES6+)**, aplicando las siguientes convenciones:

Tabla 63

Source Code Style Guide & Conventions — Landing Page

Elemento	Convención	Ejemplo
Archivos HTML	kebab-case	`index.html`
Archivos CSS	kebab-case	`styles.css` , `responsive.css`
Archivos JS	kebab-case	`i18n.js` , `main.js`

Elemento	Convención	Ejemplo
Carpetas	kebab-case	`assets/css/` , `assets/img/` , `assets/locales/`
Variables CSS	kebab-case con prefijo `--`	`--color-primary` , `--space-4`
Clases CSS	BEM simplificado (bloque__elemento--modificador)	`.hero__title` , `.card--active`
IDs HTML	kebab-case	`#primary-nav` , `#hero-title`
Variables JS	camelCase	`currentLang` , `headerHeight`
Constantes JS	UPPER_SNAKE_CASE	`DEFAULT_LANG` , `STORAGE_KEY`
Funciones JS	camelCase	`detectInitialLang()` , `applyTranslations()`
Atributos `data-*`	kebab-case	`data-i18n` , `data-i18n-attr` , `data-lang`

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

Se adoptan las guías **Google HTML/CSS Style Guide** y **HTML Style Guide and Coding Conventions** (W3Schools), con las siguientes reglas adicionales, como se detalla en la Tabla 63:

- Indentación de 2 espacios.
- Uso de comillas dobles en HTML y comillas simples en JavaScript.
- Punto y coma obligatorio al final de cada sentencia en JavaScript.
- Declaración de variables con `const` por defecto y `let` cuando sea necesario; se evita `var` .
- Uso de `"use strict"` en cada archivo JavaScript.
- Comentarios en inglés con formato `/* ... */` para bloques y `// ...` para líneas.
- Separación de responsabilidades: estructura (HTML), presentación (CSS) y comportamiento (JS).
- Uso de `aria-*` y roles semánticos para accesibilidad.

Aplicaciones móviles

Las aplicaciones móviles se desarrollan con **Kotlin** (Android nativo) y **Kotlin Multiplatform (KMP)** para la lógica compartida con iOS, siguiendo las convenciones oficiales del lenguaje:

Tabla 64

Source Code Style Guide & Conventions — Aplicaciones móviles

Elemento	Convención	Ejemplo
Clases	PascalCase	`AppointmentRepository` , `PatientViewModel`
Interfaces	PascalCase con prefijo descriptivo	`AppointmentRepository`
Funciones	camelCase	`getAvailableSlots()`
Variables	camelCase	`availableSlots` , `patientId`
Constantes	UPPER_SNAKE_CASE	`MAX_WAITING_LIST_SIZE`
Paquetes	minúsculas separadas por punto	`pe.edu.upc.saludya.appointments`
Archivos Kotlin	PascalCase	`AppointmentViewModel.kt`
Recursos XML	snake_case	`activity_main.xml` , `ic_check_in.xml`
Strings	snake_case	`app_name` , `btn_reserve`

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

Se adoptan las guías **Android Kotlin Style Guide** y **Kotlin Coding Conventions**, con las siguientes reglas adicionales, como se detalla en la Tabla 64:

- Indentación de 4 espacios.
- Longitud máxima de línea: 100 caracteres.
- Uso de `val` por defecto y `var` solo cuando sea necesario.
- Uso de `data class` para modelos de dominio.
- Uso de `sealed class` para estados de UI.
- Uso de corrutinas para operaciones asíncronas.
- Nomenclatura en inglés para todos los identificadores.
- Comentarios KDoc para clases y funciones públicas.

Servicios web

Los servicios web se desarrollan con **Spring Boot** (Java) y **OpenAPI Specification** para la documentación, siguiendo las convenciones oficiales:

Tabla 65

Source Code Style Guide & Conventions — Servicios web

Elemento	Convención	Ejemplo
Clases	PascalCase	`AppointmentController`, `PatientService`
Métodos	camelCase	`createAppointment()`, `findPatientById()`
Variables	camelCase	`appointmentId`, `patientName`
Constantes	UPPER_SNAKE_CASE	`MAX_APPOINTMENTS_PER_DAY`
Paquetes	minúsculas separadas por punto	`pe.edu.upc.saludya.appointments`
Endpoints REST	kebab-case en plural	`/api/v1/appointments`, `/api/v1/patients`
Archivos `.feature` (Gherkin)	kebab-case	`reserve-appointment.feature`
Tablas de base de datos	snake_case en plural	`appointments`, `patients`
Columnas de base de datos	snake_case	`created_at`, `patient_id`

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

Se adoptan las guías **Google Java Style Guide**, **Spring Boot Features** y **Gherkin Conventions for Readable Specifications** (para los archivos `.feature`), con las siguientes reglas adicionales, como se detalla en la Tabla 65:

- Indentación de 4 espacios.
- Uso de anotaciones de Spring (`@RestController`, `@Service`, `@Repository`).
- Uso de DTOs para la transferencia de datos entre capas.
- Uso de `Optional` para valores que pueden ser nulos.
- Documentación de endpoints con **OpenAPI** y **Swagger UI**.
- Criterios de aceptación en **Gherkin** con estructura Given-When-Then.

4.1.4. Software Deployment Configuration

En esta sección se describe la configuración de despliegue de la solución **SaludYa**, incluyendo los pasos necesarios para publicar cada uno de los productos digitales que la componen: el Landing Page, las aplicaciones móviles (pacientes y personal de salud) y los servicios web. Asimismo, se presenta el **Deployment Diagram** del modelo C4, que ilustra la distribución física de los componentes de software sobre la infraestructura de hardware y servicios en la nube.

Landing Page

El Landing Page se despliega como un sitio estático alojado en **GitHub Pages**, aprovechando la integración directa con el repositorio de GitHub del equipo.

Tabla 66

Software Deployment Configuration — Landing Page

Paso	Acción
1	Asegurar que el archivo `index.html` se encuentre en la raíz del repositorio `saludya-landing`
2	Acceder a **Settings → Pages** en el repositorio de GitHub
3	Seleccionar la rama `main` y la carpeta `/ (root)` como fuente
4	Guardar la configuración y esperar la publicación
5	Verificar el despliegue en la URL generada: `https://ruwalabs.github.io/saludya-landing/`
6	(Opcional) Configurar un dominio personalizado `saludya.pe` mediante registros CNAME

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

Tecnologías involucradas: HTML5, CSS3, JavaScript (ES6+), Font Awesome 6.5.2, como se detalla en la Tabla 66.

Aplicaciones móviles

Las aplicaciones móviles se distribuyen mediante **Firebase App Distribution** para las pruebas con usuarios de validación, y se publican en **Google Play Store** y **App Store** para la versión final.

Tabla 67

Software Deployment Configuration — Aplicaciones móviles

Paso	Acción
1	Generar el APK/AAB firmado desde Android Studio (Android) o el archivo IPA desde Xcode (iOS)
2	Crear un proyecto en **Firebase Console** y habilitar **App Distribution**
3	Subir el APK/AAB o IPA a Firebase App Distribution
4	Invitar a los testers mediante correo electrónico o enlace público
5	Recopilar feedback de los usuarios de validación
6	Publicar la versión final en **Google Play Console** y **App Store Connect**

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

Tecnologías involucradas: Kotlin, Kotlin Multiplatform (KMP), Android Studio, Xcode, Firebase App Distribution, como se detalla en la Tabla 67.

Servicios web

Los servicios web se despliegan en **AWS EC2**, con base de datos **PostgreSQL** desplegada en la misma infraestructura.

Tabla 68

Software Deployment Configuration — Servicios web

Paso	Acción
1	Crear una cuenta en **AWS** y aprovisionar una instancia **EC2**
2	Conectar el repositorio de GitHub del backend (`saludya-web-services`)
3	Configurar las variables de entorno (credenciales de base de datos, claves JWT, etc.)
4	Instalar y configurar la base de datos PostgreSQL en la instancia EC2
5	Configurar el comando de build y el comando de inicio (`./mvnw spring-boot:run` o `java -jar app.jar`)

Paso	Acción
6	Desplegar y verificar la URL pública del servicio
7	Acceder a la documentación OpenAPI en `/swagger-ui.html`

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

Tecnologías involucradas: Spring Boot, Java, PostgreSQL, OpenAPI, Swagger UI, AWS EC2, como se detalla en la Tabla 68.

Deployment Diagram (C4 Model)

El **Deployment Diagram** ilustra la distribución física de los componentes de SaludYa sobre la infraestructura de hardware y servicios en la nube, como se detalla en la Tabla 69:

Tabla 69

Deployment Diagram (C4 Model)

Nodo	Tipo	Componentes desplegados
Dispositivo del paciente	Hardware (móvil)	App pacientes (Kotlin/KMP)
Dispositivo del personal de salud	Hardware (móvil)	App personal de salud (Kotlin/KMP)
Navegador del visitante	Hardware (PC/móvil)	Landing Page (HTML/CSS/JS)
GitHub Pages	Cloud (static hosting)	Landing Page
Firebase App Distribution	Cloud (distribución)	APK/AAB/IPA de las apps
AWS EC2	Cloud (IaaS)	Servicios web (Spring Boot)
PostgreSQL (AWS EC2)	Cloud (IaaS)	Base de datos relacional
Firebase Cloud Messaging	Cloud (push)	Notificaciones push a las apps

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

Relaciones entre nodos:

- El navegador del visitante accede al Landing Page alojado en GitHub Pages vía HTTPS.
- Los dispositivos móviles descargan las apps desde Firebase App Distribution (pruebas) o las tiendas oficiales (producción).
- Las apps se comunican con los servicios web desplegados en AWS EC2 mediante HTTPS/REST.
- Los servicios web acceden a PostgreSQL para la persistencia de datos.
- Los servicios web envían notificaciones push a las apps a través de Firebase Cloud Messaging.

4.2. Landing Page & Mobile Application Implementation

4.2.1. Sprint 1

4.2.1.1. Sprint Planning 1

A continuación se presentan los aspectos principales de la reunión de planificación del Sprint 1 de **SaludYa**. En esta reunión el equipo **RuwaLabs** definió el Sprint Goal, el alcance de user stories enfocado en el paciente y la capacidad de trabajo del primer sprint, como se detalla en la Tabla 70.

Tabla 70

Sprint Planning 1 — resumen de la reunión

Campo	Valor
Sprint #	Sprint 1
Sprint Planning Background	
Date	2026-09-26
Time	7:00 PM
Location	Reunión virtual mediante Jitsi Meet
Prepared By	Montoya Nina, Paula Fernanda
Attendees (to planning meeting)	Aguilar Untiveros, Rodrigo Fabrizio / Meza Solórzano, Didier Sebastian / Montoya Nina, Paula Fernanda / Ramos Mera, Neo Daniel / Torres Juárez, Alisee Muriel
Sprint 1 – 1 Review Summary	No aplica (primer sprint del proyecto).
Sprint 1 – 1 Retrospective Summary	No aplica (primer sprint del proyecto).
Sprint Goal & User Stories	
Sprint 1 Goal	Habilitar el registro y la reserva de citas médicas del paciente en la aplicación móvil y publicar el Landing Page con la propuesta de valor de SaludYa.
Sprint 1 Velocity	99 story points
Sum of Story Points	99

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

El **Sprint Goal** se redactó siguiendo el enfoque propuesto por Scrum.org, priorizando el resultado (Outcome), el impacto (Impact) y la validación (Event):

> **Sprint 1 Goal:** *Nuestro **focus** está en habilitar el registro y la reserva de citas médicas del paciente desde la aplicación móvil y en presentar la propuesta de valor de SaludYa en el Landing Page.*
>
> *Creemos que esto **delivers** un acceso rápido, sin colas e informado a los pacientes de establecimientos públicos de salud.*
>
> *Esto se confirmará (**confirmed**) cuando un paciente pueda verificar su identidad, registrar su cuenta, iniciar sesión, reservar, consultar y cancelar una cita desde la aplicación, y cuando un visitante pueda comprender la propuesta de valor y acceder a la descarga desde el Landing Page publicado.*

La **velocity** del Sprint 1 se estableció en **99 story points**, equivalente a la suma de la estimación de las 30 user stories comprometidas (US-00 a US-29) del Product Backlog (Tabla 22). Al tratarse del primer sprint

del proyecto, el equipo no cuenta con una velocity histórica, por lo que se toma como referencia el total comprometido; este valor se ajustará en los siguientes sprints en función de la velocidad real observada.

El alcance del Sprint 1 se compone de las user stories orientadas al paciente y al Landing Page, agrupadas por épica, como se detalla en la Tabla 71.

Tabla 71

Alcance del Sprint 1 por épica

Épica	User Stories	Story Points
EP0: Plataforma de Presentación y Captación	US-00	2
EP1: Authentication & Identity Management	US-01, US-02, US-03, US-04, US-15, US-17, US-18, US-19, US-23, US-24, US-25, US-26, US-27, US-28, US-29	49
EP2: Appointments & Booking Engine	US-05, US-06, US-07, US-08, US-09, US-10, US-16	34
EP3: Reassignment Protocol	US-20, US-21	8
EP4: Arrival & QR Check-in System	US-11, US-12, US-13, US-14, US-22	14
Total	**30 user stories**	**99**

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

4.2.1.2. Aspect Leaders and Collaborators

En el Sprint 1 el equipo organizó el trabajo en función de los productos y bounded contexts incluidos en el alcance: el **Landing Page** y la aplicación móvil del paciente, compuesta por **Identity & Access Management (IAM)**, **Appointments & Booking**, **Dynamic Waitlist & Reassignment**, **Arrival & QR Check-in** y **Hospital Operations & Configuration**, junto con los servicios web que los soportan. La siguiente matriz LACX (Leadership-and-Collaboration Matrix) indica, para cada aspecto, quién asume el rol de **líder (L)** y quién de **colaborador (C)**, como se detalla en la Tabla 72. Esta organización guarda relación con la posterior asignación de tasks en el Sprint Backlog.

Tabla 72

Leadership-and-Collaboration Matrix (LACX) — Sprint 1

Team Member (Last Name, First Name)	GitHub Username	Landing Page	IAM	Appointments & Booking	Dynamic Waitlist & Reassignment	Arrival & QR Check-in	Hospital Operations & Configuration
Aguilar Untiveros, Rodrigo Fabrizio	Rodri2712	L	C	C	C	C	L
Meza Solórzano, Didier Sebastian	DidierSebas	C	L	C	C	C	C

Team Member (Last Name, First Name)	GitHub Username	Landing Page	IAM	Appointments & Booking	Dynamic Waitlist & Reassignment	Arrival & QR Check-in	Hospital Operations & Configuration
Montoya Nina, Paula Fernanda	SeviNyO	C	C	L	C	C	C
Ramos Mera, Neo Daniel	norahccccc	C	C	C	L	C	C
Torres Juárez, Alisee Muriel	lLisee1	C	C	C	C	L	C

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa. L = Leader, C = Collaborator.

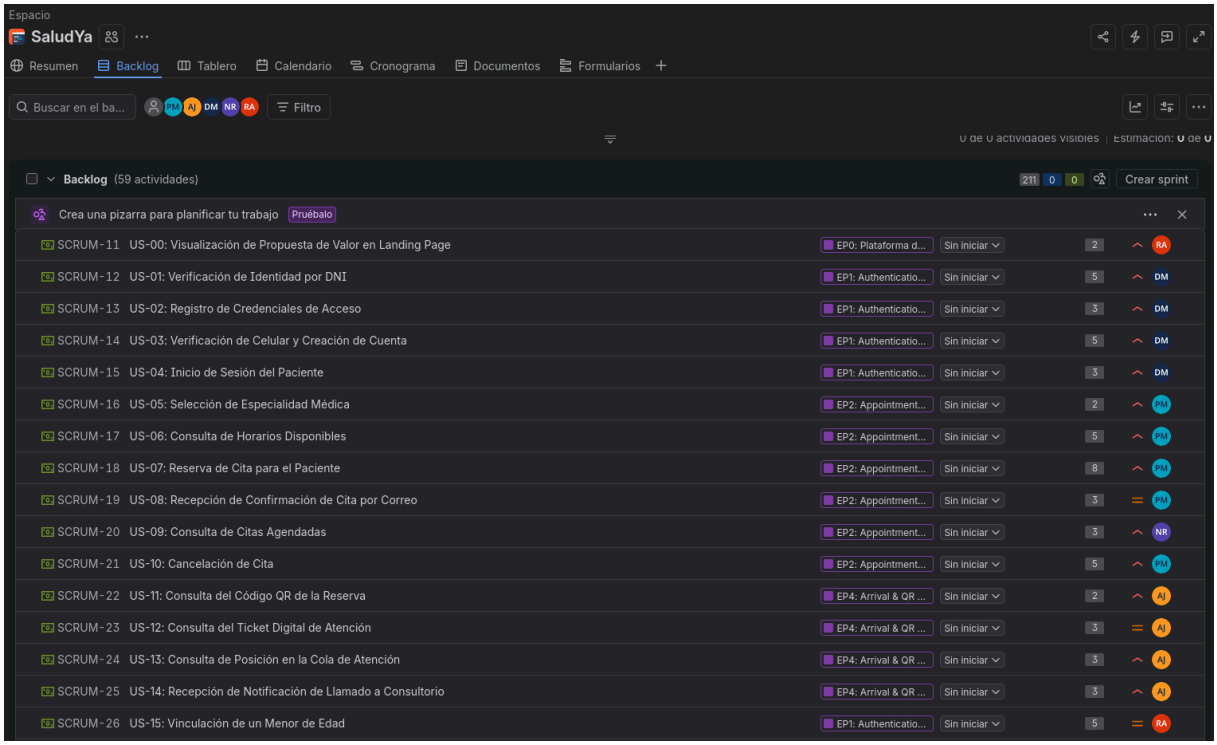
4.2.1.3. Sprint Backlog 1

El Sprint 1 tiene como objetivo habilitar el registro, la reserva y el seguimiento de las citas del paciente en la aplicación móvil, así como publicar el Landing Page con la propuesta de valor de SaludYa. Para su gestión, el equipo utilizó **Jira** como herramienta de control; el tablero del backlog del proyecto se puede revisar en el siguiente enlace y en la Figura 153.

- Link Jira: https://ruwalabs-salud-ya.atlassian.net/

Figura 153

Backlog del proyecto SaludYa en Jira



Nota. Captura del tablero Backlog del proyecto SaludYa en Jira.

A partir de las user stories comprometidas, el equipo descompuso el trabajo en las siguientes tasks (work--items), con su estimación en horas, responsable y estado, como se detalla en la Tabla 73.

Tabla 73

Sprint Backlog 1

#	User Story	Work-Item / Task	Estimation (Hours)	Assigned To	Status
1	US-00 · Visualización de Propuesta de Valor en Landing Page	T-01 · Diseño del Landing Page (Figma)	6	Rodrigo	Done
2	US-00 · Visualización de Propuesta de Valor en Landing Page	T-02 · Desarrollo frontend del Landing Page	12	Rodrigo	Done
3	US-00 · Visualización de Propuesta de Valor en Landing Page	T-03 · Despliegue del Landing Page (GitHub Pages)	4	Rodrigo	Done
4	US-01 · Verificación de Identidad por DNI	T-04 · Backend: validación de identidad por DNI	8	Didier	Done
5	US-01 · Verificación de Identidad por DNI	T-05 · Frontend: pantalla de verificación de identidad	6	Didier	Done
6	US-02 · Registro de Credenciales de Acceso	T-06 · Backend: registro de credenciales y verificación por correo	8	Didier	Done
7	US-02 · Registro de Credenciales de Acceso	T-07 · Frontend: formulario de credenciales de acceso	6	Didier	Done
8	US-03 · Verificación de Celular y Creación de Cuenta	T-08 · Backend: verificación de contacto y creación de cuenta	6	Didier	Done
9	US-03 · Verificación de Celular y Creación de Cuenta	T-09 · Frontend: pantalla de verificación por correo	6	Didier	Done
10	US-04 · Inicio de Sesión del Paciente	T-10 · Backend: autenticación y emisión de sesión	6	Didier	Done
11	US-04 · Inicio de Sesión del Paciente	T-11 · Frontend: pantalla de inicio de sesión	5	Didier	Done
12	US-05 · Selección de Especialidad Médica	T-12 · Frontend: pantalla de selección de especialidad	5	Paula	Done
13	US-06 · Consulta de Horarios Disponibles	T-13 · Backend: consulta de horarios disponibles	8	Paula	Done
14	US-06 · Consulta de Horarios Disponibles	T-14 · Frontend: pantalla de calendario y horarios	6	Paula	Done

#	User Story	Work-Item / Task	Estimation (Hours)	Assigned To	Status
15	US-07 · Reserva de Cita para el Paciente	T-15 · Backend: registro de reserva de cita	10	Paula	Done
16	US-07 · Reserva de Cita para el Paciente	T-16 · Frontend: flujo de reserva (fecha, hora y confirmación)	10	Paula	Done
17	US-08 · Recepción de Confirmación de Cita por Correo	T-17 · Backend: envío de confirmación de cita por correo	6	Paula	Done
18	US-09 · Consulta de Citas Agendadas	T-18 · Backend: consulta de citas agendadas	6	Paula	Done
19	US-09 · Consulta de Citas Agendadas	T-19 · Frontend: pantalla de citas pendientes	6	Neo	Done
20	US-10 · Cancelación de Cita	T-20 · Backend: cancelación de cita	6	Paula	Done
21	US-10 · Cancelación de Cita	T-21 · Frontend: acción de cancelación de cita	5	Paula	Done
22	US-11 · Consulta del Código QR de la Reserva	T-22 · Backend: generación del código QR de la reserva	6	Alise	Done
23	US-11 · Consulta del Código QR de la Reserva	T-23 · Frontend: pantalla del código QR de la reserva	5	Alise	Done
24	US-12 · Consulta del Ticket Digital de Atención	T-24 · Backend: consulta del ticket digital de atención	6	Alise	Done
25	US-12 · Consulta del Ticket Digital de Atención	T-25 · Frontend: pantalla del ticket de atención	5	Alise	Done
26	US-13 · Consulta de Posición en la Cola de Atención	T-26 · Backend: consulta de posición en la cola de atención	6	Alise	Done
27	US-13 · Consulta de Posición en la Cola de Atención	T-27 · Frontend: pantalla de posición en la cola	5	Alise	Done
28	US-14 · Recepción de Notificación de Llamado a Consultorio	T-28 · Backend: notificación de llamado a consultorio	6	Alise	Done
29	US-14 · Recepción de Notificación de Llamado a Consultorio	T-29 · Frontend: aviso de llamado en la aplicación	5	Neo	Done
30	US-15 · Vinculación de un Menor de Edad	T-30 · Backend: vinculación de un menor de edad	8	Didier	Done

#	User Story	Work-Item / Task	Estimation (Hours)	Assigned To	Status
31	US-15 · Vinculación de un Menor de Edad	T-31 · Frontend: pantalla de registro de menor	6	Rodrigo	Done
32	US-16 · Reserva de Cita para un Menor Vinculado	T-32 · Backend: reserva de cita para un menor vinculado	8	Paula	Done
33	US-16 · Reserva de Cita para un Menor Vinculado	T-33 · Frontend: selección de menor en el flujo de reserva	6	Paula	Done
34	US-17 · Consulta de Menores Vinculados	T-34 · Frontend: pantalla de menores vinculados	5	Rodrigo	Done
35	US-18 · Consulta del Perfil de un Menor	T-35 · Frontend: pantalla del perfil de un menor	5	Rodrigo	Done
36	US-19 · Desvinculación de un Menor de Edad	T-36 · Backend: desvinculación de un menor de edad	5	Didier	Done
37	US-19 · Desvinculación de un Menor de Edad	T-37 · Frontend: acción de desvinculación de menor	5	Rodrigo	Done
38	US-20 · Aceptación de Adelanto de Cita	T-38 · Backend: reasignación de cupos y oferta de adelanto	10	Neo	Done
39	US-20 · Aceptación de Adelanto de Cita	T-39 · Frontend: pantalla de aceptación de reasignación	6	Neo	Done
40	US-21 · Rechazo de Adelanto de Cita	T-40 · Frontend: acción de rechazo de reasignación	5	Neo	Done
41	US-22 · Salida Voluntaria de la Cola de Atención	T-41 · Backend: salida voluntaria de la cola de atención	5	Alise	Done
42	US-22 · Salida Voluntaria de la Cola de Atención	T-42 · Frontend: acción de dejar la cola	5	Alise	Done
43	US-23 · Solicitud de Enlace de Recuperación de Contraseña	T-43 · Backend: generación del enlace de recuperación	6	Didier	Done
44	US-23 · Solicitud de Enlace de Recuperación de Contraseña	T-44 · Frontend: pantalla de solicitud de recuperación	5	Didier	Done
45	US-24 · Restablecimiento de Contraseña	T-45 · Backend: restablecimiento de contraseña	6	Didier	Done
46	US-24 · Restablecimiento de Contraseña	T-46 · Frontend: pantalla de nueva contraseña	5	Didier	Done

#	User Story	Work-Item / Task	Estimation (Hours)	Assigned To	Status
47	US-25 · Solicitud de Ayuda por Pérdida de Acceso a la Cuenta	T-47 · Backend: registro de solicitud de ayuda de acceso	5	Didier	Done
48	US-25 · Solicitud de Ayuda por Pérdida de Acceso a la Cuenta	T-48 · Frontend: pantalla de solicitud de ayuda	5	Didier	Done
49	US-26 · Consulta del Perfil del Paciente	T-49 · Backend: consulta del perfil del paciente	5	Didier	Done
50	US-26 · Consulta del Perfil del Paciente	T-50 · Frontend: pantalla del perfil del paciente	5	Rodrigo	Done
51	US-27·Edición de Datos de Contacto	T-51 · Backend: actualización de datos de contacto	6	Didier	Done
52	US-27·Edición de Datos de Contacto	T-52 · Frontend: pantalla de edición de datos de contacto	5	Rodrigo	Done
53	US-28 · Cierre de Sesión del Paciente	T-53 · Backend: cierre de sesión	4	Didier	Done
54	US-28 · Cierre de Sesión del Paciente	T-54 · Frontend: acción de cierre de sesión	4	Didier	Done
55	US-29 · Aviso de Sesión Expirada	T-55 · Frontend: aviso de sesión expirada	4	Didier	Done
56	— (Task transversal)	T-56 · Backend: despliegue de los servicios web	8	Neo	Done

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa. El estado de las tasks corresponde al avance real del Sprint 1.

4.2.1.4. Development Evidence for Sprint Review

Durante el Sprint 1 se avanzó en la implementación del backend, la aplicación móvil para pacientes y la landing page de SaludYa. El trabajo se distribuyó en ramas por funcionalidad, con commits que registran la construcción de los módulos y sus ajustes de integración.

En el backend, desarrollado con Java y Spring Boot, se implementaron las API de identidad y acceso, las reservas y la configuración operativa. La rama `develop` incorpora la integración de IAM con el registro de llegada, los ajustes de reasignación según la tolerancia de check-in y la verificación por correo mediante SMTP. Este último cambio reemplaza el envío de códigos por SMS. También se registraron avances en la recuperación de cuenta y la configuración de despliegue en AWS EC2.

En la aplicación Android, desarrollada con Kotlin y Jetpack Compose, se implementaron los recorridos de recuperación de acceso, la presentación y las capas de reservas, la consulta del QR de reserva, el ticket y la cola de atención. El módulo de configuración incorpora la validación y desvinculación de menores. Las ramas de dashboard y notificaciones registran avances en navegación y presentación; estos commits documentan ese alcance, sin implicar que todos sus recorridos estén integrados de extremo a extremo.

La landing page reúne la presentación del producto y recibió ajustes en la imagen principal y los vídeos. Su repositorio también contiene el flujo de publicación con GitHub Pages. Los avances de implementación se respaldan con los commits seleccionados de los tres repositorios, como se detalla en la Tabla 74.

Tabla 74

Commits de implementación de SaludYa para la revisión del Sprint 1

Repository	Branch	Commit Id	Commit Message	Committed On
RuwaLabs/backend-saludya	iam	d6101f67c169deea63…	feat(iam): expose authorized REST APIs and context facade	03/10/2026
RuwaLabs/backend-saludya	booking	f5823addf79daa1c7b…	feat: Booking DDD Complete	03/10/2026
RuwaLabs/backend-saludya	reassignment	d4daffc364ace3799c…	feat: ddl with acl impl	26/09/2026
RuwaLabs/backend-saludya	hospitalconfig	89934b6cd4f503e9b4…	feat(config): add configuration rest controller	26/09/2026
RuwaLabs/backend-saludya	develop	4fb041aac3b5912de3…	feat: integrate IAM and Arrival with cross-context ACLs; complete patient/admin flows	06/10/2026
RuwaLabs/backend-saludya	develop	89c4529678231739c7…	feat: reassignment arrival by slot check-in tolerance and fix chain candidate selection	03/10/2026
RuwaLabs/backend-saludya	develop	8965e195d438b3c653…	fix: auth smtp done, sms removed, flow reworked	06/10/2026
RuwaLabs/backend-saludya	develop	50e5453fe97c7f4822…	feat: recovery audition	06/10/2026
RuwaLabs/backend-saludya	develop	6a747f8b711abe3e69…	chore: add Render deployment	06/10/2026
RuwaLabs/frontend-kotlin-saludya	iam	ca1cb00eceb91035cf…	feat(iam): add recovery screens and session confirmation flows	07/10/2026
RuwaLabs/frontend-kotlin-saludya	iam	9c5a3199d760155e0f…	feat(iam): validate recovery links and	07/10/2026

Repository	Branch	Commit Id	Commit Message	Committed On
			reset patient passwords	
RuwaLabs/frontend-kotlin-saludya	iam	71074b8f6a4328ec18[illegible]	feat(iam): register assisted account recovery requests	07/10/2026
RuwaLabs/frontend-kotlin-saludya	patient-reservations	a91f61d5996836674e[illegible]	feat(patient-reservations): add domain, use cases and REST infrastructure	06/10/2026
RuwaLabs/frontend-kotlin-saludya	patient-reservations	10161af7987f2c04f8[illegible]	feat(patient-reservations): alinear presentacion con patrones del proyecto	09/10/2026
RuwaLabs/frontend-kotlin-saludya	patient-check-in	a07e66938201261[illegible]	feat(check-in): add Reservation QR code Lookup, Digital Service Ticket Inquiry, Service Queue, Call to the Consulting Room	07/10/2026
RuwaLabs/frontend-kotlin-saludya	patient-check-in	93f0b140cf2af30ef025[illegible]	fix(check-in): update us qr code, ticket, attention queue	09/10/2026
RuwaLabs/frontend-kotlin-saludya	patient-configuration	c52be13f80184c6976[illegible]	feat(configuration): validate minor identity against admission records	06/10/2026
RuwaLabs/frontend-kotlin-saludya	patient-configuration	8cd77e9d6a86af270b[illegible]	feat(configuration): add confirm dialog and unlink flow to linked minor	06/10/2026
RuwaLabs/frontend-kotlin-saludya	patient-dashboard	b0a5ab82c4aae6a7ba[illegible]	feat(design-system): add BottomNavigationBar and bottom nav icons	04/10/2026
RuwaLabs/frontend-kotlin-saludya	notifications-reassignment	e4edf27e77712c18bb[illegible]	feat: ui composables done, presentation almost done. Dashboard needs	09/10/2026

Repository	Branch	Commit Id	Commit Message	Committed On
			other bounded contexts	
RuwaLabs/frontend-kotlin-saludya	develop	8b721c467ba0c0e0c85[illegible]	feat(main): custom tab indicator covering icon and label	09/10/2026
RuwaLabs/saludya-landing	main	28d248984503e0e8f1[illegible]	chore: initial commit - SaludYa landing page	28/09/2026
RuwaLabs/saludya-landing	main	4c1a8378572855facb6[illegible]	feat(hero): update hero img with best practices	28/09/2026
RuwaLabs/saludya-landing	main	e722cfdec71da9c8db[illegible]	ci: add GitHub Pages deployment workflow	03/10/2026
RuwaLabs/saludya-landing	main	d90b6f674b999ef94e[illegible]	feat: default videos updated	09/10/2026

Nota. Elaboración del equipo RuwaLabs a partir del historial de Git de los repositorios de SaludYa, consultado el 9 de octubre de 2026. Los mensajes se conservan tal como fueron registrados y las fechas corresponden a la fecha de commit. La columna Branch identifica una rama que contiene el commit; no atribuye su creación original a esa rama. Se presentan commits representativos de implementación, sin incluir los commits de edición del reporte.

4.2.1.5. Testing Suite Evidence for Sprint Review

Las pruebas del Sprint 1 verifican las reglas de identidad y acceso, los servicios REST y la protección de los recursos del paciente. En el backend de SaludYa se utiliza JUnit Jupiter con AssertJ para comprobar los componentes de seguridad y validación. Las pruebas de integración emplean Spring Boot Test y MockMvc para ejecutar solicitudes a los endpoints, revisar sus respuestas y comprobar los cambios en la persistencia con el perfil `test`.

La suite de IAM cubre el registro, la verificación por correo, el inicio de sesión, la recuperación de contraseña, el cierre y la expiración de sesión, y la vinculación de menores. Incluye casos de datos inválidos, correos duplicados, credenciales incorrectas, enlaces vencidos y acceso a información de otro paciente. La protección de recursos también se verifica para reservas, check-in y propuestas de reasignación.

El archivo `iam.feature` expresa escenarios de aceptación en Gherkin. Su cobertura automatizada se implementa en las clases JUnit de integración y seguridad; el archivo funciona como especificación de aceptación y no cuenta con un ejecutor Cucumber configurado en el proyecto. Algunos de sus escenarios contemplan roles internos, mientras que el alcance móvil de este entregable corresponde al paciente.

En Android se utilizan JUnit, pruebas de corrutinas y MockWebServer para validar formularios, ViewModels y repositorios HTTP. Las pruebas instrumentadas con Compose UI Test comprueban las pantallas de identidad, registro, acceso, recuperación y sesión. Los commits que incorporan o modifican estas pruebas se presentan con su repositorio, rama, identificador, mensaje original y fecha, como se detalla en la Tabla 75.

Repositorio de pruebas del backend y especificación BDD: RuwaLabs/backend-saludya.

Repositorio de pruebas Android: RuwaLabs/frontend-kotlin-saludya.

Tabla 75

Commits relacionados con la suite de pruebas del Sprint 1

Repository	Branch	Commit Id	Commit Message	Committed On
RuwaLabs/backend--saludya	iam	38265f3ed2efd7b08c5dd8	test(iam): cover report scenarios on H2 and PostgreSQL	03/10/2026
RuwaLabs/backend--saludya	develop	4fb041aac3b5912de3a32	feat: integrate IAM and Arrival with cross-context ACLs; complete patient/admin flows	03/10/2026
RuwaLabs/backend--saludya	develop	8965e195d438b3c658b2	fix: add smtp done, sms removed, flow reworked	06/10/2026
RuwaLabs/frontend--kotlin-saludya	iam	0c6ffdd50e27eafe346187	feat(iam): validate patient registration credentials	07/10/2026
RuwaLabs/frontend--kotlin-saludya	iam	9c5a3199d760155e0f2bb	feat(iam): validate recovery links and reset patient passwords	07/10/2026
RuwaLabs/frontend--kotlin-saludya	iam	71074b8f6a4328ec18753	feat(iam): register assisted account recovery requests	07/10/2026
RuwaLabs/frontend--kotlin-saludya	iam	ca1cb00eceb91035cf7595	feat(iam): add recovery screens and session confirmation flows	07/10/2026

Nota. Elaboración del equipo RuwaLabs a partir del historial de Git, consultado el 9 de octubre de 2026. Se verificaron los archivos de prueba modificados por cada commit. Los mensajes y fechas se conservan del repositorio; la rama indicada contiene el commit. Esta sección presenta la suite implementada; las capturas de los componentes publicados y las referencias visuales de la aplicación se presentan en el apartado 4.2.1.6.

Los archivos de prueba y su alcance se organizan por componente y tipo de testing, como se detalla en la Tabla 76.

Tabla 76

Suite de pruebas y escenarios de aceptación de SaludYa para el Sprint 1

Componente	Tipo de prueba	Archivos	Alcance relacionado
Backend IAM	Unitarias y de componentes	IamSecurityTests.java	DNI e identidad, indisponibilidad del provee-

Componente	Tipo de prueba	Archivos	Alcance relacionado
			dor, normalización de nombres, BCrypt, cifrado de notificaciones y límites de solicitudes.
Backend IAM	Integración REST y persistencia	IamIntegrationTests.java	Registro y acceso (US-01 a US-04), menores (US-15 y US-17 a US-19), contacto (US-27), recuperación (US-23 a US-25) y sesión (US-28 y US-29).
Backend, recursos del paciente	Integración y autorización	ResourceOwnershipSecurityTests.java	Impedir consultas o acciones sobre reservas, check-in y ofertas de reasignación de otro paciente; rechazo de QR sin reserva válida.
Backend IAM	Especificación de aceptación en Gherkin	iam.feature	Escenarios Given/When/Then de registro, menores, recuperación y permisos; cobertura ejecutable en JUnit.
Android IAM	Unitarias de validación y estado	Pruebas JVM de IAM	`IdentityValidatorTest`, `CredentialsValidatorTest`, `IdentityViewModelTest` y `RegistrationViewMod` datos de identidad y credenciales (US-01 y US-02).
Android IAM	Repositorios HTTP, acceso y recuperación	Pruebas JVM de IAM	Repositorios remotos, `AccessViewModelsTest`, `PasswordResetLinkTest`, `RecoveryActionsRepositoryTest` y `RecoveryAndSessionViewModelsTest` verificación por correo, acceso, recuperación y sesión.
Android IAM	Interfaz instrumentada	Pruebas de pantallas IAM	`IdentityScreenTest`, `RegistrationScreenTest`, `AccessScreensTest` y `RecoveryAndSessionScre` formularios, mensajes y

Componente	Tipo de prueba	Archivos	Alcance relacionado
			navegación de los recorridos de IAM.

Nota. Elaboración del equipo RuwaLabs a partir de los archivos de prueba del backend y la aplicación Android. La verificación de registro y acceso se realiza por correo, de acuerdo con el cambio implementado en IAM.

4.2.1.6. Execution Evidence for Sprint Review

En esta sección se presentan las vistas publicadas de la Landing Page y la documentación del backend de SaludYa, junto con las pantallas del paciente diseñadas para la aplicación Android. Las capturas permiten revisar la presentación de la solución, los contratos disponibles en Swagger y los recorridos previstos para el paciente durante el Sprint 1.

Landing Page

La Landing Page se encuentra publicada en GitHub Pages y presenta el problema, la propuesta de SaludYa y la información del equipo RuwaLabs. Las siguientes capturas corresponden a su visualización en un navegador de escritorio.

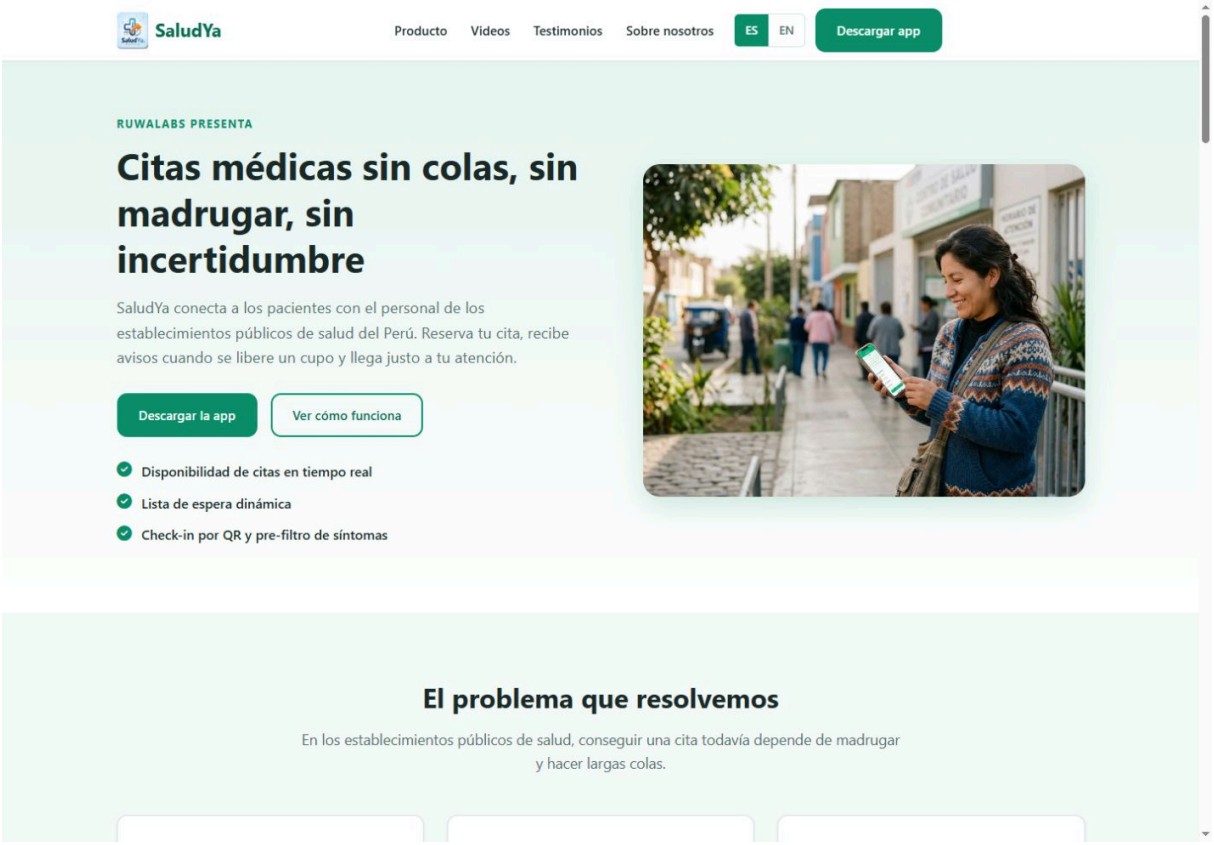
Enlace de despliegue: Landing Page de SaludYa.

Inicio de la Landing Page

La vista inicial presenta el mensaje «Citas médicas sin colas, sin madrugar, sin incertidumbre», la imagen principal y los accesos para conocer el producto y consultar su disponibilidad, como se observa en la Figura 154.

Figura 154

Inicio de la Landing Page — SaludYa



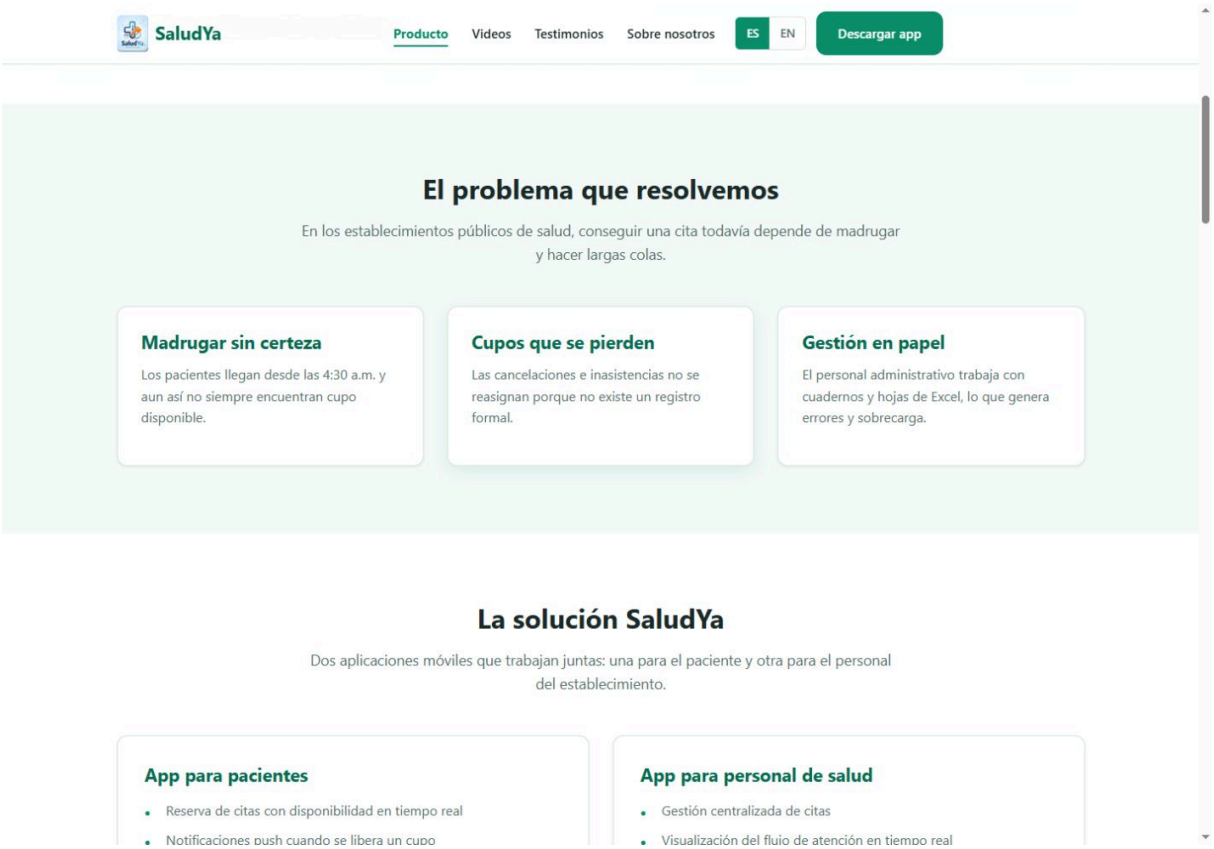
Nota. Captura de la Landing Page publicada en GitHub Pages, tomada el 9 de octubre de 2026.

Problema identificado

La sección describe la incertidumbre al buscar una cita y la pérdida de cupos por cancelaciones e inasistencias, como se observa en la Figura 155.

Figura 155

Problema identificado — SaludYa



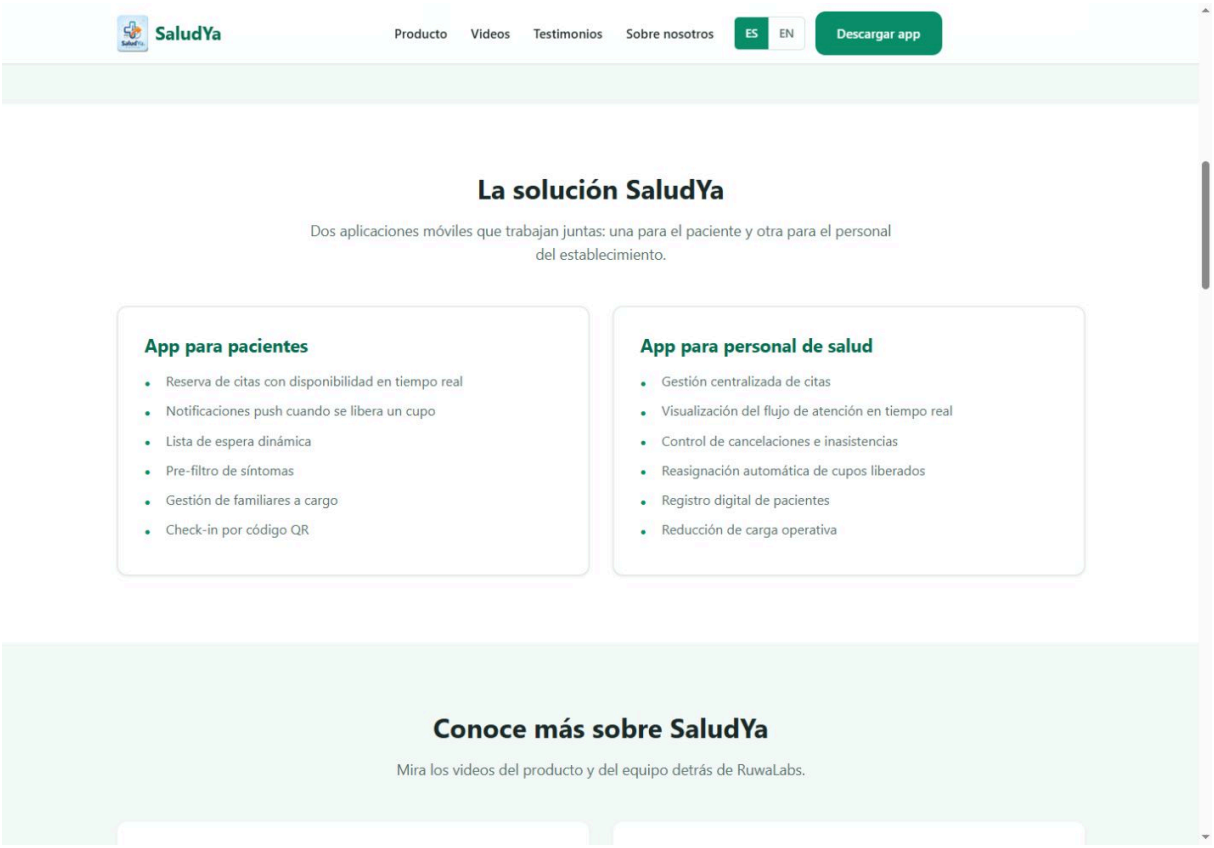
Nota. Captura de la Landing Page publicada en GitHub Pages, tomada el 9 de octubre de 2026.

Propuesta de solución

La sección reúne las funciones propuestas para el paciente, entre ellas la reserva de citas, la lista de espera, las notificaciones y el registro de llegada mediante QR; también presenta la aplicación para el personal de salud, como se observa en la Figura 156.

Figura 156

Propuesta de solución — SaludYa



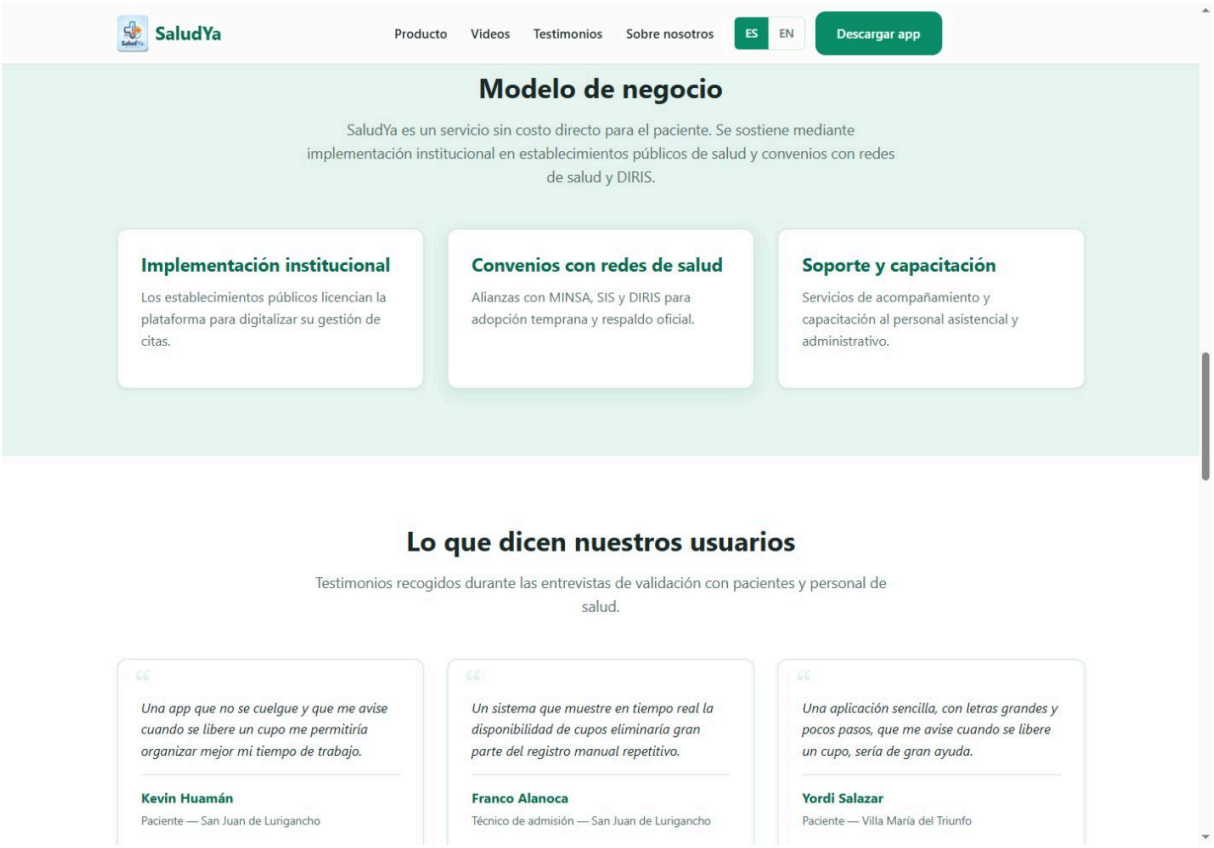
Nota. Captura de la Landing Page publicada en GitHub Pages, tomada el 9 de octubre de 2026.

Modelo de negocio

La página presenta el modelo propuesto de implementación institucional, convenios con redes de salud y soporte. Estos contenidos describen la propuesta comercial del proyecto, como se observa en la Figura 157.

Figura 157

Modelo de negocio — SaludYa



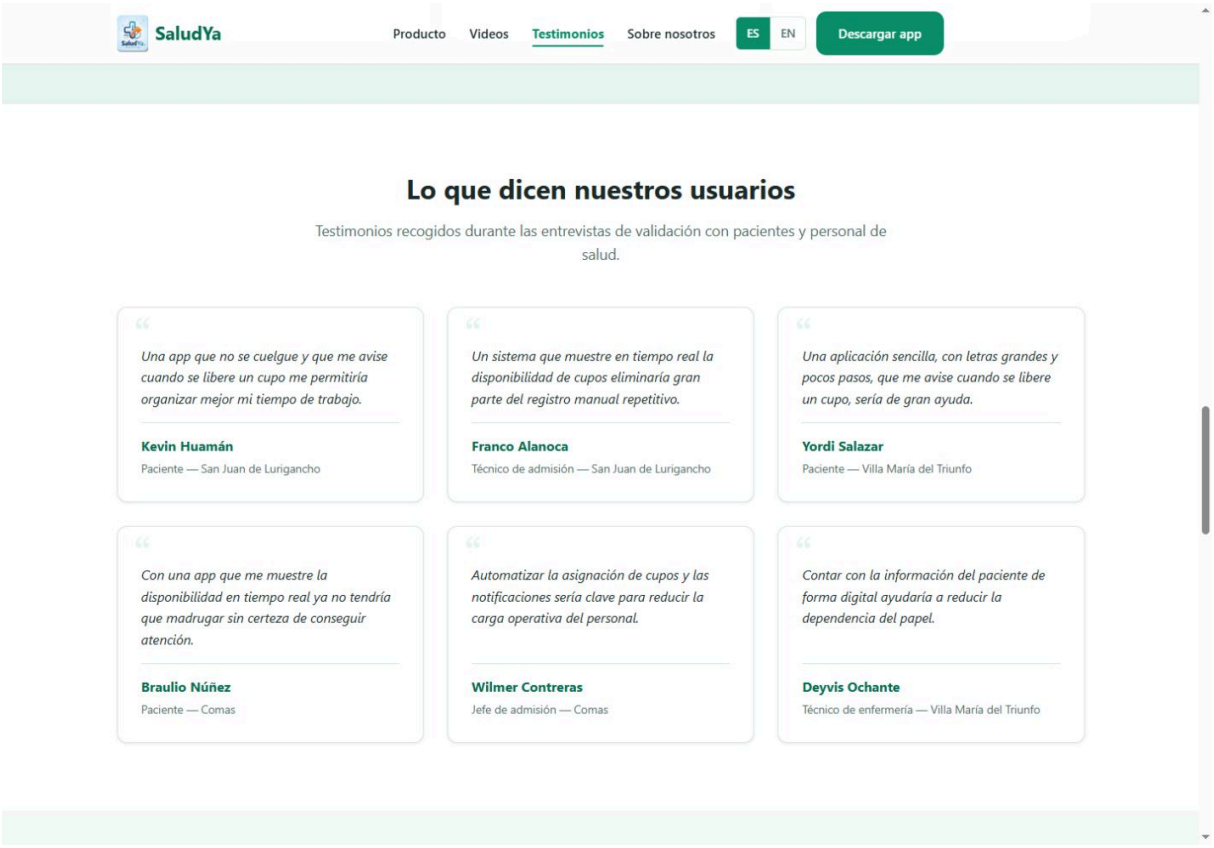
Nota. Captura de la Landing Page publicada en GitHub Pages, tomada el 9 de octubre de 2026.

Testimonios

Los testimonios publicados recogen opiniones sobre la disponibilidad de citas y la digitalización del registro, obtenidas durante las entrevistas de validación, como se observa en la Figura 158.

Figura 158

Testimonios — SaludYa



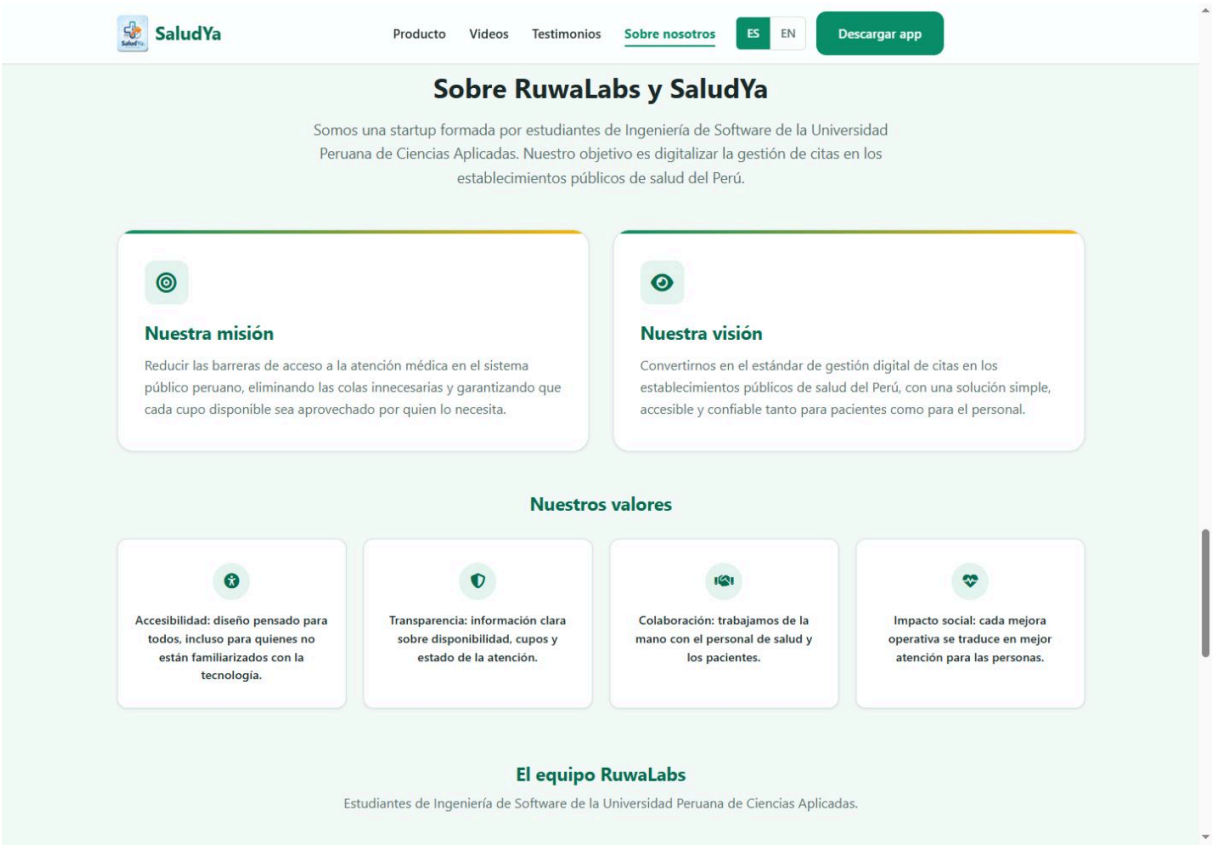
Nota. Captura de la Landing Page publicada en GitHub Pages, tomada el 9 de octubre de 2026.

Presentación de RuwaLabs

La sección explica el propósito del equipo y presenta la misión y visión de SaludYa, como se observa en la Figura 159.

Figura 159

Presentación de RuwaLabs — SaludYa



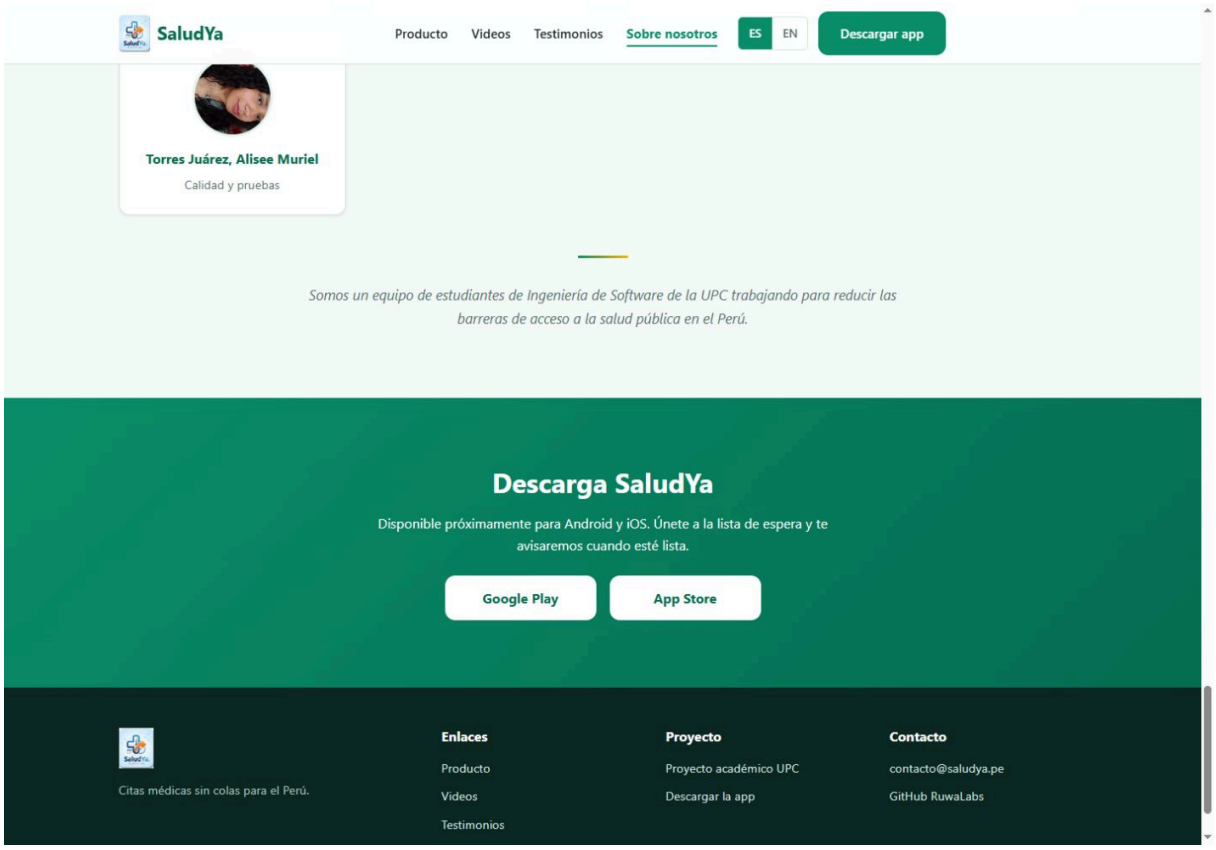
Nota. Captura de la Landing Page publicada en GitHub Pages, tomada el 9 de octubre de 2026.

Disponibilidad de la aplicación

La sección anuncia la próxima disponibilidad para Android e iOS y muestra los botones de las tiendas. En esta captura, estos accesos todavía corresponden al anuncio de disponibilidad futura, como se observa en la Figura 160.

Figura 160

Disponibilidad de la aplicación — SaludYa



Nota. Captura de la Landing Page publicada en GitHub Pages, tomada el 9 de octubre de 2026.

Aplicación Android

La aplicación Android de SaludYa se distribuye mediante un APK de prueba generado a partir de la rama `develop` del repositorio Kotlin, commit `74a4b7f`. La compilación finalizó correctamente y se verificó la firma del paquete. Esta versión requiere Android 7.0 o superior y utiliza el backend publicado en `http://3.129.217.49:8080/`.

Enlace de despliegue: Descargar APK de SaludYa para Android.

Las imágenes siguientes presentan las pantallas del paciente. Las notas de cada figura identifican su origen.

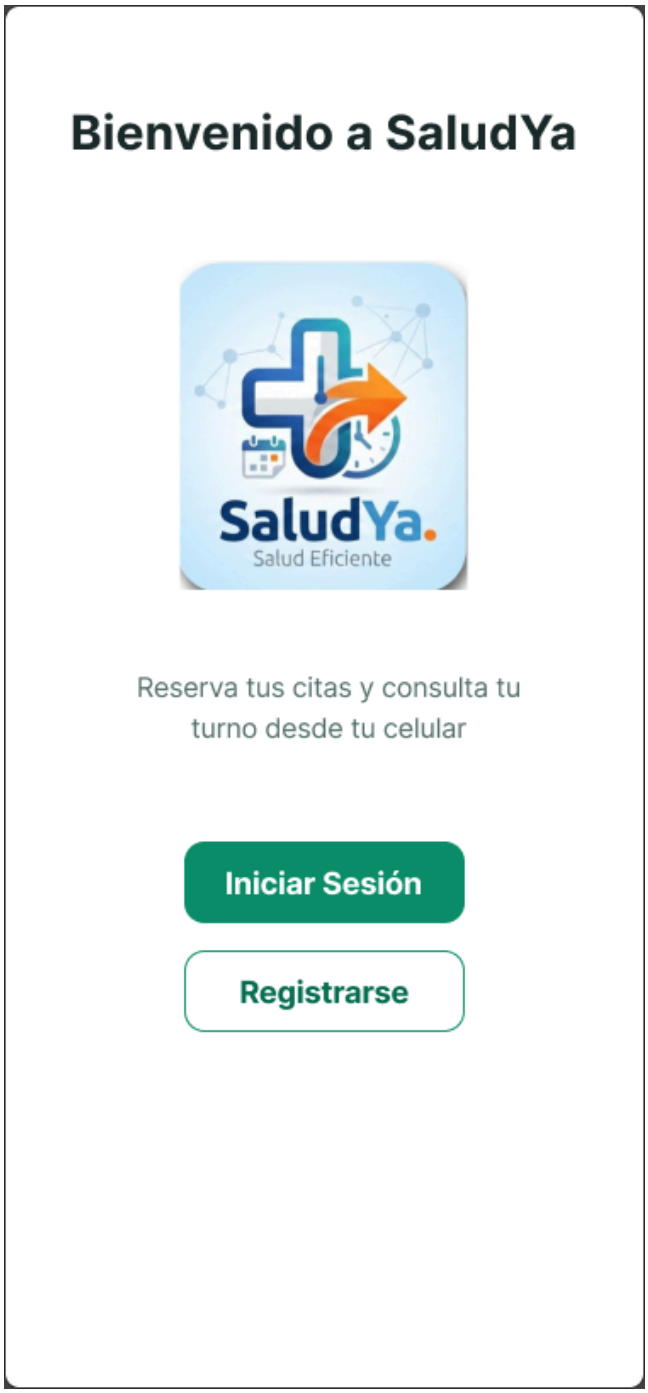
Archivo de interfaz: SaludYa en Figma.

Bienvenida

La pantalla presenta la identidad visual de SaludYa y los accesos para iniciar sesión o crear una cuenta, como se observa en la Figura 161.

Figura 161

Bienvenida — SaludYa



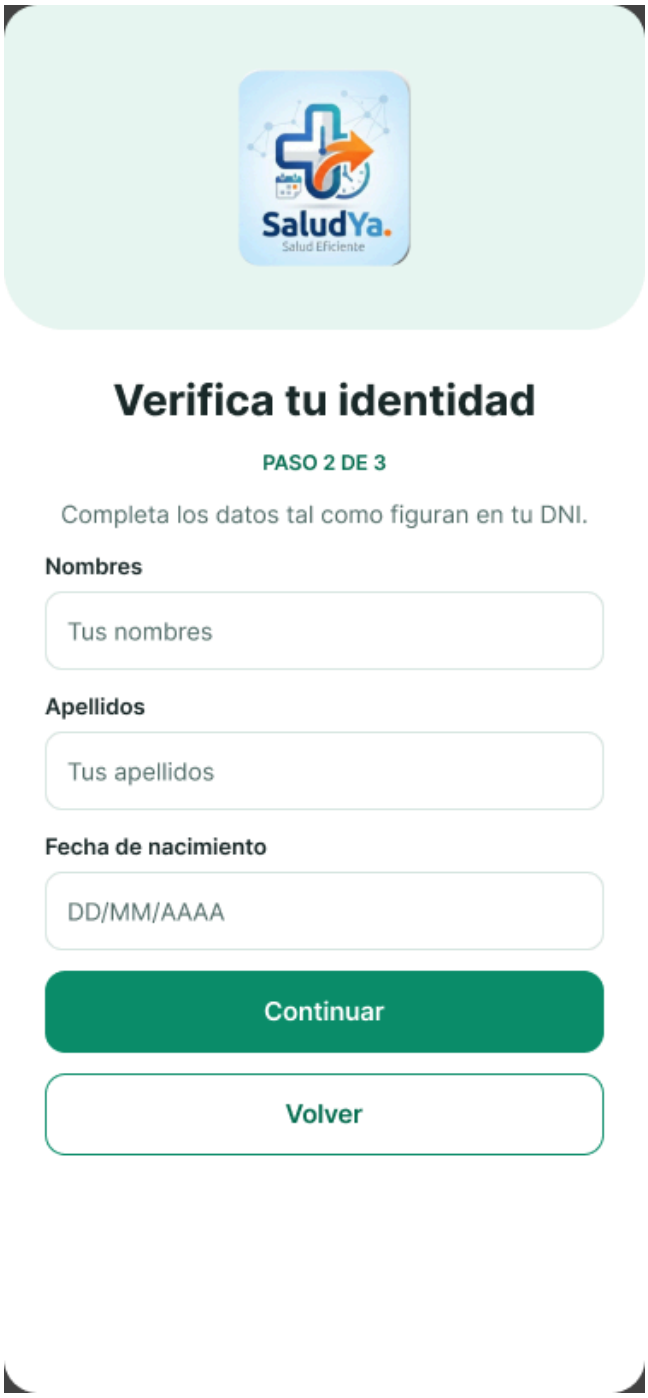
Nota. Interfaz del paciente exportada de Figma; referencia visual, no captura de ejecución del APK.

Verificación de identidad

El formulario solicita los datos de identidad del paciente antes de continuar con la creación de su cuenta, como se observa en la Figura 162.

Figura 162

Verificación de identidad — SaludYa



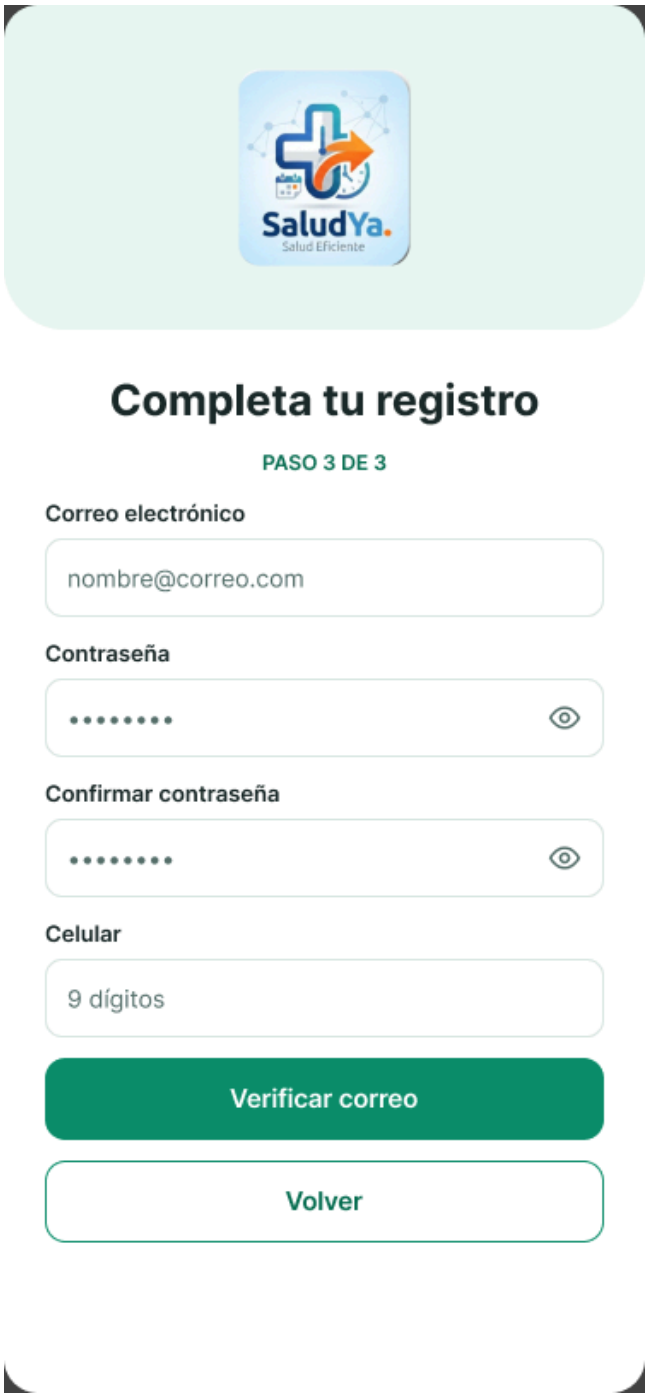
Nota. Interfaz del paciente exportada de Figma; referencia visual, no captura de ejecución del APK.

Registro de credenciales

El paciente registra su correo, contraseña y datos de contacto después de verificar su identidad, como se observa en la Figura 163.

Figura 163

Registro de credenciales — SaludYa



Nota. Interfaz del paciente exportada de Figma; referencia visual, no captura de ejecución del APK.

Verificación por correo

La pantalla solicita el código de verificación enviado al correo del paciente y permite reenviarlo, como se observa en la Figura 164.

Figura 164

Verificación por correo — SaludYa

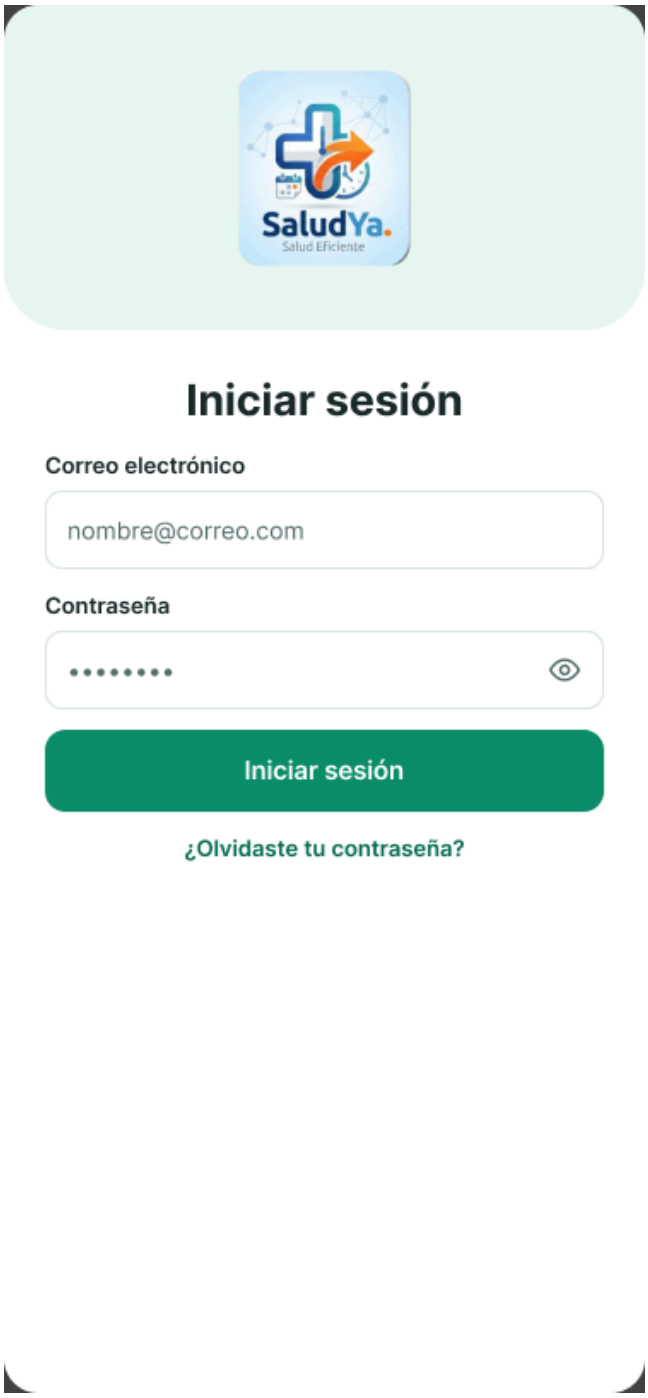
Nota. Interfaz del paciente exportada de Figma; referencia visual, no captura de ejecución del APK.

Inicio de sesión

El paciente ingresa su correo y contraseña para acceder a su cuenta, como se observa en la Figura 165.

Figura 165

Inicio de sesión — SaludYa



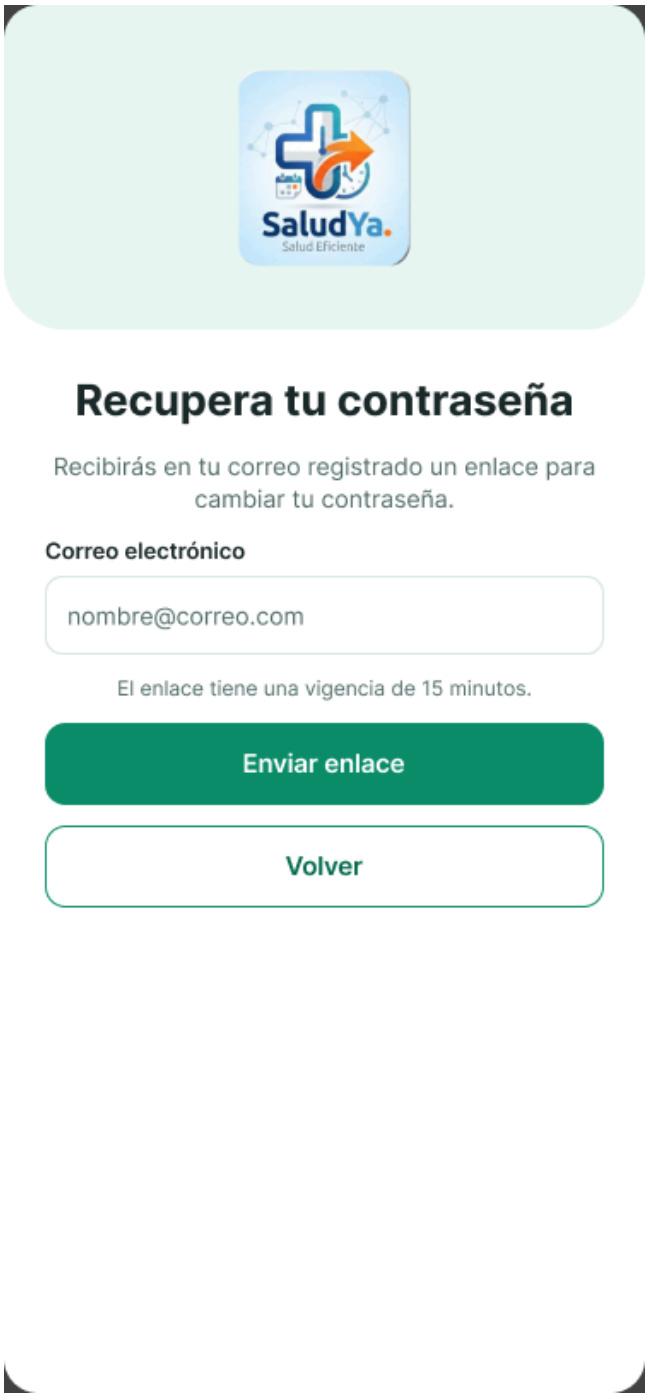
Nota. Interfaz del paciente exportada de Figma; referencia visual, no captura de ejecución del APK.

Recuperación de acceso

El formulario permite solicitar un enlace de recuperación al correo registrado, como se observa en la Figura 166.

Figura 166

Recuperación de acceso — SaludYa



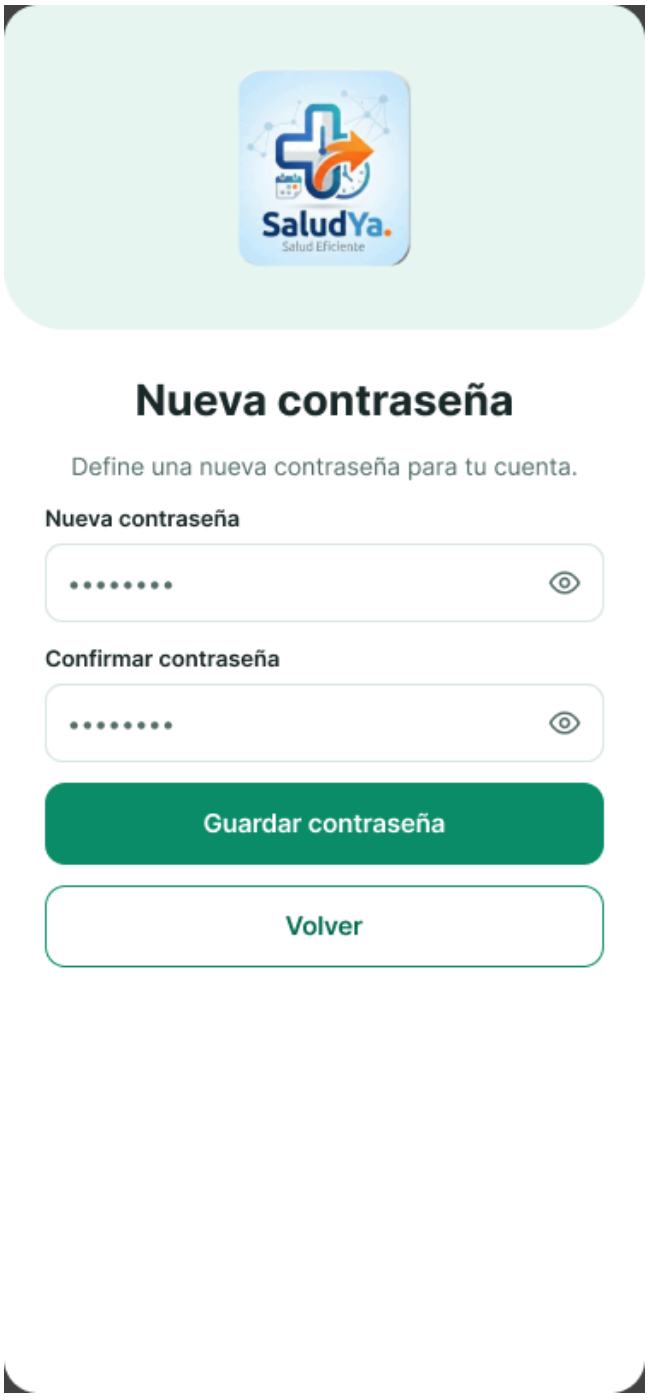
Nota. Interfaz del paciente exportada de Figma; referencia visual, no captura de ejecución del APK.

Restablecimiento de contraseña

El paciente define y confirma una nueva contraseña mediante el recorrido de recuperación, como se observa en la Figura 167.

Figura 167

Restablecimiento de contraseña — SaludYa



Nota. Interfaz del paciente exportada de Figma; referencia visual, no captura de ejecución del APK.

Inicio del paciente

La vista principal reúne las citas pendientes y los accesos al historial y a la reserva de una nueva cita, como se observa en la Figura 168.

Figura 168

Inicio del paciente — SaludYa

Nota. Interfaz del paciente exportada de Figma; referencia visual, no captura de ejecución del APK.

Reserva de cita

El recorrido de reserva presenta las opciones necesarias para elegir una cita según la disponibilidad del establecimiento, como se observa en la Figura 169.

Figura 169

Reserva de cita — SaludYa

Nota. Interfaz del paciente exportada de Figma; referencia visual, no captura de ejecución del APK.

Confirmación de reserva

La pantalla presenta la confirmación y los datos de la cita seleccionada, como se observa en la Figura 170.

Figura 170

Confirmación de reserva — SaludYa

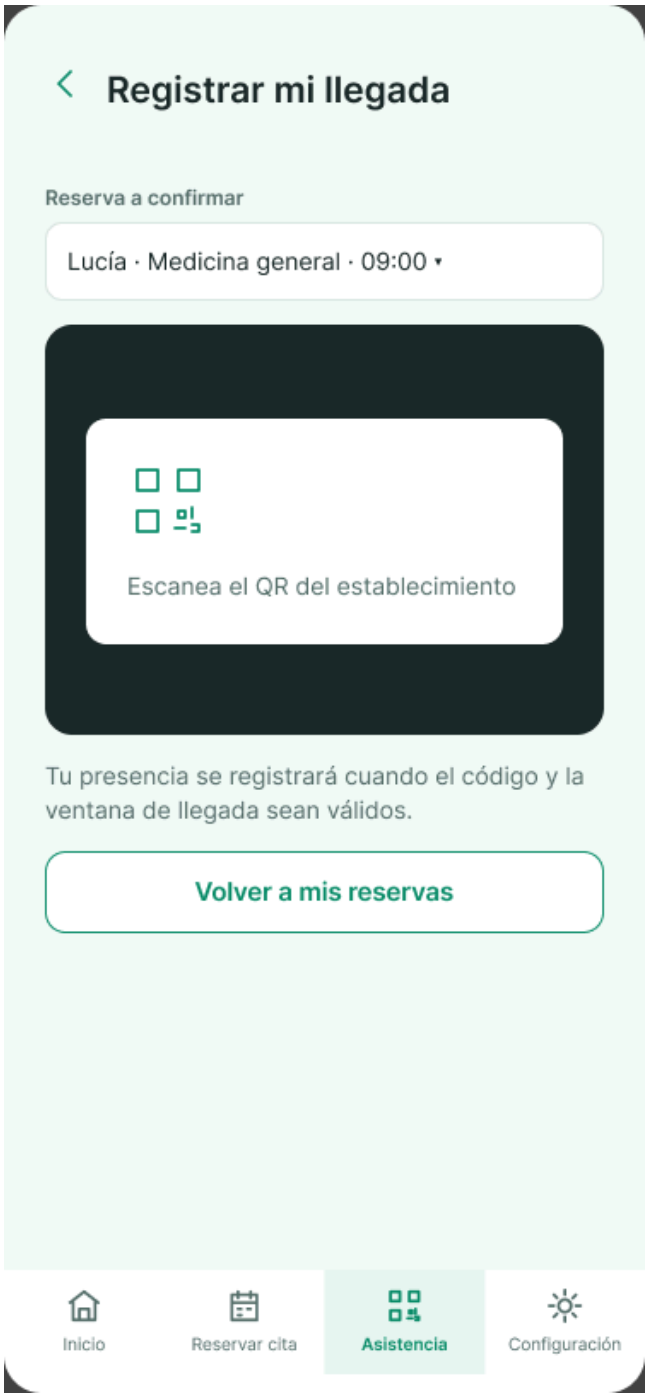
Nota. Interfaz del paciente exportada de Figma; referencia visual, no captura de ejecución del APK.

Registro de llegada

La interfaz indica al paciente que debe escanear el QR del establecimiento para registrar su llegada y confirmar su presencia, como se observa en la Figura 171.

Figura 171

Registro de llegada — SaludYa



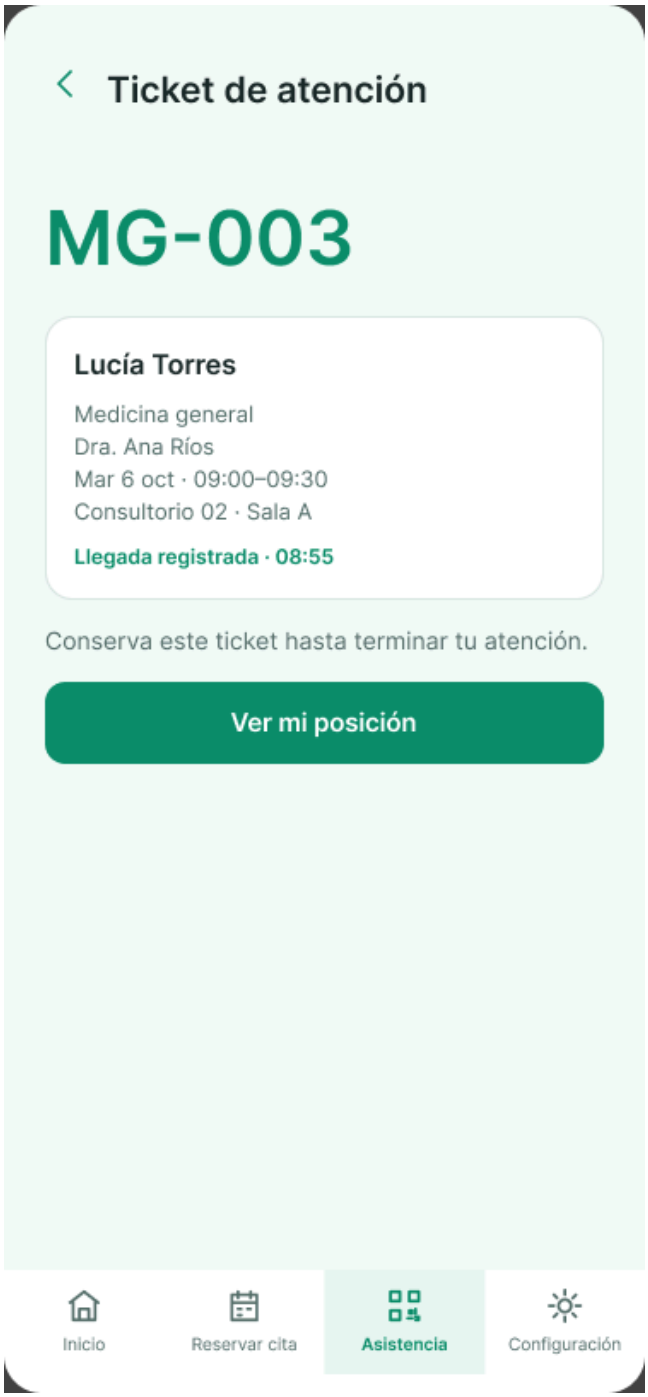
Nota. Interfaz del paciente exportada de Figma; referencia visual, no captura de ejecución del APK.

Ticket de atención

La pantalla presenta el ticket del paciente como parte del seguimiento de su turno de atención, como se observa en la Figura 172.

Figura 172

Ticket de atención — SaludYa



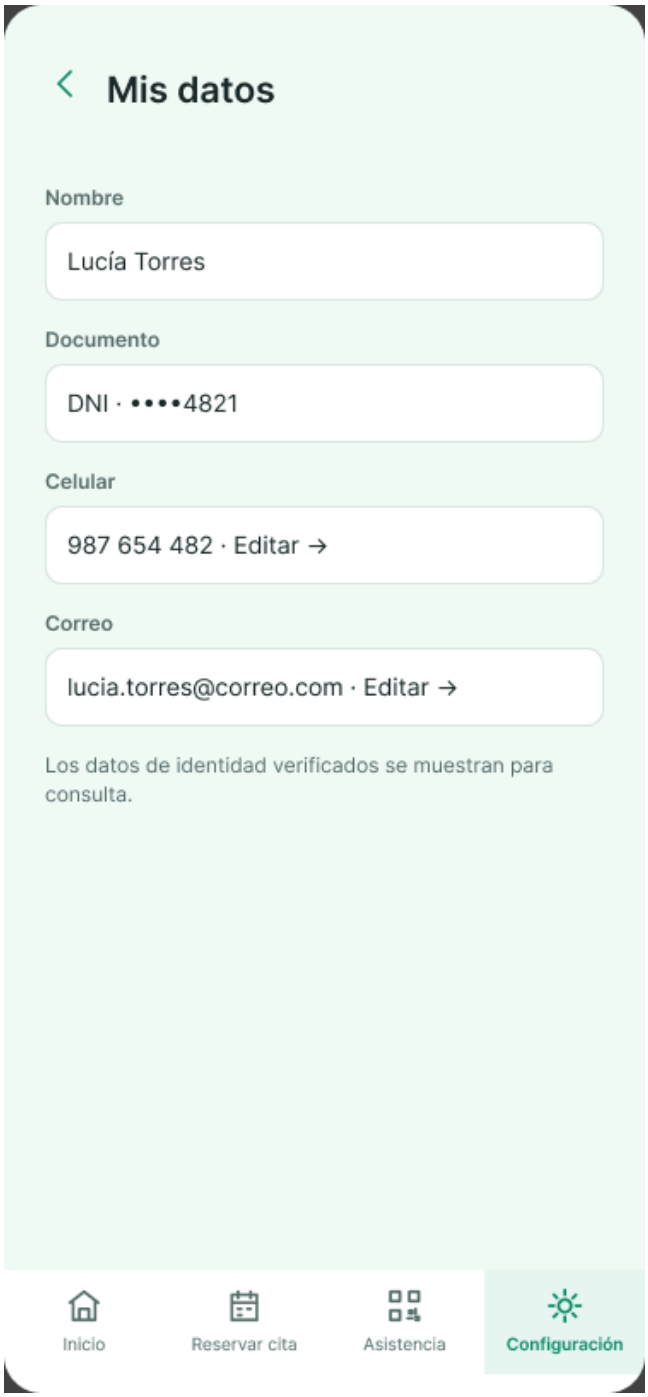
Nota. Interfaz del paciente exportada de Figma; referencia visual, no captura de ejecución del APK.

Perfil del paciente

El perfil reúne los datos personales y de contacto, junto con las acciones disponibles para la cuenta, como se observa en la Figura 173.

Figura 173

Perfil del paciente — SaludYa



Nota. Interfaz del paciente exportada de Figma; referencia visual, no captura de ejecución del APK.

Menores vinculados

La vista permite consultar los menores vinculados al paciente y acceder a las acciones de gestión del vínculo, como se observa en la Figura 174.

Figura 174

Menores vinculados — SaludYa

Nota. Interfaz del paciente exportada de Figma; referencia visual, no captura de ejecución del APK.

Notificaciones del paciente

La pantalla reúne los avisos dirigidos al paciente para consultar los cambios y novedades relacionados con sus citas, como se observa en la Figura 175.

Figura 175

Notificaciones del paciente — SaludYa

Nota. Interfaz del paciente exportada de Figma; referencia visual, no captura de ejecución del APK.

Backend: documentación publicada en Swagger

El backend dispone de una interfaz Swagger UI que presenta los endpoints y sus contratos HTTP. Las capturas muestran la documentación accesible del servicio desplegado; no constituyen resultados de pruebas ni acreditan por sí solas la ejecución exitosa de las operaciones de negocio.

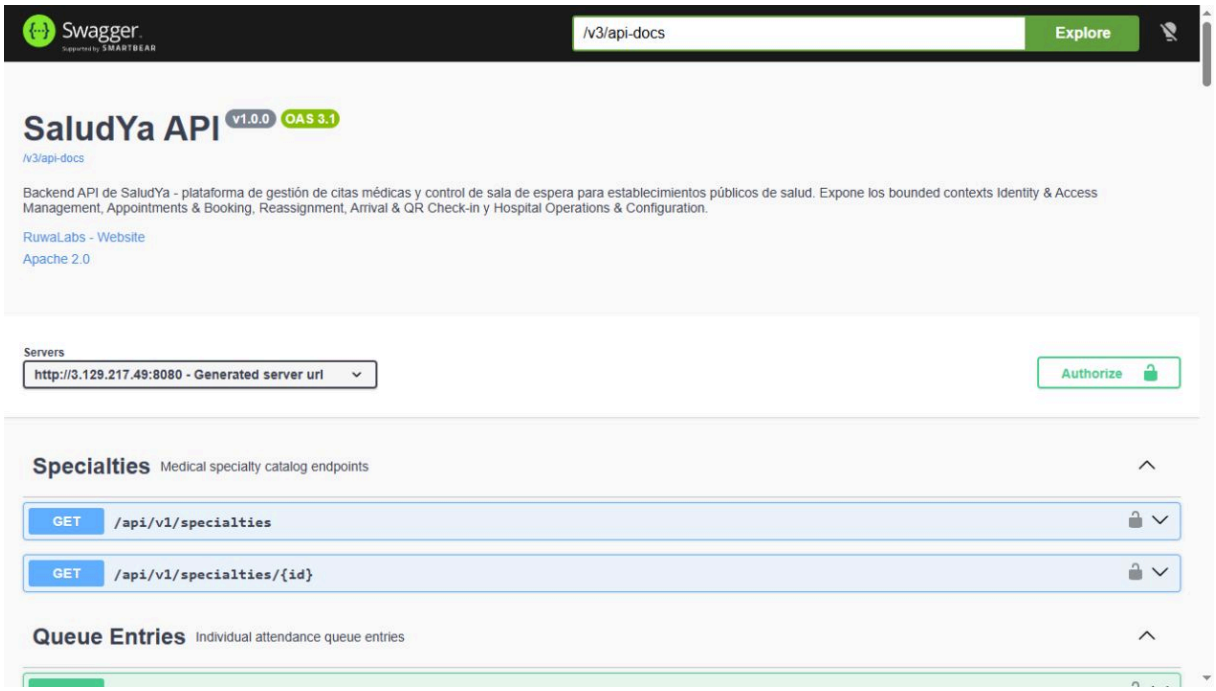
Enlace de despliegue: Swagger UI de SaludYa.

Vista general de Swagger

La interfaz identifica el servicio como SaludYa API y presenta el servidor y los grupos de endpoints publicados, como se observa en la Figura 176.

Figura 176

Vista general de Swagger — SaludYa



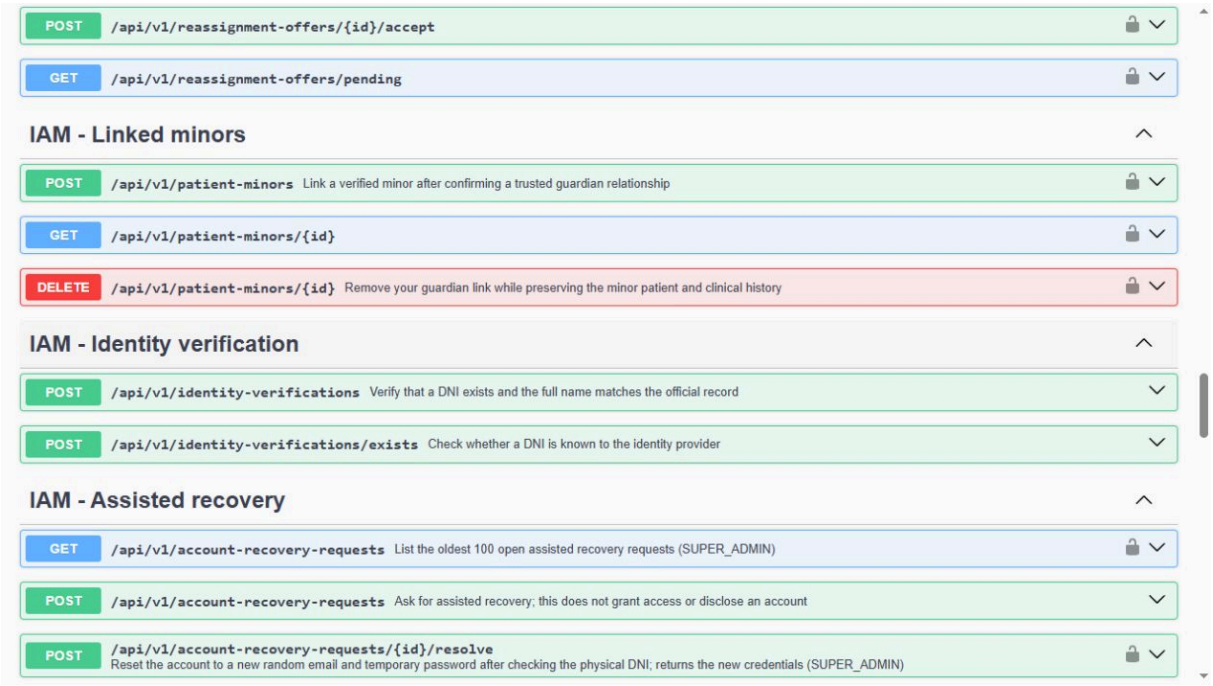
Nota. Captura de Swagger UI del backend desplegado de SaludYa, tomada el 9 de octubre de 2026.

Verificación de identidad en el backend

El grupo IAM - Identity verification documenta las operaciones para verificar la identidad y consultar la existencia de un DNI, como se observa en la Figura 177.

Figura 177

Verificación de identidad en el backend — SaludYa



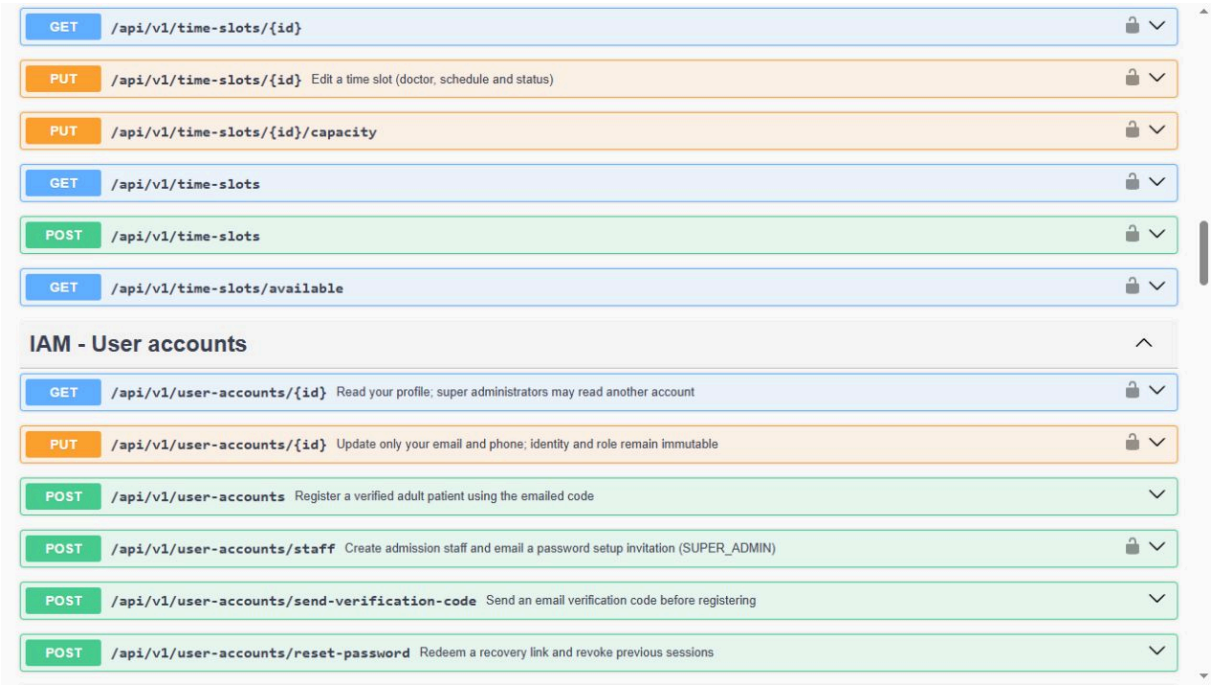
Nota. Captura de Swagger UI del backend desplegado de SaludYa, tomada el 9 de octubre de 2026.

Cuentas y credenciales

El grupo IAM - User accounts presenta los contratos de cuentas, registro y recuperación de acceso, como se observa en la Figura 178.

Figura 178

Cuentas y credenciales — SaludYa



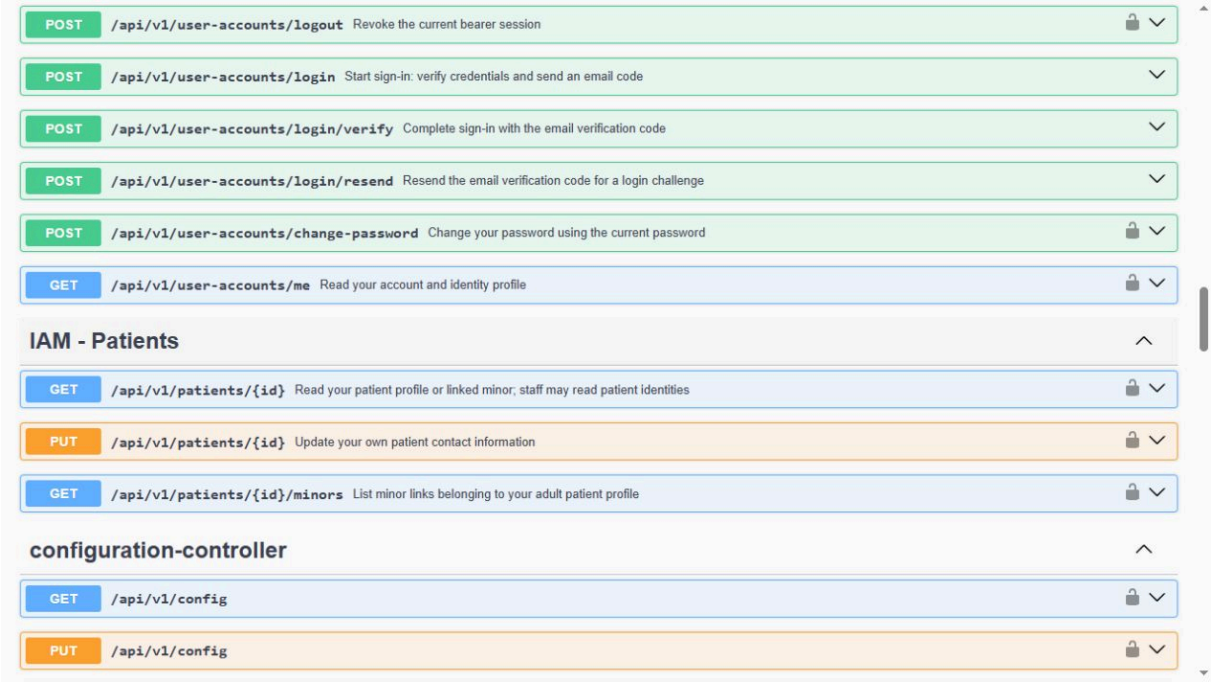
Nota. Captura de Swagger UI del backend desplegado de SaludYa, tomada el 9 de octubre de 2026.

Perfil del paciente

El grupo IAM - Patients documenta la consulta y actualización del perfil y la consulta de menores vinculados, como se observa en la Figura 179.

Figura 179

Perfil del paciente — SaludYa



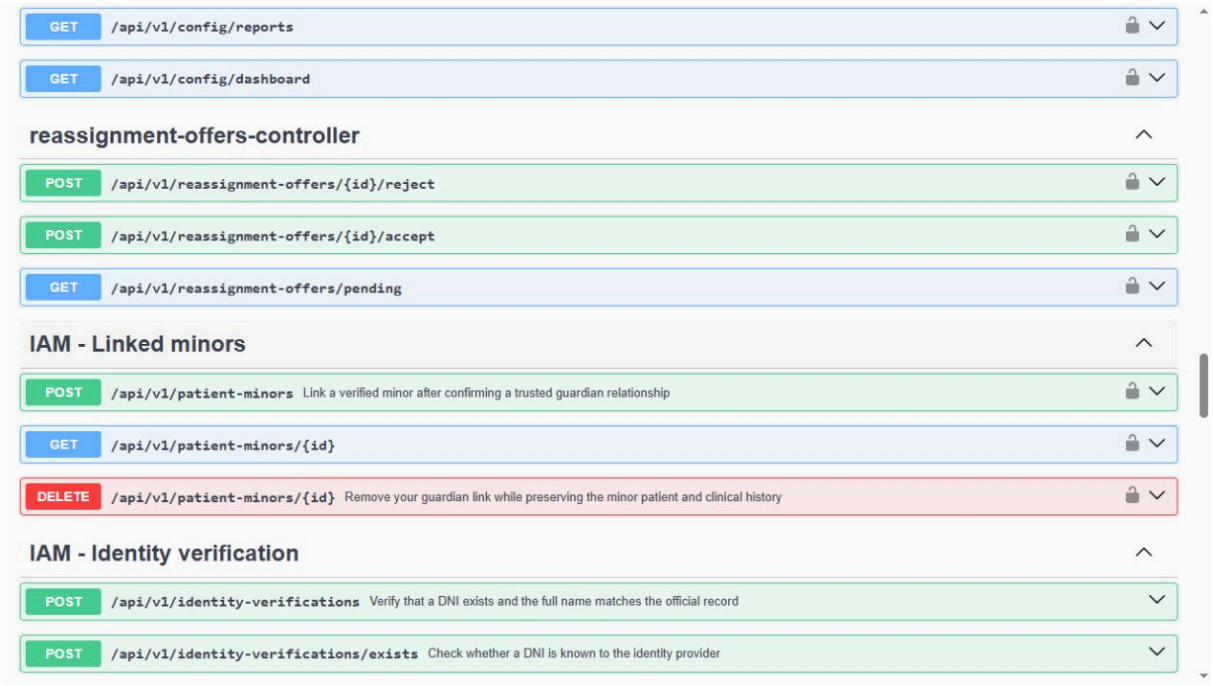
Nota. Captura de Swagger UI del backend desplegado de SaludYa, tomada el 9 de octubre de 2026.

Vinculación de menores

El grupo IAM - Linked minors presenta los contratos para crear, consultar y eliminar el vínculo con un menor, como se observa en la Figura 180.

Figura 180

Vinculación de menores — SaludYa



Nota. Captura de Swagger UI del backend desplegado de SaludYa, tomada el 9 de octubre de 2026.

Reservas de citas

El grupo Appointments presenta las operaciones para registrar y consultar reservas, cancelarlas y consultar las citas de un paciente, como se observa en la Figura 181.

Figura 181

Reservas de citas — SaludYa

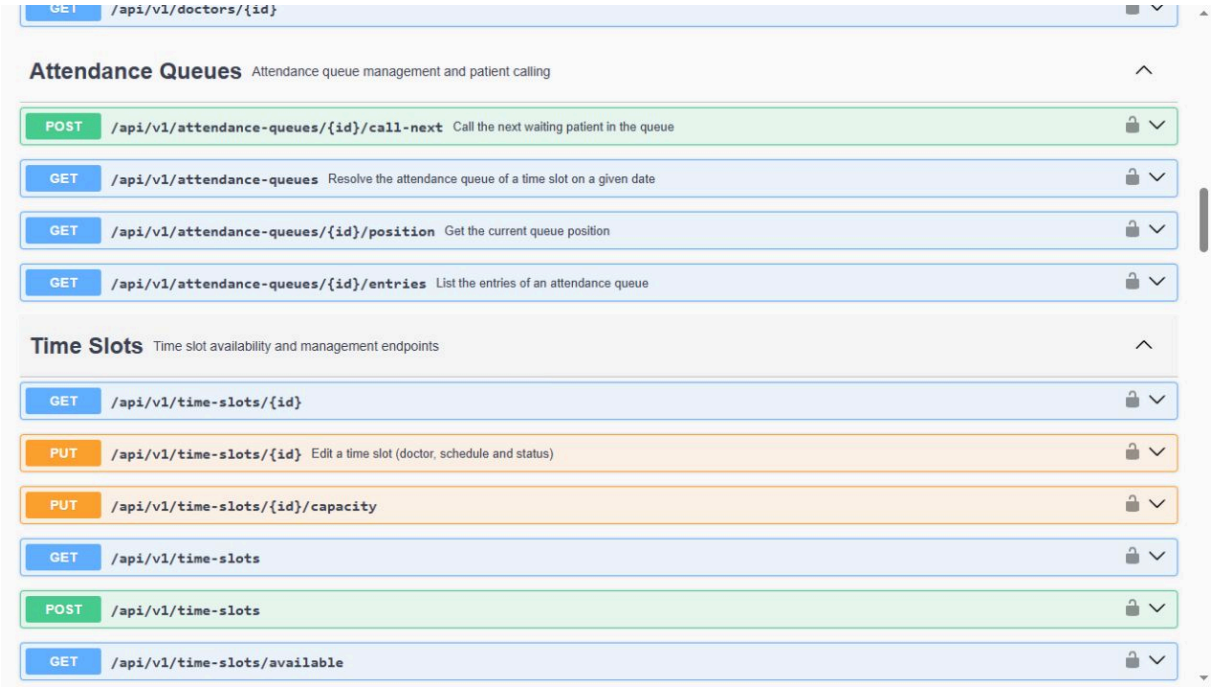
Nota. Captura de Swagger UI del backend desplegado de SaludYa, tomada el 9 de octubre de 2026.

Disponibilidad de horarios

El grupo Time Slots documenta la consulta de intervalos y disponibilidad para la reserva de citas, como se observa en la Figura 182.

Figura 182

Disponibilidad de horarios — SaludYa



Nota. Captura de Swagger UI del backend desplegado de SaludYa, tomada el 9 de octubre de 2026.

Registro de asistencia

El grupo Check-ins presenta los contratos de registro de llegada por QR o código y las consultas relacionadas con el estado de asistencia, como se observa en la Figura 183.

Figura 183

Registro de asistencia — SaludYa

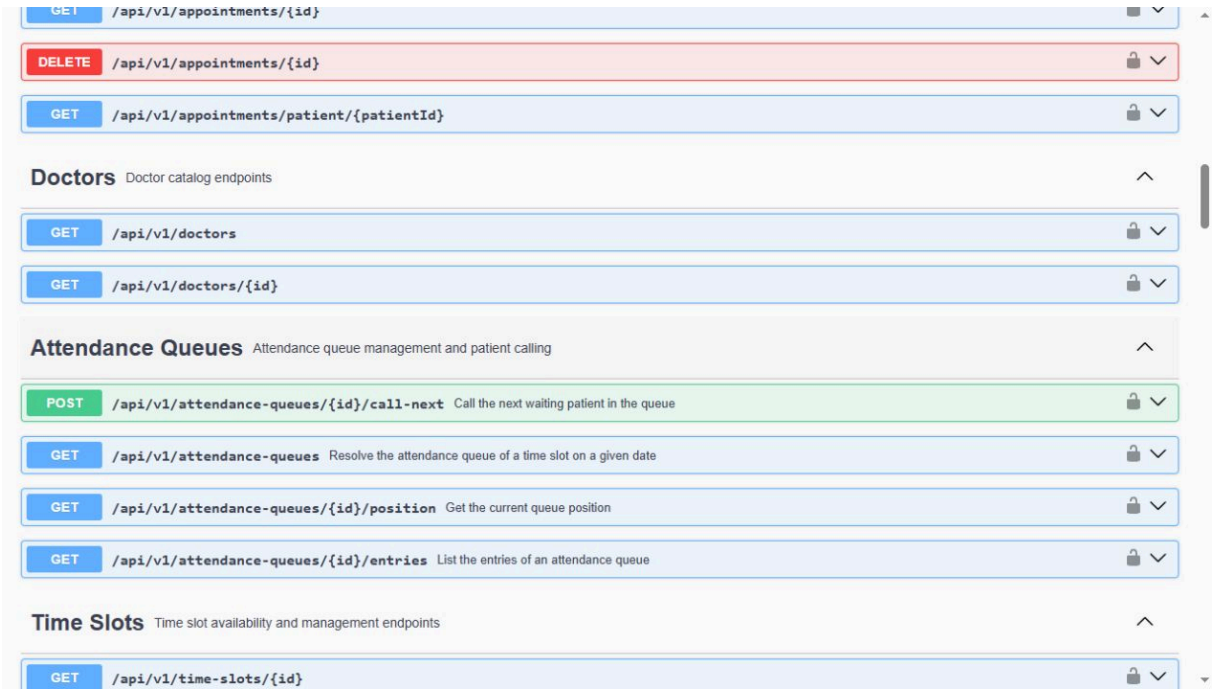
Nota. Captura de Swagger UI del backend desplegado de SaludYa, tomada el 9 de octubre de 2026.

Cola de atención

El grupo Attendance Queues documenta las consultas de cola, posición y pacientes en espera, junto con la operación de llamado, como se observa en la Figura 184.

Figura 184

Cola de atención — SaludYa



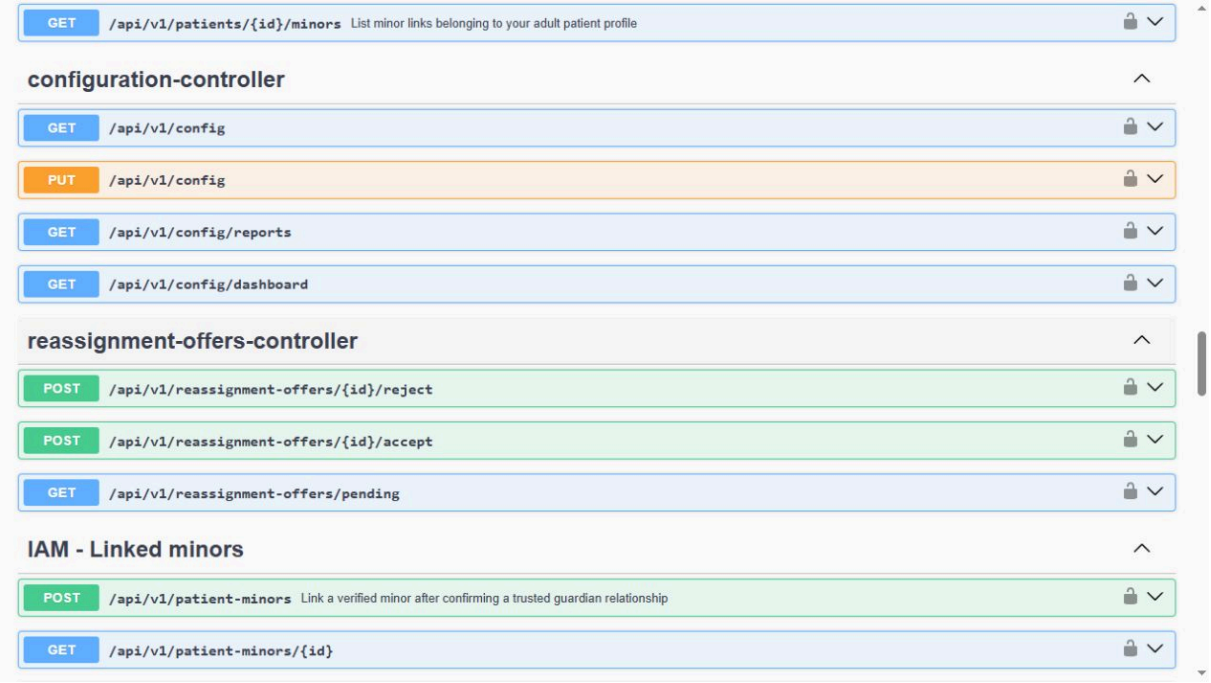
Nota. Captura de Swagger UI del backend desplegado de SaludYa, tomada el 9 de octubre de 2026.

Ofertas de reasignación

El grupo reassignment-offers-controller documenta la consulta de ofertas pendientes y las acciones para aceptar o rechazar una oferta, como se observa en la Figura 185.

Figura 185

Ofertas de reasignación — SaludYa



Nota. Captura de Swagger UI del backend desplegado de SaludYa, tomada el 9 de octubre de 2026.

4.2.1.7. Services Documentation Evidence for Sprint Review

En esta sección se presenta la evidencia de la documentación de los Web Services del backend de SaludYa correspondiente a este Sprint. La documentación se generó con springdoc-openapi (OpenAPI 3) a partir de las anotaciones del código y se encuentra desplegada y navegable en Swagger UI, lo que permite consultar cada operación y ejecutarla con datos de muestra mediante la opción *Try it out*. Se documentaron los servicios de los cinco *bounded contexts* del sistema Identity & Access Management, Appointments & Booking, Arrival & QR Check-in, Reassignment y Hospital Operations & Configuration, alcanzando 62 operaciones distribuidas en 53 rutas.

La especificación OpenAPI se publica en `/v3/api-docs` y se exportó al repositorio de Web Services (`docs/api/openapi.json`) para su versionado. La API utiliza autenticación HTTP Bearer con JWT (esquema `bearerAuth`); los endpoints públicos (registro, inicio de sesión, verificación de identidad y recuperación de cuenta) están marcados con `@SecurityRequirements` y no requieren token, mientras que el resto exige un token vigente obtenido tras el inicio de sesión.

- **Swagger UI (documentación desplegada):** http://3.129.217.49:8080/swagger-ui/index.html
- **Especificación OpenAPI (JSON):** http://3.129.217.49:8080/v3/api-docs
- **Repositorio de Web Services:** https://github.com/RuwaLabs/backend-saludya

Commits relacionados con la documentación para este Sprint

Commit	Mensaje	Relación
a592333	docs(api): add OpenAPI spec export and Web Services endpoint documentation	Exportación del documento OpenAPI (`docs/api/openapi.json`) y documentación de endpoints del Sprint

Commit	Mensaje	Relación
fdf27c4	chore: drop hardcoded OpenAPI server so Swagger targets the current host	Ajuste para que Swagger UI resuelva las peticiones contra el host desplegado
a870d04	docs(iam): add setup guide and API request examples	Guía de configuración y ejemplos de peticiones de IAM
acb79f5	fix(openapi): remove legacy ACME configuration	Corrección de la configuración OpenAPI
955abb6	feat: initializing project	Configuración base de la documentación OpenAPI del proyecto

Tabla de endpoints documentados

A continuación se detalla, para cada endpoint, la acción implementada, el verbo HTTP y la sintaxis de llamada, los parámetros admitidos, un ejemplo de petición y de respuesta con datos de muestra, la explicación de la respuesta y el enlace a su documentación desplegada. La URL base es `http://3.129.217.49:8080`.

IAM — Identity & Access Management

#	Acción implementada	Método	Sintaxis de llamada (endpoint)	Parámetros	Petición (ejemplo)	Respuesta (ejemplo)	Explicación del response	Documentación
1	Enviar código de verificación antes de registrar la cuenta	POST	`/api/v1/user-accounts/send-verification-code`	body: `email`	`{"email":"kevi` *cuerpo* `man@gmail.com"}`	`202` `{"message":"Si la` *cuerpo* `dirección es válida, recibirás un código de verificación."}`	Acepta la solicitud y envía un código de 6 dígitos al correo; no revela si el correo ya existe	Swagger
2	Registrar al paciente verificado	POST	`/api/v1/user-accounts`	body: `dni, name, lastname, birthDate, phone, email, password, code`	`{"dni":"74218365","name":"Kevin","lastname":"Guzmán","birthDate":"2003-0`	`201` `{"id":1,"userId":10,"dni":"74218365","name":"Kevin","lastnam`	Crea la cuenta y devuelve el recurso del paciente; el header `Location` apunta al recurso creado	Swagger
3	Iniciar sesión (valida credenciales	POST	`/api/v1/user-accounts/login`	body: `email, password`	`{"email":"kevi` *cuerpo* `man@gmail.com","password":"SaludYa#2026"}`	`200` `{"challengeId":"84c1d40-...","maskedEmail":"k***@gmail.com`	Valida las credenciales y devuelve el de-	Swagger

#	Acción implementada	Método	Sintaxis de llamada (endpoint)	Parámetros	Petición (ejemplo)	Respuesta (ejemplo)	Explicación del response	Documentación
	y envía código)						safío con el correo enmascarado; un mensaje único cubre credenciales inválidas	
4	Completar inicio de sesión con el código	POST	`/api/v1/user-accounts/login/verify`	body: `challengeId`, `code`	`{"challengeId" 200 c1d40-...","code":"172305"}`	`{"accessToken":"eyJhbGciOiJIUzI1NiJ9...","tokenType":"Beare`	Devuelve el token de acceso (JWT) y los identificadores del paciente y su rol	Swagger
5	Reenviar el código del desafío de login	POST	`/api/v1/user-accounts/login/resend`	body: `challengeId`	`{"challengeId" 202 c1d40-...}`	*sin cuerpo*	Genera y reenvía un nuevo código para el mismo desafío	Swagger
6	Cerrar sesión (revoca el token actual)	POST	`/api/v1/user-accounts/logout`	header: `Authorization: Bearer <token>`	*(sin cuerpo)*	`204` · *sin cuerpo*	Revoca la sesión Bearer vigente; responde sin contenido	Swagger
7	Solicitar enlace de recuperación de contraseña	POST	`/api/v1/user-accounts/recover-password`	body: `email`	`{"email":"kevi`	`202` `{"message":"If an active account exists, a recovery email will be sent."}`	Confirma la recepción de forma genérica, sin revelar si el correo está registrado	Swagger

#	Acción implementada	Método	Sintaxis de llamada (endpoint)	Parámetros	Petición (ejemplo)	Respuesta (ejemplo)	Explicación del response	Documentación
8	Restablecer contraseña con el enlace de recuperación	POST	`/api/v1/user-accounts/reset-password`	body: `token`, `password`, `confirmPassword`	`{"token":"d41 …8f0…","password":"NuevaA#2026!","confirmPassword":"N…`	`204` *sin cuerpo*	Canjea el enlace de un solo uso y revoca las sesiones previas; responde sin contenido	Swagger
9	Cambiar la contraseña usando la actual	POST	`/api/v1/user-accounts/change-password`	header: `Authorization` · body: `currentPassword`, `password`, `confirmPassword`	`{"currentPass…":"SaludYa2026!","password":"NuevaA#2026!","confirmPass…`	`204` *sin cuerpo*	Actualiza la contraseña del usuario autenticado; responde sin contenido	Swagger
10	Leer la cuenta y el perfil propios	GET	`/api/v1/user-accounts/me`	header: `Authorization`	*(sin cuerpo)*	`200` `{"id":10,"role":"PATIENT","email":"kevin.huaman@gmail.com…`	Devuelve el perfil de la cuenta autenticada	Swagger
11	Leer un perfil por id	GET	`/api/v1/user-accounts/{id}`	path: `id` · header: `Authorization`	`/api/v1/user-accounts/10`	`200` `{"id":10,"role":"PATIENT","email":"kevin.huaman@gmail.com…`	Devuelve el perfil propio; un SUPER_ADMIN puede consultar otra cuenta	Swagger
12	Actualizar correo y celular	PUT	`/api/v1/user-accounts/{id}`	path: `id` · body: `email`, `phone`	`{"email":"kevi…vo@gmail.com","phone":"987111222"}`	`200` `{"id":10,"email":"kevin.nuevo@gmail.com","phone":"9871112…`	Actualiza solo correo y celular; la identidad y el rol permanecen inmutables	Swagger

#	Acción implementada	Método	Sintaxis de llamada (endpoint)	Parámetros	Petición (ejemplo)	Respuesta (ejemplo)	Explicación del response	Documentación
13	Crear cuenta de personal de admisión	POST	/api/v1/user-accounts/staff	header: Authorization (SUPER_ADMIN) · body: dni, name, lastname, birthDate, phone, email	{"dni":"70000(	201 {"id":15,"role":"ADMISION_STAFF","email":"franco@saludya.	name":"Franco","lastname":"Vilca","birthDate":"1999-	Swagger
							Crea la cuenta del personal y envía una invitación para definir contraseña	
14	Verificar identidad por DNI y nombre	POST	/api/v1/identity-verific	body: dni, name, lastname	{"dni":"74218	200 {"verified":true}	name":"Kevin","lastname":"Aguirre"}	Swagger
							Indica si el DNI existe y el nombre completo coincide con el registro oficial	
15	Comprobar si un DNI es conocido	POST	/api/v1/identity-verifications/exists	body: dni	{"dni":"74218	200 {"exists":true}		Swagger
							Indica si el DNI es conocido por el proveedor de identidad	
16	Leer el perfil de paciente	GET	/api/v1/patients/{id}	path: id · header: Authorization	/api/v1/patients/1	200 {"id":1,"userId":10,"dni":"74218365","name":"Kevin","lastnam		Swagger
							Devuelve el paciente propio o un menor vinculado	
17	Actualizar datos de contacto del paciente	PUT	/api/v1/patients/{id}	path: id · body: email, phone	{"email":"kevi	200 {"id":1,"userId":10,"dni":"74218365","name":"Kevin",...}	vo@gmail.com","phone":"999999222"}	Swagger
							Actualiza el contacto del paciente propio y devuelve el recurso actualizado	

#	Acción implementada	Método	Sintaxis de llamada (endpoint)	Parámetros	Petición (ejemplo)	Respuesta (ejemplo)	Explicación del response	Documentación
18	Listar menores vinculados	GET	`/api/v1/patients/{id}/minors`	path: `id` · header: `Authorization`	`/api/v1/patients/5/minors`	`200` `[{"id":5,"patientId":88,"tutorId":1}]`	Lista los vínculos de tutoría del paciente autenticado	Swagger
19	Vincular a un menor verificado	POST	`/api/v1/patient-minors`	header: `Authorization` · body: `dni, name, lastname, birthDate, confirmFiliation`	`{"dni":"76543210","name":"Ana","lastname":"Torres","birthDate":"2015-08-20","confirmFiliation":true}`	`201` `{"id":5,"patientId":88,"tutorId":1}`	Crea el vínculo de tutoría tras confirmar la filiación; `Location` apunta al recurso	Swagger
20	Leer un vínculo de tutoría	GET	`/api/v1/patient-minors/{id}`	path: `id` · header: `Authorization`	`/api/v1/patient-minors/5`	`200` `{"id":5,"patientId":88,"tutorId":1}`	Devuelve el vínculo de tutoría solicitado	Swagger
21	Desvincular a un menor	DELETE	`/api/v1/patient-minors/{id}`	path: `id` · header: `Authorization`	`/api/v1/patient-minors/5`	`204` · *sin cuerpo*	Elimina el vínculo de tutoría conservando la historia clínica del menor	Swagger
22	Obtener instrucciones de soporte	GET	`/api/v1/account-recovery-requests/support`	—	*(sin cuerpo)*	`200` `{"instructions":"Acude al área de admisión con tu DNI original...","phone":"999888777"}`	Devuelve las instrucciones y el teléfono de soporte (público)	Swagger
23	Solicitar recuperación asistida	POST	`/api/v1/account-recovery-requests`	body: `dni, contactEmail`	`{"dni":"74218365","contactEmail":"ramiro.tamara@gmail.com"}`	`202` `{"message":"Request received...","instructions":"...","phone":"999888777"}`	Registra la solicitud; no otorga acceso	Swagger

#	Acción implementada	Método	Sintaxis de llamada (endpoint)	Parámetros	Petición (ejemplo)	Respuesta (ejemplo)	Explicación del response	Documentación
							ni revela cuentas	
24	Listar solicitudes de recuperación abiertas	GET	`/api/v1/account-recovery-requests`	header: `Authorization` (SUPER_ADMIN)	*(sin cuerpo)*	`200` [{"id":"3f7b...","dni":"74218365","contactEmail":"familiar@gm	Lista las 100 solicitudes abiertas más antiguas	Swagger
25	Resolver solicitud de recuperación asistida	POST	`/api/v1/account-recovery-requests/{id}/resolve`	path: `id` · header: `Authorization` (SUPER_ADMIN) · body: `identityCheckedInPerson`	{"identityChec	`200` nPerson":true} {"email":"newuser@saludya.local","password":"Temp#a1B2c	Restablece la cuenta con correo y clave temporal tras verificar el DNI físico (SUPER_ADMIN)	Swagger

Appointments & Booking

#	Acción implementada	Método	Sintaxis de llamada (endpoint)	Parámetros	Petición (ejemplo)	Respuesta (ejemplo)	Explicación del response	Documentación
26	Reservar una cita	POST	/api/v1/appointments	body: `patientId`, `timeSlotId`	{"patientId":1,	`201` SlotId":31, {"id":142,"timeSlotId":31,"patientId":1,"bookingOrder":3,"boo	Crea la cita y devuelve el recurso; `Location` apunta a la cita creada	Swagger
27	Listar citas con filtros	GET	/api/v1/appointments	query: `patientId`, `timeSlotId`, `doctorId`, `specialtyId`, `date`, `status`	/api/v1/appointments? patientId=1&status=RESERVED	`200` [{"id":142,"timeSlotId":31,"patientId":1,"status":"RESERVED",.	Devuelve las citas filtradas; si el actor es paciente, `patientId` es obligatorio	Swagger

#	Acción implementada	Método	Sintaxis de llamada (endpoint)	Parámetros	Petición (ejemplo)	Respuesta (ejemplo)	Explicación del response	Documentación
28	Obtener una cita por id	GET	/api/v1/appointments/{id}	path: id	/api/v1/appointments/142	200 {"id":142,"timeslotId":31,"patientId":1,"bookingOrder":3,"stat	Devuelve la cita; error si no existe o no es gestionable por el usuario	Swagger
29	Listar citas de un paciente	GET	/api/v1/appointments/patient/{patientId}	path: patientId · query: status	/api/v1/appointments/patient/1?status=CONFIRMED	200 [{"id":142,"patientId":1,"status":"CONFIRMED",...}]	Lista las citas del paciente indicado	Swagger
30	Cancelar una cita	DELETE	/api/v1/appointments/{id}	path: id	/api/v1/appointments/142	200 {"id":142,"status":"CANCELLED",...}	Cancela la cita y devuelve el recurso con estado CANCELLED	Swagger
31	Listar doctores por especialidad	GET	/api/v1/doctors	query: specialtyId (obligatorio)	/api/v1/doctors?specialtyId=1	200 [{"id":7,"specialtyId":1,"name":"Ana","lastname":"Rojas"}]	Devuelve los doctores de la especialidad indicada	Swagger
32	Obtener un doctor por id	GET	/api/v1/doctors/{id}	path: id	/api/v1/doctors/7	200 {"id":7,"specialtyId":1,"name":"Ana","lastname":"Rojas"}	Devuelve el doctor solicitado	Swagger
33	Listar especialidades	GET	/api/v1/specialties		/api/v1/specialties	200 [{"id":1,"name":"Medicina general","description":"Atención médica primaria"}]	Devuelve el catálogo de especialidades	Swagger
34	Obtener una especialidad por id	GET	/api/v1/specialties/{id}	path: id	/api/v1/specialties/1	200 {"id":1,"name":"Medicina general","description":"Atención médica primaria"}	Devuelve la especialidad solicitada	Swagger

#	Acción implementada	Método	Sintaxis de llamada (endpoint)	Parámetros	Petición (ejemplo)	Respuesta (ejemplo)	Explicación del response	Documentación
35	Slots disponibles por especialidad y fecha	GET	`/api/v1/time-slots/available`	query: `specialtyId, date` (obligatorios)	`/api/v1/time-slots/available?specialtyId=1&date=2026-10-15`	200 [{"id":31,"doctorId":7,"date":"2026-10-15","startHour":"09:00","3","maxCapacity":5,"currentBookings":2,"status":"AVAILABLE"	Devuelve los slots con cupo disponible para la especialidad y fecha	Swagger
36	Slots de un doctor en una fecha	GET	`/api/v1/time-slots`	query: `doctorId, date` (obligatorios)	`/api/v1/time-slots?doctorId=7&date=2026-10-15`	200 [{"id":31,"doctorId":7,"date":"2026-10-15","status":"AVAILABLE	Devuelve los slots del doctor en la fecha indicada	Swagger
37	Obtener un slot por id	GET	`/api/v1/time-slots/{id}`	path: `id`	`/api/v1/time-slots/31`	200 {"id":31,"doctorId":7,"date":"2026-10-15","startHour":"09:00",	Devuelve el slot solicitado	Swagger
38	Crear un slot de atención	POST	`/api/v1/time-slots`	body: `doctorId, date, startHour, endHour, room, maxCapacity`	{"doctorId":7," 3","maxCapacity":5}	201 ":"2026-10-15","startHour":"09:00","endHour":"09:30","roo {"id":32,"doctorId":7,"date":"2026-10-16","startHour":"09:00",	Crea el slot y devuelve el recurso creado	Swagger
39	Actualizar la capacidad de un slot	PUT	`/api/v1/time-slots/{id}/capacity`	path: `id` · body: `maxCapacity`	{"maxCapacity	200 {"id":31,"maxCapacity":8,"currentBookings":2,"status":"AVAIL	Actualiza la capacidad máxima del slot	Swagger
40	Editar un slot (doctor, horario y estado)	PUT	`/api/v1/time-slots/{id}`	path: `id` · body: `doctorId, startHour, endHour, status`	{"doctorId":7," Hour":"10	200 ndHou {"id":31,"doctorId":7,"startHour":"10:00","endHour":"10:30","s	Edita los datos del slot	Swagger "status":"AVAILABLE"}

Arrival & QR Check-in

#	Acción implementada	Método	Sintaxis de llamada (endpoint)	Parámetros	Petición (ejemplo)	Respuesta (ejemplo)	Explicación del response	Documentación
41	Registrar check-in por token QR	POST	/api/v1/check-ins/qr	body: qrToken	{"qrToken":"ey	201 {"checkInId":15,"appointmentId":142,"queueEntryId":9,"posit	Registra la llegada, crea el ticket y la entrada en la cola; devuelve la posición	Swagger
42	Registrar check-in por código de reserva	POST	/api/v1/check-ins/code	body: bookingCode	{"bookingCod	201 SV-000142"} {"checkInId":15,"appointmentId":142,"queueEntryId":9,"posit	Alternativa manual: registra la llegada con el código de reserva	Swagger
43	Obtener el ticket digital de un check-in	GET	/api/v1/check-ins/{id}	path: id	/api/v1/check-ins/15	200 {"id":15,"appointmentId":142,"bookingCode":"RSV-000142","t general","status":"VALID"}	Devuelve el ticket digital del check-in	Swagger
44	Obtener el ticket digital de una cita	GET	/api/v1/check-ins/app {appointmentId}	path: appointmentId	/api/v1/check-ins/appointment/142	200 {"id":15,"appointmentId":142,"turnCode":"A-003","status":"VA	Devuelve el ticket digital asociado a la cita	Swagger
45	Generar el token QR firmado de una cita	GET	/api/v1/check-ins/app {appointmentId}/qr-token	path: appointmentId	/api/v1/check-ins/appointment/142/qr-token	200 {"qrToken":"eyJhbGciOiJIUzI1NiJ9.qr.14f..."}	Genera el token QR firmado y temporal de la cita	Swagger
46	Posición del paciente en la cola por cita	GET	/api/v1/check-ins/app {appointmentId}/position	path: appointmentId	/api/v1/check-ins/appointment/142/position	200 {"position":3,"totalInQueue":8}	Devuelve la posición actual del paciente y el total en cola	Swagger

#	Acción implementada	Método	Sintaxis de llamada (endpoint)	Parámetros	Petición (ejemplo)	Respuesta (ejemplo)	Explicación del response	Documentación
47	Obtener una entrada de cola	GET	`/api/v1/queue-entries/{id}`	path: `id`	`/api/v1/queue-entries/9`	`200` {"id":9,"attendanceQueueId":4,"checkInId":15,"position":3,"sta	Devuelve la entrada de cola (el paciente puede leer la suya)	Swagger
48	Marcar una entrada como ausente	POST	`/api/v1/queue-entries/{id}/absent`	path: `id`	`/api/v1/queue-entries/9/absent`	`204` · *sin cuerpo*	Marca al paciente como ausente y cierra su entrada	Swagger
49	Salir voluntariamente de la cola	POST	`/api/v1/queue-entries/{id}/leave`	path: `id`	`/api/v1/queue-entries/9/leave`	`204` · *sin cuerpo*	El paciente abandona la cola; la cita se marca como ausente	Swagger
50	Iniciar la atención de una entrada llamada	POST	`/api/v1/queue-entries/{id}/start`	path: `id`	`/api/v1/queue-entries/9/start`	`200` {"id":9,"status":"IN_ATTENTION",...} `IN_ATTENTION`	Cambia la entrada	Swagger
51	Finalizar la atención de una entrada	POST	`/api/v1/queue-entries/{id}/finish`	path: `id`	`/api/v1/queue-entries/9/finish`	`200` {"id":9,"status":"ATTENDED","attendedAt":"2026-10-15T09:25:	Marca la entrada como atendida	Swagger
52	Resolver la cola de un slot en una fecha	GET	`/api/v1/attendance-queues`	query: `timeSlotId`, `date` (obligatorios)	`/api/v1/attendance-queues?timeSlotId=21&date=2026-10-15`	`200` - {"id":4,"timeSlotId":21,"date":"2026-10-15","status":"OPEN"}	Devuelve la de atención del slot y fecha; error si no existe	Swagger

#	Acción implementada	Método	Sintaxis de llamada (endpoint)	Parámetros	Petición (ejemplo)	Respuesta (ejemplo)	Explicación del response	Documentación
53	Listar las entradas de una cola	GET	`/api/v1/attendance/queues/{id}/entries`	path: `id`	`/api/v1/attendance/queues/4/entries`	200 - `[{"id":9,"position":3,"status":"WAITING",...}]`	Lista las entradas (pacientes) de la cola	Swagger
54	Posición actual en la cola	GET	`/api/v1/attendance/queues/{id}/position`	path: `id`	`/api/v1/attendance/queues/4/position`	200 - `{"position":3,"totalInQueue":8}`	Devuelve la posición y el total de la cola	Swagger
55	Llamar al siguiente paciente	POST	`/api/v1/attendance/queues/{id}/call-next`	path: `id`	`/api/v1/attendance/queues/4/call-next`	200 - `{"id":9,"position":3,"status":"CALLED","calledAt":"2026-10-15T`	Llama al siguiente paciente en espera y devuelve su entrada	Swagger

Reassignment

#	Acción implementada	Método	Sintaxis de llamada (endpoint)	Parámetros	Petición (ejemplo)	Respuesta (ejemplo)	Explicación del response	Documentación
56	Listar ofertas de cupo pendientes	GET	`/api/v1/reassignment-offers/pending`	header: `Authorization` · query: `appointmentId` (opcional)	`/api/v1/reassignment-offers/pending`	200 - `[{"id":21,"appointmentId":142,"originalAppointmentId":130,"f`	Devuelve las ofertas pendientes del usuario y sus menores	Swagger
57	Aceptar una oferta de reasignación	POST	`/api/v1/reassignment-offers/{id}/accept`	path: `id`	`/api/v1/reassignment-offers/21/accept`	200 - `{"id":21,"status":"ACCEPTED","respondedAt":"2026-10-09T09:0`	Acepta la oferta y agenda la cita en el cupo liberado	Swagger
58	Rechazar una oferta de reasignación	POST	`/api/v1/reassignment-offers/{id}/reject`	path: `id`	`/api/v1/reassignment-offers/21/reject`	200 - `{"id":21,"status":"REJECTED","respondedAt":"2026-10-09T09:0`	Rechaza la oferta y libera el cupo al si-	Swagger

#	Acción implementada	Método	Sintaxis de llamada (endpoint)	Parámetros	Petición (ejemplo)	Respuesta (ejemplo)	Explicación del response	Documentación
							guiente candidato	

Hospital Operations & Configuration

#	Acción implementada	Método	Sintaxis de llamada (endpoint)	Parámetros	Petición (ejemplo)	Respuesta (ejemplo)	Explicación del response	Documentación
59	Leer la configuración del establecimiento	GET	`/api/v1/config`	—	`/api/v1/config`	`200` {"id":1,"maxCapacityPerSlot":5,"bookingOrderScope":"PER_S	Devuelve la configuración vigente del establecimiento	Swagger
60	Actualizar la configuración del establecimiento	PUT	`/api/v1/config`	body: `maxCapacityPerSlot`, `checkInToleranceMinutes`, `postCallToleranceMinutes`, `reassignmentResponseTimeoutMin`, `bookingCutoffTime`, `cancellationDeadlineHours`, `attendanceQueueVisible`	{"maxCapacity	`200` lot":6,"checkInToleranceMinutes":15,"postCallToleranceM {"id":1,"maxCapacityPerSlot":6,...,"updatedAt":"2026-10-09T0	Actualiza la configuración y devuelve el recurso actualizado	Swagger
61	Obtener el panel de métricas del día	GET	`/api/v1/config/dashbc`	query: `date` (opcional)	`/api/v1/config/dashbc` `date=2026-10-`	`200` {"configurationId":1,"metrics": [{"name":"appointmentsToday","value":24}, {"name":"inQueue","value":8}],"externalDataAvailable":true,...	Devuelve las métricas operativas del día	Swagger
62	Generar un reporte por rango de fechas	GET	`/api/v1/config/reports`	query: `from`, `to`, `format` (obligatorios; `format` por defecto `JSON`)	`/api/v1/config/reports?from=2026-10-01&to=2026-10-07&format=JSON`	`200` {"format":"JSON","generatedAt":"2026-10-09T12:00:00Z","cont	Genera el reporte del rango indicado en el formato solicitado	Swagger

Evidencias de interacción

A continuación se presenta una muestra del modelo de evidencia de la interacción con la documentación desplegada (Swagger UI). Para cada endpoint se incluye el JSON de envío, la captura y una descripción. Los endpoints elegidos son públicos.

1. `POST /api/v1/identity-verifications` - Verificar identidad por DNI

JSON de envío:

{ "dni": "74500834", "name": "Neo Daniel", "lastname": "Ramos Mera" }

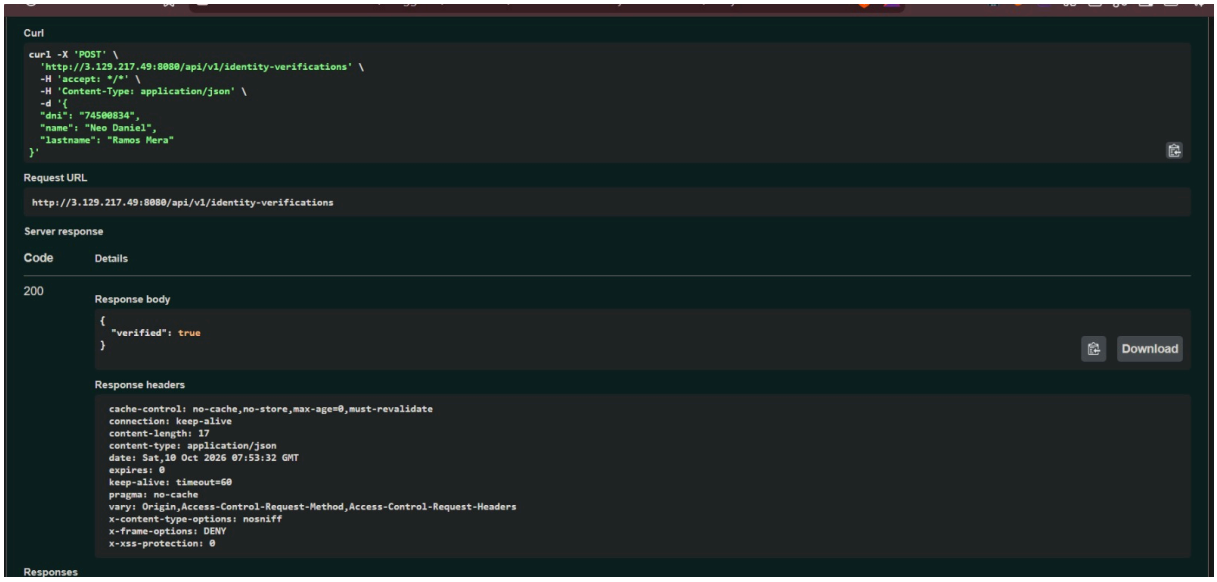


Figura 186. Ejecución de `POST /api/v1/identity-verifications` en Swagger UI con datos de muestra; respuesta `200` con `{"verified": true}`.

2. `POST /api/v1/identity-verifications/exists` - Comprobar si un DNI es conocido

JSON de envío:

{ "dni": "74500834" }

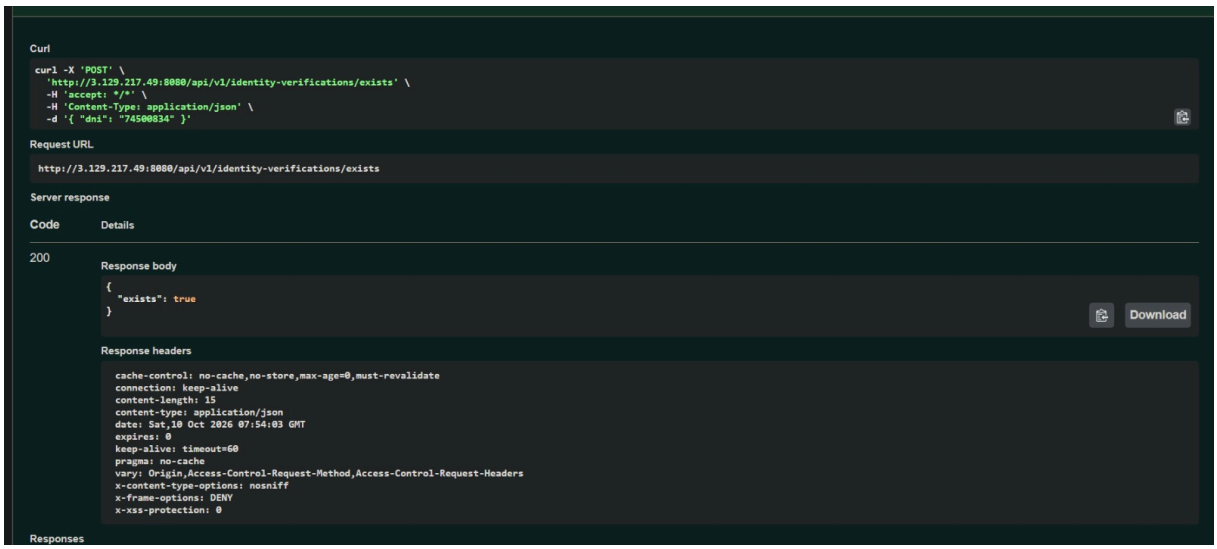


Figura 187. Ejecución de `POST /api/v1/identity-verifications/exists` respuesta `200` con `{"exists": true}`.

3. `POST /api/v1/user-accounts/send-verification-code` - Enviar código de verificación

JSON de envío:

{ "email": "doriangray292929@gmail.com" }

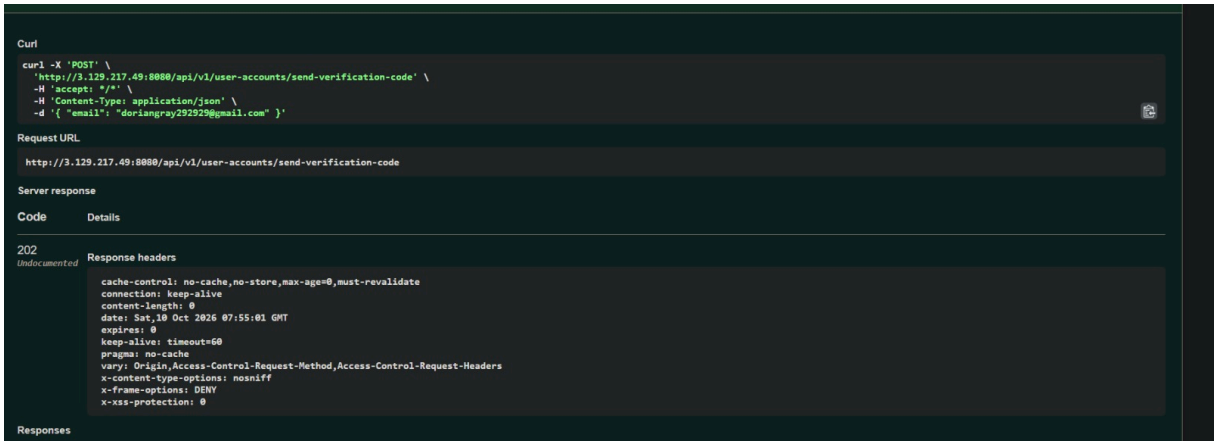


Figura 188. Ejecución de `POST /api/v1/user-accounts/send-verification-code` respuesta `202` (sin cuerpo).

4. `POST /api/v1/user-accounts/recover-password` - Solicitar recuperación de contraseña

JSON de envío:

```
{ "email": "doriangray292929@gmail.com" }
```

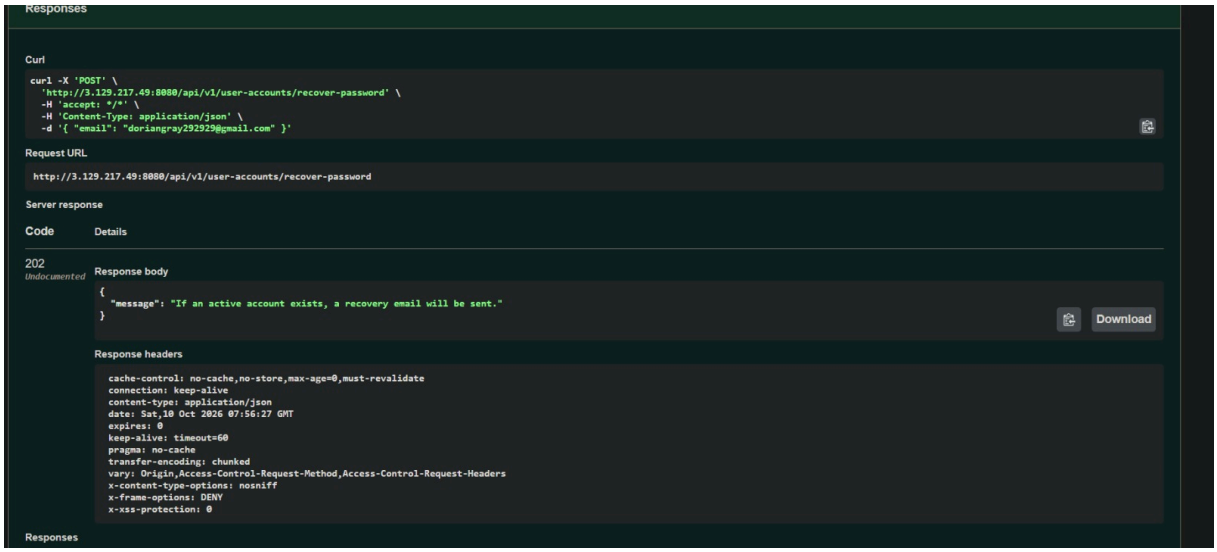


Figura 189. Ejecución de `POST /api/v1/user-accounts/recover-password` respuesta `202` con un mensaje genérico.

4.2.1.8. Software Deployment Evidence for Sprint Review

En esta sección se presenta la evidencia de las actividades de despliegue (Deployment) realizadas durante este Sprint. El despliegue se enfocó en los Web Services de SaludYa: el backend (Spring Boot) se desplegó en una instancia AWS EC2 con Ubuntu, Java 25 (Temurin) y PostgreSQL, publicándose el servicio mediante un systemd unit y automatizando el proceso con scripts de Bash. Los productos digitales del alcance Landing Page, Web Services y aplicaciones móviles cuentan con su respectiva estrategia de despliegue; en esta entrega, la evidencia corresponde a los Web Services.

Las actividades realizadas durante el Sprint fueron:

- **Creación de la cuenta y de los recursos en el proveedor cloud (AWS):** alta de la cuenta, creación del *key pair* para el acceso SSH, lanzamiento de la instancia EC2 y configuración de la red (security group con los puertos necesarios).

- **Configuración del servidor:** instalación de Java 25 (Temurin), PostgreSQL y Git, y creación del usuario y la base de datos del backend.
- **Externalización de secretos:** las variables sensibles (JWT, QR, clave de cifrado de notificaciones, SMTP, ApiPeru y bootstrap) se almacenan en `/etc/saludya/saludya.env`, fuera del repositorio, y systemd las carga mediante `EnvironmentFile`, de modo que sobreviven a cada despliegue.
- **Automatización del despliegue:** tres scripts de Bash (`setup-server.sh`, `setup-env.sh` y `deploy.sh`) y un servicio `systemd` (`saludya.service`) que recompila el JAR y reinicia el backend.
- **Verificación:** el backend queda disponible y se comprueba consultando el endpoint `/v3/api-docs`.

Scripts de despliegue

Para automatizar el aprovisionamiento y el despliegue del backend se utilizaron tres scripts de Bash.

`setup-server.sh`

Instala el entorno base del servidor (Java 25 con Temurin, PostgreSQL y Git) y crea el rol y la base de datos del backend.

```
#!/usr/bin/env bash
set -euo pipefail

DB_NAME="${DB_NAME:-saludyadb}"
DB_USER="${DB_USER:-saludya}"
DB_PASSWORD="${DB_PASSWORD:?Define DB_PASSWORD antes de ejecutar el script}"
if [ "$(id -u)" -ne 0 ]; then
 echo "ERROR: ejecuta con sudo ->  sudo bash setup-server.sh" >&2
 exit 1
fi

echo "==> [1/3] Instalando Java 25 (Temurin) ..."
apt-get update
apt-get install -y wget apt-transport-https gpg
mkdir -p /etc/apt/keyrings
wget -qO- https://packages.adoptium.net/artifactory/api/gpg/key/public \
 | gpg --dearmor > /etc/apt/keyrings/adoptium.gpg
. /etc/os-release
echo "deb [signed-by=/etc/apt/keyrings/adoptium.gpg] https://packages.adoptium.net/artifactory/deb
${VERSION_CODENAME} main" \
> /etc/apt/sources.list.d/adoptium.list
apt-get update
apt-get install -y temurin-25-jdk

echo "==> [2/3] Instalando PostgreSQL y git ..."
apt-get install -y postgresql git
systemctl enable --now postgresql

echo "==> [3/3] Creando usuario y base de datos ..."
# Crea el rol si no existe, o actualiza su password si ya existe.
sudo -u postgres psql -v ON_ERROR_STOP=1 <<SQL
DO \$\$
BEGIN
  IF NOT EXISTS (SELECT FROM pg_roles WHERE rolname = '${DB_USER}') THEN
```

```
    CREATE ROLE ${DB_USER} LOGIN PASSWORD '${DB_PASSWORD}';
  ELSE
    ALTER ROLE ${DB_USER} WITH LOGIN PASSWORD '${DB_PASSWORD}';
  END IF;
END
\$\$;
SQL

# Crea la base de datos si no existe, con el usuario como owner.
if ! sudo -u postgres psql -tAc "SELECT 1 FROM pg_database WHERE datname='${DB_NAME}'" | grep -q 1; then
  sudo -u postgres createdb -O "${DB_USER}" "${DB_NAME}"
fi

sudo -u postgres psql -v ON_ERROR_STOP=1 -c "GRANT ALL PRIVILEGES ON DATABASE ${DB_NAME} TO
${DB_USER};"
sudo -u postgres psql -v ON_ERROR_STOP=1 -d "${DB_NAME}" -c "GRANT ALL ON SCHEMA public TO ${DB_USER};"

echo
echo "================ RESUMEN ================"
java -version 2>&1 | head -1
sudo -u postgres psql -tAc "SELECT version();"
echo "Base de datos : ${DB_NAME}"
echo "Usuario       : ${DB_USER}"
echo "Password      : [no se muestra]"
echo "========================================="
echo "Siguiente paso ->  sudo bash setup-env.sh"
```

`setup-env.sh`

Genera el archivo `/etc/saludya/saludya.env` con todas las variables del backend (base de datos, secretos JWT/QR/cifrado, verificación de identidad, correo y bootstrap), con permisos `600` para mantener los secretos protegidos.

```
#!/usr/bin/env bash
#
# setup-env.sh - Crea /etc/saludya/saludya.env con TODAS las variables del backend.
#
#   Las variables viven FUERA del repositorio, y systemd las carga con
#   EnvironmentFile. Por eso sobreviven a cada "git pull" / despliegue.
#
# Uso:
#   sudo bash setup-env.sh
#
# Personaliza con variables de entorno:
#   DB_PASSWORD=<valor-seguro> BOOTSTRAP_PASSWORD=<valor-seguro> sudo -E bash setup-env.sh
#
set -euo pipefail

DB_HOST="${DB_HOST:-localhost}"
```

```
DB_PORT="${DB_PORT:-5432}"
DB_NAME="${DB_NAME:-saludyadb}"
DB_USER="${DB_USER:-saludya}"
DB_PASSWORD="${DB_PASSWORD:?Define DB_PASSWORD antes de ejecutar el script}"
BOOTSTRAP_PASSWORD="${BOOTSTRAP_PASSWORD:?Define BOOTSTRAP_PASSWORD antes de ejecutar el script}"
if [ "$(id -u)" -ne 0 ]; then
 echo "ERROR: ejecuta con sudo ->  sudo bash setup-env.sh" >&2
 exit 1
fi

mkdir -p /etc/saludya
cat > /etc/saludya/saludya.env <<EOF
SPRING_PROFILES_ACTIVE=prod

# --- Base de datos (PostgreSQL local) ---
DB_HOST=${DB_HOST}
DB_PORT=${DB_PORT}
DB_NAME=${DB_NAME}
DB_USER=${DB_USER}
DB_PASSWORD=${DB_PASSWORD}

# --- Secretos JWT / QR / cifrado de notificaciones ---
APPLICATION_JWT_SECRET=<GENERATE_A_UNIQUE_SECRET>
APPLICATION_QR_SECRET=<GENERATE_A_UNIQUE_SECRET>
IAM_NOTIFICATIONS_ENCRYPTION_KEY=<GENERATE_A_BASE64_ENCODED_KEY>

# --- Verificacion de identidad (ApiPeru) ---
IAM_IDENTITY_MODE=apiperu
IAM_IDENTITY_URL=https://api.apiperu.dev/dni
IAM_IDENTITY_API_KEY=<IDENTITY_PROVIDER_API_KEY>

# --- Administrador inicial (bootstrap) ---
IAM_BOOTSTRAP_ENABLED=true
IAM_BOOTSTRAP_EMAIL=<BOOTSTRAP_ADMIN_EMAIL>
IAM_BOOTSTRAP_PASSWORD=${BOOTSTRAP_PASSWORD}

# --- Correo (Gmail SMTP) ---
MAIL_ENABLED=true
MAIL_USERNAME=<SMTP_USERNAME>
MAIL_FROM=<SMTP_FROM_ADDRESS>
MAIL_APP_PASSWORD=<SMTP_APP_PASSWORD>
IAM_NOTIFICATIONS_LOG_CODES=true
EOF

chmod 600 /etc/saludya/saludya.env
chown root:root /etc/saludya/saludya.env

echo "OK -> /etc/saludya/saludya.env creado (permisos 600)."
```

```
echo "Editar  ->  sudo nano /etc/saludya/saludya.env"
echo "Aplicar  ->  sudo systemctl restart saludya"
```

deploy.sh

Despliega o actualiza el backend en la EC2: obtiene el código (`git clone` la primera vez o `git pull` después), compila el JAR con Maven (`./mvnw`), (re)genera el servicio `systemd` , lo reinicia y verifica que el backend responda.

```
#!/usr/bin/env bash
#
# deploy.sh - Despliega o ACTUALIZA el backend SaludYa en la EC2.
#
#   * Clona el repo la primera vez, o hace "git pull" en las siguientes
#   * Compila el JAR con Maven (./mvnw)
#   * (Re)genera el servicio systemd y lo reinicia
#   * Verifica que responda
#
# Uso:
#   bash deploy.sh
#
# Requisitos: haber corrido antes setup-server.sh y setup-env.sh
#
set -euo pipefail

REPO_URL="${REPO_URL:-https://github.com/RuwaLabs/backend-saludya.git}"
BRANCH="${BRANCH:-develop}"
APP_DIR="${APP_DIR:-$HOME/backend-saludya}"
ENV_FILE="${ENV_FILE:-/etc/saludya/saludya.env}"
SERVICE="saludya"
JAR="$APP_DIR/saludya/target/saludya-0.0.1-SNAPSHOT.jar"

if [ ! -f "$ENV_FILE" ]; then
  echo "ERROR: falta $ENV_FILE. Corre primero ->  sudo bash setup-env.sh" >&2
  exit 1
fi

echo "==> [1/4] Obteniendo codigo ..."
if [ -d "$APP_DIR/.git" ]; then
  git -C "$APP_DIR" fetch origin "$BRANCH"
  git -C "$APP_DIR" reset --hard "origin/$BRANCH"
else
  git clone --branch "$BRANCH" "$REPO_URL" "$APP_DIR"
fi

echo "==> [2/4] Compilando (Maven) ..."
cd "$APP_DIR/saludya"
./mvnw -q clean package -DskipTests
```

```
echo "==> [3/4] Configurando servicio systemd ..."
sudo tee /etc/systemd/system/saludya.service >/dev/null <<EOF
[Unit]
Description=SaludYa backend
After=network.target postgresql.service

[Service]
User=$USER
WorkingDirectory=$APP_DIR
EnvironmentFile=$ENV_FILE
ExecStart=/usr/bin/java -Xmx384m -XX:+UseSerialGC -jar $JAR
SuccessExitStatus=143
Restart=on-failure
RestartSec=10

[Install]
WantedBy=multi-user.target
EOF

sudo systemctl daemon-reload
sudo systemctl enable "$SERVICE" >/dev/null 2>&1 || true
sudo systemctl restart "$SERVICE"

echo "==> [4/4] Verificando ..."
for _ in $(seq 1 60); do
 code="$(curl -s -o /dev/null -w '%{http_code}' http://localhost:8080/v3/api-docs || true)"
 if [ "$code" = "200" ]; then
  echo "OK: backend arriba -> http://localhost:8080/v3/api-docs (200)"
  break
 fi
 sleep 2
done

sudo systemctl status "$SERVICE" --no-pager | head -15
echo
echo "Logs en vivo ->  journalctl -u $SERVICE -f"
```

Evidencias de despliegue

A continuación se presentan las capturas del proceso de despliegue en AWS.

Creación del key pair (acceso SSH)

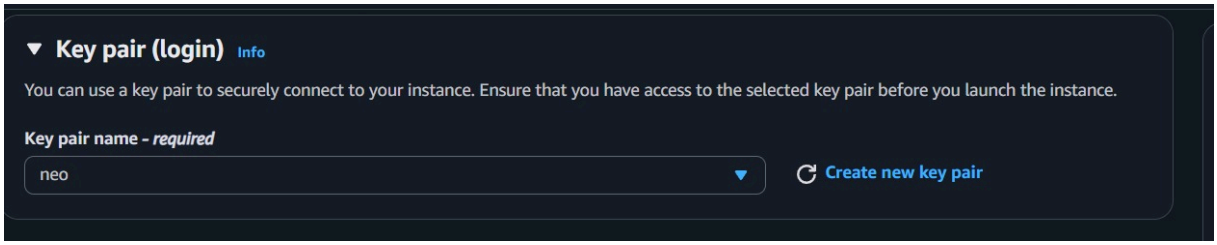


Figura 190. Creación del key pair en AWS, necesario para acceder por SSH a la instancia.

Instancia EC2 (resumen)

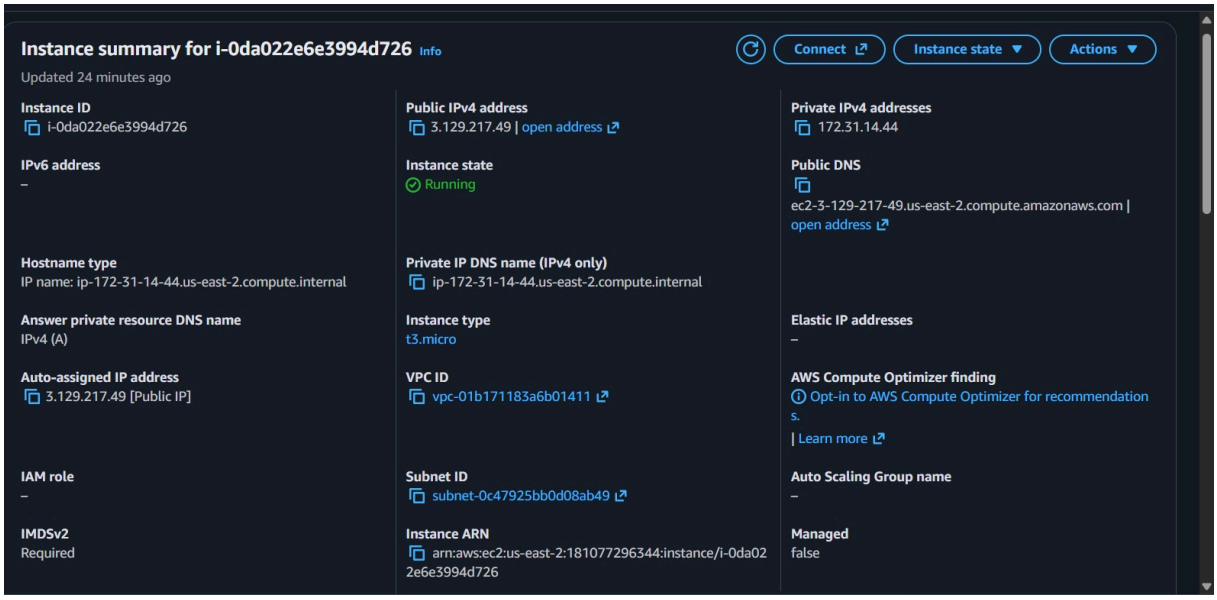


Figura 191. Resumen de la instancia EC2 donde se desplegó el backend (Web Services).

Configuración de red (security group)

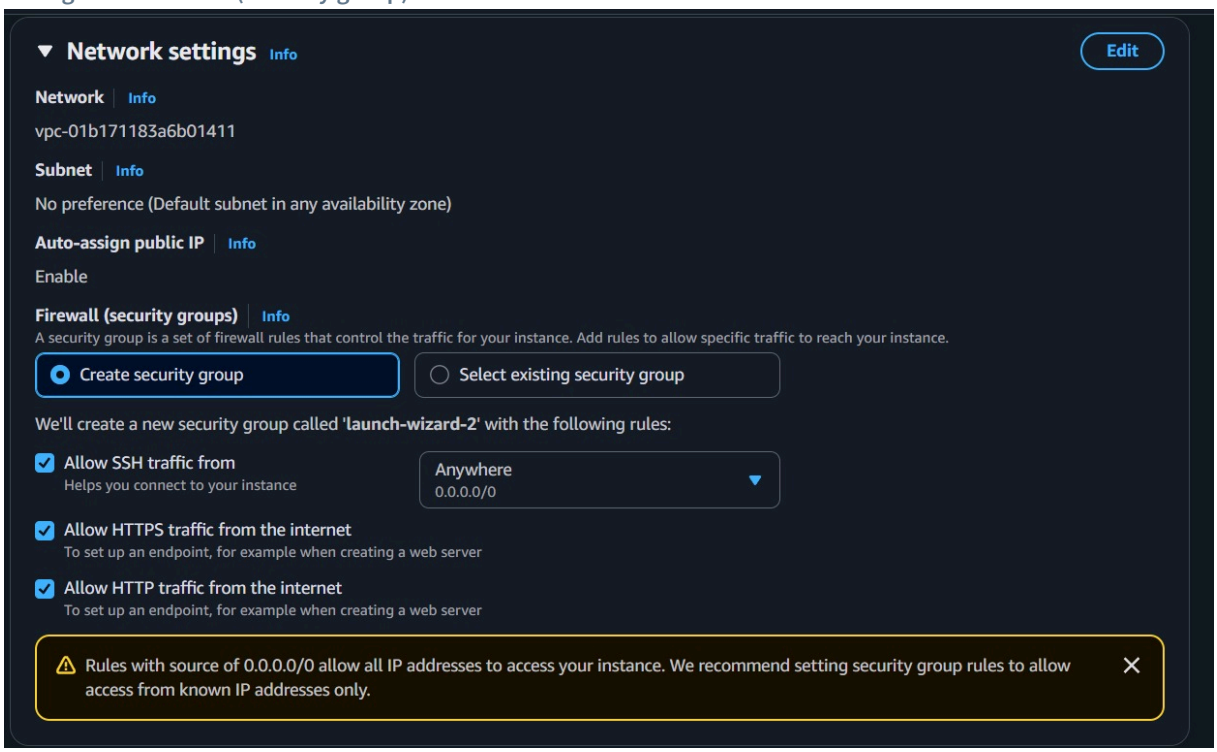


Figura 192. Configuración de red de la instancia, con los puertos habilitados para el acceso al backend.

Acceso SSH a la instancia

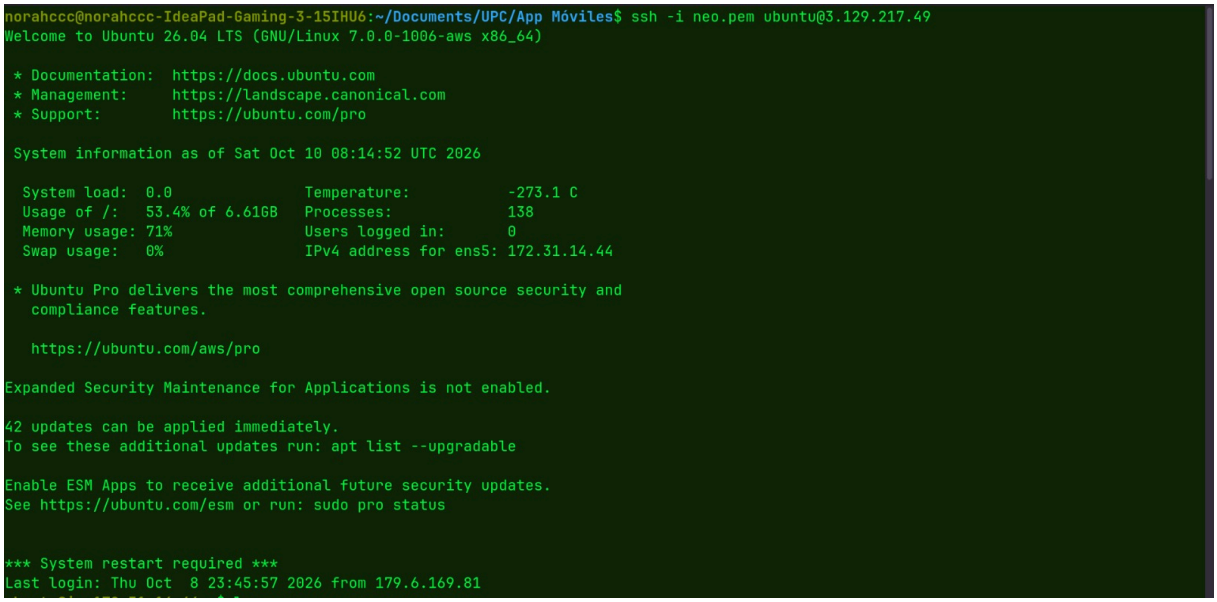


Figura 193. Acceso por SSH a la instancia EC2 del backend.

Scripts de despliegue en el servidor

Figura 194. Listado del directorio del servidor donde se observan los scripts de despliegue (`setup-server.sh` *,* `setup-env.sh` *y* `deploy.sh` *).*

Landing Page

El Landing Page de SaludYa se desplegó como un sitio estático utilizando **GitHub Pages**, aprovechando la integración nativa con el repositorio del proyecto y la ausencia de costos asociados para proyectos académicos. El despliegue se realizó sobre la rama `main` del repositorio RuwaLabs/saludya-landing, y el sitio quedó publicado de forma automática en la URL:

URL de despliegue: https://ruwalabs.github.io/saludya-landing/

Las actividades realizadas durante el Sprint para el despliegue del Landing Page fueron:

- **Creación del repositorio en GitHub:** se creó el repositorio `saludya-landing` dentro de la organización RuwaLabs, con la estructura de carpetas `assets/css`, `assets/js`, `assets/img`, `assets/locales` y el archivo `index.html` en la raíz.
- **Configuración de GitHub Pages:** se habilitó la publicación del sitio desde la rama `main` y la carpeta `/ (root)` del repositorio.
- **Automatización del despliegue:** se incorporó un flujo de trabajo de GitHub Actions (`.github/workflows/deploy.yml`) que publica automáticamente el sitio cada vez que se hace `push` a la rama `main` .
- **Verificación del despliegue:** se comprobó el acceso público al sitio mediante la URL generada por GitHub Pages y se validó la carga de estilos, scripts e imágenes.

A continuación se presentan las capturas de las actividades realizadas.

Configuración de GitHub Pages

En la configuración del repositorio se habilitó GitHub Pages como fuente de publicación, seleccionando la rama `main` y la carpeta `/ (root)`. La plataforma genera automáticamente la URL pública del sitio a partir del nombre de la organización y del repositorio.

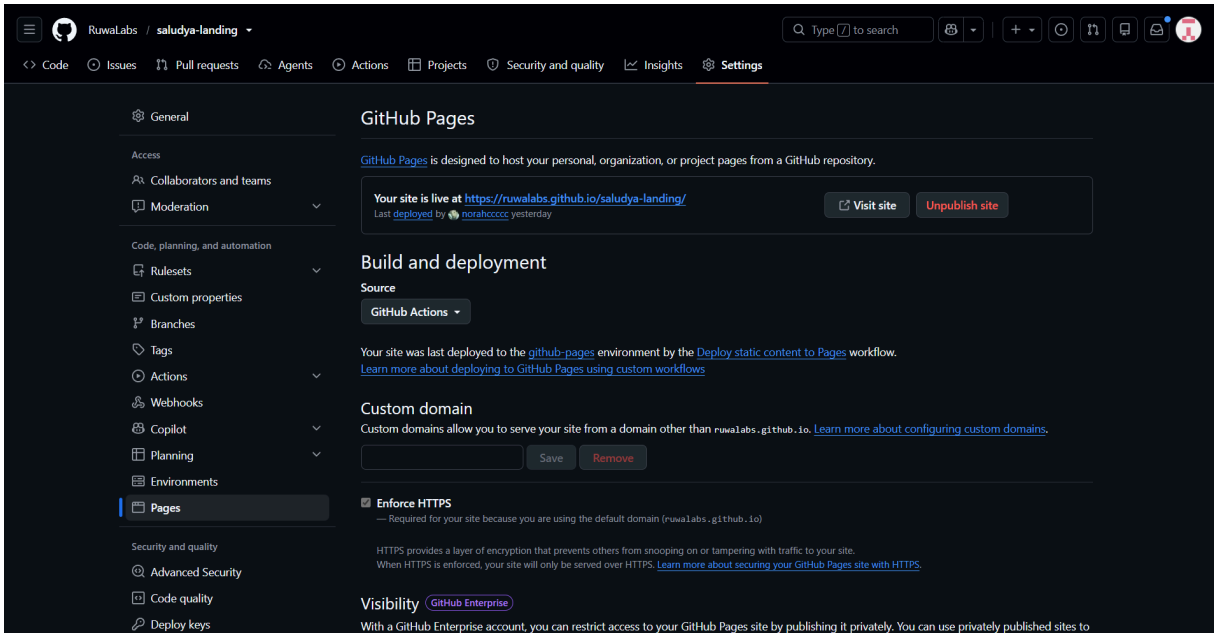


Figura 195. Configuración de GitHub Pages en el repositorio del Landing Page. Nota. Captura de la sección Settings → Pages del repositorio `saludya-landing` en GitHub.

Flujo de trabajo de GitHub Actions

Para automatizar la publicación del Landing Page, se incorporó un workflow de GitHub Actions (`.github/workflows/deploy.yml`) que se ejecuta en cada `push` a la rama `main`. El workflow instala las dependencias necesarias, prepara el sitio estático y lo publica en GitHub Pages.

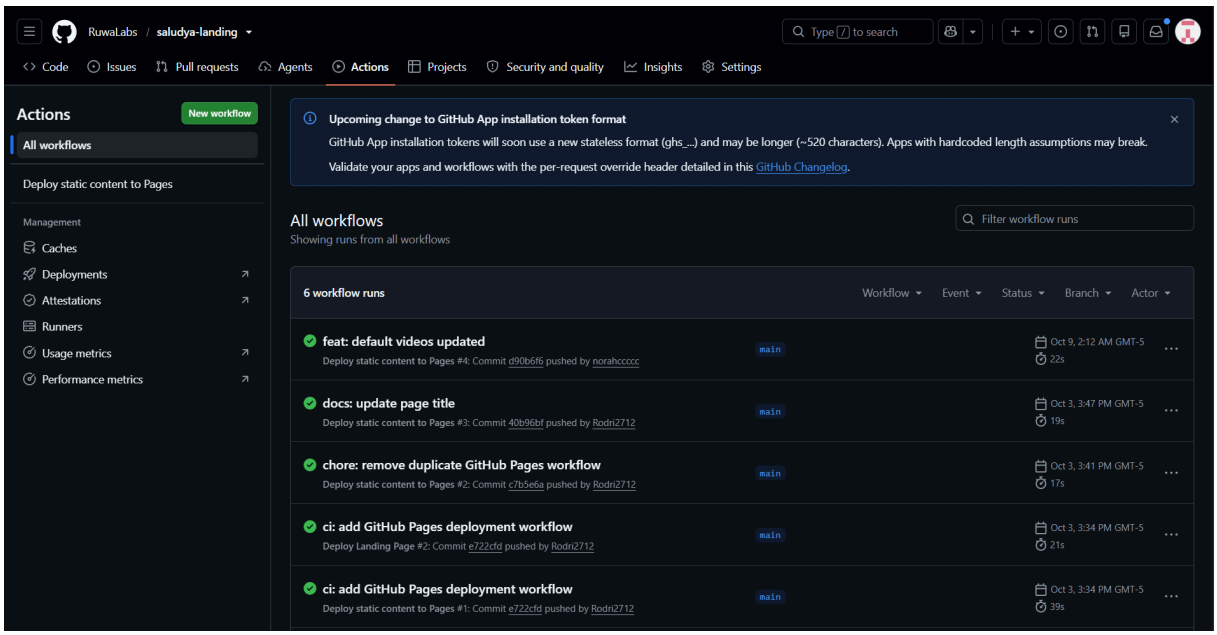


Figura 196. Ejecución exitosa del workflow de despliegue. Nota. Captura de la pestaña Actions del repositorio `saludya-landing`, donde se observa la ejecución exitosa del workflow `Deploy to GitHub Pages`.

Verificación del despliegue

Finalmente, se verificó el acceso público al Landing Page mediante la URL generada por GitHub Pages, comprobando la correcta carga de la página principal, los estilos, los scripts de internacionalización y las imágenes del Hero.

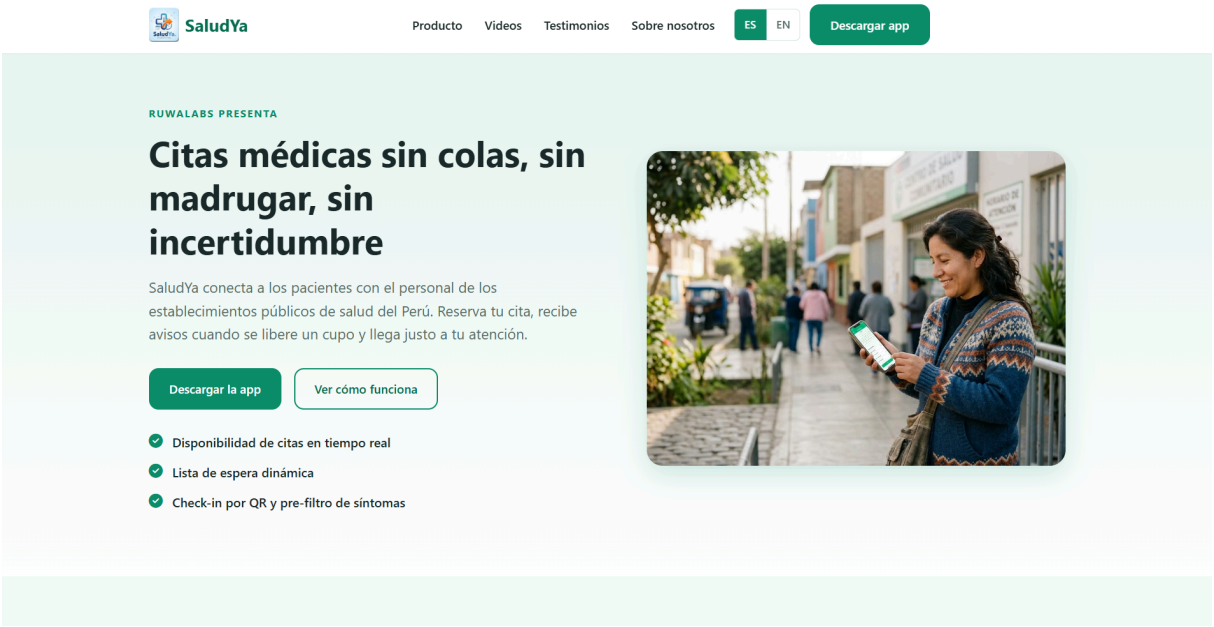


Figura 197. Landing Page de SaludYa publicado en GitHub Pages. Nota. Captura del Landing Page accesible en `https://ruwalabs.github.io/saludya-landing/` .

Con estas actividades, el Landing Page quedó publicado, disponible para su consulta pública y con despliegue automatizado ante cada cambio en la rama `main`, cumpliendo con el objetivo del Sprint 1 de presentar la propuesta de valor de SaludYa.

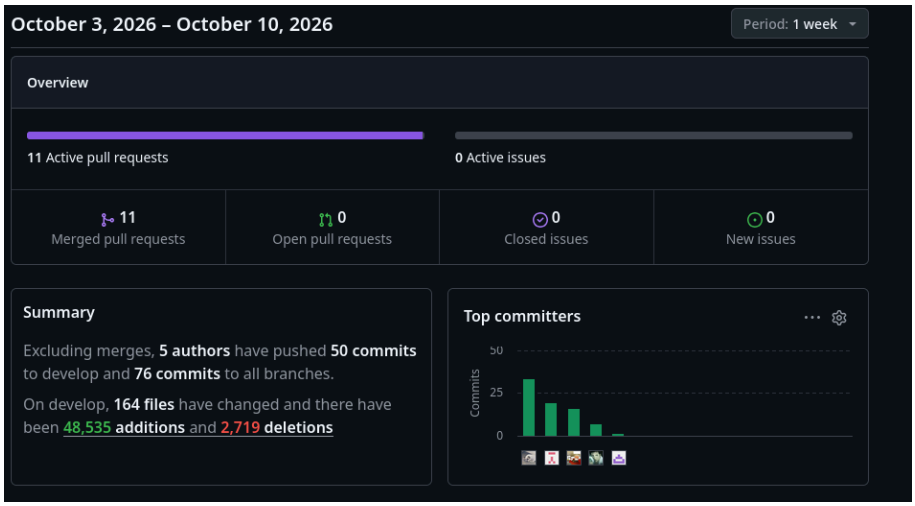
4.2.1.9. Team Collaboration Insights during Sprint

Durante el Sprint 1, el equipo **RuwaLabs** implementó el **Landing Page**, los **Web Services** (backend) y la **aplicación móvil para pacientes** (frontend). Todos los integrantes participaron en la implementación de los productos del alcance, distribuidos por producto y por bounded context, según lo definido en la matriz LACX del apartado 4.2.1.2.

La colaboración se registra en GitHub a través de los commits y *pull requests* de cada repositorio. El analítico de actividad del periodo del Sprint evidencia la participación de los cinco integrantes —con **11 pull requests** integrados y **50 commits** en `develop` realizados por **5 autores**—, como se observa en la Figura 198 y la Figura 199.

Figura 198

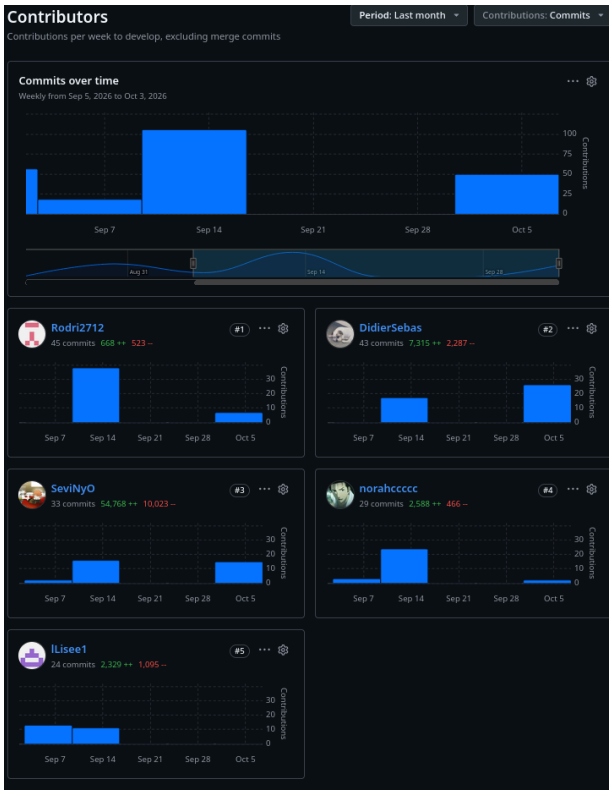
Actividad del equipo durante el Sprint 1



Nota. Captura del panel Overview de GitHub correspondiente al periodo del Sprint 1.

Figura 199

Contribuciones por integrante durante el Sprint 1



Nota. Captura del panel Contributors de GitHub correspondiente al periodo del Sprint 1.

La implementación se distribuyó de la siguiente manera:

- **Web Services (backend):** Didier (Identity & Access Management), Paula (Appointments & Booking), Neo (Dynamic Waitlist & Reassignment) y Alise (Arrival & QR Check-in), con Rodrigo a cargo de Hospital Operations & Configuration.

- **Mobile Applications (frontend):** Didier (IAM), Paula (reservas), Alise (check-in), Rodrigo (configuración) y Neo (dashboard y notificaciones/reasignación).
- **Landing Page:** Rodrigo.

La interpretación de los analíticos muestra que la actividad del Sprint se concentró en la integración de los bounded contexts y en la construcción de los recorridos del paciente, con una participación equilibrada de los cinco integrantes del equipo.

4.2.2. Sprint 2

4.2.2.1. Sprint Planning 2

A continuación se presentan los aspectos principales de la reunión de planificación del Sprint 2 de **SaludYa**, organizada por Paula Montoya el **25 de octubre de 2026** mediante **Discord**. En esta reunión el equipo **RuwaLabs** definió el Sprint Goal, el alcance de user stories enfocado en el personal administrativo y la capacidad de trabajo del segundo sprint, como se detalla en la Tabla 79.

Tabla 79

Sprint Planning 2 — resumen de la reunión

Campo	Valor
Sprint #	Sprint 2
Sprint Planning Background	
Date	2026-10-25
Time	17:00
Location	Reunión virtual mediante Discord
Prepared By	Montoya Nina, Paula Fernanda
Attendees (to planning meeting)	Aguilar Untiveros, Rodrigo Fabrizio / Meza Solórzano, Didier Sebastian / Montoya Nina, Paula Fernanda / Ramos Mera, Neo Daniel / Torres Juárez, Alisee Muriel
Sprint 1 Review Summary	El primer sprint se enfocó en la aplicación para pacientes, los servicios web y el Landing Page; el informe presenta evidencias de desarrollo, pruebas, ejecución y despliegue de estos productos. El segundo sprint extiende la solución al flujo administrativo.
Sprint 1 Retrospective Summary	La organización por módulos y responsables del primer sprint se retoma para el segundo. Se prioriza la integración de la navegación y los flujos administrativos para mantener una experiencia coherente entre las funcionalidades.
Sprint Goal & User Stories	
Sprint 2 Goal	Facilitar al personal de admisión y al Super Admin la operación diaria del establecimiento —acceso, agenda, llegada y cola, reglas e indicadores— desde la aplicación móvil administrativa; validar los flujos comprometidos al cierre del 18/11/2026.
Sprint 2 Velocity	84 story points
Sum of Story Points	84

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

El **Sprint Goal** se formula desde el resultado para los usuarios, su impacto y una condición de comprobación, siguiendo el enfoque del Statement:

> **Sprint 2 Goal:** *Nuestro **focus** está en permitir que el personal de admisión y el Super Admin gestionen la jornada de atención desde la aplicación móvil de SaludYa, con acceso según rol, calendario operativo, registro de llegada, cola presencial, reglas e indicadores.*
>
> *Creemos que esto **delivers** una operación más ordenada y visible para el establecimiento y una atención presencial más ágil para los pacientes.*
>
> *Esto se confirmará (**confirmed**) cuando el personal pueda iniciar sesión, consultar y editar bloques del calendario, registrar llegadas por QR o código, gestionar el llamado y la atención en cola, consultar indicadores y exportar un reporte; y el Super Admin pueda acceder y configurar reglas operativas, con los flujos demostrados en la aplicación y contrastados con los criterios de aceptación de las 24 historias comprometidas.*

La **velocity** del Sprint 2 se estableció en **84 story points**, equivalente a la suma de las 24 historias incluidas en este sprint. El alcance se distribuye por responsable como se detalla en la Tabla 80.

Tabla 80

Alcance del Sprint 2 por módulo

Módulo	Responsable	User Stories	Story Points
Calendario	Montoya Nina, Paula Fernanda	US-36, US-37, US-38, US-39, US-40, US-42	18
Cita y atención presencial	Torres Juárez, Alisee Muriel	US-43, US-44, US-45, US-46, US-47, US-48, US-49	30
Configuración	Aguilar Untiveros, Rodrigo Fabrizio	US-33, US-34, US-35	10
Inicio e indicadores	Meza Solórzano, Didier Sebastian	US-41, US-50, US-51, US-52, US-53	17
Login e integración	Ramos Mera, Neo Daniel	US-30, US-31, US-32	9
Total	**5 integrantes**	**24 user stories**	**84**

4.2.2.2. Aspect Leaders and Collaborators

En el Sprint 2 el equipo organizó el trabajo en función de los módulos funcionales de la aplicación móvil administrativa: **Acceso y cuentas (IAM)**, **Inicio e indicadores**, **Agenda operativa**, **Cita y atención presencial**, **Configuración operativa** e **Integración de la aplicación**. La siguiente matriz LACX (Leadership-and-Collaboration Matrix) indica, para cada aspecto, quién asume el rol de **líder (L)** y quién de **colaborador (C)**, como se detalla en la Tabla 81. Esta organización guarda relación con la posterior asignación de tasks en el Sprint Backlog 2.

Tabla 81

Leadership-and-Collaboration Matrix (LACX) — Sprint 2

Team Member (Last Name, First Name)	GitHub Username	Acceso y cuentas (IAM)	Inicio e indicadores	Agenda operativa	Cita y atención presencial	Configuración operativa	Integración de la aplicación
Aguilar Untiveros, Rodrigo Fabrizio	Rodri2712	C	C	C	C	L	C
Meza Solórzano, Didier Sebastian	DidierSebas	C	L	C	C	C	C
Montoya Nina, Paula Fernanda	SeviNyO	C	C	L	C	C	C
Ramos Mera, Neo Daniel	norahccccc	L	C	C	C	C	L
Torres Juárez, Alisee Muriel	lLisee1	C	C	C	L	C	C

Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa. L = Leader, C = Collaborator.

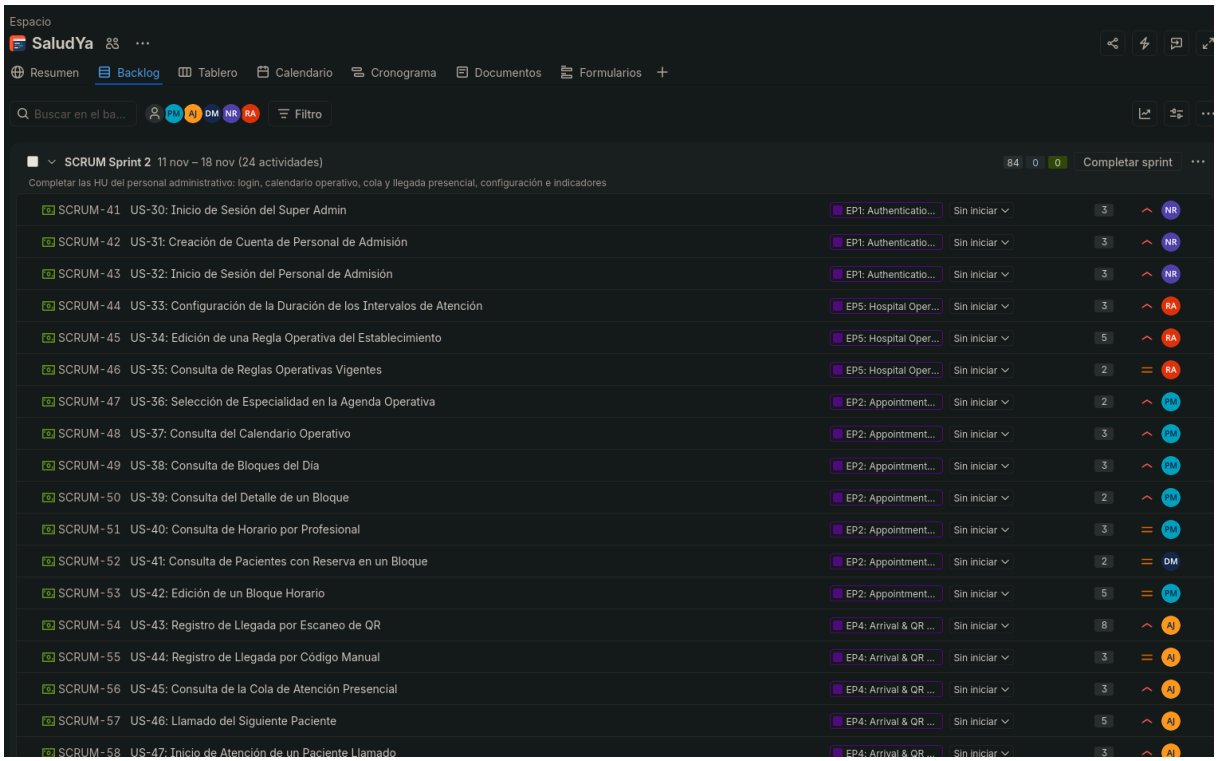
4.2.2.3. Sprint Backlog 2

El Sprint 2 tiene como objetivo habilitar la gestión diaria de la atención para el personal administrativo desde la aplicación móvil Flutter/Dart. Para su gestión, el equipo utiliza **Jira** como herramienta de control; el tablero del proyecto se puede revisar en el siguiente enlace y en la Figura 200.

- Link Jira: https://ruwalabs-salud-ya.atlassian.net/

Figura 200

Tablero de Jira del Sprint 2



Nota. Captura del tablero SCRUM del Sprint 2 en Jira, con las 24 user stories del sprint y sus responsables.

A partir de las user stories comprometidas, el equipo descompuso el trabajo en las siguientes tasks (work--items), con su estimación en horas, responsable y estado, como se detalla en la Tabla 82.

Tabla 82

Sprint Backlog 2

Sprint #	User Story Id	User Story Title	Work-Item / Task Id	Work-Item / Task Title	Description	Estimation (Hours)	Assigned To	Status
Sprint 2	SCRUM-41 · US-30	Inicio de Sesión del Super Admin	S2-T01	Acceso Super Admin	Construir formulario y control de sesión por rol.	6	Neo	To-do
Sprint 2	SCRUM-42 · US-31	Creación de Cuenta de Personal de Admisión	S2-T02	Alta de personal	Construir formulario de creación y validaciones de cuenta.	6	Neo	To-do

Sprint #	User Story Id	User Story Title	Work-Item / Task Id	Work-Item / Task Title	Description	Estimation (Hours)	Assigned To	Status
Sprint 2	SCRUM-43 · US-32	Inicio de Sesión del Personal de Admisión	S2-T03	Acceso de admisión	Implementar inicio de sesión y navegación al módulo administrativo.	6	Neo	To-do
Sprint 2	SCRUM-44 · US-33	Configuración de la Duración de los Intervalos	S2-T04	Duración de intervalos	Crear controles y validación de intervalos de atención.	6	Rodrigo	To-do
Sprint 2	SCRUM-45 · US-34	Edición de una Regla Operativa del Establecimiento	S2-T05	Editar regla operativa	Crear formulario de edición y confirmación de cambios.	10	Rodrigo	To-do
Sprint 2	SCRUM-46 · US-35	Consulta de Reglas Operativas Vigentes	S2-T06	Consultar reglas vigentes	Mostrar reglas y estados de la configuración.	4	Rodrigo	To-do
Sprint 2	SCRUM-47 · US-36	Selección de Especialidad en la Agenda Operativa	S2-T07	Selector de especialidad	Permitir elegir especialidad y filtrar la agenda .	4	Paula	To-do
Sprint 2	SCRUM-48 · US-37	Consulta del Calendario Operativo	S2-T08	Calendario mensual	Mostrar fecas con bloques y estado	6	Paula	To-do

Sprint #	User Story Id	User Story Title	Work-Item / Task Id	Work-Item / Task Title	Description	Estimation (Hours)	Assigned To	Status
					sin bloques.			
Sprint 2	SCRUM-49 · US-38	Consulta de Bloques del Día	S2-T09	Bloques por día	Mostrar bloques correspondientes a la fecha seleccionada.	6	Paula	To-do
Sprint 2	SCRUM-50 · US-39	Consulta del Detalle de un Bloque	S2-T10	Detalle de bloque	Mostrar horario, profesional y disponibilidad del bloque.	4	Paula	To-do
Sprint 2	SCRUM-51 · US-40	Consulta de Horario por Profesional	S2-T11	Horario profesional	Filtrar y consultar horarios por profesional.	6	Paula	To-do
Sprint 2	SCRUM-52 · US-41	Consulta de Pacientes con Reserva en un Bloque	S2-T12	Reservas de bloque	Mostrar pacientes con reserva en un bloque.	4	Didier	To-do
Sprint 2	SCRUM-53 · US-42	Edición de un Bloque Horario	S2-T13	Editar bloque	Editar horario y validar cambios de un bloque.	10	Paula	To-do
Sprint 2	SCRUM-54 · US-43	Registro de Llegada por Escaneo de QR	S2-T14	Escanear llegada	Integrar cámara y lectura de QR para registrar	16	Alisee	To-do

Sprint #	User Story Id	User Story Title	Work-Item / Task Id	Work-Item / Task Title	Descrip-tion	Esti-mation (Hours)	Assigned To	Status
					la llega-da.			
Sprint 2	SCRUM-55 · US-44	Registro de Lle-gada por Código Manual	S2-T15	Código manual	Capturar y validar el código de reser-va alter-nativo.	6	Alisee	To-do
Sprint 2	SCRUM-56 · US-45	Consulta de la Cola de Aten-ción Pre-sencial	S2-T16	Cola pre-sencial	Mostrar pacientes y estados de la cola.	6	Alisee	To-do
Sprint 2	SCRUM-57 · US-46	Llama-do del Si-guiente Paciente	S2-T17	Llamar si-guiente	Agregar acción de llamado y actualizar el estado en cola.	10	Alisee	To-do
Sprint 2	SCRUM-58 · US-47	Inicio de Atención de un Pa-ciente Llamado	S2-T18	Iniciar atención	Registrar inicio de atención de pa-ciente lla-mado.	6	Alisee	To-do
Sprint 2	SCRUM-59 · US-48	Finaliza-ción de la Atención de un Pa-ciente	S2-T19	Finalizar atención	Registrar cierre de la aten-ción y ac-tualizar la cola.	6	Alisee	To-do
Sprint 2	SCRUM-60 · US-49	Registro de Ausen-cia de un Paciente	S2-T20	Registrar ausencia	Marcar ausencia y reflejar-la en la cola.	10	Alisee	To-do
Sprint 2	SCRUM-61 · US-50	Consulta de Citas Cancela-	S2-T21	Cance-laciones del día	Mostrar lista de ci-tas cance-	4	Didier	To-do

Sprint #	User Story Id	User Story Title	Work-Item / Task Id	Work-Item / Task Title	Descrip-tion	Esti-mation (Hours)	Assigned To	Status
		das del Día			ladas para la jornada.			
Sprint 2	SCRUM-62 · US-51	Visualización de Indicadores del Día	S2-T22	Indicadores diarios	Mostrar métricas operativas del día.	10	Didier	To-do
Sprint 2	SCRUM-63 · US-52	Visualización de Demanda por Especialidad	S2-T23	Demanda por especialidad	Mostrar demanda desagregada por especialidad.	6	Didier	To-do
Sprint 2	SCRUM-64 · US-53	Exportación de Reporte Operativo	S2-T24	Exportar reporte	Seleccionar filtros y exportar el reporte operativo.	10	Didier	To-do

*Nota. Elaboración del equipo RuwaLabs para el proyecto SaludYa.

4.2.2.4. Development Evidence for Sprint Review

El desarrollo del Sprint 2 se concentró en la aplicación Flutter/Dart para el personal de admisión y el Super Admin. Los avances se distribuyeron en ramas por módulo.

- **Acceso administrativo (`login`).** Inicio de sesión con verificación por correo, recuperación de acceso y validación del rol de admisión o Super Admin (US-30 y US-32).
- **Inicio del personal (`home`).** Resumen de reservas y cancelaciones del día, listado de pacientes por bloque y estado de llegada o atención (US-41 y US-50). El módulo utiliza la sesión del personal para consultar el backend y reutiliza los accesos de registro de llegada y cola.
- **Agenda operativa (`schedule`).** Selección de especialidad y profesional, calendario, bloques del día, detalle y edición de bloque (US-36 a US-40 y US-42).
- **Llegada y cola presencial (`qr`).** Registro por escaneo o código manual, consulta de cola y acciones para llamar, iniciar, finalizar o marcar una ausencia (US-43 a US-49).
- **Reglas operativas (`settings`).** Consulta y edición de configuración, capacidad, intervalos y otras reglas del establecimiento (US-33 a US-35).

En paralelo, la aplicación del paciente en Kotlin avanzó en `patient-configuration` : se definió la API del perfil, se inyectó el cliente con Hilt y se conectó la pantalla de configuración a la cuenta autenticada. Se registra

como avance complementario, fuera del alcance principal de Flutter; estos commits todavía no están en `develop` de Kotlin.

En las ramas revisadas no se identificó evidencia suficiente para dar por completadas la creación de cuentas de admisión (US-31), la vista completa de indicadores (US-51), la demanda por especialidad (US-52) ni la exportación de reportes (US-53). El resumen de Inicio sí presenta conteos de reservas y cancelaciones de la jornada.

La Tabla 83 reúne los commits que respaldan estos avances, con el mensaje original y la rama que los contiene.

Tabla 83

Commits de implementación de SaludYa para la revisión del Sprint 2

Repository	Branch	Commit Id	Commit Message	Committed On
RuwaLabs/frontend-flutter-saludya	develop	5a7044a52569acb529…	feat: Inicializando proyecto con Flutter	30/10/2026
RuwaLabs/frontend-flutter-saludya	login	05b445712c8231e600…	feat(auth): implement staff login and password recovery	31/10/2026
RuwaLabs/frontend-flutter-saludya	login	13e53e875a6ec09612…	fix(auth): point Flutter API to deployed backend	31/10/2026
RuwaLabs/frontend-flutter-saludya	home	f54100022b0e10daa8…	feat(home): load staff dashboard and reservation data from backend	31/10/2026
RuwaLabs/frontend-flutter-saludya	home	fbcac20dc85553635a…	feat(home): add staff overview and reservation detail screens	31/10/2026
RuwaLabs/frontend-flutter-saludya	home	10adb156d630baec97…	feat(home): open staff dashboard after authentication	31/10/2026
RuwaLabs/frontend-flutter-saludya	home	783650498032adb4f5…	feat(home): connect arrival and queue shortcuts to existing QR module	31/10/2026
RuwaLabs/frontend-flutter-saludya	home	893db0f707d0f8cca3…	fix(home): resolve staff identity and handle queue lifecycle states	31/10/2026

Repository	Branch	Commit Id	Commit Message	Committed On
RuwaLabs/frontend-flutter-saludya	home	820f2f2d50af9d8423[illegible]	chore(home): update scanner dependency and align analyzer style	31/10/2026
RuwaLabs/frontend-flutter-saludya	schedule	cc152859ef4388321[illegible]	Domain, Application terminados	30/10/2026
RuwaLabs/frontend-flutter-saludya	schedule	d689c208bc2a7b65b[illegible]	Pantallas avanzadas	01/11/2026
RuwaLabs/frontend-flutter-saludya	qr	72862e156c858951827[illegible]	feat(qr): implement arrival check-in and attendance qr	31/10/2026
RuwaLabs/frontend-flutter-saludya	settings	cc0e20c1199b59d4e7[illegible]	feat(settings): implement configuration API integration	31/10/2026
RuwaLabs/frontend-flutter-saludya	settings	b5966b61d604d16062[illegible]	feat(settings): implement general settings screen	31/10/2026
RuwaLabs/frontend-flutter-saludya	settings	0cad45c182e13a2eef[illegible]	feat(settings): implement additional rules screen	31/10/2026
RuwaLabs/frontend-flutter-saludya	settings	83ca3e19eee6ba8262[illegible]	feat(settings): implement capacity editing screen	31/10/2026
RuwaLabs/frontend-flutter-saludya	settings	27ac50911e64ca2617[illegible]	feat(settings): implement attention intervals screen	31/10/2026
RuwaLabs/frontend-flutter-saludya	settings	0ce4c7d61eb05b4266[illegible]	fix(settings): improve capacity validation feedback	31/10/2026
RuwaLabs/frontend-kotlin-saludya	patient-configuration	3d3448871ed1d9ae9e[illegible]	chore: configure backend API connection	11/11/2026
RuwaLabs/frontend-kotlin-saludya	patient-configuration	3cb21d274752fd1924[illegible]	feat: define patient profile API endpoints	11/11/2026
RuwaLabs/frontend-kotlin-saludya	patient-configuration	749a7261c054377af8[illegible]	feat: provide PatientApi through Hilt	11/11/2026

Repository	Branch	Commit Id	Commit Message	Committed On
RuwaLabs/frontend-kotlin-saludya	patient-configuration	4079dc2f376ec5b613fcb1d4e8[illegible]a33a1c	feat: implement authenticated patient profile repository	31/11/2026
RuwaLabs/frontend-kotlin-saludya	patient-configuration	77961665061a9accb0[illegible]84a46a6a[illegible]5351a1	feat: display authenticated patient profile in configuration	31/11/2026
RuwaLabs/frontend-kotlin-saludya	patient-configuration	b93f2b93af9f51406cd[illegible]9bc7fc4bc71c	feat: handle consultation cancellation in patient profile viewmodels	31/11/2026

Nota. Elaboración del equipo RuwaLabs a partir del historial de Git, consultado el 12 de noviembre de 2026. La rama indicada contiene el commit y no implica que el cambio ya se haya integrado a develop.

4.2.2.5. Testing Suite Evidence for Sprint Review

Las pruebas automatizadas identificadas para el Sprint 2 se concentran en la aplicación administrativa de Flutter. Con `flutter_test` se comprueban pantallas y cambios de estado; las pruebas de repositorios y del cliente HTTP usan respuestas simuladas para verificar rutas, parámetros y manejo de errores sin depender del servidor publicado.

La Tabla 84 enlaza los commits que incorporaron los archivos de prueba. El commit de Agenda incluye tanto la implementación como sus pruebas.

Tabla 84

Commits de la suite de pruebas del Sprint 2

Repository	Branch	Commit Id	Commit Message	Committed On
RuwaLabs/frontend--flutter-saludya	login	05b445712c8231e600abe[illegible]2f17dbb1282d8036f	feat(auth): implement staff login and password recovery	31/10/2026
RuwaLabs/frontend--flutter-saludya	home	a9d67c53661b5089dd3b3[illegible]3a4fe55e72	test(home): cover dashboard data navigation and recovery states	31/10/2026
RuwaLabs/frontend--flutter-saludya	schedule	d689c208bc2a7b65bf06b[illegible]pantallas avanzadas	[illegible]a72e75afd6c	01/11/2026

Nota. Se verificaron los archivos de prueba modificados por los commits. Los mensajes y fechas se conservan del historial de Git.

La Tabla 85 detalla qué comportamiento verifica cada grupo de pruebas.

Tabla 85

Inventario de pruebas automatizadas del Sprint 2

Componente	Rama	Archivo	Comportamiento verificado
Acceso	`login`	auth_page_test.dart	Pantalla de inicio de sesión sin registro público y navegación a la solicitud de recuperación por correo.
Inicio: repositorio HTTP	`home`	home_repository_test.dart	Consultas autenticadas y filtros por fecha y bloque, datos vacíos, errores HTTP, y estados de llegada y atención (US-41 y US-50).
Inicio: controlador	`home`	home_controller_test.dart	Prevalencia de la respuesta más reciente, finalización segura al desmontar y limpieza de datos al fallar la sesión.
Inicio: pantallas	`home`	home_page_test.dart	Totales, navegación a cancelaciones y pacientes, reintento, accesos a calendario y QR, y desplazamiento con texto ampliado.
Agenda: navegación	`schedule`	main_schedule_test.dart	Entrada a Agenda y navegación al calendario.
Agenda: cliente HTTP	`schedule`	saludya_api_test.dart	Consulta de profesionales y bloques filtrados, y actualización del estado del bloque sin enviar el consultorio como campo editable.
Agenda: ViewModels	`schedule`	schedule_view_models_test.dart	Filtrado de especialidades y de bloques por especialidad y fecha.

El 12 de noviembre de 2026 se ejecutó `flutter test` por separado en los commits `820f2f2` de `home` y `d689c20` de `schedule`. Pasaron 21 pruebas en `home` (19 de Inicio y 2 de acceso heredadas de `login`) y 5 en `schedule`: 26 en total, sin duplicar las pruebas de acceso.

No se encontraron nuevas pruebas automatizadas en los cambios revisados de QR y reglas de Flutter, ni en los commits del perfil de paciente de Kotlin. Las pruebas de Kotlin documentadas en el Sprint 1 se conservan como evidencia de ese sprint. Queda pendiente validar los recorridos de Flutter de punta a punta con una cuenta de personal y el backend desplegado.

4.2.2.7. Services Documentation Evidence for Sprint Review

En este Sprint se continuó el trabajo sobre los servicios web de SaludYa. La documentación OpenAPI ya implementada se mantiene publicada junto con el backend desplegado en AWS EC2; el alcance del Sprint 2 no incorporó nuevos controladores, sino que la aplicación móvil administrativa consume las operaciones existentes de Identity & Access Management, Appointments & Booking, Arrival & QR Check-

-in y Hospital Operations & Configuration. La documentación comprende 62 operaciones distribuidas en 53 rutas.

La especificación OpenAPI se publica en `/v3/api-docs` y se mantiene exportada en el repositorio de Web Services (`docs/api/openapi.json`) para su versionado. La API utiliza autenticación HTTP Bearer con JWT (esquema `bearerAuth`); los endpoints públicos (registro, inicio de sesión, verificación de identidad y recuperación de cuenta) están marcados con `@SecurityRequirements` y no requieren token, mientras que el resto exige un token vigente obtenido tras el inicio de sesión. En este Sprint se incorporaron ajustes en el flujo de creación de cuentas de personal de admisión y en la recuperación de acceso mediante código enviado por correo, reflejados en la especificación publicada.

- **Swagger UI (documentación desplegada):** http://3.129.217.49:8080/swagger-ui/index.html
- **Especificación OpenAPI (JSON):** http://3.129.217.49:8080/v3/api-docs
- **Repositorio de Web Services:** https://github.com/RuwaLabs/backend-saludya

Commits relacionados con la documentación para este Sprint

Commit	Mensaje	Relación
4a8038a	feat(iam): add email-code password recovery and staff seed	Creación de cuentas de personal de admisión y recuperación por código de correo usada por el acceso administrativo
40f9623	fix(iam): use Jackson 3 for staff seed	Ajuste de serialización del personal de admisión para la respuesta documentada del endpoint
a592333	docs(api): add OpenAPI spec export and Web Services endpoint documentation	Exportación del documento OpenAPI (`docs/api/openapi.json`) que se mantiene publicado

Tabla de endpoints documentados

A continuación se detalla, para cada endpoint, la acción implementada, el verbo HTTP y la sintaxis de llamada, los parámetros admitidos, un ejemplo de petición y de respuesta con datos de muestra, la explicación de la respuesta y el enlace a su documentación desplegada. La URL base es `http://3.129.217.49:8080` . Las operaciones corresponden a las user stories US-30 a US-53 del Sprint 2: IAM (US-30 a US-32), Appointments & Booking (US-36 a US-42 y US-50), Arrival & QR Check-in (US-43 a US-49) y Hospital Operations & Configuration (US-33 a US-35, US-51 a US-53).

IAM — Identity & Access Management

#	Acción implementada	Método	Sintaxis de llamada (endpoint)	Parámetros	Petición (ejemplo)	Respuesta (ejemplo)	Explicación del response	Documentación
1	Iniciar sesión del personal y del Super Admin	POST	/api/v1/user-accounts/login	body: email, password	{"email":"fran	200 aludya.local","password":"Saludya#2026"} {"challengeId":"8f2e1d40-...","maskedEmail":"f***@saludya.lo	Valida las credenciales y envía un código de verificación al correo; no revela	Swagger

#	Acción implementada	Método	Sintaxis de llamada (endpoint)	Parámetros	Petición (ejemplo)	Respuesta (ejemplo)	Explicación del response	Documentación
							si el correo existe	
2	Completar inicio de sesión con el código	POST	`/api/v1/user-accounts/login/verify`	body: `challengeId`, `code`	`{"challengeId":"1d40-...","code":"123456"}`	`200` · `{"accessToken":"eyJhbGciOiJIUzI1NiJ9...","tokenType":"Bearer"`	Devuelve el token de acceso (JWT) y el rol del usuario (`ADMISSION_STAFF` o `SUPER_ADMIN`)	Swagger
3	Reenviar el código del desafío de login	POST	`/api/v1/user-accounts/login/resend`	body: `challengeId`	`{"challengeId":"1d40-..."}`	`200` · *sin cuerpo*	Genera y reenvía un nuevo código para el mismo desafío	Swagger
4	Cerrar sesión (revoca el token actual)	POST	`/api/v1/user-accounts/logout`	header: `Authorization: Bearer <token>`	*(sin cuerpo)*	`200` · *sin cuerpo*	Revoca la sesión Bearer vigente; responde sin contenido	Swagger
5	Crear cuenta de personal de admisión	POST	`/api/v1/user-accounts/staff`	header: `Authorization` (SUPER_ADMIN) · body: `dni`, `name`, `lastname`, `birthDate`, `phone`, `email`	`{"dni":"70000000","name":"Franco","lastname":"Saludya","birthDate":"1999-0...`	`200` · `{"id":15,"role":"ADMISSION_STAFF","email":"franco@saludya.l...`	Crea la cuenta del personal y envía una invitación para definir contraseña	Swagger

Appointments & Booking

#	Acción implementada	Método	Sintaxis de llamada (endpoint)	Parámetros	Petición (ejemplo)	Respuesta (ejemplo)	Explicación del response	Documentación
6	Listar especialidades	GET	`/api/v1/specialties`	—	`/api/v1/specialties`	`200` · `[{"id":1,"name":"Medicina general","description":"Atención médica primaria"}]`	Devuelve el catálogo de especialidades	Swagger

#	Acción implementada	Método	Sintaxis de llamada (endpoint)	Parámetros	Petición (ejemplo)	Respuesta (ejemplo)	Explicación del response	Documentación
7	Listar doctores por especialidad	GET	`/api/v1/doctors`	query: `specialtyId` (obligatorio)	`/api/v1/doctors?specialtyId=1`	200 [{"id":7,"specialtyId":1,"name":"Ana","lastname":"Rojas"}]	Devuelve los doctores de la especialidad indicada	Swagger
8	Slots de un doctor en una fecha	GET	`/api/v1/time-slots`	query: `doctorId`, `date` (obligatorios)	`/api/v1/time-slots?doctorId=7&date=2026-11-12`	200 [{"id":31,"doctorId":7,"date":"2026-11-12","startHour":"09:00","3","maxCapacity":5,"currentBookings":2,"status":"AVAILABLE"	Devuelve los slots del doctor en la fecha indicada	Swagger
9	Obtener un slot por id	GET	`/api/v1/time-slots/{id}`	path: `id`	`/api/v1/time-slots/31`	200 {"id":31,"doctorId":7,"date":"2026-11-12","startHour":"09:00","3","maxCapacity":5,"currentBookings":2,"status":"AVAILABLE"	Devuelve el slot solicitado con su disponibilidad	Swagger
10	Editar un slot (doctor, horario y estado)	PUT	`/api/v1/time-slots/{id}`	path: `id` · body: `doctorId`, `startHour`, `endHour`, `status`	{"doctorId":7,"	200 Hour":"10:00","endHour":"10:30","status":"AVAILABLE"} {"id":31,"doctorId":7,"startHour":"10:00","endHour":"10:30","st	Edita los datos del slot y devuelve el recurso actualizado	Swagger
11	Actualizar la capacidad de un slot	PUT	`/api/v1/time-slots/{id}/capacity`	path: `id` · body: `maxCapacity`	{"maxCapacity	200 {"id":31,"maxCapacity":8,"currentBookings":2,"status":"AVAIL	Actualiza la capacidad máxima del slot	Swagger
12	Listar citas con filtros	GET	`/api/v1/appoi`	query: `patientId`, `doctorId`, `date`, `timeSlotId`, `specialtyId`, `status`	`/api/v1/appoi` `timeSlotId=31&date=2026-11-12&status=RESERVED`	200 nts? [{"id":142,"timeSlotId":31,"patientId":1,"bookingOrder":3,"boo	Devuelve filtradas del bloque o de la jornada	Swagger
13	Listar citas canceladas del día	GET	`/api/v1/appoi`	query: `date`, `status=C`	`/api/v1/appoi` `date=2026-11-12&status=CANCELLED`	200 nts? [{"id":139,"timeSlotId":28,"patientId":4,"status":"CANCELLED"	Lista las canceladas de la jornada para iden-	Swagger

#	Acción implementada	Método	Sintaxis de llamada (endpoint)	Parámetros	Petición (ejemplo)	Respuesta (ejemplo)	Explicación del response	Documentación
							tificar cupos liberados	

Arrival & QR Check-in

#	Acción implementada	Método	Sintaxis de llamada (endpoint)	Parámetros	Petición (ejemplo)	Respuesta (ejemplo)	Explicación del response	Documentación
14	Registrar check-in por token QR	POST	`/api/v1/check-ins/qr`	body: `qrToken`	`{"qrToken":"ey`	200 `{"checkInId":15,"appointmentId":142,"queueEntryId":9,"posit`	Registra la llegada, crea el ticket y la entrada en la cola; devuelve la posición	Swagger
15	Registrar check-in por código de reserva	POST	`/api/v1/check-ins/code`	body: `bookingCode`	`{"bookingCod`	200 `{"checkInId":16,"appointmentId":143,"queueEntryId":10,"posi`	Alternativa manual: registra la llegada con el código de reserva	Swagger
16	Resolver la cola de un slot en una fecha	GET	`/api/v1/attendant/queues`	query: `timeSlotId, date` (obligatorios)	`/api/v1/attend queues?timeS`	200 `{"id":4,"timeSlotId":31,"date":"2026-11-12","status":"OPEN"}`	Devuelve la de atención del slot y fecha; error si no existe	Swagger
17	Listar las entradas de una cola	GET	`/api/v1/attendant/queues/{id}/entries`	path: `id`	`/api/v1/attend queues/4/entries`	200 `[{"id":9,"attendanceQueueId":4,"checkInId":15,"position":3,"st`	Lista las entradas (pacientes) de la cola	Swagger
18	Posición actual en la cola	GET	`/api/v1/attendant/queues/{id}/position`	path: `id`	`/api/v1/attend queues/4/position`	200 `{"position":3,"totalInQueue":8}`	Devuelve la posición y el	Swagger

#	Acción implementada	Método	Sintaxis de llamada (endpoint)	Parámetros	Petición (ejemplo)	Respuesta (ejemplo)	Explicación del response	Documentación
							total de la cola	
19	Llamar al siguiente paciente	POST	`/api/v1/attendance-queues/{id}/call-next`	path: `id`	`/api/v1/attendance-queues/4/call-next`	`200` - `{"id":9,"attendanceQueueId":4,"checkInId":15,"position":1,"sta`	Llama al siguiente paciente en espera y devuelve su entrada	Swagger
20	Iniciar la atención de una entrada llamada	POST	`/api/v1/queue-entries/{id}/start`	path: `id`	`/api/v1/queue-entries/9/start`	`200` `{"id":9,"attendanceQueueId":4,"checkInId":15,"position":1,"sta` `IN_ATTENTION`	Cambia la entrada a	Swagger
21	Finalizar la atención de una entrada	POST	`/api/v1/queue-entries/{id}/finish`	path: `id`	`/api/v1/queue-entries/9/finish`	`200` `{"id":9,"attendanceQueueId":4,"status":"ATTENDED","attended`	Marca la entrada como atendida	Swagger
22	Marcar una entrada como ausente	POST	`/api/v1/queue-entries/{id}/absent`	path: `id`	`/api/v1/queue-entries/10/absent`	`200` `{"id":10,"attendanceQueueId":4,"status":"ABSENT"}`	Marca al paciente como ausente y cierra su entrada	Swagger

Hospital Operations & Configuration

#	Acción implementada	Método	Sintaxis de llamada (endpoint)	Parámetros	Petición (ejemplo)	Respuesta (ejemplo)	Explicación del response	Documentación
23	Leer la configuración del establecimiento	GET	`/api/v1/config`	—	`/api/v1/config`	`200` `{"id":1,"maxCapacityPerSlot":5,"bookingOrderScope":"PER_S`	Devuelve la configuración vigente del establecimiento	Swagger

#	Acción implementada	Método	Sintaxis de llamada (endpoint)	Parámetros	Petición (ejemplo)	Respuesta (ejemplo)	Explicación del response	Documentación
24	Actualizar la configuración del establecimiento	PUT	`/api/v1/config`	body: `maxCapacityPerSlot`, `bookingOrderScope`, `checkInToleranceMinutes`, `postCallToleranceMinutes`, `reassignmentResponseTimeoutMin`, `bookingCutoffTime`, `cancellationDeadlineHours`, `attendanceQueueVisible`	`{"maxCapacity`	200 `lot":6,"checkInToleranceMinutes":20,"postCallToleranceM` `{"id":1,"maxCapacityPerSlot":6,...,"updatedAt":"2026-11-12T0`	Actualiza la configuración y devuelve el recurso actualizado	Swagger
25	Obtener el panel de métricas del día	GET	`/api/v1/config/dashbc`	query: `date` (opcional)	`/api/v1/config/dashboard?date=2026-11-`	200 `{"configurationId":1,"metrics":[{"name":"scheduledToday","value":42},{"name":"pendingToday","value":12},{"name":"cancelledToday","value":7},{"name":"attendedToday","value":28},{"name":"absentToday","value":2}],"externalDataAvailable":fa`	Devuelve las métricas operativas del día	Swagger
26	Generar un reporte por rango de fechas	GET	`/api/v1/config/reports`	query: `from`, `to`, `format` (obligatorios; `format` por defecto `JSON`)	`/api/v1/config/reports?from=`	200 `{"format":"CSV","generatedAt":"2026-11-12T09:20:00Z","conte` `general;18;2;1;1;1;11;1;9"}`	Genera el reporte del rango indicado en el formato solicitado	Swagger `to=2026-11-12&format=CSV`

Evidencias de la documentación desplegada

A continuación se presentan las capturas de la documentación desplegada en Swagger UI para los controladores que consume la aplicación móvil administrativa. Cada captura muestra las operaciones disponibles del controlador y su protección mediante autenticación Bearer.

1. IAM - User accounts

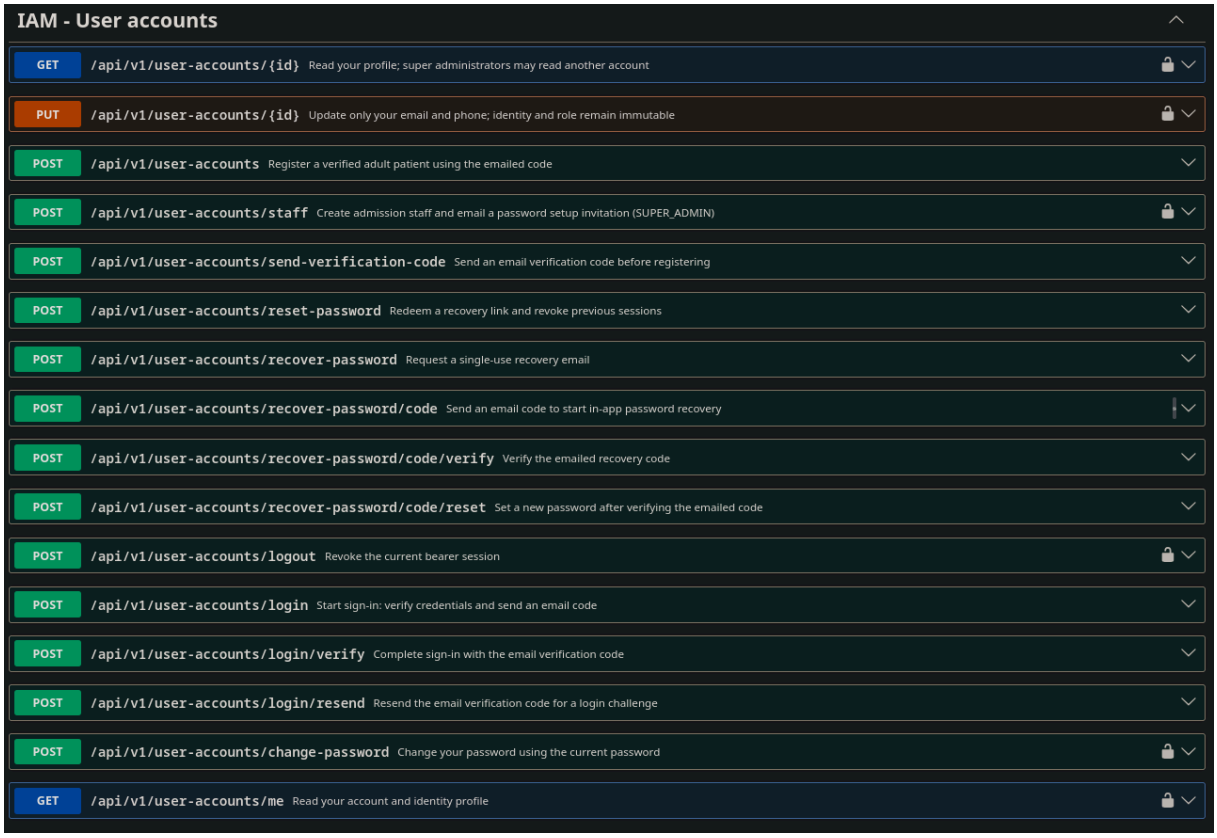


Figura 201. Documentación desplegada del controlador IAM - User accounts en Swagger UI.

2. Specialties

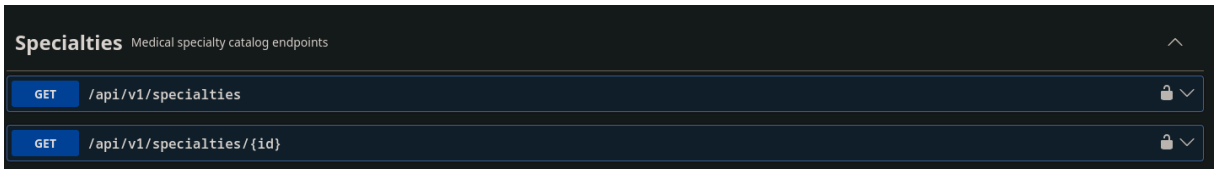


Figura 202. Documentación desplegada del controlador Specialties en Swagger UI.

3. Doctors

Figura 203. Documentación desplegada del controlador Doctors en Swagger UI.

4. Time Slots

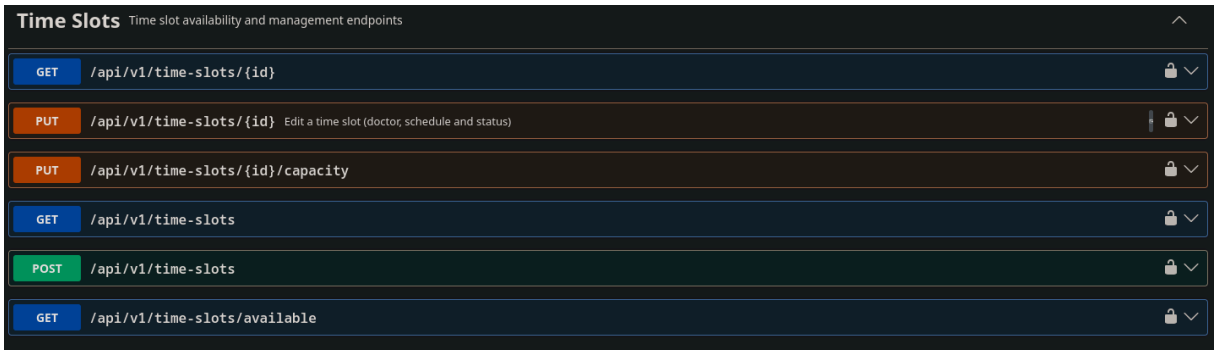


Figura 204. Documentación desplegada del controlador Time Slots en Swagger UI.

5. Appointments

Figura 205. Documentación desplegada del controlador Appointments en Swagger UI.

6. Check-ins

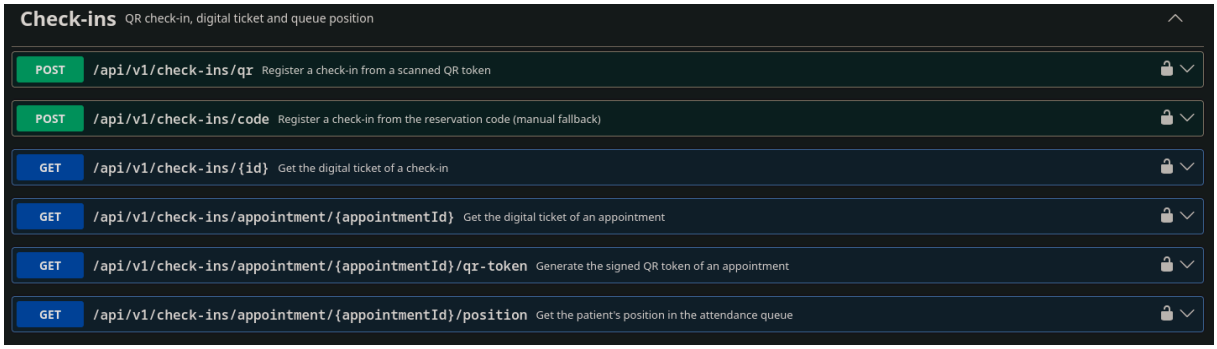


Figura 206. Documentación desplegada del controlador Check-ins en Swagger UI.

7. Attendance Queues

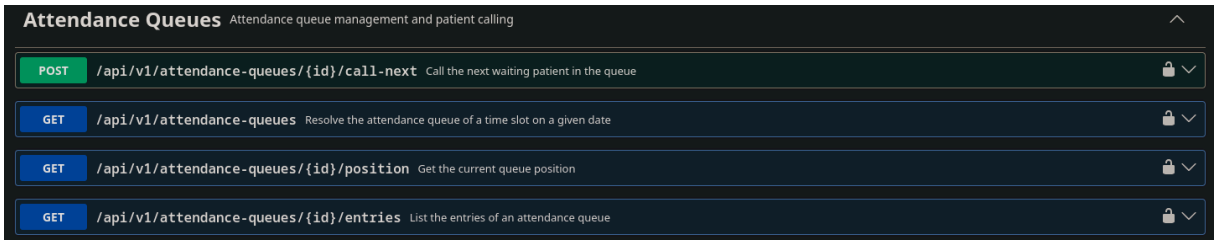


Figura 207. Documentación desplegada del controlador Attendance Queues en Swagger UI.

8. Queue Entries

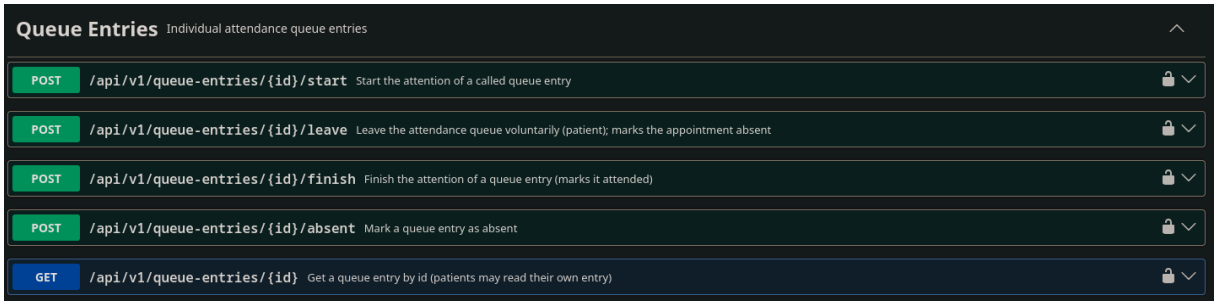


Figura 208. Documentación desplegada del controlador Queue Entries en Swagger UI.

9. configuration-controller

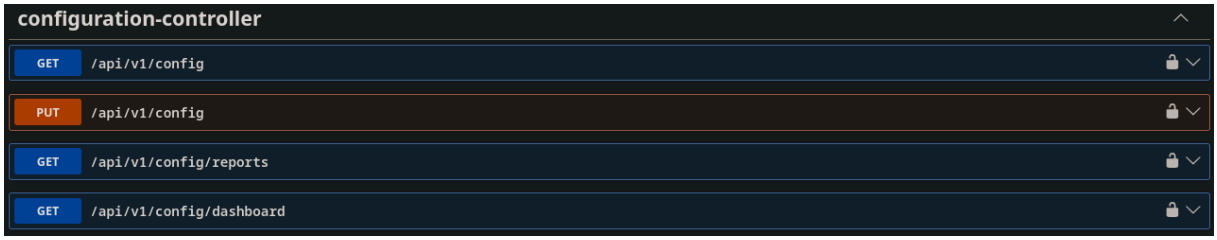


Figura 209. Documentación desplegada del controlador configuration-controller en Swagger UI.

4.2.2.8. Software Deployment Evidence for Sprint Review

El despliegue del backend, del Landing Page y de la aplicación Android ya se realizó en el Sprint 1 y se documentó en la sección 4.2.1.8. En este avance no se repite ese despliegue: la Figura 210 muestra la aplicación Android instalada y ejecutándose con el backend desplegado.

Figura 210

Aplicación Android de SaludYa en ejecución

Nota. Captura de la aplicación Android instalada y ejecutándose con el backend desplegado.

El despliegue de la aplicación Flutter (personal y administración) no forma parte de este avance; queda pendiente para una entrega posterior.

4.3. Validation Interviews

4.3.1. Diseño de Entrevistas

En esta sección se presenta el diseño de las entrevistas de validación aplicadas a los segmentos objetivo de **SaludYa**. Se elaboraron preguntas principales y complementarias orientadas a evaluar la comprensión de la propuesta de valor, la navegación, el diseño visual y la intención de uso de la **Landing Page**, considerando buenas prácticas de diseño de entrevistas. La información recolectada —características demográficas (edad, distrito de residencia, ocupación), habilidades, afinidad tecnológica, dispositivos preferidos, canales de interacción, objetivos y frustraciones— sirve como insumo para construir y sustentar los arquetipos (User Personas).

Las entrevistas se dirigieron a los dos segmentos identificados: **Segmento 1 — Pacientes de zonas urbanas periféricas** y **Segmento 2 — Personal asistencial y administrativo de establecimientos públicos de salud**. Los bloques de preguntas fueron los siguientes.

Perfil del participante

- ¿Cuál es su edad y distrito de residencia?
- ¿Con qué frecuencia usa el celular para trámites y qué tan cómodo se siente, en una escala del 1 al 10?
- ¿Cuál es su rol en el establecimiento de salud? (segmento personal)

Comprensión de la propuesta de valor

- Al abrir la página, ¿qué cree que ofrece SaludYa? Explíquelo con sus palabras.
- ¿Entendió para quién está dirigida la solución, es decir, pacientes y personal de salud?
- ¿El mensaje principal, “Citas médicas sin colas…”, le resulta claro?
- ¿Identificó qué problema resuelve la solución y cómo lo resuelve?

Navegación y arquitectura de la información

- ¿Encontró fácilmente las secciones: Producto, Videos, Testimonios, Sobre nosotros y Descarga?
- ¿La organización de la información le pareció lógica y ordenada?
- ¿Usó el selector de idioma (español o inglés)? ¿Le resultó útil?
- ¿Faltó alguna sección que esperaba encontrar?

Diseño visual, usabilidad y diseño inclusivo

- ¿El diseño le pareció claro y agradable a la vista?
- ¿El contraste, el tamaño de las letras y los botones le permitieron leer y usar la página con facilidad?
- ¿Los botones de acción, como descargar y conocer más, son visibles y fáciles de ubicar?
- ¿La página funcionó bien en su celular, en la versión móvil?

Contenido

- ¿Los testimonios influyeron en su confianza hacia SaludYa?
- ¿La sección de modelo de negocio o “sobre nosotros” aportó algo a su decisión?

Conversión e intención

- ¿Descargaría la aplicación? ¿Por qué sí o por qué no?
- ¿Qué cambiaría o agregaría para convencerlo de usar SaludYa?
- Del 1 al 5, ¿qué tan probable es que recomiende la página a otra persona?
- ¿Hay algo más que quiera comentar?

4.3.2. Registro de Entrevistas

Para cada segmento se realizaron entrevistas de validación de la **Landing Page**, registradas en video como evidencia. A continuación se presentan, para cada entrevista, los datos del participante, el enlace del video, la captura del cuadro de video, el timing de inicio y un resumen descriptivo de las respuestas, incluyendo las características objetivas y subjetivas del entrevistado.

Entrevista 1 — Segmento: Personal administrativo

Campo	Detalle
Nombre(s) y apellidos	Alvaro
Edad	46 años
Distrito	Santa
Ocupación / rol	Personal administrativo del establecimiento de salud
Entrevistador(a)	Alisee
Enlace del video	https://youtu.be/OYaoAsbkC0s
Inicio de la entrevista	00:01

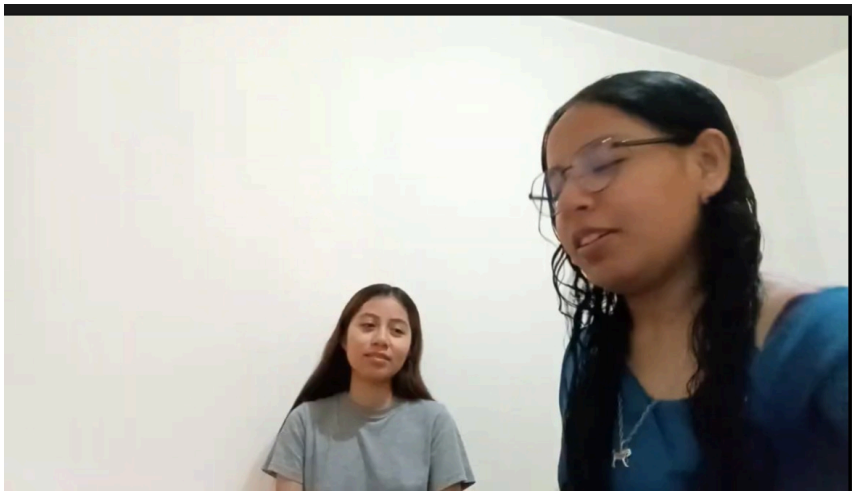


Figura 211. Cuadro de video de la entrevista 1 — personal administrativo.

Resumen. Alvaro es un adulto de 46 años, residente en el distrito de Santa, con rol administrativo en un establecimiento de salud. Usa el celular a diario para realizar trámites y califica su comodidad con el dispositivo en 7 de 10. Al explorar la Landing Page interpretó correctamente la propuesta de valor: que SaludYa ofrece soluciones al aglomeramiento del público y a la gestión de citas, aportando agilidad tanto al paciente como al personal de salud; además, el mensaje principal “Citas médicas sin colas” le resultó claro. En cuanto a la navegación, encontró las secciones de forma ordenada y la página se abrió por defecto en español. No echó en falta ninguna sección, considerándola completa. Sobre el diseño, lo percibió claro y agradable, y valoró que no fuera saturado en colores; el contraste, el tamaño de las letras y los botones le permitieron leer y usar la página con facilidad, incluso con una ligera dificultad visual. Verificó el funcionamiento en su celular Android (de gama no alta), donde la página cargó de forma veloz y correcta. Los testimonios de personas reales influyeron en su confianza y la sección “sobre nosotros” / modelo de negocio reforzó su decisión. Manifestó que descargaría, usaría y recomendaría la aplicación, y otorgó la máxima probabilidad de recomendación (5 de 5). No cambiaría nada y pidió que la aplicación mantenga un rendimiento ágil en equipos móviles. **Características del arquetipo:** adulto, personal administrativo, usuario frecuente de móvil, orientado a la practicidad, valora la claridad visual y el bajo ruido cromático, prioriza la velocidad y la confianza (respaldada por testimonios), prefiere el español y usa un dispositivo Android de gama media/baja.

Transcripción.

Entrevistador: Ah, buenas tardes. Bueno, vamos a empezar con la entrevista para la validación de la landing page. Bueno, eh, vamos a empezar con la presentación, así.

Entrevistador: Que le preguntaría, ¿cuál es su edad y subdistrito de residencia?

Entrevistado: Buenas tardes, joven. Mi edad es 46 años y mi distrito es Santa.

Entrevistador: Okay. ¿Con qué frecuencia usa el celular para trámites y qué tan cómodo se siente?

Entrevistado: Uso mi celular a diario para trámites y en una escala del 1 al 10 me sentiría en un siete de comodidad.

Entrevistador: Okay. ¿Y cuál es su rol en el establecimiento de salud? Eh, me encargo de la administración.

Entrevistador: Okay. Eh, al abrir la página, ¿qué cree que ofrece Salud?

Entrevistado: Por lo que he podido contemplar, están ofreciendo soluciones al aglomeramiento del público y a disipar, ¿no?, este tema de las citas, que es lo más tedioso que puede tener una persona cuando tiene alguna emergencia médica.

Entrevistado: Y por el lado del personal, o sea, por el lado de la institución de salud, eh, también brinda una agilidad para la atención de los pacientes.

Entrevistador: Okay. Y entonces, el mensaje principal, «citas médicas sin colas», ¿le resulta claro?

Entrevistado: Claro, este, clarísimo. Se refiere a, por lo que yo puedo entender y por lo que me parece que cualquier persona podría entender, es dejar de lado las colas y empezar a ir de una forma más práctica, ¿no?, para así conseguir el resultado de una buena atención.

Entrevistador: ¿Encontró fácilmente las secciones, como por ejemplo los productos, el testimonio, sobre nosotros, los videos?

Entrevistado: Sí, este, me parece una forma ordenada y con una facilidad para poder encontrar lo que uno necesita.

Entrevistador: ¿Usó quizás el selector de idioma para español o inglés, o lo habrá notado en la aplicación, eh, perdón, en la page?

Entrevistado: Hm, mira, este, yo lo abrí y salió en español de frente.

Entrevistador: Okay. Okay. Eh, ¿quizás faltó alguna sección que esperaba encontrar?

Entrevistado: Mira, la verdad es la primera vez que veo una app de este tipo y no, para mí está todo correcto.

Entrevistador: Okay. ¿El diseño le pareció claro y agradable a la vista?

Entrevistado: [Resoplido]

Entrevistado: Personalmente sí, porque he visto otras páginas que tienen diseños como que un poco más subidos de colores, más saturados, ¿no? Y eso como que me madrea un poco y me distrae, pero en el caso de tu página sí está todo bien.

Entrevistador: Okay. Y por ejemplo, el contraste, el tamaño de las letras y los botones, ¿le permitieron leer y utilizar la página con facilidad?

Entrevistado: Sí, estuvieron bien porque yo últimamente tengo ya poco de fallas en la visión, no creo que ya necesito anteojos, pero sí está bien.

Entrevistador: Okay. Y por ejemplo, los botones de acción, eh, como descargar y conocer más, eh, ¿fueron visibles y fáciles de ubicar?

Entrevistado: Sí, claro. Están a la vista. Se ven en la pantalla del celular.

Entrevistador: Okay. Entonces, la página le funcionó bien en su celular. Para hacer un resumen.

Entrevistado: Sí, este, a pesar que ahorita estoy con un Android que no es de muy alta gama, sí resultó todo veloz y correcto.

Entrevistador: Okay. ¿Quizás haya visto una parte acerca de los testimonios? Esos testimonios, ¿influyeron en su confianza hacia la aplicación Salud Jack?

Entrevistado: La verdad sí, porque uno siempre se tiene que basar también no solo en lo que uno ve, ¿no?, sino también en las pruebas de cada caso, de cada seguimiento que uno hace.

Entrevistado: Y al haber estos testimonios de personas reales que ya aprobaron, eh, sí, sí, sí, sí influyó.

Entrevistador: Okay. Y la sección de modelo de negocio o «sobre nosotros», ¿aportó algo a su decisión, por ejemplo, elegir nuestra aplicación?

Entrevistado: Claro, este, como te digo, no todo tiene un complemento, todo tiene que ser algo ya general y sí, sí, en medida. Sí.

Entrevistador: Okay. Y entonces, eh, para preguntar acerca de la aplicación directamente, ¿usted la descargaría?

Entrevistado: Sí, joven. Sí, sí. Me parece que es una aplicación muy práctica y sí la descargaría. Sí, la utilizaría y la recomendaría también.

Entrevistador: Claro, claro. Y del uno al cinco, ¿qué tan probable es que recomiende la página?

Entrevistado: Cinco. Cinco.

Entrevistador: Un cinco. Okay. Eh, ¿quizás usted cambiaría algo, agregaría algo para convencerlo de utilizar Salud Ya, o ya se siente realmente convencido?

Entrevistado: Yo no cambiaría nada. Para mí está bien tal y como está.

Entrevistador: Okay. Bueno, para terminar, ¿hay algo más que quiera comentar quizá de la landing page o de la aplicación?

Entrevistado: Este, sí, que de verdad cumpla, ¿no?, con las expectativas que yo tengo, sobre todo, ¿no? Y que sea este, ¿cómo te digo?, que no se vuelva lento, que sea práctico, que mantenga una velocidad adecuada para los equipos celulares.

Entrevistador: Bueno, eso sería todo por la entrevista de hoy. De todas formas, gracias por las respuestas y bueno, eso sería todo.

Entrevistado: Ya, joven, ya gracias por la entrevista. Estamos en contacto.

Entrevista 2 — Segmento: Paciente

Campo	Detalle
Nombre(s) y apellidos	Michelle
Edad	20 años
Distrito	Punta Hermosa
Ocupación / rol	Paciente

Campo	Detalle
Entrevistador(a)	Neo
Enlace del video	https://drive.google.com/file/d/19-pnt9joLZRH_xgGoZjRtWLqU26qJOTC/view?usp=sharing
Inicio de la entrevista	00:00

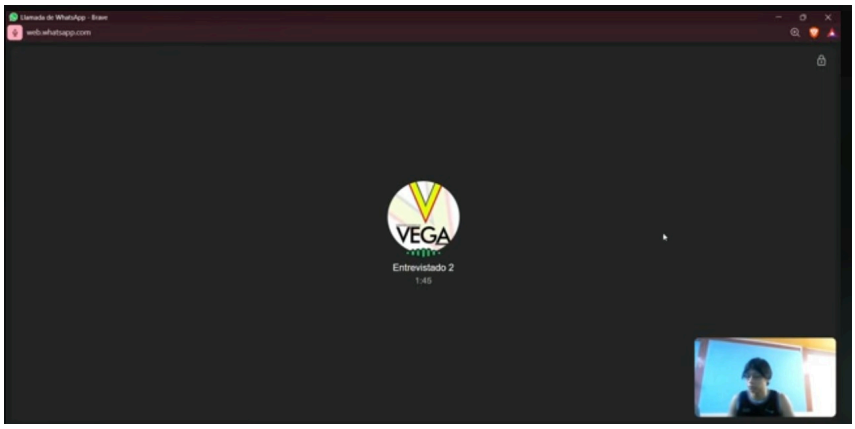


Figura 212. Cuadro de video de la entrevista 2 — paciente.

Resumen. Michelle es una joven de 20 años, residente en Punta Hermosa y usuaria del servicio como paciente. Usa el celular con frecuencia para realizar trámites. Interpretó la propuesta de valor de SaludYa como una forma de agilizar las citas y ahorrar tiempo, entendió que está dirigida tanto a pacientes como al personal de salud, y le resultó claro el mensaje “Citas médicas sin colas”. Consideró la navegación intuitiva y la organización de la información lógica y ordenada, sin echar en falta secciones, y usó la página en español. Sobre el diseño, lo percibió claro e intuitivo, con un contraste, tamaños de letra y botones adecuados, y sin problemas en la versión móvil. Los testimonios influyeron en su confianza y la sección “sobre nosotros” / modelo de negocio le generó confianza. Indicó que descargaría la aplicación por la confianza y los testimonios, no cambiaría nada por considerarla completa, y su probabilidad de recomendación fue de 4 a 5. **Características del arquetipo:** adulta joven (20 años), paciente, usuaria frecuente de móvil, valora la confianza y las recomendaciones de otras personas, prefiere el español y prioriza la rapidez y la simplicidad.

Transcripción.

Entrevistador: Buenos días. Hoy estamos con Michelle para la entrevista de validación de la Landing Page. Muy buenos días, Michelle. Gracias por participar.

Michelle: Buenos días, con gusto.

Entrevistador: Bueno, vamos a empezar con el perfil del participante. ¿Cuál es su edad y su distrito de residencia?

Michelle: Tengo 20 años y vivo en Punta Hermosa.

Entrevistador: ¿Con qué frecuencia usa el celular para trámites? ¿Qué tan cómodo se siente?

Michelle: Lo ocupo frecuentemente, pero no es tan común.

Entrevistador: ¿Y cuál es su rol en el establecimiento de salud?

Michelle: Eh… no, no pertenezco al establecimiento de salud. Soy paciente.

Entrevistador: Perfecto, gracias por la aclaración. Ahora, comprensión de la propuesta de valor. Al abrir la página, ¿qué cree que ofrece SaludYa? Explíquelo con sus palabras.

Michelle: Agilizar las citas y ahorrar tiempo.

Entrevistador: ¿Entendió para quién está dirigida la solución, es decir, pacientes y personal de salud?

Michelle: Sí.

Entrevistador: ¿El mensaje principal, «Citas médicas sin colas…», le resulta claro?

Michelle: Sí.

Entrevistador: ¿Identificó qué problema resuelve y cómo lo resuelve?

Michelle: Sí, para agilizar el tiempo.

Entrevistador: Muy bien. Pasamos a navegación y arquitectura de la información. ¿Encontró fácilmente las secciones: Producto, Videos, Testimonios, Sobre nosotros, Descarga?

Michelle: Sí, es intuitiva.

Entrevistador: ¿La organización de la información le pareció lógica y ordenada?

Michelle: Sí.

Entrevistador: ¿Usó el selector de idioma, español o inglés? ¿Le resultó útil?

Michelle: Idioma español.

Entrevistador: ¿Faltó alguna sección que esperaba encontrar?

Michelle: No.

Entrevistador: Muy bien. Ahora, diseño visual, usabilidad y diseño inclusivo. ¿El diseño le pareció claro y agradable?

Michelle: Sí, es intuitivo.

Entrevistador: ¿El contraste, el tamaño de las letras y los botones le permitieron leer y usar la página con facilidad?

Michelle: Sí.

Entrevistador: ¿Los botones de acción, como descargar y conocer más, son visibles y fáciles de ubicar?

Michelle: Sí.

Entrevistador: ¿La página funcionó bien en su celular, en la versión móvil?

Michelle: Sí, no he tenido ningún problema.

Entrevistador: Perfecto. Ahora, contenido. ¿Los testimonios influyeron en su confianza hacia SaludYa?

Michelle: Sí.

Entrevistador: ¿La sección de modelo de negocio o «sobre nosotros» aportó algo a su decisión?

Michelle: Sí, me da confianza.

Entrevistador: Muy bien. Conversión e intención. ¿Descargaría la aplicación? ¿Por qué sí o por qué no?

Michelle: Sí, porque me genera confianza y por los testimonios que había escuchado.

Entrevistador: ¿Qué cambiaría o agregaría para convencerlo de usar SaludYa?

Michelle: Pues creo que está muy completa.

Entrevistador: Del 1 al 5, ¿qué tan probable es que recomiende la página a otra persona?

Michelle: 4, quizá 5.

Entrevistador: Muy bien. Para cerrar, ¿hay algo más que quiera comentar?

Michelle: No.

Entrevistador: Bueno, eso sería todo por la entrevista de hoy. Muchas gracias por las respuestas, Michelle. Eso sería todo.

Michelle: Gracias a ustedes. Estamos en contacto.

4.3.3. Evaluaciones según heurísticas

En esta sección se presentan las evaluaciones de la experiencia de usuario de **SaludYa** aplicando el formato de **Evaluación de User Experience según Heurísticas** (heurísticas de usabilidad, arquitectura de información y diseño inclusivo). Las evaluaciones fueron realizadas por el equipo **RouteGuard** y se organizan en dos bloques: (1) la aplicación Android para pacientes y (2) la aplicación Flutter para el personal de admisión y la administración del establecimiento. A continuación se registran los hallazgos, su severidad y las recomendaciones para cada problema identificado.

Evaluación 1 — Aplicación Android (paciente)

UX Heuristics & Principles Evaluation — Usability · Inclusive Design · Information Architecture

Tabla 86

Datos de la evaluación

Campo	Valor
CARRERA	Ingeniería de Software
CURSO	1ACC0238 Aplicaciones para Dispositivos Móviles
NRC	4945
PROFESORES	Mayta Guillermo, Jorge Luis
AUDITOR	RouteGuard (Marcelo Pareja, Mathias de la Cruz, Manuel Francia, Nickolas Ramirez)
CLIENTE(S)	Paula Montoya y Neo Ramos (grupo auditado: RuwaLabs)

Nota. Elaboración del equipo RouteGuard; la evaluación se realizó sobre los mock-ups de SaludYa.

SITE o APP A EVALUAR

SaludYa — aplicación móvil Android para pacientes. La evaluación cubre los mock-ups de las secciones **01 · Registro e inicio de sesión**, **02 · Perfil y menores**, **03 · Inicio y reservas**, **04 · Check-in y atención** y **05 · Configuración** del archivo de Figma.

TAREAS A EVALUAR

El alcance de esta evaluación incluye la revisión de la usabilidad de las siguientes tareas:

1. Registro de un usuario nuevo (pasos 1 a 3 y verificación de correo)
2. Verificación de identidad con DNI durante el registro
3. Inicio de sesión de un paciente
4. Recuperación de contraseña por enlace al correo
5. Solicitud de ayuda cuando se perdió el acceso al correo
6. Consulta del perfil y de los datos personales del paciente
7. Vinculación de un menor y verificación de su identidad
8. Consulta de la lista y el detalle de menores vinculados

9. Consulta de citas pendientes desde el inicio
10. Consulta del detalle de una cita y cancelación de una reserva
11. Registro de llegada escaneando el QR del establecimiento
12. Consulta de la posición en la cola y del ticket digital
13. Atención de la llamada al consultorio y salida de la cola
14. Registro de llegada de un menor y consulta de su ticket
15. Recuperación de errores de llegada (QR no válido, fuera de plazo, ventana no habilitada, inasistencia)
16. Cambio de celular y de correo con verificación por código
17. Consulta del detalle de un menor, sincronización de datos y desvinculación
18. Uso de la vista del representado («Hola, Mateo»)
19. Filtrado del historial de citas por fecha
20. Inicio de una reserva: elegir especialidad, beneficiario y fecha
21. Cierre de sesión

No están incluidas en esta versión de la evaluación las siguientes tareas:

1. Registro de personal por el Super Admin
2. Selección de horario y confirmación final de la reserva
3. Notificaciones y reasignación (sección 06)
4. Recuperación de contraseña por código (pantallas ocultas en el prototipo)
5. Cambio de contraseña desde Configuración (la pantalla no está en el prototipo)
6. Sección 09 · Variantes interactivas y estados
7. Flujos de Personal y de Administración (ver Evaluación 2)

Alcance de la evaluación. La evaluación integra los hallazgos de las secciones 01 a 05. Los problemas de las secciones 04 y 05 que ya estaban cubiertos por los de las secciones 01 a 03 no se repiten.

ESCALA DE SEVERIDAD

Los errores se puntúan tomando en cuenta la siguiente escala de severidad:

Nivel	Descripción
1	**Problema superficial:** puede ser fácilmente superado por el usuario y ocurre con muy poca frecuencia. No necesita ser arreglado a no ser que exista disponibilidad de tiempo.
2	**Problema menor:** puede ocurrir un poco más frecuentemente o es un poco más difícil de superar para el usuario. Se le debería asignar una prioridad baja de cara al siguiente release.
3	**Problema mayor:** ocurre frecuentemente o los usuarios no son capaces de resolverlo. Es importante que sea corregido y se le debe asignar una prioridad alta.
4	**Problema muy grave:** un error de gran impacto que impide al usuario continuar con el uso de la herramienta. Es imperativo que sea corregido antes del lanzamiento.

Tabla 87

TABLA RESUMEN de la evaluación heurística

#	Problema	Escala de severidad	Heurística / Principio violado(a)
--	--	--	--
1	El registro indica «PASO 3 DE 3» pero aún falta la verificación del correo	2	Usability: Visibilidad del estado del sistema
2	El mensaje «Correo verificado» dice que se validó el número	2	Usability: Consistencia y estándares

#	Problema	Escala de severidad	Heurística / Principio violado(a)
3	El login pide correo pero el mensaje de error habla de documento	3	Usability: Consistencia y estándares
4	Los campos de contraseña no muestran requisitos	3	Usability: Prevención de errores
5	«Reenviar código» es un texto plano sin estado ni tiempo de espera	2	Usability: Visibilidad del estado del sistema
6	Toda la interfaz está solo en español, sin selector de idioma	2	Inclusive Design: Ofrece opciones
7	Nombres distintos para la misma acción o pantalla («Reservar una cita», «Reservar cita», «Reserva tu cita», «Mis citas»)	1	Information Architecture: Is it usable?
8	El paciente llamado no ve cuánto tiempo le queda para llegar	3	Usability: Visibilidad del estado del sistema
9	«Has sido atendida» usa género fijo y no considera a quien representa a un menor	2	Inclusive Design: Considera la situación
10	El mismo ícono se usa para estados positivos, neutros y negativos	2	Usability: Consistencia y estándares
11	Avisos y errores terminan en «Entendido» sin una acción para recuperarse	2	Usability: Ayudar a reconocer, diagnosticar y recuperarse de errores
12	Los campos de solo lectura se ven como campos editables	2	Usability: Consistencia y estándares
13	La pantalla que confirma el cambio de celular se titula «Verificar correo»	2	Usability: Consistencia entre el sistema y el mundo real
14	«Sincronizar» es un término técnico y una acción manual poco clara	2	Usability: Consistencia entre el sistema y el mundo real
15	El mensaje «Revisa el dato ingresado» no dice qué campo falló	2	Usability: Ayudar a reconocer, diagnosticar y recuperarse de errores
16	Configuración no ofrece idioma, notificaciones ni opciones de accesibilidad	2	Inclusive Design: Ofrece opciones

Nota. Elaboración del equipo RouteGuard sobre los mock-ups de SaludYa.

DESCRIPCIÓN DE PROBLEMAS

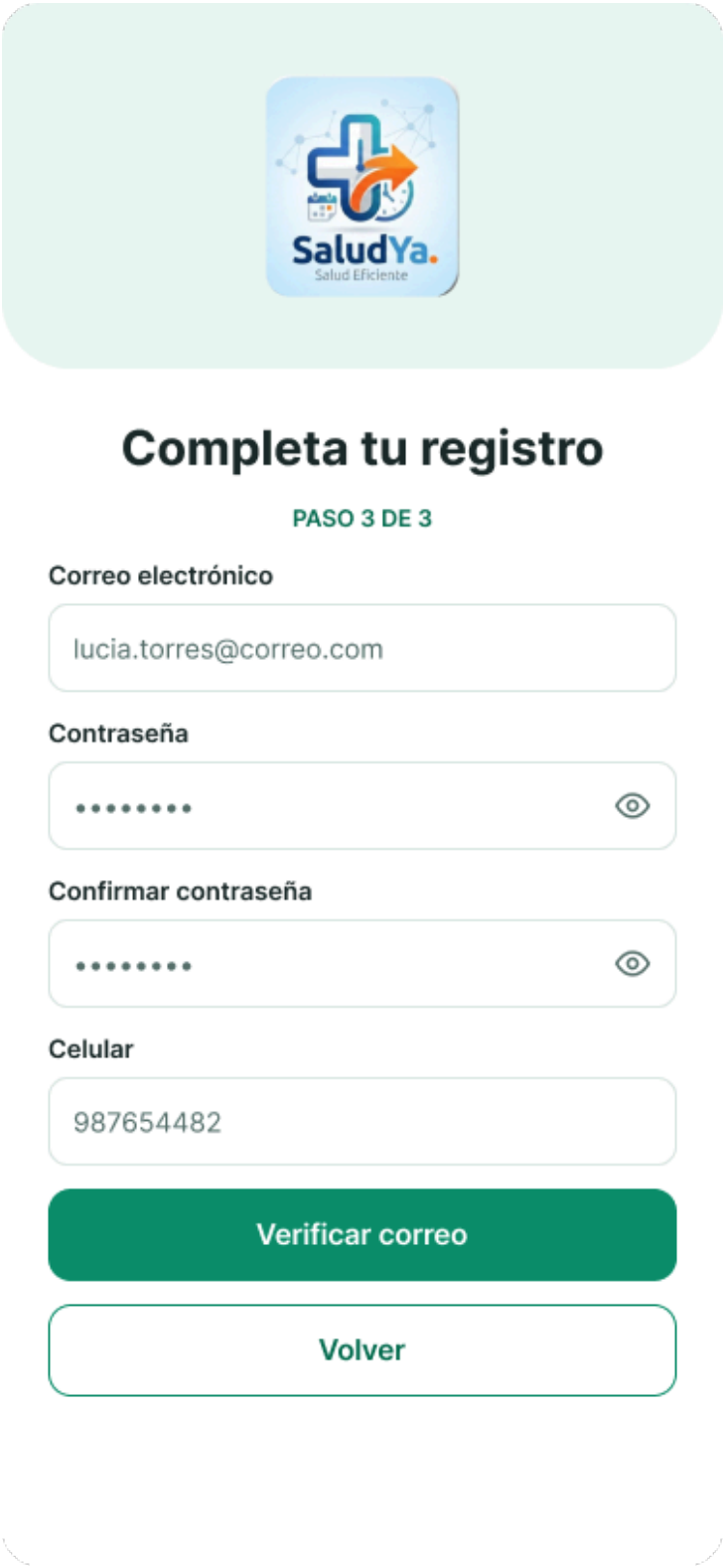
PROBLEMA #1: El registro indica «PASO 3 DE 3» pero aún falta la verificación del correo

Severidad: 2 **Heurística violada:** Usabilidad - Visibilidad del estado del sistema

Problema: La pantalla «03 · Registro · Datos de acceso» muestra «PASO 3 DE 3», pero al pulsar «Verificar correo» el usuario llega a la pantalla «04 · Registro · Verificar correo», que no tiene indicador de paso. El usuario cree que terminó y encuentra un paso más, por lo que no sabe cuánto falta para crear su cuenta.

Figura 213

Registro - Datos de acceso



Nota. Captura del mock-up de SaludYa evaluada por el equipo RouteGuard.

Figura 214

Registro - Verificar correo

Nota. Captura del mock-up de SaludYa evaluada por el equipo RouteGuard.

Recomendación: Pasar a «PASO 4 DE 4» o mostrar que la verificación del correo forma parte del paso 3, y mantener el indicador en la pantalla de verificación.

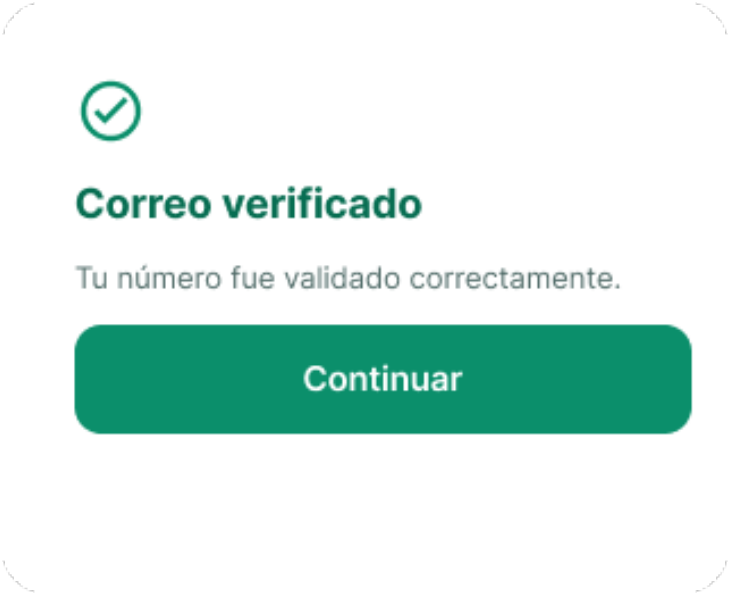
PROBLEMA #2: El mensaje «Correo verificado» dice que se validó el número

Severidad: 2 **Heurística violada:** Usabilidad - Consistencia y estándares

Problema: El diálogo «Registro · Correo validado» tiene el título «Correo verificado», pero el texto indica «Tu número fue validado correctamente». El usuario no sabe si se verificó su correo o su celular, y el mensaje puede hacerle pensar que debe verificar otro dato.

Figura 215

Diálogo correo verificado



Nota. Captura del mock-up de SaludYa evaluada por el equipo RouteGuard.

Recomendación: Cambiar el texto a «Tu correo fue validado correctamente» para que coincida con el título y con el paso realizado.

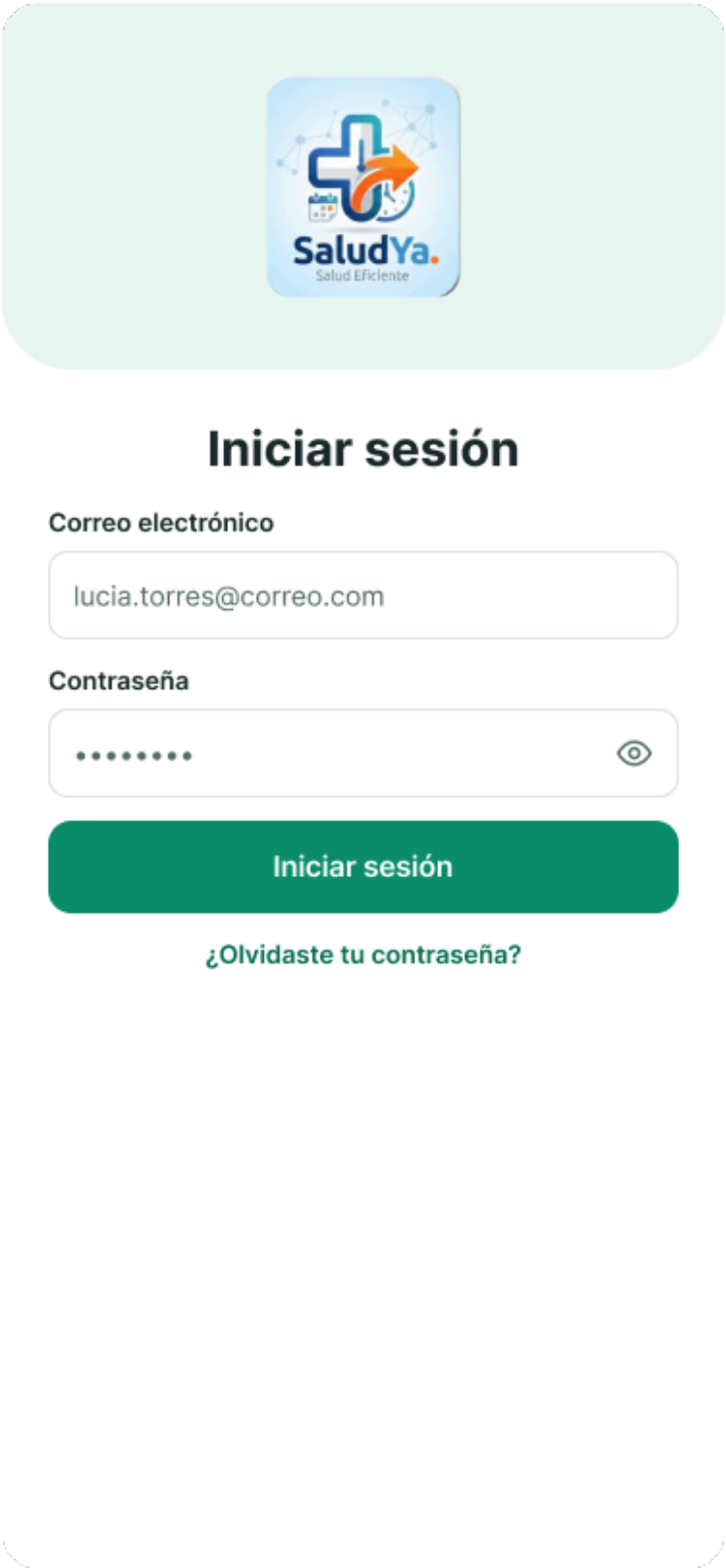
PROBLEMA #3: El login pide correo pero el mensaje de error habla de documento

Severidad: 3 **Heurística violada:** Usabilidad - Consistencia y estándares

Problema: La pantalla «05 · Login · Paciente» solicita correo electrónico y contraseña, pero el diálogo «Credenciales incorrectas» dice «El documento o la contraseña son incorrectos». Además, el registro empieza con el DNI, por lo que el usuario no tiene claro con qué dato debe ingresar. Esto genera reintentos fallidos y frustración.

Figura 216

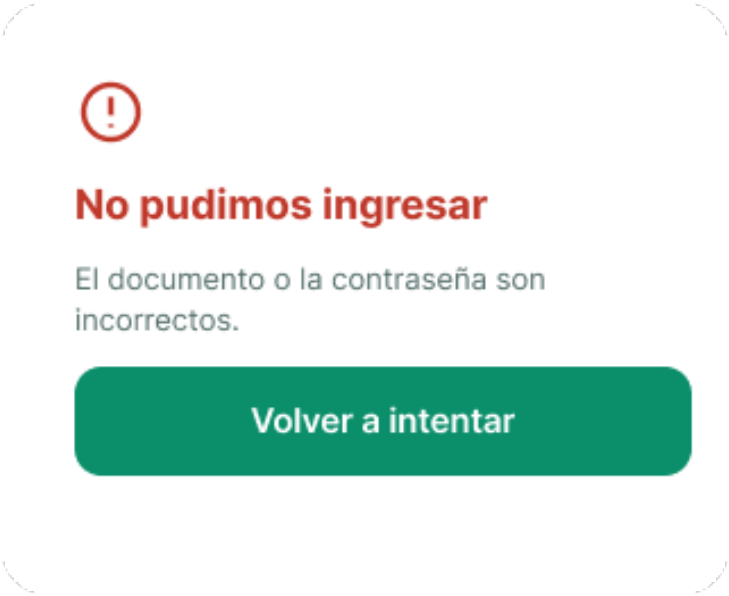
Login paciente



Nota. Captura del mock-up de SaludYa evaluada por el equipo RouteGuard.

Figura 217

Diálogo credenciales incorrectas



Nota. Captura del mock-up de SaludYa evaluada por el equipo RouteGuard.

Recomendación: Unificar el identificador de acceso (correo o DNI) en el login y en sus mensajes, e indicar en el registro con qué dato se ingresará después.

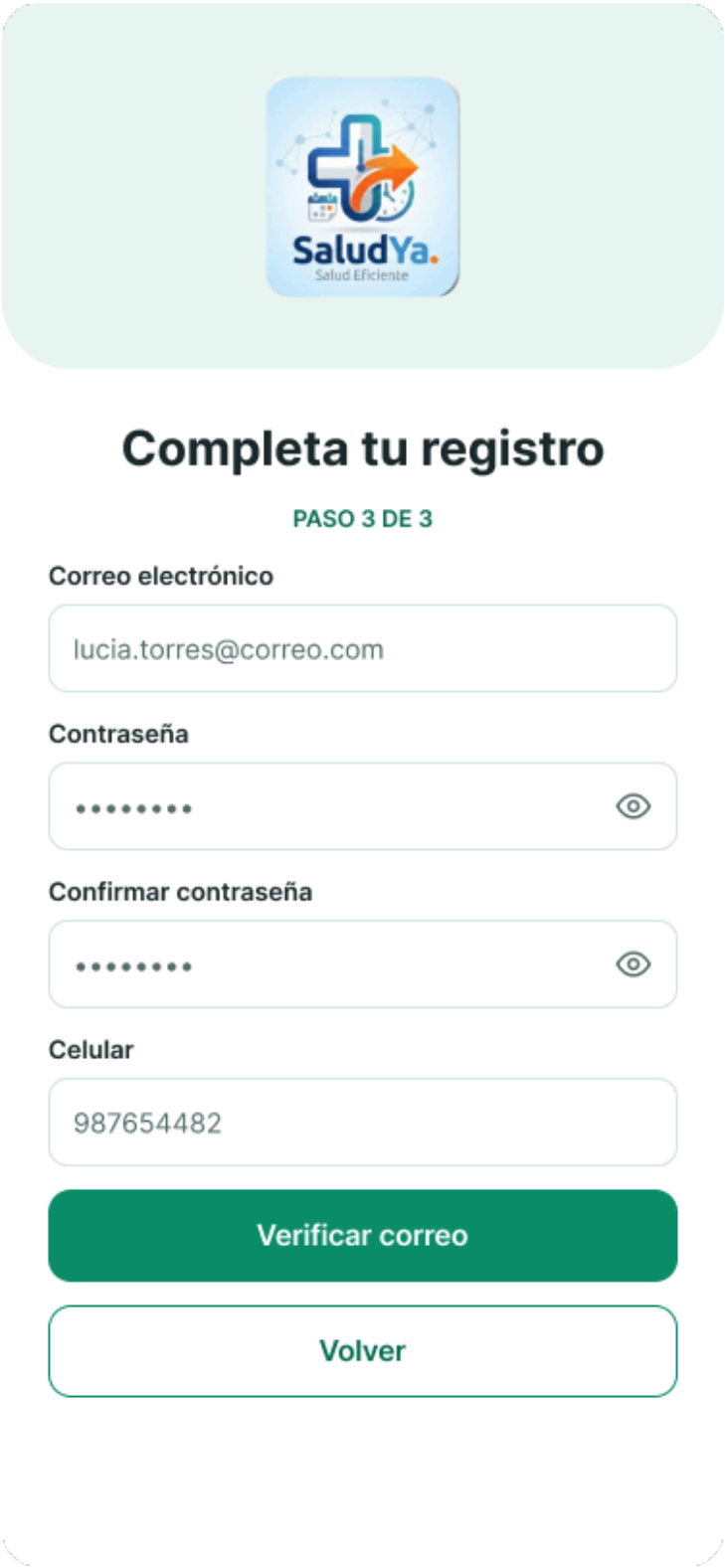
PROBLEMA #4: Los campos de contraseña no muestran requisitos

Severidad: 3 **Heurística violada:** Usabilidad - Prevención de errores

Problema: Las pantallas «03 · Registro · Datos de acceso» y «10 · Recuperación · Nueva contraseña» piden crear una contraseña sin indicar longitud mínima, caracteres requeridos ni nivel de seguridad. El usuario descubre las reglas solo al fallar, y el único aviso previsto es el diálogo de contraseñas diferentes.

Figura 218

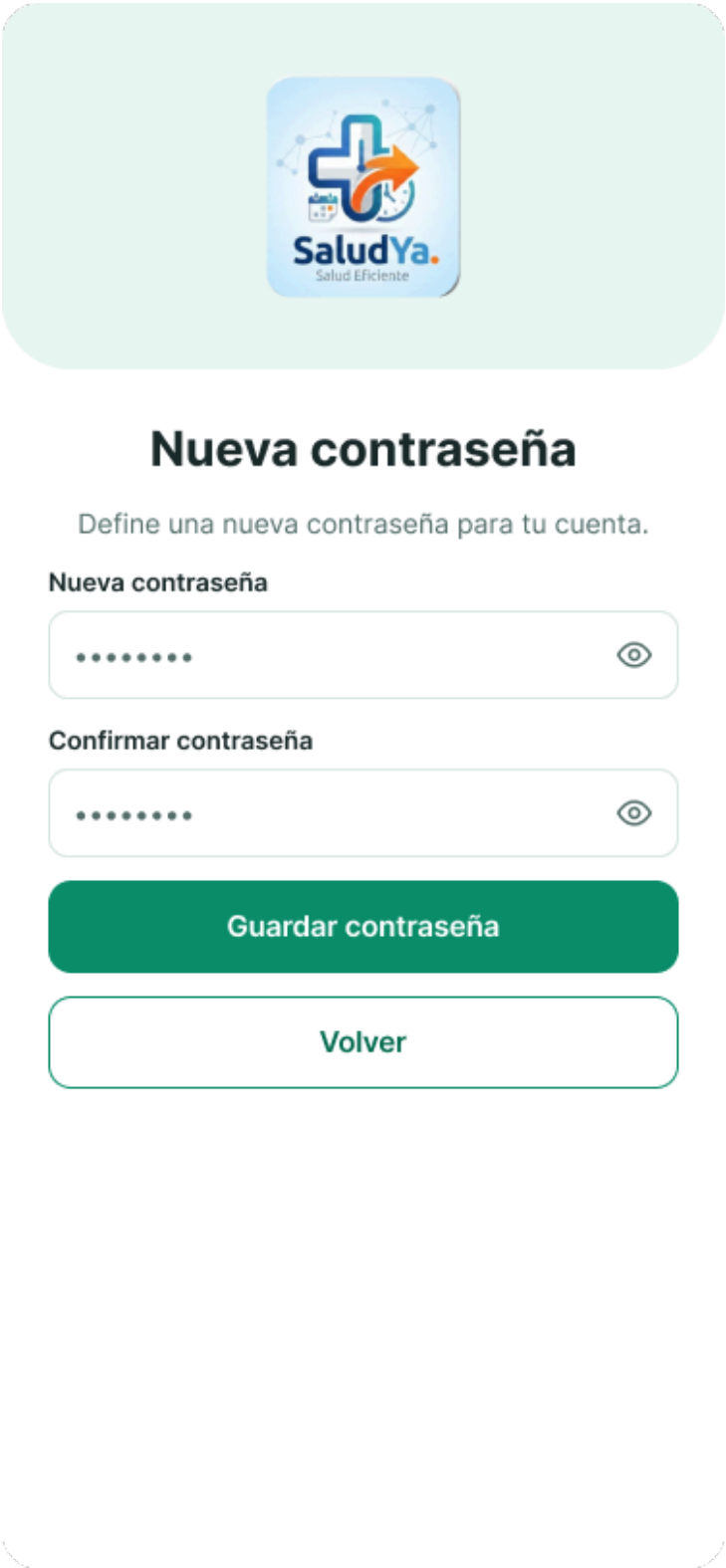
Registro - Datos de acceso



Nota. Captura del mock-up de SaludYa evaluada por el equipo RouteGuard.

Figura 219

Recuperación - Nueva contraseña



Nota. Captura del mock-up de SaludYa evaluada por el equipo RouteGuard.

Recomendación: Mostrar bajo el campo los requisitos de la contraseña y un indicador de seguridad, y validar mientras el usuario escribe.

PROBLEMA #5: «Reenviar código» es un texto plano sin estado ni tiempo de espera

Severidad: 2 **Heurística violada:** Usabilidad - Visibilidad del estado del sistema

Problema: En «04 · Registro · Verificar correo» la acción «Reenviar código» es un texto de 18 px de alto que no indica si ya se envió, cuánto falta para poder reenviarlo ni la vigencia del código. El usuario no sabe si debe esperar o pulsarlo de nuevo.

Figura 220

Registro - Verificar correo

Nota. Captura del mock-up de SaludYa evaluada por el equipo RouteGuard.

Recomendación: Mostrar un contador («Reenviar en 00:30»), confirmar el envío con un aviso y presentar la acción como un botón con área táctil adecuada.

PROBLEMA #6: Toda la interfaz está solo en español, sin selector de idioma

Severidad: 2 **Heurística violada:** Inclusive Design - Ofrece opciones

Problema: En las pantallas revisadas todos los textos están en español y no hay una opción visible para cambiar de idioma. El proyecto requiere internacionalización (i18n) con inglés (en_US) y español latino-americano (es_419), y el idioma por defecto de la interfaz debe ser el inglés.

Figura 221

Dashboard inicio

Nota. Captura del mock-up de SaludYa evaluada por el equipo RouteGuard.

Recomendación: Incluir en «Configuración» un selector de idioma (English / Español), definir inglés como idioma por defecto y preparar las cadenas para traducción.

PROBLEMA #7: Nombres distintos para la misma acción o pantalla («Reservar una cita», «Reservar cita», «Reserva tu cita», «Mis citas»)

Severidad: 1 **Heurística violada:** Information Architecture - Is it usable?

Problema: La acción de reservar aparece como «Reservar una cita» (botón de inicio), «Reservar cita» (navegación y estado vacío) y «Reserva tu cita» (título). La lista de citas se llama «Citas pendientes» y en la pantalla de error «Mis citas». Estas variaciones pueden hacer dudar si se trata de la misma función.

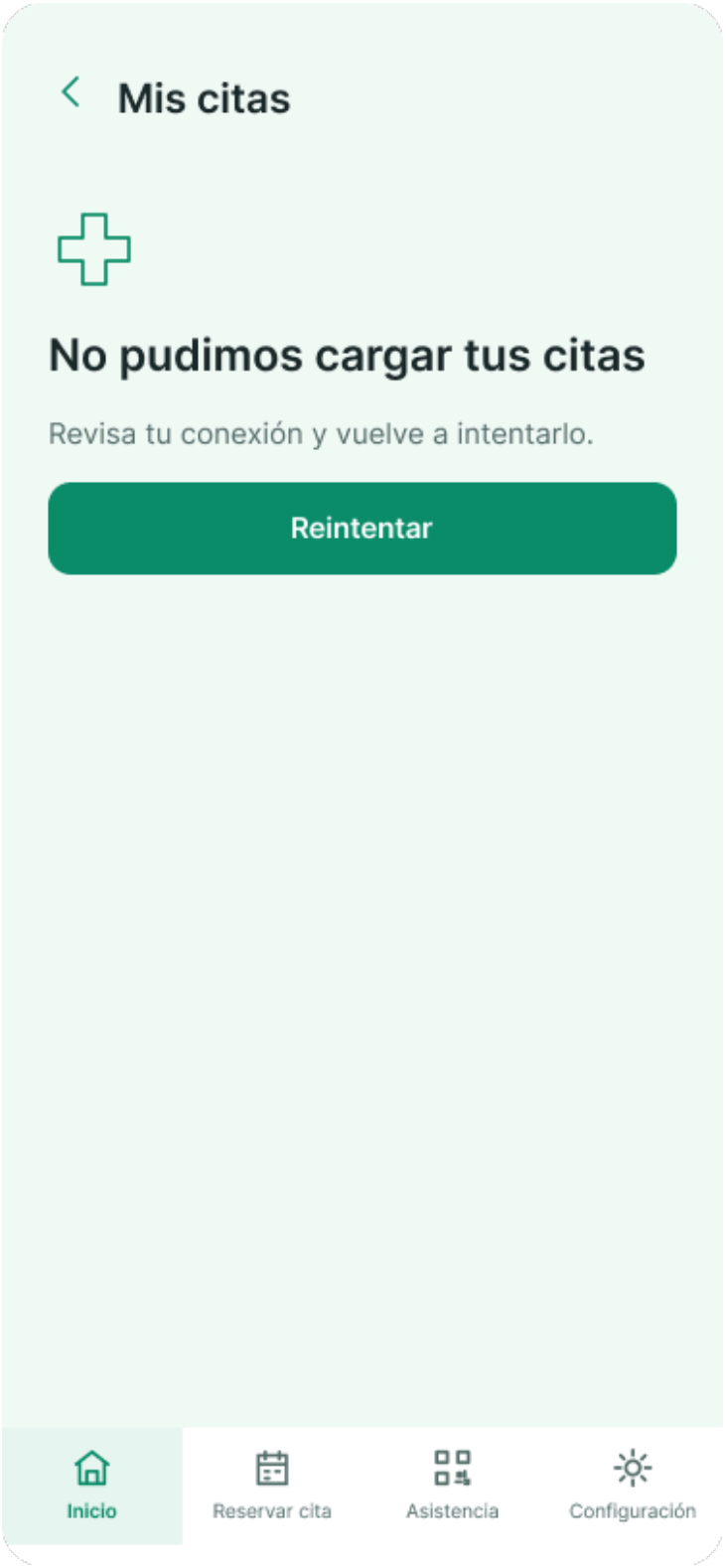
Figura 222

Dashboard inicio

Nota. Captura del mock-up de SaludYa evaluada por el equipo RouteGuard.

Figura 223

Error de carga



Nota. Captura del mock-up de SaludYa evaluada por el equipo RouteGuard.

Figura 224

Reserva - Especialidades

Nota. Captura del mock-up de SaludYa evaluada por el equipo RouteGuard.

Recomendación: Definir un sistema de etiquetas y aplicar el mismo nombre en botones, navegación y títulos.

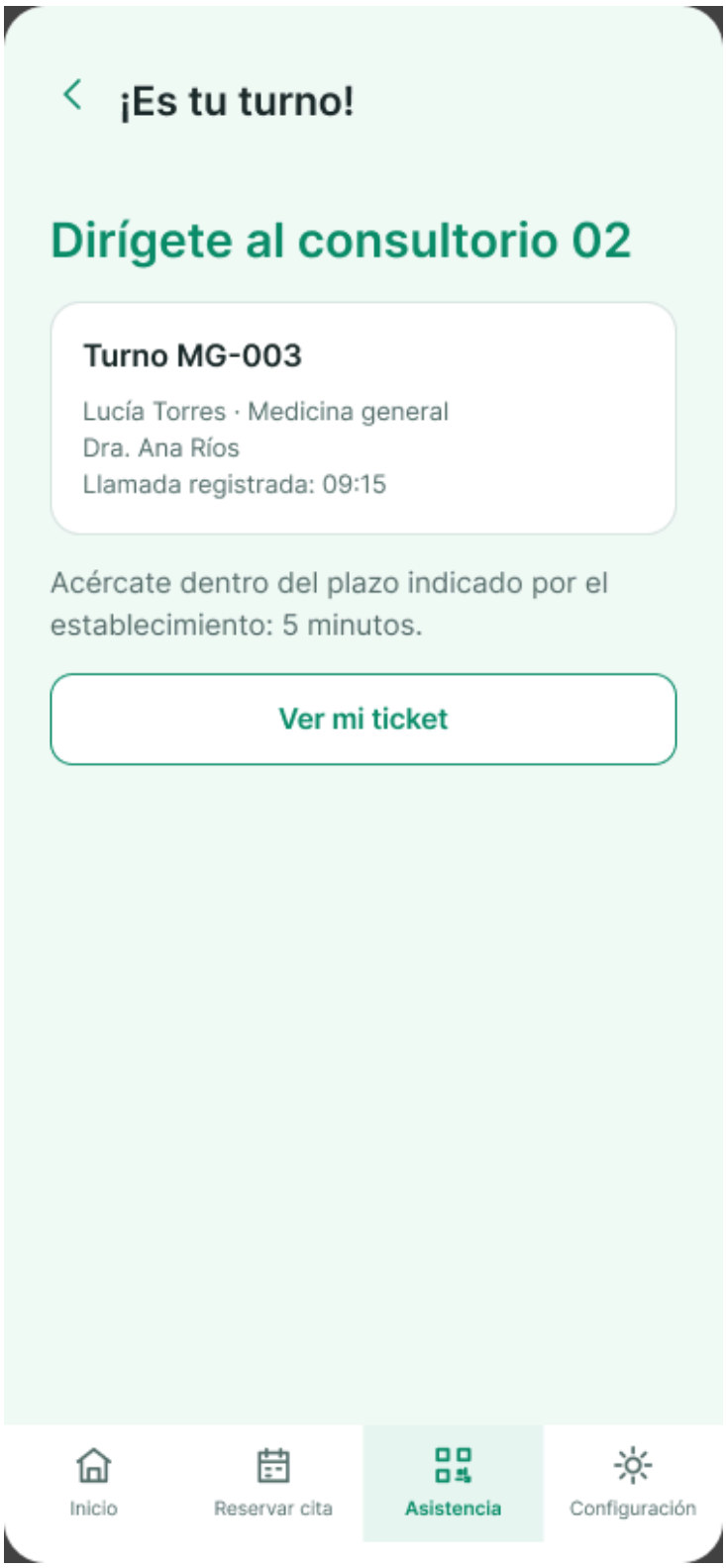
PROBLEMA #8: El paciente llamado no ve cuánto tiempo le queda para llegar

Severidad: 3 **Heurística violada:** Usabilidad - Visibilidad del estado del sistema

Problema: En «A05 · Llamada a consultorio» el mensaje dice «Acércate dentro del plazo indicado por el establecimiento: 5 minutos» y solo muestra la hora de la llamada («Llamada registrada: 09:15»). No hay contador regresivo ni hora límite, aunque el personal sí ve «Quedan 02:30» en su pantalla. Si el paciente no llega a tiempo, «A11 · Fuera del plazo» le avisa recién cuando la cita ya quedó como inasistencia. El paciente puede perder su turno sin haber visto cuánto tiempo tenía.

Figura 225

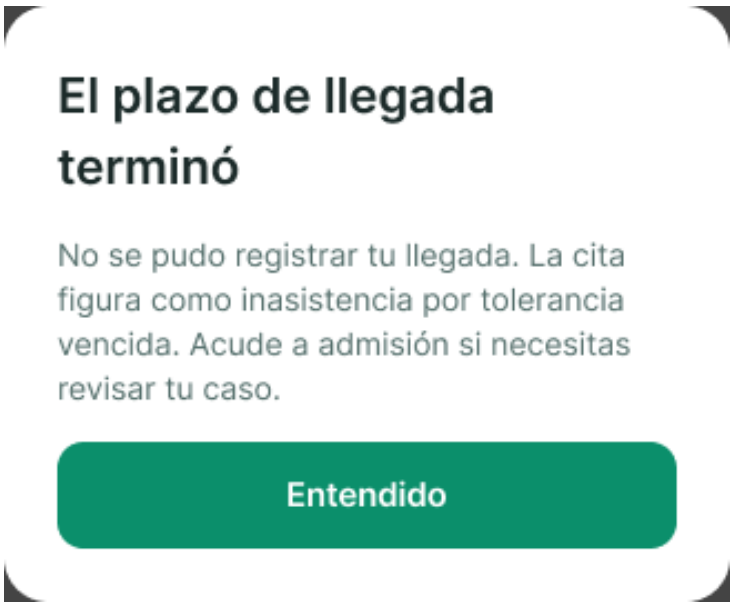
A05 - Llamada a consultorio



Nota. Captura del mock-up de SaludYa evaluada por el equipo RouteGuard.

Figura 226

A11 - Fuera del plazo



Nota. Captura del mock-up de SaludYa evaluada por el equipo RouteGuard.

Recomendación: Mostrar en A05 un contador regresivo y la hora límite («Llega antes de las 09:20»), enviar una notificación al llamar y otra cuando falten 2 minutos. En A11 explicar cómo recuperar el turno.

PROBLEMA #9: «Has sido atendida» usa género fijo y no considera a quien representa a un menor

Severidad: 2 **Heurística violada:** Diseño inclusivo - Considera la situación

Problema: «A07 · Atención finalizada» dice «Has sido atendida», con género femenino fijo. La app la usan hombres y personas de otros géneros, y además un adulto puede ver los tickets de un menor (A15 · Ticket de menor, que es de Mateo). El mensaje no se adapta a ninguno de esos casos.

Figura 227

A07 - Atención finalizada

Nota. Captura del mock-up de SaludYa evaluada por el equipo RouteGuard.

Figura 228

A15 - Ticket de menor

Nota. Captura del mock-up de SaludYa evaluada por el equipo RouteGuard.

Recomendación: Usar un texto neutro («Tu atención finalizó») o personalizarlo según el beneficiario («La atención de Mateo finalizó»).

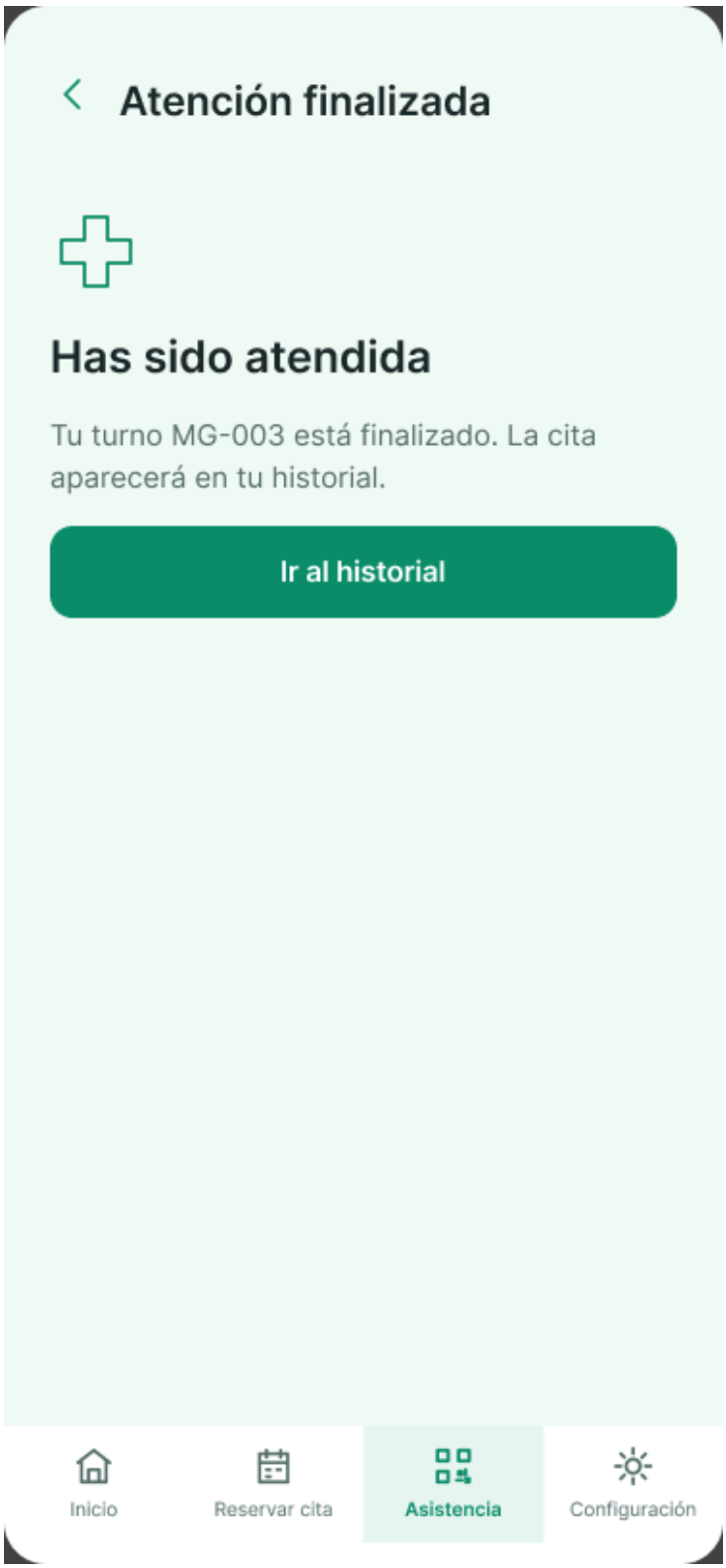
PROBLEMA #10: El mismo ícono se usa para estados positivos, neutros y negativos

Severidad: 2 **Heurística violada:** Usabilidad - Consistencia y estándares

Problema: La cruz médica verde aparece igual en «A03 · Salida registrada», «A06 · Sin llegada registrada», «A07 · Atención finalizada», «A16 · Turno ausente» y «C12 · Sin menores». Un resultado positivo (atendida) y uno negativo (inasistencia) se ven iguales, y el usuario debe leer el texto para saber qué pasó. En los diálogos de error se usa otro estilo (alerta roja), por lo que el criterio tampoco es uniforme.

Figura 229

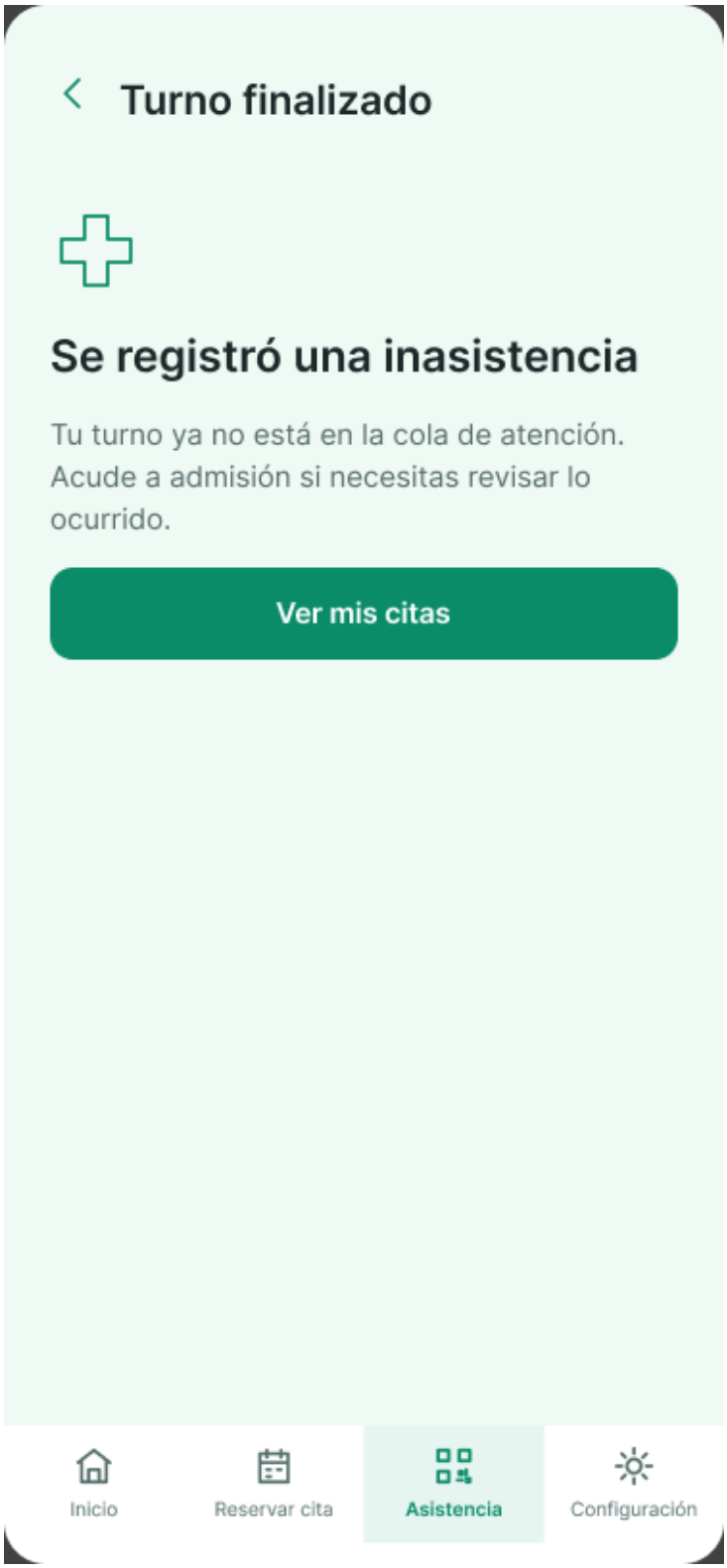
A07 - Atención finalizada



Nota. Captura del mock-up de SaludYa evaluada por el equipo RouteGuard.

Figura 230

A16 - Turno ausente



Nota. Captura del mock-up de SaludYa evaluada por el equipo RouteGuard.

Recomendación: Usar un ícono distinto por tipo de estado (check para éxito, reloj para espera, alerta para problema) y combinar ícono, color y texto.

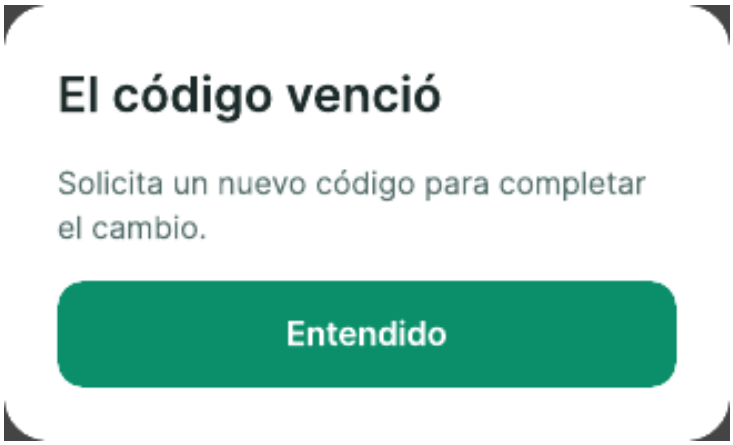
PROBLEMA #11: Avisos y errores terminan en «Entendido» sin una acción para recuperarse

Severidad: 2 **Heurística violada:** Usabilidad - Ayudar a reconocer, diagnosticar y recuperarse de errores

Problema: «C15 · Código vencido» dice «Solicita un nuevo código para completar el cambio», pero su único botón es «Entendido», así que el usuario debe cerrar el diálogo y buscar «Reenviar código». «C18 · Vínculo no acreditado» remite a «admisión» sin ningún canal de contacto. «A11 · Fuera del plazo» y «A16 · Turno ausente» informan la inasistencia, pero solo ofrecen «Entendido» o «Ver mis citas», sin reprogramar ni contactar. El usuario queda sin siguiente paso.

Figura 231

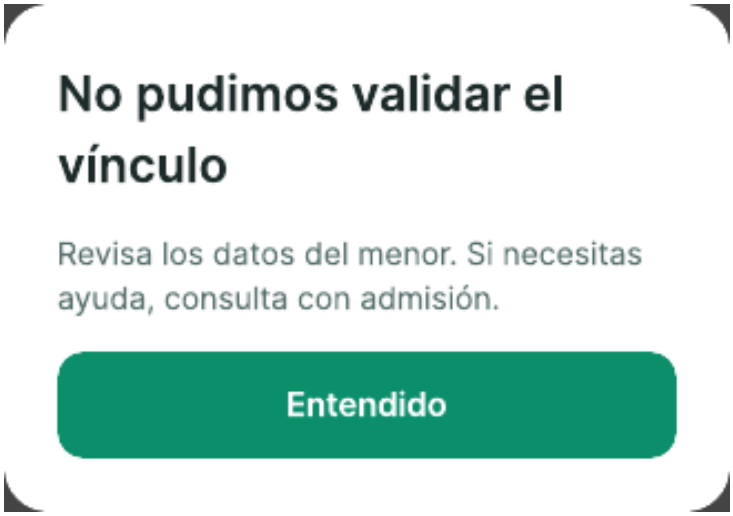
C15 - Código vencido



Nota. Captura del mock-up de SaludYa evaluada por el equipo RouteGuard.

Figura 232

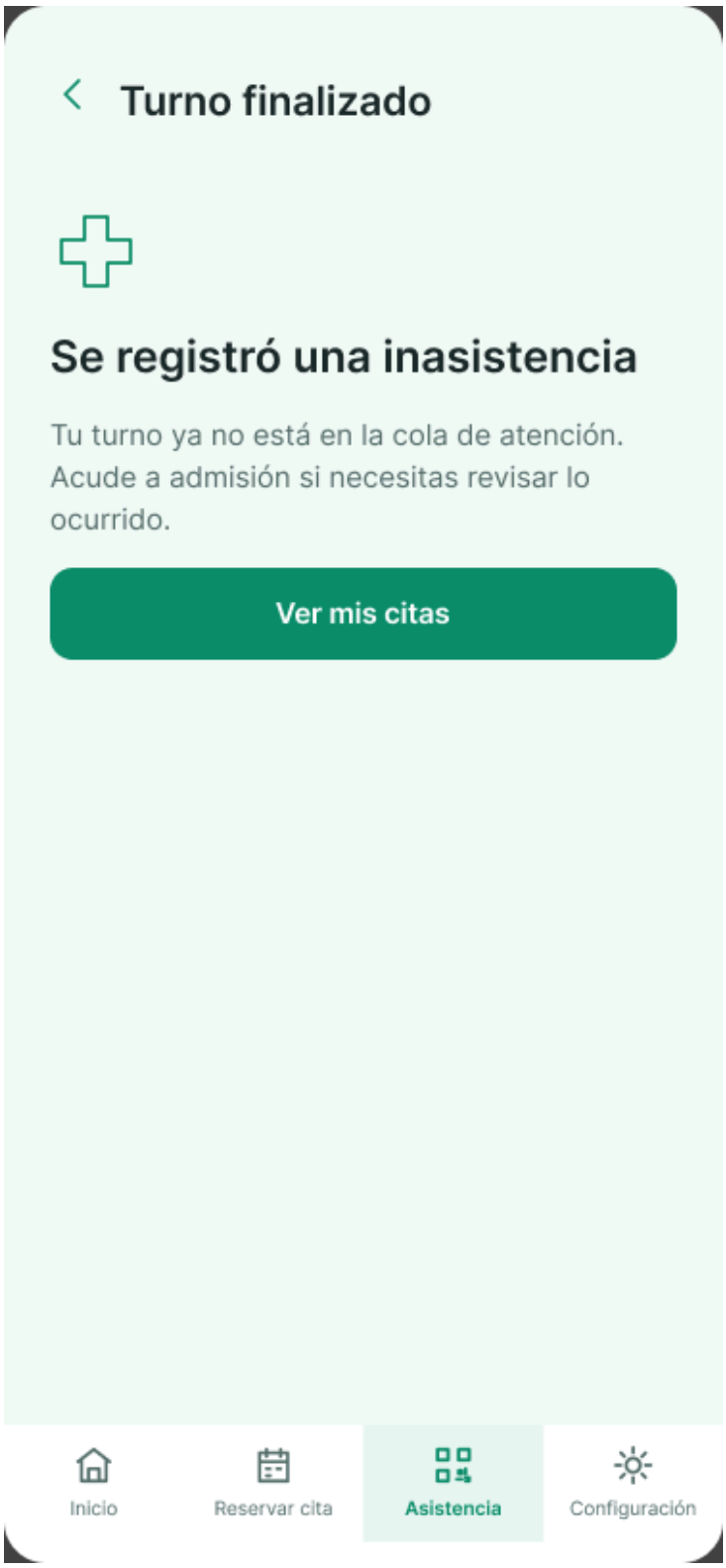
C18 - Vínculo no acreditado



Nota. Captura del mock-up de SaludYa evaluada por el equipo RouteGuard.

Figura 233

A16 - Turno ausente



Nota. Captura del mock-up de SaludYa evaluada por el equipo RouteGuard.

Recomendación: Ofrecer la acción de recuperación en el mismo diálogo («Solicitar nuevo código», «Reservar de nuevo», «Contactar a admisión» con teléfono o dirección) y reservar «Entendido» para avisos que no requieren acción.

PROBLEMA #12: Los campos de solo lectura se ven como campos editables

Severidad: 2 **Heurística violada:** Usabilidad - Consistencia y estándares

Problema: En «C02 · Mis datos», «Nombre» y «Documento» (solo lectura) tienen el mismo cuadro blanco que «Celular» y «Correo», que sí se pueden cambiar. En estos últimos, la única parte accionable es el pequeño texto «Editar →» dentro del cuadro. El usuario toca el nombre y no pasa nada, y puede no notar el enlace «Editar». La nota que lo explica («Los datos de identidad verificados se muestran para consulta») está al final y en letra pequeña.

Figura 234

C02 - Mis datos

Nota. Captura del mock-up de SaludYa evaluada por el equipo RouteGuard.

Recomendación: Mostrar los datos de solo lectura como texto (sin cuadro) o con estilo deshabilitado y un ícono de candado. Hacer que todo el campo editable sea tocable y con un ícono de lápiz visible.

PROBLEMA #13: La pantalla que confirma el cambio de celular se titula «Verificar correo»

Severidad: 2 **Heurística violada:** Usabilidad - Consistencia entre el sistema y el mundo real

Problema: «C05 · Confirmar cambio por correo» se titula «Verificar correo», pero su texto dice «…enviado a tu correo registrado para confirmar el cambio de celular». El usuario que quiere cambiar el celular lee que debe «verificar el correo» y puede creer que cambió algo del correo. «C06» usa el mismo título para verificar realmente un correo nuevo, así que dos tareas distintas comparten pantalla de aspecto idéntico.

Figura 235

C05 - Confirmar cambio por correo

Nota. Captura del mock-up de SaludYa evaluada por el equipo RouteGuard.

Figura 236

C06 - Verificar correo

Nota. Captura del mock-up de SaludYa evaluada por el equipo RouteGuard.

Recomendación: Titular cada pantalla según la tarea («Confirmar cambio de celular» y «Verificar tu nuevo correo») y repetir el dato que se está cambiando.

PROBLEMA #14: «Sincronizar» es un término técnico y una acción manual poco clara

Severidad: 2 **Heurística violada:** Usabilidad - Consistencia entre el sistema y el mundo real

Problema: «C07 · Parientes vinculados» ofrece «Sincronizar vínculos» y «C08 · Detalle del menor» ofrece «Sincronizar datos». No se explica qué se sincroniza, con qué ni cuándo conviene hacerlo. Una persona no técnica no sabe si debe pulsarlo, y la sincronización debería ser automática. El resultado («C13 · Vínculos sincronizados») tampoco aclara qué cambió.

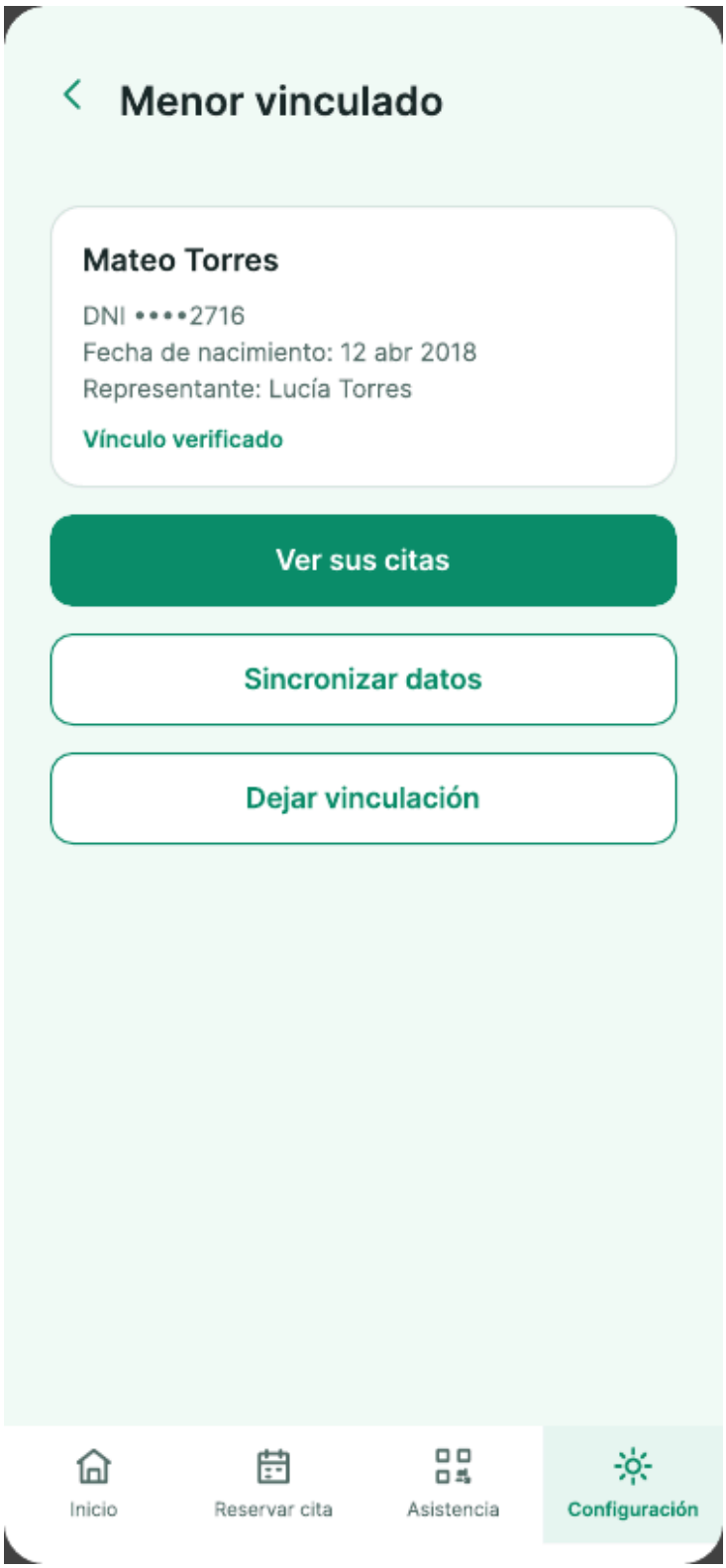
Figura 237

C07 - Parientes vinculados

Nota. Captura del mock-up de SaludYa evaluada por el equipo RouteGuard.

Figura 238

C08 - Detalle del menor



Nota. Captura del mock-up de SaludYa evaluada por el equipo RouteGuard.

Recomendación: Reemplazarlo por una actualización automática o por un texto comprensible («Actualizar datos de Mateo») con una línea que explique para qué sirve, y mostrar la fecha de la última actualización.

PROBLEMA #15: El mensaje «Revisa el dato ingresado» no dice qué campo falló

Severidad: 2 **Heurística violada:** Usabilidad - Ayudar a reconocer, diagnosticar y recuperarse de errores

Problema: «C17 · Dato inválido» dice «Ingresa un correo válido o un celular de 9 dígitos, según el campo». El mismo mensaje sirve para dos pantallas distintas y obliga al usuario a deducir cuál es el suyo. Además, aparece en un diálogo que tapa el campo con error.

Figura 239

C17 - Dato inválido

Nota. Captura del mock-up de SaludYa evaluada por el equipo RouteGuard.

Recomendación: Mostrar el error junto al campo con un mensaje específico («El celular debe tener 9 dígitos» o «Ingresa un correo válido, por ejemplo nombre@correo.com») y validar mientras se escribe.

PROBLEMA #16: Configuración no ofrece idioma, notificaciones ni opciones de accesibilidad

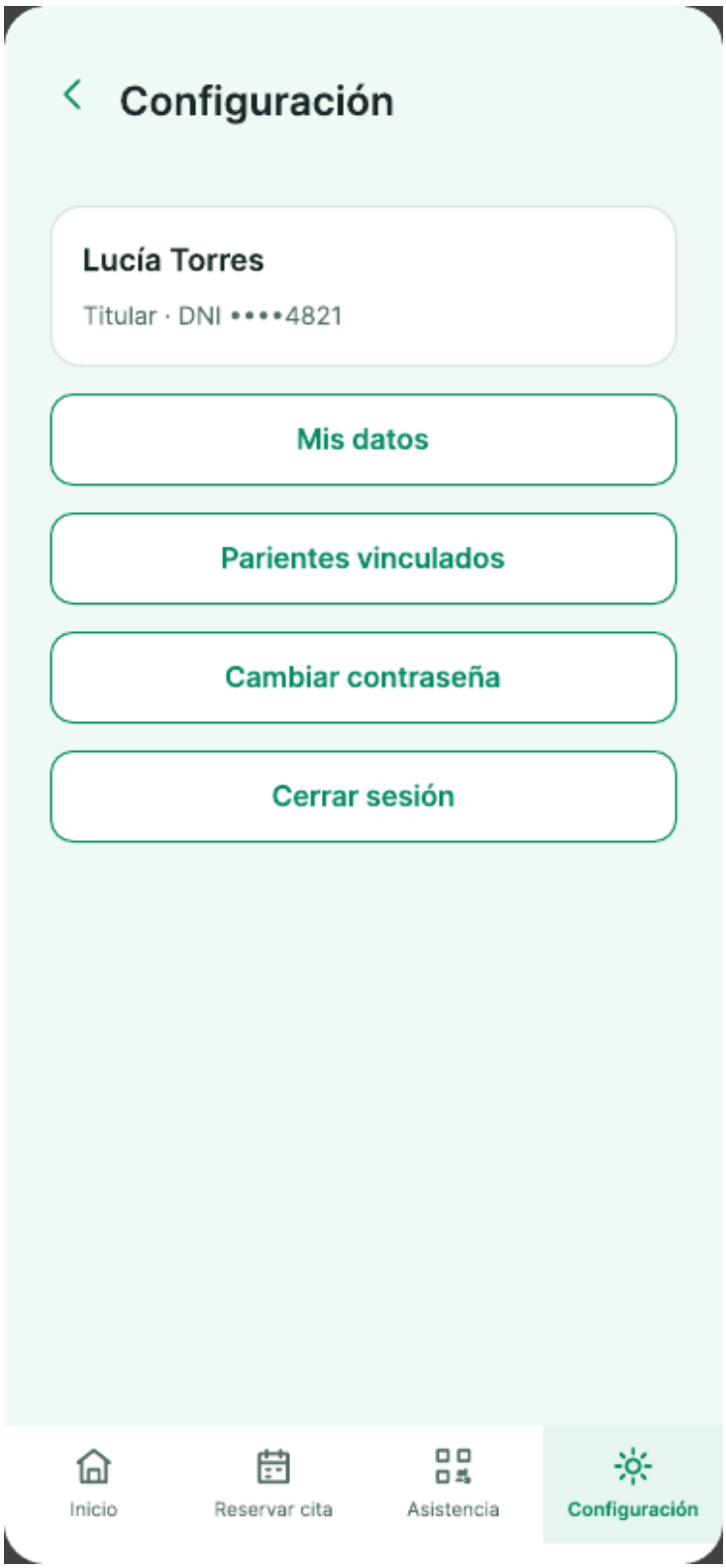
Severidad: 2 **Heurística violada:** Diseño inclusivo - Ofrece opciones

Problema: «C01 · Configuración» solo tiene cuatro botones iguales: «Mis datos», «Parientes vinculados», «Cambiar contraseña» y «Cerrar sesión». No hay selector de idioma (el proyecto exige español e inglés), ajustes de notificaciones ni opciones de accesibilidad como tamaño de texto o contraste. «Cerrar sesión» tiene el mismo peso visual que las funciones de navegación.

Relación con otros hallazgos: el selector de idioma ya se reportó en el problema #6; este problema amplía el hallazgo a las opciones de notificación y accesibilidad.

Figura 240

C01 - Configuración



Nota. Captura del mock-up de SaludYa evaluada por el equipo RouteGuard.

Recomendación: Agregar un selector de idioma (inglés por defecto), preferencias de notificación y tamaño de texto, y separar «Cerrar sesión» del resto como acción distinta.

Evaluación 2 — Aplicación Flutter (personal y administración)

UX Heuristics & Principles Evaluation — Usability · Inclusive Design · Information Architecture

Tabla 88

Datos de la evaluación

Campo	Valor
CARRERA	Ingeniería de Software
CURSO	1ACC0238 Aplicaciones para Dispositivos Móviles
NRC	4945
PROFESORES	Mayta Guillermo, Jorge Luis
AUDITOR	RouteGuard (Marcelo Pareja, Mathias de la Cruz, Manuel Francia, Nickolas Ramirez)
CLIENTE(S)	Paula Montoya y Neo Ramos (grupo auditado: RuwaLabs)

Nota. Elaboración del equipo RouteGuard; la evaluación se realizó sobre los mock-ups de SaludYa.

SITE o APP A EVALUAR

SaludYa — aplicación Flutter para el personal de admisión y la administración del establecimiento. La evaluación cubre los mock-ups de las secciones **07 · Citas y atención · Personal** y **08 · Configuración e indicadores · Administración** del archivo de Figma.

TAREAS A EVALUAR

El alcance de esta evaluación incluye la revisión de la usabilidad de las siguientes tareas:

1. Consultar el inicio del personal (citas pendientes y canceladas)
2. Consultar el calendario operativo y los bloques del día
3. Consultar, editar y guardar un bloque de atención
4. Elegir especialidad, profesional y horario
5. Gestionar la cola presencial (llamar, iniciar y terminar atención)
6. Marcar a un paciente como ausente
7. Registrar la llegada de un paciente por QR o por código manual
8. Consultar citas canceladas y pacientes con reserva
9. Consultar y editar las reglas del establecimiento (capacidad, tolerancia, plazos, intervalos)
10. Consultar indicadores diarios y demanda por especialidad
11. Exportar un reporte por rango de fechas

No están incluidas en esta versión de la evaluación las siguientes tareas:

1. Flujos del paciente (ver Evaluación 1)
2. Sección 09 · Variantes interactivas y estados
3. Gestión de cuentas del personal (registro por el Super Admin)

Alcance de la evaluación. La evaluación se concentra en los problemas propios de las pantallas de personal y administración.

ESCALA DE SEVERIDAD

Los errores se puntúan tomando en cuenta la siguiente escala de severidad:

Nivel	Descripción
1	**Problema superficial:** puede ser fácilmente superado por el usuario y ocurre con muy poca frecuencia. No necesita ser arreglado a no ser que exista disponibilidad de tiempo.
2	**Problema menor:** puede ocurrir un poco más frecuentemente o es un poco más difícil de superar para el usuario. Se le debería asignar una prioridad baja de cara al siguiente release.

Nivel	Descripción
3	**Problema mayor:** ocurre frecuentemente o los usuarios no son capaces de resolverlo. Es importante que sea corregido y se le debe asignar una prioridad alta.
4	**Problema muy grave:** un error de gran impacto que impide al usuario continuar con el uso de la herramienta. Es imperativo que sea corregido antes del lanzamiento.

Tabla 89

TABLA RESUMEN de la evaluación heurística

#	Problema	Escala de severidad	Heurística / Principio violado(a)
1	Los estados de bloques, turnos y pacientes se distinguen solo por texto verde	2	Inclusive Design: Proporciona experiencias comparables
2	Editar un bloque no muestra qué reservas se verán afectadas	3	Usability: Prevención de errores
3	Elegir profesional y horario no indica la tarea ni la selección actual	2	Usability: Visibilidad del estado del sistema
4	«Terminar atención» se ejecuta sin confirmación ni opción de deshacer	2	Usability: Libertad y control del usuario
5	«Marcar ausente» es el botón más llamativo y su confirmación usa otro color	2	Usability: Prevención de errores
6	La pestaña «Cita» con ícono de QR se confunde con «Calendario»	2	Information Architecture: Is it findable?
7	Las reglas del establecimiento están repartidas en tres lugares	2	Information Architecture: Is it findable?
8	«Prioridad de reasignación: Por especialidad» contradice la nota que la explica	2	Usability: Consistencia entre el sistema y el mundo real
9	El mensaje de validación de capacidad mezcla reglas que no corresponden	2	Usability: Ayudar a reconocer, diagnosticar y recuperarse de errores
10	Los indicadores no muestran un control para cambiar la fecha o el periodo	2	Usability: Visibilidad del estado del sistema
11	Descargar un reporte requiere dos pasos y no indica formato ni archivo	2	Usability: Eficiencia de uso

Nota. Elaboración del equipo RouteGuard sobre los mock-ups de SaludYa.

DESCRIPCIÓN DE PROBLEMAS

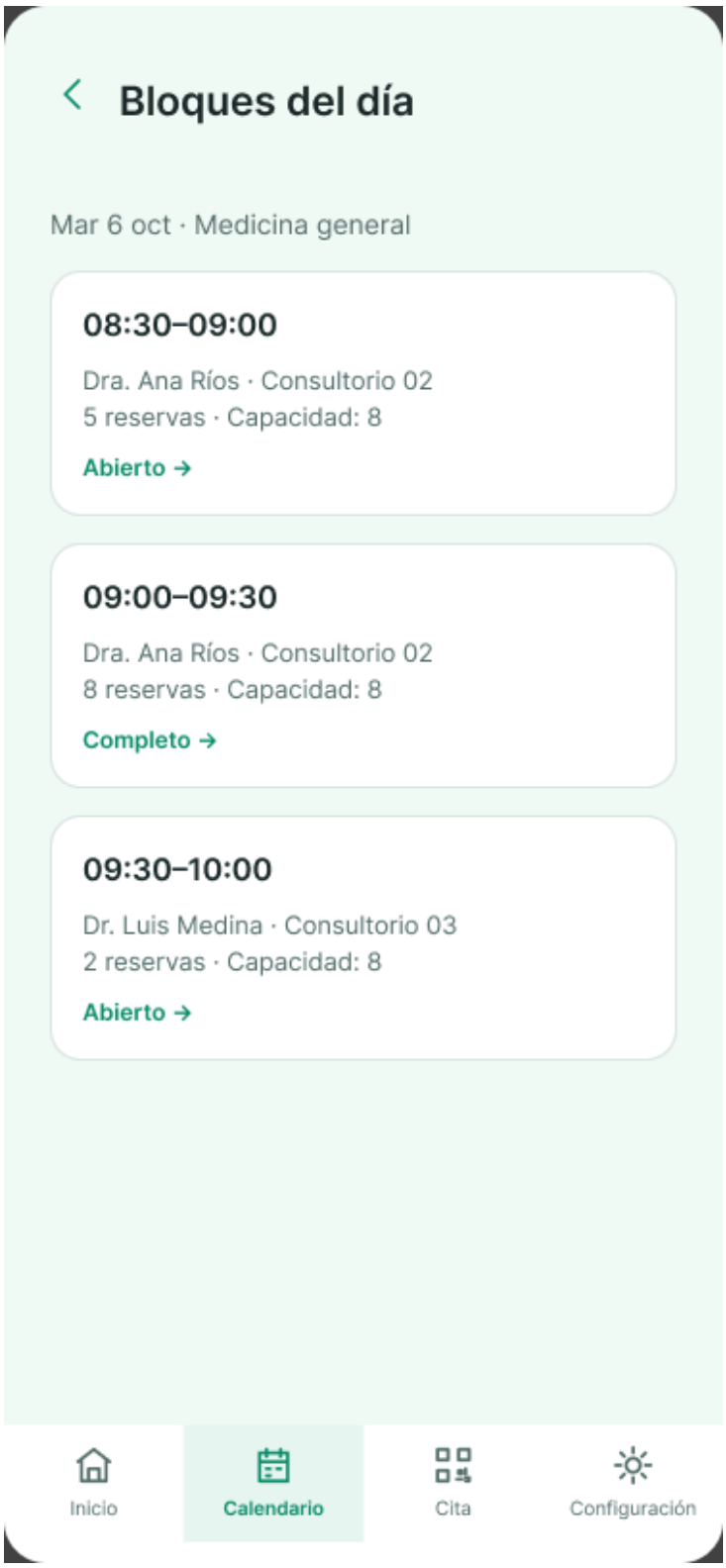
PROBLEMA #1: Los estados de bloques, turnos y pacientes se distinguen solo por texto verde

Severidad: 2 **Heurística violada:** Diseño inclusivo - Proporciona experiencias comparables

Problema: En «P04 · Bloques de hoy» los estados «Abierto →» y «Completo →» se muestran con el mismo texto verde pequeño, así que un bloque lleno no se distingue de uno con cupos. Pasa lo mismo en «P11 · Cola presencial» («En atención» y «En espera») y en «P20 · Pacientes con reserva» («Llegada registrada», «Pendiente de llegada» y «En atención»). Para escanear rápido una lista de decenas de ítems hay que leer cada texto, y una persona con daltonismo no tiene otra pista.

Figura 241

P04 - Bloques de hoy



Nota. Captura del mock-up de SaludYa evaluada por el equipo RouteGuard.

Figura 242

P11 - Cola presencial

Nota. Captura del mock-up de SaludYa evaluada por el equipo RouteGuard.

Figura 243

P20 - Pacientes con reserva

Nota. Captura del mock-up de SaludYa evaluada por el equipo RouteGuard.

Recomendación: Usar etiquetas tipo «chip» con ícono, forma y color distintos por estado (por ejemplo, rojo con candado para «Completo», verde para «Abierto») y separar el estado de la flecha de navegación.

PROBLEMA #2: Editar un bloque no muestra qué reservas se verán afectadas

Severidad: 3 **Heurística violada:** Usabilidad - Prevención de errores

Problema: «P05» indica que el bloque 09:00–09:30 tiene «Capacidad: 8 · Reservas: 8». En «P07 · Editar bloque» se puede cambiar la hora, la especialidad, el profesional y el estado, y solo hay «Confirmar cambios» y «Cancelar». No se muestra cuántos pacientes se afectan ni cuáles, y si hay conflicto «P23» solo dice «El cambio entra en conflicto con reservas confirmadas o con otro bloque del profesional», sin indicar cuál. Un cambio mal hecho puede mover o cancelar citas de hasta 8 pacientes sin que el personal lo sepa.

Figura 244

P07 - Editar bloque

Nota. Captura del mock-up de SaludYa evaluada por el equipo RouteGuard.

Figura 245

P23 - Cambio incompatible

Nota. Captura del mock-up de SaludYa evaluada por el equipo RouteGuard.

Recomendación: Mostrar, antes de confirmar, un resumen del impacto («Afecta a 8 reservas») con la lista y las opciones (notificar, reasignar, cancelar). En P23 indicar la reserva o el bloque exacto con el que hay conflicto.

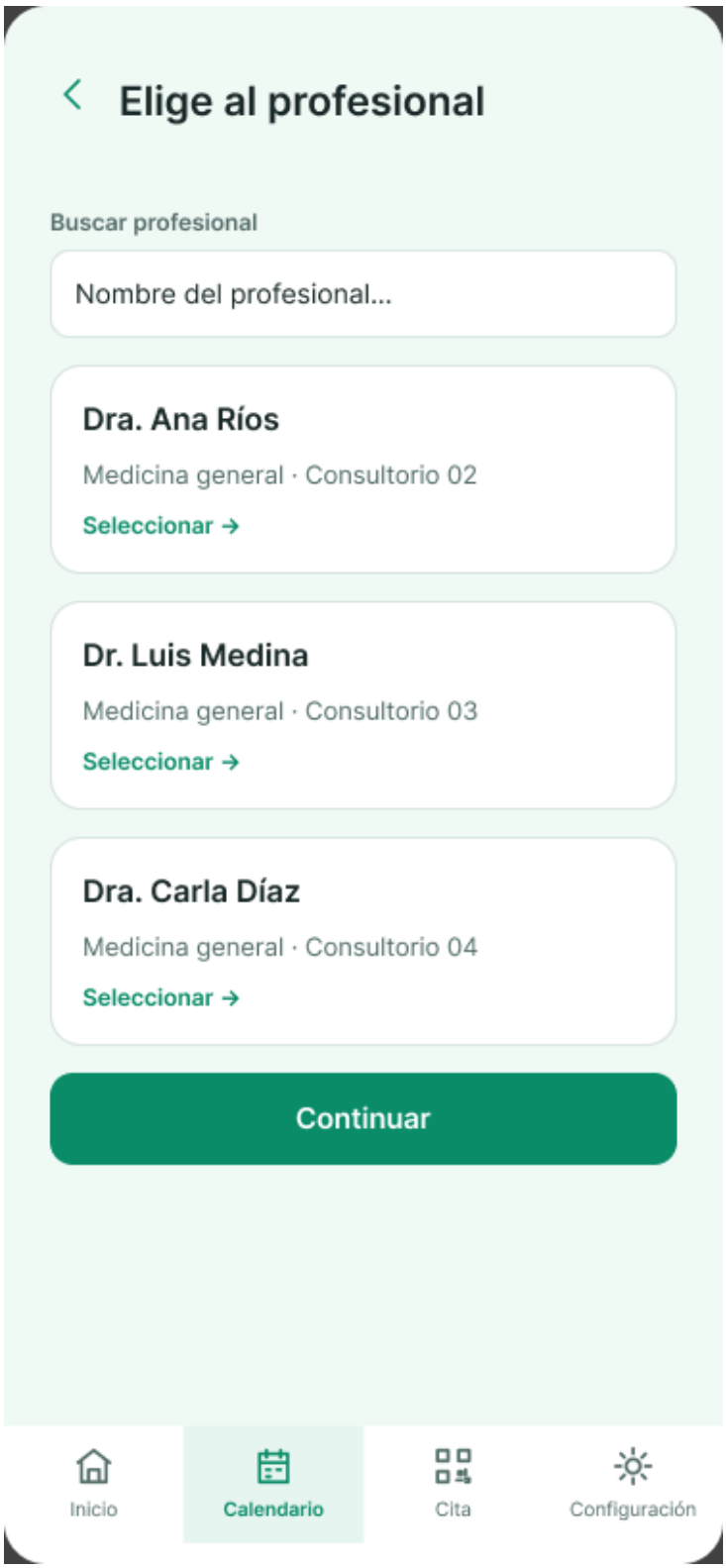
PROBLEMA #3: Elegir profesional y horario no indica la tarea ni la selección actual

Severidad: 2 **Heurística violada:** Usabilidad - Visibilidad del estado del sistema

Problema: «P08 · Elegir profesional», «P09 · Elegir horario» y «P10 · Horario por profesional» no dicen para qué se elige (¿reasignar una cita?, ¿crear una reserva?, ¿consultar?). En P08 cada tarjeta tiene «Seleccionar →» pero ninguna muestra estado seleccionado antes de «Continuar». En P09 y P10 el horario elegido solo se indica con el texto pequeño «· Seleccionado», sin resaltar la tarjeta. Además, P09 y P10 son casi idénticas.

Figura 246

P08 - Elegir profesional



Nota. Captura del mock-up de SaludYa evaluada por el equipo RouteGuard.

Figura 247

P09 - Elegir horario

Nota. Captura del mock-up de SaludYa evaluada por el equipo RouteGuard.

Figura 248

P10 - Horario por profesional

Nota. Captura del mock-up de SaludYa evaluada por el equipo RouteGuard.

Recomendación: Indicar la tarea en el título o subtítulo («Reasignar cita de Lucía Torres»), resaltar la tarjeta elegida (borde, fondo, check) y fusionar P09 y P10 en un solo selector.

PROBLEMA #4: «Terminar atención» se ejecuta sin confirmación ni opción de deshacer

Severidad: 2 **Heurística violada:** Usabilidad - Libertad y control del usuario

Problema: En «P11 · Cola presencial» el botón principal «Terminar atención de N.º 1» cierra la atención y retira al paciente de la cola («P17 · Atención finalizada» aparece después). No hay confirmación previa ni un «Deshacer», y el botón está a solo 14 px de «Llamar al siguiente». Un toque equivocado deja a un paciente fuera de la cola sin forma visible de revertirlo.

Figura 249

P11 - Cola presencial

Nota. Captura del mock-up de SaludYa evaluada por el equipo RouteGuard.

Figura 250

P17 - Atención finalizada

Nota. Captura del mock-up de SaludYa evaluada por el equipo RouteGuard.

Recomendación: Pedir confirmación breve («¿Terminar la atención de María Pérez?») o mostrar un aviso con «Deshacer» durante unos segundos, y separar mejor los dos botones.

PROBLEMA #5: «Marcar ausente» es el botón más llamativo y su confirmación usa otro color

Severidad: 2 **Heurística violada:** Usabilidad - Prevención de errores

Problema: En «P13 · Plazo de llamada vencido» el botón rojo relleno «Marcar ausente» está arriba y es el más prominente, mientras «Iniciar atención» (la opción que puede ser correcta si el paciente llegó) es un botón de borde. Luego, en «P16 · Confirmar ausencia» el botón «Confirmar ausencia» es verde, el color de las acciones positivas, aunque es una acción que libera el cupo y marca inasistencia. El color no es coherente y la confirmación se ve igual que cualquier otra.

Figura 251

P13 - Plazo de llamada vencido

Nota. Captura del mock-up de SaludYa evaluada por el equipo RouteGuard.

Figura 252

P16 - Confirmar ausencia

Nota. Captura del mock-up de SaludYa evaluada por el equipo RouteGuard.

Recomendación: Dar prioridad visual a «Iniciar atención» y dejar «Marcar ausente» como acción secundaria de advertencia. En P16 usar el mismo color de advertencia en «Confirmar ausencia» y dejar «Volver» como opción segura destacada.

PROBLEMA #6: La pestaña «Cita» con ícono de QR se confunde con «Calendario»

Severidad: 2 **Heurística violada:** Arquitectura de información - Is it findable?

Problema: La barra inferior del personal tiene «Inicio», «Calendario», «Cita» y «Configuración». «Calendario» y «Cita» se refieren casi a lo mismo (citas), pero «Cita» abre la cola de atención y el registro de llegada, y su ícono es un QR. El ícono de «Configuración» es un sol. Quien busca la cola de atención no sabe si está en «Calendario» o en «Cita».

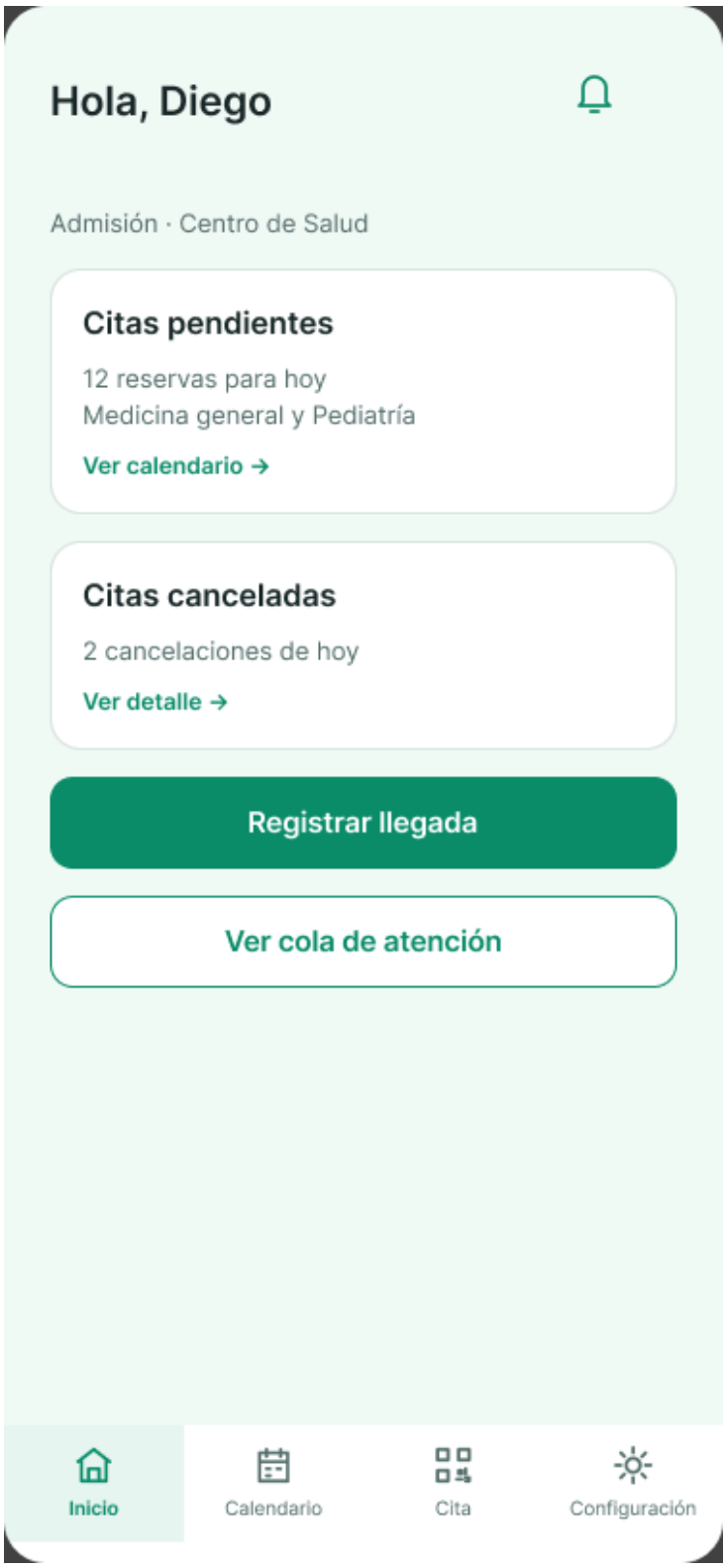
Figura 253

P11 - Cola presencial

Nota. Captura del mock-up de SaludYa evaluada por el equipo RouteGuard.

Figura 254

P01 - Inicio del personal



Nota. Captura del mock-up de SaludYa evaluada por el equipo RouteGuard.

Recomendación: Renombrar según la tarea («Cola» o «Atención»), usar un ícono acorde (personas en fila) y cambiar el sol por un engranaje.

PROBLEMA #7: Las reglas del establecimiento están repartidas en tres lugares

Severidad: 2 **Heurística violada:** Arquitectura de información - Is it findable?

Problema: «H01 · Configuración general» muestra cuatro reglas (capacidad, tolerancia de llegada, plazo después de la llamada, respuesta a reasignación) y un botón «Ver más reglas». «H02 · Más reglas» muestra otras cuatro (hora límite para reservar, anticipación para cancelar, prioridad de reasignación, posición de cola visible) y un botón aparte «Intervalos de atención» que abre «H13». Todas son reglas del mismo tipo, pero están en tres niveles. Para ver o cambiar una regla hay que recordar en cuál se encuentra.

Figura 255

H01 - Configuración general

Nota. Captura del mock-up de SaludYa evaluada por el equipo RouteGuard.

Figura 256

H02 - Reglas adicionales

Nota. Captura del mock-up de SaludYa evaluada por el equipo RouteGuard.

Recomendación: Reunir todas las reglas en una sola lista agrupada por tema (Reservas, Llegada y atención, Reasignación) con buscador o secciones, y dejar la edición como destino de cada fila.

PROBLEMA #8: «Prioridad de reasignación: Por especialidad» contradice la nota que la explica

Severidad: 2 **Heurística violada:** Usabilidad - Consistencia entre el sistema y el mundo real

Problema: En «H02» la tarjeta indica «Prioridad de reasignación: Por especialidad», pero la nota inferior dice «La prioridad de reasignación sigue el orden de reserva. La atención sigue el orden de llegada». No se entiende cuál es el criterio real (especialidad u orden de reserva), y la nota, que es la única explicación, está en letra pequeña. Una regla de este tipo afecta a muchos pacientes.

Figura 257

H02 - Reglas adicionales

Nota. Captura del mock-up de SaludYa evaluada por el equipo RouteGuard.

Recomendación: Definir un solo criterio y expresarlo claro («Se reasigna primero a quien reservó antes, dentro de la misma especialidad»), con la explicación junto a la regla y en un tamaño legible.

PROBLEMA #9: El mensaje de validación de capacidad mezcla reglas que no corresponden

Severidad: 2 **Heurística violada:** Usabilidad - Ayudar a reconocer, diagnosticar y recuperarse de errores

Problema: «H05 · Validación de regla» dice «La capacidad debe ser mayor que cero. La tolerancia de llegada no puede superar la duración del intervalo. Revisa los campos antes de guardar». Al editar la capacidad, el segundo mensaje no tiene relación y el usuario no sabe cuál de las dos reglas falló. Además aparece como diálogo, tapando el campo.

Figura 258

H05 - Validación de regla

Nota. Captura del mock-up de SaludYa evaluada por el equipo RouteGuard.

Recomendación: Mostrar solo el mensaje que corresponde al campo editado, junto al campo y en rojo («La capacidad debe ser mayor que 0»).

PROBLEMA #10: Los indicadores no muestran un control para cambiar la fecha o el periodo

Severidad: 2 **Heurística violada:** Usabilidad - Visibilidad del estado del sistema

Problema: «H06 · Indicadores del día» y «H07 · Demanda» muestran una fecha fija («Mar 6 oct 2026») sin ningún control visible para cambiarla, aunque «H09 · Periodo sin datos» muestra otra fecha («1 oct 2026») y «H08» permite elegir un rango para exportar. En las pantallas revisadas no se ve cómo la administración puede consultar otro día o semana, que es el uso básico de un panel de indicadores.

Figura 259

H06 - Indicadores diarios

Nota. Captura del mock-up de SaludYa evaluada por el equipo RouteGuard.

Figura 260

H09 - Periodo sin datos

Nota. Captura del mock-up de SaludYa evaluada por el equipo RouteGuard.

Recomendación: Agregar un selector de fecha o periodo visible en la cabecera de los indicadores (con atajos como «Hoy», «Ayer», «Últimos 7 días») y reflejar el periodo elegido en el título.

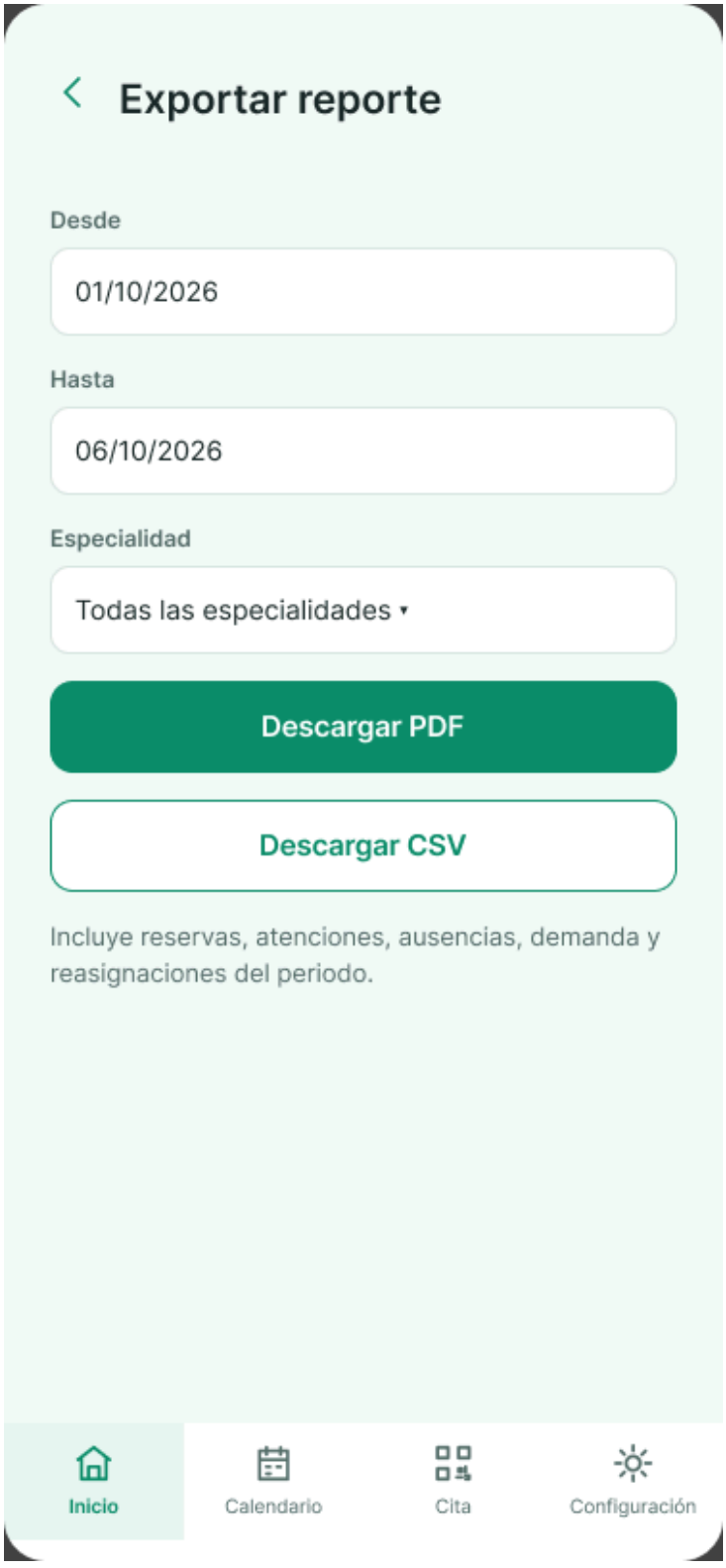
PROBLEMA #11: Descargar un reporte requiere dos pasos y no indica formato ni archivo

Severidad: 2 **Heurística violada:** Usabilidad - Eficiencia de uso

Problema: En «H08» el usuario elige «Descargar PDF» o «Descargar CSV», y luego «H11 · Exportación preparada» muestra un diálogo con otro botón «Descargar archivo». El segundo paso es redundante y el diálogo no repite qué archivo es (PDF o CSV), el periodo, el nombre ni el tamaño.

Figura 261

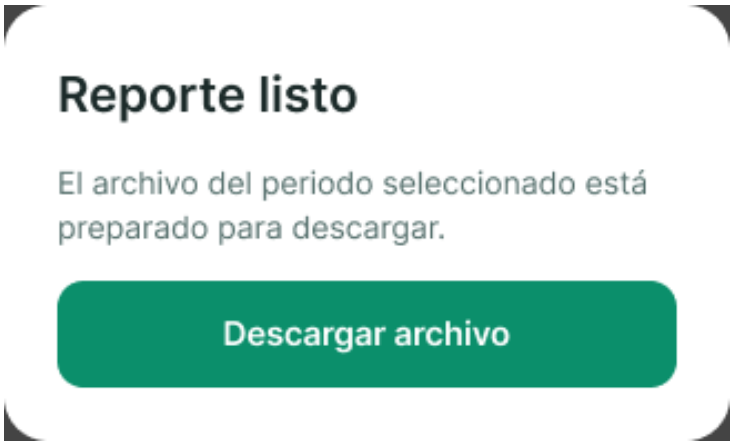
H08 - Exportación



Nota. Captura del mock-up de SaludYa evaluada por el equipo RouteGuard.

Figura 262

H11 - Exportación preparada



Nota. Captura del mock-up de SaludYa evaluada por el equipo RouteGuard.

Recomendación: Iniciar la descarga al tocar el botón de formato (con una barra de progreso si demora) y mostrar el nombre del archivo y el formato en la confirmación.

Conclusiones

Conclusiones y recomendaciones

A partir del análisis realizado y de la validación de las hipótesis planteadas, se presentan las siguientes conclusiones y recomendaciones:

Conclusiones

- La problemática planteada sobre la ineficiencia en el acceso a citas en establecimientos públicos de salud se confirmó a través del análisis de contexto y entrevistas, demostrando que la dependencia exclusiva de canales presenciales o telefónicos genera colas prolongadas y desinformación.
- El supuesto de negocio respecto a la demanda no atendida por soluciones digitales fue validado con éxito, dado que todos los pacientes entrevistados consideraron de alta utilidad consultar la disponibilidad de cupos en tiempo real desde sus dispositivos.
- Al contrastar las suposiciones con el comportamiento real del segmento de pacientes, se evidenció que la mayoría no ha utilizado aplicaciones médicas previamente. Sin embargo, se sienten cómodos realizando trámites desde sus celulares, lo que confirma la viabilidad de adopción de la plataforma.
- Las hipótesis operativas orientadas a mejorar el control del flujo de atención fueron validadas por el personal administrativo, quienes confirmaron que la gestión manual actual mediante cuadernos y hojas de cálculo provoca pérdida de información y desorganización operativa.
- Se concluye que la lista de espera dinámica y las notificaciones automatizadas responden directamente a los criterios de éxito del proyecto, asegurando el reaprovechamiento de cupos cancelados y mitigando los viajes innecesarios de los usuarios.
- La hipótesis sobre la reducción del tiempo de espera percibido es respaldada por la estructuración técnica del sistema, el cual incluye un mecanismo de registro de llegada (*check-in*) por código QR diseñado para descongestionar las ventanillas físicas.

Recomendaciones

- **Despliegue incremental basado en el Product Backlog:** Se recomienda ejecutar la priorización del MVP definida en los Sprints 1 y 2, liberando inicialmente el motor de reservas (*Appointments & Booking*), la validación de identidad y el *check-in* por QR, debido a que estos componentes representan el núcleo de valor para descongestionar el flujo presencial de pacientes.
- **Evolución hacia la Reprogramación Inteligente:** Para futuras versiones, se recomienda integrar un sistema de sugerencias inteligentes de horarios y la posibilidad de reprogramación automática, con el objetivo de resolver de manera proactiva los cruces de agenda y mejorar la experiencia del módulo central de citas.
- **Ampliación de mecanismos de Check-in:** A partir de las oportunidades identificadas en el contexto *Arrival & QR Check-in*, se recomienda evaluar en futuras versiones mecanismos alternativos de validación, como el reconocimiento facial, para facilitar el acceso de pacientes que no cuenten con un dispositivo móvil al momento de su atención.

Bibliografía

Ministerio de Salud (MINSA). (2017, 16 de mayo). *Nuevo sistema "Web colas" reduce el tiempo de espera en hospitales*. Recuperado de https://www.gob.pe/institucion/minsa/noticias/13263-nuevo-sistema-web-colas-reduce-el-tiempo-de-espera-en-hospitales

Ministerio de Salud (MINSA). (2023, 19 de septiembre). *Minsa reduce tiempos de espera por atención en establecimientos de salud de Lima Metropolitana*. Recuperado de https://www.gob.pe/institucion/minsa/noticias/831946-minsa-reduce-tiempos-de-espera-por-atencion-en-establecimientos-de-salud-de-lima-metropolitana

Ministerio de Salud (MINSA). (2024, 1 de marzo). *Minsa fortalece sistema de "Citas en Línea"*. Recuperado de https://www.gob.pe/institucion/minsa/noticias/914419-minsa-fortalece-sistema-de-citas-en-linea

Ministerio de Salud (MINSA). (2025, 2 de agosto). *Hospital San Juan de Lurigancho: más de 93 000 pacientes accedieron a sus citas por teléfono en el primer semestre de 2025*. Recuperado de https://www.gob.pe/institucion/minsa/noticias/1224054-hospital-san-juan-de-lurigancho-mas-de-93-000-pacientes-accedieron-a-sus-citas-por-telefono-en-el-primer-semestre-de-2025

Presidencia del Consejo de Ministros (PCM). (2026). *Obtener cita médica en un establecimiento del Minsa*. Plataforma del Estado Peruano. Recuperado de https://www.gob.pe/20621-obtener-cita-medica-en-un-establecimiento-del-minsa

World Health Organization (WHO). (2025). *Global strategy on digital health 2020–2027*. World Health Organization. Recuperado de https://www.who.int/publications/i/item/9789240116870

Anexos

Anexo A: Vídeos de entrevistas realizadas

Para la identificación de necesidades de los usuarios, se entrevistó a una pequeña parte del conjunto que forma a los segmentos objetivos.

Segmento objetivo 1: Pacientes de zonas urbanas periféricas que acuden a establecimientos públicos de salud

- Entrevista #1
- Entrevista #2
- Entrevista #3

Segmento objetivo 2: Personal asistencial y administrativo de establecimientos públicos de salud

- Entrevista #1
- Entrevista #2
- Entrevista #3